



TRSGraph Database 手册

v10.0

目录

1. TRSGraph简介	4
1.1 TRSGraph 的优势	4
1.2 适用场景	4
2. 基本概念	6
2.1 数据模型	6
2.2 路径	8
2.3 点 VID	10
2.4 服务架构	12
3. 快速入门	27
3.1 本地部署	27
3.2 nGQL 命令汇总	43
4. nGQL 指南	65
4.1 nGQL 概述	65
4.2 数据类型	79
4.3 运算符	98
4.4 函数和表达式	112
4.5 通用查询语句	155
4.6 子句和选项	208
4.7 变量和复合查询	241
4.8 图空间语句	247
4.9 Tag 语句	257
4.10 Edge type 语句	265
4.11 点语句	271
4.12 边语句	278
4.13 原生索引	285
4.14 全文索引	296
4.15 查询调优与终止	304
4.16 作业管理	310
5. 配置与日志	314
5.1 配置	314
5.2 日志	340
6. 数据安全	346
6.1 验证和授权	346
6.2 SSL 加密	352

7. 同步与迁移	354
7.1 负载均衡	354
8. 导入与导出	355
8.1 导入导出工具概述	355
8.2 TRSGraph Importer	356
9. 最佳实践	366
9.1 Compaction	366
9.2 Storage 负载均衡	368
9.3 图建模设计	369
9.4 系统设计建议	373
9.5 执行计划	374
9.6 超级顶点（稠密点）处理	375
10. 客户端	377
10.1 客户端介绍	377
10.2 TRSGraph Console	378
10.3 TRSGraph CPP	383
10.4 TRSGraph Java	384
10.5 TRSGraph Python	385
11. TRSGraph Studio	386
11.1 认识 TRSGraph Studio	386
11.2 安装与登录	388
11.3 快速开始	393
11.4 全局设置	414
11.5 故障排查	415
12. 常见问题 FAQ	417
12.1 关于本手册	417
12.2 关于历史兼容性	417
12.3 关于执行报错	417
12.4 关于设计与功能	420
12.5 关于运维	422
12.6 关于连接	424
12.7 关于扩容、缩容	425
13. 附录	427
13.1 错误码	427

1. TRSGraph简介

TRSGraph 是一款分布式的、易扩展的原生图数据库，能够承载包含数千万个点和数万亿条边的超大规模数据集，并且提供毫秒级查询，可以存储庞大的图形网络并从中检索信息的数据库。它可以将图中的数据高效存储为点（Vertex）和边（Edge），还可以将属性（Property）附加到点和边上。

图数据库适合存储大多数从现实抽象出的数据类型。世界上几乎所有领域的事物都有内在联系，像关系型数据库这样的建模系统会提取实体之间的关系，并将关系单独存储到表和列中，而实体的类型和属性存储在其他列甚至其他表中，这使得数据管理费时费力。

TRSGraph 作为一个典型的图数据库，可以将丰富的关系通过边及其类型和属性自然地呈现。

1.1 TRSGraph 的优势

1.1.1 高性能

基于图数据库的特性使用 C++ 编写的 TRSGraph，可以提供毫秒级查询。众多数据库中，TRSGraph 在图数据服务领域展现了卓越的性能，数据规模越大，TRSGraph 优势就越大。

1.1.2 易扩展

TRSGraph 采用 shared-nothing 架构，支持在不停止数据库服务的情况下扩缩容。

1.1.3 易开发

TRSGraph 提供 Java、Python、C++ 和 Go 等流行编程语言的客户端，更多客户端仍在开发中。详情请参见 [TRSGraph clients](#)。

1.1.4 高可靠访问控制

TRSGraph 支持严格的角色访问控制和 LDAP（Lightweight Directory Access Protocol）等外部认证服务，能够有效提高数据安全性。详情请参见 [验证和授权](#)。

1.1.5 兼容 openCypher 查询语言

TRSGraph 查询语言，简称为 nGQL，是一种声明性的、部分兼容 openCypher 的文本查询语言，易于理解和使用。详细语法请参见 [nGQL 指南](#)。

1.1.6 面向未来硬件，读写平衡

闪存型设备有着极高的性能，并且 [价格快速下降](#)，TRSGraph 是一个面向 SSD 设计的产品，相比于基于 HDD + 大内存的产品，更适合面向未来的硬件趋势，也更容易做到读写平衡。

1.1.7 灵活数据建模

用户可以轻松地在 TRSGraph 中建立数据模型，不必将数据强制转换为关系表。而且可以自由增加、更新和删除属性。详情请参见 [数据模型](#)。

1.2 适用场景

TRSGraph 可用于各种基于图的业务场景。为节约转换各类数据到关系型数据库的时间，以及避免复杂查询，建议使用 TRSGraph。

1.2.1 欺诈检测

金融机构必须仔细研究大量的交易信息，才能检测出潜在的金融欺诈行为，并了解某个欺诈行为和设备的内在关联。这种场景可以通过图来建模，然后借助 TRSGraph，可以很容易地检测出诈骗团伙或其他复杂诈骗行为。

1.2.2 实时推荐

TRSGraph 能够及时处理访问者产生的实时信息，并且精准推送文章、视频、产品和服务。

1.2.3 知识图谱

自然语言可以转化为知识图谱，存储在 TRSGraph 中。用自然语言组织的问题可以通过智能问答系统中的语义解析器进行解析并重新组织，然后从知识图谱中检索出问题的可能答案，提供给提问者。

1.2.4 社交网络

人际关系信息是典型的图数据，TRSGraph 可以轻松处理数十亿人和数万亿人际关系的社交网络信息，并在海量并发的情况下，提供快速的好友推荐和工作岗位查询。

最后更新: 2025年12月19日

2. 基本概念

2.1 数据模型

本文介绍 TRSGraph 的数据模型。数据模型是一种组织数据并说明它们如何相互关联的模型。

2.1.1 数据模型

TRSGraph 数据模型使用 6 种基本的数据模型：

- 图空间 (Space)
图空间用于隔离不同团队或者项目的数据。不同图空间的数据是相互隔离的，可以指定不同的存储副本数、权限、分片等。
- 点 (Vertex)
点用来保存实体对象，特点如下：
 - 点是用点标识符 (`VID`) 标识的。`VID` 在同一图空间中唯一。`VID` 是一个 `int64`，或者 `fixed_string(N)`。
 - 点可以有 0 到多个 Tag。



TRSGraph 2.x 及以下版本中的点必须包含至少一个 Tag。

- 边 (Edge)
边是用来连接点的，表示两个点之间的关系或行为，特点如下：
 - 两点之间可以有多条边。
 - 边是有方向的，不存在无向边。
 - 四元组 `<起点 VID、Edge type、边排序值 (rank)、终点 VID>` 用于唯一标识一条边。边没有 EID。
 - 一条边有且仅有一个 Edge type。
 - 一条边有且仅有一个 Rank，类型为 `int64`，默认值为 0。



Rank 可以用来区分 Edge type、起始点、目的点都相同的边。该值完全由用户自己指定。
读取时必须自行取得全部的 Rank 值后排序过滤和拼接。
不支持诸如 `next()`、`pre()`、`head()`、`tail()`、`max()`、`min()`、`lessThan()`、`moreThan()` 等函数功能，也不能通过创建索引加速访问或者条件过滤。

- 标签 (Tag)
Tag 由一组事先预定义的属性构成。
- 边类型 (Edge type)
Edge type 由一组事先预定义的属性构成。
- 属性 (Property)
属性是指以键值对 (Key-value pair) 形式表示的信息。



Note

Tag 和 Edge type 的作用，类似于关系型数据库中“点表”和“边表”的表结构。

2.1.2 有向属性图

TRSGraph 使用有向属性图模型，指点 and 边构成的图，这些边是有方向的，点和边都可以有属性。

下表为篮球运动员数据集的结构示例，包括两种类型的点（player、team）和两种类型的边（serve、follow）。

类型	名称	属性名（数据类型）	说明
Tag	player	name (string) age (int)	表示球员。属性 name 表示姓名，age 表示年龄。
Tag	team	name (string)	表示球队。属性 name 表示球队名。
Edge type	serve	start_year (int) end_year (int)	表示球员效力球队的行为。该行为将球员和球队联系起来，方向是从球员到球队。属性 start_year 表示效力的起始年份，end_year 表示效力的结束年份。
Edge type	follow	degree (int)	表示推特上球员关注另一个球员的行为。该行为将两个球员联系起来，方向是从一个球员到另一个球员。属性 degree 表示喜爱程度。



Note

TRSGraph 中没有无向边，只支持有向边。



Compatibility

由于 TRSGraph 10.0 的数据模型中，允许存在“悬挂边”，因此在增删时，用户需自行保证“一条边所对应的起点和终点”的存在性。详见 [INSERT VERTEX](#)、[DELETE VERTEX](#)、[INSERT EDGE](#)、[DELETE EDGE](#)。

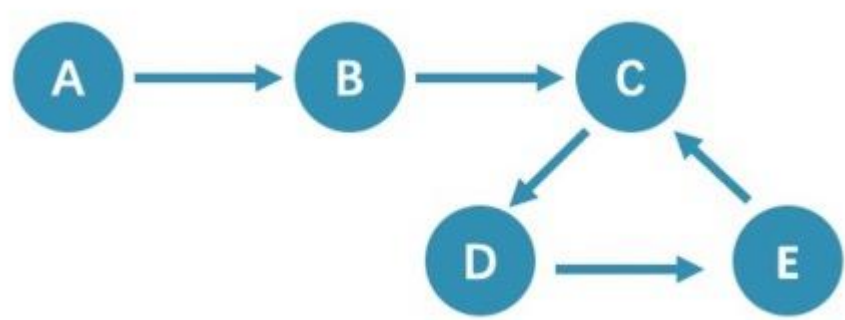
不支持 openCypher 中的 MERGE 语句。

2.2 路径

图论中一个非常重要的概念是路径，路径是指一个有限或无限的边序列，这些边连接着一系列的点。

路径的类型分为三种：`walk`、`trail`、`path`。关于路径的详细说明，请参见[维基百科](#)。

本文以下图为例进行简单介绍。



2.2.1 walk

`walk` 类型的路径由有限或无限的边序列构成。遍历时点和边可以重复。

查看示例图，由于 C、D、E 构成了一个环，因此该图包含无限个路径，例如 `A->B->C->D->E`、`A->B->C->D->E->C`、`A->B->C->D->E->C->D`。



`GO` 语句采用的是 `walk` 类型路径。

2.2.2 trail

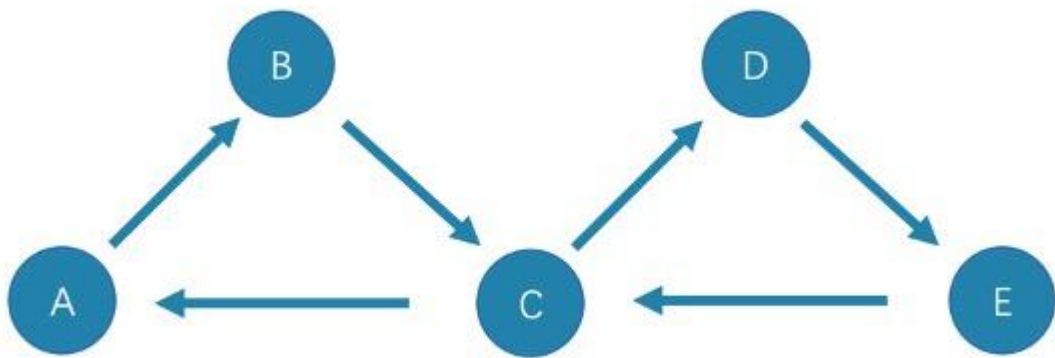
`trail` 类型的路径由有限的边序列构成。遍历时只有点可以重复，边不可以重复。柯尼斯堡七桥问题的路径类型就是 `trail`。

查看示例图，由于边不可以重复，所以该图包含有限个路径，最长路径由 5 条边组成：`A->B->C->D->E->C`。



`MATCH`、`FIND PATH` 和 `GET SUBGRAPH` 语句采用的是 `trail` 类型路径。

在 `trail` 类型中，还有 `cycle` 和 `circuit` 两种特殊的路径类型，以下图为例对这两种特殊的路径类型进行介绍。



- cycle

`cycle` 是封闭的 `trail` 类型的路径，遍历时边不可以重复，起点和终点重复，并且没有其他点重复。在此示例图中，最长路径由三条边组成：`A->B->C->A` 或 `C->D->E->C`。

- circuit

`circuit` 也是封闭的 `trail` 类型的路径，遍历时边不可以重复，除起点和终点重复外，可能存在其他点重复。在此示例图中，最长路径为：`A->B->C->D->E->C->A`。

2.2.3 path

`path` 类型的路径由有限的边序列构成。遍历时点和边都不可以重复。

查看示例图，由于点和边都不可以重复，所以该图包含有限个路径，最长路径由 4 条边组成：`A->B->C->D->E`。

最后更新: 2025年12月19日

2.3 点 VID

在一个图空间中，一个点由点的 ID 唯一标识，即 VID 或 Vertex ID。

2.3.1 VID 的特点

- VID 数据类型只可以为定长字符串 `FIXED_STRING(<N>)` 或 `INT64`。一个图空间只能选用其中一种 VID 类型。
- VID 在一个图空间中必须唯一，其作用类似于关系型数据库中的主键（索引+唯一约束）。但不同图空间中的 VID 是完全独立无关的。
- 点 VID 的生成方式必须由用户自行指定，系统不提供自增 ID 或者 UUID。
- VID 相同的点，会被认为是同一个点。例如：
- VID 相当于一个实体的唯一标号，例如一个人的身份证号。Tag 相当于实体所拥有的类型，例如"滴滴司机"和"老板"。不同的 Tag 又相应定义了两组不同的属性，例如"驾照号、驾龄、接单量、接单小号"和"工号、薪水、债务额度、商务电话"。
- 同时操作相同 VID 并且相同 Tag 的两条 `INSERT` 语句（均无 `IF NOT EXISTS` 参数），晚写入的 `INSERT` 会覆盖先写入的。
- 同时操作包含相同 VID 但是两个不同 TAG A 和 TAG B 的两条 `INSERT` 语句，对 TAG A 的操作不会影响 TAG B。
- VID 通常会被（LSM-tree 方式）索引并缓存在内存中，因此直接访问 VID 的性能最高。

2.3.2 VID 使用建议

- TRSGraph 1.x 只支持 VID 类型为 `INT64`，从 2.x 开始支持 `INT64` 和 `FIXED_STRING(<N>)`。在 `CREATE SPACE` 中通过参数 `vid_type` 可以指定 VID 类型。
- 可以使用 `id()` 函数，指定或引用该点的 VID。
- 可以使用 `LOOKUP` 或者 `MATCH` 语句，来通过属性索引查找对应的 VID。
- 性能上，直接通过 VID 找到点的语句性能最高，例如 `DELETE xxx WHERE id(xxx) == "player100"`，或者 `GO FROM "player100"` 等语句。通过属性先查找 VID，再进行图操作的性能会变差，例如 `LOOKUP | GO FROM $-.ids` 等语句，相比前者多了一次内存或硬盘的随机读（`LOOKUP`）以及一次序列化（`|`）。

2.3.3 VID 生成建议

VID 的生成工作完全交给应用端，有一些通用的建议：

- （最优）通过有唯一性的主键或者属性来直接作为 VID；属性访问依赖于 VID；
- 通过有唯一性的属性组合来生成 VID，属性访问依赖于属性索引。
- 通过 snowflake 等算法生成 VID，属性访问依赖于属性索引。
- 如果个别记录的主键特别长，但绝大多数记录的主键都很短的情况，不要将 `FIXED_STRING(<N>)` 的 `N` 设置成超大，这会浪费大量内存和硬盘，也会降低性能。此时可通过 BASE64，MD5，hash 编码加拼接的方式来生成。
- 如果用 hash 方式生成 int64 VID：在有 10 亿个点的情况下，发生 hash 冲突的概率大约是 1/10。边的数量与碰撞的概率无关。

2.3.4 定义和修改 VID 与其数据类型

VID 的数据类型必须在创建图空间时定义，且一旦定义无法修改。

VID 必须在插入点时设置，且一旦设置无法修改。

2.3.5 “查询起始点”(start vid) 与全局扫描

绝大多数情况下，TRSGraph 的查询语句（`MATCH`、`GO`、`LOOKUP`）的执行计划，必须要通过一定方式找到查询起始点的 VID（`start vid`）。

定位 `start vid` 只有两种方式：

1. 例如 `GO FROM "player100" OVER` 是在语句中显式的指明 `start vid` 是 "player100"；
2. 例如 `LOOKUP ON player WHERE player.name == "Tony Parker"` 或者 `MATCH (v:player {name:"Tony Parker"})`，是通过属性 `player.name` 的索引来定位到 `start vid`；

最后更新: 2025年12月19日

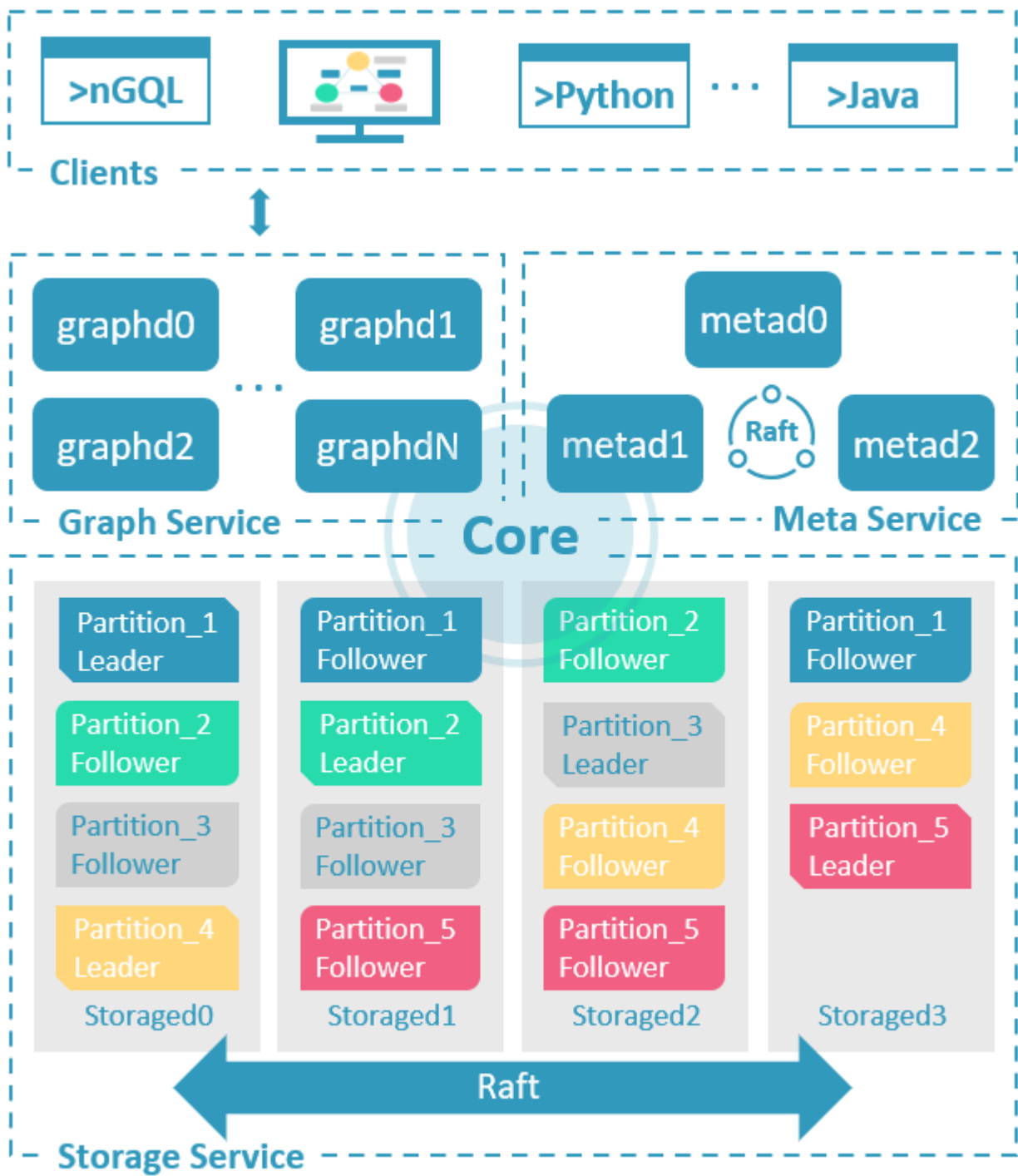
2.4 服务架构

2.4.1 TRSGraph 架构总览

TRSGraph 由三种服务构成：Graph 服务、Meta 服务和 Storage 服务，是一种存储与计算分离的架构。

每个服务都有可执行的二进制文件 and 对应进程，用户可以使用这些二进制文件在一个或多个计算机上部署 TRSGraph 集群。

下图展示了 TRSGraph 集群的经典架构。



Meta 服务

在 TRSGraph 架构中，Meta 服务是由 trsgraph-metad 进程提供的，负责数据管理，例如 Schema 操作、集群管理和用户权限管理等。

Meta 服务的详细说明，请参见 [Meta 服务](#)。

Graph 服务和 Storage 服务

TRSGraph 采用计算存储分离架构。Graph 服务负责处理计算请求，Storage 服务负责存储数据。它们由不同的进程提供，Graph 服务是由 trsgraph-graphd 进程提供，Storage 服务是由 trsgraph-storaged 进程提供。计算存储分离架构的优势如下：

- 易扩展

分布式架构保证了 Graph 服务和 Storage 服务的灵活性，方便扩容和缩容。

- 高可用

如果提供 Graph 服务的服务器有一部分出现故障，其余服务器可以继续为客户端提供服务，而且 Storage 服务存储的数据不会丢失。服务恢复速度较快，甚至能做到用户无感知。

- 节约成本

计算存储分离架构能够提高资源利用率，而且可根据业务需求灵活控制成本。

- 更多可能性

基于分离架构的特性，Graph 服务将可以在更多类型的存储引擎上单独运行，Storage 服务也可以为多种目的的计算引擎提供服务。

Graph 服务和 Storage 服务的详细说明，请参见 [Graph 服务](#)和 [Storage 服务](#)。

最后更新: 2025年12月19日

2.4.2 Meta 服务

本文介绍 Meta 服务的架构和功能。

Meta 服务架构

Meta 服务是由 trsgraph-metad 进程提供的，用户可以根据场景配置 trsgraph-metad 进程数量：

- 测试环境中，用户可以在 TRSGraph 集群中部署 1 个或 3 个 trsgraph-metad 进程。如果要部署 3 个，用户可以将它们部署在 1 台机器上，或者分别部署在不同的机器上。
- 生产环境中，建议在 TRSGraph 集群中部署 3 个 trsgraph-metad 进程。请将这些进程部署在不同的机器上以保证高可用。

所有 trsgraph-metad 进程构成了基于 Raft 协议的集群，其中一个进程是 leader，其他进程都是 follower。

leader 是由多数派选举出来，只有 leader 能够对客户端或其他组件提供服务，其他 follower 作为候补，如果 leader 出现故障，会在所有 follower 中选举出新的 leader。



leader 和 follower 的数据通过 Raft 协议保持一致，因此 leader 故障和选举新 leader 不会导致数据不一致。更多关于 Raft 的介绍见 [Storage 服务](#)。

Meta 服务功能

管理用户账号

Meta 服务中存储了用户的账号和权限信息，当客户端通过账号发送请求给 Meta 服务，Meta 服务会检查账号信息，以及该账号是否有对应的请求权限。

更多 TRSGraph 的访问控制说明，请参见[身份验证](#)。

管理分片

Meta 服务负责存储和管理分片的位置信息，并且保证分片的负载均衡。

管理图空间

TRSGraph 支持多个图空间，不同图空间内的数据是安全隔离的。Meta 服务存储所有图空间的元数据（非完整数据），并跟踪数据的变更，例如增加或删除图空间。

管理 SCHEMA 信息

TRSGraph 是强类型图数据库，它的 Schema 包括 Tag、Edge type、Tag 属性和 Edge type 属性。

Meta 服务中存储了 Schema 信息，同时还负责 Schema 的添加、修改和删除，并记录它们的版本。

更多 TRSGraph 的 Schema 信息，请参见[数据模型](#)。

管理 TTL 信息

Meta 服务存储 TTL（Time To Live）定义信息，可以用于设置数据生命周期。数据过期后，会由 Storage 服务进行处理，具体过程参见 [TTL](#)。

管理作业

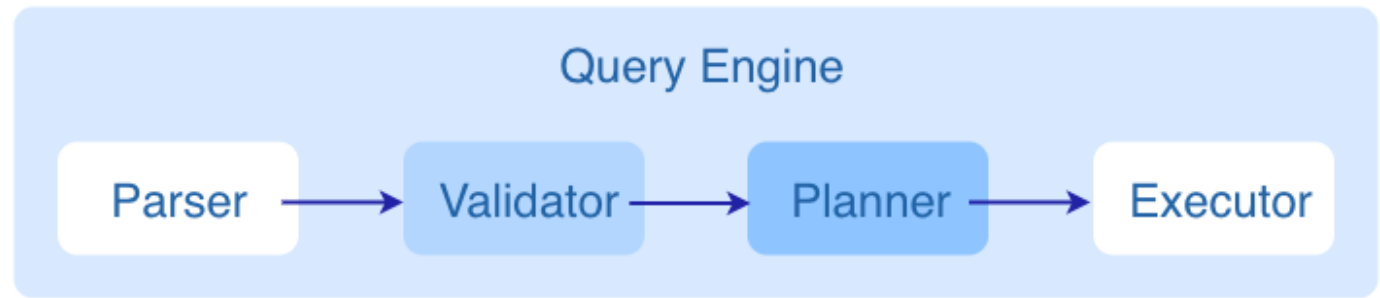
Meta 服务中的作业管理模块负责作业的创建、排队、查询和删除。

最后更新: 2025年12月19日

2.4.3 Graph 服务

Graph 服务主要负责处理查询请求，包括解析查询语句、校验语句、生成执行计划以及按照执行计划执行四个大步骤，本文将基于这些步骤介绍 Graph 服务。

Graph 服务架构



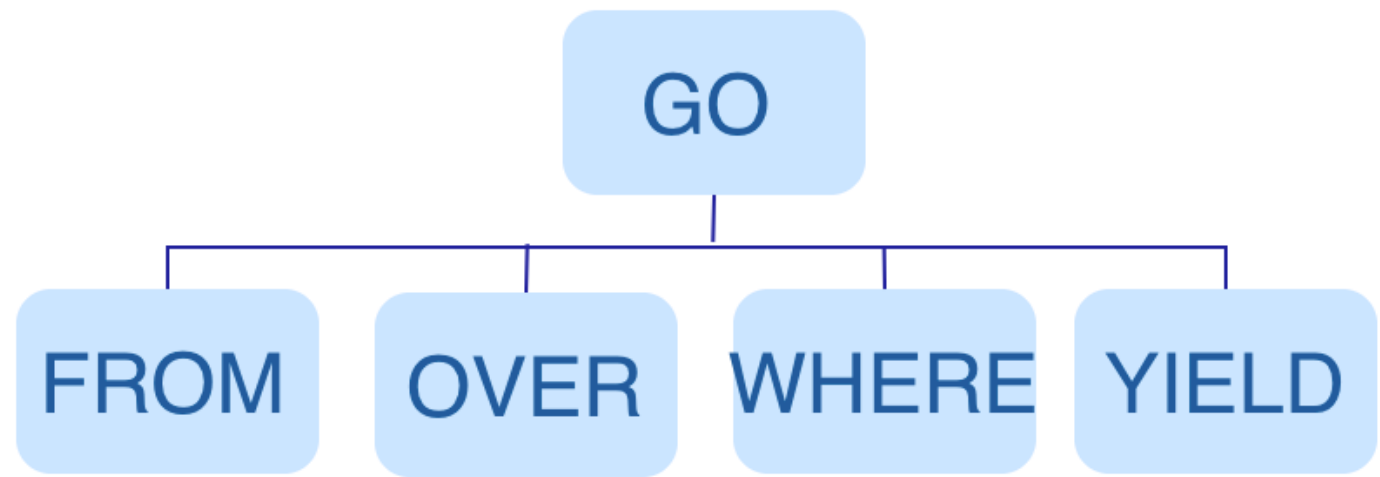
查询请求发送到 Graph 服务后，会由如下模块依次处理：

- 1. Parser：词法语法解析模块。
- 2. Validator：语义校验模块。
- 3. Planner：执行计划与优化器模块。
- 4. Executor：执行引擎模块。

Parser

Parser 模块收到请求后，通过 Flex（词法分析工具）和 Bison（语法分析工具）生成的词法语法解析器，将语句转换为抽象语法树（AST），在语法解析阶段会拦截不符合语法规则的语句。

例如 `GO FROM "Tim" OVER like WHERE properties(edge).likeness > 8.0 YIELD dst(edge)` 语句转换的 AST 如下。



Validator

Validator 模块对生成的 AST 进行语义校验，主要包括：

- 校验元数据信息

校验语句中的元数据信息是否正确。

例如解析 `OVER`、`WHERE` 和 `YIELD` 语句时，会查找 Schema 校验 Edge type、Tag 的信息是否存在，或者插入数据时校验插入的数据类型和 Schema 中的是否一致。

- 校验上下文引用信息

校验引用的变量是否存在或者引用的属性是否属于变量。

例如语句 `$var = GO FROM "Tim" OVER like YIELD dst(edge) AS ID; GO FROM $var.ID OVER serve YIELD dst(edge)`，Validator 模块首先会检查变量 `var` 是否定义，其次再检查属性 `ID` 是否属于变量 `var`。

- 校验类型推断

推断表达式的结果类型，并根据子句校验类型是否正确。

例如 `WHERE` 子句要求结果是 `bool`、`null` 或者 `empty`。

- 校验 `*` 代表的信息

查询语句中包含 `*` 时，校验子句时需要将 `*` 涉及的 Schema 都进行校验。

例如语句 `GO FROM "Tim" OVER * YIELD dst(edge), properties(edge).likeness, dst(edge)`，校验 `OVER` 子句时需要校验所有的 Edge type，如果 Edge type 包含 `like` 和 `serve`，该语句会展开为 `GO FROM "Tim" OVER like,serve YIELD dst(edge), properties(edge).likeness, dst(edge)`。

- 校验输入输出

校验管道符 `()` 前后的一致性。

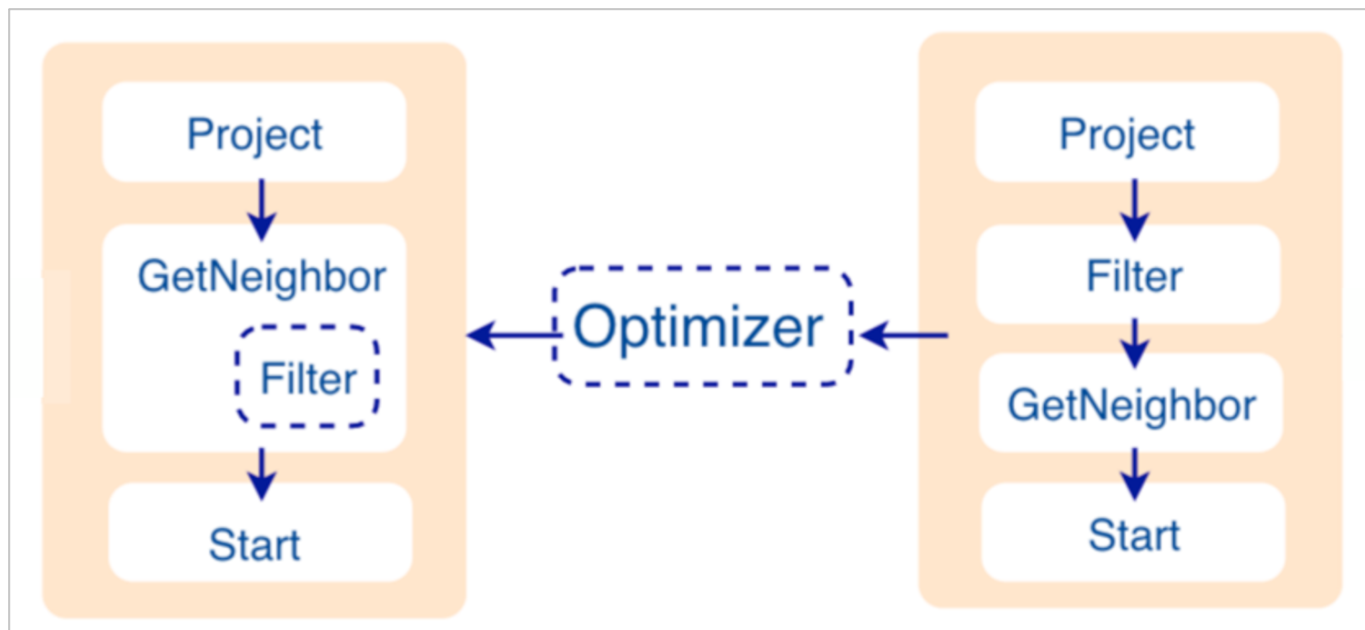
例如语句 `GO FROM "Tim" OVER like YIELD dst(edge) AS ID | GO FROM $-.ID OVER serve YIELD dst(edge)`，Validator 模块会校验 `$-.ID` 在管道符左侧是否已经定义。

校验完成后，Validator 模块还会生成一个默认可执行，但是未进行优化的执行计划，存储在目录 `src/planner` 内。

Planner

如果配置文件 `trsgraph-graphd.conf` 中 `enable_optimizer` 设置为 `false`，Planner 模块不会优化 Validator 模块生成的执行计划，而是直接交给 Executor 模块执行。

如果配置文件 `trsgraph-graphd.conf` 中 `enable_optimizer` 设置为 `true`，Planner 模块会对 Validator 模块生成的执行计划进行优化。如下图所示。



• 优化前

如上图右侧未优化的执行计划，每个节点依赖另一个节点，例如根节点 `Project` 依赖 `Filter`、`Filter` 依赖 `GetNeighbor`，最终找到叶子节点 `Start`，才能开始执行（并非真正执行）。

在这个过程中，每个节点会有对应的输入变量和输出变量，这些变量存储在一个哈希表中。由于执行计划不是真正执行，所以哈希表中每个 key 的 value 值都为空（除了 `Start` 节点，起始数据会存储在该节点的输入变量中）。哈希表定义在仓库 `trsgraph-graph` 内的 `src/context/ExecutionContext.cpp` 中。

例如哈希表的名称为 `ResultMap`，在建立 `Filter` 这个节点时，定义该节点从 `ResultMap["GN1"]` 中读取数据，然后将结果存储在 `ResultMap["Filter2"]` 中，依次类推，将每个节点的输入输出都确定好。

• 优化过程

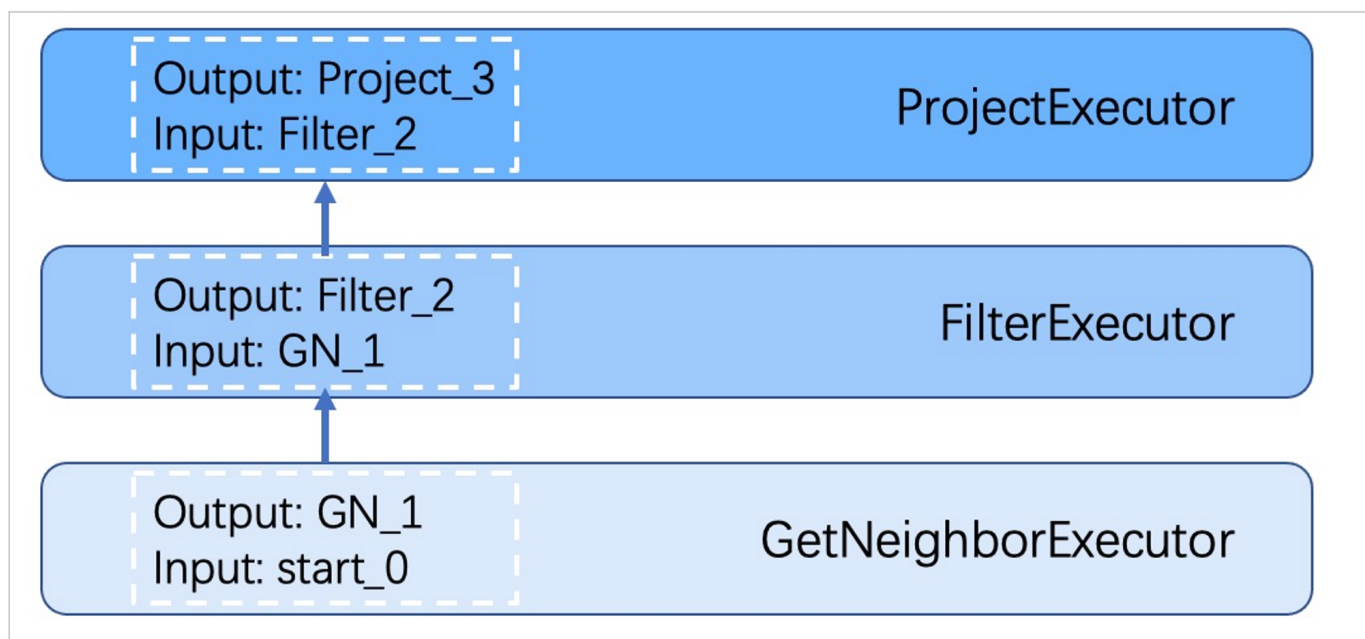
Planner 模块目前的优化方式是 RBO（rule-based optimization），即预定义优化规则，然后对 Validator 模块生成的默认执行计划进行优化。新的优化规则 CBO（cost-based optimization）正在开发中。优化代码存储在仓库 `trsgraph-graph` 的目录 `src/optimizer/` 内。

RBO 是一个自底向上的探索过程，即对于每个规则而言，都会由执行计划的根节点（示例是 `Project`）开始，一步步向下探索到最底层的节点，在过程中查看是否可以匹配规则。

如上图所示，探索到节点 `Filter` 时，发现依赖的节点是 `GetNeighbor`，匹配预先定义的规则，就会将 `Filter` 融入到 `GetNeighbor` 中，然后移除节点 `Filter`，继续匹配下一个规则。在执行阶段，当算子 `GetNeighbor` 调用 Storage 服务的接口获取一个点的邻边时，Storage 服务内部会直接将不符合条件的边过滤掉，这样可以极大地减少传输的数据量，该优化称为过滤下推。

Executor

Executor 模块包含调度器（Scheduler）和执行器（Executor），通过调度器调度执行计划，让执行器根据执行计划生成对应的执行算子，从叶子节点开始执行，直到根节点结束。如下图所示。



每一个执行计划节点都一一对应一个执行算子，节点的输入输出在优化执行计划时已经确定，每个算子只需要拿到输入变量中的值进行计算，最后将计算结果放入对应的输出变量中即可，所以只需要从节点 `start` 一步步执行，最后一个算子的输出变量会作为最终结果返回给客户端。

最后更新: 2025年12月19日

2.4.4 Storage 服务

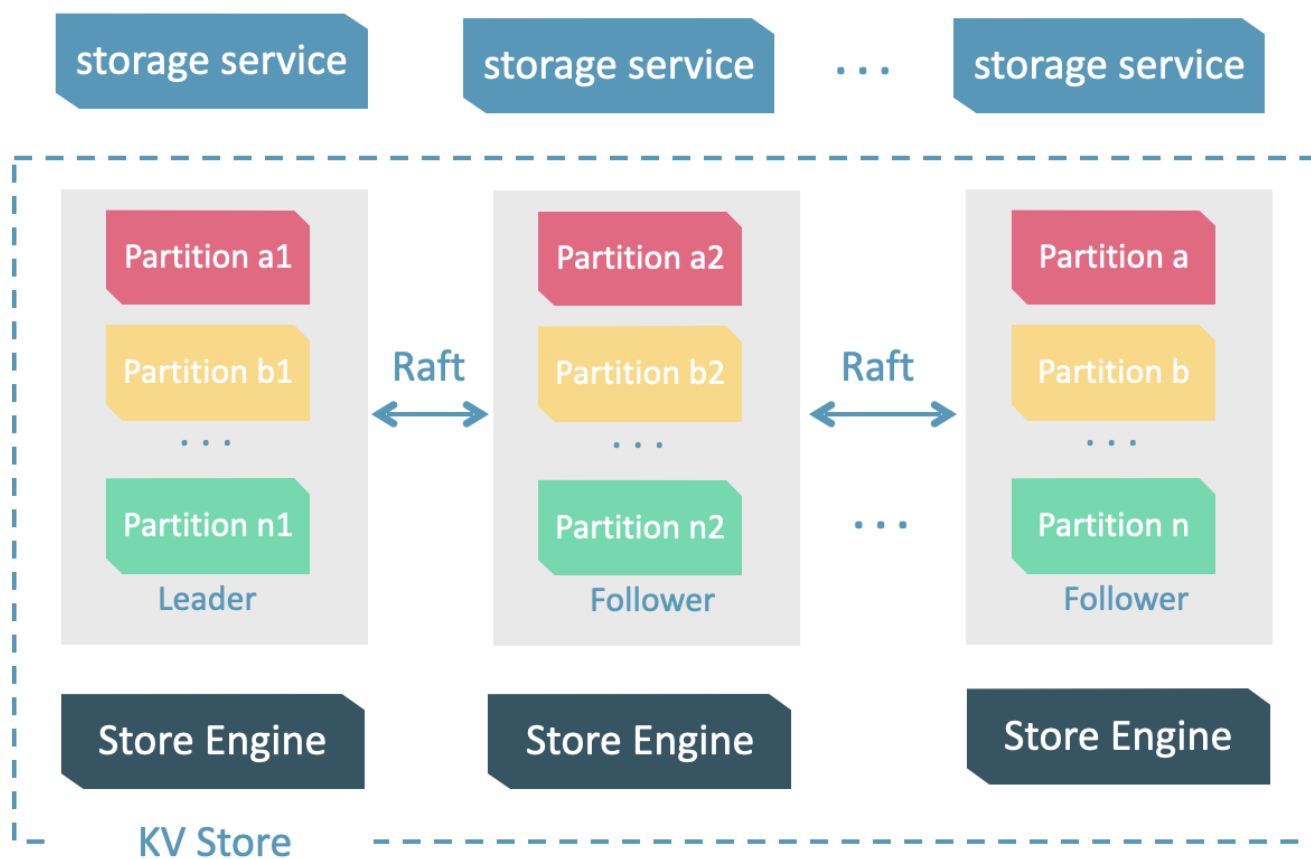
TRSGraph 的存储包含两个部分，一个是 Meta 相关的存储，称为 Meta 服务，在前文已有介绍。

另一个是具体数据相关的存储，称为 Storage 服务。其运行在 trsgraph-storaged 进程中。本文仅介绍 Storage 服务的架构设计。

优势

- 高性能（自研 KVStore）
- 易水平扩展（Shared-nothing 架构，不依赖 NAS 等硬件设备）
- 强一致性（Raft）
- 高可用性（Raft）
- 支持向第三方系统进行同步（例如全文索引）

Storage 服务架构



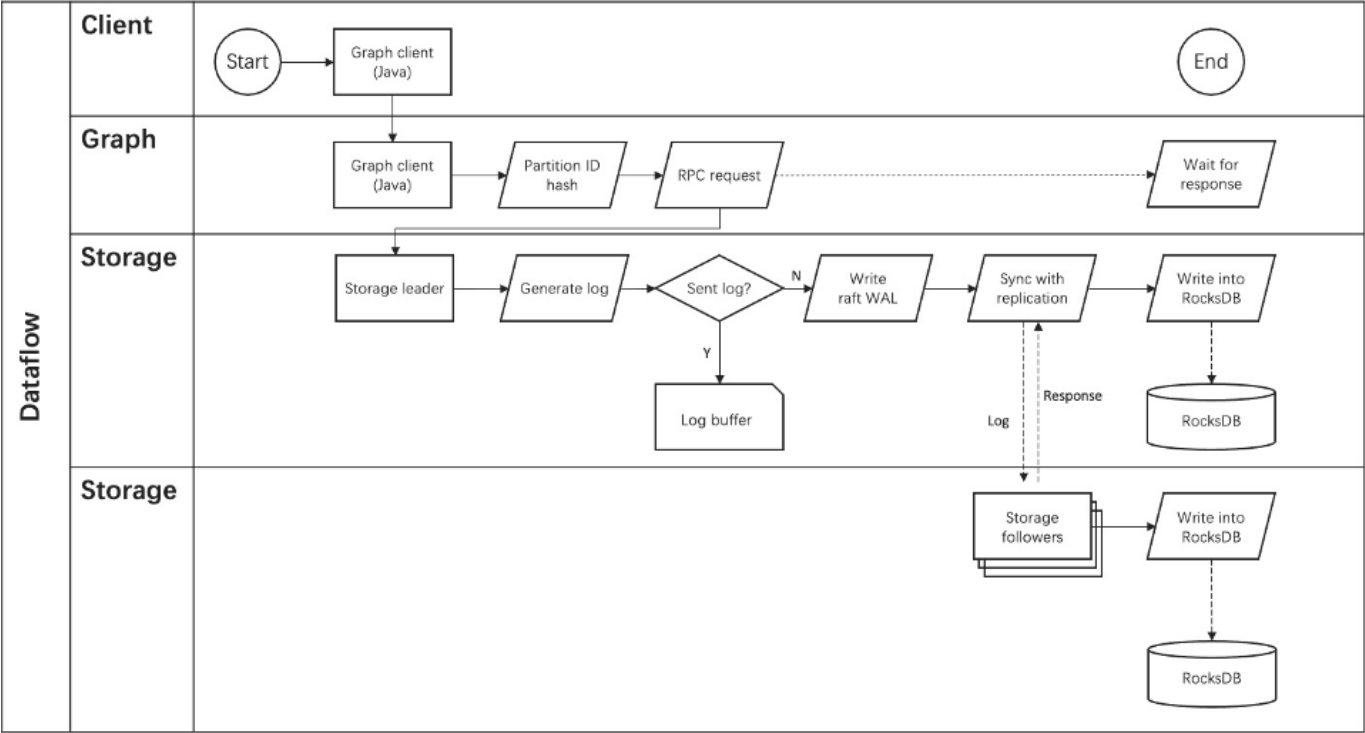
Storage 服务是由 trsgraph-storaged 进程提供的，用户可以根据场景配置 trsgraph-storaged 进程数量，例如测试环境 1 个，生产环境 3 个。

所有 trsgraph-storaged 进程构成了基于 Raft 协议的集群，整个服务架构可以分为三层，从上到下依次为：

- Storage interface 层
Storage 服务的最上层，定义了一系列和图相关的 API。API 请求会在这一层被翻译成一组针对分片的 KV 操作，例如：
 - getNeighbors：查询一批点的出边或者入边，返回边以及对应的属性，并且支持条件过滤。
 - insert vertex/edge：插入一条点或者边及其属性。
 - getProps：获取一个点或者一条边的属性。正是这一层的存在，使得 Storage 服务变成了真正的图存储，否则 Storage 服务只是一个 KV 存储服务。
- Consensus 层
Storage 服务的中间层，实现了 Multi Group Raft，保证强一致性和高可用性。
- Store Engine 层
Storage 服务的最底层，是一个单机版本地存储引擎，提供对本地数据的 get、put、scan 等操作。相关接口存储在 KVStore.h 和 KVEngine.h 文件，用户可以根据业务需求定制开发相关的本地存储插件。

下文将基于架构介绍 Storage 服务的部分特性。

Storage 写入流程



KVStore

TRSGraph 使用自行开发的 KVStore，而不是其他开源 KVStore，原因如下：

- 需要高性能 KVStore。
- 需要以库的形式提供，实现高效计算下推。对于强 Schema 的 TRSGraph 来说，计算下推时如何提供 Schema 信息，是高效的关键。
- 需要数据强一致性。

基于上述原因，TRSGraph 使用 RocksDB 作为本地存储引擎，实现了自己的 KVStore，有如下优势：

- 对于多硬盘机器，TRSGraph 只需配置多个不同的数据目录即可充分利用多硬盘的并发能力。
- 由 Meta 服务统一管理所有 Storage 服务，可以根据所有分片的分布情况和状态，手动进行负载均衡。



不支持自动负载均衡是为了防止自动数据搬迁影响线上业务。

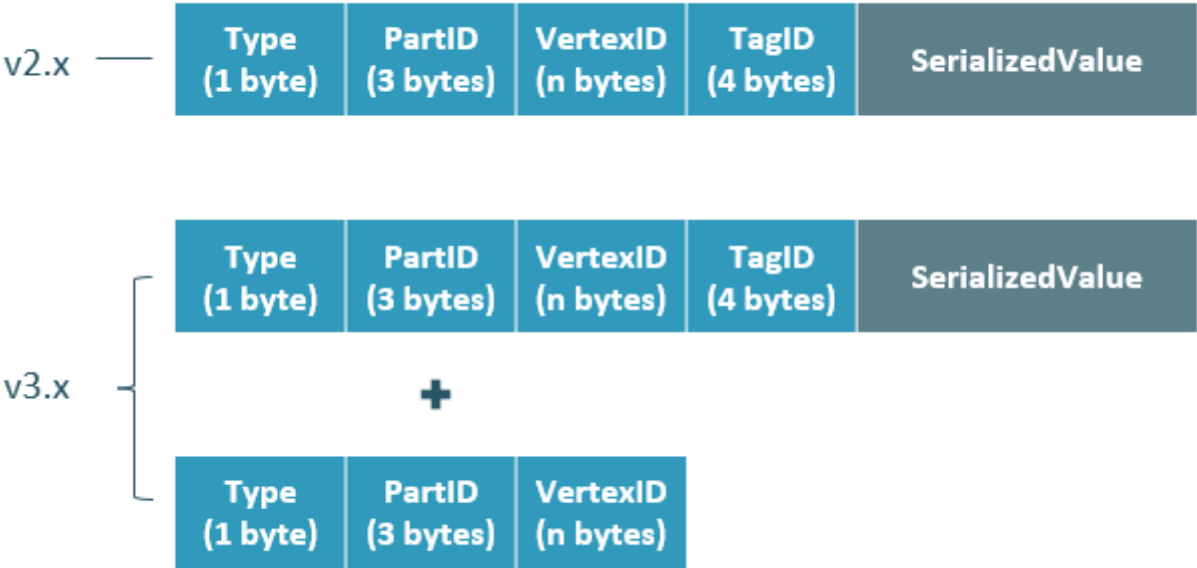
- 定制预写日志（WAL），每个分片都有自己的 WAL。
- 支持多个图空间，不同图空间相互隔离，每个图空间可以设置自己的分片数和副本数。

数据存储格式

图存储的主要数据是点和边，TRSGraph 将点和边的信息存储为 key，同时将点和边的属性信息存储在 value 中，以便更高效地使用属性过滤。

• 点数据存储格式

相比 TRSGraph 2.x 版本，3.x 版本在开启无 Tag 的点配置后，每个点多了一个不含 TagID 字段并且无 value 的 key。



字段	说明
Type	key 类型。长度为 1 字节。
PartID	数据分片编号。长度为 3 字节。此字段主要用于 Storage 负载均衡（balance）时方便根据前缀扫描整个分片的数据。
VertexID	点 ID。当点 ID 类型为 int 时，长度为 8 字节；当点 ID 类型为 string 时，长度为创建图空间时指定的 fixed_string 长度。
TagID	点关联的 Tag ID。长度为 4 字节。
SerializedValue	序列化的 value，用于保存点的属性信息。

• 边数据存储格式

Type (1 byte)	PartID (3 bytes)	VertexID (n bytes)	EdgeType (4 bytes)	Rank (8 bytes)	VertexID (n bytes)	Placeholder (1 byte)	SerializedValue
------------------	---------------------	-----------------------	-----------------------	-------------------	-----------------------	-------------------------	-----------------

字段	说明
Type	key 类型。长度为 1 字节。
PartID	数据分片编号。长度为 3 字节。此字段主要用于 Storage 负载均衡（balance）时方便根据前缀扫描整个分片的数据。
VertexID	点 ID。前一个 VertexID 在出边里表示起始点 ID，在入边里表示目的点 ID；后一个 VertexID 出边里表示目的点 ID，在入边里表示起始点 ID。
Edge type	边的类型。大于 0 表示出边，小于 0 表示入边。长度为 4 字节。
Rank	用来处理两点之间有多个同类型边的情况。用户可以根据自己的需求进行设置，例如存放交易时间、交易流水号等。长度为 8 字节，
Placeholder	预留字段。长度为 1 字节。
SerializedValue	序列化的 value，用于保存边的属性信息。

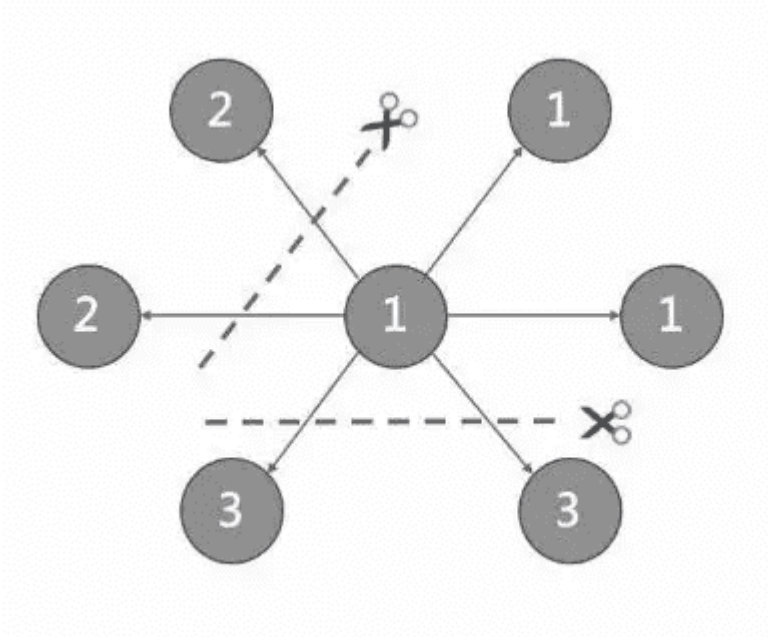
属性说明

TRSGraph 使用强类型 Schema。

对于点或边的属性信息，TRSGraph 会将属性信息编码后按顺序存储。由于定长属性的长度是固定的，查询时可以根据偏移量快速查询。在解码之前，需要先从 Meta 服务中查询具体的 Schema 信息（并缓存）。同时为了支持在线变更 Schema，在编码属性时，会加入对应的 Schema 版本信息。

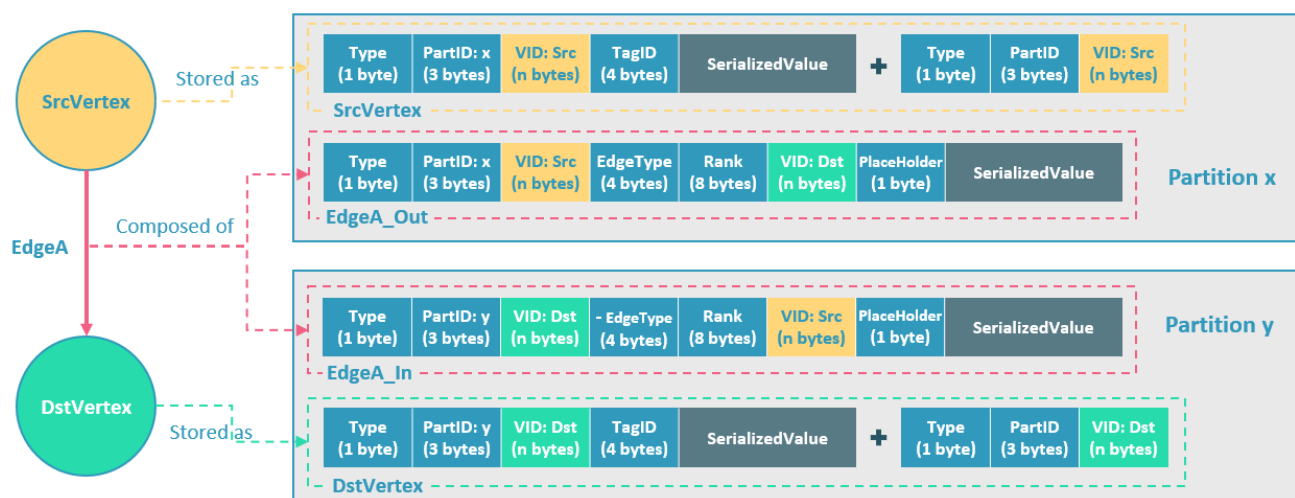
数据分片

由于超大规模关系网络的节点数量高达百亿到千亿，而边的数量更会高达万亿，即使仅存储点和边两者也远大于一般服务器的容量。因此需要有方法将图元素切割，并存储在不同逻辑分片（Partition）上。TRSGraph 采用边分割的方式。



切边与存储放大

TRSGraph 中逻辑上的一条边对应着硬盘上的两个键值对 (key-value pair)，在边的数量和属性较多时，存储放大现象较明显。边的存储方式如下图所示。



上图以最简单的两个点和一条边为例，起点 **SrcVertex** 通过边 **EdgeA** 连接目的点 **DstVertex**，形成路径 $(SrcVertex) - [EdgeA] \rightarrow (DstVertex)$ 。这两个点和一条边会以 6 个键值对的形式保存在存储层的两个不同分片，即 **Partition x** 和 **Partition y** 中，详细说明如下：

- 点 **SrcVertex** 的键值保存在 **Partition x** 中。
- 边 **EdgeA** 的第一份键值，这里用 **EdgeA_Out** 表示，与 **SrcVertex** 一同保存在 **Partition x** 中。key 的字段有 Type、PartID (x)、VID (Src，即点 **SrcVertex** 的 ID)、EdgeType (符号为正，代表边方向为出)、Rank (0)、VID (Dst，即点 **DstVertex** 的 ID) 和 Placeholder。SerializedValue 即 Value，是序列化的边属性。
- 点 **DstVertex** 的键值保存在 **Partition y** 中。
- 边 **EdgeA** 的第二份键值，这里用 **EdgeA_In** 表示，与 **DstVertex** 一同保存在 **Partition y** 中。key 的字段有 Type、PartID (y)、VID (Dst，即点 **DstVertex** 的 ID)、EdgeType (符号为负，代表边方向为入)、Rank (0)、VID (Src，即点 **SrcVertex** 的 ID) 和 Placeholder。SerializedValue 即 Value，是序列化的边属性，与 **EdgeA_Out** 中该部分的完全相同。

EdgeA_Out 和 **EdgeA_In** 以方向相反的两条边的形式存在于存储层，二者组合成了逻辑上的一条边 **EdgeA**。**EdgeA_Out** 用于从起点开始的遍历请求，例如 $(a) - [] \rightarrow ()$ ；**EdgeA_In** 用于指向目的点的遍历请求，或者说从目的点开始，沿着边的方向逆序进行的遍历请求，例如 $() - [] \rightarrow (a)$ 。

如 **EdgeA_Out** 和 **EdgeA_In** 一样，TRSGraph 冗余了存储每条边的信息，导致存储边所需的实际空间翻倍。因为边对应的 key 占用的硬盘空间较小，但 value 占用的空间与属性值的长度和数量成正比，所以，当边的属性值较大或数量较多时候，硬盘空间占用量会比较大。

分片算法

分片策略采用静态 Hash 的方式，即对点 VID 进行取模操作，同一个点的所有 Tag、出边和入边信息都会存储到同一个分片，这种方式极大地提升了查询效率。



Note

创建图空间时需指定分片数量，分片数量设置后无法修改，建议设置时提前满足业务将来的扩容需求。

多机集群部署时，分片分布在集群内的不同机器上。分片数量在 CREATE SPACE 语句中指定，此后不可更改。

如果需要将某些点放置在相同的分片（例如在一台机器上），可以参考公式或代码。

下文用简单代码说明 VID 和分片的关系。

```
// 如果 ID 长度为 8，为了兼容 1.0，将数据类型视为 int64。
uint64_t vid = 0;
if (id.size() == 8) {
    memcpy(static_cast<void*>(&vid), id.data(), 8);
} else {
    MurmurHash2 hash;
    vid = hash(id.data());
}
PartitionID pId = vid % numParts + 1;
```

简单来说，上述代码是将一个固定的字符串进行哈希计算，转换成数据类型为 int64 的数字（int64 数字的哈希计算结果是数字本身），将数字取模，然后加 1，即：

```
pId = vid % numParts + 1;
```

示例的部分参数说明如下。

参数	说明
<code>%</code>	取模运算。
<code>numParts</code>	VID 所在图空间的分片数，即 <code>CREATE SPACE</code> 语句中的 <code>partition_num</code> 值。
<code>pId</code>	VID 所在分片的 ID。

例如有 100 个分片，VID 为 1、101 和 1001 的三个点将会存储在相同的分片。分片 ID 和机器地址之间的映射是随机的，所以不能假定任何两个分片位于同一台机器上。

Raft

关于 RAFT 的简单介绍

分布式系统中，同一份数据通常会有多个副本，这样即使少数副本发生故障，系统仍可正常运行。这就需要一定的技术手段来保证多个副本之间的一致性。

基本原理：Raft 就是一种用于保证多副本一致性的协议。Raft 采用多个副本之间竞选的方式，赢得“超过半数”副本投票的（候选）副本成为 Leader，由 Leader 代表所有副本对外提供服务；其他 Follower 作为备份。当该 Leader 出现异常后（通信故障、运维命令等），其余 Follower 进行新一轮选举，投票出一个新的 Leader。Leader 和 Follower 之间通过心跳的方式相互探测是否存活，并以 Raft-wal 的方式写入硬盘，超过多个心跳仍无响应的副本会被认为发生故障。



因为 Raft-wal 需要定期写硬盘，如果硬盘写能力瓶颈会导致 Raft 心跳失败，导致重新发起选举。硬盘 IO 严重堵塞情况下，会导致长期无法选举出 Leader。

读写流程：对于客户端的每个写入请求，Leader 会将该写入以 Raft-wal 的方式，将该条同步给其他 Follower，并只有在“超过半数”副本都成功收到 Raft-wal 后，才会返回客户端该写入成功。对于客户端的每个读取请求，都直接访问 Leader，而 Follower 并不参与读请求服务。

故障流程：场景 1：考虑一个配置为单副本（图空间）的集群；如果系统只有一个副本时，其自身就是 Leader；如果其发生故障，系统将完全不可用。场景 2：考虑一个配置为 3 副本（图空间）的集群；如果系统有 3 个副本，其中一个副本是 Leader，其他 2 个副本是 Follower；即使原 Leader 发生故障，剩下两个副本仍可投票出一个新的 Leader（以及一个 Follower），此时系统仍可使用；但是当这 2 个副本中任一者再次发生故障后，由于投票人数不足，系统将完全不可用。



Raft 多副本的方式与 HDFS 多副本的方式是不同的，Raft 基于“多数派”投票，因此副本数量不能是偶数。

MULTI GROUP RAFT

由于 Storage 服务需要支持集群分布式架构，所以基于 Raft 协议实现了 Multi Group Raft，即每个分片的所有副本共同组成一个 Raft group，其中一个副本是 leader，其他副本是 follower，从而实现强一致性和高可用性。Raft 的部分实现如下。

由于 Raft 日志不允许空洞，TRSGraph 使用 Multi Group Raft 缓解此问题，分片数量较多时，可以有效提高 TRSGraph 的性能。但是分片数量太多会增加开销，例如 Raft group 内部存储的状态信息、WAL 文件，或者负载过低时的批量操作。

实现 Multi Group Raft 有 2 个关键点：

- 共享 Transport 层

每一个 Raft group 内部都需要向对应的 peer 发送消息，如果不能共享 Transport 层，会导致连接的开销巨大。

- 共享线程池

如果不共享一组线程池，会造成系统的线程数过多，导致大量的上下文切换开销。

批量 (BATCH) 操作

TRSGraph 中，每个分片都是串行写日志，为了提高吞吐，写日志时需要做批量操作，但是由于 TRSGraph 利用 WAL 实现一些特殊功能，需要对批量操作进行分组，这是 TRSGraph 的特色。

例如无锁 CAS 操作需要之前的 WAL 全部提交后才能执行，如果一个批量写入的 WAL 里包含了 CAS 类型的 WAL，就需要拆分成粒度更小的几个组，还要保证这几组 WAL 串行提交。

LEADER 切换 (TRANSFER LEADERSHIP)

leader 切换对于负载均衡至关重要，当把某个分片从一台机器迁移到另一台机器时，首先会检查分片是不是 leader，如果是的话，需要先切换 leader，数据迁移完毕之后，通常还要重新均衡 leader 分布。

对于 leader 来说，提交 leader 切换命令时，就会放弃自己的 leader 身份，当 follower 收到 leader 切换命令时，就会发起选举。

成员变更

为了避免脑裂，当一个 Raft group 的成员发生变化时，需要有一个中间状态，该状态下新旧 group 的多数派需要有重叠的部分，这样就防止了新的 group 或旧的 group 单方面做出决定。为了更加简化，Diego Ongaro 在自己的博士论文中提出每次只增减一个 peer 的方式，以保证新旧 group 的多数派总是有重叠。TRSGraph 也采用了这个方式，只不过增加成员和移除成员的实现有所区别。具体实现方式请参见 Raft Part class 里 addPeer/removePeer 的实现。

与 HDFS 的区别

Storage 服务基于 Raft 协议实现的分布式架构，与 HDFS 的分布式架构有一些区别。例如：

- Storage 服务本身通过 Raft 协议保证一致性，副本数量通常为奇数，方便进行选举 leader，而 HDFS 存储具体数据的 DataNode 需要通过 NameNode 保证一致性，对副本数量没有要求。
- Storage 服务只有 leader 副本提供读写服务，而 HDFS 的所有副本都可以提供读写服务。
- Storage 服务无法修改副本数量，只能在创建图空间时指定副本数量，而 HDFS 可以调整副本数量。
- Storage 服务是直接访问文件系统，而 HDFS 的上层（例如 HBase）需要先访问 HDFS，再访问到文件系统，远程过程调用（RPC）次数更多。

总而言之，Storage 服务更加轻量级，精简了一些功能，架构没有 HDFS 复杂，可以有效提高小块存储的读写性能。

最后更新: 2025年12月19日

3. 快速入门

3.1 本地部署

3.1.1 安装 TRSGraph

- 解压 tar 包

```
tar xzf trsgraph.tar.gz
```

后续操作

- 启动 TRSGraph
- 连接 TRSGraph

最后更新: 2025年12月19日

3.1.2 启动 TRSGraph 服务

TRSGraph 支持通过脚本管理服务。

使用脚本管理服务

使用脚本 `trsgraph.service` 管理服务，包括启动、停止、重启、中止和查看。



`trsgraph.service` 的默认路径是 `/usr/local/trsgraph/scripts`，如果修改过安装路径，请使用实际路径。

语法

```
$ sudo /usr/local/trsgraph/scripts/trsgraph.service
[-v] [-c <config_file_path>]
<start|stop|restart|kill|status>
<metad|graphd|storaged|all>
```

参数	说明
<code>-v</code>	显示详细调试信息。
<code>-c</code>	指定配置文件路径，默认路径为 <code>/usr/local/trsgraph/etc/</code> 。
<code>start</code>	启动服务。
<code>stop</code>	停止服务。
<code>restart</code>	重启服务。
<code>kill</code>	中止服务。
<code>status</code>	查看服务状态。
<code>metad</code>	管理 Meta 服务。
<code>graphd</code>	管理 Graph 服务。
<code>storaged</code>	管理 Storage 服务。
<code>all</code>	管理所有服务。

启动 TRSGraph 服务

执行如下命令启动服务：

```
$ sudo /usr/local/trsgraph/scripts/trsgraph.service start all
[INFO] Starting trsgraph-metad...
[INFO] Done
[INFO] Starting trsgraph-graphd...
[INFO] Done
[INFO] Starting trsgraph-storaged...
[INFO] Done
```

停止 TRSGraph 服务



请勿使用 `kill -9` 命令强制终止进程，否则可能较小概率出现数据丢失。

执行如下命令停止 TRSGraph 服务：

```
$ sudo /usr/local/trsgraph/scripts/trsgraph.service stop all
[INFO] Stopping trsgraph-metad...
[INFO] Done
[INFO] Stopping trsgraph-graphd...
[INFO] Done
[INFO] Stopping trsgraph-storaged...
[INFO] Done
```

查看 TRSGraph 服务

执行如下命令查看 TRSGraph 服务状态：

```
$ sudo /usr/local/trsgraph/scripts/trsgraph.service status all
```

- 如果返回如下结果，表示 TRSGraph 服务正常运行。

```
[INFO] trsgraph-metad(33fd35e): Running as 29020, Listening on 9559
[INFO] trsgraph-graphd(33fd35e): Running as 29095, Listening on 9669
[WARN] trsgraph-storaged after v3.0.0 will not start service until it is added to cluster.
[WARN] See Manage Storage hosts: ADD HOSTS in https://ai.tris.net.cn/trsgraph/
[INFO] trsgraph-storaged(33fd35e): Running as 29147, Listening on 9779
```



正常启动 TRSGraph 后，`trsgraph-storaged` 进程的端口显示红色。这是因为 `trsgraph-storaged` 在启动流程中会等待 `trsgraph-metad` 添加当前 Storage 服务，当前 Storage 服务收到 Ready 信号后才会正式启动服务。从 3.0.0 版本开始，在配置文件中添加的 Storage 节点无法直接读写，配置文件的作用仅仅是将 Storage 节点注册至 Meta 服务中。必须使用 `ADD HOSTS` 命令后，才能正常读写 Storage 节点。

- 如果返回类似如下结果，表示 TRSGraph 服务异常，可以根据异常服务信息进一步排查。

```
[INFO] trsgraph-metad: Running as 25600, Listening on 9559
[INFO] trsgraph-graphd: Exited
[INFO] trsgraph-storaged: Running as 25646, Listening on 9779
```

TRSGraph 服务由 Meta 服务、Graph 服务和 Storage 服务共同提供，这三种服务的配置文件都保存在安装目录的 `etc` 目录内，默认路径为 `/usr/local/trsgraph/etc/`，用户可以检查相应的配置文件排查问题。

下一步

- 连接 TRSGraph

最后更新: 2025年12月19日

3.1.3 连接 TRSGraph

本文介绍如何使用原生命令行客户端 TRSGraph Console 连接 TRSGraph。



Caution

首次连接到 TRSGraph 后，必须先注册 Storage 服务，才能正常查询数据。

TRSGraph 支持多种类型的客户端，包括命令行客户端、可视化界面客户端和流行编程语言客户端。

前提条件

- TRSGraph 服务已启动。
- 运行 TRSGraph Console 的机器和运行 TRSGraph 的服务器网络互通。
- TRSGraph Console 的版本兼容 TRSGraph 的版本。



Note

版本相同的 TRSGraph Console 和 TRSGraph 兼容程度最高，版本不同的 TRSGraph Console 连接 TRSGraph 时，可能会有兼容问题，或者无法连接并报错 `incompatible version between client and server`。

操作步骤

1. 在命令行界面中，切换工作目录至 `trsgraph-console` 文件所在目录。
2. 执行如下命令连接 TRSGraph。

- Linux 或 macOS

```
$ ./trsgraph-console -addr <ip> -port <port> -u <username> -p <password>  
[-t 120] [-e "nGQL_statement" | -f filename.nGQL]
```

- Windows

```
> trsgraph-console.exe -addr <ip> -port <port> -u <username> -p <password>  
[-t 120] [-e "nGQL_statement" | -f filename.nGQL]
```

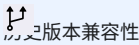
参数说明如下。

参数	说明
<code>-h/-help</code>	显示帮助菜单。
<code>-addr/-address</code>	设置要连接的 Graph 服务的 IP 或主机名。默认地址为 127.0.0.1。
<code>-P/-port</code>	设置要连接的 Graph 服务的端口。默认端口为 9669。
<code>-u/-user</code>	设置 TRSGraph 账号的用户名。未启用身份认证时，可以使用任意已存在的用户名（默认为 <code>root</code> ）。
<code>-p/-password</code>	设置用户名对应的密码。未启用身份认证时，密码可以填写任意字符。
<code>-t/-timeout</code>	设置整数类型的连接超时时间。单位为毫秒，默认值为 120。
<code>-e/-eval</code>	设置字符串类型的 nGQL 语句。连接成功后会执行一次该语句并返回结果，然后自动断开连接。
<code>-f/-file</code>	设置存储 nGQL 语句的文件的存储路径。连接成功后会执行该文件内的 nGQL 语句并返回结果，执行完毕后自动断开连接。
<code>-enable_ssl</code>	连接 TRSGraph 时使用 SSL 加密。
<code>-ssl_root_ca_path</code>	指定 CA 证书的存储路径。
<code>-ssl_cert_path</code>	指定 CRT 证书的存储路径。
<code>-ssl_private_key_path</code>	指定私钥文件的存储路径。

最后更新: 2025年12月19日

3.1.4 注册 Storage 服务

首次连接到 TRSGraph 后，需要先添加 Storage 主机，并确认主机都处于在线状态。



版本兼容性

从 TRSGraph 3.0.0 版本开始，必须先使用 `ADD HOSTS` 添加主机，才能正常通过 Storage 服务读写数据。

前提条件

已连接 TRSGraph 服务。

操作步骤

1. 添加 Storage 主机。

执行如下命令添加主机：

```
ADD HOSTS <ip>:<port> [,<ip>:<port> ...];
```

示例：

```
trsgraph> ADD HOSTS 192.168.10.100:9779, 192.168.10.101:9779, 192.168.10.102:9779;
```



Caution

请确保添加的主机 IP 和配置文件 `trsgraph-storaged.conf` 中 `local_ip` 配置的 IP 一致，否则会导致添加 Storage 主机失败。关于配置文件的详情，参见[配置管理](#)。

2. 检查主机状态，确认全部在线。

```
trsgraph> SHOW HOSTS;
+-----+-----+-----+-----+-----+-----+-----+
| Host          | Port | Status | Leader count | Leader distribution | Partition distribution | Version |
+-----+-----+-----+-----+-----+-----+-----+
| "192.168.10.100" | 9779 | "ONLINE" | 0           | "No valid partition" | "No valid partition" | "10.0" |
| "192.168.10.101" | 9779 | "ONLINE" | 0           | "No valid partition" | "No valid partition" | "10.0" |
| "192.168.10.102" | 9779 | "ONLINE" | 0           | "No valid partition" | "No valid partition" | "10.0" |
+-----+-----+-----+-----+-----+-----+-----+
```

在返回结果的 Status 列，可以看到所有 Storage 主机都在线。

最后更新: 2025年12月19日

3.1.5 使用常用 nGQL (CRUD 命令)

本文介绍 TRSGraph 查询语言的基础语法，包括用于 Schema 创建和常用增删改查操作的语句。

如需了解更多语句的用法，参见 [nGQL 指南](#)。

使用说明

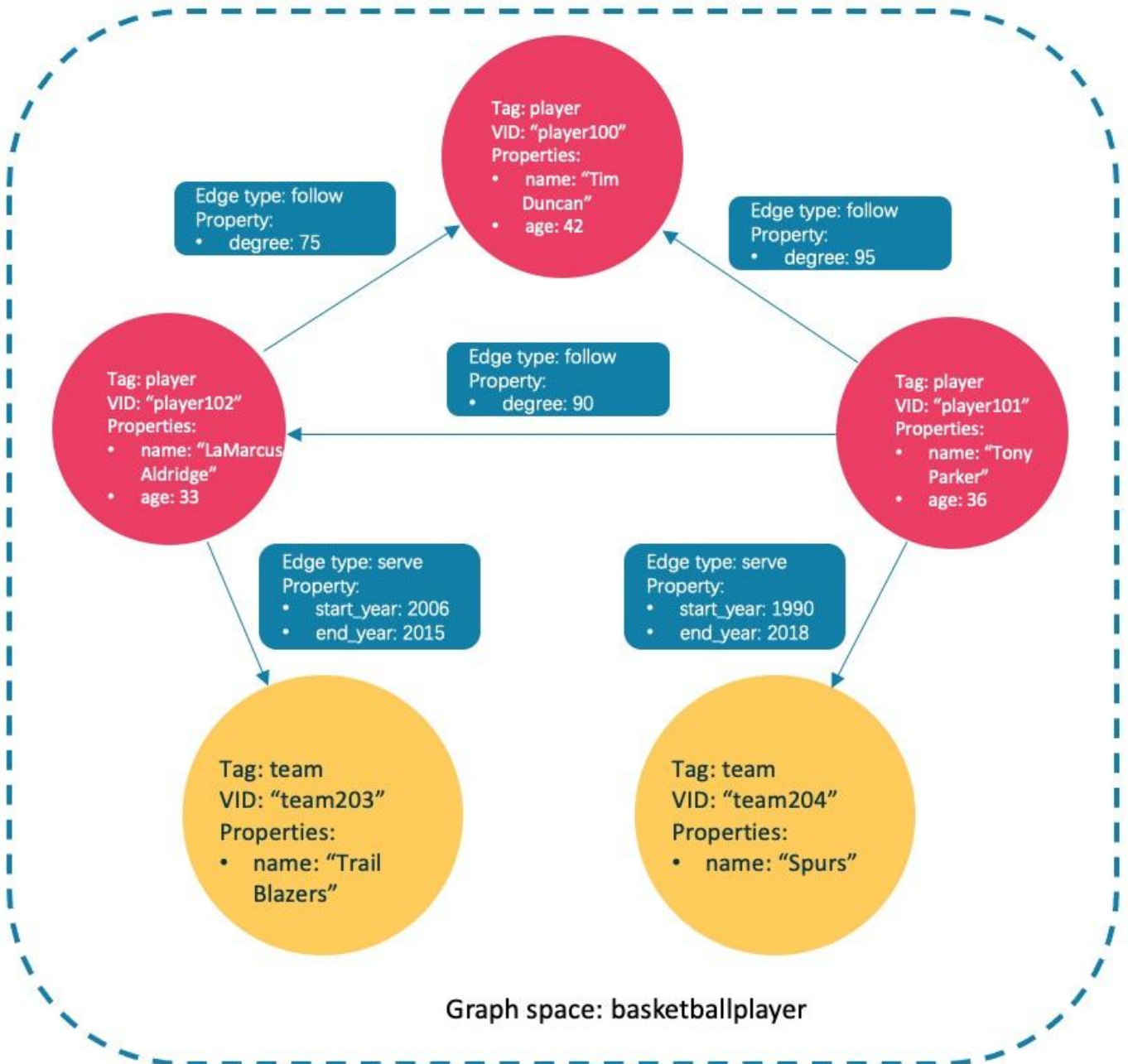
图空间和 SCHEMA

一个 TRSGraph 实例由一个或多个图空间组成。每个图空间都是物理隔离的，用户可以在同一个实例中使用不同的图空间存储不同的数据集。为了在图空间中插入数据，需要为图数据库定义一个 Schema。TRSGraph 的 Schema 是由如下几部分组成。

组成部分	说明
点 (Vertex)	表示现实世界中的实体。一个点可以有 0 到多个标签。
标签 (Tag)	点的类型，定义了一组描述点类型的属性。
边 (Edge)	表示两个点之间有方向的关系。
边类型 (Edge type)	边的类型，定义了一组描述边的类型的属性。

更多信息，请参见[数据结构](#)。

本文将使用下图的数据集演示基础操作的语法。



异步实现创建和修改



Caution

在 TRSGraph 中，下列创建和修改操作是异步实现的。要在下一个心跳周期之后才能生效，否则访问会报错。为确保数据同步，后续操作能顺利进行，请等待 2 个心跳周期（20 秒）。

- `CREATE SPACE`
- `CREATE TAG`
- `CREATE EDGE`
- `ALTER TAG`
- `ALTER EDGE`
- `CREATE TAG INDEX`
- `CREATE EDGE INDEX`



Note

默认心跳周期是 10 秒。修改心跳周期参数 `heartbeat_interval_secs`，请参见配置简介。

第一步：创建和选择图空间

NGQL 语法

- 创建图空间

```
CREATE SPACE [IF NOT EXISTS] <graph_space_name> (
  [partition_num = <partition_number>],
  [replica_factor = <replica_number>],
  vid_type = {FIXED_STRING(<N>) | INT64}
)
[COMMENT = '<comment>'];
```

参数详情请参见 [CREATE SPACE](#)。

- 列出创建成功的图空间

```
trsgraph> SHOW SPACES;
```

- 选择已创建的图空间

```
USE <graph_space_name>;
```

示例

1. 执行如下语句创建名为 `basketballplayer` 的图空间。

```
trsgraph> CREATE SPACE basketballplayer(partition_num=15, replica_factor=1, vid_type=fixed_string(30));
```



Note

如果报错提示 `[ERROR (-1005)]: Host not enough!`，请检查是否已添加 Storage 主机。

2. 执行命令 `SHOW HOSTS` 检查分片的分布情况，确保平衡分布。

```
trsgraph> SHOW HOSTS;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Host | IP | Port | Status | Partition Count | Replication Factor | Vid Type | Comment |
```

Host	Port	Status	Leader count	Leader distribution	Partition distribution	Version
"storage0"	9779	"ONLINE"	5	"basketballplayer:5"	"basketballplayer:5"	"10.0"
"storage1"	9779	"ONLINE"	5	"basketballplayer:5"	"basketballplayer:5"	"10.0"
"storage2"	9779	"ONLINE"	5	"basketballplayer:5"	"basketballplayer:5"	"10.0"

如果 Leader distribution 分布不均匀，请执行命令 `BALANCE LEADER` 重新分配。更多信息，请参见 [Storage 负载均衡](#)。

3. 选择图空间 `basketballplayer`。

```
trsgraph[(none)]> USE basketballplayer;
```

用户可以执行命令 `SHOW SPACES` 查看创建的图空间。

```
trsgraph> SHOW SPACES;
+-----+
| Name |
+-----+
| "basketballplayer" |
+-----+
```

第二步：创建 Tag 和 Edge type

NGQL 语法

```
CREATE {TAG | EDGE} [IF NOT EXISTS] {<tag_name> | <edge_type_name>}
(
  <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']
  [{, <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']} ...]
)
[TTL_DURATION = <ttl_duration>]
[TTL_COL = <prop_name>]
[COMMENT = '<comment>'];
```

参数详情请参见 [CREATE TAG](#) 和 [CREATE EDGE](#)。

示例

创建 Tag: `player` 和 `team`，以及 Edge type: `follow` 和 `serve`。说明如下表。

名称	类型	属性
player	Tag	name (string), age (int)
team	Tag	name (string)
follow	Edge type	degree (int)
serve	Edge type	start_year (int), end_year (int)

```
trsgraph> CREATE TAG player(name string, age int);

trsgraph> CREATE TAG team(name string);

trsgraph> CREATE EDGE follow(degree int);

trsgraph> CREATE EDGE serve(start_year int, end_year int);
```

第三步：插入数据

用户可以使用 `INSERT` 语句，基于现有的 Tag 插入点，或者基于现有的 Edge type 插入边。

NGQL 语法

• 插入点

```
INSERT VERTEX [IF NOT EXISTS] [tag_props, [tag_props] ...]
VALUES <vid>: ([prop_value_list])

tag_props:
  tag_name ([prop_name_list])

prop_name_list:
```



```
[prop_name [, prop_name] ...]

prop_value_list:
[ prop_value [, prop_value] ...]
```

`vid` 是 Vertex ID 的缩写, `vid` 在一个图空间中是唯一的。参数详情请参见 [INSERT VERTEX](#)。

- 插入边

```
INSERT EDGE [IF NOT EXISTS] <edge_type> ( <prop_name_list> ) VALUES
<src_vid> -> <dst_vid>[@<rank>] : ( <prop_value_list> )
[, <src_vid> -> <dst_vid>[@<rank>] : ( <prop_value_list> ), ...];

<prop_name_list> ::=
[ <prop_name> [, <prop_name> ] ...]

<prop_value_list> ::=
[ <prop_value> [, <prop_value> ] ...]
```

参数详情请参见 [INSERT EDGE](#)。

示例

- 插入代表球员和球队的点。

```
trsgraph> INSERT VERTEX player(name, age) VALUES "player100":("Tim Duncan", 42);

trsgraph> INSERT VERTEX player(name, age) VALUES "player101":("Tony Parker", 36);

trsgraph> INSERT VERTEX player(name, age) VALUES "player102":("LaMarcus Aldridge", 33);

trsgraph> INSERT VERTEX team(name) VALUES "team203":("Trail Blazers"), "team204":("Spurs");
```

- 插入代表球员和球队之间关系的边。

```
trsgraph> INSERT EDGE follow(degree) VALUES "player101" -> "player100":(95);

trsgraph> INSERT EDGE follow(degree) VALUES "player101" -> "player102":(90);

trsgraph> INSERT EDGE follow(degree) VALUES "player102" -> "player100":(75);

trsgraph> INSERT EDGE serve(start_year, end_year) VALUES "player101" -> "team204":(1999, 2018),"player102" -> "team203":(2006, 2015);
```

第四步: 查询数据

- `GO` 语句可以根据指定的条件遍历数据库。`GO` 语句从一个或多个点开始, 沿着一条或多条边遍历, 返回 `YIELD` 子句中指定的信息。
- `FETCH` 语句可以获得点或边的属性。
- `LOOKUP` 语句是基于索引的, 和 `WHERE` 子句一起使用, 查找符合特定条件的数据。
- `MATCH` 语句是查询图数据最常用的, 可以灵活的描述各种图模式, 但是它依赖索引去匹配 TRSGraph 中的数据模型, 性能也还需要调优。

NGQL 语法

- `GO`

```
GO [[<M> TO] <N> {STEP|STEPS}] FROM <vertex_list>
OVER <edge_type_list> [{REVERSELY | BIDIRECT}]
[ WHERE <conditions> ]
YIELD [DISTINCT] <return_list>
[ { SAMPLE <sample_list> | <limit_by_list_clause> } ]
[ { GROUP BY {<col_name> | expression} | <position>} YIELD <col_name>]
```

```
[| ORDER BY <expression> [{ASC | DESC}]]
[| LIMIT [<offset>,<number_rows>];
```

- **FETCH**

- 查询 Tag 属性

```
FETCH PROP ON {<tag_name>[, tag_name ...] | *}
<vid> [, vid ...]
YIELD <return_list> [AS <alias>];
```

- 查询边属性

```
FETCH PROP ON <edge_type> <src_vid> -> <dst_vid>[@<rank>] [, <src_vid> -> <dst_vid> ...]
YIELD <output>;
```

- **LOOKUP**

```
LOOKUP ON {<vertex_tag> | <edge_type>}
[WHERE <expression> [AND <expression> ...]]
YIELD <return_list> [AS <alias>];

<return_list>
  <prop_name> [AS <col_alias>] [, <prop_name> [AS <prop_alias>] ...];
```

- **MATCH**

```
MATCH <pattern> [<clause_1>] RETURN <output> [<clause_2>];
```

GO 语句示例

- 从 VID 为 `player101` 的球员开始, 沿着边 `follow` 找到连接的球员。

```
trsgaph> GO FROM "player101" OVER follow YIELD id($$);
+-----+
| id($$) |
+-----+
| "player100" |
```

```
| "player102" |
+-----+
```

- 从 VID 为 `player101` 的球员开始，沿着边 `follow` 查找年龄大于或等于 35 岁的球员，并返回他们的姓名和年龄，同时重命名对应的列。

```
trsgaph> GO FROM "player101" OVER follow WHERE properties($$).age >= 35 \
YIELD properties($$).name AS Teammate, properties($$).age AS Age;
+-----+
| Teammate      | Age |
+-----+
| "Tim Duncan"  | 42  |
+-----+
```

子句/符号	说明
<code>YIELD</code>	指定该查询需要返回的值或结果。
<code>\$\$</code>	表示边的终点。
<code>\</code>	表示换行继续输入。

- 从 VID 为 `player101` 的球员开始，沿着边 `follow` 查找连接的球员，然后检索这些球员的球队。为了合并这两个查询请求，可以使用管道符或临时变量。
- 使用管道符

```
trsgraph> GO FROM "player101" OVER follow YIELD dst(edge) AS id | \
GO FROM $-.id OVER serve YIELD properties($$).name AS Team, \
properties($^).name AS Player;
+-----+
| Team      | Player      |
+-----+
| "Trail Blazers" | "LaMarcus Aldridge" |
+-----+
```

子句/符号	说明
<code>\$^</code>	表示边的起点。
<code> </code>	组合多个查询的管道符，将前一个查询的结果集用于后一个查询。
<code>\$-</code>	表示管道符前面的查询输出的结果集。

- 使用临时变量



当复合语句作为一个整体提交给服务器时，其中的临时变量会在语句结束时被释放。

```
trsgraph> $var = GO FROM "player101" OVER follow YIELD dst(edge) AS id; \
GO FROM $var.id OVER serve YIELD properties($$).name AS Team, \
properties($^).name AS Player;
+-----+
| Team      | Player      |
+-----+
| "Trail Blazers" | "LaMarcus Aldridge" |
+-----+
```

FETCH 语句示例

查询 VID 为 `player100` 的球员的属性。

```
trsgraph> FETCH PROP ON player "player100" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 42, name: "Tim Duncan"} |
+-----+
```



LOOKUP 和 MATCH 的示例在下文的索引部分查看。

其他操作

修改点和边

用户可以使用 UPDATE 语句或 UPSERT 语句修改现有数据。

UPSERT 是 UPDATE 和 INSERT 的结合体。当使用 UPSERT 更新一个点或边，如果它不存在，数据库会自动插入一个新的点或边。



每个 partition 内部，UPSERT 操作是一个串行操作，所以执行速度比执行 INSERT 或 UPDATE 慢很多。其仅在多个 partition 之间有并发。

nGQL 语法

- UPDATE 点

```
UPDATE VERTEX <vid> SET <properties to be updated>
[WHEN <condition>] [YIELD <columns>];
```

- UPDATE 边

```
UPDATE EDGE ON <edge_type> <source vid> -> <destination vid> [@rank]
SET <properties to be updated> [WHEN <condition>] [YIELD <columns to be output>];
```

- UPSERT 点或边

```
UPSERT {VERTEX <vid> | EDGE <edge_type>} SET <update_columns>
[WHEN <condition>] [YIELD <columns>];
```

示例

- 用 UPDATE 修改 VID 为 player100 的球员的 name 属性，然后用 FETCH 语句检查结果。

```
trsgaph> UPDATE VERTEX "player100" SET player.name = "Tim";

trsgaph> FETCH PROP ON player "player100" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
```

```
| {age: 42, name: "Tim"} |
+-----+
```

- 用 `UPDATE` 修改某条边的 `degree` 属性，然后用 `FETCH` 检查结果。

```
trsgraph> UPDATE EDGE ON follow "player101" -> "player100" SET degree = 96;

trsgraph> FETCH PROP ON follow "player101" -> "player100" YIELD properties(edge);
+-----+
| properties(EDGE) |
+-----+
| {degree: 96}      |
+-----+
```

- 用 `INSERT` 插入一个 VID 为 `player111` 的点，然后用 `UPSET` 更新它。

```
trsgraph> INSERT VERTEX player(name,age) VALUES "player111":("David West", 38);

trsgraph> UPSERT VERTEX "player111" SET player.name = "David", player.age = $^.player.age + 11 \
    WHEN $^.player.name == "David West" AND $^.player.age > 20 \
    YIELD $^.player.name AS Name, $^.player.age AS Age;
+-----+
| Name   | Age |
+-----+
| "David" | 49  |
+-----+
```

删除点和边

nGQL 语法

- 删除点

```
DELETE VERTEX <vid1>[, <vid2>...]
```

- 删除边

```
DELETE EDGE <edge_type> <src_vid> -> <dst_vid>[@<rank>]
[, <src_vid> -> <dst_vid>...]
```

示例

- 删除点

```
trsgraph> DELETE VERTEX "player111", "team203";
```

- 删除边

```
trsgraph> DELETE EDGE follow "player101" -> "team204";
```

使用索引

用户可以通过 `CREATE INDEX` 语句为 Tag 和 Edge type 增加索引。



Caution

`MATCH` 和 `LOOKUP` 语句的执行都依赖索引，但是索引会导致写性能大幅降低。请不要随意在生产环境中使用索引，除非很清楚使用索引对业务的影响。

必须为“已写入但未构建索引”的数据重建索引，否则无法在 `MATCH` 和 `LOOKUP` 语句中返回这些数据。参见重建索引。

nGQL 语法

- 创建索引

```
CREATE {TAG | EDGE} INDEX [IF NOT EXISTS] <index_name>
ON {<tag_name> | <edge_name>} ({<prop_name_list>}) [COMMENT = '<comment>'];
```

• 重建索引

```
REBUILD {TAG | EDGE} INDEX <index_name>;
```



为没有指定长度的变量属性创建索引时，需要指定索引长度。在 utf-8 编码中，一个中文字符占 3 字节，请根据变量属性长度设置合适的索引长度。例如 10 个中文字符，索引长度需要为 30。详情请参见[创建索引](#)。

基于索引的 LOOKUP 和 MATCH 示例

确保 LOOKUP 或 MATCH 有一个索引可用。如果没有，请先创建索引。

找到 Tag 为 player 的点的信息，它的 name 属性值为 Tony Parker。

```
// 为 name 属性创建索引 player_index_1。
trsgraph> CREATE TAG INDEX IF NOT EXISTS player_index_1 ON player(name(20));

// 重建索引确保能对已存在数据生效。
trsgraph> REBUILD TAG INDEX player_index_1
+-----+
| New Job Id |
+-----+
| 31         |
+-----+

// 使用 LOOKUP 语句检索点的属性。
trsgraph> LOOKUP ON player WHERE player.name == "Tony Parker" \
        YIELD properties(vertex).name AS name, properties(vertex).age AS age;
+-----+-----+
| name      | age |
+-----+-----+
| "Tony Parker" | 36 |
+-----+-----+

// 使用 MATCH 语句检索点的属性。
trsgraph> MATCH (v:player{name:"Tony Parker"}) RETURN v;
+-----+-----+
| v      |
+-----+-----+
| ("player101":player{age: 36, name: "Tony Parker"}) |
+-----+-----+
```

最后更新: 2025年12月19日

3.2 nGQL 命令汇总

3.2.1 函数

数学函数

函数	说明
<code>double abs(double x)</code>	返回 x 的绝对值。
<code>double floor(double x)</code>	返回小于或等于 x 的最大整数。
<code>double ceil(double x)</code>	返回大于或等于 x 的最小整数。
<code>double round(double x)</code>	返回离 x 最近的整数值，如果 x 恰好在中间，则返回离 0 较远的整数。
<code>double sqrt(double x)</code>	返回 x 的平方根。
<code>double cbrt(double x)</code>	返回 x 的立方根。
<code>double hypot(double x, double y)</code>	返回直角三角形（直角边长为 x 和 y ）的斜边长。
<code>double pow(double x, double y)</code>	返回 x^y 的值。
<code>double exp(double x)</code>	返回 e^x 的值。
<code>double exp2(double x)</code>	返回 2^x 的值。
<code>double log(double x)</code>	返回以自然数 e 为底 x 的对数。
<code>double log2(double x)</code>	返回以 2 为底 x 的对数。
<code>double log10(double x)</code>	返回以 10 为底 x 的对数。
<code>double sin(double x)</code>	返回 x 的正弦值。
<code>double asin(double x)</code>	返回 x 的反正弦值。
<code>double cos(double x)</code>	返回 x 的余弦值。
<code>double acos(double x)</code>	返回 x 的反余弦值。
<code>double tan(double x)</code>	返回 x 的正切值。
<code>double atan(double x)</code>	返回 x 的反正切值。
<code>double rand()</code>	返回 $[0,1)$ 内的随机浮点数。
<code>int rand32(int min, int max)</code>	返回 $[\text{min}, \text{max})$ 内的一个随机 32 位整数。用户可以只传入一个参数，该参数会判定为 <code>max</code> ，此时 <code>min</code> 默认为 0。如果不传入参数，此时会从带符号的 32 位 <code>int</code> 范围内随机返回。
<code>int rand64(int min, int max)</code>	返回 $[\text{min}, \text{max})$ 内的一个随机 64 位整数。用户可以只传入一个参数，该参数会判定为 <code>max</code> ，此时 <code>min</code> 默认为 0。如果不传入参数，此时会从带符号的 64 位 <code>int</code> 范围内随机返回。
<code>bit_and()</code>	逐位做 AND 操作。
<code>bit_or()</code>	逐位做 OR 操作。
<code>bit_xor()</code>	逐位做 XOR 操作。
<code>int size()</code>	返回列表或映射中元素的数量，或字符串的长度。
<code>int range(int start, int end, int step)</code>	返回 $[\text{start}, \text{end}]$ 中指定步长的值组成的列表。步长 <code>step</code> 默认为 1。
<code>int sign(double x)</code>	返回 x 的正负号。如果 x 为 0，则返回 0。如果 x 为负数，则返回 -1。如果 x 为正数，则返回 1。
<code>double e()</code>	返回自然对数的底 e (2.718281828459045)。
<code>double pi()</code>	返回数学常数 π (3.141592653589793)。
<code>double radians()</code>	将角度转换为弧度。 <code>radians(180)</code> 返回 3.141592653589793。

聚合函数

函数	说明
avg()	返回参数的平均值。
count()	语法： <code>count({expr *})</code> 。 <code>count()</code> 返回总行数（包括 NULL）。 <code>count(expr)</code> 返回满足表达式的非空值的总数。 <code>count()</code> 和 <code>size()</code> 是不同的。
max()	返回参数的最大值。
min()	返回参数的最小值。
collect()	<code>collect()</code> 函数返回一个符合表达式返回结果的列表。该函数可以将多条记录或值合并进一个列表，实现数据聚合。
std()	返回参数的总体标准差。
sum()	返回参数的和。

字符串函数

函数	说明
<code>int strcmp(string a, string b)</code>	比较两个字符串（不区分大小写）。当 <code>a=b</code> 时，返回 0，当 <code>a>b</code> 是，返回大于 0 的数，当 <code>a<b</code> 时，返回小于 0 的数。
<code>string lower(string a)</code>	返回小写形式的字符串。
<code>string toLower(string a)</code>	和 <code>lower()</code> 相同。
<code>string upper(string a)</code>	返回大写形式的字符串。
<code>string toUpper(string a)</code>	和 <code>upper()</code> 相同。
<code>int length(a)</code>	返回给定字符串的长度或路径的长度，单位分别是字节和跳数。
<code>string trim(string a)</code>	删除字符串头部和尾部的空格。
<code>string ltrim(string a)</code>	删除字符串头部的空格。
<code>string rtrim(string a)</code>	删除字符串尾部的空格。
<code>string left(string a, int count)</code>	返回字符串左侧 <code>count</code> 个字符组成的子字符串。如果 <code>count</code> 超过字符串 <code>a</code> 的长度，则返回字符串 <code>a</code> 。
<code>string right(string a, int count)</code>	返回字符串右侧 <code>count</code> 个字符组成的子字符串。如果 <code>count</code> 超过字符串 <code>a</code> 的长度，则返回字符串 <code>a</code> 。
<code>string lpad(string a, int size, string letters)</code>	在字符串 <code>a</code> 的左侧填充 <code>letters</code> 字符串，并返回 <code>size</code> 长度的字符串。
<code>string rpad(string a, int size, string letters)</code>	在字符串 <code>a</code> 的右侧填充 <code>letters</code> 字符串，并返回 <code>size</code> 长度的字符串。
<code>string substr(string a, int pos, int count)</code>	从字符串 <code>a</code> 的指定位置 <code>pos</code> 开始（不包括 <code>pos</code> 位置的字符），提取右侧的 <code>count</code> 个字符，组成新的字符串并返回。
<code>string substring(string a, int pos, int count)</code>	和 <code>substr()</code> 相同。
<code>string reverse(string)</code>	逆序返回字符串。
<code>string replace(string a, string b, string c)</code>	将字符串 <code>a</code> 中的子字符串 <code>b</code> 替换为字符串 <code>c</code> 。
<code>list split(string a, string b)</code>	在子字符串 <code>b</code> 处拆分字符串 <code>a</code> ，返回一个字符串列表。
<code>concat()</code>	<code>concat()</code> 函数至少需要两个或以上字符串参数，并将所有参数连接成一个字符串。 语法： <code>concat(string1,string2,...)</code>
<code>concat_ws()</code>	<code>concat_ws()</code> 函数将两个或以上字符串参数与预定义的分隔符（separator）相连接。
<code>extract()</code>	<code>extract()</code> 从指定字符串中提取符合正则表达式的子字符串。
<code>json_extract()</code>	<code>json_extract()</code> 将指定 JSON 字符串转换为 map 类型。

• 日期时间函数

函数	说明
int now()	根据当前系统返回当前时间戳。
timestamp timestamp()	根据当前系统返回当前时间戳。
date date()	根据当前系统返回当前日期（UTC 时间）。
time time()	根据当前系统返回当前时间（UTC 时间）。
datetime datetime()	根据当前系统返回当前日期和时间（UTC 时间）。

• Schema 相关函数

• 原生 nGQL 语句适用

函数	说明
id(vertex)	返回点 ID。数据类型和点 ID 的类型保持一致。
map properties(vertex)	返回点的所有属性。
map properties(edge)	返回边的所有属性。
string type(edge)	返回边的 Edge type。
src(edge)	返回边的起始点 ID。数据类型和点 ID 的类型保持一致。
dst(edge)	返回边的目的点 ID。数据类型和点 ID 的类型保持一致。
int rank(edge)	返回边的 rank。
vertex	返回点的信息。包括点 ID、Tag、属性和值。
edge	返回边的信息。包括 Edge type、起始点 ID、目的点 ID、rank、属性和值。
vertices	返回子图中的点的信息。详情参见 GET SUBGRAPH 。
edges	返回子图中的边的信息。详情参见 GET SUBGRAPH 。
path	返回路径信息。详情参见 FIND PATH 。

• openCypher 兼容语句适用

函数	说明
id(<vertex>)	返回点 ID。数据类型和点 ID 的类型保持一致。
list tags(<vertex>)	返回点的 Tag，与 labels() 作用相同。
list labels(<vertex>)	返回点的 Tag，与 tags() 作用相同，用于兼容 openCypher 语法。
map properties(<vertex_or_edge>)	返回点或边的所有属性。
string type(<edge>)	返回边的 Edge type。
src(<edge>)	返回边的起始点 ID。数据类型和点 ID 的类型保持一致。
dst(<edge>)	返回边的目的点 ID。数据类型和点 ID 的类型保持一致。
vertex startNode(<path>)	获取一条边或一条路径并返回它的起始点 ID。
string endNode(<path>)	获取一条边或一条路径并返回它的目的点 ID。
int rank(<edge>)	返回边的 rank。

列表函数

函数	说明
keys(expr)	返回一个列表，包含字符串形式的点、边或映射的所有属性。
labels(vertex)	返回点的 Tag 列表。
nodes(path)	返回路径中所有点的列表。
range(start, end [, step])	返回 [start,end] 范围内固定步长的列表，默认步长 step 为 1。
relationships(path)	返回路径中所有关系的列表。
reverse(list)	返回将原列表逆序排列的新列表。
tail(list)	返回不包含原列表第一个元素的新列表。
head(list)	返回列表的第一个元素。
last(list)	返回列表的最后一个元素。
reduce()	将表达式逐个应用于列表中的元素，然后和累加器中的当前结果累加，最后返回完整结果。

类型转换函数

函数	说明
bool toBoolean()	将字符串转换为布尔。
float toFloat()	将整数或字符串转换为浮点数。
string toString()	将任意数据类型转换为字符串类型。
int toInteger()	将浮点或字符串转换为整数。
set toSet()	将列表或集合转换为集合。
int hash()	hash() 函数返回参数的哈希值。其参数可以是数字、字符串、列表、布尔值、NULL 等类型的值，或者计算结果为这些类型的表达式。

谓词函数

谓词函数只返回 true 或 false，通常用于 WHERE 子句中。

```
<predicate>(<variable> IN <list> WHERE <condition>)
```

函数	说明
exists()	如果指定的属性在点、边或映射中存在，则返回 true，否则返回 false。
any()	如果指定的谓词适用于列表中的至少一个元素，则返回 true，否则返回 false。
all()	如果指定的谓词适用于列表中的每个元素，则返回 true，否则返回 false。
none()	如果指定的谓词不适用于列表中的任何一个元素，则返回 true，否则返回 false。
single()	如果指定的谓词适用于列表中的唯一一个元素，则返回 true，否则返回 false。

• 条件表达式函数

函数	说明
CASE	使用条件来过滤 nGQL 查询语句的结果，常用于 <code>YIELD</code> 和 <code>RETURN</code> 子句中。 <code>CASE</code> 表达式会遍历所有条件，并在满足第一个条件时停止读取后续条件，然后返回结果。 如果不满足任何条件，将通过 <code>ELSE</code> 子句返回结果。如果没有 <code>ELSE</code> 子句且不满足任何条件，则返回 <code>NULL</code> 。
coalesce()	返回所有表达式中第一个非空元素。

3.2.2 通用查询语句

• MATCH

```
MATCH <pattern> [<clause_1>] RETURN <output> [<clause_2>];
```

模式	示例	说明
匹配点	<code>(v)</code>	用户可以在一对括号中使用自定义变量来表示模式中的点。例如 <code>(v)</code> 。
匹配 Tag	<code>MATCH (v:player) RETURN v</code>	用户可以在点的右侧用 <code>:<tag_name></code> 表示模式中的 Tag。
匹配多 Tag	<code>MATCH (v:player:team) RETURN v</code>	用户可以用英文冒号 (:) 匹配多 Tag 的点。
匹配点的属性	<pre>MATCH (v:player{name:"Tim Duncan"}) RETURN v MATCH (v) WITH v, properties(v) as props, keys(properties(v)) as kk WHERE [i in kk where props[i] == "Tim Duncan"] RETURN v</pre>	用户可以在 Tag 的右侧用 <code>{<prop_name>: <prop_value>}</code> 表示模式中点的属性；或者不指定 Tag 直接匹配点的属性。
匹配单点 ID	<code>MATCH (v) WHERE id(v) == 'player101' RETURN v</code>	用户可以使用点 ID 去匹配点。 <code>id()</code> 函数可以检索点的 ID。
匹配多点 ID	<pre>MATCH (v:player { name: 'Tim Duncan' })-- (v2) WHERE id(v2) IN ["player101", "player102"] RETURN v2</pre>	要匹配多个点的 ID，可以用 <code>WHERE id(v) IN [vid_list]</code> 。
匹配连接的点	<pre>MATCH (v:player{name:"Tim Duncan"})--(v2) RETURN v2.player.name AS Name</pre>	用户可以使用 <code>--</code> 符号表示两个方向的边，并匹配这些边连接的点。用户可以在 <code>--</code> 符号上增加 <code><</code> 或 <code>></code> 符号指定边的方向。
匹配路径	<pre>MATCH p=(v:player{name:"Tim Duncan"})-- >(v2) RETURN p</pre>	连接起来的点和边构成了路径。用户可以使用自定义变量命名路径。
匹配边	<pre>MATCH (v:player{name:"Tim Duncan"})-[e]- (v2) RETURN e MATCH ()<-[e]-() RETURN e</pre>	除了用 <code>--</code> 、 <code>--></code> 、 <code><--</code> 表示未命名的边之外，用户还可以在方括号中使用自定义变量命名边。例如 <code>-[e]-</code> 。
匹配 Edge type	<code>MATCH ()-[e:follow]->() RETURN e</code>	和点一样，用户可以用 <code>:<edge_type></code> 表示模式中的 Edge type，例如 <code>-[e:follow]-</code> 。
匹配边的属性	<pre>MATCH (v:player{name:"Tim Duncan"})-[e:follow{degree:95}]->(v2) RETURN e MATCH ()-[e]->() WITH e, properties(e) as props, keys(properties(e)) as kk WHERE [i in kk where props[i] == 90] RETURN e</pre>	用户可以用 <code>{<prop_name>: <prop_value>}</code> 表示模式中 Edge type 的属性，例如 <code>[e:follow{likeness:95}]</code> ；或者不指定 Edge type 直接匹配边的属性。
匹配多个 Edge type	<code>MATCH (v:player{name:"Tim Duncan"})-[e:follow :serve]->(v2) RETURN e</code>	使用 <code> </code> 可以匹配多个 Edge type，例如 <code>[e:follow :serve]</code> 。第一个 Edge type 前的英文冒号 (:) 不可省略，后续 Edge type 前的英文冒号可以省略，例如 <code>[e:follow serve]</code> 。
匹配多条边	<pre>MATCH (v:player{name:"Tim Duncan"})-[]- >(v2)<-[e:serve]->(v3) RETURN v2, v3</pre>	用户可以扩展模式，匹配路径中的多条边。
匹配定长路径	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*2]->(v2) RETURN DISTINCT v2 AS Friends</pre>	用户可以在模式中使用 <code>:<edge_type>*<hop></code> 匹配定长路径。 <code>hop</code> 必须是一个非负整数。 <code>e</code> 的数据类型是列表。
匹配变长路径	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*1..3]->(v2) RETURN v2 AS Friends</pre>	<code>minHop</code> ：可选项。表示路径的最小长度。 <code>minHop</code> 必须是一个非负整数，默认为 1。 <code>maxHop</code> ：可选项。表示路径的最大长度。 <code>maxHop</code> 必须是一个非负整数，默认为无穷大。 <code>e</code> 的数据类型是列表。
匹配多个 Edge type 的变长路径	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[e:follow serve*2]->(v2) RETURN DISTINCT v2</pre>	用户可以在变长或定长模式中指定多个 Edge type。 <code>hop</code> 、 <code>minHop</code> 和 <code>maxHop</code> 对所有 Edge type 都生效。 <code>e</code> 的数据类型是列表。

模式	示例	说明
检索点或边的信息	<pre>MATCH (v:player{name:"Tim Duncan"}) RETURN v MATCH (v:player{name:"Tim Duncan"})-[e]->(v2) RETURN e</pre>	使用 <code>RETURN {<vertex_name> <edge_name>}</code> 检索点或边的所有信息。
检索点 ID	<pre>MATCH (v:player{name:"Tim Duncan"}) RETURN id(v)</pre>	使用 <code>id()</code> 函数检索点 ID。
检索 Tag	<pre>MATCH (v:player{name:"Tim Duncan"}) RETURN labels(v)</pre>	使用 <code>labels()</code> 函数检索点上的 Tag 列表。 检索列表 <code>labels(v)</code> 中的第 N 个元素, 可以使用 <code>labels(v)[n-1]</code> 。
检索点或边的单个属性	<pre>MATCH (v:player{name:"Tim Duncan"}) RETURN v.player.age</pre>	使用 <code>RETURN {<vertex_name> <edge_name>}.<property></code> 检索单个属性。 使用 <code>AS</code> 设置属性的别名。
检索点或边的所有属性	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) RETURN properties(v2)</pre>	使用 <code>properties()</code> 函数检索点或边的所有属性。
检索 Edge type	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[e]->() RETURN DISTINCT type(e)</pre>	使用 <code>type()</code> 函数检索匹配的 Edge type。
检索路径	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[*3]->() RETURN p</pre>	使用 <code>RETURN <path_name></code> 检索匹配路径的所有信息。
检索路径中的点	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) RETURN nodes(p)</pre>	使用 <code>nodes()</code> 函数检索路径中的所有点。
检索路径中的边	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) RETURN relationships(p)</pre>	使用 <code>relationships()</code> 函数检索路径中的所有边。
检索路径长度	<pre>MATCH p=(v:player{name:"Tim Duncan"})-[*..2]->(v2) RETURN p AS Paths, length(p) AS Length</pre>	使用 <code>length()</code> 函数检索路径的长度。

• OPTIONAL MATCH

模式	示例	说明
作为 <code>MATCH</code> 语句的可选项去匹配图数据库中的模式	<pre>MATCH (m)-[]->(n) WHERE id(m)=="player100" OPTIONAL MATCH (n)-[]->(l) RETURN id(m),id(n),id(l)</pre>	如果图数据库中没有对应的模式, 对应的列返回 <code>NULL</code> 。

• LOOKUP

```
LOOKUP ON {<vertex_tag> | <edge_type>}
[WHERE <expression> [AND <expression> ...]]
YIELD <return_list> [AS <alias>]
```

模式	示例	说明
检索点	<code>LOOKUP ON player WHERE player.name == "Tony Parker" YIELD player.name AS name, player.age AS age</code>	返回 Tag 为 <code>player</code> 且 <code>name</code> 为 <code>Tony Parker</code> 的点。
检索边	<code>LOOKUP ON follow WHERE follow.degree == 90 YIELD follow.degree</code>	返回 Edge type 为 <code>follow</code> 且 <code>degree</code> 为 <code>90</code> 的边。
通过 Tag 列出所有点	<code>LOOKUP ON player YIELD properties(vertex),id(vertex)</code>	查找所有 Tag 为 <code>player</code> 的点 VID。
通过 Edge type 列出边	<code>LOOKUP ON follow YIELD edge AS e</code>	查找 Edge type 为 <code>follow</code> 的所有边的信息。
统计点	<code>LOOKUP ON player YIELD id(vertex) YIELD COUNT(*) AS Player_Count</code>	统计 Tag 为 <code>player</code> 的点。
统计边	<code>LOOKUP ON follow YIELD edge as e YIELD COUNT(*) AS Like_Count</code>	统计 Edge type 为 <code>follow</code> 的边。

• GO

```
GO [[<M> TO] <N> {STEP|STEPS} ] FROM <vertex_list>
OVER <edge_type_list> [{REVERSELY | BIDIRECT}]
[ WHERE <conditions> ]
YIELD [DISTINCT] <return_list>
[{SAMPLE <sample_list> | LIMIT <limit_list>}]
[| GROUP BY {col_name | expr | position} YIELD <col_name>]
[| ORDER BY <expression> [{ASC | DESC}]]
[| LIMIT [<offset_value>,< >] <number_rows>]
```

示例	说明
<code>GO FROM "player102" OVER serve YIELD dst(edge)</code>	返回 <code>player102</code> 所属队伍。
<code>GO 2 STEPS FROM "player102" OVER follow YIELD dst(edge)</code>	返回距离 <code>player102</code> 两跳的朋友。
<code>GO FROM "player100", "player102" OVER serve WHERE properties(edge).start_year > 1995 YIELD DISTINCT properties(\$\$.name AS team_name, properties(edge).start_year AS start_year, properties(\$^).name AS player_name</code>	添加过滤条件。
<code>GO FROM "player100" OVER follow, serve YIELD properties(edge).degree, properties(edge).start_year</code>	遍历多个 Edge type。属性没有值时，会显示 <code>NULL</code> 。
<code>GO FROM "player100" OVER follow REVERSELY YIELD src(edge) AS destination</code>	返回 <code>player100</code> 入方向的邻居点。
<code>GO FROM "player100" OVER follow REVERSELY YIELD src(edge) AS id GO FROM \$-.id OVER serve WHERE properties(\$^).age > 20 YIELD properties(\$^).name AS FriendOf, properties(\$\$.name AS Team</code>	查询 <code>player100</code> 的朋友和朋友所属队伍。
<code>GO FROM "player102" OVER follow YIELD dst(edge) AS both</code>	返回 <code>player102</code> 所有邻居点。
<code>GO 2 STEPS FROM "player100" OVER follow YIELD src(edge) AS src, dst(edge) AS dst, properties(\$\$.age AS age GROUP BY \$-.dst YIELD \$-.dst AS dst, collect_set(\$-.src) AS src, collect(\$-.age) AS age</code>	根据年龄分组。

• FETCH

• 获取点的属性值

```
FETCH PROP ON {<tag_name>[, tag_name ...] | *}
<vid> [, vid ...]
YIELD <return_list> [AS <alias>]
```

示例

```
FETCH PROP ON player "player100" YIELD properties(vertex)
```

说明

在 `FETCH` 语句中指定 Tag 获取对应点的属性值。

```
FETCH PROP ON player "player100" YIELD player.name AS name
```

使用 `YIELD` 子句指定返回的属性。

```
FETCH PROP ON player "player101", "player102", "player103" YIELD
properties(vertex)
```

指定多个点 ID 获取多个点的属性值，点之间用英文逗号 (,) 分隔。

```
FETCH PROP ON player, t1 "player100", "player103" YIELD
properties(vertex)
```

在 `FETCH` 语句中指定多个 Tag 获取属性值。Tag 之间用英文逗号 (,) 分隔。

```
FETCH PROP ON * "player100", "player106", "team200" YIELD
properties(vertex)
```

在 `FETCH` 语句中使用 `*` 获取当前图空间所有标签里，点的属性值。

• 获取边的属性值

```
FETCH PROP ON <edge_type> <src_vid> -> <dst_vid>[@<rank>] [, <src_vid> -> <dst_vid> ...]
YIELD <output>;
```

示例

```
FETCH PROP ON serve "player100" -> "team204" YIELD properties(edge)
```

说明

获取连接 `player100` 和 `team204` 的边 `serve` 的所有属性值。

```
FETCH PROP ON serve "player100" -> "team204" YIELD serve.start_year
```

使用 `YIELD` 子句指定返回的属性。

```
FETCH PROP ON serve "player100" -> "team204", "player133" ->
"team202" YIELD properties(edge)
```

指定多个边模式 (`<src_vid> -> <dst_vid>[@<rank>]`) 获取多个边的属性值。模式之间用英文逗号 (,) 分隔。

```
FETCH PROP ON serve "player100" -> "team204"@1 YIELD
properties(edge)
```

要获取 `rank` 不为 0 的边，请在 `FETCH` 语句中设置 `rank`。

```
GO FROM "player101" OVER follow YIELD follow._src AS s, follow._dst
AS d | FETCH PROP ON follow $-.s -> $-.d YIELD follow.degree
```

返回从点 `player101` 开始的 `follow` 边的 `degree` 值。

```
$var = GO FROM "player101" OVER follow YIELD follow._src AS s,
follow._dst AS d; FETCH PROP ON follow $var.s -> $var.d YIELD
follow.degree
```

自定义变量构建查询。

. SHOW

语句	语法	示例	说明
SHOW CHARSET	SHOW CHARSET	SHOW CHARSET	显示当前的字符集。
SHOW COLLATION	SHOW COLLATION	SHOW COLLATION	显示当前的排序规则。
SHOW CREATE SPACE	SHOW CREATE SPACE <space_name>	SHOW CREATE SPACE basketballplayer	显示指定图空间的创建语句。
SHOW CREATE TAG/EDGE	SHOW CREATE {TAG <tag_name> EDGE <edge_name>}	SHOW CREATE TAG player	显示指定 Tag/Edge type 的基本信息。
SHOW HOSTS	SHOW HOSTS [GRAPH STORAGE META]	SHOW HOSTS SHOW HOSTS GRAPH	显示 Graph、Storage、Meta 服务主机信息、版本信息。
SHOW INDEX STATUS	SHOW {TAG EDGE} INDEX STATUS	SHOW TAG INDEX STATUS	重建原生索引的作业状态，以便确定重建索引是否成功。
SHOW INDEXES	SHOW {TAG EDGE} INDEXES	SHOW TAG INDEXES	列出当前图空间内的所有 Tag 和 Edge type（包括属性）的索引。
SHOW PARTS	SHOW PARTS [<part_id>]	SHOW PARTS	显示图空间中指定分片或所有分片的信息。
SHOW ROLES	SHOW ROLES IN <space_name>	SHOW ROLES in basketballplayer	显示分配给用户的角色信息。
SHOW SNAPSHOTS	SHOW SNAPSHOTS	SHOW SNAPSHOTS	显示所有快照信息。
SHOW SPACES	SHOW SPACES	SHOW SPACES	显示现存的图空间。
SHOW STATS	SHOW STATS	SHOW STATS	显示最近 STATS 作业收集的图空间统计信息。
SHOW TAGS/ EDGES	SHOW TAGS EDGES	SHOW TAGS、SHOW EDGES	显示当前图空间内的所有 Tag/Edge type。
SHOW USERS	SHOW USERS	SHOW USERS	显示用户信息。
SHOW SESSIONS	SHOW SESSIONS	SHOW SESSIONS	显示所有会话信息。
SHOW SESSIONS	SHOW SESSION <Session_Id>	SHOW SESSION 1623304491050858	指定会话 ID 进行查看。
SHOW QUERIES	SHOW [ALL] QUERIES	SHOW QUERIES	查看当前 Session 中正在执行的查询请求信息。
SHOW META LEADER	SHOW META LEADER	SHOW META LEADER	显示当前 Meta 集群的 leader 信息。

3.2.3 子句和选项

子句	语法	示例	说明
GROUP BY	<code>GROUP BY <var> YIELD <var>, <aggregation_function(var)></code>	<code>GO FROM "player100" OVER follow BIDIRECT YIELD \$\$player.name as Name GROUP BY \$-.Name YIELD \$-.Name as Player, count(*) AS Name_Count</code>	查找所有连接到 player100 的点，并根据 他们的姓名进行分组， 返回姓名的出现次数。
LIMIT	<code>YIELD <var> [LIMIT [<offset_value>], <number_rows>]</code>	<code>GO FROM "player100" OVER follow REVERSELY YIELD \$\$player.name AS Friend, \$\$player.age AS Age ORDER BY \$-.Age, \$-.Friend LIMIT 1, 3</code>	从排序结果中返回第 2 行开始的 3 行数据。
SKIP	<code>RETURN <var> [SKIP <offset>] [LIMIT <number_rows>]</code>	<code>MATCH (v:player{name:"Tim Duncan"}) --> (v2) RETURN v2.player.name AS Name, v2.player.age AS Age ORDER BY Age DESC SKIP 1</code>	用户可以单独使用 SKIP <offset> 设置偏 移量，后面不需要添加 LIMIT <number_rows>。
SAMPLE	<code><go_statement> SAMPLE <sample_list>;</code>	<code>GO 3 STEPS FROM "player100" OVER * YIELD properties(\$\$).name AS NAME, properties(\$\$).age AS Age SAMPLE [1,2,3];</code>	在结果集中均匀取样并 返回指定数量的数据。
ORDER BY	<code><YIELD clause> ORDER BY <expression> [ASC DESC] [, <expression> [ASC DESC] ...]</code>	<code>FETCH PROP ON player "player100", "player101", "player102", "player103" YIELD player.age AS age, player.name AS name ORDER BY \$-.age ASC, \$-.name DESC</code>	ORDER BY 子句指定输出 结果的排序规则。
RETURN	<code>RETURN [<vertex_name> <edge_name> <vertex_name>.<property> <edge_name>.<property> ...]</code>	<code>MATCH (v:player) RETURN v.player.name, v.player.age LIMIT 3</code>	返回点的属性为 name 和 age 的前三行值。
TTL	<code>CREATE TAG <tag_name>(<property_name_1> <property_value_1>, <property_name_2> <property_value_2>, ...) ttl_duration= <value_int>, ttl_col = <property_name></code>	<code>CREATE TAG t2(a int, b int, c string) ttl_duration= 100, ttl_col = "a"</code>	创建 Tag 并设置 TTL 选 项。
WHERE	<code>WHERE {<vertex edge_alias>.<property_name> <> == <[...]> <value>...}</code>	<code>MATCH (v:player) WHERE v.player.name == "Tim Duncan" XOR (v.player.age < 30 AND v.player.name == "Yao Ming") OR NOT (v.player.name == "Yao Ming" OR v.player.name == "Tim Duncan") RETURN v.player.name, v.player.age</code>	WHERE 子句可以根据条 件过滤输出结果，通常 用于 GO 和 LOOKUP 语 句，MATCH 和 WITH 语 句。
YIELD	<code>YIELD [DISTINCT] <col> [AS <alias>] [, <col> [AS <alias>] ...] [WHERE <conditions>];</code>	<code>GO FROM "player100" OVER follow YIELD dst(edge) AS ID FETCH PROP ON player \$-.ID YIELD player.age AS Age YIELD AVG(\$-.Age) as Avg_age, count(*)as Num_friends</code>	查找 player100 关注的 player，并计算他们的 平均年龄。
WITH	<code>MATCH \$expressions WITH {nodes() labels() ...}</code>	<code>MATCH p=(v:player{name:"Tim Duncan"})--() WITH nodes(p) AS n UNWIND n AS n1 RETURN DISTINCT n1</code>	WITH 子句可以获取并处 理查询前半部分的结 果，并将处理结果作为 输入传递给查询的后半 部分。
UNWIND	<code>UNWIND <list> AS <alias> <RETURN clause></code>	<code>UNWIND [1,2,3] AS n RETURN n</code>	拆分列表。

3.2.4 图空间语句

语句	语法	示例	说明
CREATE SPACE	<pre>CREATE SPACE [IF NOT EXISTS] <graph_space_name> ([partition_num = <partition_number>], [replica_factor = <replica_number>], vid_type = {FIXED_STRING(<N>) INT[64]}) [COMMENT = '<comment>']</pre>	<pre>CREATE SPACE my_space_1 (vid_type=FIXED_STRING(30))</pre>	创建一个新的图空间。
CREATE SPACE	<pre>CREATE SPACE <new_graph_space_name> AS <old_graph_space_name></pre>	<pre>CREATE SPACE my_space_4 as my_space_3</pre>	克隆现有图空间的 Schema。
USE	<pre>USE <graph_space_name></pre>	<pre>USE space1</pre>	指定一个图空间，或切换到另一个图空间，将其作为后续查询的工作空间。
SHOW SPACES	<pre>SHOW SPACES</pre>	<pre>SHOW SPACES</pre>	列出 TRSGraph 示例中的所有图空间。
DESCRIBE SPACE	<pre>DESC[RIBE] SPACE <graph_space_name></pre>	<pre>DESCRIBE SPACE basketballplayer</pre>	显示指定图空间的信息。
CLEAR SPACE	<pre>CLEAR SPACE [IF EXISTS] <graph_space_name></pre>	清空图空间中的点和边，但不会删除图空间本身以及其中的 Schema 信息。	
DROP SPACE	<pre>DROP SPACE [IF EXISTS] <graph_space_name></pre>	<pre>DROP SPACE basketballplayer</pre>	删除指定图空间的所有内容。

3.2.5 TAG 语句

语句	语法	示例	说明
CREATE TAG	<pre>CREATE TAG [IF NOT EXISTS] <tag_name> (<prop_name> <data_type> [NULL NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>'] [{, <prop_name> <data_type> [NULL NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']} ...]) [TTL_DURATION = <ttl_duration>] [TTL_COL = <prop_name>] [COMMENT = '<comment>']</pre>	<pre>CREATE TAG woman(name string, age int, married bool, salary double, create_time timestamp) TTL_DURATION = 100, TTL_COL = "create_time"</pre>	通过指定名称创建一个 Tag。
DROP TAG	<pre>DROP TAG [IF EXISTS] <tag_name></pre>	<pre>DROP TAG test;</pre>	删除当前工作空间内所有点上的指定 Tag。
ALTER TAG	<pre>ALTER TAG <tag_name> <alter_definition> [, alter_definition] ...] [ttl_definition [, ttl_definition] ...] [COMMENT = '<comment>']</pre>	<pre>ALTER TAG t1 ADD (p3 int, p4 string)</pre>	修改 Tag 的结构。例如增删属性、修改数据类型，也可以为属性设置、修改 TTL (Time-To-Live)。
SHOW TAGS	<pre>SHOW TAGS</pre>	<pre>SHOW TAGS</pre>	显示当前图空间内的所有 Tag 名称。
DESCRIBE TAG	<pre>DESC[RIBE] TAG <tag_name></pre>	<pre>DESCRIBE TAG player</pre>	查看指定 Tag 的详细信息，例如字段名称、数据类型等。
DELETE TAG	<pre>DELETE TAG <tag_name_list> FROM <VID></pre>	<pre>DELETE TAG test1 FROM "test"</pre>	删除指定点上的指定 Tag。

3.2.6 Edge type 语句

语句	语法	示例	说明
CREATE EDGE	<pre>CREATE EDGE [IF NOT EXISTS] <edge_type_name> (<prop_name> <data_type> [NULL NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>'] [{, <prop_name> <data_type> [NULL NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']} ...]) [TTL_DURATION = <ttl_duration>] [TTL_COL = <prop_name>] [COMMENT = '<comment>']</pre>	<pre>CREATE EDGE e1(p1 string, p2 int, p3 timestamp) TTL_DURATION = 100, TTL_COL = "p2"</pre>	指定名称创建一个 Edge type。
DROP EDGE	<pre>DROP EDGE [IF EXISTS] <edge_type_name></pre>	<pre>DROP EDGE e1</pre>	删除当前工作空间内的指定 Edge type。
ALTER EDGE	<pre>ALTER EDGE <edge_type_name> <alter_definition> [, alter_definition] ...] [ttl_definition [, ttl_definition] ...] [COMMENT = '<comment>']</pre>	<pre>ALTER EDGE e1 ADD (p3 int, p4 string)</pre>	修改 Edge type 的结构。
SHOW EDGES	<pre>SHOW EDGES</pre>	<pre>SHOW EDGES</pre>	显示当前图空间内的所有 Edge type 名称。
DESCRIBE EDGE	<pre>DESC[RIBE] EDGE <edge_type_name></pre>	<pre>DESCRIBE EDGE follow</pre>	查看指定 Edge type 的详细信息，例如字段名称、数据类型等。

3.2.7 点语句

语句	语法	示例	说明
INSERT VERTEX	<pre>INSERT VERTEX [IF NOT EXISTS] [tag_props, [tag_props] ...] VALUES <vid>: ([prop_value_list])</pre>	<pre>INSERT VERTEX t2 (name, age) VALUES "13":("n3", 12), "14":("n4", 8)</pre>	在 TRSGraph 实例的指定图空间中插入一个或多个点。
DELETE VERTEX	<pre>DELETE VERTEX <vid> [, <vid> ...]</pre>	<pre>DELETE VERTEX "team1"</pre>	删除点，以及点关联的出边和入边。
UPDATE VERTEX	<pre>UPDATE VERTEX ON <tag_name> <vid> SET <update_prop> [WHEN <condition>] [YIELD <output>]</pre>	<pre>UPDATE VERTEX ON player "player101" SET age = age + 2</pre>	修改点上 Tag 的属性值。
UPSERT VERTEX	<pre>UPSERT VERTEX ON <tag> <vid> SET <update_prop> [WHEN <condition>] [YIELD <output>]</pre>	<pre>UPSERT VERTEX ON player "player667" SET age = 31</pre>	结合 UPDATE 和 INSERT，如果点存在，会修改点的属性值；如果点不存在，会插入新的点。

3.2.8 边语句

语句	语法	示例	说明
INSERT EDGE	<pre>INSERT EDGE [IF NOT EXISTS] <edge_type> (<prop_name_list>) VALUES <src_vid> -> <dst_vid>[@<rank>] : (<prop_value_list>) [, <src_vid> -> <dst_vid>[@<rank>] : (<prop_value_list>), ...]</pre>	<pre>INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", 1)</pre>	在 TRSGraph 实例的指定图空间中插入一条或多条边。
DELETE EDGE	<pre>DELETE EDGE <edge_type> <src_vid> -> <dst_vid>[@<rank>] [, <src_vid> -> <dst_vid>[@<rank>] ...]</pre>	<pre>DELETE EDGE serve "player100" -> "team204"@0</pre>	删除边。一次可以删除一条或多条边。
UPDATE EDGE	<pre>UPDATE EDGE ON <edge_type> <src_vid> -> <dst_vid> [@<rank>] SET <update_prop> [WHEN <condition>] [YIELD <output>]</pre>	<pre>UPDATE EDGE ON serve "player100" -> "team204"@0 SET start_year = start_year + 1</pre>	修改边上 Edge type 的属性。
UPSERT EDGE	<pre>UPSERT EDGE ON <edge_type> <src_vid> -> <dst_vid> [@rank] SET <update_prop> [WHEN <condition>] [YIELD <properties>]</pre>	<pre>UPSERT EDGE on serve "player666" -> "team200"@0 SET end_year = 2021</pre>	结合 UPDATE 和 INSERT，如果边存在，会更新边的属性；如果边不存在，会插入新的边。

3.2.9 索引

• 原生索引

索引配合 `LOOKUP` 和 `MATCH` 语句使用。

语句	语法	示例	说明
CREATE INDEX	<pre>CREATE {TAG EDGE} INDEX [IF NOT EXISTS] <index_name> ON {<tag_name> <edge_name>} {([<prop_name_list>]) [COMMENT = '<comment>']}</pre>	<pre>CREATE TAG INDEX player_index on player()</pre>	对 Tag、EdgeType 或其属性创建原生索引。
SHOW CREATE INDEX	<pre>SHOW CREATE {TAG EDGE} INDEX <index_name></pre>	<pre>show create tag index index_2</pre>	创建 Tag 或者 Edge type 时使用的 nGQL 语句，其中包含索引的详细信息，例如其关联的属性。
SHOW INDEXES	<pre>SHOW {TAG EDGE} INDEXES</pre>	<pre>SHOW TAG INDEXES</pre>	列出当前图空间内的所有 Tag 和 Edge type（包括属性）的索引。
DESCRIBE INDEX	<pre>DESCRIBE {TAG EDGE} INDEX <index_name></pre>	<pre>DESCRIBE TAG INDEX player_index_0</pre>	查看指定索引的信息，包括索引的属性名称（Field）和数据类型（Type）。
REBUILD INDEX	<pre>REBUILD {TAG EDGE} INDEX [<index_name_list>]</pre>	<pre>REBUILD TAG INDEX single_person_index</pre>	重建索引。索引功能不会自动对其创建之前已存在的存量数据生效。
SHOW INDEX STATUS	<pre>SHOW {TAG EDGE} INDEX STATUS</pre>	<pre>SHOW TAG INDEX STATUS</pre>	查看索引名称和对应的状态。
DROP INDEX	<pre>DROP {TAG EDGE} INDEX [IF EXISTS] <index_name></pre>	<pre>DROP TAG INDEX player_index_0</pre>	删除当前图空间中已存在的索引。

全文索引

语法	示例	说明
<pre>SIGN IN TEXT SERVICE [(<elastic_ip:port> [,<username>, <password>]), (<elastic_ip:port>), ...]</pre>	<pre>SIGN IN TEXT SERVICE (127.0.0.1:9200)</pre>	TRSGraph 的全文索引是基于 Elasticsearch 实现，部署 Elasticsearch 集群之后，可以使用 <code>SIGN IN</code> 语句登录 Elasticsearch 客户端。
<pre>SHOW TEXT SEARCH CLIENTS</pre>	<pre>SHOW TEXT SEARCH CLIENTS</pre>	列出文本搜索客户端。
<pre>SIGN OUT TEXT SERVICE</pre>	<pre>SIGN OUT TEXT SERVICE</pre>	退出所有文本搜索客户端。
<pre>CREATE FULLTEXT {TAG EDGE} INDEX index_nametag_nameedge_nameprop_nameprop_nameanalyzer_name"]</pre>	<pre>CREATE FULLTEXT TAG INDEX trsgraph_index_1 ON player(name)</pre>	创建全文索引。
<pre>SHOW FULLTEXT INDEXES</pre>	<pre>SHOW FULLTEXT INDEXES</pre>	显示全文索引。
<pre>REBUILD FULLTEXT INDEX</pre>	<pre>REBUILD FULLTEXT INDEX</pre>	重建全文索引。
<pre>DROP FULLTEXT INDEX index_name</pre>	<pre>DROP FULLTEXT INDEX trsgraph_index_1</pre>	删除全文索引。
<pre>LOOKUP ON {tagedge_typeindex_nametextreturn_listoffset], number_rows</pre>	<pre>LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"Chris") YIELD id(vertex)</pre>	使用查询选项。

3.2.10 子图和路径

类型	语法	示例	说明
子图	<pre>GET SUBGRAPH [WITH PROP] [step_count] {STEP STEPS}] FROM {vidvidedge_typeedge_typevertex_alias] [,EDGES AS edge_alias]</pre>	<pre>GET SUBGRAPH 1 STEPS FROM "player100" YIELD VERTICES AS nodes, EDGES AS relationships</pre>	指定 Edge type 的起始点可以到达的点和边的信息，返回子图信息。
路径	<pre>FIND { SHORTEST ALL NOLOOP } PATH [WITH PROP] FROM vertex_id_listvertex_id_listedge_type_listWHERE clauseNaliasM</pre>	<pre>FIND SHORTEST PATH FROM "player102" TO "team204" OVER * YIELD path as p</pre>	查找指定起始点和目的点之间的路径。返回的路径格式类似于 (vertex_idedge_type_name>@rank>]->(vertex_id)。

3.2.11 查询调优

类型	语法	示例	说明
EXPLAIN	<pre>EXPLAIN [format="row" "dot"] <your_nGQL_statement></pre>	<pre>EXPLAIN format="row" SHOW TAGS EXPLAIN format="dot" SHOW TAGS</pre>	输出 nGQL 语句的执行计划，但不会执行 nGQL 语句。
PROFILE	<pre>PROFILE [format="row" "dot"] <your_nGQL_statement></pre>	<pre>PROFILE format="row" SHOW TAGS PROFILE format="dot" SHOW TAGS</pre>	执行 nGQL 语句，然后输出执行计划和执行概要。

3.2.12 运维

• SUBMIT JOB BALANCE

语法	说明
<code>SUBMIT JOB BALANCE LEADER</code>	启动任务均衡分布所有图空间中的 leader。该命令会返回任务 ID。

• 作业管理

语法	说明
<code>SUBMIT JOB</code> <code>COMPACT</code>	触发 RocksDB 的长耗时 <code>compact</code> 操作。
<code>SUBMIT JOB FLUSH</code>	将内存中的 RocksDB memfile 写入硬盘。
<code>SUBMIT JOB STATS</code>	启动一个作业，该作业对当前图空间进行统计。作业完成后，用户可以使用 <code>SHOW STATS</code> 语句列出统计结果。
<code>SHOW JOB <job_id></code>	显示当前图空间内指定作业和相关任务的信息。Meta 服务将 <code>SUBMIT JOB</code> 请求解析为多个任务，然后分配给进程 <code>trsgraph-storaged</code> 。
<code>SHOW JOBS</code>	列出当前图空间内所有未过期的作业。
<code>STOP JOB</code>	停止当前图空间内未完成的作业。
<code>RECOVER JOB</code>	重新执行当前图空间内失败的作业，并返回已恢复的作业数量。

• 终止查询

语法	示例	说明
<code>KILL QUERY (session=<session_id>, plan=<plan_id>)</code>	<code>KILL</code> <code>QUERY (SESSION=1625553545984255, PLAN=163)</code>	在一个会话中执行命令终止另一个会话中的查询。

• 终止会话

语法	示例	说明
<code>KILL {SESSION SESSIONS} <SessionId></code>	<code>KILL SESSION 1672887983842984</code>	终止单个会话。
<code>SHOW SESSIONS YIELD \$-.SessionId AS sid [WHERE <filter_clause>] KILL {SESSION SESSIONS} \$-.sid</code>	<code>SHOW SESSIONS YIELD \$-.SessionId AS sid, \$-.CreateTime as CreateTime ORDER BY \$-.CreateTime ASC LIMIT 2 KILL SESSIONS \$-.sid</code>	终止多个会话。
<code>SHOW SESSIONS KILL SESSIONS \$-.SessionId</code>	<code>SHOW SESSIONS KILL SESSIONS \$-.SessionId</code>	终止所有会话。

最后更新: 2025年12月19日

4. nGQL 指南

4.1 nGQL 概述

4.1.1 什么是 nGQL

nGQL是 TRSGraph 使用的的声明式图查询语言，支持灵活高效的图模式，而且 nGQL 是为开发和运维人员设计的类 SQL 查询语言，易于学习。

nGQL 可以做什么

- 支持图遍历
- 支持模式匹配
- 支持聚合
- 支持修改图
- 支持访问控制
- 支持聚合查询
- 支持索引
- 支持大部分 openCypher 9 图查询语法（不支持修改和控制语法）

占位标识符和占位符值

TRSGraph 查询语言 nGQL 参照以下标准设计：

- (Draft) ISO/IEC JTC1 N14279 SC 32 - Database_Languages - GQL
- (Draft) ISO/IEC JTC1 SC32 N3228 - SQL_Property_Graph_Queries - SQLPGQ
- OpenCypher 9

在模板代码中，任何非关键字、字面值或标点符号的标记都是占位符标识符或占位符值。

本文中 nGQL 语法符号的说明如下。

符号	含义
<>	语法元素的名称。
:	定义元素的公式。
[]	可选元素。
{}	显式的指定元素。
	所有可选的元素。
...	可以重复多次。

例如插入一个点的 nGQL 语法：

```
INSERT VERTEX [IF NOT EXISTS] [tag_props, [tag_props] ...]
VALUES <vid>: ([prop_value_list])

tag_props:
  tag_name ([prop_name_list])

prop_name_list:
```

```
[prop_name [, prop_name] ...]

prop_value_list:
[prop_value [, prop_value] ...]
```

示例语句：

```
trsgraph> CREATE TAG IF NOT EXISTS player(name string, age int);
trsgraph> INSERT VERTEX IF NOT EXISTS player(name,age) VALUES "player100":("Tim Duncan", 42);
```

关于 openCypher 兼容性

原生 NGQL 和 OPENCYPHER 的关系

原生 nGQL 是由 TRSGraph 自行创造和实现的图查询语言。openCypher 是由 openCypher Implementers Group 组织所开源和维护的图查询语言，最新版本为 openCypher 9。

由于 nGQL 语言部分兼容了 openCypher，这个部分在本文中称为 openCypher 兼容语句。

Note

nGQL 语言 = 原生 nGQL 语句 + openCypher 兼容语句

NGQL 完全兼容 OPENCYPHER 9 吗？

不。

OpenCypher 兼容性

nGQL 设计目标仅为兼容部分的 DQL 语句(match, optional match, with等)。

不计划兼容任何 DDL，DML，DCL；

不计划兼容 Bolt 协议；

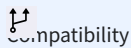
不计划兼容 APOC 与 GDS。

在本文搜索 "compatibility" 或者 “兼容性” 查看具体不兼容的细节。

在 [Issues](#) 中已经列出已知的兼容错误。如果发现这种类型的新问题，请提交问题并附带 incompatible 标签。

NGQL 和 OPENCYPHER 9 的主要差异有哪些？

类别	openCypher 9	nGQL
Schema	弱 Schema	强 Schema
相等运算符	=	==
数学求幂	^	使用 pow(x, y) 替代 ^。
边 Rank	无此概念	用 @rank 设置。
语句	-	不支持 openCypher 9 的所有 DML 语句（如 CREATE、MERGE 等），不支持所有的 DCL，和支持部分 MATCH、OPTIONAL MATCH 语法和函数。
语句文本换行	换行符	\ + 换行符
Label 与 Tag 是不同的概念	Label 用于寻找点（点的索引）。	Tag 用于定义点的一种类型及相应的属性，无索引功能。
预编译与参数化查询	支持	仅支持参数化查询。



- 请注意 openCypher 9 和 Cypher 在语法和许可上有不同：
- 1. Cypher 要求所有 Cypher 语句必须“显式地在一个事务中”执行，而 openCypher 没有这样的要求。另外，nGQL 没有事务及隔离性。
 - 2. Cypher 企业版功能有多种的约束（constraints），包括 Unique node property constraints、Node property existence constraints、Relationship property existence constraints、Node key constraints。OpenCypher 标准中没有约束。而 nGQL 是强 Schema 系统，前述的约束大多通过 Schema 定义可实现（包括 NOT NULL），唯一不能支持的功能是“属性值唯一性”（UNIQUE constraint）。
 - 3. Cypher 有 APoC，openCypher 9 没有 APoC。Cypher 有 Bolt 协议支持要求，openCypher 9 没有。

是否支持 TINKERPOP GREMLIN？

不支持。也没有计划。

是否支持 W3C 的 RDF（SPARQL）或 GRAPHQL 等？

不支持。也没有计划。

TRSGraph 的数据模型是属性图，是一个强 Schema 系统，不支持 RDF 标准。

nGQL 也不支持 SPARQL 和 GraphQL。

最后更新: 2025年12月19日

4.1.2 模式

模式（pattern）和图模式匹配，是图查询语言的核心功能，本文介绍 TRSGraph 设计的各种模式，部分还未实现。

单点模式

点用一对括号来描述，通常包含一个名称。例如：

```
(a)
```

示例为一个简单的模式，描述了单个点，并使用变量 `a` 命名该点。

多点关联模式

多个点通过边相连是常见的结构，模式用箭头来描述两个点之间的边。例如：

```
(a)-[]->(b)
```

示例为一个简单的数据结构：两个点和一条连接两个点的边，两个点分别为 `a` 和 `b`，边是有方向的，从 `a` 到 `b`。

这种描述点和边的方式可以扩展到任意数量的点和边，例如：

```
(a)-[]->(b)<-[]-(c)
```

这样的一系列点和边称为 **路径**（path）。

只有在涉及某个点时，才需要命名这个点。如果不涉及这个点，则可以省略名称，例如：

```
(a)-[]->()<-[]-(c)
```

Tag 模式



nGQL 中的 **Tag** 概念与 openCypher 中的 **Label** 有一些不同。例如，必须创建一个 **Tag** 之后才能使用它，而且 **Tag** 还定义了属性的类型。

模式除了简单地描述图中的点之外，还可以描述点的 **Tag**。例如：

```
(a:User)-[]->(b)
```

模式也可以描述有多个 **Tag** 的点，例如：

```
(a:User:Admin)-[]->(b)
```

属性模式

点和边是图的基本结构。nGQL 在这两种结构上都可以增加属性，方便实现更丰富的模型。

在模式中，属性的表示方式为：用花括号括起一些键值对，用英文逗号分隔，并且需要指定属性所属的 **Tag** 或者 **Edge type**。

例如一个点有两个属性：

```
(a:player{name: "Tim Duncan", age: 42})
```

在这个点上可以有一条边是：

```
(a)-[e:follow{degree: 95}]->(b)
```


边模式

描述一条边最简单的方法是使用箭头连接两个点。

可以用以下方式描述边以及它的方向性。如果不关心边的方向，可以省略箭头，例如：

```
(a) - [] - (b)
```

和点一样，边也可以命名。一对方括号用于分隔箭头，变量放在两者之间。例如：

```
(a) - [r] -> (b)
```

和点上的 Tag 一样，边也可以有类型。描述边的类型，例如：

```
(a) - [r:REL_TYPE] -> (b)
```

和点上的 Tag 不同，一条边只能有一种 Edge type。但是如果我们想描述多个可选 Edge type，可以用管道符号 (|) 将可选值分开，例如：

```
(a) - [r:TYPE1|TYPE2] -> (b)
```

和点一样，边的名称可以省略，例如：

```
(a) - [:REL_TYPE] -> (b)
```

变长模式

在图中指定边的长度来描述多条边（以及中间的点）组成的一条长路径，不需要使用多个点和边来描述。例如：

```
(a) - [*2] -> (b)
```

该模式描述了 3 点 2 边组成的图，它们都在一条路径上（长度为 2），等价于：

```
(a) - [] -> () - [] -> (b)
```

也可以指定长度范围，这样的边模式称为 `variable-length edges`，例如：

```
(a) - [*3..5] -> (b)
```

`*3..5` 表示最小长度为 3，最大长度为 5。

该模式描述了 4 点 3 边、5 点 4 边或 6 点 5 边组成的图。

也可以指定长度范围的上限或下限，也可以两个都不指定，例如：

```
(a) - [*..5] -> (b) // 最小长度为 1，最大长度为 5。
(a) - [*3..] -> (b) // 最小长度为 3，最大长度为无穷大。
(a) - [*] -> (b)    // 最小长度为 1，最大长度为无穷大。
```

路径变量

一系列连接的点和边称为 `路径`。nGQL 允许使用变量来命名路径，例如：

```
p = (a) - [*3..5] -> (b) // 从 a 到 b，最小长度为 3，最大长度为 5 的路径
```

可以在 MATCH 语句中使用路径变量。

最后更新: 2025年12月19日

4.1.3 注释

本文介绍 nGQL 中的注释方式。

Examples

```
trsgraph> RETURN 1+1;      # 这条注释延续到行尾。
trsgraph> RETURN 1+1;      // 这条注释延续到行尾。
trsgraph> RETURN 1 /* 这是一条行内注释 */ + 1 == 2;
trsgraph> RETURN 11 +      \
/* 多行注释                \
用反斜线来换行。          \
*/ 12;
```



- nGQL 语句中的反斜线 (\) 代表换行。
- 如果 nGQL 语句以 # 或者 // 开头，不会执行该行命令并且会返回提示 `StatementEmpty`。

openCypher 兼容性

- 在 nGQL 中，用户必须在行末使用反斜线 (\) 来换行，即使是在使用 /* */ 符号的多行注释内。
- 在 openCypher 中不需要使用反斜线换行。

```
/* openCypher 风格：
这条注释
延续了不止
一行 */
MATCH (n:label)
RETURN n;
```

```
/* 原生 nGQL 风格： \
这条注释          \
延续了不止        \
一行 */           \
MATCH (n:tag) \
RETURN n;
```

最后更新: 2025年12月19日

4.1.4 大小写区分

标识符区分大小写

以下语句会出现错误，因为 `my_space` 和 `MY_SPACE` 是两个不同的图空间。

```
trsgraph> CREATE SPACE IF NOT EXISTS my_space (vid_type=FIXED_STRING(30));
trsgraph> use MY_SPACE;
[ERROR (-1005)]: SpaceNotFound;
```

关键字不区分大小写

以下语句是等价的，因为 `show` 和 `spaces` 是关键字。

```
trsgraph> show spaces;
trsgraph> SHOW SPACES;
trsgraph> SHOW spaces;
trsgraph> show SPACES;
```

函数不区分大小写

函数名称不区分大小写，例如 `count()`、`COUNT()`、`couNT()` 是等价的。

```
trsgraph> WITH [NULL, 1, 1, 2, 2] As a \
          UNWIND a AS b \
          RETURN count(b), COUNT(*), couNT(DISTINCT b);
+-----+-----+-----+
| count(b) | COUNT(*) | couNT(distinct b) |
+-----+-----+-----+
| 4        | 5        | 2                  |
+-----+-----+-----+
```

最后更新: 2025年12月19日

4.1.5 关键字

关键字是 nGQL 中具有特殊含义的词，例如 `CREATE TAG` 语句中的 `CREATE` 和 `TAG`。需要经过特殊处理才能作为标识符使用的关键字被称为保留关键字，而能够直接作为标识符使用的这部分关键字被称为非保留关键字。

不建议在创建 Schema 时使用关键字。如果必须使用关键字，请注意一下规则：

- 使用保留关键字或特殊字符作为标识符时，必须用反引号（```）包围，例如 ``AND``。否则，会返回语义错误，创建失败。
- 使用非保留关键字作为标识符时：
 - 若其包含大写字母，必须用反引号（```）包围，例如 ``Comment``。否则，虽然能创建成功，系统会自动将标识符转化为全小写字母，也即 ``comment``。
 - 若其为全小写字母，可以不使用反引号（```）包围。

```
trsgaph> CREATE TAG TAG(name string);
[ERROR (-1004)]: SyntaxError: syntax error near `TAG'

trsgaph> CREATE TAG `TAG` (name string);
Execution succeeded

trsgaph> CREATE TAG SPACE(name string);
Execution succeeded

trsgaph> CREATE TAG 中文(简体 string);
Execution succeeded

trsgaph> CREATE TAG `¥%特殊 字符&*+~*/` (`q~! () = wer` string);
Execution succeeded
```

保留关键字

```
ACROSS
ADD
ALTER
AND
AS
ASC
ASCENDING
BALANCE
BOOL
BY
CASE
CHANGE
COMPACT
CREATE
DATE
DATETIME
DELETE
DESC
DESCENDING
DESCRIBE
DISTINCT
DOUBLE
DOWNLOAD
DROP
DURATION
EDGE
EDGES
EXISTS
EXPLAIN
FALSE
FETCH
FIND
FIXED_STRING
FLOAT
FLUSH
FROM
GEOGRAPHY
GET
GO
GRANT
IF
IGNORE_EXISTED_INDEX
IN
INDEX
INDEXES
INGEST
INSERT
INT
```

```
INT16
INT32
INT64
INT8
INTERSECT
IS
JOIN
LEFT
LIST
LOOKUP
MAP
MATCH
MINUS
NO
NOT
NULL
OF
ON
OR
ORDER
OVER
OVERWRITE
PATH
PROP
REBUILD
RECOVER
REMOVE
RESTART
RETURN
REVERSELY
REVOKE
SET
SHOW
STEP
STEPS
STOP
STRING
SUBMIT
TAG
TAGS
TIME
TIMESTAMP
TO
TRUE
UNION
UNWIND
UPDATE
UPSERT
UPTO
USE
VERTEX
VERTICES
WHEN
WHERE
WITH
XOR
YIELD
```

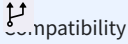
非保留关键字

```
ACCOUNT
ADMIN
AGENT
ALL
ALLSHORTESTPATHS
ANALYZER
ANY
ATOMIC_EDGE
AUTO
BASIC
BIDIRECT
BOTH
CHARSET
CLEAR
CLIENTS
COLLATE
COLLATION
COMMENT
CONFIGS
CONTAINS
DATA
DBA
DEFAULT
DIVIDE
DRAINER
DRAINERS
ELASTICSEARCH
ELSE
END
ENDS
ES_QUERY
```

FORCE
FORMAT
FULLTEXT
GOD
GRANTS
GRAPH
GROUP
GROUPS
GUEST
HDFS
HOST
HOSTS
HTTP
HTTPS
INTO
IP
JOB
JOBS
KILL
LEADER
LIMIT
LINESTRING
LISTENER
LOCAL
MERGE
META
NEW
NOLOOP
NONE
OFFSET
OPTIONAL
OUT
PART
PARTITION_NUM
PARTS
PASSWORD
PLAN
POINT
POLYGON
PROFILE
QUERIES
QUERY
READ
REDUCE
RENAME
REPLICA_FACTOR
RESET
ROLE
ROLES
S2_MAX_CELLS
S2_MAX_LEVEL
SAMPLE
SEARCH
SERVICE
SESSION
SESSIONS
SHORTEST
SHORTESTPATH
SIGN
SINGLE
SKIP
SNAPSHOT
SNAPSHOTS
SPACE
SPACES
STARTS
STATS
STATUS
STORAGE
SUBGRAPH
SYNC
TEXT
TEXT_SEARCH
THEN
TOP
TTL_COL
TTL_DURATION
USER
USERS
UUID
VALUE
VALUES
VARIABLES
VID_TYPE
WHITELIST
WRITE
ZONE
ZONES

4.1.6 nGQL 风格指南

nGQL 没有严格的构建格式要求，但根据恰当而统一的风格创建 nGQL 语句有利于提高可读性、避免歧义。在同一组织或项目中使用相同的 nGQL 风格有利于降低维护成本，规避因格式混乱或误解造成的问题。本文为写作 nGQL 语句提供了风格参考。



nGQL 风格与 [Cypher Style Guide](#) 不同。

换行

1. 换行写子句。

不推荐：

```
GO FROM "player100" OVER follow REVERSELY YIELD src(edge) AS id;
```

推荐：

```
GO FROM "player100" \
OVER follow REVERSELY \
YIELD src(edge) AS id;
```

2. 换行写复合语句中的不同语句。

不推荐：

```
GO FROM "player100" OVER follow REVERSELY YIELD src(edge) AS id | GO FROM $-.id \
OVER serve WHERE properties($^).age > 20 YIELD properties($^).name AS FriendOf, properties($$).name AS Team;
```

推荐：

```
GO FROM "player100" \
OVER follow REVERSELY \
YIELD src(edge) AS id | \
GO FROM $-.id OVER serve \
WHERE properties($^).age > 20 \
YIELD properties($^).name AS FriendOf, properties($$).name AS Team;
```

3. 子句长度超过 80 个字符时，在合适的位置换行。

不推荐：

```
MATCH (v:player{name:"Tim Duncan"})-[e]->(v2) \
WHERE (v2.player.name STARTS WITH "Y" AND v2.player.age > 35 AND v2.player.age < v.player.age) OR (v2.player.name STARTS WITH "T" AND v2.player.age < 45 AND v2.player.age > v.player.age) \
RETURN v2;
```

推荐：

```
MATCH (v:player{name:"Tim Duncan"})-[e]->(v2) \
WHERE (v2.player.name STARTS WITH "Y" AND v2.player.age > 35 AND v2.player.age < v.player.age) \
OR (v2.player.name STARTS WITH "T" AND v2.player.age < 45 AND v2.player.age > v.player.age) \
RETURN v2;
```



即使子句不超过 80 个字符，如需换行后有助于理解，也可将子句再次分行。

标识符命名

在 nGQL 语句中，关键字、标点符号、空格以外的字符内容都是标识符。推荐的标识符命名方式如下。

1. 使用单数名词命名 Tag，用原型动词或动词短语构成 Edge type。

不推荐：

```
MATCH p=(v:players)-[e:are_following]-(v2) \
RETURN nodes(p);
```

推荐：

```
MATCH p=(v:player)-[e:follow]-(v2) \
RETURN nodes(p);
```

2. 标识符用蛇形命名法，以下划线（_）连接单词，且所有字母小写。

不推荐：

```
MATCH (v:basketballTeam) \
RETURN v;
```

推荐：

```
MATCH (v:basketball_team) \
RETURN v;
```

3. 语法关键词大写，变量小写。

不推荐：

```
match (V:player) return V limit 5;
```

推荐：

```
MATCH (v:player) RETURN v LIMIT 5;
```

Pattern

1. 分行写 Pattern 时，在表示边的箭头右侧换行，而不是左侧。

不推荐：

```
MATCH (v:player{name: "Tim Duncan", age: 42}) \
-[e:follow]->()-[e2:serve]->()<--(v2) \
RETURN v, e, v2;
```

推荐：

```
MATCH (v:player{name: "Tim Duncan", age: 42})-[e:follow]-> \
()-[e2:serve]->()<--(v2) \
RETURN v, e, v2;
```

2. 将无需查询的点和边匿名化。

不推荐：

```
MATCH (v:player)-[e:follow]->(v2) \
RETURN v;
```

推荐：

```
MATCH (v:player)-[:follow]->() \
RETURN v;
```

3. 将非匿名点放在匿名点的前面。

不推荐：


```
MATCH ()-[:follow]->(v) \
RETURN v;
```

推荐：

```
MATCH (v)<-[:follow]-( ) \
RETURN v;
```

字符串

字符串用双引号包围。

不推荐：

```
RETURN 'Hello TRSGraph!';
```

推荐：

```
RETURN "Hello TRSGraph!\\"123\\"";
```



字符串中需要嵌套单引号或双引号时，用反斜线 (\) 转义。例如：

```
RETURN "\"The database is amazing,\" the user says.\"";
```

结束语句

1. 用英文分号 (;) 结束 nGQL 语句。

不推荐：

```
FETCH PROP ON player "player100" YIELD properties(vertex)
```

推荐：

```
FETCH PROP ON player "player100" YIELD properties(vertex);
```

2. 使用管道符 (|) 分隔的复合语句，仅在最后一行末用英文分号结尾。在管道符前使用英文分号会导致语句执行失败。

不支持：

```
GO FROM "player100" \
OVER follow \
YIELD dst(edge) AS id; | \
GO FROM $-.id \
OVER serve \
YIELD properties($$).name AS Team, properties($^).name AS Player;
```

支持：

```
GO FROM "player100" \
OVER follow \
YIELD dst(edge) AS id | \
GO FROM $-.id \
OVER serve \
YIELD properties($$).name AS Team, properties($^).name AS Player;
```

3. 在包含自定义变量的复合语句中，用英文分号结束定义变量的语句。不按规则加分号或使用管道符结束该语句会导致执行失败。

不支持：

```
$var = GO FROM "player100" \
OVER follow \
YIELD dst(edge) AS id \
GO FROM $var.id \
OVER serve \
YIELD properties($$).name AS Team, properties($^).name AS Player;
```

也不支持：

```
$var = GO FROM "player100" \
OVER follow \
YIELD dst(edge) AS id | \
GO FROM $var.id \
OVER serve \
YIELD properties($$).name AS Team, properties($^).name AS Player;
```

支持：

```
$var = GO FROM "player100" \
OVER follow \
YIELD dst(edge) AS id; \
GO FROM $var.id \
OVER serve \
YIELD properties($$).name AS Team, properties($^).name AS Player;
```

.....

最后更新: 2025年12月19日

4.2 数据类型

4.2.1 数值

nGQL 支持整数和浮点数。

整数

nGQL 支持带符号的 64 位整数（INT64）、32 位整数（INT32）、16 位整数（INT16）和 8 位整数（INT8）。

类型	声明关键字	范围
INT64	INT64 或 INT	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
INT32	INT32	-2,147,483,648 ~ 2,147,483,647
INT16	INT16	-32,768 ~ 32,767
INT8	INT8	-128 ~ 127

浮点数

nGQL 支持单精度浮点（FLOAT）和双精度浮点（DOUBLE）。

类型	声明关键字	范围	精度
FLOAT	FLOAT	3.4E +/- 38	6~7 位
DOUBLE	DOUBLE	1.7E +/- 308	15~16 位

nGQL 支持科学计数法，例如 1e2、1.1e2、.3e4、1.e4、-1234E-10。



不支持 MySQL 中的 DECIMAL 数据类型。

数值的读写

在写入和读取不同类型的数据时，nGQL 的行为遵守以下规则：

数值类型	设置为 VID	设置为属性类型	读取该类型的属性值得到的类型
INT64	支持	支持	INT64
INT32	不支持	支持	INT64
INT16	不支持	支持	INT64
INT8	不支持	支持	INT64
FLOAT	不支持	支持	DOUBLE
DOUBLE	不支持	支持	DOUBLE

例如，nGQL 不支持设置 INT8 类型的 VID，但支持将 TAG 或 Edge type 的某个属性类型设置为 INT8。当使用 nGQL 语句读取 INT8 类型的属性时，获取到的值的类型为 INT64。

- TRSGraph 支持写入多种进制的数值：

- 十进制，例如 123456。
- 十六进制，例如 0x1e240。
- 八进制，例如 0361100。

但 TRSGraph 会将写入的非十进制数值解析为十进制的值保存。读取到的值为十进制。

例如，属性 score 的类型为 INT，通过 INSERT 语句为其赋值 0xb，使用 FETCH 等语句查询该属性值获取到的结果是 11，即将十六进制的 0xb 转换为十进制后的值。

- 将 FLOAT/DOUBLE 类型的数值插入 INT 类型的列，会将数值四舍五入取整。

最后更新: 2025年12月19日

4.2.2 布尔

TRSGraph 使用关键字 `BOOL` 声明布尔数据类型，可选值为 `true` 或 `false`。

nGQL 支持以如下方式使用布尔值：

- 将属性值的数据类型定义为布尔。
 - 在 `WHERE` 子句中用布尔值作为判断条件。
-

最后更新: 2025年12月19日

4.2.3 字符串

TRSGraph 支持定长字符串和变长字符串。

声明与表示方式

nGQL 中的字符串声明方式如下：

- 使用关键字 `STRING` 声明变长字符串。
- 使用关键字 `FIXED_STRING(<length>)` 声明定长字符串，`<length>` 为字符串长度，例如 `FIXED_STRING(32)`。

字符串的表示方式为用双引号或单引号包裹，例如 `"Hello, Cooper"` 或 `'Hello, Cooper'`。

字符串读写

nGQL 支持以如下方式使用字符串：

- 将 `VID` 的数据类型定义为定长字符串。
- 将变长字符串设置为 Schema 名称，包括图空间、Tag、Edge type 和属性的名称。
- 将属性值的数据类型定义为定长或变长字符串。

例如：

- 将属性值的类型定义为定长字符串

```
trsgraph> CREATE TAG IF NOT EXISTS t1 (p1 FIXED_STRING(10));
```

- 将属性值的类型定义为变长字符串

```
trsgraph> CREATE TAG IF NOT EXISTS t2 (p2 STRING);
```

如果尝试写入的定长字符串超出长度限制：

- 当该定长字符串为属性值时，写入会成功，TRSGraph 将截断字符串，仅存入符合长度限制的部分。
- 当该定长字符串为 VID 时，写入会失败，TRSGraph 将报错。

转义字符

反斜杠（\）在字符串中为转义字符，用于表示特殊字符。

例如，如果想在字符串中包含一个双引号（"），不能直接写 `"Hello "world""`，因为这会引起语法错误。相反，需要使用反斜杠（\）来转义双引号，如 `"Hello \"world\""`。

```
trsgraph> RETURN "Hello \"world\""
+-----+
| "Hello "world" " |
+-----+
| "Hello "world" " |
+-----+
```

同样，反斜杠本身也需要转义，因为它是一个特殊字符。例如，要在字符串中包含一个反斜杠，需要写成 `"Hello \\ world"`。

```
trsgraph> RETURN "Hello \\ world"
+-----+
| "Hello \ world" |
+-----+
| "Hello \ world" |
+-----+
```

更多转义字符的示例，请参见 [Escape character examples](#)。

OpenCypher 兼容性

openCypher、Cypher 和 nQL 之间有一些细微区别，例如下面 openCypher 的示例，不能将单引号替换为双引号。

```
# File: Literals.feature
Feature: Literals

Background:
  Given any graph
  Scenario: Return a single-quoted string
    When executing query:
      """
      RETURN '' AS literal
      """
    Then the result should be, in any order:
      | literal |
      | ''      | # Note: it should return single-quotes as openCypher required.
    And no side effects
```

Cypher 的返回结果同时支持单引号和双引号，nQL 遵循 Cypher 的方式。

```
trigraph > YIELD '' AS quote1, "" AS quote2, """" AS quote3, """" AS quote4
+-----+-----+-----+-----+
| quote1 | quote2 | quote3 | quote4 |
+-----+-----+-----+-----+
| ""      | ""      | """"    | """"    |
+-----+-----+-----+-----+
```

.....

最后更新: 2025年12月19日

4.2.4 日期和时间类型

本文介绍日期和时间的类型，包括 `DATE`、`TIME`、`DATETIME`、`TIMESTAMP` 和 `DURATION`。

注意事项

- 在插入时间类型的属性值时，TRSGraph 会根据配置文件中 `timezone_name` 参数指定的时区，将该 `DATE`、`TIME`、`DATETIME` 转换成相应的世界协调时间（UTC）时间。



如需修改当前时区，请同时修改所有服务的配置文件中的 `timezone_name` 参数。

- 函数 `date()`、`time()` 和 `datetime()` 可以指定时区进行转换，例如 `datetime("2017-03-04 22:30:40.003000+08:00")` 或 `datetime("2017-03-04T22:30:40.003000[Asia/Shanghai]")`。
- 函数 `date()`、`time()`、`datetime()` 和 `timestamp()` 可以用空值获取当前的日期或时间。
- 函数 `date()`、`time()`、`datetime()` 和 `duration()` 可以用属性名称获取自身的某一个具体属性值，例如 `date().month` 获取当前月份、`time("02:59:40").minute` 获取传入时间的分钟数。
- 进行时间运算时建议使用 `duration()` 计算时刻的偏移。此外还支持 `date()` 和 `date()` 的加减、`timestamp()` 和 `timestamp()` 的加减。
- 设置时间的年份为负数时，需要使用 Map 类型数据。

openCypher 兼容性

- 支持年、月、日、时、分、秒、毫秒、微秒，不支持纳秒。
- 不支持函数 `localdatetime()`。
- 不支持大部分字符串时间格式，支持 `YYYY-MM-DDThh:mm:ss` 和 `YYYY-MM-DD hh:mm:ss`。
- 支持单个数字的字符串时间格式，例如 `time("1:1:1")`。

DATE

`DATE` 包含日期，但是不包含时间。TRSGraph 检索和显示 `DATE` 的格式为 `YYYY-MM-DD`。支持的范围是 `-32768-01-01` 到 `32767-12-31`。

`date()` 支持的属性名称包括 `year`、`month` 和 `day`。`date()` 支持输入 `YYYY`、`YYYY-MM` 或 `YYYY-MM-DD`，未输入的月份或日期默认为 `01`。

```
trsgraph> RETURN DATE({year:-123, month:12, day:3});
+-----+
| date({year:-123, month:12, day:3}) |
+-----+
| -123-12-03 |
+-----+

trsgraph> RETURN DATE("23333");
+-----+
| date("23333") |
+-----+
| 23333-01-01 |
+-----+

trsgraph> RETURN DATE("2023-12-12") - DATE("2023-12-11");
+-----+
| (date("2023-12-12")-date("2023-12-11")) |
+-----+
| 1 |
+-----+
```


TIME

`TIME` 包含时间，但是不包含日期。TRSGraph 检索和显示 `TIME` 的格式为 `hh:mm:ss.msmsmsususus`。支持的范围是 `00:00:00.000000` 到 `23:59:59.999999`。

`time()` 支持的属性名称包括 `hour`、`minute` 和 `second`。

DATETIME

`DATETIME` 包含日期和时间。TRSGraph 检索和显示 `DATETIME` 的格式为 `YYYY-MM-DDThh:mm:ss.msmsmsususus`。支持的范围是 `-32768-01-01T00:00:00.000000` 到 `32767-12-31T23:59:59.999999`。

- `datetime()` 支持的属性名称包括 `year`、`month`、`day`、`hour`、`minute` 和 `second`。
- `datetime()` 可将 `TIMESTAMP` 类型的日期值转换成 `DATETIME` 类型的日期值。`TIMESTAMP` 类型的日期值取值范围：`0~9223372036`。
- `datetime()` 支持 `int` 类型的参数，该 `int` 参数表示时间戳。

```
# 获取当前时间。
trsgraph> RETURN datetime();
+-----+
| datetime() |
+-----+
| 2022-08-29T06:37:08.933000 |
+-----+

# 获取当前时间的小时。
trsgraph> RETURN datetime().hour;
+-----+
| datetime().hour |
+-----+
| 6 |
+-----+

# 将时间戳转换成 DATETIME 类型的格式。
trsgraph> RETURN datetime(timestamp(1625469277));
+-----+
| datetime(timestamp(1625469277)) |
+-----+
| 2021-07-05T07:14:37.000000 |
+-----+

trsgraph> RETURN datetime(1625469277);
+-----+
| datetime(1625469277) |
+-----+
| 2021-07-05T07:14:37.000000 |
+-----+
```

TIMESTAMP

`TIMESTAMP` 包含日期和时间。支持的范围是 UTC 时间的 `1970-01-01T00:00:01` 到 `2262-04-11T23:47:16`。

`TIMESTAMP` 还有以下特点：

- 以时间戳形式存储和显示。例如 `1615974839`，表示 `2021-03-17T17:53:59`。
- 查询 `TIMESTAMP` 类型属性值的方式包括时间戳整数和 `timestamp()` 函数。
- 插入 `TIMESTAMP` 类型属性值的方式包括时间戳整数、`timestamp()` 函数和 `now()` 函数。
- `timestamp()` 函数支持传入空值获取当前时间戳；同时支持传入整数以标识该整数为时间戳，整数取值：`0~9223372036`。
- `timestamp()` 函数可将 `DATETIME` 类型的日期值转换成 `TIMESTAMP` 类型的日期值，且传入的 `DATETIME` 类型的日期值为 `string` 类型。
- 底层存储的数据格式为 64 位 `int`。

```
# 传入当前时间。
trsgraph> RETURN timestamp();
+-----+
| timestamp() |
+-----+
| 1625469277 |
+-----+
```

```
# 传入指定时间。
trsgraph> RETURN timestamp("2022-01-05T06:18:43");
+-----+
| timestamp("2022-01-05T06:18:43") |
+-----+
| 1641363523 |
+-----+

# 传入 datetime()。
trsgraph> RETURN timestamp(datetime("2022-08-29T07:53:10.939000"));
+-----+
| timestamp(datetime("2022-08-29T07:53:10.939000")) |
+-----+
| 1661759590 |
+-----+
```



传入 `timestamp()` 函数的时间字符串不支持包含毫秒和微秒，但是通过 `timestamp(datetime())` 传入的时间字符串支持包含毫秒和微秒。

DURATION

DURATION 是一段连续的时间，由 `years`、`months`、`days`、`hours`、`minutes`、`seconds` 六个Key自由组合成的Map类型数据表示。例如 `duration({years: 12, months: 5, days: 14, hours: 16, minutes: 12, seconds: 70})`。

DURATION 还有以下特点：

- 不支持为 **DURATION** 类型数据创建索引。
- 可以用于对指定时间进行计算。

示例

1. 创建 Tag，名称为 `date1`，包含 `DATE`、`TIME` 和 `DATETIME` 三种类型。

```
trsgraph> CREATE TAG IF NOT EXISTS date1(p1 date, p2 time, p3 datetime);
```

2. 插入点，名称为 `test1`。

```
trsgraph> INSERT VERTEX date1(p1, p2, p3) VALUES "test1":(date("2021-03-17"), time("17:53:59"), datetime("2017-03-04T22:30:40.003000[Asia/Shanghai]"));
```

3. 查询 `test1` 的属性 `p1` 是否为 2021-03-17。

```
trsgraph> MATCH (v:date1) RETURN v.date1.p1 == date("2021-03-17");
+-----+
| (v.date1.p1==date("2021-03-17")) |
+-----+
| true |
+-----+
```

4. 获取 `test1` 的属性 `p1` 的月份。

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS date1_index ON date1(p1);
trsgraph> REBUILD TAG INDEX date1_index;
trsgraph> MATCH (v:date1) RETURN v.date1.p1.month;
+-----+
| v.date1.p1.month |
+-----+
| 3 |
+-----+
```

5. 查找 Tag `date1` 中属性 `p3` 小于 2023-01-01T00:00:00.000000 的值。

```
trsgraph> MATCH (v:date1) \
WHERE v.date1.p3 < datetime("2023-01-01T00:00:00.000000") \
RETURN v.date1.p3;
+-----+
| v.date1.p3 |
+-----+
| 2017-03-04T14:30:40.003000 |
+-----+
```

6. 创建 Tag，名称为 `school`，包含 `TIMESTAMP` 类型。

```
trsgraph> CREATE TAG IF NOT EXISTS school(name string , found_time timestamp);
```

7. 插入点，名称为 `DUT`，存储时间为 "1988-03-01T08:00:00"。

```
# 时间戳形式插入，1988-03-01T08:00:00 对应的时间戳为 573177600，转换为 UTC 时间为 573206400。
trsgraph> INSERT VERTEX school(name, found_time) VALUES "DUT":("DUT", 573206400);

# 日期和时间格式插入。
trsgraph> INSERT VERTEX school(name, found_time) VALUES "DUT":("DUT", timestamp("1988-03-01T08:00:00"));
```

8. 插入点，名称为 `dut`，用 `now()` 或 `timestamp()` 函数存储时间。

```
# 用 now() 函数存储时间
trsgraph> INSERT VERTEX school(name, found_time) VALUES "dut":("dut", now());

# 用 timestamp() 函数存储时间
trsgraph> INSERT VERTEX school(name, found_time) VALUES "dut":("dut", timestamp());
```

还可以使用 `WITH` 语句设置具体日期时间或进行计算，例如：

```
trsgraph> WITH time({hour: 12, minute: 31, second: 14, millisecond:111, microsecond: 222}) AS d RETURN d;
+-----+
| d      |
+-----+
| 12:31:14.111222 |
+-----+

trsgraph> WITH date({year: 1984, month: 10, day: 11}) AS x RETURN x + 1;
+-----+
| (x+1)  |
+-----+
| 1984-10-12 |
+-----+

trsgraph> WITH date('1984-10-11') as x, duration({years: 12, days: 14, hours: 99, minutes: 12}) as d \
RETURN x + d AS sum, x - d AS diff;
+-----+-----+
| sum      | diff      |
+-----+-----+
| 1996-10-29 | 1972-09-23 |
+-----+-----+
```

最后更新: 2025年12月19日

4.2.5 NULL

默认情况下，插入点或边时，属性值可以为 `NULL`，用户也可以设置属性值不允许为 `NULL`（`NOT NULL`），即插入点或边时必须设置该属性的值，除非创建属性时已经设置默认值。

NULL 的逻辑操作

`AND`、`OR`、`XOR` 和 `NOT` 的真值表如下。

a	b	a AND b	a OR b	a XOR b	NOT a
false	false	false	false	false	true
false	null	false	null	null	true
false	true	false	true	true	true
true	false	false	true	true	false
true	null	null	true	null	false
true	true	true	true	false	false
null	false	false	null	null	null
null	null	null	null	null	null
null	true	null	true	null	null

OpenCypher 兼容性

TRSGraph 中，NULL 的比较和操作与 openCypher 不同，后续也可能会有变化。

NULL 的比较

TRSGraph 中，NULL 的比较操作不兼容 openCypher。

NULL 的操作和返回

TRSGraph 中，对 NULL 的操作以及返回结果不兼容 openCypher。

示例

使用 NOT NULL

创建 Tag，名称为 `player`，指定属性 `name` 为 `NOT NULL`。

```
trsgraph> CREATE TAG IF NOT EXISTS player(name string NOT NULL, age int);
```

使用 `SHOW` 命令查看创建 Tag 语句，属性 `name` 为 `NOT NULL`，属性 `age` 为默认的 `NULL`。

```
trsgraph> SHOW CREATE TAG player;
+-----+-----+
| Tag      | Create Tag                                     |
+-----+-----+
| "student" | "CREATE TAG `player` (                         |
|           |   `name` string NOT NULL,                     |
|           |   `age` int64 NULL                            |
|           | ) ttl_duration = 0, ttl_col = ""             |
+-----+-----+
```

插入点 `Kobe`，属性 `age` 可以为 `NULL`。

```
trsgraph> INSERT VERTEX player(name, age) VALUES "Kobe":("Kobe",null);
```

使用 NOT NULL 并设置默认值

创建 Tag，名称为 `player`，指定属性 `age` 为 NOT NULL，并设置默认值 18。

```
trsgraph> CREATE TAG IF NOT EXISTS player(name string, age int NOT NULL DEFAULT 18);
```

插入点 `Kobe`，只设置属性 `name`。

```
trsgraph> INSERT VERTEX player(name) VALUES "Kobe":("Kobe");
```

查询点 `Kobe`，属性 `age` 为默认值 18。

```
trsgraph> FETCH PROP ON player "Kobe" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 18, name: "Kobe"} |
+-----+
```

最后更新: 2025年12月19日

4.2.6 列表

列表（List）是复合数据类型，一个列表是一组元素的序列，可以通过元素在序列中的位置访问列表中的元素。

列表用左方括号（[]）和右方括号（]）包裹多个元素，各个元素之间用英文逗号（,）隔开。元素前后的空格在列表中被忽略，因此可以使用换行符、制表符和空格调整格式。

OpenCypher 兼容性

复合数据类型（例如 List、Set、Map）不能存储为点或边的属性。

列表操作

对列表进行操作可以使用预设的列表函数，也可以使用下标表达式过滤列表内的元素。

下标表达式语法

```
[M]
[M..N]
[M..]
[..
```

nGQL 的下标支持从前往后查询，从 0 开始，0 表示第一个元素，1 表示第二个元素，以此类推；也支持从后往前查询，从 -1 开始，-1 表示最后一个元素，-2 表示倒数第二个元素，以此类推。

- [M]：表示下标为 M 的元素。
- [M..N]：表示 $M \leq \text{下标} < N$ 的元素。N 为 0 时，返回为空。
- [M..]：表示 $M \leq \text{下标}$ 的元素。
- [..\text{下标} < N 的元素。N 为 0 时，返回为空。



- 越界的下标返回为空，未越界的可以正常返回。
- $M \geq N$ 时，返回为空。
- 查询单个元素时，如果 M 为 null，返回报错 BAD_TYPE；范围查询时，M 或 N 为 null，返回为 null。

示例

```
# 返回列表 [1,2,3]
trsgraph> RETURN list[1, 2, 3] AS a;
+-----+
| a      |
+-----+
| [1, 2, 3] |
+-----+

# 返回列表 [1,2,3,4,5] 中位置下标为 3 的元素。列表的位置下标是从 0 开始，因此返回的元素为 4。
trsgraph> RETURN range(1,5)[3];
+-----+
| range(1,5)[3] |
+-----+
| 4              |
+-----+

# 返回列表 [1,2,3,4,5] 中位置下标为-2 的元素。列表的最后一个元素的位置下标是-1，因此-2 是指倒数第二个元素，即 4。
trsgraph> RETURN range(1,5)[-2];
+-----+
| range(1,5)[-2] |
+-----+
| 4              |
+-----+

# 返回列表 [1,2,3,4,5] 中下标位置从 0 到 3（不包括 3）的元素。
trsgraph> RETURN range(1,5)[0..3];
+-----+
| range(1,5)[0..3] |
+-----+
```

```

+-----+
| [1, 2, 3] |
+-----+

# 返回列表 [1,2,3,4,5] 中位置下标大于 2 的元素。
trsgraph> RETURN range(1,5)[3..] AS a;
+-----+
| a |
+-----+
| [4, 5] |
+-----+

# 返回列表内下标小于 3 的元素。
trsgraph> WITH list[1, 2, 3, 4, 5] AS a \
RETURN a[..3] AS r;
+-----+
| r |
+-----+
| [1, 2, 3] |
+-----+

# 筛选列表 [1,2,3,4,5] 中大于 2 的元素，将这些元素分别做运算并返回。
trsgraph> RETURN [n IN range(1,5) WHERE n > 2 | n + 10] AS a;
+-----+
| a |
+-----+
| [13, 14, 15] |
+-----+

# 返回列表内第一个至倒数第二个（包括）的元素。
trsgraph> YIELD list[1, 2, 3][0..-1] AS a;
+-----+
| a |
+-----+
| [1, 2] |
+-----+

# 返回列表内倒数第三个至倒数第一个（不包括）的元素。
trsgraph> YIELD list[1, 2, 3, 4, 5][-3..-1] AS a;
+-----+
| a |
+-----+
| [3, 4] |
+-----+

# 设置变量，返回列表内下标为 1、2 的元素。
trsgraph> $var = YIELD 1 AS f, 3 AS t; \
YIELD list[1, 2, 3][$var.f..$var.t] AS a;
+-----+
| a |
+-----+
| [2, 3] |
+-----+

# 越界的下标返回为空，未越界的可以正常返回。
trsgraph> RETURN list[1, 2, 3, 4, 5] [0..10] AS a;
+-----+
| a |
+-----+
| [1, 2, 3, 4, 5] |
+-----+

trsgraph> RETURN list[1, 2, 3] [-5..5] AS a;
+-----+
| a |
+-----+
| [1, 2, 3] |
+-----+

# [0..0] 时返回为空。
trsgraph> RETURN list[1, 2, 3, 4, 5] [0..0] AS a;
+-----+
| a |
+-----+
| [] |
+-----+

# M ≥ N 时，返回为空。
trsgraph> RETURN list[1, 2, 3, 4, 5] [3..1] AS a;
+-----+
| a |
+-----+
| [] |
+-----+

# 范围查询时，下标有 null 时，返回为 null。
trsgraph> WITH list[1,2,3] AS a \
RETURN a[0..null] as r;
+-----+
| r |
+-----+
| __NULL__ |
+-----+

```

```

# 将列表 [1,2,3,4,5] 中的元素分别做运算，然后将列表去掉表头并返回。
trsgraph> RETURN tail([n IN range(1, 5) | 2 * n - 10]) AS a;
+-----+
| a      |
+-----+
| [-6, -4, -2, 0] |
+-----+

# 将列表 [1,2,3] 中的元素判断为真，然后返回。
trsgraph> RETURN [n IN range(1, 3) WHERE true | n] AS r;
+-----+
| r      |
+-----+
| [1, 2, 3] |
+-----+

# 返回列表 [1,2,3] 的长度。
trsgraph> RETURN size(list[1,2,3]);
+-----+
| size([1,2,3]) |
+-----+
| 3             |
+-----+

# 将列表 [92,90] 中的元素做运算，然后在 where 子句中进行条件判断。
trsgraph> GO FROM "player100" OVER follow WHERE properties(edge).degree NOT IN [x IN [92, 90] | x + $$player.age] \
YIELD dst(edge) AS id, properties(edge).degree AS degree;
+-----+-----+
| id      | degree |
+-----+-----+
| "player101" | 95    |
| "player102" | 90    |
+-----+-----+

# 将 MATCH 语句的查询结果作为列表中的元素进行运算并返回。
trsgraph> MATCH p = (n:player{name:"Tim Duncan"})-[:follow]->(m) \
RETURN [n IN nodes(p) | n.player.age + 100] AS r;
+-----+
| r      |
+-----+
| [142, 136] |
| [142, 141] |
+-----+

```

OpenCypher 兼容性

- 在 openCypher 中，查询越界元素时返回 `null`，而在 nGQL 中，查询单个越界元素时返回 `OUT_OF_RANGE`。

```

trsgraph> RETURN range(0,5)[-12];
+-----+
| range(0,5)[-12] |
+-----+
| OUT_OF_RANGE    |
+-----+

```

- 复合数据类型（例如 `set`、`map`、`list`）不能存储为点或边的属性。
- 建议修改图建模方式：将复合数据类型建模为点的邻边，而不是该点的自身属性，每条邻边可以动态增删，并且可以设置邻边的 `Rank` 值来控制邻边的顺序。
- `List` 中不支持 `pattern`，例如 `[(src)-[]->(m) | m.name]`。

最后更新: 2025年12月19日

4.2.7 集合

集合（Set）是复合数据类型，集合中是一组元素，与列表（List）不同的是，集合中的元素是无序的，且不允许重复。

集合用左花括号（{）和右花括号（}）包裹多个元素，各个元素之间用英文逗号（,）隔开。元素前后的空格在集合中被忽略，因此可以使用换行符、制表符和空格调整格式。

OpenCypher 兼容性

- 复合数据类型（例如 List、Set、Map）不能存储为点或边的属性。
- 在 OpenCypher 中，集合不是一个数据类型，而在 nGQL 中，用户可以使用集合。

示例

```
# 返回集合 {1,2,3}。
trsgraph> RETURN set{1, 2, 3} AS a;
+-----+
| a      |
+-----+
| {3, 2, 1} |
+-----+

# 返回集合 {1,2,1}，因为集合不允许重复元素，会返回 {1,2}，且顺序是无序的。
trsgraph> RETURN set{1, 2, 1} AS a;
+-----+
| a      |
+-----+
| {2, 1} |
+-----+

# 判断集合中是否有指定元素 1。
trsgraph> RETURN 1 IN set{1, 2} AS a;
+-----+
| a      |
+-----+
| true   |
+-----+

# 计算集合中的元素数量。
trsgraph> YIELD size(set{1, 2, 1}) AS a;
+---+
| a  |
+---+
| 2  |
+---+

# 返回目标点属性值组成的集合。
trsgraph> GO FROM "player100" OVER follow \
        YIELD set(properties($$).name,properties($$).age) as a;
+-----+
| a      |
+-----+
| {36, "Tony Parker"} |
| {41, "Manu Ginobili"} |
+-----+
```

最后更新: 2025年12月19日

4.2.8 映射

映射（Map）是复合数据类型。一个映射是一组键值对（Key-Value）的无序集合。在映射中，Key 是字符串类型，Value 可以是任何数据类型。用户可以通过 `map['<key>']` 的方法获取映射中的元素。

映射用左花括号（{）和右花括号（}）包裹多个键值对，各个键值对之间用英文逗号（,）隔开。键值对前后的空格在映射中被忽略，因此可以使用换行符、制表符和空格调整格式。

OpenCypher 兼容性

- 复合数据类型（例如 List、Set、Map）不能存储为点或边的属性。
- 不支持映射投影（map projection）。

示例

```
# 返回简单的映射。
trsgraph> YIELD map{key1: 'Value1', Key2: 'Value2'} as a;
+-----+
| a                |
+-----+
| {Key2: "Value2", key1: "Value1"} |
+-----+

# 返回列表类型的映射。
trsgraph> YIELD map{listKey: [{inner: 'Map1'}, {inner: 'Map2'}]} as a;
+-----+
| a                |
+-----+
| {listKey: [{inner: "Map1"}, {inner: "Map2"}]} |
+-----+

# 返回混合类型的映射。
trsgraph> RETURN map{a: LIST[1,2], b: SET{1,2,1}, c: "hee"} as a;
+-----+
| a                |
+-----+
| {a: [1, 2], b: {2, 1}, c: "hee"} |
+-----+

# 返回映射中的指定元素。
trsgraph> RETURN map{a: LIST[1,2], b: SET{1,2,1}, c: "hee"}["b"] AS b;
+-----+
| b                |
+-----+
| {2, 1} |
+-----+

# 判断映射中是否有指定key，暂不支持判断value。
trsgraph> RETURN "a" IN MAP{a:1, b:2} AS a;
+-----+
| a                |
+-----+
| true |
+-----+
```

最后更新: 2025年12月19日

4.2.9 类型转换

类型转换是指将表达式的类型转换为另一个类型。

TRSGraph 支持显式地转换类型。详情参见[类型转换函数](#)。

示例

```
# 将列表拆分，并各自转换为布尔值显示。
trsgraph> UNWIND [true, false, 'true', 'false', NULL] AS b \
    RETURN toBoolean(b) AS b;
+-----+
| b      |
+-----+
| true   |
+-----+
| false  |
+-----+
| true   |
+-----+
| false  |
+-----+
| __NULL__ |
+-----+

# 将整数或字符串转换为浮点数。
trsgraph> RETURN toFloat(1), toFloat('1.3'), toFloat('1e3'), toFloat('not a number');
+-----+-----+-----+-----+
| toFloat(1) | toFloat("1.3") | toFloat("1e3") | toFloat("not a number") |
+-----+-----+-----+-----+
| 1.0        | 1.3            | 1000.0         | __NULL__                |
+-----+-----+-----+-----+
```

最后更新: 2025年12月19日

4.2.10 地理空间

地理空间（GEOGRAPHY）是由经纬度构成的表示地理空间信息的数据类型。TRSGraph 当前支持简单地理要素中的 Point、LineString 和 Polygon 三种地理形状。支持 SQL-MM 3 中的部分核心 geo 解析、构造、格式设置、转换、谓词和度量等函数。

GEOGRAPHY

GEOGRAPHY 的基本类型是点，由经纬度确定一个点，例如 "POINT(3 8)" 表示经度为 3°，纬度为 8°。多个点可以构成线段或多边形。

类型	示例	说明
Point	"POINT(3 8)"	点类型
LineString	"LINESTRING(3 8, 4.7 73.23)"	线段类型
Polygon	"POLYGON((0 1, 1 2, 2 3, 0 1))"	多边形类型



请勿直接插入上述类型的 GEOGRAPHY 数据，例如 `INSERT VERTEX any_shape(geo) VALUES "1":("POINT(1 1)")`，需要使用 `geo` 函数指定数据类型后才能插入，例如 `INSERT VERTEX any_shape(geo) VALUES "1":(ST_GeogFromText("POINT(1 1)"))`。

示例

```
//创建 Tag，允许存储任意形状地理空间数据类型。
trsgraph> CREATE TAG IF NOT EXISTS any_shape(geo geography);

//创建 Tag，只允许存储点形状地理空间数据类型。
trsgraph> CREATE TAG IF NOT EXISTS only_point(geo geography(point));

//创建 Tag，只允许存储线段形状地理空间数据类型。
trsgraph> CREATE TAG IF NOT EXISTS only_linestring(geo geography(linestring));

//创建 Tag，只允许存储多边形形状地理空间数据类型。
trsgraph> CREATE TAG IF NOT EXISTS only_polygon(geo geography(polygon));

//创建 Edge type，允许存储任意形状地理空间数据类型。
trsgraph> CREATE EDGE IF NOT EXISTS any_shape_edge(geo geography);

//创建存储多边形地理空间的点。
trsgraph> INSERT VERTEX any_shape(geo) VALUES "103":(ST_GeogFromText("POLYGON((0 1, 1 2, 2 3, 0 1))"));

//创建存储多边形地理空间的边。
trsgraph> INSERT EDGE any_shape_edge(geo) VALUES "201"->"302":(ST_GeogFromText("POLYGON((0 1, 1 2, 2 3, 0 1))"));

//查询点 103 的属性 geo。
trsgraph> FETCH PROP ON any_shape "103" YIELD ST_ASText(any_shape.geo);
+-----+
| ST_ASText(any_shape.geo) |
+-----+
| "POLYGON((0 1, 1 2, 2 3, 0 1))" |
+-----+

//查询边 201->302 的属性 geo。
trsgraph> FETCH PROP ON any_shape_edge "201"->"302" YIELD ST_ASText(any_shape_edge.geo);
+-----+
| ST_ASText(any_shape_edge.geo) |
+-----+
| "POLYGON((0 1, 1 2, 2 3, 0 1))" |
+-----+

//为 geo 属性创建索引并使用 LOOKUP 查询。
trsgraph> CREATE TAG INDEX IF NOT EXISTS any_shape_geo_index ON any_shape(geo);
trsgraph> REBUILD TAG INDEX any_shape_geo_index;
trsgraph> LOOKUP ON any_shape YIELD ST_ASText(any_shape.geo);
+-----+
| ST_ASText(any_shape.geo) |
+-----+
| "POLYGON((0 1, 1 2, 2 3, 0 1))" |
+-----+
```

为 geo 属性创建索引时，还可以指定 geo 索引的参数。说明如下。

参数	默认值	说明
s2_max_level	30	S2 cell 用于填充的最大等级。取值： 1 ~ 30 。设置为小于默认值时，意味着会使用较大的单元格进行填充。
s2_max_cells	8	S2 cell 用于填充的最大数量，可以限制填充时的工作量。取值： 1 ~ 30 。对于复杂形状的区域（例如细矩形），可以使用更大的值。



指定如上两个参数对 Point 类型属性没有影响，Point 类型属性的 s2_max_level 强制为 30 。

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS any_shape_geo_index ON any_shape(geo) with (s2_max_level=30, s2_max_cells=8);
```

更多索引说明请参见[索引介绍](#)。

最后更新: 2025年12月19日

4.3 运算符

4.3.1 比较符

TRSGraph 支持的比较符如下。

符号	说明
<code>==</code>	相等
<code>!=, <></code>	不等于
<code>></code>	大于
<code>>=</code>	大于等于
<code><</code>	小于
<code><=</code>	小于等于
<code>IS NULL</code>	为 NULL
<code>IS NOT NULL</code>	不为 NULL
<code>IS EMPTY</code>	不存在
<code>IS NOT EMPTY</code>	存在

比较操作的结果是 `true` 或者 `false`。



- 比较不同类型的值通常没有定义，结果可能是 `NULL` 或其它。
- `EMPTY` 当前仅用于判断，不支持函数或者运算操作，包括且不限于 `GROUP BY`、`count()`、`sum()`、`max()`、`hash()`、`collect()`、`+`、`*`。

OpenCypher 兼容性

openCypher 中没有 `EMPTY`，因此不支持在 `MATCH` 语句中使用 `EMPTY`。

示例

`==`

字符串比较时，会区分大小写。不同类型的值不相等。



nGQL 中的相等符号是 `==`，openCypher 中的相等符号是 `=`。

```
trsgraph> RETURN 'A' == 'a', toUpper('A') == toUpper('a'), toLower('A') == toLower('a');
+-----+-----+-----+
| ("A"=="a") | (toUpper("A")==toUpper("a")) | (toLower("A")==toLower("a")) |
+-----+-----+-----+
| false      | true                          | true                          |
+-----+-----+-----+

trsgraph> RETURN '2' == 2, toInteger('2') == 2;
+-----+-----+
| ("2"==2) | (toInteger("2")==2) |
+-----+-----+
```

```
| false | true |
+-----+-----+
```

>

```
trsgraph> RETURN 3 > 2;
+-----+
| (3>2) |
+-----+
| true  |
+-----+

trsgraph> WITH 4 AS one, 3 AS two \
      RETURN one > two AS result;
+-----+
| result |
+-----+
| true   |
+-----+
```

>=

```
trsgraph> RETURN 2 >= "2", 2 >= 2;
+-----+-----+
| (2>="2") | (2>=2) |
+-----+-----+
| __NULL__ | true   |
+-----+-----+
```

<

```
trsgraph> YIELD 2.0 < 1.9;
+-----+
| (2<1.9) |
+-----+
| false   |
+-----+
```

<=

```
trsgraph> YIELD 0.11 <= 0.11;
+-----+
| (0.11<=0.11) |
+-----+
| true          |
+-----+
```

!=

```
trsgraph> YIELD 1 != '1';
+-----+
| (1!="1") |
+-----+
| true     |
+-----+
```

IS [NOT] NULL

```
# 返回关于 null 的一些判断结果。
trsgraph> RETURN null IS NULL AS value1, null == null AS value2, null != null AS value3;
+-----+-----+-----+
| value1 | value2 | value3 |
+-----+-----+-----+
| true   | __NULL__ | __NULL__ |
+-----+-----+-----+

# 返回关于 NULL 的一些属性信息。
trsgraph> RETURN length(NULL), size(NULL), count(NULL), NULL IS NULL, NULL IS NOT NULL, sin(NULL), NULL + NULL, [1, NULL] IS NULL;
+-----+-----+-----+-----+-----+-----+-----+-----+
| length(NULL) | size(NULL) | count(NULL) | NULL IS NULL | NULL IS NOT NULL | sin(NULL) | (NULL+NULL) | [1,NULL] IS NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
| __NULL__     | __NULL__   | 0           | true         | false           | __NULL__  | __NULL__    | false           |
+-----+-----+-----+-----+-----+-----+-----+-----+

# 创建 map 数据, 判断 map 的 name 属性是否不为 NULL。
trsgraph> WITH {name: null} AS `map` \
      RETURN `map`.name IS NOT NULL;
+-----+
| map.name IS NOT NULL |
+-----+
| false                |
+-----+

# 创建 map1、map2、map3 的数据, 并判断返回其 name 属性是否为 NULL。
trsgraph> WITH {name: 'Mats', name2: 'Pontus'} AS map1, \
```

```

    {name: null} AS map2, {notName: 0, notName2: null } AS map3 \
    RETURN map1.name IS NULL, map2.name IS NOT NULL, map3.name IS NULL;
+-----+-----+-----+
| map1.name IS NULL | map2.name IS NOT NULL | map3.name IS NULL |
+-----+-----+-----+
| false             | false                 | true                |
+-----+-----+-----+

# 查询所有 Tag 为 player 的点数据，判断并返回点的 age 属性是否为 NULL、name 属性是否不为 NULL、empty 属性是否为 NULL。
trsgraph> MATCH (n:player) \
    RETURN n.player.age IS NULL, n.player.name IS NOT NULL, n.player.empty IS NULL;
+-----+-----+-----+
| n.player.age IS NULL | n.player.name IS NOT NULL | n.player.empty IS NULL |
+-----+-----+-----+
| false                | true                      | true                    |
| false                | true                      | true                    |
...

```

IS [NOT] EMPTY

```

# 判断 null 是否存在。
trsgraph> RETURN null IS EMPTY;
+-----+
| NULL IS EMPTY |
+-----+
| false         |
+-----+

# 判断 a 字符串是否存在。
trsgraph> RETURN "a" IS NOT EMPTY;
+-----+
| "a" IS NOT EMPTY |
+-----+
| true              |
+-----+

# 遍历所有 player100 指向的目的点，并返回目的点数据 name 属性存在的点 id。
trsgraph> GO FROM "player100" OVER * WHERE properties($$).name IS NOT EMPTY YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "team204"  |
| "player101" |
| "player125" |
+-----+

```

最后更新: 2025年12月19日

4.3.2 布尔符

TRSGraph 支持的布尔符如下。

符号	说明
AND	逻辑与
OR	逻辑或
NOT	逻辑非
XOR	逻辑异或

对于以上运算的优先级，请参见 [运算优先级](#)。

对于带有 `NULL` 的逻辑运算，请参见 [NULL](#)。



非 0 数字不能转换为布尔值。

最后更新: 2025年12月19日

4.3.3 管道符

nGQL 支持使用管道符 (|) 将多个查询组合起来。

openCypher 兼容性

管道符仅适用于原生 nGQL。

语法

nGQL 和 SQL 之间的一个主要区别是子查询的组成方式。

- 在 SQL 中，子查询是嵌套在查询语句中的。
- 在 nGQL 中，子查询是通过类似 shell 中的管道符 (|) 实现的。

示例

```
trsgraph> GO FROM "player100" OVER follow \
          YIELD dst(edge) AS dstid, properties($$).name AS Name | \
          GO FROM $-.dstid OVER follow YIELD dst(edge);
```

```
+-----+
| dst(EDGE) |
+-----+
| "player100" |
| "player102" |
| "player125" |
| "player100" |
+-----+
```

必须在 `YIELD` 子句中为需要的返回结果设置别名，才能在管道符右侧使用引用符 `$-`，例如示例中的 `$-.dstid`。

性能提示

TRSGraph 中的管道对性能有影响，以 `A | B` 为例，体现在以下几个方面：

1. 管道是同步操作。也即需要管道之前的子句 `A` 执行完毕后，数据才能整体进入管道子句。
2. 如果 `A` 发大量数据给 `|`，整个查询请求的总体时延可能会非常大。此时可以尝试拆分这个语句：
 - a. 应用程序发送 `A`，
 - b. 将收到的返回结果在应用程序拆分，
 - c. 并发送给多个 graphd，
 - d. 每个 graphd 执行部分 `B`。

这样通常比单个 graphd 执行完整地 `A | B` 要快很多。

最后更新: 2025年12月19日

4.3.4 集合运算符

合并多个请求时，可以使用集合运算符，包括 `UNION`、`UNION ALL`、`INTERSECT` 和 `MINUS`。

所有集合运算符的优先级相同，如果一个 nQL 语句中有多个集合运算符，TRSGraph 会从左到右进行计算，除非用括号指定顺序。



集合运算符前后的查询语句中定义的变量名及顺序必需保持一致，例如 `RETURN a,b,c UNION RETURN a,b,c` 中的 `a,b,c` 的名称及顺序需要保持一致。

UNION、UNION DISTINCT、UNION ALL

```
<left> UNION [DISTINCT | ALL] <right> [ UNION [DISTINCT | ALL] <right> ...]
```

- 运算符 `UNION DISTINCT`（或使用缩写 `UNION`）返回两个集合 A 和 B 的并集，不包含重复的元素。
- 运算符 `UNION ALL` 返回两个集合 A 和 B 的并集，包含重复的元素。
- `left` 和 `right` 必须有相同数量的列和数据类型。如果需要转换数据类型，请参见[类型转换](#)。

示例

```
# 返回两个查询结果的并集，不包含重复的元素。
trsgraph> GO FROM "player102" OVER follow YIELD dst(edge) \
  UNION \
  GO FROM "player100" OVER follow YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "player100" |
| "player101" |
| "player125" |
+-----+

# 查询 Tag 为 player 的点，根据名称排序后获取前 3 条数据，并与数组组合并返回，不包含重复的元素。
trsgraph> MATCH (v:player) \
  WITH v.player.name AS n \
  RETURN n ORDER BY n LIMIT 3 \
  UNION \
  UNWIND ["Tony Parker", "Ben Simmons"] AS n \
  RETURN n;
+-----+
| n |
+-----+
| "Amar'e Stoudemire" |
| "Aron Baynes" |
| "Ben Simmons" |
| "Tony Parker" |
+-----+

# 返回两个查询结果的并集，包含重复的元素。
trsgraph> GO FROM "player102" OVER follow YIELD dst(edge) \
  UNION ALL \
  GO FROM "player100" OVER follow YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "player100" |
| "player101" |
| "player101" |
| "player125" |
+-----+

# 查询 Tag 为 player 的点，根据名称排序后获取前 3 条数据，并与数组组合并返回，包含重复的元素。
trsgraph> MATCH (v:player) \
  WITH v.player.name AS n \
  RETURN n ORDER BY n LIMIT 3 \
  UNION ALL \
  UNWIND ["Tony Parker", "Ben Simmons"] AS n \
  RETURN n;
+-----+
| n |
+-----+
| "Amar'e Stoudemire" |
| "Aron Baynes" |
| "Ben Simmons" |
| "Tony Parker" |
| "Ben Simmons" |
```

```

+-----+
# UNION 也可以和 YIELD 语句一起使用，去重时会检查每一行的所有列，每列都相同时才会去重。
trsgraph> GO FROM "player102" OVER follow \
    YIELD dst(edge) AS id, properties(edge).degree AS Degree, properties($$).age AS Age \
    UNION /* DISTINCT */ \
    GO FROM "player100" OVER follow \
    YIELD dst(edge) AS id, properties(edge).degree AS Degree, properties($$).age AS Age;
+-----+
| id          | Degree | Age |
+-----+
| "player100" | 75     | 42  |
| "player101" | 75     | 36  |
| "player101" | 95     | 36  |
| "player125" | 95     | 41  |
+-----+

```

INTERSECT

```
<left> INTERSECT <right>
```

- 运算符 `INTERSECT` 返回两个集合 A 和 B 的交集。
- `left` 和 `right` 必须有相同数量的列和数据类型。如果需要转换数据类型，请参见[类型转换](#)。

示例

```

# 返回两个查询结果的交集。
trsgraph> GO FROM "player102" OVER follow \
    YIELD dst(edge) AS id, properties(edge).degree AS Degree, properties($$).age AS Age \
    INTERSECT \
    GO FROM "player100" OVER follow \
    YIELD dst(edge) AS id, properties(edge).degree AS Degree, properties($$).age AS Age;
+-----+
| id | Degree | Age |
+-----+
+-----+

# 返回 player102 的邻居和边数据与 player100 的邻居和边数据之间的交集。
trsgraph> MATCH (v:player)-[e:follow]->(v2) \
    WHERE id(v) == "player102" \
    RETURN id(v2) AS id, e.degree AS Degree, v2.player.age AS Age \
    INTERSECT \
    MATCH (v:player)-[e:follow]->(v2) \
    WHERE id(v) == "player100" \
    RETURN id(v2) AS id, e.degree AS Degree, v2.player.age AS Age;
+-----+
| id | Degree | Age |
+-----+
+-----+

# 返回 [1,2] 与 [1,2,3,4] 的交集。
trsgraph> UNWIND [1,2] AS a RETURN a \
    INTERSECT \
    UNWIND [1,2,3,4] AS a \
    RETURN a;
+----+
| a |
+----+
| 1 |
| 2 |
+----+

```

MINUS

```
<left> MINUS <right>
```

运算符 `MINUS` 返回两个集合 A 和 B 的差异，即 $A-B$ 。请注意 `left` 和 `right` 的顺序， $A-B$ 表示在集合 A 中，但是不在集合 B 中的元素。

示例

```

# 返回在第一个查询结果中，但是不在第二个查询结果中的元素。
trsgraph> GO FROM "player100" OVER follow YIELD dst(edge) \
    MINUS \
    GO FROM "player102" OVER follow YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "player125" |
+-----+

# 返回在 player102 的邻居中，但不在 player100 的邻居中的元素。

```

```

trsgraph> GO FROM "player102" OVER follow YIELD dst(edge) AS id \
MINUS \
GO FROM "player100" OVER follow YIELD dst(edge) AS id;
+-----+
| id      |
+-----+
| "player100" |
+-----+

# 返回在 player102 的邻居中, 但不在 player100 的邻居中的元素。
trsgraph> MATCH (v:player)-[e:follow]->(v2) \
WHERE id(v) == "player102" \
RETURN id(v2) AS id \
MINUS \
MATCH (v:player)-[e:follow]->(v2) \
WHERE id(v) == "player100" \
RETURN id(v2) AS id;
+-----+
| id      |
+-----+
| "player100" |
+-----+

# 返回 [1,2,3] 中不与 4 相同的元素。
trsgraph> UNWIND [1,2,3] AS a RETURN a \
MINUS \
WITH 4 AS a \
RETURN a;
+----+
| a |
+----+
| 1 |
| 2 |
| 3 |
+----+

```

集合运算符和管道符的优先级

当查询包含集合运算符和管道符 (|) 时, **管道符**的优先级高。例如 `GO FROM 1 UNION GO FROM 2 | GO FROM 3` 相当于 `GO FROM 1 UNION (GO FROM 2 | GO FROM 3)`。

示例

```

trsgraph> GO FROM "player102" OVER follow \
YIELD dst(edge) AS play_dst \
UNION \
GO FROM "team200" OVER serve REVERSELY \
YIELD src(edge) AS play_src \
| GO FROM $-.play_src OVER follow YIELD dst(edge) AS play_dst;
+-----+
| play_dst |
+-----+
| "player100" |
| "player101" |
| "player117" |
| "player105" |
+-----+

```

```

nebula> GO FROM "player102" OVER follow YIELD follow._dst AS play_dst \
UNION \
GO FROM "team200" OVER serve REVERSELY YIELD serve._dst AS play_dst \
| GO FROM $-.play_dst OVER follow YIELD follow._dst AS play_dst;

```

该查询会先执行红框内的语句, 然后执行绿框的 UNION 操作。

圆括号可以修改执行的优先级, 例如:

```

trsgraph> (GO FROM "player102" OVER follow \
YIELD dst(edge) AS play_dst \
UNION \
GO FROM "team200" OVER serve REVERSELY \
YIELD src(edge) AS play_dst) \
| GO FROM $-.play_dst OVER follow YIELD dst(edge) AS play_dst;

```

该查询中, 圆括号包裹的部分先执行, 即先执行 UNION 操作, 再将结果结合管道符进行下一步操作。

最后更新: 2025年12月19日

4.3.5 字符串运算符

TRSGraph 支持使用字符串运算符进行连接、搜索、匹配运算。支持的运算符如下。

名称	说明
+	连接字符串。
CONTAINS	在字符串中执行搜索。
(NOT) IN	字符串是否匹配某个值。
(NOT) STARTS WITH	在字符串的开头执行匹配。
(NOT) ENDS WITH	在字符串的结尾执行匹配。
正则表达式	通过正则表达式匹配字符串。



所有搜索或匹配都区分大小写。

示例

```
+
# 返回连接后的字符串。
trsgraph> RETURN 'a' + 'b';
+-----+
| "a"+"b" |
+-----+
| "ab"     |
+-----+
trsgraph> UNWIND 'a' AS a UNWIND 'b' AS b RETURN a + b;
+-----+
| (a+b)    |
+-----+
| "ab"     |
+-----+
```

CONTAINS

CONTAINS 要求待运算的左右两边都是字符串类型。

```
# 查询返回 id 值为 player101, 且目标点属性 name 中含有"ets"的数据。
trsgraph> MATCH (s:player)-[e:serve]->(t:team) WHERE id(s) == "player101" \
AND t.team.name CONTAINS "ets" RETURN s.player.name, e.start_year, e.end_year, t.team.name;
+-----+-----+-----+-----+
| s.player.name | e.start_year | e.end_year | t.team.name |
+-----+-----+-----+-----+
| "Tony Parker" | 2018         | 2019         | "Hornets"    |
+-----+-----+-----+-----+

# 查询返回 player101 的边属性 start_year 转为字符串后含有"19", 且点属性 name 中含有"ny"的数据。
trsgraph> GO FROM "player101" OVER serve WHERE (STRING)properties(edge).start_year CONTAINS "19" AND \
properties($^).name CONTAINS "ny" \
YIELD properties($^).name, properties(edge).start_year, properties(edge).end_year, properties($$).name;
+-----+-----+-----+-----+
| properties($^).name | properties(EDGE).start_year | properties(EDGE).end_year | properties($$).name |
+-----+-----+-----+-----+
| "Tony Parker"       | 1999                         | 2018                         | "Spurs"             |
+-----+-----+-----+-----+

# 查询返回 palyer101 指向的目标点属性 name 中不含有"ets"的数据。
trsgraph> GO FROM "player101" OVER serve !(properties($$).name CONTAINS "ets") \
YIELD properties($^).name, properties(edge).start_year, properties(edge).end_year, properties($$).name;
+-----+-----+-----+-----+
| properties($^).name | properties(EDGE).start_year | properties(EDGE).end_year | properties($$).name |
+-----+-----+-----+-----+
| "Tony Parker"       | 1999                         | 2018                         | "Spurs"             |
+-----+-----+-----+-----+
```

(NOT) IN

```
# 返回列表中是否含有某个值。
trsgraph> RETURN 1 IN [1,2,3], "Yao" NOT IN ["Yi", "Tim", "Kobe"], NULL IN ["Yi", "Tim", "Kobe"];
+-----+-----+-----+-----+
| (1 IN [1,2,3]) | ("Yao" NOT IN ["Yi","Tim","Kobe"]) | (NULL IN ["Yi","Tim","Kobe"]) |
+-----+-----+-----+-----+
| true           | true                               | __NULL__                     |
+-----+-----+-----+-----+
```

(NOT) STARTS WITH

```
# 返回是否以某个字符串开头。
trsgraph> RETURN 'apple' STARTS WITH 'app', 'apple' STARTS WITH 'a', 'apple' STARTS WITH toUpper('a');
+-----+-----+-----+-----+
| ("apple" STARTS WITH "app") | ("apple" STARTS WITH "a") | ("apple" STARTS WITH toUpper("a")) |
+-----+-----+-----+-----+
| true                         | true                       | false                             |
+-----+-----+-----+-----+

trsgraph> RETURN 'apple' STARTS WITH 'b','apple' NOT STARTS WITH 'app';
+-----+-----+-----+
| ("apple" STARTS WITH "b") | ("apple" NOT STARTS WITH "app") |
+-----+-----+-----+
| false                     | false                             |
+-----+-----+-----+
```

(NOT) ENDS WITH

```
# 返回是否以某个字符串结尾。
trsgraph> RETURN 'apple' ENDS WITH 'app', 'apple' ENDS WITH 'e', 'apple' ENDS WITH 'E', 'apple' ENDS WITH 'b';
+-----+-----+-----+-----+
| ("apple" ENDS WITH "app") | ("apple" ENDS WITH "e") | ("apple" ENDS WITH "E") | ("apple" ENDS WITH "b") |
+-----+-----+-----+-----+
| false                     | true                     | false                     | false                     |
+-----+-----+-----+-----+
```

正则表达式



当前仅 opencypher 兼容语句（`MATCH`、`WITH` 等）支持正则表达式，原生 nGQL 语句（`FETCH`、`GO`、`LOOKUP` 等）不支持正则表达式。

TRSGraph 支持使用正则表达式进行过滤，正则表达式的语法是继承自 `std::regex`，用户可以使用语法 `=~ '<regex>'` 进行正则表达式匹配。例如：

```
# 返回是否匹配正则表达式。
trsgraph> RETURN "384748.39" =~ "\d+(\.\d{2})?";
+-----+-----+
| ("384748.39" =~ "\d+(\.\d{2})?") |
+-----+-----+
| true                               |
+-----+-----+

# 查询返回符合正则表达式的点数据。
trsgraph> MATCH (v:player) WHERE v.player.name =~ 'Tony.*' RETURN v.player.name;
+-----+
| v.player.name |
+-----+
| "Tony Parker" |
+-----+
```

最后更新: 2025年12月19日

4.3.6 列表运算符

TRSGraph 支持使用列表（List）运算符进行运算。支持的运算符如下。

名称	说明
+	连接列表。
IN	元素是否存在于列表中。
[]	使用下标操作符访问列表中的元素。

示例

```
# 返回连接后的列表。
trsgraph> YIELD [1,2,3,4,5]+[6,7] AS myList;
+-----+
| myList |
+-----+
| [1, 2, 3, 4, 5, 6, 7] |
+-----+

# 返回列表中的元素数量。
trsgraph> RETURN size([NULL, 1, 2]);
+-----+
| size([NULL,1,2]) |
+-----+
| 3 |
+-----+

# 返回 NULL 是否存在列表中。
trsgraph> RETURN NULL IN [NULL, 1];
+-----+
| (NULL IN [NULL,1]) |
+-----+
| __NULL__ |
+-----+

# 返回在列表[2, 3, 4, 5]中存在，且也在列表[2, 3, 8]中存在的元素，并拆分显示。
trsgraph> WITH [2, 3, 4, 5] AS numberlist \
  UNWIND numberlist AS number \
  WITH number \
  WHERE number IN [2, 3, 8] \
  RETURN number;
+-----+
| number |
+-----+
| 2 |
| 3 |
+-----+

# 返回列表中下标为 1 的元素。
trsgraph> WITH ['Anne', 'John', 'Bill', 'Diane', 'Eve'] AS names RETURN names[1] AS result;
+-----+
| result |
+-----+
| "John" |
+-----+
```

最后更新: 2025年12月19日

4.3.7 算术运算符

TRSGraph 支持的算术运算符如下。

符号	说明
<code>+</code>	加法
<code>-</code>	减法
<code>*</code>	乘法
<code>/</code>	除法
<code>%</code>	取模
<code>-</code>	负数符号

示例

```
# 返回 1+2 的结果。
trsgraph> RETURN 1+2 AS result;
+-----+
| result |
+-----+
| 3      |
+-----+

# 返回 -10+5 的结果。
trsgraph> RETURN -10+5 AS result;
+-----+
| result |
+-----+
| -5     |
+-----+

# 返回 (3*8)%5 的结果。
trsgraph> RETURN (3*8)%5 AS result;
+-----+
| result |
+-----+
| 4      |
+-----+
```

最后更新: 2025年12月19日

4.3.8 运算符优先级

nGQL 运算符的优先级从高到低排列如下（同一行的运算符优先级相同）：

- `-`（负数）
- `!`、`NOT`
- `*`、`/`、`%`
- `-`、`+`
- `==`、`>=`、`>`、`<=`、`<`、`<>`、`!=`
- `AND`
- `OR`、`XOR`
- `=`（赋值）

如果表达式中有相同优先级的运算符，运算是从左到右进行，只有赋值操作是例外（从右到左运算）。

运算符的优先级决定运算的顺序，要显式修改运算顺序，可以使用圆括号。

示例

```
trsgraph> RETURN 2+3*5;
+-----+
| (2+ (3*5)) |
+-----+
| 17          |
+-----+

trsgraph> RETURN (2+3)*5;
+-----+
| ((2+3)*5) |
+-----+
| 25         |
+-----+
```

openCypher 兼容性

在 openCypher 中，比较操作可以任意连接，例如 `x < y <= z` 等价于 `x < y AND y <= z`。

在 nGQL 中，`x < y <= z` 等价于 `(x < y) <= z`，`(x < y)` 的结果是一个布尔值，再将布尔值和 `z` 比较，最终结果是 `NULL`。

最后更新: 2025年12月19日

4.4 函数和表达式

4.4.1 内置数学函数

本文介绍 TRSGraph 支持的数学函数。

abs()

abs() 返回指定数字的绝对值。

语法: `abs(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN abs(-10);
+-----+
| abs(-10) |
+-----+
| 10       |
+-----+

trsgraph> RETURN abs(5-6);
+-----+
| abs(5-6) |
+-----+
| 1        |
+-----+
```

floor()

floor() 返回小于或等于指定数字的最大整数。

语法: `floor(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN floor(9.9);
+-----+
| floor(9.9) |
+-----+
| 9.0        |
+-----+
```

ceil()

ceil() 返回大于或等于指定数字的最小整数。

语法: `ceil(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN ceil(9.1);
+-----+
| ceil(9.1) |
+-----+
```

10.0
+-----+

round()

round() 返回指定数字四舍五入后的值。极端情况下请注意浮点数的精度问题。

语法： round(<expression>, <precision>, <mode>)

- `expression`：结果的数据类型为 double 的表达式。
- `precision`：代表精度的整数。小于 0 时，在小数点左侧做四舍五入。
- `mode`：代表舍入模式的字符串。可选参数。可选值如下：
- `DOWN`：向零舍入。
- `CEILING`：向正无穷舍入。
- `FLOOR`：向负无穷舍入。
- `HALF_UP`：基于精度向最接近的值舍入，如果最接近的值有两个，则取远离零的值。
- `HALF_DOWN`：基于精度向最接近的值舍入，如果最接近的值有两个，则取靠近零的值。
- `HALF_EVEN`：基于精度向最接近的值舍入，如果最接近的值有两个，则取最接近的偶数值。
- 返回类型：double。

示例：

```
trsgraph> RETURN round(314.15926, 2);
+-----+
| round(314.15926,2) |
+-----+
| 314.16 |
+-----+

trsgraph> RETURN round(314.15926, -1);
+-----+
| round(314.15926,-(1)) |
+-----+
| 310.0 |
+-----+

trsgraph> RETURN round(314.15926, 2, "DOWN");
+-----+
| round(314.15926,2,"DOWN") |
+-----+
| 314.15 |
+-----+

trsgraph> RETURN round(314.15926, 2, "CEILING");
+-----+
| round(314.15926,2,"CEILING") |
+-----+
| 314.16 |
+-----+

trsgraph> RETURN round(314.15926, 2, "FLOOR");
+-----+
| round(314.15926,2,"FLOOR") |
+-----+
| 314.15 |
+-----+

trsgraph> RETURN round(314.15, 1, "HALF_UP");
+-----+
| round(314.15,1,"HALF_UP") |
+-----+
| 314.2 |
+-----+

trsgraph> RETURN round(314.15, 1, "HALF_DOWN");
+-----+
| round(314.15,1,"HALF_DOWN") |
+-----+
| 314.1 |
+-----+

trsgraph> RETURN round(314.15, 1, "HALF_EVEN");
+-----+
| round(314.15,1,"HALF_EVEN") |
+-----+
```

```
| 314.2 |  
+-----+
```

sqrt()

sqrt() 返回指定数字的平方根。

语法: sqrt(<expression>)

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN sqrt(9);  
+-----+  
| sqrt(9) |  
+-----+  
| 3.0     |  
+-----+
```

cbrt()

cbrt() 返回指定数字的立方根。

语法: cbrt(<expression>)

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN cbrt(8);  
+-----+  
| cbrt(8) |  
+-----+  
| 2.0     |  
+-----+
```

hypot()

hypot() 返回直角三角形的斜边长。

语法: hypot(<expression_x>,<expression_y>)

- `expression_x`、`expression_y` : 结果的数据类型为 double 的表达式。表示直角三角形的边长 `x` 和 `y`。
- 返回类型: double。

示例:

```
trsgraph> RETURN hypot(3,(2*2));  
+-----+  
| hypot(3,(2*2)) |  
+-----+  
| 5.0            |  
+-----+
```

pow()

pow() 返回指定数字的幂 (x^y)。

语法: `pow(<expression_x>,<expression_y>,)`

- `expression_x` : 结果的数据类型为 double 的表达式。表示底数 x 。
- `expression_y` : 结果的数据类型为 double 的表达式。表示指数 y 。
- 返回类型: double。

示例:

```
trsgraph> RETURN pow(3,3);
+-----+
| pow(3,3) |
+-----+
| 27      |
+-----+
```

exp()

`exp()` 返回自然常数 e 的幂 (e^x)。

语法: `exp(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。表示指数 x 。
- 返回类型: double。

示例:

```
trsgraph> RETURN exp(2);
+-----+
| exp(2)      |
+-----+
| 7.38905609893065 |
+-----+
```

exp2()

`exp2()` 返回2的幂 (2^x)。

语法: `exp2(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。表示指数 x 。
- 返回类型: double。

示例:

```
trsgraph> RETURN exp2(3);
+-----+
| exp2(3) |
+-----+
| 8.0     |
+-----+
```

log()

`log()` 返回以自然数 e 为底的对数 ($\log_e(N)$)。

语法: `log(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。表示真数 N 。
- 返回类型: double。

示例:

```
trsgraph> RETURN log(8);
+-----+
```

```
| log(8) |
+-----+
| 2.0794415416798357 |
+-----+
```

log2()

log2() 返回以 2 为底的对数 $(\log_2(N))$ 。

语法: `log2(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。表示真数 N 。
- 返回类型: double。

示例:

```
trsgaph> RETURN log2(8);
+-----+
| log2(8) |
+-----+
| 3.0      |
+-----+
```

log10()

log10() 返回以 10 为底的对数 $(\log_{10}(N))$ 。

语法: `log10(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。表示真数 N 。
- 返回类型: double。

示例:

```
trsgaph> RETURN log10(100);
+-----+
| log10(100) |
+-----+
| 2.0         |
+-----+
```

sin()

sin() 返回指定数字的正弦值。可以使用函数 `radians()` 将角度转化为弧度。

语法: `sin(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgaph> RETURN sin(3);
+-----+
| sin(3) |
+-----+
| 0.1411200080598672 |
+-----+
```

asin()

asin() 返回指定数字的反正弦值。可以使用函数 `radians()` 将角度转化为弧度。

语法: `asin(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN asin(0.5);
+-----+
| asin(0.5) |
+-----+
| 0.5235987755982989 |
+-----+
```

cos()

`cos()` 返回指定数字的余弦值。可以使用函数 `radians()` 将角度转化为弧度。

语法: `cos(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN cos(0.5);
+-----+
| cos(0.5) |
+-----+
| 0.8775825618903728 |
+-----+
```

acos()

`acos()` 返回指定数字的反余弦值。可以使用函数 `radians()` 将角度转化为弧度。

语法: `acos(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN acos(0.5);
+-----+
| acos(0.5) |
+-----+
| 1.0471975511965979 |
+-----+
```

tan()

`tan()` 返回指定数字的正切值。可以使用函数 `radians()` 将角度转化为弧度。

语法: `tan(<expression>)`

- `expression` : 结果的数据类型为 double 的表达式。
- 返回类型: double。

示例:

```
trsgraph> RETURN tan(0.5);
+-----+
| tan(0.5) |
+-----+
```

```
| 0.5463024898437905 |  
+-----+
```

atan()

atan() 返回指定数字的反正切值。可以使用函数 radians() 将角度转化为弧度。

语法: atan(<expression>)

- expression：结果的数据类型为 double 的表达式。
- 返回类型：double。

示例：

```
trsgraph> RETURN atan(0.5);  
+-----+  
| atan(0.5) |  
+-----+  
| 0.4636476090008061 |  
+-----+
```

rand()

rand() 返回 [0,1) 内的随机浮点数。

语法: rand()

- 返回类型：double。

示例：

```
trsgraph> RETURN rand();  
+-----+  
| rand() |  
+-----+  
| 0.6545837172298736 |  
+-----+
```

rand32()

rand32() 返回指定范围 ([min, max)) 内的随机 32 位整数。

语法: rand32(<expression_min>,<expression_max>)

- expression_min：结果的数据类型为 int 的表达式。表示最小值 min。
- expression_max：结果的数据类型为 int 的表达式。表示最大值 max。
- 返回类型：int。
- 用户可以只传入一个参数，该参数会判定为 max，此时 min 默认为 0。如果不传入参数，此时会从带符号的 32 位 int 范围内随机返回。

示例：

```
trsgraph> RETURN rand32(1,100);  
+-----+  
| rand32(1,100) |  
+-----+  
| 63 |  
+-----+
```

rand64()

rand64() 返回指定范围 ([min, max)) 内的随机 64 位整数。

语法: `rand64(<expression_min>,<expression_max>)`

- `expression_min`: 结果的数据类型为 `int` 的表达式。表示最小值 `min`。
- `expression_max`: 结果的数据类型为 `int` 的表达式。表示最大值 `max`。
- 返回类型: `int`。
- 用户可以只传入一个参数, 该参数会判定为 `max`, 此时 `min` 默认为 `0`。如果不传入参数, 此时会从带符号的 64 位 `int` 范围内随机返回。

示例:

```
trsgraph> RETURN rand64(1,100);
+-----+
| rand64(1,100) |
+-----+
| 34             |
+-----+
```

bit_and()

`bit_and()` 返回按位进行 AND 运算后的结果。

语法: `bit_and(<expression_1>,<expression_2>)`

- `expression_1`、`expression_2`: 结果的数据类型为 `int` 的表达式。
- 返回类型: `int`。

示例:

```
trsgraph> RETURN bit_and(5,6);
+-----+
| bit_and(5,6) |
+-----+
| 4             |
+-----+
```

bit_or()

`bit_or()` 返回按位进行 OR 运算后的结果。

语法: `bit_or(<expression_1>,<expression_2>)`

- `expression_1`、`expression_2`: 结果的数据类型为 `int` 的表达式。
- 返回类型: `int`。

示例:

```
trsgraph> RETURN bit_or(5,6);
+-----+
| bit_or(5,6) |
+-----+
| 7             |
+-----+
```

bit_xor()

`bit_xor()` 返回按位进行 XOR 运算后的结果。

语法: `bit_xor(<expression_1>,<expression_2>)`

- `expression_1`、`expression_2`: 结果的数据类型为 `int` 的表达式。
- 返回类型: `int`。

示例:

```
trsgraph> RETURN bit_xor(5,6);
+-----+
| bit_xor(5,6) |
+-----+
| 3           |
+-----+
```

size()

size() 返回列表或映射中元素的数量，或者返回字符串的长度。

语法： `size(<expression>|<string>)`

- `expression`：列表或映射的表达式。
- `string`：指定的字符串。
- 返回类型：int。

示例：

```
trsgraph> RETURN size([1,2,3,4]);
+-----+
| size([1,2,3,4]) |
+-----+
| 4               |
+-----+
```

```
trsgraph> RETURN size("basketballplayer") as size;
+-----+
| size |
+-----+
| 16   |
+-----+
```

range()

range() 返回指定范围（`[start,end]`）中指定步长的值组成的列表。

语法： `range(<expression_start>,<expression_end>[,<expression_step>])`

- `expression_start`：结果的数据类型为 int 的表达式。表示起始值 `start`。
- `expression_end`：结果的数据类型为 int 的表达式。表示结束值 `end`。
- `expression_step`：结果的数据类型为 int 的表达式。表示步长 `step`，默认值为 1。
- 返回类型：list。

示例：

```
trsgraph> RETURN range(1,3*3,2);
+-----+
| range(1,(3*3),2) |
+-----+
| [1, 3, 5, 7, 9] |
+-----+
```

sign()

sign() 返回指定数字的正负号。如果数字为 0，则返回 0；如果数字为负数，则返回 -1；如果数字为正数，则返回 1。

语法： `sign(<expression>)`

- `expression`：结果的数据类型为 double 的表达式。
- 返回类型：int。

示例：

```
trsgraph> RETURN sign(10);
+-----+
| sign(10) |
+-----+
| 1         |
+-----+
```

e()

e() 返回自然对数的底 e (2.718281828459045) 。

语法： `e()`

- 返回类型：double。

示例：

```
trsgraph> RETURN e();
+-----+
| e()      |
+-----+
| 2.718281828459045 |
+-----+
```

pi()

pi() 返回数学常数 π (3.141592653589793) 。

语法： `pi()`

- 返回类型：double。

示例：

```
trsgraph> RETURN pi();
+-----+
| pi()      |
+-----+
| 3.141592653589793 |
+-----+
```

radians()

radians() 返回指定角度的弧度。

语法： `radians(<angle>)`

- 返回类型：double。

示例：

```
trsgraph> RETURN radians(180);
+-----+
| radians(180) |
+-----+
| 3.141592653589793 |
+-----+
```

最后更新: 2025年12月19日

4.4.2 聚合函数

本文介绍 TRSGraph 支持的聚合函数。

avg()

avg() 返回参数的平均值。

语法: avg(<expression>)

- 返回类型: double。

示例:

```
# 返回 Tag player 的 age 属性的平均值。
trsgraph> MATCH (v:player) RETURN avg(v.player.age);
+-----+
| avg(v.player.age) |
+-----+
| 33.294117647058826 |
+-----+
```

count()

count() 返回参数的数量。

- (原生 nGQL) 用户可以同时使用 count() 和 GROUP BY 对传参进行分组和计数，再使用 YIELD 返回结果。
- (openCypher 方式) 用户可以使用 count() 对指定的值进行计数，再使用 RETURN 返回结果。不需要使用 GROUP BY。

语法: count({<expression> | *})

- count(*) 返回总行数 (包括 NULL)。
- 返回类型: int。

示例:

```
# 将列表拆分，并返回拆分后行的数量，列表元素的数量，拆分后将行去重后的数量。
trsgraph> WITH [NULL, 1, 1, 2, 2] As a UNWIND a AS b \
RETURN count(b), count(*), count(DISTINCT b);
+-----+-----+-----+
| count(b) | count(*) | count(distinct b) |
+-----+-----+-----+
| 4        | 5        | 2                  |
+-----+-----+-----+

# 返回 player101 follow 的人，以及 follow player101 的人，即双向查询。
# 使用`count()`和`GROUP BY`进行分组和计数。
trsgraph> GO FROM "player101" OVER follow BIDIRECT \
YIELD properties($$).name AS Name \
| GROUP BY $-.Name YIELD $-.Name, count(*);
+-----+-----+
| $-.Name          | count(*) |
+-----+-----+
| "LaMarcus Aldridge" | 2        |
| "Tim Duncan"       | 2        |
| "Marco Belinelli"  | 1        |
| "Manu Ginobili"    | 1        |
| "Boris Diaw"       | 1        |
| "Dejounte Murray" | 1        |
+-----+-----+

# 使用`count()`进行计数。
trsgraph> MATCH (v1:player)-[:follow]-(v2:player) \
WHERE id(v1)== "player101" \
RETURN v2.player.name AS Name, count(*) as cnt ORDER BY cnt DESC;
+-----+-----+
| Name              | cnt |
+-----+-----+
| "LaMarcus Aldridge" | 2   |
| "Tim Duncan"       | 2   |
| "Boris Diaw"       | 1   |
| "Manu Ginobili"    | 1   |
| "Dejounte Murray" | 1   |
+-----+-----+
```

```
| "Marco Belinelli" | 1 |
+-----+-----+
```

上述示例的返回结果有两列：

- `$-.Name`：查询结果包含的姓名。
- `count(*)`：姓名出现的次数。

因为测试数据集 `basketballplayer` 中没有重复的姓名，`count(*)` 列中数字 2 表示该行的人和 `player101` 是互相 `follow` 的关系。

```
# 方法一：统计数据库中的年龄分布情况。
trsgraph> LOOKUP ON player \
  YIELD player.age As playerage \
  | GROUP BY $-.playerage \
  YIELD $-.playerage as age, count(*) AS number \
  | ORDER BY $-.number DESC, $-.age DESC;

+-----+-----+
| age | number |
+-----+-----+
| 34 | 4 |
| 33 | 4 |
| 30 | 4 |
| 29 | 4 |
| 38 | 3 |
+-----+-----+
...

# 方法二：统计数据库中的年龄分布情况。
trsgraph> MATCH (n:player) \
  RETURN n.player.age as age, count(*) as number \
  ORDER BY number DESC, age DESC;

+-----+-----+
| age | number |
+-----+-----+
| 34 | 4 |
| 33 | 4 |
| 30 | 4 |
| 29 | 4 |
| 38 | 3 |
+-----+-----+
...

# 统计 Tim Duncan 关联的边数。
trsgraph> MATCH (v:player{name:"Tim Duncan"}) -[e]- (v2) \
  RETURN count(e);

+-----+
| count(e) |
+-----+
| 13 |
+-----+

# 多跳查询，统计 Tim Duncan 关联的边数，返回两列（不去重和去重）。
trsgraph> MATCH (n:player {name : "Tim Duncan"})-[]->(friend:player)-[]->(fof:player) \
  RETURN count(fof), count(DISTINCT fof);

+-----+-----+
| count(fof) | count(distinct fof) |
+-----+-----+
| 4 | 3 |
+-----+-----+
```

max()

max() 返回参数的最大值。

语法：`max(<expression>)`

- 返回类型：与原参数相同。

示例：

```
# 返回 Tag player 的 age 属性的最大值。
trsgraph> MATCH (v:player) RETURN max(v.player.age);

+-----+
| max(v.player.age) |
+-----+
| 47 |
+-----+
```

min()

min() 返回参数的最小值。

语法： min(<expression>)

- 返回类型：与原参数相同。

示例：

```
# 返回 Tag player 的 age 属性的最小值。
trsgraph> MATCH (v:player) RETURN min(v.player.age);
+-----+
| min(v.player.age) |
+-----+
| 20                |
+-----+
```

collect()

collect() 返回一个符合表达式返回结果的列表。该函数可以将多条记录或值合并进一个列表，实现数据聚合。

语法： collect(<expression>)

- 返回类型： list。

示例：

```
# 将列表拆分为单独的行。
trsgraph> UNWIND [1, 2, 1] AS a \
          RETURN a;
+---+
| a |
+---+
| 1 |
| 2 |
| 1 |
+---+

# 将列表拆分为单独的行记录后再合并为一个列表返回。
trsgraph> UNWIND [1, 2, 1] AS a \
          RETURN collect(a);
+-----+
| collect(a) |
+-----+
| [1, 2, 1]  |
+-----+

# 统计并返回列表中相同元素的数量。
trsgraph> UNWIND [1, 2, 1] AS a \
          RETURN a, collect(a), size(collect(a));
+-----+-----+
| a | collect(a) | size(collect(a)) |
+-----+-----+
| 2 | [2]        | 1                 |
| 1 | [1, 1]     | 2                 |
+-----+-----+

# 降序排列，限制输出行数为 3，然后将结果输出到列表中。
trsgraph> UNWIND ["c", "b", "a", "d" ] AS p \
          WITH p AS q \
          ORDER BY q DESC LIMIT 3 \
          RETURN collect(q);
+-----+
| collect(q) |
+-----+
| ["d", "c", "b"] |
+-----+

# 将列表拆分后去重，再合并为新的列表返回。
trsgraph> WITH [1, 1, 2, 2] AS coll \
          UNWIND coll AS x \
          WITH DISTINCT x \
          RETURN collect(x) AS ss;
+-----+
| ss      |
+-----+
| [1, 2]  |
+-----+

# 将 Tag player 的 age 属性值合并为列表返回。
trsgraph> MATCH (n:player) \
          RETURN collect(n.player.age);
+-----+
```



```
| collect(n.player.age) |
+-----+
| [32, 32, 34, 29, 41, 40, 33, 25, 40, 37, ... |
...

# 基于年龄聚合姓名。
trsgraph> MATCH (n:player) \
    RETURN n.player.age AS age, collect(n.player.name);
+-----+
| age | collect(n.player.name) |
+-----+
| 24 | ["Giannis Antetokounmpo"] |
| 20 | ["Luka Doncic"] |
| 25 | ["Joel Embiid", "Kyle Anderson"] |
+-----+
...

# 将 player100 的目的点 name 属性值聚合，结果合并为列表返回。
trsgraph> GO FROM "player100" OVER serve \
    YIELD properties($$).name AS name \
    | GROUP BY $-.name \
    YIELD collect($-.name) AS name;
+-----+
| name |
+-----+
| ["Spurs"] |
+-----+

# 将 Tag player 的 age 属性值聚合，结果合并为列表返回。
trsgraph> LOOKUP ON player \
    YIELD player.age AS playerage \
    | GROUP BY $-.playerage \
    YIELD collect($-.playerage) AS playerage;
+-----+
| playerage |
+-----+
| [22] |
| [47] |
| [43] |
| [25, 25] |
+-----+
...
```

std()

std() 返回参数的总体标准差。

语法： `std(<expression>)`

- 返回类型：double。

示例：

```
# 返回 Tag player 的 age 属性值的标准差。
trsgraph> MATCH (v:player) RETURN std(v.player.age);
+-----+
| std(v.player.age) |
+-----+
| 6.423895701687502 |
+-----+
```

sum()

sum() 返回参数的和。

语法： `sum(<expression>)`

- 返回类型：与原参数相同。

示例：

```
# 返回 Tag player 的 age 属性值的总和。
trsgraph> MATCH (v:player) RETURN sum(v.player.age);
+-----+
| sum(v.player.age) |
+-----+
| 1698 |
+-----+
```

聚合示例

```
trsgraph> GO FROM "player100" OVER follow YIELD dst(edge) AS dst, properties($$.age AS age \
| GROUP BY $-.dst \
YIELD \
$-.dst AS dst, \
toInteger((sum($-.age)/count($-.age)))+avg(distinct $-.age+1)+1 AS statistics;
+-----+-----+
| dst          | statistics |
+-----+-----+
| "player125"  | 84.0       |
| "player101"  | 74.0       |
+-----+-----+
```

最后更新: 2025年12月19日

4.4.3 内置字符串函数

本文介绍 TRSGraph 支持的字符串函数。

注意事项

- 字符串的表示方式为用双引号或单引号包裹。
- 和 SQL 一样，nGQL 的字符索引（位置）从 1 开始。但是 C 语言的字符索引是从 0 开始的。

strcasecmp()

strcasecmp() 比较两个字符串（不区分大小写）。

语法： `strcasecmp(<string_a>,<string_b>)`

- `string_a`、`string_b`：待比较的字符串。
- 返回类型：int。
- 当 `string_a = string_b` 时，返回 0，当 `string_a > string_b` 是，返回大于 0 的数，当 `string_a < string_b` 时，返回小于 0 的数。

示例：

```
# 比较字符串 "a" 和 "aa"
trsgraph> RETURN strcasecmp("a","aa");
+-----+
| strcasecmp("a","aa") |
+-----+
| -97                  |
+-----+
```

lower() 和 toLower()

lower() 和 toLower() 都可以返回指定字符串的小写形式。

语法： `lower(<string>)`、`toLower(<string>)`

- `string`：指定的字符串。
- 返回类型：string。

示例：

```
# 返回 "Basketball_Player" 小写形式
trsgraph> RETURN lower("Basketball_Player");
+-----+
| lower("Basketball_Player") |
+-----+
| "basketball_player"       |
+-----+
```

upper() 和 toUpper()

upper() 和 toUpper() 都可以返回指定字符串的大写形式。

语法： `upper(<string>)`、`toUpper(<string>)`

- `string`：指定的字符串。
- 返回类型：string。

示例：

```
# 返回 "Basketball_Player" 大写形式
trsgraph> RETURN upper("Basketball_Player");
+-----+
| upper("Basketball_Player") |
+-----+
| "BASKETBALL_PLAYER"      |
+-----+
```

length()

length() 返回：

- 指定字符串的长度，单位：字节。
- 路径的长度，单位：跳。

语法： `length({<string>|<path>})`

- `string`：指定的字符串。
- `path`：指定的路径，使用变量表示。
- 返回类型：int。

示例：

```
# 返回字符串 "basketball" 的长度
trsgraph> RETURN length("basketball");
+-----+
| length("basketball") |
+-----+
| 10                    |
+-----+
```

```
# 返回 p 到 v2 路径的长度
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-->(v2) return length(p);
+-----+
| length(p) |
+-----+
| 1          |
| 1          |
| 1          |
+-----+
```

trim()

trim() 删除指定字符串头部和尾部的空格。

语法： `trim(<string>)`

- `string`：指定的字符串。
- 返回类型：string。

示例：

```
# 裁剪掉字符串 " basketball player " 头部和尾部的空格
trsgraph> RETURN trim(" basketball player ");
+-----+
| trim(" basketball player ") |
+-----+
| "basketball player"         |
+-----+
```

ltrim()

ltrim() 删除字符串头部的空格。

语法: `ltrim(<string>)`

- `string`: 指定的字符串。
- 返回类型: `string`。

示例:

```
# 裁剪掉字符串 " basketball player " 头部的空格
trsgraph> RETURN ltrim(" basketball player ");
+-----+
| ltrim(" basketball player ") |
+-----+
| "basketball player "         |
+-----+
```

rtrim()

`rtrim()` 删除字符串尾部的空格。

语法: `rtrim(<string>)`

- `string`: 指定的字符串。
- 返回类型: `string`。

示例:

```
# 裁剪掉字符串 " basketball player " 尾部的空格
trsgraph> RETURN rtrim(" basketball player ");
+-----+
| rtrim(" basketball player ") |
+-----+
| " basketball player"         |
+-----+
```

left()

`left()` 返回指定字符串头部若干个字符组成的子字符串。

语法: `left(<string>,<count>)`

- `string`: 指定的字符串。
- `count`: 指定从头部开始的字符数量。如果 `count` 超过字符串的长度, 则返回字符串本身。
- 返回类型: `string`。

示例:

```
# 返回字符串 "basketball_player" 从头部开始的 6 个字符组成的字符串
trsgraph> RETURN left("basketball_player",6);
+-----+
| left("basketball_player",6) |
+-----+
| "basket"                   |
+-----+
```

right()

`right()` 返回指定字符串尾部若干个字符组成的子字符串。

语法: `right(<string>,<count>)`

- `string`: 指定的字符串。
- `count`: 指定从尾部开始的字符数量。如果 `count` 超过字符串的长度, 则返回字符串本身。
- 返回类型: `string`。

示例：

```
# 返回字符串 "basketball_player" 从尾部 6 个字符组成的字符串
trsgraph> RETURN right("basketball_player",6);
+-----+
| right("basketball_player",6) |
+-----+
| "player"                     |
+-----+
```

lpad()

lpad() 在指定字符串的头部填充字符串至指定长度，并返回结果字符串。

语法： lpad(<string>,<count>,<letters>)

- `string`：指定的字符串。
- `count`：指定从尾部开始将要返回的字符数量。如果 `count` 少于 `string` 字符串的长度，则只返回 `string` 字符串从前到后的 `count` 个字符。
- `letters`：从头部填充的字符串。
- 返回类型：string。

示例：

```
# 在字符串 "abcd" 头部填充 "b" 字符，延长字符串长度至 10
trsgraph> RETURN lpad("abcd",10,"b");
+-----+
| lpad("abcd",10,"b") |
+-----+
| "bbbbbbabcd"       |
+-----+

# 返回字符串 "abcd" 的前三个字符
trsgraph> RETURN lpad("abcd",3,"b");
+-----+
| lpad("abcd",3,"b") |
+-----+
| "abc"              |
+-----+
```

rpadd()

rpadd() 在指定字符串的尾部填充字符串至指定长度，并返回结果字符串。

语法： rpadd(<string>,<count>,<letters>)

- `string`：指定的字符串。
- `count`：指定从头部开始将要返回的字符数量。如果 `count` 少于 `string` 字符串的长度，则只返回 `string` 字符串从前到后的 `count` 个字符。
- `letters`：从尾部填充的字符串。
- 返回类型：string。

示例：

```
# 在字符串 "abcd" 尾部填充 "b" 字符，延长字符串长度至 10
trsgraph> RETURN rpadd("abcd",10,"b");
+-----+
| rpadd("abcd",10,"b") |
+-----+
| "abcdbbbbbb"        |
+-----+

# 返回字符串 "abcd" 的前三个字符
trsgraph> RETURN rpadd("abcd",3,"b");
+-----+
| rpadd("abcd",3,"b") |
+-----+
| "abc"               |
+-----+
```

substr() 和 substring()

substr() 和 substring() 从指定字符串的指定位置开始（不包括开始位置的字符），提取后面的若干个字符，组成新的字符串并返回。

语法: substr(<string>,<pos>,<count>) 、 substring(<string>,<pos>,<count>)

- string：指定的字符串。
- pos：指定开始的位置，即字符索引，数据类型为 int。
- count：指定从开始位置往后提取的字符数量。
- 返回类型：string。

SUBSTR() 和 SUBSTRING() 的返回说明

- 如果 pos 为 0，表示从指定字符串头部开始提取（包括第一个字符）。
- 如果 pos 大于最大字符索引，则返回空字符串。
- 如果 pos 是负数，则返回 BAD_DATA。
- 如果省略 count，则返回从 pos 位置开始到字符串末尾的子字符串。
- 如果 count 为 0，则返回空字符串。
- 使用 NULL 作为任何参数会出现错误。

 openCypher 兼容性

在 openCypher 中，如果字符串 a 为 null，会返回 null。

示例：

```
# 返回字符串 "abcdefg" 从下标 2 开始的 4 个字符组成的子字符串
trsggraph> RETURN substr("abcdefg",2,4);
+-----+
| substr("abcdefg",2,4) |
+-----+
| "cdef"                |
+-----+

# 返回字符串 "abcdefg" 从下标 0 开始的 4 个字符组成的子字符串
trsggraph> RETURN substr("abcdefg",0,4);
+-----+
| substr("abcdefg",0,4) |
+-----+
| "abcd"                |
+-----+

# 返回字符串 "abcdefg" 从下标 0 开始到结尾的字符组成的子字符串
trsggraph> RETURN substr("abcdefg",2);
+-----+
| substr("abcdefg",2)   |
+-----+
| "cdefg"               |
+-----+
```

reverse()

reverse() 逆序返回指定的字符串。

语法: reverse(<string>)

- string：指定的字符串。
- 返回类型：string。

示例：

```
# 逆序返回字符串 "abcdefg"
trsgraph> RETURN reverse("abcdefg");
+-----+
| reverse("abcdefg") |
+-----+
| "gfedcba"          |
+-----+
```

replace()

replace() 将指定字符串中的子字符串 a 替换为字符串 b。

语法: `replace(<string>,<substr_a>,<string_b>)`

- `string`: 指定的字符串。
- `substr_a`: 子字符串 a。
- `string_b`: 字符串 b。
- 返回类型: string。

示例:

```
trsgraph> RETURN replace("abcdefg","cd","AAAAA");
+-----+
| replace("abcdefg","cd","AAAAA") |
+-----+
| "abAAAAAefg"                    |
+-----+
```

split()

split() 将子字符串 b 识别为分隔符，分隔指定字符串，并返回分隔后的字符串列表。

语法: `split(<string>,<substr>)`

- `string`: 指定的字符串。
- `substr`: 子字符串 b。
- 返回类型: list。

示例:

```
trsgraph> RETURN split("basketballplayer","a");
+-----+
| split("basketballplayer","a") |
+-----+
| ["b", "sketb", "llpl", "yer"] |
+-----+
```

concat()

concat() 返回所有参数连接成的字符串。

语法: `concat(<string1>,<string2>,...)`

- 函数至少需要两个或以上字符串参数。如果字符串参数只有一个，则返回该字符串参数本身。
- 如果任何一个的字符串参数为 `NULL`，则 `concat()` 函数返回值为 `NULL`。
- 返回类型: string。

示例:

```
//拼接 1, 2, 3
trsgraph> RETURN concat("1","2","3") AS r;
+-----+
| r      |
+-----+
```



```
+-----+
| "123" |
+-----+

//字符串参数有 NULL
trsgraph> RETURN concat("1","2",NULL) AS r;
+-----+
| r      |
+-----+
|  __NULL__ |
+-----+

trsgraph> GO FROM "player100" over follow \
        YIELD concat(src(edge), properties($^).age, properties($$).name, properties(edge).degree) AS A;
+-----+
| A      |
+-----+
| "player10042Tony Parker95" |
| "player10042Manu Ginobili95" |
+-----+
```

concat_ws()

concat_ws() 返回用分隔符（separator）连接的所有字符串。

语法: concat_ws(<separator>,<string1>,<string2>,...)

- 函数至少需要两个或以上字符串参数。
- 如果分隔符为 NULL 时，concat_ws() 函数才返回 NULL。
- 如果分隔符不为 NULL，字符串参数只有一个，则返回该字符串参数本身。
- 字符串参数存在 NULL 值时，忽略 NULL 值，继续连接下一个参数。

示例:

```
//分隔符为 +, 连接 a, b, c。
trsgraph> RETURN concat_ws("+","a","b","c") AS r;
+-----+
| r      |
+-----+
| "a+b+c" |
+-----+

//分隔符为 NULL。
neubla> RETURN concat_ws(NULL,"a","b","c") AS r;
+-----+
| r      |
+-----+
|  __NULL__ |
+-----+

//分隔符为 +, 字符串参数有 NULL。
trsgraph> RETURN concat_ws("+","a",NULL,"b","c") AS r;
+-----+
| r      |
+-----+
| "a+b+c" |
+-----+

//分隔符为 +。字符串参数只有一个
trsgraph> RETURN concat_ws("+","a") AS r;
+-----+
| r      |
+-----+
| "a"    |
+-----+

trsgraph> GO FROM "player100" over follow \
        YIELD concat_ws(" ",src(edge), properties($^).age, properties($$).name, properties(edge).degree) AS A;
+-----+
| A      |
+-----+
| "player100 42 Tony Parker 95" |
| "player100 42 Manu Ginobili 95" |
+-----+
```

extract()

extract() 从指定字符串中提取符合正则表达式的子字符串。

语法: `extract(<string>,"<regular_expression>")`

- `string`: 指定的字符串。
- `regular_expression`: 正则表达式。
- 返回类型: `list`。

示例:

```
trsgraph> MATCH (a:player)-[b:serve]-(c:team{name: "Lakers"}) \
    WHERE a.player.age > 45 \
    RETURN extract(a.player.name, "\\w+") AS result;
+-----+
| result |
+-----+
| ["Shaquille", "O", "Neal"] |
+-----+

trsgraph> MATCH (a:player)-[b:serve]-(c:team{name: "Lakers"}) \
    WHERE a.player.age > 45 \
    RETURN extract(a.player.name, "hello") AS result;
+-----+
| result |
+-----+
| []      |
+-----+
```

json_extract() 函数

`json_extract()` 将指定 JSON 字符串转换为 `map` 类型。

语法: `extract(<string>)`

- `string`: 指定字符串, 为 JSON 格式。
- 返回类型: `map`。



- 目前仅支持 `Bool`、`Double`、`Int`、`String` 和 `NULL` 类型数据。
- 仅支持深度为 1 的 `Map` 嵌套, 如果嵌套深度为 2 及以上, 嵌套项保留为空。

示例:

```
trsgraph> YIELD json_extract('{ "a": 1, "b": {}, "c": { "d": true } }') AS result;
+-----+
| result |
+-----+
| {a: 1, b: {}, c: {d: true}} |
+-----+
```

最后更新: 2025年12月19日

4.4.4 内置日期时间函数

TRSGraph 支持以下内置日期时间函数。

函数	说明
int now()	根据当前系统返回当前时间戳。
timestamp timestamp()	根据当前系统返回当前时间戳。
date date()	根据当前系统返回当前日期（UTC 时间）。
time time()	根据当前系统返回当前时间（UTC 时间）。
datetime datetime()	根据当前系统返回当前日期和时间（UTC 时间）。
map duration()	持续时间。可以用于对指定时间进行计算。

详细信息参见[日期和时间类型](#)。

示例

```
trsgraph> RETURN now(), timestamp(), date(), time(), datetime();
+-----+-----+-----+-----+-----+
| now() | timestamp() | date() | time() | datetime() |
+-----+-----+-----+-----+-----+
| 1640057560 | 1640057560 | 2021-12-21 | 03:32:40.351000 | 2021-12-21T03:32:40.351000 |
+-----+-----+-----+-----+-----+
```

最后更新: 2025年12月19日

4.4.5 Schema 相关函数

本文介绍 TRSGraph 支持的 Schema 相关的函数。

Schema 相关的函数分为两类：

- 适用于原生 nGQL 语句
- 适用于 openCypher 兼容语句

原生 nGQL 语句适用

原生 nGQL 语句的 `YIELD` 和 `WHERE` 子句中可以使用如下介绍的函数。



由于 `vertex`、`edge`、`vertices`、`edges`、`path` 属于关键字，使用时需要用 `AS <alias>` 设置别名才能正常使用。例如 `GO FROM "player100" OVER follow YIELD edge AS e;`

ID(VERTEX)

`id(vertex)` 返回点 ID。

语法： `id(vertex)`

- 返回类型：和点 ID 的类型保持一致。

示例：

```
trsgraph> LOOKUP ON player WHERE player.age > 45 YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player144" |
| "player140" |
+-----+
```

PROPERTIES(VERTEX)

`properties(vertex)` 返回点的所有属性。

语法： `properties(vertex)`

- 返回类型：map。

示例：

```
trsgraph> LOOKUP ON player WHERE player.age > 45 \
YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 47, name: "Shaquille O'Neal"} |
| {age: 46, name: "Grant Hill"} |
+-----+
```

用户也可以使用属性引用符（`$^` 和 `$$`）替代函数 `properties()` 中的 `vertex` 参数来获取点的所有属性。

- `$^` 表示探索开始时的点数据。例如 `GO FROM "player100" OVER follow reversely YIELD properties($^)` 中，`$^` 指 `player100` 这个点。
- `$$` 表示探索结束的终点数据。

`properties($^)` 和 `properties($$)` 一般用于 `GO` 语句中。更多信息，请参见属性引用符。



Caution

用户可以通过 `properties().<property_name>` 来获取点的指定属性。但是不建议使用这种方式获取指定属性，因为 `properties()` 函数返回所有属性，这样会降低查询性能。

PROPERTIES(EDGE)

`properties(edge)` 返回边的所有属性。

语法: `properties(edge)`

- 返回类型: `map`。

示例:

```
trsgraph> GO FROM "player100" OVER follow \
  YIELD properties(edge);
+-----+
| properties(EDGE) |
+-----+
| {degree: 95}      |
| {degree: 95}      |
+-----+
```



Warning

用户可以通过 `properties(edge).<property_name>` 来获取边的指定属性。但是不建议使用这种方式获取指定属性，因为 `properties(edge)` 函数返回边的所有属性，这样会降低查询性能。

TYPE(EDGE)

`type(edge)` 返回边的 Edge type。

语法: `type(edge)`

- 返回类型: `string`。

示例:

```
trsgraph> GO FROM "player100" OVER follow \
  YIELD src(edge), dst(edge), type(edge), rank(edge);
+-----+-----+-----+-----+
| src(EDGE) | dst(EDGE) | type(EDGE) | rank(EDGE) |
+-----+-----+-----+-----+
| "player100" | "player101" | "follow" | 0 |
| "player100" | "player125" | "follow" | 0 |
+-----+-----+-----+-----+
```

SRC(EDGE)

`src(edge)` 返回边的起始点 ID。

语法: `src(edge)`

- 返回类型: 和点 ID 的类型保持一致。

示例:

```
trsgraph> GO FROM "player100" OVER follow \
  YIELD src(edge), dst(edge);
+-----+-----+
| src(EDGE) | dst(EDGE) |
+-----+-----+
| "player100" | "player101" |
| "player100" | "player125" |
+-----+-----+
```



`src(edge)` 和 `properties($^)` 查找起始点的语义不同。`src(edge)` 始终表示图数据库中边的起始点 ID，而 `properties($^)` 表示探索开始时的点数据，例如示例中 `GO FROM "player100"` 中 `player100` 这个点的数据。

DST(EDGE)

`dst(edge)` 返回边的目的点 ID。

语法: `dst(edge)`

- 返回类型: 和点 ID 的类型保持一致。

示例:

```
trsgaph> GO FROM "player100" OVER follow \
        YIELD src(edge), dst(edge);
+-----+-----+
| src(EDGE) | dst(EDGE) |
+-----+-----+
| "player100" | "player101" |
| "player100" | "player125" |
+-----+-----+
```



`dst(edge)` 始终表示图数据库中边的目的点 ID。

RANK(EDGE)

`rank(edge)` 返回边的 rank。

语法: `rank(edge)`

- 返回类型: int。

示例:

```
trsgaph> GO FROM "player100" OVER follow \
        YIELD src(edge), dst(edge), rank(edge);
+-----+-----+-----+
| src(EDGE) | dst(EDGE) | rank(EDGE) |
+-----+-----+-----+
| "player100" | "player101" | 0 |
| "player100" | "player125" | 0 |
+-----+-----+-----+
```

VERTEX

`vertex` 返回点的信息。包括点 ID、Tag、属性和值。需要用 `AS <alias>` 设置别名。

语法: `vertex`

示例:

```
trsgaph> LOOKUP ON player WHERE player.age > 45 YIELD vertex AS v;
+-----+
| v |
+-----+
| ("player144" :player{age: 47, name: "Shaquille O'Neal"}) |
| ("player140" :player{age: 46, name: "Grant Hill"}) |
+-----+
```

EDGE

`edge` 返回边的信息。包括 Edge type、起始点 ID、目的点 ID、rank、属性和值。需要用 `AS <alias>` 设置别名。

语法: `edge`

示例：

```
trsgraph> GO FROM "player100" OVER follow YIELD edge AS e;
+-----+
| e      |
+-----+
| [[:follow "player100"->"player101" @0 {degree: 95}]] |
| [[:follow "player100"->"player125" @0 {degree: 95}]] |
+-----+
```

VERTICES

vertices 返回子图中的点的信息。详情参见 [GET SUBGRAPH](#)。

EDGES

edges 返回子图中的边的信息。详情参见 [GET SUBGRAPH](#)。

PATH

path 返回路径信息。详情参见 [FIND PATH](#)。

openCypher 兼容语句适用

openCypher 兼容语句的 [RETURN](#) 和 [WHERE](#) 子句中可以使用如下介绍的函数。

ID()

id() 返回点 ID。

语法： `id(<vertex>)`

- 返回类型：和点 ID 的类型保持一致。

示例：

```
trsgraph> MATCH (v:player) RETURN id(v);
+-----+
| id(v)      |
+-----+
| "player129" |
| "player115" |
| "player106" |
| "player102" |
| ...        |
+-----+
```

TAGS() 和 LABELS()

tags() 和 labels() 返回点的 Tag。

语法： `tags(<vertex>)`、 `labels(<vertex>)`

- 返回类型：list。

示例：

```
trsgraph> MATCH (v) WHERE id(v) == "player100" \
RETURN tags(v);
+-----+
| tags(v) |
+-----+
| ["player"] |
+-----+
```

PROPERTIES()

properties() 返回点或边的所有属性。

语法： `properties(<vertex_or_edge>)`

- 返回类型：map。

示例：

```

trsgraph> MATCH (v:player)-[e:follow]-() RETURN properties(v),properties(e);
+-----+-----+
| properties(v) | properties(e) |
+-----+-----+
| {age: 31, name: "Stephen Curry"} | {degree: 90} |
| {age: 47, name: "Shaquille O'Neal"} | {degree: 100} |
| {age: 34, name: "LeBron James"} | {degree: 13} |
...

```

TYPE()

type() 返回边的 Edge type。

语法: `type(<edge>)`

- 返回类型: string。

示例:

```

trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->() \
RETURN type(e);
+-----+
| type(e) |
+-----+
| "serve" |
| "follow" |
| "follow" |
+-----+

```

TYPEID()

typeid() 返回边的 Edge type 的内部 ID 值, 可以用正负来判断方向。

语法: `typeid(<edge>)`

- 返回类型: int。

示例:

```

trsgraph> MATCH (v:player)-[e:follow]- (v2) RETURN e,typeid(e), \
CASE WHEN typeid(e) > 0 \
THEN "正向" ELSE "反向" END AS direction \
LIMIT 5;
+-----+-----+-----+
| e | typeid(e) | direction |
+-----+-----+-----+
| [:follow "player127"->"player114" @0 {degree: 90}] | 5 | "正向" |
| [:follow "player127"->"player148" @0 {degree: 70}] | 5 | "正向" |
| [:follow "player148"->"player127" @0 {degree: 80}] | -5 | "反向" |
| [:follow "player147"->"player136" @0 {degree: 90}] | 5 | "正向" |
| [:follow "player136"->"player147" @0 {degree: 90}] | -5 | "反向" |
+-----+-----+-----+

```

SRC()

src() 返回边的起始点 ID。

语法: `src(<edge>)`

- 返回类型: 和点 ID 的类型保持一致。

示例:

```

trsgraph> MATCH ()-[e]->(v:player{name:"Tim Duncan"}) \
RETURN src(e);
+-----+
| src(e) |
+-----+
| "player125" |
| "player113" |
| "player102" |
...

```

DST()

dst() 返回边的目的点 ID。

语法: `dst(<edge>)`

- 返回类型: 和点 ID 的类型保持一致。

示例:

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->() \
            RETURN dst(e);
+-----+
| dst(e) |
+-----+
| "team204" |
| "player101" |
| "player125" |
+-----+
```

STARTNODE()

`startNode()` 获取一条路径并返回它的起始点信息, 包括点 ID、Tag、属性和值。

语法: `startNode(<path>)`

示例:

```
trsgraph> MATCH p = (a :player {name : "Tim Duncan"})-[r:serve]-(t) \
            RETURN startNode(p);
+-----+
| startNode(p) |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+
```

ENDNODE()

`endNode()` 获取一条路径并返回它的目的点信息, 包括点 ID、Tag、属性和值。

语法: `endNode(<path>)`

示例:

```
trsgraph> MATCH p = (a :player {name : "Tim Duncan"})-[r:serve]-(t) \
            RETURN endNode(p);
+-----+
| endNode(p) |
+-----+
| ("team204" :team{name: "Spurs"}) |
+-----+
```

RANK()

`rank()` 返回边的 rank。

语法: `rank(<edge>)`

- 返回类型: `int`。

示例:

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->() \
            RETURN rank(e);
+-----+
| rank(e) |
+-----+
| 0 |
| 0 |
| 0 |
+-----+
```

最后更新: 2025年12月19日

4.4.6 列表函数

本文介绍 TRSGraph 支持的列表（List）函数。部分列表函数在原生 nGQL 语句和 openCypher 兼容语句中的语法不同。

注意事项

- 和 SQL 一样，nGQL 的字符索引（位置）从 1 开始。但是 C 语言的字符索引是从 0 开始的。

通用

RANGE()

`range()` 返回指定整数范围 `[start,end]` 内固定步长的列表。

语法: `range(start, end [, step])`

- `step`: 可选参数。步长。默认为 1。
- 返回类型: `list`。

示例:

```
trsgraph> RETURN range(1,9,2);
+-----+
| range(1,9,2) |
+-----+
| [1, 3, 5, 7, 9] |
+-----+
```

REVERSE()

`reverse()` 返回将原列表逆序排列的新列表。

语法: `reverse(<list>)`

- 返回类型: `list`。

示例:

```
trsgraph> WITH [NULL, 4923, 'abc', 521, 487] AS ids \
RETURN reverse(ids);
+-----+
| reverse(ids) |
+-----+
| [487, 521, "abc", 4923, __NULL__] |
+-----+
```

TAIL()

`tail()` 返回不包含原列表第一个元素的新列表。

语法: `tail(<list>)`

- 返回类型: `list`。

示例:

```
trsgraph> WITH [NULL, 4923, 'abc', 521, 487] AS ids \
RETURN tail(ids);
+-----+
| tail(ids) |
+-----+
| [4923, "abc", 521, 487] |
+-----+
```

HEAD()

`head()` 返回列表的第一个元素。

语法: `head(<list>)`

- 返回类型: 与原列表内的元素类型相同。

示例:

```
trsgraph> WITH [NULL, 4923, 'abc', 521, 487] AS ids \
RETURN head(ids);
+-----+
| head(ids) |
+-----+
|  NULL    |
+-----+
```

LAST()

`last()` 返回列表的最后一个元素。

语法: `last(<list>)`

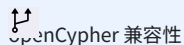
- 返回类型: 与原列表内的元素类型相同。

示例:

```
trsgraph> WITH [NULL, 4923, 'abc', 521, 487] AS ids \
RETURN last(ids);
+-----+
| last(ids) |
+-----+
|  487      |
+-----+
```

REDUCE()

`reduce()` 将表达式逐个应用于列表中的元素，然后和累加器中的当前结果累加，最后返回完整结果。该函数将遍历给定列表中的每个元素 `e`，在 `e` 上运行表达式并和累加器的当前结果累加，将新的结果存储在累加器中。这个函数类似于函数式语言（如 Lisp 和 Scala）中的 `fold` 或 `reduce` 方法。



openCypher 兼容性

在 openCypher 中，`reduce()` 函数没有定义。nGQL 使用了 Cypher 方式实现 `reduce()` 函数。

语法: `reduce(<accumulator> = <initial>, <variable> IN <list> | <expression>)`

- `accumulator`: 在遍历列表时保存累加结果。
- `initial`: 为 `accumulator` 提供初始值的表达式或值。
- `variable`: 为列表引入一个变量，决定使用列表中的哪个元素。
- `list`: 列表或列表表达式。
- `expression`: 该表达式将对列表中的每个元素运行一次，并将结果累加至 `accumulator`。
- 返回类型: 取决于提供的参数，以及表达式的语义。

示例:

```
trsgraph> RETURN reduce(totalNum = -4 * 5, n IN [1, 2] | totalNum + n * 2) AS r;
+-----+
| r      |
+-----+
| -14    |
+-----+

trsgraph> MATCH p = (n:player{name:"LeBron James"})<-[:follow]-(m) \
RETURN nodes(p)[0].player.age AS src1, nodes(p)[1].player.age AS dst2, \
       reduce(totalAge = 100, n IN nodes(p) | totalAge + n.player.age) AS sum;
+-----+-----+-----+
| src1 | dst2 | sum |
+-----+-----+-----+
| 34    | 31    | 165 |
| 34    | 29    | 163 |
| 34    | 33    | 167 |
```

```
| 34 | 26 | 160 |
| 34 | 34 | 168 |
| 34 | 37 | 171 |
+-----+
trsgraph> LOOKUP ON player WHERE player.name == "Tony Parker" YIELD id(vertex) AS VertexID \
| GO FROM $-.VertexID over follow \
WHERE properties(edge).degree != reduce(totalNum = 5, n IN range(1, 3) | properties($$).age + totalNum + n) \
YIELD properties($$).name AS id, properties($$).age AS age, properties(edge).degree AS degree;
+-----+
| id | age | degree |
+-----+
| "Tim Duncan" | 42 | 95 |
| "LaMarcus Aldridge" | 33 | 90 |
| "Manu Ginobili" | 41 | 95 |
+-----+
```

原生 nGQL 语句适用

KEYS()

keys() 返回一个列表，包含字符串形式的点、边的所有属性。

语法: keys({vertex | edge})

- 返回类型: list。

示例:

```
trsgraph> LOOKUP ON player \
WHERE player.age > 45 \
YIELD keys(vertex);
+-----+
| keys (VERTEX) |
+-----+
| ["age", "name"] |
| ["age", "name"] |
+-----+
```

LABELS()

labels() 返回点的 Tag 列表。

语法: labels(verte)

- 返回类型: list。

示例:

```
trsgraph> FETCH PROP ON * "player101", "player102", "team204" \
YIELD labels(vertex);
+-----+
| labels (VERTEX) |
+-----+
| ["player"] |
| ["player"] |
| ["team"] |
+-----+
```

openCypher 兼容语句适用

KEYS()

keys() 返回一个列表，包含字符串形式的点、边或映射的所有属性。

语法: keys(<vertex_or_edge>)

- 返回类型: list。

示例:

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->() \
RETURN keys(e);
+-----+
| keys (e) |
+-----+
| ["end_year", "start_year"] |
```

```
| ["degree"] |
| ["degree"] |
+-----+
```

LABELS()

labels() 返回点的 Tag 列表。

语法: labels(<vertex>)

- 返回类型: list。

示例:

```
trsgraph> MATCH (v)-[e:serve]->() \
            WHERE id(v)=="player100" \
            RETURN labels(v);
+-----+
| labels(v) |
+-----+
| ["player"] |
+-----+
```

NODES()

nodes() 返回路径中所有点的列表。包括点 ID、Tag、属性和值。

语法: nodes(<path>)

- 返回类型: list。

示例:

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-->(v2) \
            RETURN nodes(p);
+-----+
| nodes(p) |
+-----+
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"team204" :team{name: "Spurs"}}] |
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"player101" :player{age: 36, name: "Tony Parker"}}] |
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"player125" :player{age: 41, name: "Manu Ginobili"}}] |
+-----+
```

RELATIONSHIPS()

relationships() 返回路径中所有关系的列表。

语法: relationships(<path>)

- 返回类型: list。

示例:

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-->(v2) \
            RETURN relationships(p);
+-----+
| relationships(p) |
+-----+
| [{"serve" "player100"->"team204" @0 {end_year: 2016, start_year: 1997}}] |
| [{"follow" "player100"->"player101" @0 {degree: 95}}] |
| [{"follow" "player100"->"player125" @0 {degree: 95}}] |
+-----+
```

4.4.7 类型转换函数

本文介绍 TRSGraph 支持的类型转换函数。

toBoolean()

toBoolean() 将字符串转换为布尔。

语法: `toBoolean(<value>)`

- 返回类型: bool。

示例:

```
trsgraph> UNWIND [true, false, 'true', 'false', NULL] AS b \
  RETURN toBoolean(b) AS b;
+-----+
| b      |
+-----+
| true   |
| false  |
| true   |
| false  |
| __NULL__ |
+-----+
```

toFloat()

toFloat() 将整数或字符串转换为浮点数。

语法: `toFloat(<value>)`

- 返回类型: float。

示例:

```
trsgraph> RETURN toFloat(1), toFloat('1.3'), toFloat('1e3'), toFloat('not a number');
+-----+-----+-----+-----+
| toFloat(1) | toFloat("1.3") | toFloat("1e3") | toFloat("not a number") |
+-----+-----+-----+-----+
| 1.0        | 1.3            | 1000.0         | __NULL__                |
+-----+-----+-----+-----+
```

toString()

toString() 将任意非复合数据类型数据转换为字符串类型。

语法: `toString(<value>)`

- 返回类型: string。

示例:

```
trsgraph> RETURN toString(9669) AS int2str, toString(null) AS null2str;
+-----+-----+
| int2str | null2str |
+-----+-----+
| "9669"  | __NULL__ |
+-----+-----+
```

toInteger()

toInteger() 将浮点或字符串转换为整数。

语法: `toInteger(<value>)`

- 返回类型: int。

示例：

```
trsgraph> RETURN toInteger(1), toInteger('1'), toInteger('1e3'), toInteger('not a number');
+-----+
| toInteger(1) | toInteger("1") | toInteger("1e3") | toInteger("not a number") |
+-----+
| 1           | 1           | 1000           | __NULL__           |
+-----+
```

toSet()

toSet() 将列表或集合转换为集合。

语法： `toSet(<value>)`

- 返回类型： `set`。

示例：

```
trsgraph> RETURN toSet(list[1,2,3,1,2]) AS list2set;
+-----+
| list2set |
+-----+
| {3, 1, 2} |
+-----+
```

hash()

hash() 返回参数的哈希值。其参数可以是数字、字符串、列表、布尔值、NULL 等类型的值，或者计算结果为这些类型的表达式。

hash() 函数采用 MurmurHash2 算法，种子（seed）为 0xc70f6907UI。用户可以在 [MurmurHash2.h](#) 中查看其源代码。

在 Java 中的调用方式如下：

```
MurmurHash2.hash64("to_be_hashed".getBytes(),"to_be_hashed".getBytes().length, 0xc70f6907)
```

语法： `hash(<string>)`

- 返回类型： `int`。

示例：

```
trsgraph> RETURN hash("abcde");
+-----+
| hash("abcde") |
+-----+
| 811036730794841393 |
+-----+

trsgraph> YIELD hash([1,2,3]);
+-----+
| hash([1,2,3]) |
+-----+
| 11093822460243 |
+-----+

trsgraph> YIELD hash(NULL);
+-----+
| hash(NULL) |
+-----+
| -1 |
+-----+

trsgraph> YIELD hash(toLower("HELLO TRSGRAPH"));
+-----+
| hash(toLower("HELLO TRSGRAPH")) |
+-----+
| -8481157362655072082 |
+-----+
```

4.4.8 条件表达式函数

本文介绍 TRSGraph 支持的条件表达式函数。

CASE

`CASE` 表达式使用条件来过滤传参。和 openCypher 一样，nGQL 提供两种形式的 `CASE` 表达式：简单形式和通用形式。

`CASE` 表达式会遍历所有条件，并在满足第一个条件时停止读取后续条件，然后返回结果。如果不满足任何条件，将通过 `ELSE` 子句返回结果。如果没有 `ELSE` 子句且不满足任何条件，则返回 `NULL`。

简单形式

- 语法

```
CASE <comparer>
WHEN <value> THEN <result>
[WHEN ...]
[ELSE <default>]
END
```



Caution

`CASE` 表达式一定要用 `END` 结尾。

参数	说明
<code>comparer</code>	用于与 <code>value</code> 进行比较的值或者有效表达式。
<code>value</code>	和 <code>comparer</code> 进行比较，如果匹配，则满足此条件。
<code>result</code>	如果 <code>value</code> 匹配 <code>comparer</code> ，则返回该 <code>result</code> 。
<code>default</code>	如果没有条件匹配，则返回该 <code>default</code> 。

- 示例

```
trsgraph> RETURN \
  CASE 2+3 \
  WHEN 4 THEN 0 \
  WHEN 5 THEN 1 \
  ELSE -1 \
  END \
  AS result;
+-----+
| result |
+-----+
| 1      |
+-----+

trsgraph> GO FROM "player100" OVER follow \
  YIELD properties($$).name AS Name, \
  CASE properties($$).age > 35 \
  WHEN true THEN "Yes" \
  WHEN false THEN "No" \
  ELSE "Nah" \
  END \
  AS Age_above_35;
+-----+-----+
| Name      | Age_above_35 |
+-----+-----+
| "Tony Parker" | "Yes"        |
| "Manu Ginobili" | "Yes"        |
+-----+-----+
```

通用形式

- 语法


```
CASE
WHEN <condition> THEN <result>
[WHEN ...]
[ELSE <default>]
END
```

参数	说明
<code>condition</code>	如果条件 <code>condition</code> 为 <code>true</code> ，表示满足此条件。
<code>result</code>	<code>condition</code> 为 <code>true</code> ，则返回此 <code>result</code> 。
<code>default</code>	如果没有条件匹配，则返回该 <code>default</code> 。

• 示例

```
trsgraph> YIELD \
  CASE WHEN 4 > 5 THEN 0 \
  WHEN 3+4==7 THEN 1 \
  ELSE 2 \
  END \
  AS result;
+-----+
| result |
+-----+
| 1      |
+-----+
```

```
trsgraph> MATCH (v:player) WHERE v.player.age > 30 \
  RETURN v.player.name AS Name, \
  CASE \
  WHEN v.player.name STARTS WITH "T" THEN "Yes" \
  ELSE "No" \
  END \
  AS Starts_with_T;
+-----+-----+
| Name           | Starts_with_T |
+-----+-----+
| "Tim Duncan"   | "Yes"         |
| "LaMarcus Aldridge" | "No"          |
| "Tony Parker"  | "Yes"         |
+-----+-----+
```

简单形式和通用形式的区别

为了避免误用简单形式和通用形式，用户需要了解它们的差异。请参见如下示例：

```
trsgraph> GO FROM "player100" OVER follow \
  YIELD properties($$).name AS Name, properties($$).age AS Age, \
  CASE properties($$).age \
  WHEN properties($$).age > 35 THEN "Yes" \
  ELSE "No" \
  END \
  AS Age_above_35;
+-----+-----+-----+
| Name           | Age | Age_above_35 |
+-----+-----+-----+
| "Tony Parker"  | 36  | "No"          |
| "Manu Ginobili" | 41  | "No"          |
+-----+-----+-----+
```

示例本意为当玩家年龄大于 35 时输出 `Yes`。但是查看输出结果，年龄为 36 时输出的却是 `No`。

这是因为查询使用了简单形式的 `CASE` 表达式，比较对象是 `$$player.age` 和 `$$player.age > 35`。当年龄为 36 时：

- `$$player.age` 的值为 `36`，数据类型为 `int`。
- `$$player.age > 35` 的值为 `true`，数据类型为 `boolean`。

这两种数据类型无法匹配，不满足条件，因此返回 `No`。

`coalesce()`

`coalesce()` 返回所有表达式中第一个非空元素。

语法: `coalesce(<expression_1>[,<expression_2>...])`

- 返回类型: 与原元素类型相同。

示例:

```
trsgaph> RETURN coalesce(null,[1,2,3]) as result;
+-----+
| result |
+-----+
| [1, 2, 3] |
+-----+

trsgaph> RETURN coalesce(null) as result;
+-----+
| result |
+-----+
| __NULL__ |
+-----+
```

最后更新: 2025年12月19日

4.4.9 谓词函数

谓词函数只返回 `true` 或 `false`，通常用于 `WHERE` 子句中。

TRSGraph 支持以下谓词函数。

函数	说明
<code>exists()</code>	如果指定的属性在点、边或映射中存在，则返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>any()</code>	如果指定的谓词适用于列表中的至少一个元素，则返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>all()</code>	如果指定的谓词适用于列表中的每个元素，则返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>none()</code>	如果指定的谓词不适用于列表中的任何一个元素，则返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>single()</code>	如果指定的谓词适用于列表中的唯一一个元素，则返回 <code>true</code> ，否则返回 <code>false</code> 。



Note

如果列表为空，或者列表中的所有元素都为空，则返回 `NULL`。



Compatibility

在 `openCypher` 中只定义了函数 `exists()`，其他几个函数依赖于具体实现。

语法

```
<predicate>(<variable> IN <list> WHERE <condition>)
```

示例

```
trsgraph> RETURN any(n IN [1, 2, 3, 4, 5, NULL] \
    WHERE n > 2) AS r;
+-----+
| r      |
+-----+
| true   |
+-----+

trsgraph> RETURN single(n IN range(1, 5) \
    WHERE n == 3) AS r;
+-----+
| r      |
+-----+
| true   |
+-----+

trsgraph> RETURN none(n IN range(1, 3) \
    WHERE n == 0) AS r;
+-----+
| r      |
+-----+
| true   |
+-----+

trsgraph> WITH [1, 2, 3, 4, 5, NULL] AS a \
    RETURN any(n IN a WHERE n > 2);
+-----+
| any(n IN a WHERE (n>2)) |
+-----+
| true                     |
+-----+

trsgraph> MATCH p = (n:player{name:"LeBron James"})<-[:follow]-(m) \
    RETURN nodes(p)[0].player.name AS n1, nodes(p)[1].player.name AS n2, \
    all(n IN nodes(p) WHERE n.player.name NOT STARTS WITH "D") AS b;
+-----+
| n1      | n2      | b      |
+-----+
| LeBron | LeBron | true   |
+-----+
```

```
| "LeBron James" | "Danny Green" | false |
| "LeBron James" | "Dejounte Murray" | false |
| "LeBron James" | "Chris Paul" | true |
| "LeBron James" | "Kyrie Irving" | true |
| "LeBron James" | "Carmelo Anthony" | true |
| "LeBron James" | "Dwyane Wade" | false |
+-----+

trsgraph> MATCH p = (n:player{name:"LeBron James"})-[:follow]->(m) \
RETURN single(n IN nodes(p) WHERE n.player.age > 40) AS b;
+-----+
| b |
+-----+
| true |
+-----+

trsgraph> MATCH (n:player) \
RETURN exists(n.player.id), n IS NOT NULL;
+-----+
| exists(n.player.id) | n IS NOT NULL |
+-----+
| false | true |
...

trsgraph> MATCH (n:player) \
WHERE exists(n['name']) \
RETURN n;
+-----+
| n |
+-----+
| ("player105" :player{age: 31, name: "Danny Green"}) |
| ("player109" :player{age: 34, name: "Tiago Splitter"}) |
| ("player111" :player{age: 38, name: "David West"}) |
...

```

最后更新: 2025年12月19日

4.4.10 geo 函数

geo 函数用于生成地理空间（GEOGRAPHY）数据类型的值或对其执行操作。

关于地理空间数据类型说明请参见[地理空间](#)。

函数说明

函数	返回类型	说明
ST_Point(longitude, latitude)	GEOGRAPHY	创建包含一个点的地理空间。
ST_GeogFromText(wkt_string)	GEOGRAPHY	返回与传入的 WKT 字符串形式相对应的 GEOGRAPHY。
ST_ASText(geography)	STRING	返回传入的 GEOGRAPHY 的 WKT 字符串形式。
ST_Centroid(geography)	GEOGRAPHY	以单点 GEOGRAPHY 的形式返回传入的 GEOGRAPHY 的形心。
ST_IsValid(geography)	BOOL	返回传入的 GEOGRAPHY 是否有效。
ST_Intersects(geography_1, geography_2)	BOOL	返回传入的两个 GEOGRAPHY 是否有交集。
ST_Covers(geography_1, geography_2)	BOOL	返回 geography_1 是否完全包含 geography_2。如果 geography_2 中没有位于 geography_1 外部的点，返回 True。
ST_CoveredBy(geography_1, geography_2)	BOOL	返回 geography_2 是否完全包含 geography_1。如果 geography_1 中没有位于 geography_2 外部的点，返回 True。
ST_DWithin(geography_1, geography_2, distance)	BOOL	如果 geography_1 中至少有一个点与 geography_2 中的一个点的距离小于或等于 distance 参数（以米为单位）指定的距离，则返回 True。
ST_Distance(geography_1, geography_2)	FLOAT	返回两个非空 GEOGRAPHY 之间的最短距离（以米为单位）。
S2_CellIdFromPoint(point_geography)	INT	返回覆盖点 GEOGRAPHY 的 S2 单元 ID。
S2_CoveringCellIds(geography)	ARRAY<INT64>	返回覆盖传入的 GEOGRAPHY 的 S2 单元 ID 的数组。

示例

```

trsgraph> RETURN ST_ASText(ST_Point(1,1));
+-----+
| ST_ASText(ST_Point(1,1)) |
+-----+
| "POINT(1 1)"           |
+-----+

trsgraph> RETURN ST_ASText(ST_GeogFromText("POINT(3 8)"));
+-----+
| ST_ASText(ST_GeogFromText("POINT(3 8)")) |
+-----+
| "POINT(3 8)"           |
+-----+

trsgraph> RETURN ST_ASTEXT(ST_Centroid(ST_GeogFromText("LineString(0 1,1 0)")));
+-----+
| ST_ASTEXT(ST_Centroid(ST_GeogFromText("LineString(0 1,1 0)"))) |
+-----+
| "POINT(0.5000380800773782 0.5000190382261059)" |
+-----+

trsgraph> RETURN ST_IsValid(ST_GeogFromText("POINT(3 8)"));
+-----+
| ST_IsValid(ST_GeogFromText("POINT(3 8)")) |
+-----+
| true                                     |
+-----+

trsgraph> RETURN ST_Intersects(ST_GeogFromText("LineString(0 1,1 0)"),ST_GeogFromText("LineString(0 0,1 1)"));
+-----+
| ST_Intersects(ST_GeogFromText("LineString(0 1,1 0)"),ST_GeogFromText("LineString(0 0,1 1)")) |
+-----+
| true                                     |
+-----+

trsgraph> RETURN ST_Covers(ST_GeogFromText("POLYGON((0 0,10 0,10 10,0 10,0 0))"),ST_Point(1,2));

```

```

+-----+
| ST_Covers(ST_GeogFromText("POLYGON((0 0,10 0,10 10,0 10,0 0))"),ST_Point(1,2)) |
+-----+
| true |
+-----+

trsgraph> RETURN ST_CoveredBy(ST_Point(1,2),ST_GeogFromText("POLYGON((0 0,10 0,10 10,0 10,0 0))"));
+-----+
| ST_CoveredBy(ST_Point(1,2),ST_GeogFromText("POLYGON((0 0,10 0,10 10,0 10,0 0))")) |
+-----+
| true |
+-----+

trsgraph> RETURN ST_dwithin(ST_GeogFromText("Point(0 0)"),ST_GeogFromText("Point(10 10)"),20000000000.0);
+-----+
| ST_dwithin(ST_GeogFromText("Point(0 0)"),ST_GeogFromText("Point(10 10)"),20000000000) |
+-----+
| true |
+-----+

trsgraph> RETURN ST_Distance(ST_GeogFromText("Point(0 0)"),ST_GeogFromText("Point(10 10)"));
+-----+
| ST_Distance(ST_GeogFromText("Point(0 0)"),ST_GeogFromText("Point(10 10)")) |
+-----+
| 1.5685230187677438e+06 |
+-----+

trsgraph> RETURN S2_CellIdFromPoint(ST_GeogFromText("Point(1 1)"));
+-----+
| S2_CellIdFromPoint(ST_GeogFromText("Point(1 1)")) |
+-----+
| 1153277837650709461 |
+-----+

trsgraph> RETURN S2_CoveringCellIds(ST_GeogFromText("POLYGON((0 1, 1 2, 2 3, 0 1))"));
+-----+
+
| S2_CoveringCellIds(ST_GeogFromText("POLYGON((0 1, 1 2, 2 3, 0 1))")) |
+-----+
+
| [1152391494368201343, 1153466862374223872, 1153554823304445952, 1153836298281156608, 1153959443583467520, 1154240918560178176, 1160503736791990272, 1160591697722212352] |
+-----+
+

```

最后更新: 2025年12月19日

说明：

- 必需预先定义索引。
- 必需指定点或边所属的标签或边类型。
- 使用 `YIELD` 子句指定返回的 ID。

示例：

```
LOOKUP ON player WHERE player.name == "Tony Parker" YIELD id(vertex);
```

更多信息，请参见 [LOOKUP ON](#)。

GO

用法：用于基于给定的点进行图遍历，按需返回起始点、边或目标点的信息。可以指定遍历的深度、边的类型、方向等。

场景：复杂的图遍历，比如找到某个点的朋友、朋友的朋友等。

说明：

- 结合 [属性引用符](#) (`$^` 和 `$$`) 来返回起始点或目标点的属性，例如 `YIELD $^.player.name`。
- 结合 [函数](#) `properties($^)` 和 `properties($$)` 来返回起始点或目标点的所有属性；或者在函数中指定属性名，来返回指定的属性，例如 `YIELD properties($^).name`。
- 结合 [函数](#) `src(edge)` 和 `dst(edge)` 来返回边的起始点或目标点 ID，例如 `YIELD src(edge)`。

示例：

```
GO 3 STEPS FROM "player102" OVER follow YIELD dst(edge);
```

更多信息，请参见 [GO](#)。

MATCH

用法：用于执行复杂的图模式匹配查询。

场景：复杂的图模式匹配时使用，比如寻找满足特定模式的点和边的组合。

说明：

MATCH 语句兼容 OpenCypher 的语法，但是有一些差异：

- 使用 `==` 表达相等判断而不是 `=`，例如 `WHERE player.name == "Tony Parker"`。
- 引用点的属性时，需要指定点的标签，例如 `YIELD player.name`。
- 新增 `WHERE id(v) == "player100"` 语法。
- 必须使用 `RETURN` 子句指定返回的信息。

示例：


```
MATCH (v:player{name:"Tim Duncan"})--(v2:player) \
RETURN v2.player.name AS Name;
```

更多信息，请参见 [MATCH](#)。

FIND PATH

用法：用于查询给定的起始点和目标点之间的所有路径；或者查询路径中的点和边的属性时使用。

场景：查询两个点之间的所有路径。

说明：必须使用 `YIELD` 子句指定返回信息。

示例：

```
FIND SHORTEST PATH FROM "player102" TO "team204" OVER * YIELD path AS p;
```

-----+-----+-----+-----

			+----- 返回路径为 p 列
			+----- 经由所有类型的边的出方向探索
			+----- 从给定的起点、终点 VID
			+----- 查找最短路径

更多信息，请参见 [FIND PATH](#)。

GET SUBGRAPH

用法：提取满足特定条件的图的一部分；查询子图中的点和边的属性。

场景：分析图的局部结构或特定区域时非常有用，比如提取某个人的社交网络子图，或者提取某个区域的交通网络。

说明：必须使用 `YIELD` 子句指定返回信息。

示例：

```
GET SUBGRAPH 5 STEPS FROM "player101" YIELD VERTICES AS nodes, EDGES AS relationships;
```

-----+-----+-----+-----

	+----- 从 "player101" 开始出发	+----- 返回所有的点、边
	+----- 获取 5 步的探索	

更多信息，请参见 [GET SUBGRAPH](#)。

SHOW

SHOW 语句主要用于获取数据库的元数据信息，而不是用于获取存储在数据库中的实际数据内容。这类语句通常用于查询数据库的结构和配置，如查看现有的图空间、标签、边类型、索引等。

语句	语法	示例	说明
SHOW CHARSET	SHOW CHARSET	SHOW CHARSET	显示当前的字符集。
SHOW COLLATION	SHOW COLLATION	SHOW COLLATION	显示当前的排序规则。
SHOW CREATE SPACE	SHOW CREATE SPACE <space_name>	SHOW CREATE SPACE basketballplayer	显示指定图空间的创建语句。
SHOW CREATE TAG/EDGE	SHOW CREATE {TAG <tag_name> EDGE <edge_name>}	SHOW CREATE TAG player	显示指定 Tag/Edge type 的基本信息。
SHOW HOSTS	SHOW HOSTS [GRAPH STORAGE META]	SHOW HOSTS SHOW HOSTS GRAPH	显示 Graph、Storage、Meta 服务主机信息、版本信息。
SHOW INDEX STATUS	SHOW {TAG EDGE} INDEX STATUS	SHOW TAG INDEX STATUS	重建原生索引的作业状态，以便确定重建索引是否成功。
SHOW INDEXES	SHOW {TAG EDGE} INDEXES	SHOW TAG INDEXES	列出当前图空间内的所有 Tag 和 Edge type（包括属性）的索引。
SHOW PARTS	SHOW PARTS [<part_id>]	SHOW PARTS	显示图空间中指定分片或所有分片的信息。
SHOW ROLES	SHOW ROLES IN <space_name>	SHOW ROLES in basketballplayer	显示分配给用户的角色信息。
SHOW SNAPSHOTS	SHOW SNAPSHOTS	SHOW SNAPSHOTS	显示所有快照信息。
SHOW SPACES	SHOW SPACES	SHOW SPACES	显示现存的图空间。
SHOW STATS	SHOW STATS	SHOW STATS	显示最近 STATS 作业收集的图空间统计信息。
SHOW TAGS/ EDGES	SHOW TAGS EDGES	SHOW TAGS、SHOW EDGES	显示当前图空间内的所有 Tag/Edge type。
SHOW USERS	SHOW USERS	SHOW USERS	显示用户信息。
SHOW SESSIONS	SHOW SESSIONS	SHOW SESSIONS	显示所有会话信息。
SHOW SESSIONS	SHOW SESSION <Session_Id>	SHOW SESSION 1623304491050858	指定会话 ID 进行查看。
SHOW QUERIES	SHOW [ALL] QUERIES	SHOW QUERIES	查看当前 Session 中正在执行的查询请求信息。
SHOW META LEADER	SHOW META LEADER	SHOW META LEADER	显示当前 Meta 集群的 leader 信息。

复合查询

TRSGraph 的查询语句可以组合使用，以实现更复杂的查询。

复合语句中如需引用子查询的结果，需要为该结果设置别名，并使用管道符 | 传递给下一个子查询，同时在下一个子查询中使用 \$- 引用该结果的别名。详情参见管道符。

示例：

```
trsgraph> GO FROM "player100" OVER follow \  
YIELD dst(edge) AS dstid, properties($$).name AS Name | \  

```

```
GO FROM $-.dstid OVER follow YIELD dst(edge);
```

```
+-----+
| dst(EDGE) |
+-----+
| "player100" |
| "player102" |
| "player125" |
| "player100" |
+-----+
```

管道符 `|` 仅适用于 nQL，不适用于 OpenCypher 语句，即不能在 MATCH 语句中使用管道符。如果需要使用 MATCH 语句进行复合查询，可以使用 `WITH` 子句。

示例：

```
trsgraph> MATCH (v:player)-->(v2:player) \
          WITH DISTINCT v2 AS v2, v2.player.age AS Age \
          ORDER BY Age \
          WHERE Age<25 \
          RETURN v2.player.name AS Name, Age;
+-----+-----+
| Name           | Age |
+-----+-----+
| "Luka Doncic"  | 20  |
| "Ben Simmons"  | 22  |
| "Kristaps Porzingis" | 23  |
+-----+-----+
```

更多信息

- [nQL 语句汇总](#)

最后更新: 2025年12月19日

4.5.2 MATCH

`MATCH` 语句提供基于模式（Pattern）匹配的搜索功能，其通过定义一个或多个模式，允许在 `TRSGraph` 中查找与模式匹配的数据。在检索到匹配的数据后，用户可以使用 `RETURN` 子句将其作为结果返回。

在本文中，我们将使用名为 `basketballplayer` 的测试数据集来演示 `MATCH` 语句的使用。

语法

`MATCH` 语句的语法相较于其他查询语句（如 `GO` 和 `LOOKUP`）更具灵活性。在进行查询时，`MATCH` 语句使用的路径类型是 `trail`，这意味着点可以重复出现，但边不能重复。

`MATCH` 语法的基本结构如下：

```
MATCH <pattern> [<clause_1>] RETURN <output> [<clause_2>];
```

- `pattern`：`MATCH` 语句支持匹配一个或多个模式，多个模式之间用英文逗号（,）分隔。例如 `(a)-[]->(b), (c)-[]->(d)`。Pattern 的详细说明请参见[模式](#)。
- `clause_1`：支持 `WHERE`、`WITH`、`UNWIND`、`OPTIONAL MATCH` 子句，也可以使用 `MATCH` 作为子句。
- `output`：定义需要返回输出结果的列表名称。可以使用 `AS` 设置列表的别名。
- `clause_2`：支持 `ORDER BY`、`LIMIT` 子句。

版本兼容性

- 从 3.5.0 版本开始，`MATCH` 语句支持全表扫描，即在不使用任何索引或者过滤条件的情况下可遍历图中点或边。在此之前的版本中，`MATCH` 语句在某些情况下需要索引才能执行查询或者需要使用 `LIMIT` 限制输出结果数量。
- 从 3.0.0 版本开始，为了区别不同 Tag 的属性，返回属性时必须额外指定 Tag 名称。即从 `RETURN <变量名>.<属性名>` 改为 `RETURN <变量名>.<Tag名>.<属性名>`。

使用说明

- 尽量避免执行全表扫描，因为这可能导致查询性能下降；并且如果在进行全表扫描时内存不足，可能会导致查询失败，系统会提示报错。建议使用具有过滤条件或指定 Tag、边类型的查询，例如 `MATCH (v:player) RETURN v.player.name AS Name` 语句中的 `v:player` 和 `v.player.name`。
- 可为 Tag、Edge type 或 Tag、Edge type 的某个属性创建索引，以提高查询性能。例如，用户可以为 `player` Tag 创建索引，或者为 `player` Tag 的 `name` 属性创建索引。有关索引的使用及注意事项，请参见[使用索引必读](#)。
- 目前 `MATCH` 语句无法查询到悬挂边。

使用模式

匹配点

用户可以在一对括号中使用自定义变量来表示模式中的点。例如 `(v)`。

```
# 匹配全图空间中的任意 3 个点
trsgraph> MATCH (v) \
  RETURN v \
  LIMIT 3;
+-----+
| v |
+-----+
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
| ("player106" :player{age: 25, name: "Kyle Anderson"}) |
| ("player115" :player{age: 40, name: "Kobe Bryant"}) |
+-----+
```

匹配 TAG



版本兼容性

在 TRSGraph 3.0.0 之前，匹配 Tag 的前提是 Tag 本身有索引或者 Tag 的某个属性有索引，否则，用户无法基于该 Tag 执行 `MATCH` 语句。从 TRSGraph 3.0.0 开始，匹配 Tag 可以不创建索引，但需要使用 `LIMIT` 限制输出结果数量。从 TRSGraph 3.5.0 开始，`MATCH` 语句支持全表扫描，无需为 Tag 或 Tag 的某个属性创建索引，或者使用 `LIMIT` 限制输出结果数量，即可执行 `MATCH` 语句。

用户可以在点的右侧用 `:<tag_name>` 表示模式中的 Tag。

```
# 匹配全图空间中所有 Tag 为 player 的点
trsgraph> MATCH (v:player) \
    RETURN v;

+-----+
| v      |
+-----+
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
| ("player106" :player{age: 25, name: "Kyle Anderson"})      |
| ("player115" :player{age: 40, name: "Kobe Bryant"})        |
| ...                                                    |
```

需要匹配拥有多个 Tag 的点，可以用英文冒号 (:)。

```
# 创建包含 name 属性和 age 属性的 Tag actor。
trsgraph> CREATE TAG actor (name string, age int);
# 插入 Tag actor 到点 player100。
trsgraph> INSERT VERTEX actor(name, age) VALUES "player100":("Tim Duncan", 42);
# 匹配 Tag 为 player 和 actor 的点。
trsgraph> MATCH (v:player:actor) \
    RETURN v;

+-----+
| v      |
+-----+
| ("player100" :actor{age: 42, name: "Tim Duncan"} :player{age: 42, name: "Tim Duncan"}) |
+-----+
```

匹配点的属性

用户可以在 Tag 的右侧用 `{<prop_name>: <prop_value>}` 表示模式中点的属性。

```
# 使用属性 name 搜索匹配的点。
trsgraph> MATCH (v:player{name:"Tim Duncan"}) \
    RETURN v;

+-----+
| v      |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+
```

使用 `WHERE` 子句也可以实现相同的操作：

```
# 查找类型为 player，名字为 Tim Duncan 的点。
trsgraph> MATCH (v:player) \
    WHERE v.player.name == "Tim Duncan" \
    RETURN v;

+-----+
| v      |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+
```



onCypher 兼容性

在 openCypher 9 中，`=` 是相等运算符，在 nGQL 中，`==` 是相等运算符，`=` 是赋值运算符。

使用 `WHERE` 子句直接匹配点的属性。

```
# 匹配属性中值等于 Tim Duncan 的点。
trsgraph> MATCH (v) \
    WITH v, properties(v) as props, keys(properties(v)) as kk \
    WHERE [i in kk where props[i] == "Tim Duncan"] \
    RETURN v;

+-----+
```

```

| v
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+

# 匹配 name 属性值存在于 names 列表内的起点，并返回起点和终点的数据。
trsgraph> WITH ['Tim Duncan', 'Yao Ming'] AS names \
    MATCH (v1:player)-->(v2:player) \
    WHERE v1.player.name in names \
    RETURN v1, v2;

+-----+-----+
| v1          | v2          |
+-----+-----+
| ("player133" :player{age: 38, name: "Yao Ming"}) | ("player114" :player{age: 39, name: "Tracy McGrady"}) |
| ("player133" :player{age: 38, name: "Yao Ming"}) | ("player144" :player{age: 47, name: "Shaquille O'Neal"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | ("player125" :player{age: 41, name: "Manu Ginobili"}) |
+-----+-----+

```

匹配点 ID

用户可以使用点 ID 去匹配点。 `id()` 函数可以检索点的 ID。

```

# 查找 ID 为 "player101" 的点。（注: ID 全局唯一）。
trsgraph> MATCH (v) \
    WHERE id(v) == 'player101' \
    RETURN v;

+-----+
| v
+-----+
| ("player101" :player{age: 36, name: "Tony Parker"}) |
+-----+

```

要匹配多个点的 ID，可以用 `WHERE id(v) IN [vid_list]` 或者 `WHERE id(v) IN {vid_list}`。

```

# 查找与 `Tim Duncan` 直接相连的点，并且这些点的 ID 必须是 `player101` 或 `player102`。
trsgraph> MATCH (v:player { name: 'Tim Duncan' })--(v2) \
    WHERE id(v2) IN ["player101", "player102"] \
    RETURN v2;

+-----+
| v2
+-----+
| ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
+-----+

# 查找 ID 为 player100 和 player101 的点，并返回 name 属性。
trsgraph> MATCH (v) WHERE id(v) IN {"player100", "player101"} \
    RETURN v.player.name AS name;

+-----+
| name
+-----+
| "Tony Parker" |
| "Tim Duncan" |
+-----+

```

匹配连接的点

用户可以使用 `--` 符号表示两个方向的边，并匹配这些边连接的点。



版本兼容性

在 nGQL 1.x 中，`--` 符号用于行内注释，从 nGQL 2.x 起，`--` 符号表示出边或入边，不再用于注释。

```

# name 属性值为 Tim Duncan 的点为 v，与 v 相连接的点为 v2，查找 v2 并返回其 name 属性值。
trsgraph> MATCH (v:player{name:"Tim Duncan"})--(v2:player) \
    RETURN v2.player.name AS Name;

+-----+
| Name
+-----+
| "Manu Ginobili" |
| "Manu Ginobili" |
| "Dejounte Murray" |
...

```

用户可以在 `--` 符号上增加 `<` 或 `>` 符号指定边的方向。

```

# `-->` 表示边从 v 开始，指向 v2。对于点 v 来说是出边，对于点 v2 来说是入边。
trsgraph> MATCH (v:player{name:"Tim Duncan"})-->(v2:player) \
    RETURN v2.player.name AS Name;

```

```
+-----+
| Name   |
+-----+
| "Tony Parker" |
| "Manu Ginobili" |
+-----+
```

如果需要判断目标点，可以使用 `CASE` 表达式。

```
# name 属性值为 Tim Duncan 的点为 v，与 v 相连接的点为 v2，查找 v2 并判断，如果 v2.team.name 的值不为空则返回 v2.team.name 属性值，如果 v2.player.name 的值不为空则返回 v2.player.name 属性值。
trsgaph> MATCH (v:player{name:"Tim Duncan"})--(v2) \
RETURN \
CASE WHEN v2.team.name IS NOT NULL \
THEN v2.team.name \
WHEN v2.player.name IS NOT NULL \
THEN v2.player.name END AS Name;

+-----+
| Name   |
+-----+
| "Manu Ginobili" |
| "Manu Ginobili" |
| "Spurs" |
| "Dejounte Murray" |
...

```

如果需要扩展模式，可以增加更多点和边。

```
# name 属性值为 Tim Duncan 的点为 v，指向点 v2，点 v3 也指向点 v2，返回 v3 的 name 属性值。
trsgraph> MATCH (v:player{name:"Tim Duncan"})-->(v2)<--(v3) \
RETURN v3.player.name AS Name;

+-----+
| Name   |
+-----+
| "Dejounte Murray" |
| "LaMarcus Aldridge" |
| "Marco Belinelli" |
...

```

如果不需要引用点，可以省略括号中表示点的变量。

```
# 查找 name 属性值为 Tim Duncan 的点 v，点 v3 与 v 指向同一个点，返回 v3 的 name 属性值。
trsgraph> MATCH (v:player{name:"Tim Duncan"})-->()<--(v3) \
RETURN v3.player.name AS Name;

+-----+
| Name   |
+-----+
| "Dejounte Murray" |
| "LaMarcus Aldridge" |
| "Marco Belinelli" |
...

```

匹配路径

连接起来的点和边构成了路径。用户可以使用自定义变量命名路径。

```
# 设置路径 p，其模式为 name 属性值为 Tim Duncan 的点 v 指向相邻的点 v2。返回所有符合条件的路径。
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-->(v2) \
RETURN p;

+-----+
| p |
+-----+
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:serve@0 {end_year: 2016, start_year: 1997}]->("team204" :team{name: "Spurs"})> |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})> |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})> |
+-----+
```



OpenCypher 兼容性

在 nGQL 中，`@` 符号表示边的 rank，在 openCypher 中，没有 rank 概念。

匹配边

```
# 匹配对应边并返回 3 条数据。
trsgraph> MATCH ()<-[e]-() \
RETURN e \
LIMIT 3;

+-----+
| e |
+-----+
```

```
+-----+
|[:follow "player101"->"player102" @0 {degree: 90}] |
|[:follow "player103"->"player102" @0 {degree: 70}] |
|[:follow "player135"->"player102" @0 {degree: 80}] |
+-----+
```

匹配 EDGE TYPE

和点一样，用户可以用 `:<edge_type>` 表示模式中的 Edge type，例如 `-[e:follow]-`。



版本兼容性

在 TRSGraph 3.0.0 之前，匹配 Edge Type 的前提是 Edge Type 本身有对应属性的索引，否则，用户无法基于 Edge Type 执行 `MATCH` 语句。从 TRSGraph 3.0.0 开始，匹配 Edge Type 可以不创建索引，但需要使用 `LIMIT` 限制输出结果数量，并且必须指定边的方向。从 TRSGraph 3.5.0 开始，无需为 Edge Type 创建索引或者使用 `LIMIT` 限制输出结果数量，即可使用 `MATCH` 语句匹配边。

```
# 匹配所有 edge type 为 follow 的边。
trsgraph> MATCH ()-[e:follow]->() \
    RETURN e;

+-----+
| e |
+-----+
|[:follow "player102"->"player100" @0 {degree: 75}] |
|[:follow "player102"->"player101" @0 {degree: 75}] |
|[:follow "player129"->"player116" @0 {degree: 90}] |
...
+-----+
```

匹配边的属性

用户可以用 `{<prop_name>: <prop_value>}` 表示模式中 Edge type 的属性，例如 `[e:follow{likeness:95}]`。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e:follow{degree:95}]->(v2) \
    RETURN e;

+-----+
| e |
+-----+
|[:follow "player100"->"player101" @0 {degree: 95}] |
|[:follow "player100"->"player125" @0 {degree: 95}] |
+-----+
```

使用 `WHERE` 子句直接匹配边的属性。

```
trsgraph> MATCH ()-[e]->() \
    WITH e, properties(e) as props, keys(properties(e)) as kk \
    WHERE [i in kk where props[i] == 90] \
    RETURN e;

+-----+
| e |
+-----+
|[:follow "player125"->"player100" @0 {degree: 90}] |
|[:follow "player140"->"player114" @0 {degree: 90}] |
|[:follow "player133"->"player144" @0 {degree: 90}] |
|[:follow "player133"->"player114" @0 {degree: 90}] |
...
+-----+
```

匹配多个 EDGE TYPE

使用 `|` 可以匹配多个 Edge type，例如 `[e:follow|:serve]`。第一个 Edge type 前的英文冒号 (:) 不可省略，后续 Edge type 前的英文冒号可以省略，例如 `[e:follow|serve]`。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e:follow|:serve]->(v2) \
    RETURN e;

+-----+
| e |
+-----+
|[:follow "player100"->"player101" @0 {degree: 95}] |
|[:follow "player100"->"player125" @0 {degree: 95}] |
|[:serve "player100"->"team204" @0 {end_year: 2016, start_year: 1997}] |
+-----+
```

匹配多条边

用户可以扩展模式，匹配路径中的多条边。


```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[]->(v2)<-[e:serve]-(v3) \
    RETURN v2, v3;
+-----+-----+
| v2          | v3          |
+-----+-----+
| ("team204" :team{name:"Spurs"}) | ("player104" :player{age: 32, name: "Marco Belinelli"}) |
| ("team204" :team{name:"Spurs"}) | ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("team204" :team{name:"Spurs"}) | ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
| ...          |              |
+-----+-----+
```

匹配定长路径

用户可以在模式中使用 :<edge_type>*<hop> 匹配定长路径。hop 必须是一个非负整数。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*2]->(v2) \
    RETURN DISTINCT v2 AS Friends;
+-----+-----+
| Friends          |
+-----+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
+-----+-----+
```

如果 hop 为 0，模式会匹配路径上的起始点。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[*0]->(v2) \
    RETURN v2;
+-----+-----+
| v2          |
+-----+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+-----+
```



在对匹配的多跳边进行过滤时，如对 -[e:follow*2]-> 中的 e 进行过滤，此时的 e 不再是单条边的数据类型，而是一个包含多条边的列表，例如：

以下语句可以运行但是没有返回数据，因为 e 是一个列表，没有 .degree 的属性。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*2]->(v2) \
    WHERE e.degree > 1 \
    RETURN DISTINCT v2 AS Friends;
```

这是正确的表达：

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*2]->(v2) \
    WHERE ALL(e_ in e WHERE e_.degree > 0) \
    RETURN DISTINCT v2 AS Friends;
```

进一步，这是表达对多跳边的第一跳的边属性过滤的表达：

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*2]->(v2) \
    WHERE e[0].degree > 98 \
    RETURN DISTINCT v2 AS Friends;
```

匹配变长路径

用户可以在模式中使用 :<edge_type>*[minHop..maxHop] 匹配变长路径。



如果未设置 maxHop 可能会导致 graph 服务 OOM，请谨慎执行该命令。

参数	说明
minHop	可选项。表示路径的最小长度。minHop 必须是一个非负整数，默认值为 1。
maxHop	可选项。表示路径的最大长度。maxHop 必须是一个非负整数，默认值为无穷大。

如果未指定 `minHop` 和 `maxHop`，仅设置了 `:<edge_type>*`，则二者都应用默认值，即 `minHop` 为 1，`maxHop` 为无穷大。

```

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*]->(v2) \
RETURN v2 AS Friends;
+-----+
| Friends |
+-----+
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
| ("player101" :player{age: 36, name: "Tony Parker"}) |
| ... |
+-----+

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*1..3]->(v2) \
RETURN v2 AS Friends;
+-----+
| Friends |
+-----+
| ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
| ... |
+-----+

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*1..]->(v2) \
RETURN v2 AS Friends;
+-----+
| Friends |
+-----+
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
| ("player101" :player{age: 36, name: "Tony Parker"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
| ... |
+-----+

```

用户可以使用 `DISTINCT` 关键字聚合重复结果。

```

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*1..3]->(v2:player) \
RETURN DISTINCT v2 AS Friends, count(v2);
+-----+-----+
| Friends | count(v2) |
+-----+-----+
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) | 1 |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | 4 |
| ("player101" :player{age: 36, name: "Tony Parker"}) | 3 |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) | 3 |
+-----+-----+

```

如果 `minHop` 为 0，模式会匹配路径上的起始点。例如，与上个示例相比，下面的示例设置 `minHop` 为 0。此时，因为表示 "Tim Duncan" 的点是路径的起始点，所以它在结果集中的计数为 5，比在上个示例的结果中多计一次。

```

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow*0..3]->(v2:player) \
RETURN DISTINCT v2 AS Friends, count(v2);
+-----+-----+
| Friends | count(v2) |
+-----+-----+
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) | 1 |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | 5 |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) | 3 |
| ("player101" :player{age: 36, name: "Tony Parker"}) | 3 |
+-----+-----+

```



当在模式中使用变量 `e` 匹配定长或者变长路径时，例如 `-[e:follow*0..3]->`，不支持在其他模式中引用 `e`。例如，不支持以下语句：

```

trsgraph> MATCH (v:player)-[e:like*1..3]->(n) \
WHERE (n)-[e*1..4]->(:player) \
RETURN v;

```

匹配多个 EDGE TYPE 的变长路径

用户可以在变长或定长模式中指定多个 Edge type。`hop`、`minHop` 和 `maxHop` 对所有 Edge type 都生效。

```

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e:follow|serve*2]->(v2) \
RETURN DISTINCT v2;
+-----+
| v2 |
+-----+
| ("team204" :team{name: "Spurs"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
| ("team215" :team{name: "Hornets"}) |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
+-----+

```

```
| ("player102" :player{age: 33, name: "LaMarcus Aldridge"}) |
+-----+
```

匹配多个模式

用户可以用英文逗号 (,) 分隔多个模式。

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS team_index ON team(name(20));
trsgraph> REBUILD TAG INDEX team_index;
trsgraph> MATCH (v1:player{name:"Tim Duncan"}), (v2:team{name:"Spurs"}) \
    RETURN v1,v2;
+-----+
| v1 | v2 |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) | ("team204" :team{name: "Spurs"}) |
+-----+
```

匹配最短路径

用户可以使用 `allShortestPaths` 返回起始点到目标点的所有最短路径。

```
trsgraph> MATCH p = allShortestPaths((a:player{name:"Tim Duncan"})-[e*.5]-(b:player{name:"Tony Parker"})) \
    RETURN p;
+-----+
| p |
+-----+
| <("player100" :player{age: 42, name: "Tim Duncan"})-<[:follow@0 {degree: 95}]-(player101" :player{age: 36, name: "Tony Parker"})> |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})> |
+-----+
```

用户可以使用 `shortestPath` 返回起始点到目标点的任意一条最短路径。

```
trsgraph> MATCH p = shortestPath((a:player{name:"Tim Duncan"})-[e*.5]-(b:player{name:"Tony Parker"})) \
    RETURN p;
+-----+
| p |
+-----+
| <("player100" :player{age: 42, name: "Tim Duncan"})-<[:follow@0 {degree: 95}]-(player101" :player{age: 36, name: "Tony Parker"})> |
+-----+
```

多MATCH检索

不同的模式有不同的筛选条件时，可以使用多 `MATCH`，会返回模式完全匹配的行。

```
trsgraph> MATCH (m)-[]->(n) WHERE id(m)=="player100" \
    MATCH (n)-[]->(l) WHERE id(n)=="player125" \
    RETURN id(m),id(n),id(l);
+-----+
| id(m) | id(n) | id(l) |
+-----+
| "player100" | "player125" | "team204" |
| "player100" | "player125" | "player100" |
+-----+
```

OPTIONAL MATCH检索

参见[OPTIONAL MATCH](#)。



Caution

TRSGraph 10.0 中 `MATCH` 语句的性能和资源占用得到了优化,但对性能要求较高时，仍建议使用 `GO`，`LOOKUP`，`|` 和 `FETCH` 等来替代 `MATCH`。

最后更新: 2025年12月19日

4.5.3 OPTIONAL MATCH



Caution

目前 `OPTIONAL MATCH` 为 Beta 功能，后续可能会有一定优化调整。

`OPTIONAL MATCH` 通常与 `MATCH` 语句一起使用，作为 `MATCH` 语句的可选项去匹配命中的模式，如果没有命中对应的模式，对应的列返回 `NULL`。

openCypher 兼容性

本文操作仅适用于 nGQL 中的 openCypher 方式。

使用限制

`OPTIONAL MATCH` 子句中暂不支持使用 `WHERE` 子句。

示例

`MATCH` 语句中使用 `OPTIONAL MATCH` 的示例如下：

```
# 找出特定节点 m (ID为 "player100" 的节点) 直接连接的节点，以及这些节点 n 直连的下一层节点 l (OPTIONAL MATCH (n)-[]->(l))， 如果没有找到这样的节点 l，查询也会继续， l 的返回值是 null。
trsgraph> MATCH (m)-[]->(n) WHERE id(m)=="player100" \
            OPTIONAL MATCH (n)-[]->(l) \
            RETURN id(m),id(n),id(l);

+-----+-----+-----+
| id(m) | id(n) | id(l) |
+-----+-----+-----+
| "player100" | "team204" | __NULL__ |
| "player100" | "player101" | "team204" |
| "player100" | "player101" | "team215" |
| "player100" | "player101" | "player100" |
| "player100" | "player101" | "player102" |
| "player100" | "player101" | "player125" |
| "player100" | "player125" | "team204" |
| "player100" | "player125" | "player100" |
+-----+-----+-----+
```

而使用多 `MATCH`，不使用 `OPTIONAL MATCH` 时，会返回模式完全匹配的行。示例如下：

```
# 找出特定节点 m (ID为 "player100" 的节点) 直接连接的节点，以及这些节点 n 直连的下一层节点 l，节点 l 必须存在。
trsgraph> MATCH (m)-[]->(n) WHERE id(m)=="player100" \
            MATCH (n)-[]->(l) \
            RETURN id(m),id(n),id(l);

+-----+-----+-----+
| id(m) | id(n) | id(l) |
+-----+-----+-----+
| "player100" | "player101" | "team204" |
| "player100" | "player101" | "team215" |
| "player100" | "player101" | "player100" |
| "player100" | "player101" | "player102" |
| "player100" | "player101" | "player125" |
| "player100" | "player125" | "team204" |
| "player100" | "player125" | "player100" |
+-----+-----+-----+
```

最后更新: 2025年12月19日

4.5.4 LOOKUP

`LOOKUP` 根据索引遍历数据。用户可以使用 `LOOKUP` 实现如下功能：

- 根据 `WHERE` 子句搜索特定数据。
- 通过 `Tag` 列出点：检索指定 `Tag` 的所有点 ID。
- 通过 `Edge type` 列出边：检索指定 `Edge type` 的所有边的起始点、目的点和 `rank`。
- 统计包含指定 `Tag` 的点或属于指定 `Edge type` 的边的数量。

OpenCypher 兼容性

本文操作仅适用于原生 nGQL。

注意事项

- 索引会导致写性能大幅降低。请不要随意在生产环境中使用索引，除非很清楚使用索引对业务的影响。
- 通过 `Explain` 命令查看选择的索引。



版本兼容性

在 2.5.0 版本之前，如果用 `LOOKUP` 语句基于指定属性查询时该属性没有索引，系统将报错，而不会使用其它索引。

前提条件

请确保 `LOOKUP` 语句有至少一个索引可用。

如果已经存在相关的点、边或属性，必须在新创建索引后重建索引，才能使其生效。

语法

```
LOOKUP ON {<vertex_tag> | <edge_type>}
[WHERE <expression> [AND <expression> ...]]
YIELD [DISTINCT] <return_list> [AS <alias>]
[<clause>];

<return_list>
  <prop_name> [AS <col_alias>] [, <prop_name> [AS <prop_alias>] ...];
```

- `WHERE <expression>`：指定遍历的过滤条件，还可以结合布尔运算符 `AND` 和 `OR` 一起使用。详情请参见 `WHERE`。
- `YIELD`：定义需要返回的输出。详情请参见 `YIELD`。
- `DISTINCT`：聚合输出结果，返回去重后的结果集。
- `AS`：设置别名。
- `clause`：支持 `ORDER BY`、`LIMIT` 子句。

WHERE 语句限制

在 LOOKUP 语句中使用 WHERE 子句，不支持如下操作：

- `$-` 和 `$^`。
- 在关系表达式中，不支持运算符两边都有字段名，例如 `tagName.prop1 > tagName.prop2`。
- 不支持运算表达式和函数表达式中嵌套 AliasProp 表达式。
- 不支持 XOR 运算符。
- 不支持除 `STARTS WITH` 之外的字符串操作。
- 不支持过滤 `rank()`。
- 不支持图模式。

检索点

返回 Tag 为 `player` 且 `name` 为 `Tony Parker` 的点。

```
# 创建 Tag 为 player 的复合属性索引 index_player。
trsgraph> CREATE TAG INDEX IF NOT EXISTS index_player ON player(name(30), age);

# 重建复合属性索引 index_player，返回任务 id。
trsgraph> REBUILD TAG INDEX index_player;
+-----+
| New Job Id |
+-----+
| 15         |
+-----+

# 返回所有 name 等于 Tony Parker 的点数据的 ID。
trsgraph> LOOKUP ON player \
    WHERE player.name == "Tony Parker" \
    YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player101" |
+-----+

# 查询所有 name 等于 Tony Parker 的点数据，返回 name 和 age 属性值。
trsgraph> LOOKUP ON player \
    WHERE player.name == "Tony Parker" \
    YIELD properties(vertex).name AS name, properties(vertex).age AS age;
+-----+-----+
| name      | age |
+-----+-----+
| "Tony Parker" | 36 |
+-----+-----+

# 查询所有 age 大于 45 的点数据，并返回点数据的 ID。
trsgraph> LOOKUP ON player \
    WHERE player.age > 45 \
    YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player144" |
| "player140" |
+-----+

# 查询所有 name 以 B 开头，且 age 是 22 或 30 的点数据，并返回 name 和 age 属性值。
trsgraph> LOOKUP ON player \
    WHERE player.name STARTS WITH "B" \
    AND player.age IN [22,30] \
    YIELD properties(vertex).name, properties(vertex).age;
+-----+-----+-----+
| properties(VERTEX).name | properties(VERTEX).age |
+-----+-----+-----+
| "Ben Simmons"          | 22                      |
| "Blake Griffin"        | 30                      |
+-----+-----+-----+

# 查询所有 name 等于 Kobe Bryant 的点数据，返回点的 ID 和 name 属性值，并以此为起始点进行遍历，返回遍历结果。
trsgraph> LOOKUP ON player \
    WHERE player.name == "Kobe Bryant" \
    YIELD id(vertex) AS VertexID, properties(vertex).name AS name \
    GO FROM $-.VertexID OVER serve \
    YIELD $-.name, properties(edge).start_year, properties(edge).end_year, properties($$).name;
+-----+-----+-----+-----+
| $-.name      | properties(EDGE).start_year | properties(EDGE).end_year | properties($$).name |
+-----+-----+-----+-----+
```

"Kobe Bryant" 1996	2016	"Lakers"	
-----	-----	-----	-----

检索边

返回 Edge type 为 follow 且 degree 为 90 的边。

```
# 创建 Edge type 为 follow, 属性 degree 的索引 index_follow。
trsgraph> CREATE EDGE INDEX IF NOT EXISTS index_follow ON follow(degree);

# 重建属性索引 index_follow, 返回任务 id。
trsgraph> REBUILD EDGE INDEX index_follow;
+-----+
| New Job Id |
+-----+
| 62         |
+-----+

# 查询并返回所有 degree 等于 90 的边。
trsgraph> LOOKUP ON follow \
    WHERE follow.degree == 90 YIELD edge AS e;
+-----+
| e |
+-----+
| [:follow "player109"->"player125" @0 {degree: 90}] |
| [:follow "player118"->"player120" @0 {degree: 90}] |
| [:follow "player118"->"player131" @0 {degree: 90}] |
...

# 查询所有 degree 等于 90 的边, 并返回 degree 属性值。
trsgraph> LOOKUP ON follow \
    WHERE follow.degree == 90 \
    YIELD properties(edge).degree;
+-----+
| properties(EDGE).degree |
+-----+
| 90 |
| 90 |
...

# 根据 degree 属性值升序排列, 并返回前 10 条 degree 属性值。
trsgraph> LOOKUP ON follow \
    YIELD properties(edge).degree as degree \
    ORDER BY $-.degree \
    LIMIT 10;
+-----+
| degree |
+-----+
| -1 |
| -1 |
| 9 |
| 10 |
| 13 |
| 50 |
| 55 |
| 60 |
| 70 |
| 70 |
+-----+

# 查询所有 degree 等于 60 的边, 返回边的目的点 ID 和边的 degree 属性值, 并为此起始点进行遍历, 返回遍历结果。
trsgraph> LOOKUP ON follow \
    WHERE follow.degree == 60 \
    YIELD dst(edge) AS DstVID, properties(edge).degree AS Degree | \
    GO FROM $-.DstVID OVER serve \
    YIELD $-.DstVID, properties(edge).start_year, properties(edge).end_year, properties($$.name);
+-----+
| $-.DstVID | properties(EDGE).start_year | properties(EDGE).end_year | properties($$.name) |
+-----+
| "player105" | 2010 | 2018 | "Spurs" |
| "player105" | 2009 | 2010 | "Cavaliers" |
| "player105" | 2018 | 2019 | "Raptors" |
+-----+
```

通过 Tag 列出所有的对应的点/通过 Edge type 列出边

如果需要通过 Tag 列出所有的点, 或通过 Edge type 列出边, 则 Tag、Edge type 或属性上必须有至少一个索引。

例如一个 Tag `player` 有属性 `name` 和 `age`，为了遍历所有包含 Tag `player` 的点 ID，Tag `player`、属性 `name` 或属性 `age` 中必须有一个已经创建索引。

- 查找所有 Tag 为 `player` 的点 VID。

```
# 创建名为 player 的 Tag。
trsgraph> CREATE TAG IF NOT EXISTS player(name string,age int);

# 创建 Tag 为 player 的索引 player_index。
trsgraph> CREATE TAG INDEX IF NOT EXISTS player_index on player();

# 重建索引 player_index，返回任务 id。
trsgraph> REBUILD TAG INDEX player_index;
+-----+
| New Job Id |
+-----+
| 66         |
+-----+

# 插入两个点数据。
trsgraph> INSERT VERTEX player(name,age) \
VALUES "player100":{"Tim Duncan", 42}, "player101":{"Tony Parker", 36};

# 列出所有的 player。类似于 MATCH (n:player) RETURN id(n) /*, n */。
trsgraph> LOOKUP ON player YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player100" |
| "player101" |
...

# 从结果中返回最前面的 4 行数据。
trsgraph> LOOKUP ON player YIELD id(vertex) | LIMIT 4;
+-----+
| id(VERTEX) |
+-----+
| "player105" |
| "player109" |
| "player111" |
| "player118" |
+-----+
```

- 查找 Edge type 为 `follow` 的所有边的信息。

```
# 创建名为 follow 的 Edge type。
trsgraph> CREATE EDGE IF NOT EXISTS follow(degree int);

# 创建 Edge type 为 follow 的索引 follow_index。
trsgraph> CREATE EDGE INDEX IF NOT EXISTS follow_index on follow();

# 重建索引 follow_index。
trsgraph> REBUILD EDGE INDEX follow_index;
+-----+
| New Job Id |
+-----+
| 88         |
+-----+

# 插入边数据。
trsgraph> INSERT EDGE follow(degree) \
VALUES "player100"->"player101":(95);

# 列出所有的 follow 边。类似于 MATCH (s)-[e:follow]->(d) RETURN id(s), rank(e), id(d) /*, type(e) */。
trsgraph> LOOKUP ON follow YIELD edge AS e;
+-----+
| e |
+-----+
| [:follow "player105"->"player100" @0 {degree: 70}] |
| [:follow "player105"->"player116" @0 {degree: 80}] |
| [:follow "player109"->"player100" @0 {degree: 80}] |
...
```

统计点或边

统计 Tag 为 `player` 的点和 Edge type 为 `follow` 的边。

```
# 统计 Tag 为 player 的点总数。
trsgraph> LOOKUP ON player YIELD id(vertex) \
YIELD COUNT(*) AS Player_Number;
+-----+
| Player_Number |
+-----+
| 51            |
+-----+
```



```
# 统计 Edge type 为 follow 的边总数。
trsggraph> LOOKUP ON follow YIELD edge AS e| \
    YIELD COUNT(*) AS Follow_Number;
+-----+
| Follow_Number |
+-----+
| 81             |
+-----+
```



Note

使用 `SHOW STATS` 命令也可以统计点和边。

最后更新: 2025年12月19日

4.5.5 GO

`GO` 语句是 TRSGraph 图数据库中用于从给定起始点开始遍历图的语句。`GO` 语句采用的路径类型是 `walk`，即遍历时点和边都可以重复。

openCypher 兼容性

本文操作仅适用于原生 nGQL。

语法

```
GO [[<M> TO] <N> {STEP|STEPS} ] FROM <vertex_list>
OVER <edge_type_list> [{REVERSELY | BIDIRECT}]
[ WHERE <conditions> ]
YIELD [DISTINCT] <return_list>
[{ SAMPLE <sample_list> | <limit_by_list_clause> }]
[| GROUP BY {<col_name> | expression> | <position>} YIELD <col_name>]
[| ORDER BY <expression> [{ASC | DESC}]]
[| LIMIT [<offset>,<number_rows>];

<vertex_list> ::=
    <vid> [, <vid> ...]

<edge_type_list> ::=
    <edge_type> [, <edge_type> ...]
    | *
```

```
<return_list> ::=
  <col_name> [AS <col_alias>] [, <col_name> [AS <col_alias>] ...]
```

- `<N> {STEP|STEPS}`：指定跳数。如果没有指定跳数，默认值 `N` 为 1。如果 `N` 为 0，TRSGraph 不会检索任何边。
- `M TO N {STEP|STEPS}`：遍历 `M~N` 跳的边。如果 `M` 为 0，输出结果和 `M` 为 1 相同，即 `GO 0 TO 2` 和 `GO 1 TO 2` 是相同的。
- `<vertex_list>`：用逗号分隔的点 ID 列表。
- `<edge_type_list>`：遍历的 Edge type 列表。
- `REVERSELY` | `BIDIRECT`：默认情况下检索的是 `<vertex_list>` 的出边（正向），`REVERSELY` 表示反向，即检索入边；`BIDIRECT` 为双向，即检索正向和反向。可通过 `YIELD` 返回 `<edge_type>._type` 字段判断方向，其正数为正向，负数为反向。
- `WHERE <conditions>`：指定遍历的过滤条件。用户可以在起始点、目的点和边使用 `WHERE` 子句，还可以结合 `AND`、`OR`、`NOT`、`XOR` 一起使用。详情参见 [WHERE](#)。



Note

- 遍历多个 Edge type 时，`WHERE` 子句有一些限制。例如不支持 `WHERE edge1.prop1 > edge2.prop2`。
- `GO` 语句执行时先遍历所有的点，然后再根据过滤器条件进行过滤。

- `YIELD [DISTINCT] <return_list>`：定义需要返回的输出。`<return_list>` 建议使用 [Schema 相关函数](#) 指定返回信息，当前支持 `src(edge)`、`dst(edge)`、`_type(edge)` 等，暂不支持嵌套函数。详情参见 [YIELD](#)。
- `SAMPLE <sample_list>`：用于在结果集中取样。详情参见 [SAMPLE](#)。
- `<limit_by_list_clause>`：用于在遍历过程中逐步限制输出数量。详情参见 [LIMIT](#)。
- `GROUP BY`：根据指定属性的值将输出分组。详情参见 [GROUP BY](#)。分组后需要再次使用 `YIELD` 定义需要返回的输出。
- `ORDER BY`：指定输出结果的排序规则。详情参见 [ORDER BY](#)。



Note

没有指定排序规则时，输出结果的顺序不是固定的。

- `LIMIT [<offset>[,] <number_rows>]`：限制输出结果的行数。详情参见 [LIMIT](#)。

使用说明

- `GO` 语句中的 `WHERE` 和 `YIELD` 子句通常结合属性引用符(`$^` 和 `$$`)或函数 `properties($^)` 和 `properties($$)` 指定点的属性；使用函数 `properties(edge)` 指定边的属性。用法参见 [属性引用符](#) 和 [Schema 相关函数](#)。
- `GO` 复合语句中如需引用子查询的结果，需要为该结果设置别名，并使用管道符 `|` 传递给下一个子查询，同时在下一个子查询中使用 `$-` 引用该结果的别名。详情参见 [管道符](#)。
- 当查询属性没有值时，返回结果显示 `NULL`。

场景及示例

查询起始点的直接邻居点

场景：查询某个点的直接相邻点，例如查询一个人所属队伍。

```
# 返回 player102 所属队伍。
trsgraph> GO FROM "player102" OVER serve YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
```

```
| "team203" |  
| "team204" |  
+-----+
```

查询指定跳数内的点

场景：查询一个点在指定跳数内的所有点，例如查询一个人两跳内的朋友。

```
# 返回距离 player102 两跳的朋友。  
trsgraph> GO 2 STEPS FROM "player102" OVER follow YIELD dst(edge);  
+-----+  
| dst(EDGE) |  
+-----+  
| "player101" |  
| "player125" |  
| "player100" |  
| "player102" |  
| "player125" |  
+-----+
```

```
# 查询 player100 1~2 跳内的朋友。  
trsgraph> GO 1 TO 2 STEPS FROM "player100" OVER follow \  
YIELD dst(edge) AS destination;  
+-----+  
| destination |  
+-----+  
| "player101" |  
| "player125" |  
...  
  
# 该 MATCH 查询与上一个 GO 查询具有相同的语义。  
trsgraph> MATCH (v) -[e:follow*1..2]->(v2) \  
WHERE id(v) == "player100" \  
RETURN id(v2) AS destination;  
+-----+  
| destination |  
+-----+  
| "player100" |  
| "player102" |  
...  

```

添加过滤条件

场景：查询满足特定条件的点和边，例如查询起始点和目的点之间具有特定属性的边。

```
# 使用 WHERE 添加过滤条件。  
trsgraph> GO FROM "player100", "player102" OVER serve \  
WHERE properties(edge).start_year > 1995 \  
YIELD DISTINCT properties($$).name AS team_name, properties(edge).start_year AS start_year, properties($^).name AS player_name;  
  
+-----+  
| team_name | start_year | player_name |  
+-----+  
| "Spurs" | 1997 | "Tim Duncan" |  
| "Trail Blazers" | 2006 | "LaMarcus Aldridge" |  
| "Spurs" | 2015 | "LaMarcus Aldridge" |  
+-----+
```

查询所有边

场景：查询起始点关联的所有边。

```
# 返回 player102 关联的所有边。  
trsgraph> GO FROM "player102" OVER * BIDIRECT YIELD edge AS e;  
+-----+  
| e |  
+-----+  
| [:follow "player101"->"player102" @0 {degree: 90}] |  
| [:follow "player103"->"player102" @0 {degree: 70}] |  
| [:follow "player135"->"player102" @0 {degree: 80}] |  
| [:follow "player102"->"player100" @0 {degree: 75}] |  
| [:follow "player102"->"player101" @0 {degree: 75}] |  
| [:serve "player102"->"team203" @0 {end_year: 2015, start_year: 2006}] |  
| [:serve "player102"->"team204" @0 {end_year: 2019, start_year: 2015}] |  
+-----+
```

查询多个 EDGE TYPE

场景：查询起始点关联的多个边类型可以通过设置多个Edge Type实现，也可以通过设置 * 关联所有的边类型。

```
# 遍历多个 Edge type。  
trsgraph> GO FROM "player100" OVER follow, serve \  
YIELD properties(edge).degree, properties(edge).start_year;  
+-----+
```

```
| properties(EDGE).degree | properties(EDGE).start_year |
+-----+-----+
| 95                      |      NULL      |
| 95                      |      NULL      |
|      NULL              |      1997      |
+-----+-----+
```

查询入边方向的点

```
# 返回关注 player100 的邻居点。
trsgraph> GO FROM "player100" OVER follow REVERSELY \
    YIELD src(edge) AS destination;
+-----+
| destination |
+-----+
| "player101" |
| "player102" |
...

# 该 MATCH 查询与上一个 GO 查询具有相同的语义。
trsgraph> MATCH (v)<-[e:follow]- (v2) WHERE id(v) == 'player100' \
    RETURN id(v2) AS destination;
+-----+
| destination |
+-----+
| "player101" |
| "player102" |
...
```

子查询作为起始点

场景：使用子查询的结果作为图遍历的起始点。

```
# 查询 player100 年龄大于 20 的朋友和这些朋友所属队伍。
trsgraph> GO FROM "player100" OVER follow \
    YIELD dst(edge) AS id | \
    GO FROM $.id OVER serve \
    WHERE properties($^).age > 20 \
    YIELD properties($^).name AS Friend, properties($$).name AS Team;
+-----+-----+
| Friend      | Team      |
+-----+-----+
| "Tony Parker" | "Spurs"   |
| "Tony Parker" | "Hornets" |
| "Manu Ginobili" | "Spurs"  |
+-----+-----+

# 该 MATCH 查询与上一个 GO 查询具有相同的语义。
trsgraph> MATCH (v)-[e:follow]-> (v2)-[e2:serve]->(v3) \
    WHERE id(v) == 'player100' \
    AND properties(v2).age > 20 \
    RETURN v2.player.name AS Friend, v3.team.name AS Team;
+-----+-----+
| Friend      | Team      |
+-----+-----+
| "Tony Parker" | "Spurs"   |
| "Tony Parker" | "Hornets" |
| "Manu Ginobili" | "Spurs"  |
+-----+-----+
```

使用 GROUP BY 分组

场景：使用 GROUP BY 分组，然后使用 YIELD 返回分组后的结果。

```
# 根据年龄分组。
trsgraph> GO 2 STEPS FROM "player100" OVER follow \
    YIELD src(edge) AS src, dst(edge) AS dst, properties($$).age AS age \
    | GROUP BY $-.dst \
    YIELD $-.dst AS dst, collect_set($-.src) AS src, collect($-.age) AS age;
+-----+-----+-----+
| dst      | src      | age      |
+-----+-----+-----+
| "player125" | {"player101"} | [41]     |
| "player100" | {"player125", "player101"} | [42, 42] |
| "player102" | {"player101"} | [33]     |
+-----+-----+-----+
```

使用 ORDER BY 和 LIMIT 排序和限制输出结果

```
# 分组并限制输出结果的行数。
trsgraph> $a = GO FROM "player100" OVER follow YIELD src(edge) AS src, dst(edge) AS dst; \
    GO 2 STEPS FROM $a.dst OVER follow \
    YIELD $a.src AS src, $a.dst, src(edge), dst(edge) \
    | ORDER BY $-.src | OFFSET 1 LIMIT 2;
+-----+-----+-----+-----+
| src      | $a.dst      | src(EDGE) | dst(EDGE) |
+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+
| "player100" | "player101" | "player100" | "player101" |
| "player100" | "player125" | "player100" | "player125" |
+-----+-----+-----+-----+
```

其他用法

```
# 在多个边上通过 IS NOT EMPTY 进行判断。
trsgraph> GO FROM "player100" OVER follow WHERE properties($$).name IS NOT EMPTY YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "player125" |
| "player101" |
+-----+
```

最后更新: 2025年12月19日

4.5.6 FETCH

`FETCH` 可以获取指定点或边的属性值。

openCypher 兼容性

本文操作仅适用于原生 nGQL。

获取点的属性值

语法

```
FETCH PROP ON {<tag_name>[, tag_name ...] | *}
<vid> [, vid ...]
YIELD [DISTINCT] <return_list> [AS <alias>];
```

参数	说明
<code>tag_name</code>	Tag 名称。
<code>*</code>	表示当前图空间中的所有 Tag。
<code>vid</code>	点 ID。
<code>YIELD</code>	定义需要返回的输出。详情请参见 <code>YIELD</code> 。
<code>AS</code>	设置别名。

基于 TAG 获取点的属性值

在 `FETCH` 语句中指定 Tag 获取对应点的属性值。

```
# 获取 Tag 为 player, 且 ID 为 player100 的点数据的属性值。
trsgaph> FETCH PROP ON player "player100" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 42, name: "Tim Duncan"} |
+-----+
```

获取点的指定属性值

使用 `YIELD` 子句指定返回的属性。

```
# 获取 Tag 为 player, 且 ID 为 player100 的点数据 name 的属性值。
trsgraph> FETCH PROP ON player "player100" \
    YIELD properties(vertex).name AS name;
+-----+
| name |
+-----+
| "Tim Duncan" |
+-----+
```

获取多个点的属性值

指定多个点 ID 获取多个点的属性值，点之间用英文逗号（,）分隔。

```
# 获取 Tag 为 player, 且 ID 为 player101, player102, player103 三个点数据的属性值。
trsgraph> FETCH PROP ON player "player101", "player102", "player103" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 33, name: "LaMarcus Aldridge"} |
| {age: 36, name: "Tony Parker"} |
| {age: 32, name: "Rudy Gay"} |
+-----+
```

基于多个 TAG 获取点的属性值

在 `FETCH` 语句中指定多个 Tag 获取属性值。Tag 之间用英文逗号（,）分隔。

```
# 创建新 Tag t1。
trsgraph> CREATE TAG IF NOT EXISTS t1(a string, b int);

# 为点 player100 添加 Tag t1。
trsgraph> INSERT VERTEX t1(a, b) VALUES "player100":("Hello", 100);

# 基于 Tag player 和 t1 获取点 player100 上的属性值。
trsgraph> FETCH PROP ON player, t1 "player100" YIELD vertex AS v;
+-----+
| v      |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"} :t1{a: "Hello", b: 100}) |
+-----+
```

用户可以在 `FETCH` 语句中组合多个 Tag 和多个点。

```
# 获取 Tag 为 player 或 t1, 且 ID 为 player100, player103 两个点数据的属性值。
trsgraph> FETCH PROP ON player, t1 "player100", "player103" YIELD vertex AS v;
+-----+
| v      |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"} :t1{a: "Hello", b: 100}) |
| ("player103" :player{age: 32, name: "Rudy Gay"}) |
+-----+
```

在所有标签中获取点的属性值

在 `FETCH` 语句中使用 `*` 获取当前图空间所有标签里，点的属性值。

```
# 获取当前图空间所有标签里 ID 为 player100, player106, team200 的点数据属性值。
trsgraph> FETCH PROP ON * "player100", "player106", "team200" YIELD vertex AS v;
+-----+
| v      |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"} :t1{a: "Hello", b: 100}) |
| ("player106" :player{age: 25, name: "Kyle Anderson"}) |
| ("team200" :team{name: "Warriors"}) |
+-----+
```

获取边的属性值

语法

```
FETCH PROP ON <edge_type> <src_vid> -> <dst_vid>[@<rank>] [, <src_vid> -> <dst_vid> ...]
YIELD <output>;
```

参数	说明
<code>edge_type</code>	Edge type 名称。
<code>src_vid</code>	起始点 ID，表示边的起点。
<code>dst_vid</code>	目的点 ID，表示边的终点。
<code>rank</code>	边的 rank。可选参数，默认值为 0。起始点、目的点、Edge type 和 rank 可以唯一确定一条边。
<code>YIELD</code>	定义需要返回的输出。详情请参见 <code>YIELD</code> 。

获取边的所有属性值

```
# 获取连接 player100 和 team204 的边 serve 的所有属性值。
trsgraph> FETCH PROP ON serve "player100" -> "team204" YIELD properties(edge);
+-----+
| properties(EDGE) |
+-----+
| {end_year: 2016, start_year: 1997} |
+-----+
```

获取边的指定属性值

使用 `YIELD` 子句指定返回的属性。

```
# 获取连接 player100 和 team204 的边 serve 的 start_year 属性值。
trsgraph> FETCH PROP ON serve "player100" -> "team204" \
    YIELD properties(edge).start_year;
+-----+
| properties(EDGE).start_year |
```



```
+-----+
| 1997      |
+-----+
```

获取多条边的属性值

指定多个边模式 (`<src_vid> -> <dst_vid>[@<rank>]`) 获取多个边的属性值。模式之间用英文逗号 (,) 分隔。

```
# 获取连接 player100 和 team204 的边, 连接 player133 和 team202 的边 serve 的所有属性值。
trsgraph> FETCH PROP ON serve "player100" -> "team204", "player133" -> "team202" YIELD edge AS e;
+-----+
| e          |
+-----+
| [:serve "player100"->"team204" @0 {end_year: 2016, start_year: 1997}] |
| [:serve "player133"->"team202" @0 {end_year: 2011, start_year: 2002}] |
+-----+
```

基于 RANK 获取属性值

如果有多条边, 起始点、目的点和 Edge type 都相同, 可以通过指定 rank 获取正确的边属性值。

```
# 插入不同属性值、不同 rank 的边。
trsgraph> insert edge serve(start_year,end_year) \
    values "player100"->"team204"@1:(1998, 2017);

trsgraph> insert edge serve(start_year,end_year) \
    values "player100"->"team204"@2:(1990, 2018);

# 默认返回 rank 为 0 的边。
trsgraph> FETCH PROP ON serve "player100" -> "team204" YIELD edge AS e;
+-----+
| e          |
+-----+
| [:serve "player100"->"team204" @0 {end_year: 2016, start_year: 1997}] |
+-----+

# 要获取 rank 不为 0 的边, 请在 FETCH 语句中设置 rank。
trsgraph> FETCH PROP ON serve "player100" -> "team204"@1 YIELD edge AS e;
+-----+
| e          |
+-----+
| [:serve "player100"->"team204" @1 {end_year: 2017, start_year: 1998}] |
+-----+
```

复合语句中使用 FETCH

将 `FETCH` 与原生 nGQL 结合使用是一种常见的方式, 例如和 `GO` 一起。

```
# 返回从点 player101 开始的 follow 边的 degree 值。src(edge) 获取边的起始点 ID, dst(edge) 获取边的目的点 ID。
trsgraph> GO FROM "player101" OVER follow \
    YIELD src(edge) AS s, dst(edge) AS d \
    | FETCH PROP ON follow $-s -> $-d \
    YIELD properties(edge).degree;
+-----+
| properties(EDGE).degree |
+-----+
| 95                       |
| 90                       |
| 95                       |
+-----+
```

用户也可以通过自定义变量构建类似的查询。

```
# 返回从点 player101 开始的 follow 边的 degree 值。src(edge) 获取边的起始点 ID, dst(edge) 获取边的目的点 ID。
trsgraph> $var = GO FROM "player101" OVER follow \
    YIELD src(edge) AS s, dst(edge) AS d; \
    FETCH PROP ON follow $var.s -> $var.d \
    YIELD properties(edge).degree;
+-----+
| properties(EDGE).degree |
+-----+
| 95                       |
| 90                       |
| 95                       |
+-----+
```

更多复合语句的详情, 请参见[复合查询（子句结构）](#)。

4.5.7 SHOW

SHOW CHARSET

`SHOW CHARSET` 语句显示当前的字符集。

目前可用的字符集为 `utf8` 和 `utf8mb4`。默认字符集为 `utf8`。TRSGraph 扩展 `uft8` 支持四字节字符，因此 `utf8` 和 `utf8mb4` 是等价的。

语法

```
SHOW CHARSET;
```

示例

```
trsgraph> SHOW CHARSET;
+-----+-----+-----+-----+
| Charset | Description      | Default collation | Maxlen |
+-----+-----+-----+-----+
| "utf8"  | "UTF-8 Unicode"  | "utf8_bin"        | 4      |
+-----+-----+-----+-----+
```

参数	说明
<code>Charset</code>	字符集名称。
<code>Description</code>	字符集说明。
<code>Default collation</code>	默认排序规则。
<code>Maxlen</code>	存储一个字符所需的最大字节数。

最后更新: 2025年12月19日

SHOW COLLATION

SHOW COLLATION 语句显示当前的排序规则。

目前可用的排序规则为 utf8_bin 和 utf8mb4_bin。

- 当字符集为 utf8，默认排序规则为 utf8_bin。
- 当字符集为 utf8mb4，默认排序规则为 utf8mb4_bin。

语法

```
SHOW COLLATION;
```

示例

```
trsgraph> SHOW COLLATION;
+-----+-----+
| Collation | Charset |
+-----+-----+
| "utf8_bin" | "utf8" |
+-----+-----+
```

参数	说明
Collation	排序规则名称。
Charset	与排序规则关联的字符集名称。

最后更新: 2025年12月19日

SHOW CREATE SPACE

`SHOW CREATE SPACE` 语句显示指定图空间的创建语句。

图空间的更多详细信息，请参见 [CREATE SPACE](#)。

语法

```
SHOW CREATE SPACE <space_name>;
```

示例

```
trsgraph> SHOW CREATE SPACE basketballplayer;
+-----+
| Space          | Create
+-----+-----+
| "basketballplayer" | "CREATE SPACE `basketballplayer` (partition_num = 10, replica_factor = 1, charset = utf8, collate = utf8_bin, vid_type = FIXED_STRING(32))" |
+-----+-----+
```

最后更新: 2025年12月19日

SHOW CREATE TAG/EDGE

`SHOW CREATE TAG` 语句显示指定 Tag 的基本信息。Tag 的更多信息，请参见 [CREATE TAG](#)。

`SHOW CREATE EDGE` 语句显示指定 Edge type 的基本信息。Edge type 的更多信息，请参见 [CREATE EDGE](#)。

语法

```
SHOW CREATE {TAG <tag_name> | EDGE <edge_name>;}
```

示例

```
trsgraph> SHOW CREATE TAG player;
+-----+-----+
| Tag      | Create Tag      |
+-----+-----+
| "player" | "CREATE TAG `player` (  
|           |   `name` string NULL,  
|           |   `age` int64 NULL  
|           | ) ttl_duration = 0, ttl_col = "" |
+-----+-----+

trsgraph> SHOW CREATE EDGE follow;
+-----+-----+
| Edge      | Create Edge      |
+-----+-----+
| "follow"  | "CREATE EDGE `follow` (  
|           |   `degree` int64 NULL  
|           | ) ttl_duration = 0, ttl_col = "" |
+-----+-----+
```

最后更新: 2025年12月19日

SHOW HOSTS

`SHOW HOSTS` 语句可以显示集群信息，包括端口、状态、leader、分片、版本等信息，或者指定显示 Graph、Storage、Meta 服务主机信息。

语法

```
SHOW HOSTS [GRAPH | STORAGE | META];
```



对于使用源码安装的 TRSGraph，执行添加了服务名的命令后，输出的信息中不显示版本信息。

示例

```
trsgraph> SHOW HOSTS;
+-----+-----+-----+-----+-----+-----+-----+
| Host      | Port | Status | Leader count | Leader distribution | Partition distribution | Version |
+-----+-----+-----+-----+-----+-----+-----+
| "storaged0" | 9779 | "ONLINE" | 8 | "docs:5, basketballplayer:3" | "docs:5, basketballplayer:3" | "10.0" |
| "storaged1" | 9779 | "ONLINE" | 9 | "basketballplayer:4, docs:5" | "docs:5, basketballplayer:4" | "10.0" |
| "storaged2" | 9779 | "ONLINE" | 8 | "basketballplayer:3, docs:5" | "docs:5, basketballplayer:3" | "10.0" |
+-----+-----+-----+-----+-----+-----+-----+

trsgraph> SHOW HOSTS GRAPH;
+-----+-----+-----+-----+-----+-----+
| Host      | Port | Status | Role | Git Info Sha | Version |
+-----+-----+-----+-----+-----+-----+
| "graphd" | 9669 | "ONLINE" | "GRAPH" | "3ba41bd" | "10.0" |
| "graphd1" | 9669 | "ONLINE" | "GRAPH" | "3ba41bd" | "10.0" |
| "graphd2" | 9669 | "ONLINE" | "GRAPH" | "3ba41bd" | "10.0" |
+-----+-----+-----+-----+-----+-----+

trsgraph> SHOW HOSTS STORAGE;
+-----+-----+-----+-----+-----+-----+
| Host      | Port | Status | Role | Git Info Sha | Version |
+-----+-----+-----+-----+-----+-----+
| "storaged0" | 9779 | "ONLINE" | "STORAGE" | "3ba41bd" | "10.0" |
| "storaged1" | 9779 | "ONLINE" | "STORAGE" | "3ba41bd" | "10.0" |
| "storaged2" | 9779 | "ONLINE" | "STORAGE" | "3ba41bd" | "10.0" |
+-----+-----+-----+-----+-----+-----+

trsgraph> SHOW HOSTS META;
+-----+-----+-----+-----+-----+-----+
| Host      | Port | Status | Role | Git Info Sha | Version |
+-----+-----+-----+-----+-----+-----+
| "metad2" | 9559 | "ONLINE" | "META" | "3ba41bd" | "10.0" |
| "metad0" | 9559 | "ONLINE" | "META" | "3ba41bd" | "10.0" |
| "metad1" | 9559 | "ONLINE" | "META" | "3ba41bd" | "10.0" |
+-----+-----+-----+-----+-----+-----+
```

最后更新: 2025年12月19日

SHOW INDEX STATUS

`SHOW INDEX STATUS` 语句显示重建原生索引的作业状态，以便确定重建索引是否成功。

语法

```
SHOW {TAG | EDGE} INDEX STATUS;
```

示例

```
trsgraph> SHOW TAG INDEX STATUS;
+-----+-----+
| Name                                | Index Status |
+-----+-----+
| "date1_index"                      | "FINISHED"   |
| "basketballplayer_all_tag_indexes" | "FINISHED"   |
| "any_shape_geo_index"              | "FINISHED"   |
+-----+-----+

trsgraph> SHOW EDGE INDEX STATUS;
+-----+-----+
| Name                                | Index Status |
+-----+-----+
| "follow_index"                    | "FINISHED"   |
+-----+-----+
```

相关文档

- [管理作业](#)
- [REBUILD NATIVE INDEX](#)

最后更新: 2025年12月19日

SHOW INDEXES

`SHOW INDEXES` 语句可以列出当前图空间内的所有 Tag 和 Edge type（包括属性）的索引。

语法

```
SHOW {TAG | EDGE} INDEXES;
```

示例

```
trsgraph> SHOW TAG INDEXES;
+-----+-----+-----+
| Index Name      | By Tag | Columns |
+-----+-----+-----+
| "player_index_0" | "player" | []      |
| "player_index_1" | "player" | ["name"] |
+-----+-----+-----+

trsgraph> SHOW EDGE INDEXES;
+-----+-----+-----+
| Index Name      | By Edge | Columns |
+-----+-----+-----+
| "follow_index"  | "follow" | []      |
+-----+-----+-----+
```



版本兼容性

TRSGraph 2.0.1 中，`SHOW TAG/EDGE INDEXES` 语句仅返回 `Names`。

最后更新: 2025年12月19日

SHOW PARTS

`SHOW PARTS` 语句显示图空间中指定分片或所有分片的信息。

语法

```
SHOW PARTS [<part_id>;
```

示例

```
trsgraph> SHOW PARTS;
+-----+-----+-----+-----+
| Partition ID | Leader                | Peers                | Losts |
+-----+-----+-----+-----+
| 1            | "192.168.2.1:9779"    | "192.168.2.1:9779"  | ""    |
| 2            | "192.168.2.2:9779"    | "192.168.2.2:9779"  | ""    |
| 3            | "192.168.2.3:9779"    | "192.168.2.3:9779"  | ""    |
| 4            | "192.168.2.1:9779"    | "192.168.2.1:9779"  | ""    |
| 5            | "192.168.2.2:9779"    | "192.168.2.2:9779"  | ""    |
| 6            | "192.168.2.3:9779"    | "192.168.2.3:9779"  | ""    |
| 7            | "192.168.2.1:9779"    | "192.168.2.1:9779"  | ""    |
| 8            | "192.168.2.2:9779"    | "192.168.2.2:9779"  | ""    |
| 9            | "192.168.2.3:9779"    | "192.168.2.3:9779"  | ""    |
| 10           | "192.168.2.1:9779"    | "192.168.2.1:9779"  | ""    |
+-----+-----+-----+-----+

trsgraph> SHOW PARTS 1;
+-----+-----+-----+-----+
| Partition ID | Leader                | Peers                | Losts |
+-----+-----+-----+-----+
| 1            | "192.168.2.1:9779"    | "192.168.2.1:9779"  | ""    |
+-----+-----+-----+-----+
```

返回结果的说明如下：

参数	说明
<code>Partition ID</code>	存储分片的 ID。
<code>Leader</code>	分片对应的 Raft leader 副本的信息，包括 IP（或主机名）与服务端口。
<code>Peers</code>	分片对应的所有副本（leader 与 follower）的信息，包括 IP（或主机名）与服务端口。
<code>Losts</code>	分片对应的处于离线状态的副本信息，包括 IP（或主机名）和服务端口。

最后更新: 2025年12月19日

SHOW ROLES

`SHOW ROLES` 语句显示分配给用户的角色信息。

根据登录的用户角色，返回的结果也有所不同：

- 如果登录的用户角色是 `GOD`，或者有权访问该图空间的 `ADMIN`，则返回该图空间内除 `GOD` 之外的所有用户角色信息。
- 如果登录的用户角色是有权访问该图空间 `DBA`、`USER` 或 `GUEST`，则返回自身的角色信息。
- 如果登录的用户角色没有权限访问该图空间，则返回权限错误。

关于角色的详情请参见[内置角色权限](#)。

语法

```
SHOW ROLES IN <space_name>;
```

示例

```
trsgraph> SHOW ROLES in basketballplayer;
+-----+-----+
| Account | Role Type |
+-----+-----+
| "user1" | "ADMIN"   |
+-----+-----+
```

最后更新: 2025年12月19日

SHOW SNAPSHOTS

`SHOW SNAPSHOTS` 语句显示所有快照信息。

快照的使用方式请参见[管理快照](#)。

角色要求

只有 `GOD` 角色的用户（即 `root`）才能执行 `SHOW SNAPSHOTS` 语句。

语法

```
SHOW SNAPSHOTS;
```

示例

```
trsgaph> SHOW SNAPSHOTS;
+-----+-----+-----+
| Name                               | Status | Hosts                               |
+-----+-----+-----+
| "SNAPSHOT_2020_12_16_11_13_55"    | "VALID" | "storaged0:9779, storaged1:9779, storaged2:9779" |
| "SNAPSHOT_2020_12_16_11_14_10"    | "VALID" | "storaged0:9779, storaged1:9779, storaged2:9779" |
+-----+-----+-----+
```

最后更新: 2025年12月19日

SHOW SPACES

`SHOW SPACES` 语句显示现存的图空间。

如何创建图空间，请参见 [CREATE SPACE](#)。

语法

```
SHOW SPACES;
```

示例

```
trsgraph> SHOW SPACES;
+-----+
| Name          |
+-----+
| "docs"        |
| "basketballplayer" |
+-----+
```

最后更新: 2025年12月19日

SHOW STATS

`SHOW STATS` 语句显示最近一次 `SUBMIT JOB STATS` 作业收集的图空间统计信息。

图空间统计信息包含：

- 点的总数
- 边的总数
- 每个 Tag 关联的点的总数
- 每个 Edge type 关联的边的总数



`SHOW STATS` 返回的不是实时数据。因为返回的数据是最近一次 `SUBMIT JOB STATS` 作业收集的数据。返回的数据可能包含 TTL 过期数据，该过期数据会在下次执行 `Compaction` 操作时被删除并不纳入统计。

前提条件

在需要查看统计信息的图空间中执行 `SUBMIT JOB STATS`。详情请参见 `SUBMIT JOB STATS`。



`SHOW STATS` 的结果取决于最后一次执行的 `SUBMIT JOB STATS`。如果发生过新的写入或者更改，必须再次执行 `SUBMIT JOB STATS`，否则统计数据有错误。

语法

```
SHOW STATS;
```

示例

```
# 选择图空间。
trsgraph> USE basketballplayer;

# 执行 SUBMIT JOB STATS。
trsgraph> SUBMIT JOB STATS;
+-----+
| New Job Id |
+-----+
| 98         |
+-----+

# 确认作业执行成功。
trsgraph> SHOW JOB 98;
+-----+-----+-----+-----+-----+-----+
| Job Id(TaskId) | Command(Dest) | Status | Start Time | Stop Time | Error Code |
+-----+-----+-----+-----+-----+-----+
| 98             | "STATS"       | "FINISHED" | 2021-11-01T09:33:21.000000 | 2021-11-01T09:33:21.000000 | "SUCCEEDED" |
| 0              | "stored2"     | "FINISHED" | 2021-11-01T09:33:21.000000 | 2021-11-01T09:33:21.000000 | "SUCCEEDED" |
| 1              | "stored0"     | "FINISHED" | 2021-11-01T09:33:21.000000 | 2021-11-01T09:33:21.000000 | "SUCCEEDED" |
| 2              | "stored1"     | "FINISHED" | 2021-11-01T09:33:21.000000 | 2021-11-01T09:33:21.000000 | "SUCCEEDED" |
| "Total:3"     | "Succeeded:3" | "Failed:0" | "In Progress:0" | "" | "" |
+-----+-----+-----+-----+-----+-----+

# 显示图空间统计信息。
trsgraph> SHOW STATS;
+-----+-----+-----+
| Type | Name | Count |
+-----+-----+-----+
| "Tag" | "player" | 51 |
| "Tag" | "team" | 30 |
| "Edge" | "follow" | 81 |
| "Edge" | "serve" | 152 |
| "Space" | "vertices" | 81 |
| "Space" | "edges" | 233 |
+-----+-----+-----+
```

最后更新: 2025年12月19日

SHOW TAGS/EDGES

- SHOW TAGS 语句显示当前图空间内的所有 Tag。
- SHOW EDGES 语句显示当前图空间内的所有 Edge type。

语法

```
SHOW {TAGS | EDGES};
```

示例

```
trsgraph> SHOW TAGS;
+-----+
| Name   |
+-----+
| "player" |
| "star"  |
| "team"  |
+-----+

trsgraph> SHOW EDGES;
+-----+
| Name   |
+-----+
| "follow" |
| "serve"  |
+-----+
```

最后更新: 2025年12月19日

SHOW USERS

`SHOW USERS` 语句显示用户信息。

角色要求

只有 `GOD` 角色的用户（即 `root`）才能执行 `SHOW USERS` 语句。

语法

```
SHOW USERS;
```

示例

```
trsgraph> SHOW USERS;
+-----+-----+
| Account | IP Whitelist |
+-----+-----+
| "root"  | ""           |
| "user1" | ""           |
| "user2" | "192.168.10.10" |
+-----+-----+
```

最后更新: 2025年12月19日

SHOW SESSIONS

登录 TRSGraph 数据库时，会创建对应会话，用户可以查询会话信息。

注意事项

- 执行 `exit` 退出登录时，客户端会调用 API `release`，释放会话并清除会话信息。如果没有正常退出，且没有在配置文件 `trsgraph-graphd.conf` 设置空闲会话超时时间（`session_idle_timeout_secs`），会话不会自动释放。对于未自动释放的会话，需要手动删除指定会话，详情参见[终止会话](#)。
- `SHOW SESSIONS` 查询所有 Graph 服务上的会话信息。
- `SHOW LOCAL SESSIONS` 从当前连接的 Graph 服务获取会话信息，不会查询其他 Graph 服务上的会话信息。
- `SHOW SESSION <Session_Id>` 查询指定 Session ID 的会话信息。

语法

```
SHOW [LOCAL] SESSIONS;  
SHOW SESSION <Session_Id>;
```

示例

```
trsgraph> SHOW SESSIONS;  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+  
| SessionId | UserName | SpaceName | CreateTime | UpdateTime | GraphAddr | Timezone |  
ClientIp |  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+  
| 1651220858102296 | "root" | "basketballplayer" | 2022-04-29T08:27:38.102296 | 2022-04-29T08:50:46.282921 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
| 1651199330300991 | "root" | "basketballplayer" | 2022-04-29T02:28:50.300991 | 2022-04-29T08:16:28.339038 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
| 1651112899847744 | "root" | "basketballplayer" | 2022-04-28T02:28:19.847744 | 2022-04-28T08:17:44.470210 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
| 1651041092662100 | "root" | "basketballplayer" | 2022-04-27T06:31:32.662100 | 2022-04-27T07:01:25.200978 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
| 1650959429593975 | "root" | "basketballplayer" | 2022-04-26T07:50:29.593975 | 2022-04-26T07:51:47.184810 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
| 1650958897679595 | "root" | "" | 2022-04-26T07:41:37.679595 | 2022-04-26T07:41:37.683802 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+  
  
trsgraph> SHOW SESSION 1635254859271703;  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+  
| SessionId | UserName | SpaceName | CreateTime | UpdateTime | GraphAddr | Timezone |  
ClientIp |  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+  
| 1651220858102296 | "root" | "basketballplayer" | 2022-04-29T08:27:38.102296 | 2022-04-29T08:50:54.254384 | "127.0.0.1:9669" | 0 | "127.0.0.1" |  
+-----+-----+-----+-----+-----+-----+-----+  
+-----+
```

参数	说明
SessionId	会话 ID，唯一标识一个会话。
UserName	会话的登录用户名称。
SpaceName	用户当前所使用的图空间。刚登录时为空（""）。
CreateTime	会话的创建时间，即用户认证登录的时间。时区为配置文件中 <code>timezone_name</code> 指定的时区。
UpdateTime	用户有执行操作时，会更新此时间。时区为配置文件中 <code>timezone_name</code> 指定的时区。
GraphAddr	会话的 Graph 服务 IP（或主机名）和端口。
Timezone	保留参数，暂无意义。
ClientIp	会话的客户端 IP 或主机名。

SHOW QUERIES

`SHOW QUERIES` 语句可以查看当前 Session 中正在执行的查询请求信息。



如果需要终止查询，请参见[终止查询](#)。

注意事项

- `SHOW LOCAL QUERIES` 从本地缓存获取当前 Session 中查询的状态，几乎没有延迟。
- `SHOW QUERIES` 从 Meta 服务获取所有 Session 中的查询信息。这些信息会根据参数 `session_reclaim_interval_secs` 定义的周期同步到 Meta 服务，因此在客户端获取到的信息可能属于上个同步周期。

语法

```
SHOW [LOCAL] QUERIES;
```

示例

```
trsgraph> SHOW LOCAL QUERIES;
+-----+-----+-----+-----+-----+-----+-----+-----+
| SessionID | ExecutionPlanID | User | Host | StartTime | DurationInUsec | Status | Query |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1625463842921750 | 46 | "root" | ""192.168.x.x":9669" | 2021-07-05T05:44:19.502903 | 0 | "RUNNING" | "SHOW LOCAL QUERIES;" |
+-----+-----+-----+-----+-----+-----+-----+-----+

trsgraph> SHOW QUERIES;
+-----+-----+-----+-----+-----+-----+-----+-----+
| SessionID | ExecutionPlanID | User | Host | StartTime | DurationInUsec | Status | Query |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1625456037718757 | 54 | "user1" | ""192.168.x.x":9669" | 2021-07-05T05:51:08.691318 | 1504502 | "RUNNING" | "MATCH p=(v:player)-[*1..4]-(v2) RETURN v2 AS Friends;" |
+-----+-----+-----+-----+-----+-----+-----+-----+

# 返回耗时 TOP 10 的查询。
trsgraph> SHOW QUERIES | ORDER BY $.DurationInUsec DESC | LIMIT 10;
+-----+-----+-----+-----+-----+-----+-----+-----+
| SessionID | ExecutionPlanID | User | Host | StartTime | DurationInUsec | Status | Query |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1625471375320831 | 98 | "user2" | ""192.168.x.x":9669" | 2021-07-05T07:50:24.461779 | 2608176 | "RUNNING" | "MATCH (v:player)-[*1..4]-(v2) RETURN v2 AS Friends;" |
| 1625456037718757 | 99 | "user1" | ""192.168.x.x":9669" | 2021-07-05T07:50:24.910616 | 2159333 | "RUNNING" | "MATCH (v:player)-[*1..4]-(v2) RETURN v2 AS Friends;" |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

参数说明如下。

参数	说明
SessionID	会话 ID。
ExecutionPlanID	执行计划 ID。
User	执行查询的用户名。
Host	用户连接的服务器地址和端口。
StartTime	执行查询的开始时间。
DurationInUsec	执行查询的持续时长。单位：微秒。
Status	查询的当前状态。
Query	查询语句。

最后更新: 2025年12月19日

SHOW META LEADER

`SHOW META LEADER` 语句显示当前 Meta 集群的 leader 信息。

关于 Meta 服务的详细说明请参见 [Meta 服务](#)。

语法

```
SHOW META LEADER;
```

示例

```
trsgraph> SHOW META LEADER;
+-----+-----+
| Meta Leader | secs from last heart beat |
+-----+-----+
| "127.0.0.1:9559" | 3 |
+-----+-----+
```

参数	说明
<code>Meta Leader</code>	Meta 集群的 leader 信息，包括 leader 所在服务器的 IP（或主机名）和端口。
<code>secs from last heart beat</code>	距离上次心跳的时间间隔。单位：秒。

最后更新: 2025年12月19日

4.5.8 FIND PATH

`FIND PATH` 语句查找指定起始点和目的点之间的路径。



用户可在配置文件 `trigraphd-graphd.conf` 中添加 `num_operator_threads` 参数提高 `FIND PATH` 的查询性能。`num_operator_threads` 的取值为 2~10，该值不能超过 Graph 服务所在机器的 CPU 核心个数，建议设置为 Graph 服务所在机器的 CPU 核心个数。关于配置文件的详细信息，参见 [Graph 服务配置](#)。

语法

```
FIND { SHORTEST | SINGLE SHORTEST | ALL | NOLOOP } PATH [WITH PROP] FROM <vertex_id_list> TO <vertex_id_list>
OVER <edge_type_list> [REVERSELY | BIDIRECT]
[<WHERE clause>] [UPTO <N> {STEP|STEPS}]
YIELD path as <alias>
[| ORDER BY $-.path] [| LIMIT <M>];

<vertex_id_list> ::=
    [vertex_id [, vertex_id] ...]
```

- `SHORTEST`：查找所有最短路径。
- `SINGLE SHORTEST`：查找所有最短路径，随机返回其中一条。
- `ALL`：查找所有路径。
- `NOLOOP`：查找非循环路径。
- `WITH PROP`：展示点和边的属性。不添加本参数则隐藏属性。
- `<vertex_id_list>`：点 ID 列表。多个点用英文逗号 (,) 分隔。支持 `$-` 和 `$var`。
- `<edge_type_list>`：Edge type 列表。多个 Edge type 用英文逗号 (,) 分隔。`*` 表示所有 Edge type。
- `REVERSELY | BIDIRECT`：`REVERSELY` 表示反向，`BIDIRECT` 表示双向。
- `<WHERE clause>`：可以使用 `WHERE` 子句过滤边属性。
- `UPTO <N> {STEP|STEPS}`：路径的最大跳数。默认值为 5。
- `ORDER BY $-.path`：将返回结果进行排序。
- `LIMIT <M>`：指定返回的最大行数。



`FIND PATH` 语句检索的路径类型为 `trail`，即检索的路径只有点可以重复，边不可以重复。详情请参见[路径](#)。

限制

- 指定起始点和目的点的列表后，会返回起始点和目的点所有组合的路径。
- 搜索所有路径时可能会出现循环。
- 使用 `WHERE` 子句时只能过滤边属性，暂不支持过滤点属性，且不支持函数。
- `graphd` 是单进程查询，会占用很多内存。

示例

返回的路径格式类似于 (<vertex_id>)-[:<edge_type_name>@<rank>]->(<vertex_id>)。

```
# 查找并返回 player102 到 team204 的最短路径。
trsggraph> FIND SHORTEST PATH FROM "player102" TO "team204" OVER * YIELD path AS p;
+-----+
| p                |
+-----+
| <("player102")-[:serve@ {}]->("team204")> |
+-----+
```

```
# 查找并返回带属性值的 team204 到 player100 的最短反向路径。
trsggraph> FIND SHORTEST PATH WITH PROP FROM "team204" TO "player100" OVER * REVERSELY YIELD path AS p;
+-----+
| p                |
+-----+
| <("team204" :team{name: "Spurs"})<[:serve@ {end_year: 2016, start_year: 1997}]-("player100" :player{age: 42, name: "Tim Duncan"})> |
+-----+
```

```
# 查找并返回起点为 player100,player130 而终点为 player132,player133 的 18 跳之内双向最短路径。
trsggraph> FIND SHORTEST PATH FROM "player100", "player130" TO "player132", "player133" OVER * BIDIRECT UPTO 18 STEPS YIELD path as p;
+-----+
|
|
|
|
+-----+
| <("player100")<[:follow@ {}]-("player144")<[:follow@ {}]-
("player133")>
| <("player100")-[:serve@ {}]->("team204")<[:serve@ {}]-("player138")-[:serve@ {}]->("team225")<[:serve@ {}]-
("player132")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player112")-[:serve@ {}]->("team204")<[:serve@ {}]-("player114")<[:follow@ {}]-
("player133")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player109")-[:serve@ {}]->("team204")<[:serve@ {}]-("player114")<[:follow@ {}]-
("player133")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player104")-[:serve@ 20182019 {}]->("team204")<[:serve@ {}]-("player114")<[:follow@ {}]-
("player133")>
| ...
|
|
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player112")-[:serve@ {}]->("team204")<[:serve@ {}]-("player138")-[:serve@ {}]->("team225")<
[:serve@ {}]-("player132")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player109")-[:serve@ {}]->("team204")<[:serve@ {}]-("player138")-[:serve@ {}]->("team225")<
[:serve@ {}]-("player132")>
| ...
|
|
+-----+
+
```

```
# 查找 player100 和 player130 分别与 player132 和 player133 之间跳数最长为 18 步的最短路径。
trsggraph> FIND SINGLE SHORTEST PATH FROM "player100", "player130" TO "player132", "player133" OVER * BIDIRECT UPTO 18 STEPS YIELD path as p;
+-----+
|
|
|
|
+-----+
| <("player100")<[:follow@ {}]-("player144")<[:follow@ {}]-
("player133")>
| <("player100")-[:serve@ {}]->("team204")<[:serve@ {}]-("player138")-[:serve@ {}]->("team225")<[:serve@ {}]-
("player132")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player112")-[:serve@ {}]->("team204")<[:serve@ {}]-("player114")<[:follow@ {}]-
("player133")>
| <("player130")-[:serve@ {}]->("team219")<[:serve@ {}]-("player112")-[:serve@ {}]->("team204")<[:serve@ {}]-("player138")-[:serve@ {}]->("team225")<
[:serve@ {}]-("player132")>
|
+-----+
+
```

```
# 查找所有从 player100 到 team204, 并且 degree 为空或者大于等于 0 的路径。
trsggraph> FIND ALL PATH FROM "player100" TO "team204" OVER * WHERE follow.degree is EMPTY or follow.degree >=0 YIELD path AS p;
+-----+
| p                |
+-----+
| <("player100")-[:serve@ {}]->("team204")>
| <("player100")-[:follow@ {}]->("player125")-[:serve@ {}]->("team204")>
| <("player100")-[:follow@ {}]->("player101")-[:serve@ {}]->("team204")>
| ...
|
+-----+
```

```
# 查找所有从 player100 到 team204 无环路径。
trsggraph> FIND NOLOOP PATH FROM "player100" TO "team204" OVER * YIELD path AS p;
+-----+
| p                |
+-----+
| <("player100")-[:serve@ {}]->("team204")>
|
```

```
| <("player100")-[:follow@ {}]->("player125")-[:serve@ {}]->("team204")> |
| <("player100")-[:follow@ {}]->("player101")-[:serve@ {}]->("team204")> |
| <("player100")-[:follow@ {}]->("player101")-[:follow@ {}]->("player125")-[:serve@ {}]->("team204")> |
| <("player100")-[:follow@ {}]->("player101")-[:follow@ {}]->("player102")-[:serve@ {}]->("team204")> |
| ... |
+-----+
```

FAQ

是否支持 WHERE 子句，以实现图遍历过程中的条件过滤？

支持使用 WHERE 子句过滤，但只能过滤边属性，不支持过滤点属性。

如示例中的 WHERE follow.degree is EMPTY or follow.degree >= 0。

.....

最后更新: 2025年12月19日

4.5.9 GET SUBGRAPH

`GET SUBGRAPH` 语句查询并返回一个通过从指定点出发对图进行游走而生成的子图。在 `GET SUBGRAPH` 语句中，用户可以指定游走的步数以及游走所经过的边的类型或方向。

语法

```
GET SUBGRAPH [WITH PROP] [<step_count> {STEP|STEPS}] FROM {<vid>, <vid>...}
[({IN | OUT | BOTH} <edge_type>, <edge_type>...)]
[WHERE <expression> [AND <expression> ...]]
YIELD [VERTICES AS <vertex_alias>] [, EDGES AS <edge_alias>];
```

- `WITH PROP`：展示属性。不添加本参数则隐藏属性。
- `step_count`：指定从起始点开始的跳数，返回从 0 到 `step_count` 跳的子图。必须是非负整数。默认值为 1。
- `vid`：指定起始点 ID。
- `edge_type`：指定 Edge type。可以用 `IN`、`OUT` 和 `BOTH` 来指定起始点上该 Edge type 的方向。默认为 `BOTH`。
- `WHERE`：指定遍历的过滤条件，可以结合布尔运算符 `AND` 使用。
- `YIELD`：定义需要返回的输出。可以仅返回点或边。必须设置别名。



Note

`GET SUBGRAPH` 语句检索的路径类型为 `trail`，即检索的路径只有点可以重复，边不可以重复。详情请参见[路径](#)。

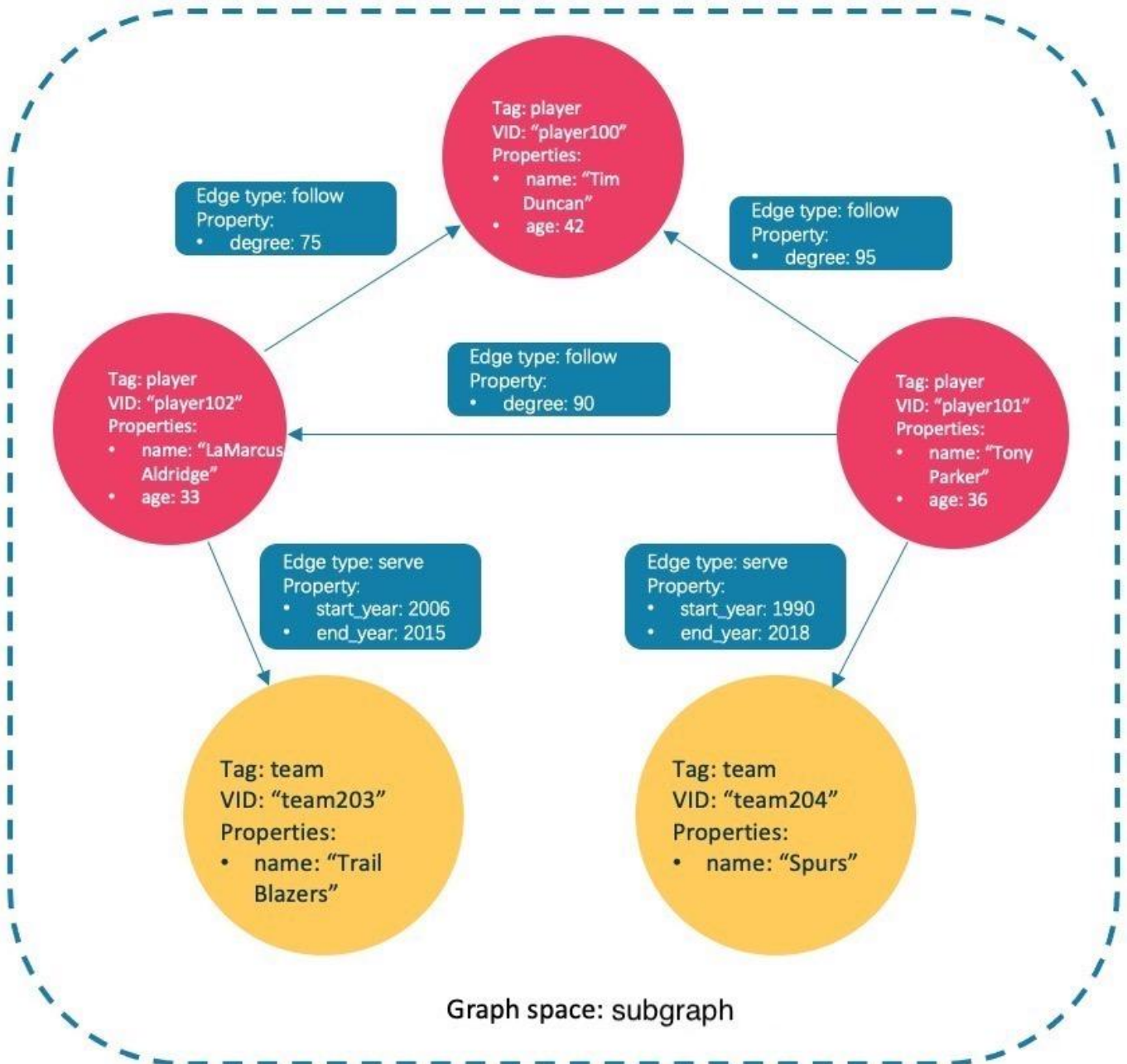
WHERE 语句限制

在 `GET SUBGRAPH` 语句中使用 `WHERE` 子句，注意以下限制：

- 仅支持 `AND` 运算符。
- 仅支持过滤目的点，点的格式为 `$$.tagName.propName`。
- 支持过滤边，边的格式为 `edge_type.propName`。
- 支持数学函数、聚合函数、字符串函数、日期时间函数、列表函数中的通用函数和类型转化函数。
- 不支持聚合函数、Schema 相关函数、条件表达式函数、谓词函数、`geo` 函数和自定义函数，列表函数中除通用函数以外的函数。

示例

以下面的示例图进行演示。



插入测试数据:

```
trsgraph> CREATE SPACE IF NOT EXISTS subgraph(partition_num=15, replica_factor=1, vid_type=fixed_string(30));
trsgraph> USE subgraph;
trsgraph> CREATE TAG IF NOT EXISTS player(name string, age int);
trsgraph> CREATE TAG IF NOT EXISTS team(name string);
trsgraph> CREATE EDGE IF NOT EXISTS follow(degree int);
trsgraph> CREATE EDGE IF NOT EXISTS serve(start_year int, end_year int);
trsgraph> INSERT VERTEX player(name, age) VALUES "player100":("Tim Duncan", 42);
trsgraph> INSERT VERTEX player(name, age) VALUES "player101":("Tony Parker", 36);
trsgraph> INSERT VERTEX player(name, age) VALUES "player102":("LaMarcus Aldridge", 33);
trsgraph> INSERT VERTEX team(name) VALUES "team203":("Trail Blazers"), "team204":("Spurs");
trsgraph> INSERT EDGE follow(degree) VALUES "player101" -> "player100":(95);
trsgraph> INSERT EDGE follow(degree) VALUES "player101" -> "player102":(90);
```



```

trsgraph> INSERT EDGE follow(degree) VALUES "player102" -> "player100":(75);
trsgraph> INSERT EDGE serve(start_year, end_year) VALUES "player101" -> "team204":(1999, 2018), "player102" -> "team203":(2006, 2015);

```

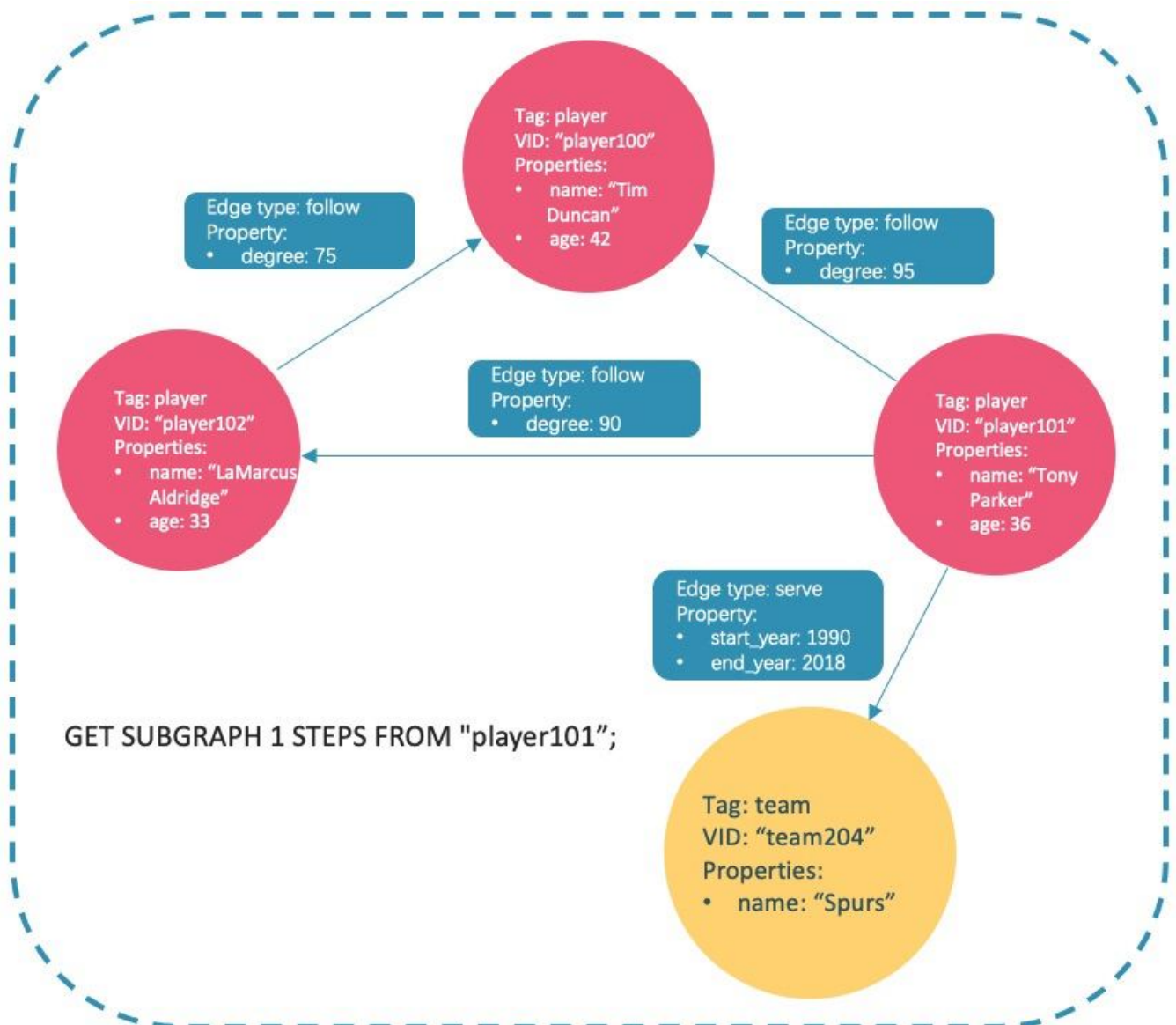
- 查询从点 `player101` 开始、0~1 跳、所有 Edge type 的子图。

```

trsgraph> GET SUBGRAPH 1 STEPS FROM "player101" YIELD VERTICES AS nodes, EDGES AS relationships;
+-----+-----+-----+
| nodes | | |
+-----+-----+-----+
| [{"player101" :player{}}] | [{"serve "player101"->"team204" @0 {}}, {"follow "player101"->"player100" @0 {}}, {"follow "player101"->"player102" @0 {}}] |
+-----+-----+-----+
| [{"team204" :team{}}, {"player100" :player{}}, {"player102" :player{}}] | [{"follow "player102"->"player100" @0 {}}, {"follow "player102"->"team203" @0 {}}] |
+-----+-----+-----+

```

返回的子图如下。



- 查询从点 `player101` 开始、0~1 跳、`follow` 类型的入边的子图。

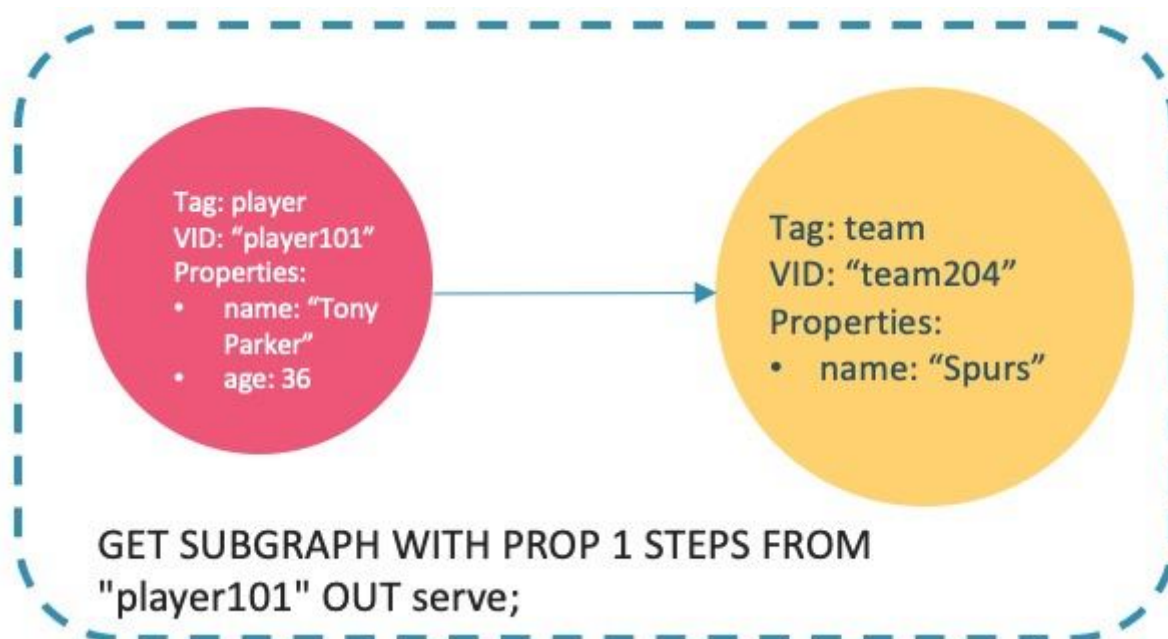
```
trsgraph> GET SUBGRAPH 1 STEPS FROM "player101" IN follow YIELD VERTICES AS nodes, EDGES AS relationships;
+-----+-----+
| nodes | relationships |
+-----+-----+
| [{"player101" :player{}}] | [] |
+-----+-----+
```

因为 `player101` 没有 `follow` 类型的入边。所以仅返回点 `player101`。

- 查询从点 `player101` 开始、0~1 跳、`serve` 类型的出边的子图，同时展示边的属性。

```
trsgraph> GET SUBGRAPH WITH PROP 1 STEPS FROM "player101" OUT serve YIELD VERTICES AS nodes, EDGES AS relationships;
+-----+-----+
| nodes | relationships |
+-----+-----+
| [{"player101" :player{age: 36, name: "Tony Parker"}}] | [[:serve "player101"->"team204" @0 {end_year: 2018, start_year: 1999}]] |
| [{"team204" :team{name: "Spurs"}}] | [] |
+-----+-----+
```

返回的子图如下。



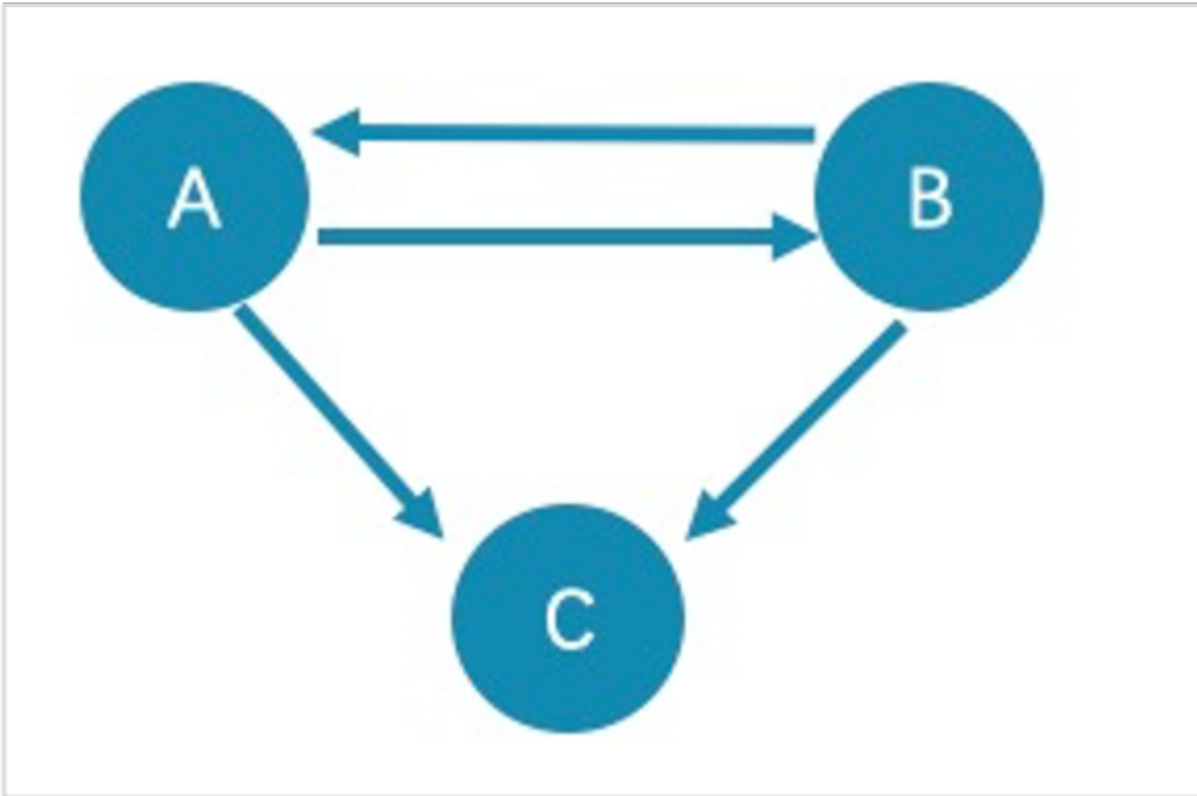
- 查询从点 `player101` 开始、0~2 跳、`follow` 类型边 `degree` 大于 90，年龄大于 30 的子图，同时展示边的属性。

```
trsgraph> GET SUBGRAPH WITH PROP 2 STEPS FROM "player101" \
  WHERE follow.degree > 90 AND $$player.age > 30 \
  YIELD VERTICES AS nodes, EDGES AS relationships;
+-----+-----+
| nodes | relationships |
+-----+-----+
| [{"player101" :player{age: 36, name: "Tony Parker"}}] | [[:follow "player101"->"player100" @0 {degree: 95}]] |
| [{"player100" :player{age: 42, name: "Tim Duncan"}}] | [] |
+-----+-----+
```

FAQ

为什么返回结果中会出现超出 `STEP_COUNT` 跳数之外的关系？

为了展示子图的完整性，会在满足条件的所有点上额外查询一跳。例如下图。



- 用 `GET SUBGRAPH 1 STEPS FROM "A"`; 查询的满足结果的路径是 `A->B`、`B->A` 和 `A->C`，为了子图的完整性，会在满足结果的点上额外查询一跳，即 `B->C`。
- 用 `GET SUBGRAPH 1 STEPS FROM "A" IN follow`; 查询的满足结果的路径是 `B->A`，在满足结果的点上额外查询一跳，即 `A->B`。

如果只是查询满足条件的路径或点，建议使用 `MATCH` 或 `GO` 语句。例如：

```

trsgraph> MATCH p= (v:player) -- (v2) WHERE id(v)=="A" RETURN p;

trsgraph> GO 1 STEPS FROM "A" OVER follow YIELD src(edge),dst(edge);
  
```

为什么返回结果中会出现低于 `STEP_COUNT` 跳数的关系？

查询到没有多余子图数据时会停止查询，且不会返回空值。

```

# 查询从点 player101 开始、0~100 跳、follow 类型的出边的子图。
trsgraph> GET SUBGRAPH 100 STEPS FROM "player101" OUT follow YIELD VERTICES AS nodes, EDGES AS relationships;
+-----+-----+
| nodes                                | relationships                                |
+-----+-----+
| [{"player101" :player{}}]           | [[:follow "player101"->"player100" @0 {}], [:follow "player101"->"player102" @0 {}]] |
| [{"player100" :player{}}, {"player102" :player{}}] | [[:follow "player102"->"player100" @0 {}]] |
+-----+-----+
  
```

最后更新: 2025年12月19日

4.6 子句和选项

4.6.1 GROUP BY

GROUP BY 子句可以用于聚合数据。

openCypher 兼容性

本文操作仅适用于原生 nGQL。

用户也可以使用 openCypher 方式的 count() 函数聚合数据。

```
trsgraph> MATCH (v:player)<-[:follow]-(:player) RETURN v.player.name AS Name, count(*) as cnt ORDER BY cnt DESC;
+-----+-----+
| Name           | cnt |
+-----+-----+
| "Tim Duncan"   | 10  |
| "LeBron James" | 6   |
| "Tony Parker"  | 5   |
| "Chris Paul"   | 4   |
| "Manu Ginobili"| 4   |
+-----+-----+
...
```

语法

GROUP BY 子句可以聚合相同值的行，然后进行计数、排序和计算等操作。

GROUP BY 子句可以在管道符 (|) 之后和 YIELD 子句之前使用。

```
| GROUP BY <var> YIELD <var>, <aggregation_function(var)>
```

aggregation_function() 函数支持 avg()、sum()、max()、min()、count()、collect()、std()。

示例

```
# 查找所有连接到 player100 的点，并根据他们的姓名进行分组，返回姓名的出现次数。
trsgraph> GO FROM "player100" OVER follow BIDIRECT \
  YIELD properties($$).name as Name \
  | GROUP BY $-.Name \
  YIELD $-.Name as Player, count(*) AS Name_Count;
+-----+-----+
| Player           | Name_Count |
+-----+-----+
| "Shaquille O'Neal" | 1          |
| "Tiago Splitter"  | 1          |
| "Manu Ginobili"   | 2          |
| "Boris Diaw"      | 1          |
| "LaMarcus Aldridge" | 1          |
| "Tony Parker"     | 2          |
| "Marco Belinelli" | 1          |
| "Dejounte Murray" | 1          |
| "Danny Green"     | 1          |
| "Aron Baynes"     | 1          |
+-----+-----+

# 查找所有连接到 player100 的点，并根据起始点进行分组，返回 degree 的总和。
trsgraph> GO FROM "player100" OVER follow \
  YIELD src(edge) AS player, properties(edge).degree AS degree \
  | GROUP BY $-.player \
  YIELD sum($-.degree);
+-----+
| sum($-.degree) |
+-----+
| 190             |
+-----+
```

sum() 函数详情请参见内置数学函数。

隐式分组

在上述 nGQL 语句中明确写出 `GROUP BY` 并起到分组字段作用的用法称为 `GROUP BY` 显示用法；而在 openCypher 语句中 `GROUP BY` 的用法是隐式的，即在语句中不用写出 `GROUP BY` 也可起到分组字段的作用。nGQL 语句中显示地 `GROUP BY` 用法与 openCypher 语句中的隐式地 `GROUP BY` 用法相同，并且 nGQL 语句兼容 openCypher 的用法，即也支持隐式地使用 `GROUP BY`。有关 `GROUP BY` 的隐式用法，请参见[Stack Overflow](#)。

例如：查询 34 岁以上的球员中完全重叠服役的区间。

```
trsgraph> LOOKUP ON player WHERE player.age > 34 YIELD id(vertex) AS v | \
GO FROM $-.v OVER serve YIELD serve.start_year AS start_year, serve.end_year AS end_year | \
YIELD $-.start_year, $-.end_year, count(*) AS count | \
ORDER BY $-.count DESC | LIMIT 5;
```

\$-.start_year	\$-.end_year	count
2018	2019	3
2007	2012	2
1998	2004	2
2017	2018	2
2010	2011	2

最后更新: 2025年12月19日

4.6.2 LIMIT

`LIMIT` 子句限制输出结果的行数。`LIMIT` 在原生 nGQL 语句和 openCypher 兼容语句中的用法有所不同。

- 在原生 nGQL 语句中，一般需要在 `LIMIT` 子句前使用管道符，可以直接在 `LIMIT` 语句后设置或者省略偏移量参数。
- 在 openCypher 兼容语句中，不允许在 `LIMIT` 子句前使用管道符，可以使用 `SKIP` 指明偏移量。



在原生 nGQL 或 openCypher 方式中使用 `LIMIT` 时，使用 `ORDER BY` 子句限制输出顺序非常重要，否则会输出一个不可预知的子集。

原生 nGQL 语句中的 LIMIT

在原生 nGQL 中，`LIMIT` 有通用语法和 `GO` 语句中的专属语法。

原生 NGQL 中的通用 LIMIT 语法

原生 nGQL 中的通用 `LIMIT` 语法与 `SQL` 中的 `LIMIT` 原理相同。`LIMIT` 子句接收一个或两个参数，参数的值必须是非负整数，且必须用在管道符之后。语法和说明如下：

... LIMIT [<offset>,,] <number_rows>;	
参数	说明
<code>offset</code>	偏移量，即定义从哪一行开始返回。索引从 0 开始。默认值为 0，表示从第一行开始返回。
<code>number_rows</code>	返回的总行数。

示例：

```
# 从结果中返回最前面的 3 行数据。
trsgraph> LOOKUP ON player YIELD id(vertex) \
    LIMIT 3;
+-----+
| id(VERTEX) |
+-----+
| "player100" |
| "player101" |
| "player102" |
+-----+

# 从排序后结果中返回第 2 行开始的 3 行数据。
trsgraph> GO FROM "player100" OVER follow REVERSELY \
    YIELD properties($$).name AS Friend, properties($$).age AS Age \
    ORDER BY $-.Age, $-.Friend \
    LIMIT 1, 3;
+-----+-----+
| Friend          | Age |
+-----+-----+
| "Danny Green"   | 31  |
| "Aron Baynes"   | 32  |
| "Marco Belinelli" | 32  |
+-----+-----+
```

GO 语句中的 LIMIT

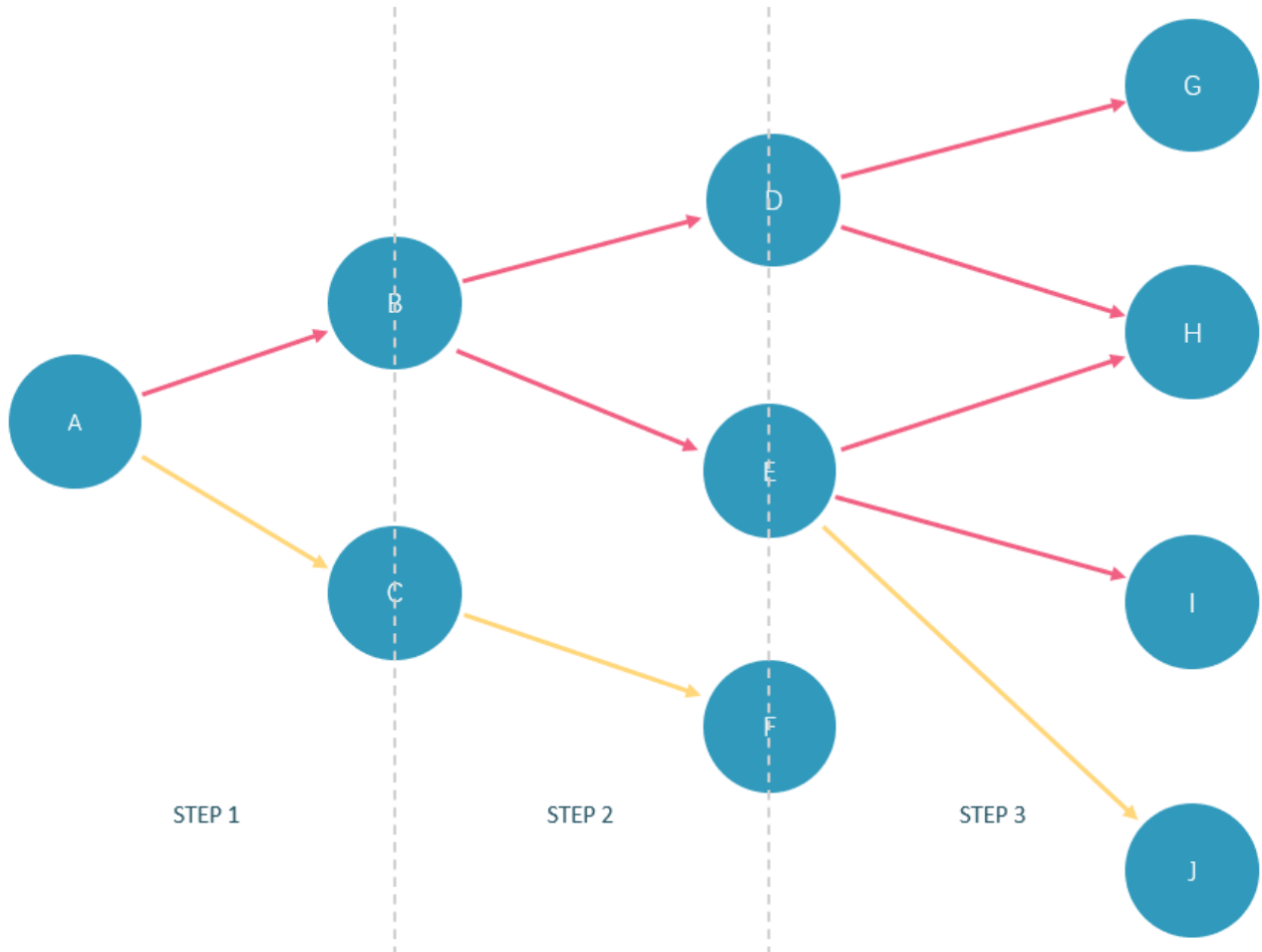
`GO` 语句中的 `LIMIT` 除了支持原生 nGQL 中的通用语法外，还支持根据边限制输出结果数量。

语法：

```
<go_statement> LIMIT <limit_list>;
```

`limit_list` 是一个列表，列表中的元素必须为自然数，且元素数量必须与 `GO` 语句中的 `STEPS` 的最大数相同。下文以 `GO 1 TO 3 STEPS FROM "A" OVER * LIMIT <limit_list>` 为例详细介绍 `LIMIT` 的这种用法。

- 列表 `limit_list` 必须包含 3 个自然数元素，例如 `GO 1 TO 3 STEPS FROM "A" OVER * LIMIT [1,2,4]`。
- `LIMIT [1,2,4]` 中的 1 表示系统在第一步时自动选择 1 条边继续遍历，2 表示在第二步时选择 2 条边继续遍历，4 表示在第三步时选择 4 条边继续遍历。
- 因为 `GO 1 TO 3 STEPS` 表示返回第一到第三步的所有遍历结果，因此下图中所有红色边和它们的原点与目的点都会被这条 `GO` 语句匹配上，而黄色边表示 `GO` 语句遍历时没有选择的路径。如果不是 `GO 1 TO 3 STEPS` 而是 `GO 3 STEPS`，则只会匹配上第三步的红色边和它们两端的点。



在 `basketballplayer` 数据集中的执行示例如下：

```
trsgraph> GO 3 STEPS FROM "player100" \
  OVER * \
  YIELD properties($$).name AS NAME, properties($$).age AS Age \
  LIMIT [3,3,3];
+-----+
| NAME          | Age    |
+-----+
| "Tony Parker" | 36     |
| "Manu Ginobili" | 41    |
| "Spurs"       | __NULL__ |
+-----+

trsgraph> GO 3 STEPS FROM "player102" OVER * BIDIRECT\
  YIELD dst(edge) \
  LIMIT [rand32(5),rand32(5),rand32(5)];
+-----+
| dst(EDGE)     |
+-----+
| "player100"   |
| "player100"   |
+-----+
```

openCypher 兼容语句中的 LIMIT

在 MATCH 等 openCypher 兼容语句中使用 LIMIT 不需要加管道符。语法和说明如下：

```
... [SKIP <offset>] [LIMIT <number_rows>];
```

参数	说明
offset	偏移量，即定义从哪一行开始返回。索引从 0 开始。默认值为 0，表示从第一行开始返回。
number_rows	返回的总行数量。

offset 和 number_rows 可以使用表达式，但是表达式的结果必须是非负整数。



两个整数组成的分数表达式会自动向下取整。例如 8/6 向下取整为 1。

单独使用 LIMIT

LIMIT 可以单独使用，返回指定数量的结果。

```
trsgraph> MATCH (v:player) RETURN v.player.name AS Name, v.player.age AS Age \
ORDER BY Age LIMIT 5;
+-----+
| Name          | Age |
+-----+
| "Luka Doncic"  | 20  |
| "Ben Simmons" | 22  |
| "Kristaps Porzingis" | 23  |
| "Giannis Antetokounmpo" | 24  |
| "Kyle Anderson" | 25  |
+-----+
```

单独使用 SKIP

SKIP 可以单独使用，用于设置偏移量，返回指定位置之后的数据。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) --> (v2) \
RETURN v2.player.name AS Name, v2.player.age AS Age \
ORDER BY Age DESC SKIP 1;
+-----+
| Name          | Age |
+-----+
| "Manu Ginobili" | 41  |
| "Tony Parker"  | 36  |
+-----+

trsgraph> MATCH (v:player{name:"Tim Duncan"}) --> (v2) \
RETURN v2.player.name AS Name, v2.player.age AS Age \
ORDER BY Age DESC SKIP 1+1;
+-----+
| Name          | Age |
+-----+
| "Tony Parker"  | 36  |
+-----+
```

同时使用 SKIP 与 LIMIT

同时使用 SKIP 与 LIMIT 可以返回从指定位置开始的指定数量的数据。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) --> (v2) \
RETURN v2.player.name AS Name, v2.player.age AS Age \
ORDER BY Age DESC SKIP 1 LIMIT 1;
+-----+
| Name          | Age |
+-----+
| "Manu Ginobili" | 41  |
+-----+
```


4.6.3 SAMPLE

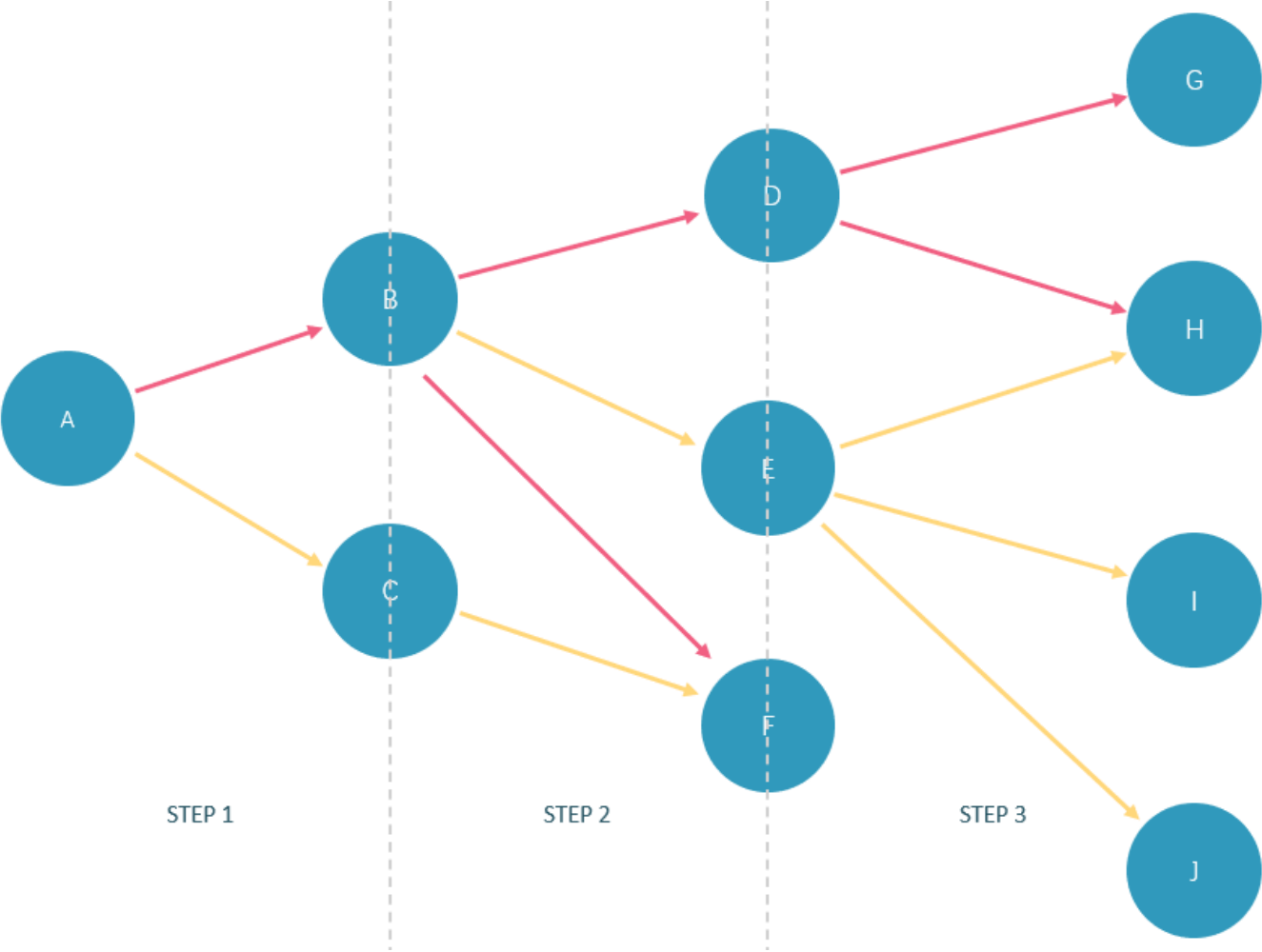
`SAMPLE` 子句用于在结果集中均匀取样并返回指定数量的数据。

`SAMPLE` 仅能在 `GO` 语句中使用，语法如下：

```
<go_statement> SAMPLE <sample_list>;
```

`sample_list` 是一个列表，列表中的元素必须为自然数，且元素数量必须与 `GO` 语句中的 `STEPS` 的最大数相同。下文以 `GO 1 TO 3 STEPS FROM "A" OVER * SAMPLE <sample_list>` 为例详细介绍 `SAMPLE` 的用法。

- 列表 `sample_list` 必须包含 3 个自然数元素，例如 `GO 1 TO 3 STEPS FROM "A" OVER * SAMPLE [1,2,4]`。
- `SAMPLE [1,2,4]` 中的 1 表示系统在第一步时自动选择 1 条边继续遍历，2 表示在第二步时选择 2 条边继续遍历，4 表示在第三步时选择 4 条边继续遍历。如果某一步没有匹配的边或者匹配到的边数量小于指定数量，则按实际数量返回。
- 因为 `GO 1 TO 3 STEPS` 表示返回第一到第三步的所有遍历结果，因此下图中所有红色边和它们的原点与目的点都会被这条 `GO` 语句匹配上，而黄色边表示 `GO` 语句遍历时没有选择的路径。如果不是 `GO 1 TO 3 STEPS` 而是 `GO 3 STEPS`，则只会匹配上第三步的红色边和它们两端的点。



在 `basketballplayer` 数据集中的执行示例如下：

```
trigraph> GO 3 STEPS FROM "player100" \
OVER * \
YIELD properties($$).name AS NAME, properties($$).age AS Age \
SAMPLE [1,2,3];
+-----+
| NAME      | Age  |
+-----+
| "Tony Parker" | 36   |
| "Manu Ginobili" | 41   |
```

```
| "Spurs"          | __NULL__ |
+-----+-----+

trsgraph> GO 1 TO 3 STEPS FROM "player100" \
OVER * \
YIELD properties($$).name AS NAME, properties($$).age AS Age \
SAMPLE [2,2,2];
+-----+-----+
| NAME          | Age      |
+-----+-----+
| "Manu Ginobili" | 41       |
| "Spurs"         | __NULL__ |
| "Tim Duncan"   | 42       |
| "Spurs"         | __NULL__ |
| "Manu Ginobili" | 41       |
| "Spurs"         | __NULL__ |
+-----+-----+
```

最后更新: 2025年12月19日

4.6.4 ORDER BY

`ORDER BY` 子句指定输出结果的排序规则。

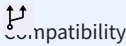
- 在原生 nGQL 中，必须在 `YIELD` 子句之后使用管道符 `()` 和 `ORDER BY` 子句。
- 在 openCypher 方式中，不允许使用管道符。在 `RETURN` 子句之后使用 `ORDER BY` 子句。

排序规则分为如下两种：

- `ASC`（默认）：升序。
- `DESC`：降序。

原生 nGQL 语法

```
<YIELD clause>
| ORDER BY <expression> [ASC | DESC] [, <expression> [ASC | DESC] ...];
```



原生 nGQL 语法中，`ORDER BY` 命令后必须使用引用符 `$-.`。但在 2.5.0 之前的版本中不需要。

示例

```
trsgraph> FETCH PROP ON player "player100", "player101", "player102", "player103" \
  YIELD properties(vertex).age AS age, properties(vertex).name AS name \
  | ORDER BY $-.age ASC, $-.name DESC;

+-----+-----+
| age | name |
+-----+-----+
| 32 | "Rudy Gay" |
| 33 | "LaMarcus Aldridge" |
| 36 | "Tony Parker" |
| 42 | "Tim Duncan" |
+-----+-----+

trsgraph> $var = GO FROM "player100" OVER follow \
  YIELD dst(edge) AS dst; \
  ORDER BY $var.dst DESC;

+-----+
| dst |
+-----+
| "player125" |
| "player101" |
+-----+
```

OpenCypher 方式语法

```
<RETURN clause>
ORDER BY <expression> [ASC | DESC] [, <expression> [ASC | DESC] ...];
```

示例

```
trsgraph> MATCH (v:player) RETURN v.player.name AS Name, v.player.age AS Age \
  ORDER BY Name DESC;

+-----+-----+
| Name | Age |
+-----+-----+
| "Yao Ming" | 38 |
| "Vince Carter" | 42 |
| "Tracy McGrady" | 39 |
| "Tony Parker" | 36 |
| "Tim Duncan" | 42 |
+-----+-----+
...

# 首先以年龄排序，如果年龄相同，再以姓名排序。
trsgraph> MATCH (v:player) RETURN v.player.age AS Age, v.player.name AS Name \
  ORDER BY Age DESC, Name ASC;

+-----+-----+
| Age | Name |
+-----+-----+
```

```
| 47 | "Shaquille O'Neal" |
| 46 | "Grant Hill"      |
| 45 | "Jason Kidd"       |
| 45 | "Steve Nash"       |
+-----+
...
```

NULL 值的排序

升序排列时，会在输出的最后列出 NULL 值，降序排列时，会在输出的开头列出 NULL 值。

```
trsgaph> MATCH (v:player{name:"Tim Duncan"}) --> (v2) \
          RETURN v2.player.name AS Name, v2.player.age AS Age \
          ORDER BY Age;
+-----+
| Name      | Age  |
+-----+
| "Tony Parker" | 36   |
| "Manu Ginobili" | 41   |
| __NULL__    | __NULL__ |
+-----+

trsgaph> MATCH (v:player{name:"Tim Duncan"}) --> (v2) \
          RETURN v2.player.name AS Name, v2.player.age AS Age \
          ORDER BY Age DESC;
+-----+
| Name      | Age  |
+-----+
| __NULL__    | __NULL__ |
| "Manu Ginobili" | 41   |
| "Tony Parker" | 36   |
+-----+
```

最后更新: 2025年12月19日

4.6.5 RETURN

`RETURN` 子句定义了 nGQL 查询的输出结果。如果需要返回多个字段，用英文逗号 (,) 分隔。

`RETURN` 可以引导子句或语句：

- `RETURN` 子句可以用于 nGQL 中的 openCypher 方式语句中，例如 `MATCH` 或 `UNWIND`。
- `RETURN` 可以单独使用，输出表达式的结果。

openCypher 兼容性

本文操作仅适用于 nGQL 中的 openCypher 方式。关于原生 nGQL 如何定义输出结果，请参见 `YIELD`。

`RETURN` 不支持如下 openCypher 功能：

- 使用不在英文字母表中的字符作为变量名。例如：

```
MATCH (`点 1`:player) \
RETURN `点 1`;
```

- 设置一个模式，并返回该模式匹配的所有元素。例如：

```
MATCH (v:player) \
RETURN (v)-[e]->(v2);
```

历史版本兼容性

- 在 nGQL 1.x 中，`RETURN` 适用于原生 nGQL，语法为 `RETURN <var_ref> IF <var_ref> IS NOT NULL`。
- 从 nGQL 2.0 开始，`RETURN` 不适用于原生 nGQL。

Map 顺序说明

`RETURN` 返回 Map 时，Key 的顺序是未定义的。

```
trsgraph> RETURN {age: 32, name: "Marco Belinelli"};
+-----+
| {age:32,name:"Marco Belinelli"} |
+-----+
| {age: 32, name: "Marco Belinelli"} |
+-----+

trsgraph> RETURN {zage: 32, name: "Marco Belinelli"};
+-----+
| {zage:32,name:"Marco Belinelli"} |
+-----+
| {name: "Marco Belinelli", zage: 32} |
+-----+
```

返回点或边

使用 `RETURN {<vertex_name> | <edge_name>}` 返回点或边的所有信息。

```
// 返回点
trsgraph> MATCH (v:player) \
    RETURN v;
+-----+
| v |
+-----+
| ("player104" :player{age: 32, name: "Marco Belinelli"}) |
| ("player107" :player{age: 32, name: "Aron Baynes"}) |
| ("player116" :player{age: 34, name: "LeBron James"}) |
| ("player120" :player{age: 29, name: "James Harden"}) |
| ("player125" :player{age: 41, name: "Manu Ginobili"}) |
+-----+
...
```

```
// 返回边
trsgraph> MATCH (v:player)-[e]->() \
    RETURN e;
+-----+
| e      |
+-----+
| [:follow "player104"->"player100" @0 {degree: 55}] |
| [:follow "player104"->"player101" @0 {degree: 50}] |
| [:follow "player104"->"player105" @0 {degree: 60}] |
| [:serve "player104"->"team200" @0 {end_year: 2009, start_year: 2007}] |
| [:serve "player104"->"team208" @0 {end_year: 2016, start_year: 2015}] |
+-----+
...
```

返回点 ID

使用 `id()` 函数返回点 ID。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) \
    RETURN id(v);
+-----+
| id(v) |
+-----+
| "player100" |
+-----+
```

返回 Tag

使用 `labels()` 函数返回点上的 Tag 列表。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) \
    RETURN labels(v);
+-----+
| labels(v) |
+-----+
| ["player"] |
+-----+
```

返回列表 `labels(v)` 中的第 N 个元素，可以使用 `labels(v)[n-1]`。例如下面示例使用 `labels(v)[0]` 检索第一个元素。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) \
    RETURN labels(v)[0];
+-----+
| labels(v)[0] |
+-----+
| "player"      |
+-----+
```

返回属性

返回点的属性时，必需指定属性所属的 Tag，因为点可以有多个 Tag，并且同一个属性名可以在不同的 Tag 上出现。

支持指定点的 Tag 返回该 Tag 的所有属性；也支持指定点的 Tag 和某个属性名，返回该 Tag 的指定属性。

```
// 返回点的属性
trsgraph> MATCH (v:player) \
    RETURN v.player, v.player.name, v.player.age \
    LIMIT 3;
+-----+-----+-----+
| v.player | v.player.name | v.player.age |
+-----+-----+-----+
| {age: 33, name: "LaMarcus Aldridge"} | "LaMarcus Aldridge" | 33 |
| {age: 25, name: "Kyle Anderson"} | "Kyle Anderson" | 25 |
| {age: 40, name: "Kobe Bryant"} | "Kobe Bryant" | 40 |
+-----+-----+-----+
```

返回边的属性时，无需指定属性所属的 Edge type，因为边只能有一个 Edge type。

```
// 返回边的属性
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->() \
    RETURN e.start_year, e.degree \
    LIMIT 3;
+-----+-----+
| e.start_year | e.degree |
+-----+-----+
| __NULL__ | 95 |
| __NULL__ | 95 |
+-----+-----+
```

```
| 1997          | __NULL__ |
+-----+-----+
```

使用 `properties()` 函数返回点或边的所有属性。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) \
RETURN properties(v2);
+-----+
| properties(v2) |
+-----+
| {name: "Spurs"} |
| {age: 36, name: "Tony Parker"} |
| {age: 41, name: "Manu Ginobili"} |
+-----+
```

返回 Edge type

使用 `type()` 函数返回匹配的 Edge type。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[e]->() \
RETURN DISTINCT type(e);
+-----+
| type(e) |
+-----+
| "serve" |
| "follow" |
+-----+
```

返回路径

使用 `RETURN <path_name>` 返回匹配路径的所有信息。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[*3]->() \
RETURN p;
+-----+
+
|
| p
|
+-----+
+
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 90}]->("player102" :player{age: 33, name: "LaMarcus Aldridge"})-[:serve@0 {end_year: 2019, start_year: 2015}]->("team204" :team{name: "Spurs"})> |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 90}]->("player102" :player{age: 33, name: "LaMarcus Aldridge"})-[:serve@0 {end_year: 2015, start_year: 2006}]->("team203" :team{name: "Trail Blazers"})> |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 90}]->("player102" :player{age: 33, name: "LaMarcus Aldridge"})-[:follow@0 {degree: 75}]->("player101" :player{age: 36, name: "Tony Parker"})> |
+-----+
+
...
```

返回路径中的点

使用 `nodes()` 函数返回路径中的所有点。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) \
RETURN nodes(p);
+-----+
| nodes(p) |
+-----+
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"team204" :team{name: "Spurs"}}] |
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"player101" :player{age: 36, name: "Tony Parker"}}] |
| [{"player100" :player{age: 42, name: "Tim Duncan"}}, {"player125" :player{age: 41, name: "Manu Ginobili"}}] |
+-----+
```

返回路径中的边

使用 `relationships()` 函数返回路径中的所有边。

```
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[]->(v2) \
RETURN relationships(p);
+-----+
| relationships(p) |
+-----+
| [{"serve" "player100"->"team204" @0 {end_year: 2016, start_year: 1997}}] |
| [{"follow" "player100"->"player101" @0 {degree: 95}}] |
| [{"follow" "player100"->"player125" @0 {degree: 95}}] |
+-----+
```

返回路径长度

使用 `length()` 函数检索路径的长度。

```

trsgraph> MATCH p=(v:player{name:"Tim Duncan"})-[*..2]->(v2) \
RETURN p AS Paths, length(p) AS Length;
+-----+
|
| Paths
| Length |
+-----+
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:serve@0 {end_year: 2016, start_year: 1997}]->("team204" :team{name:
"Spurs"})> | 1 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony
Parker"})> | 1 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu
Ginobili"})> | 1 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:serve@0 {end_year:
2018, start_year: 1999}]->("team204" :team{name: "Spurs"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:serve@0 {end_year:
2019, start_year: 2018}]->("team215" :team{name: "Hornets"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 95}]-
>("player100" :player{age: 42, name: "Tim Duncan"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 90}]-
>("player102" :player{age: 33, name: "LaMarcus Aldridge"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})-[:follow@0 {degree: 95}]-
>("player125" :player{age: 41, name: "Manu Ginobili"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})-[:serve@0 {end_year:
2018, start_year: 2002}]->("team204" :team{name: "Spurs"})> | 2 |
| <("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})-[:follow@0 {degree:
90}]->("player100" :player{age: 42, name: "Tim Duncan"})> | 2 |
+-----+

```

返回所有元素

使用星号 (*) 返回匹配模式中的所有元素。

```

trsgraph> MATCH (v:player{name:"Tim Duncan"}) \
RETURN *;
+-----+
| v |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) |
+-----+

trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->(v2) \
RETURN *;
+-----+
| v | e |
+-----+
| v2 |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) | [:follow "player100"->"player101" @0 {degree: 95}] | ("player101" :player{age: 36,
name: "Tony Parker"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | [:follow "player100"->"player125" @0 {degree: 95}] | ("player125" :player{age: 41,
name: "Manu Ginobili"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | [:serve "player100"->"team204" @0 {end_year: 2016, start_year: 1997}] | ("team204" :team{name:
"Spurs"}) |
+-----+

```

重命名字段

使用语法 `AS <alias>` 重命名输出结果中的字段。

```

trsgraph> MATCH (v:player{name:"Tim Duncan"})-[:serve]->(v2) \
RETURN v2.team.name AS Team;
+-----+
| Team |
+-----+
| "Spurs" |
+-----+

trsgraph> RETURN "Amber" AS Name;
+-----+
| Name |
+-----+
| "Amber" |
+-----+

```


返回不存在的属性

如果匹配的结果中，某个属性不存在，会返回 `NULL`。

```
trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->(v2) \
    RETURN v2.player.name, type(e), v2.player.age;
+-----+
| v2.player.name | type(e) | v2.player.age |
+-----+
| "Manu Ginobili" | "follow" | 41             |
| __NULL__         | "serve"  | __NULL__       |
| "Tony Parker"   | "follow" | 36             |
+-----+
```

返回表达式结果

`RETURN` 语句可以返回字面量、函数或谓词等表达式的结果。

```
trsgraph> MATCH (v:player{name:"Tony Parker"})-->(v2:player) \
    RETURN DISTINCT v2.player.name, "Hello"+" graphs!", v2.player.age > 35;
+-----+
| v2.player.name | ("Hello"+" graphs!") | (v2.player.age>35) |
+-----+
| "LaMarcus Aldridge" | "Hello graphs!"       | false              |
| "Tim Duncan"         | "Hello graphs!"       | true               |
| "Manu Ginobili"      | "Hello graphs!"       | true               |
+-----+

trsgraph> RETURN 1+1;
+-----+
| (1+1) |
+-----+
| 2      |
+-----+

trsgraph> RETURN 1- -1;
+-----+
| (1-- (1)) |
+-----+
| 2          |
+-----+

trsgraph> RETURN 3 > 1;
+-----+
| (3>1) |
+-----+
| true  |
+-----+

trsgraph> RETURN 1+1, rand32(1, 5);
+-----+
| (1+1) | rand32(1,5) |
+-----+
| 2      | 1            |
+-----+
```

返回唯一字段

使用 `DISTINCT` 可以删除结果集中的重复字段。

```
# 未使用 DISTINCT。
trsgraph> MATCH (v:player{name:"Tony Parker"})--(v2:player) \
    RETURN v2.player.name, v2.player.age;
+-----+
| v2.name | v2.age |
+-----+
| "Tim Duncan" | 42 |
| "LaMarcus Aldridge" | 33 |
| "Marco Belinelli" | 32 |
| "Boris Diaw" | 36 |
| "Dejounte Murray" | 29 |
| "Tim Duncan" | 42 |
| "LaMarcus Aldridge" | 33 |
| "Manu Ginobili" | 41 |
+-----+

# 使用 DISTINCT。
trsgraph> MATCH (v:player{name:"Tony Parker"})--(v2:player) \
    RETURN DISTINCT v2.player.name, v2.player.age;
+-----+
| v2.name | v2.age |
+-----+
| "Tim Duncan" | 42 |
| "LaMarcus Aldridge" | 33 |
+-----+
```

	"Marco Belinelli"		32	
	"Boris Diaw"		36	
	"Dejounte Murray"		29	
	"Manu Ginobili"		41	
+-----+				

最后更新: 2025年12月19日

4.6.6 TTL

TTL（Time To Live）是一个定义数据生存期的机制。当数据达到其预设的生存时间后，它会被自动从图数据库中删除。这种功能尤其适合于只需要暂时存储的数据，例如临时会话或缓存数据。

openCypher 兼容性

本文操作仅适用于原生 nGQL。

注意事项

- 不能修改带有 TTL 选项的属性的 Schema。
- TTL 和 INDEX 共存问题：
 - 如果一个 Tag/Edge type 的其中一属性已有 INDEX，则不能为其设置 TTL，也不能为该 Tag 的其他属性设置 TTL。
 - 如果已有 TTL，可以再添加 INDEX。

TTL 选项

nGQL 支持的 TTL 选项如下。

选项	说明
<code>ttl_col</code>	指定一个现有的要设置存活时间的属性。属性的数据类型必须是 <code>int</code> 或者 <code>timestamp</code> 。
<code>ttl_duration</code>	指定时间戳差值，默认单位：秒。时间戳差值必须为 64 位非负整数。属性值和时间戳差值之和如果小于当前时间戳，属性就会过期。如果 <code>ttl_duration</code> 为 0，属性永不过期。 可在配置文件 <code>trsgraph-storaged.conf</code>（默认路径 <code>/usr/local/nightly/etc/</code>）中设置 <code>ttl_use_ms</code> 为 <code>true</code> 将默认单位设为毫秒。

Warning

- 在设置 `ttl_use_ms` 为 `true` 前，请确保没有为属性设置 TTL，否则会因为过期时间缩短，导致数据被错误地删除。
- 在设置 `ttl_use_ms` 为 `true` 后，即设置 `ttl_duration` 的默认单位为毫秒后，`ttl_col` 的默认单位仍然为秒，它的数据类型必须是 `int`，并且需要手动转换属性值为毫秒。例如设置 `ttl_col` 为 `a`，则需要将 `a` 的值转换为毫秒，如当 `a` 的值为 `now()`，则需要将 `a` 的值设置为 `now() * 1000`。

使用 TTL 选项

在使用 TTL 功能之前，必须先创建一个时间戳或整数属性，并在 TTL 选项中指定了它。数据库不会自动为您创建或管理这个时间戳属性。

在插入时间戳或整数属性的值时，建议使用 `now()` 函数或者当前时间戳来代表当前时间。

TAG 或 EDGE TYPE 已存在

如果 Tag 和 Edge type 已经创建，请使用 `ALTER` 语句更新 Tag 或 Edge type。

```
# 创建 Tag。
trsgraph> CREATE TAG IF NOT EXISTS t1 (a timestamp);

# ALTER 修改 Tag，添加 TTL 选项。
trsgraph> ALTER TAG t1 TTL_COL = "a", TTL_DURATION = 5;

# 插入点，插入后 5 秒过期。
trsgraph> INSERT VERTEX t1(a) VALUES "101":(now());
```

TAG 或 EDGE TYPE 不存在

创建 Tag 或 Edge type 时可以同时设置 TTL 选项。详情请参见 [CREATE TAG](#) 和 [CREATE EDGE](#)。

```
# 创建 Tag 并设置 TTL 选项。
trsgraph> CREATE TAG IF NOT EXISTS t2(a int, b int, c string) TTL_DURATION= 100, TTL_COL = "a";

# 插入点。过期时间戳为 1648197238 (1648197138 + 100)。
trsgraph> INSERT VERTEX t2(a, b, c) VALUES "102":(1648197138, 30, "Hello");
```

属性过期



Caution

- 当为一个 Tag 或 Edge type 的属性设置 TTL 并该属性的值为 `NULL` 时，TTL 功能不会生效，即该属性永不过期。
- 如果为一个 Tag 或 Edge type 新增默认值为 `now()` 的属性并且该属性设置了 TTL，该 Tag 或 Edge type 相关的历史数据不会过期，因为历史数据的该属性值为当前时间戳。

点属性过期

点属性过期有如下影响：

- 如果一个点仅有一个 Tag，点上的一个属性过期，点也会过期。
- 如果一个点有多个 Tag，点上的一个属性过期，和该属性相同 Tag 的其他属性也会过期，但是点不会过期，点上其他 Tag 的属性保持不变。

边属性过期

因为一条边仅有一个 Edge type，边上的一个属性过期，边也会过期。

过期处理

属性过期后，对应的过期数据仍然存储在硬盘上，但是查询时会过滤过期数据。

TRSGraph 自动删除过期数据后，会在下一次 **Compaction** 过程中回收硬盘空间。



Note

如果关闭 **TTL 选项**，上一次 **Compaction** 之后的过期数据将可以被查询到。

删除存活时间

删除存活时间可以使用如下几种方法：

- 删除设置存活时间的属性。

```
trsgraph> ALTER TAG t1 DROP (a);
```

- 设置 `t1_col` 为空字符串。

```
trsgraph> ALTER TAG t1 TTL_COL = "";
```

- 设置 `t1_duration` 为 0。本操作可以保留 TTL 选项，属性永不过期，且属性的 Schema 无法修改。

```
trsgraph> ALTER TAG t1 TTL_DURATION = 0;
```

最后更新: 2025年12月19日

4.6.7 WHERE

`WHERE` 子句可以根据条件过滤输出结果。

`WHERE` 子句通常用于如下查询：

- 原生 nGQL，例如 `GO` 和 `LOOKUP` 语句。
- openCypher 方式，例如 `MATCH` 和 `WITH` 语句。

openCypher 兼容性

过滤 Rank 是原生 nGQL 功能。如需在 openCypher 兼容语句中直接获取 Rank 值，可以使用 `rank()` 函数，例如 `MATCH (:player)-[e:follow]->()`
`RETURN rank(e);`

基础用法



下文示例中的 `$$`、`$$^` 等是引用符号，详情请参见引用符。

用布尔运算符定义条件

在 `WHERE` 子句中使用布尔运算符 `NOT`、`AND`、`OR` 和 `XOR` 定义条件。关于运算符的优先级，请参见运算符优先级。

```
trsgaph> MATCH (v:player) \
  WHERE v.player.name == "Tim Duncan" \
  XOR (v.player.age < 30 AND v.player.name == "Yao Ming") \
  OR NOT (v.player.name == "Yao Ming" OR v.player.name == "Tim Duncan") \
  RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
| "Danny Green" | 31           |
| "Tiago Splitter" | 34          |
| "David West" | 38           |
...
```

```
trsgaph> GO FROM "player100" \
  OVER follow \
  WHERE properties(edge).degree > 90 \
  OR properties($$).age != 33 \
  AND properties($$).name != "Tony Parker" \
  YIELD properties($$);
+-----+
| properties($$) |
+-----+
| {age: 41, name: "Manu Ginobili"} |
+-----+
```

过滤属性

在 `WHERE` 子句中使用点或边的属性定义条件。

- 过滤点属性：

```
trsgaph> MATCH (v:player)-[e]->(v2) \
  WHERE v2.player.age < 25 \
  RETURN v2.player.name, v2.player.age;

+-----+-----+
| v2.player.name | v2.player.age |
+-----+-----+
| "Ben Simmons"  | 22            |
| "Luka Doncic"  | 20            |
| "Kristaps Porzingis" | 23          |
+-----+-----+
```

```
trsgaph> GO FROM "player100" OVER follow \
  WHERE $^.player.age >= 42 \
  YIELD dst(edge);

+-----+
| dst(EDGE) |
+-----+
| "player101" |
| "player125" |
+-----+
```

- 过滤边属性：

```
trsgaph> MATCH (v:player)-[e]->() \
  WHERE e.start_year < 2000 \
  RETURN DISTINCT v.player.name, v.player.age;

+-----+-----+
| v.player.name | v.player.age |
+-----+-----+
| "Tony Parker" | 36           |
| "Tim Duncan"  | 42           |
| "Grant Hill"  | 46           |
...

```

```
trsgaph> GO FROM "player100" OVER follow \
  WHERE follow.degree > 90 \
  YIELD dst(edge);

+-----+
| dst(EDGE) |
+-----+
| "player101" |
| "player125" |
+-----+
```

过滤动态计算属性

```
trsgraph> MATCH (v:player) \
  WHERE v[toLower("AGE")] < 21 \
  RETURN v.player.name, v.player.age;

+-----+-----+
| v.name | v.age |
+-----+-----+
| "Luka Doncic" | 20    |
+-----+-----+
```

过滤现存属性

```
trsgraph> MATCH (v:player) \
  WHERE exists(v.player.age) \
  RETURN v.player.name, v.player.age;

+-----+-----+
| v.player.name | v.player.age |
+-----+-----+
| "Danny Green" | 31           |
| "Tiago Splitter" | 34          |
| "David West"   | 38           |
...

```

过滤 RANK

在 nGQL 中，如果多个边拥有相同的起始点、目的点和属性，则它们的唯一区别是 rank 值。在 `WHERE` 子句中使用 rank 过滤边。

```
# 创建测试数据。
trsgraph> CREATE SPACE IF NOT EXISTS test (vid_type=FIXED_STRING(30));
trsgraph> USE test;
```

```

trsgraph> CREATE EDGE IF NOT EXISTS e1(p1 int);
trsgraph> CREATE TAG IF NOT EXISTS person(p1 int);
trsgraph> INSERT VERTEX person(p1) VALUES "1":(1);
trsgraph> INSERT VERTEX person(p1) VALUES "2":(2);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@0:(10);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@1:(11);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@2:(12);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@3:(13);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@4:(14);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@5:(15);
trsgraph> INSERT EDGE e1(p1) VALUES "1"->"2"@6:(16);

# 通过 rank 过滤边, 查找 rank 大于 2 的边。
trsgraph> GO FROM "1" \
  OVER e1 \
  WHERE rank(edge) > 2 \
  YIELD src(edge), dst(edge), rank(edge) AS Rank, properties(edge).p1 | \
  ORDER BY $-.Rank DESC;

+-----+-----+-----+-----+
| src(EDGE) | dst(EDGE) | Rank | properties(EDGE).p1 |
+-----+-----+-----+-----+
| "1"       | "2"       | 6    | 16                  |
| "1"       | "2"       | 5    | 15                  |
| "1"       | "2"       | 4    | 14                  |
| "1"       | "2"       | 3    | 13                  |
+-----+-----+-----+-----+

# 通过 rank 过滤边, 查找 rank 值等于 0 的 follow 边。
trsgraph> MATCH (v)-[e:follow]->() \
  WHERE rank(e)==0 \
  RETURN *;

+-----+-----+
| v      | e      |
+-----+-----+
| ("player142" :player{age: 29, name: "Klay Thompson"}) | [:follow "player142"->"player117" @0 {degree: 90}] |
| ("player139" :player{age: 34, name: "Marc Gasol"})     | [:follow "player139"->"player138" @0 {degree: 99}] |
| ("player108" :player{age: 36, name: "Boris Diaw"})     | [:follow "player108"->"player100" @0 {degree: 80}] |
| ("player108" :player{age: 36, name: "Boris Diaw"})     | [:follow "player108"->"player101" @0 {degree: 80}] |
...

```

过滤 PATTERN

```

trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->(t) \
  WHERE (v)-[e]->(t:team) \
  RETURN (v)-->();

+-----+
+
| (v)-->() = (v)-->()
+-----+
+
| [<("player100" :player{age: 42, name: "Tim Duncan"})-[:serve@0 {end_year: 2016, start_year: 1997}]->("team204" :team{name: "Spurs"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})>] |
+-----+
+

trsgraph> MATCH (v:player{name:"Tim Duncan"})-[e]->(t) \
  WHERE NOT (v)-[e]->(t:team) \
  RETURN (v)-->();

+-----+
+
| (v)-->() = (v)-->()
+-----+
+
| [<("player100" :player{age: 42, name: "Tim Duncan"})-[:serve@0 {end_year: 2016, start_year: 1997}]->("team204" :team{name: "Spurs"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})>] |
| [<("player100" :player{age: 42, name: "Tim Duncan"})-[:serve@0 {end_year: 2016, start_year: 1997}]->("team204" :team{name: "Spurs"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player101" :player{age: 36, name: "Tony Parker"})>,
<("player100" :player{age: 42, name: "Tim Duncan"})-[:follow@0 {degree: 95}]->("player125" :player{age: 41, name: "Manu Ginobili"})>] |
+-----+
+

```

过滤字符串

在 WHERE 子句中使用 STARTS WITH、ENDS WITH 或 CONTAINS 可以匹配字符串的特定部分。匹配时区分大小写。

STARTS WITH

STARTS WITH 会从字符串的起始位置开始匹配。

```

# 查询姓名以 T 开头的 player 信息。
trsgraph> MATCH (v:player) \

```

```
WHERE v.player.name STARTS WITH "T" \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
| "Tony Parker" | 36           |
| "Tiago Splitter" | 34          |
| "Tim Duncan" | 42           |
| "Tracy McGrady" | 39          |
+-----+
```

如果使用小写 `t` (`STARTS WITH "t"`)，会返回空集，因为数据库中没有以小写 `t` 开头的姓名。

```
trsgaph> MATCH (v:player) \
WHERE v.player.name STARTS WITH "t" \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
Empty set (time spent 5080/6474 us)
```

ENDS WITH

`ENDS WITH` 会从字符串的结束位置开始匹配。

```
trsgaph> MATCH (v:player) \
WHERE v.player.name ENDS WITH "r" \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
| "Tony Parker" | 36           |
| "Tiago Splitter" | 34          |
| "Vince Carter" | 42           |
+-----+
```

CONTAINS

`CONTAINS` 会检查关键字是否匹配字符串的某一部分。

```
trsgaph> MATCH (v:player) \
WHERE v.player.name CONTAINS "Pa" \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
| "Paul George" | 28           |
| "Tony Parker" | 36           |
| "Paul Gasol" | 38           |
| "Chris Paul" | 33           |
+-----+
```

结合 NOT 使用

用户可以结合布尔运算符 `NOT` 一起使用，否定字符串匹配条件。

```
trsgaph> MATCH (v:player) \
WHERE NOT v.player.name ENDS WITH "R" \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
| "Danny Green" | 31           |
| "Tiago Splitter" | 34          |
| "David West" | 38           |
| "Russell Westbrook" | 30          |
...
```

过滤列表

匹配列表中的值

使用 `IN` 运算符检查某个值是否在指定列表中。

```
trsgaph> MATCH (v:player) \
WHERE v.player.age IN range(20,25) \
RETURN v.player.name, v.player.age;
+-----+
| v.player.name | v.player.age |
+-----+
```



```
| "Ben Simmons" | 22 |
| "Giannis Antetokounmpo" | 24 |
| "Kyle Anderson" | 25 |
| "Joel Embiid" | 25 |
| "Kristaps Porzingis" | 23 |
| "Luka Doncic" | 20 |
+-----+

trsgraph> LOOKUP ON player \
  WHERE player.age IN [25,28] \
  YIELD properties(vertex).name, properties(vertex).age;
+-----+
| properties(VERTEX).name | properties(VERTEX).age |
+-----+
| "Kyle Anderson" | 25 |
| "Damian Lillard" | 28 |
| "Joel Embiid" | 25 |
| "Paul George" | 28 |
| "Ricky Rubio" | 28 |
+-----+
```

结合 NOT 使用

```
trsgraph> MATCH (v:player) \
  WHERE v.player.age NOT IN range(20,25) \
  RETURN v.player.name AS Name, v.player.age AS Age \
  ORDER BY Age;
+-----+
| Name | Age |
+-----+
| "Kyrie Irving" | 26 |
| "Cory Joseph" | 27 |
| "Damian Lillard" | 28 |
| "Paul George" | 28 |
| "Ricky Rubio" | 28 |
...

```

最后更新: 2025年12月19日

4.6.8 YIELD

- `YIELD` 定义 nGQL 查询的输出结果。
- `YIELD` 可以引导子句或语句：
- `YIELD` 子句用于原生 nGQL 语句中，例如 `GO`、`FETCH` 或 `LOOKUP`，必须通过 `YIELD` 子句定义返回结果。
 - `YIELD` 语句可以在独立查询或复合查询中使用。

openCypher 兼容性

本文操作仅适用于原生 nGQL。关于 openCypher 方式如何定义输出结果，请参见 `RETURN`。

- `YIELD` 在 nGQL 和 openCypher 中有不同的函数：
- 在 openCypher 中，`YIELD` 用于在 `CALL[...YIELD]` 子句中指定过程调用的输出。
- Note

nGQL 不支持 `CALL[...YIELD]`。
- 在 nGQL 中，`YIELD` 和 openCypher 中的 `RETURN` 类似。
- Note

下文示例中的 `$$`、`$-` 等是引用符号，详情请参见引用属性。

YIELD 子句

语法	
<pre>YIELD [DISTINCT] <col> [AS <alias>] [, <col> [AS <alias>] ...];</pre>	
参数	说明
<code>DISTINCT</code>	聚合输出结果，返回去重后的结果集。
<code>col</code>	要返回的字段。如果没有为字段设置别名，返回结果中的列名为 <code>col</code> 。
<code>alias</code>	<code>col</code> 的别名。使用关键字 <code>AS</code> 进行设置，设置后返回结果中的列名为该别名。

使用 YIELD 子句

- `GO` 语句中使用 `YIELD`：
- ```
trsgraph> GO FROM "player100" OVER follow \
 YIELD properties($$).name AS Friend, properties($$).age AS Age;
+-----+-----+
| Friend | Age |
+-----+-----+
| "Tony Parker" | 36 |
```

```
| "Manu Ginobili" | 41 |
+-----+-----+
```

- `FETCH` 语句中使用 `YIELD`：

```
trsgraph> FETCH PROP ON player "player100" \
 YIELD properties(vertex).name;
+-----+-----+
| properties(VERTEX).name |
+-----+-----+
| "Tim Duncan" |
+-----+-----+
```

- `LOOKUP` 语句中使用 `YIELD`：

```
trsgraph> LOOKUP ON player WHERE player.name == "Tony Parker" \
 YIELD properties(vertex).name, properties(vertex).age;
+-----+-----+
| properties(VERTEX).name | properties(VERTEX).age |
+-----+-----+
| "Tony Parker" | 36 |
+-----+-----+
```

YIELD 语句

语法

```
YIELD [DISTINCT] <col> [AS <alias>] [, <col> [AS <alias>] ...]
[WHERE <conditions>];
```

| 参数                      | 说明                                                               |
|-------------------------|------------------------------------------------------------------|
| <code>DISTINCT</code>   | 聚合输出结果，返回去重后的结果集。                                                |
| <code>col</code>        | 要按返回的字段。如果没有为字段设置别名，返回结果中的列名为 <code>col</code> 。                 |
| <code>alias</code>      | <code>col</code> 的别名。使用关键字 <code>AS</code> 进行设置，设置后返回结果中的列名为该别名。 |
| <code>conditions</code> | 在 <code>WHERE</code> 子句中设置的过滤条件。详情请参见 <code>WHERE</code> 。       |

复合查询中使用 YIELD 语句

在复合查询中，`YIELD` 语句可以接收、过滤、修改之前语句的结果集，然后输出。

```
查找 player100 关注的 player，并计算他们的平均年龄。
trsgraph> GO FROM "player100" OVER follow \
 YIELD dst(edge) AS ID \
 | FETCH PROP ON player $-.ID \
 YIELD properties(vertex).age AS Age \
 | YIELD AVG($-.Age) as Avg_age, count(*)as Num_friends;
+-----+-----+
| Avg_age | Num_friends |
+-----+-----+
| 38.5 | 2 |
+-----+-----+

查找 player101 关注的 player，返回 degree 大于 90 的 player。
trsgraph> $var1 = GO FROM "player101" OVER follow \
 YIELD properties(edge).degree AS Degree, dst(edge) as ID; \
 YIELD $var1.ID AS ID WHERE $var1.Degree > 90;
+-----+
| ID |
+-----+
| "player100" |
| "player125" |
+-----+

查找 player 中年龄大于 30 且小于 32 的点，返回去掉重复属性的值。
trsgraph> LOOKUP ON player \
 WHERE player.age < 32 and player.age >30 \
 YIELD DISTINCT properties(vertex).age as v;
+-----+
| v |
+-----+
| 31 |
+-----+
```

独立使用 YIELD 语句

`YIELD` 可以计算表达式并返回结果。

```
trsgraph> YIELD rand32(1, 6);
+-----+
| rand32(1,6) |
+-----+
| 3 |
+-----+

trsgraph> YIELD "Hel" + "\tlo" AS string1, ", World!" AS string2;
+-----+-----+
| string1 | string2 |
+-----+-----+
| "Hel\tlo" | ", World!" |
+-----+-----+

trsgraph> YIELD hash("Tim") % 100;
+-----+
| (hash("Tim")%100) |
+-----+
| 42 |
+-----+

trsgraph> YIELD \
CASE 2+3 \
WHEN 4 THEN 0 \
WHEN 5 THEN 1 \
ELSE -1 \
END \
AS result;
+-----+
| result |
+-----+
| 1 |
+-----+

trsgraph> YIELD 1- -1;
+-----+
| (1--(1)) |
+-----+
| 2 |
+-----+
```

最后更新: 2025年12月19日

### 4.6.9 WITH

`WITH` 子句可以获取并处理查询前半部分的结果，并将处理结果作为输入传递给查询的后半部分。

#### openCypher 兼容性

本文操作仅适用于 openCypher 方式。



在原生 nGQL 中，有与 `WITH` 类似的管道符，但它们的工作方式不同。不要在 openCypher 方式中使用管道符，也不要原生 nGQL 中使用 `WITH` 子句。

#### 组成复合查询

使用 `WITH` 子句可以组合语句，将一条语句的输出转换为另一条语句的输入。

##### 示例 1

1. 匹配一个路径。
2. 通过 `nodes()` 函数将路径上的所有点输出到一个列表。
3. 将列表拆分为行。
4. 去重后返回点的信息。

```
查询 Tag 为 player, name 属性为 Tim Duncan 的点的路径，并返回去重后路径中的所有的点信息列表。
trsgaph> MATCH p=(v:player{name:"Tim Duncan"})--() \
 WITH nodes(p) AS n \
 UNWIND n AS n1 \
 RETURN DISTINCT n1;

+-----+
| n1 |
+-----+
| ("player100":player{age: 42, name: "Tim Duncan"}) |
| ("player101":player{age: 36, name: "Tony Parker"}) |
| ("team204":team{name: "Spurs"}) |
| ("player102":player{age: 33, name: "LaMarcus Aldridge"}) |
| ("player125":player{age: 41, name: "Manu Ginobili"}) |
| ("player104":player{age: 32, name: "Marco Belinelli"}) |
| ("player144":player{age: 47, name: "Shaquille O'Neal"}) |
| ("player105":player{age: 31, name: "Danny Green"}) |
| ("player113":player{age: 29, name: "Dejounte Murray"}) |
| ("player107":player{age: 32, name: "Aron Baynes"}) |
| ("player109":player{age: 34, name: "Tiago Splitter"}) |
| ("player108":player{age: 36, name: "Boris Diaw"}) |
+-----+
```

##### 示例 2

1. 匹配点 ID 为 `player100` 的点。
2. 通过 `labels()` 函数将点的所有 Tag 输出到一个列表。
3. 将列表拆分为行。
4. 返回结果。

```
查询 id 为 player100 的点，并返回点的所有 Tag。
trsgaph> MATCH (v) \
 WHERE id(v)=="player100" \
 WITH labels(v) AS tags_unf \
 UNWIND tags_unf AS tags_f \
 RETURN tags_f;

+-----+
| tags_f |
+-----+
| "player" |
+-----+
```

## 过滤聚合查询

`WITH` 可以在聚合查询中作为过滤器使用。

```
查询所有起始点和目的点均有 player 点类型的边，并对所有目的点去重，再根据 age 属性升序排列，返回 age 属性值小于 25 的目的点数据。
trsgraph> MATCH (v:player)-->(v2:player) \
 WITH DISTINCT v2 AS v2, v2.player.age AS Age \
 ORDER BY Age \
 WHERE Age<25 \
 RETURN v2.player.name AS Name, Age;
+-----+-----+
| Name | Age |
+-----+-----+
"Luka Doncic"	20
"Ben Simmons"	22
"Kristaps Porzingis"	23
+-----+-----+
```

## collect() 之前处理输出

在 `collect()` 函数将输出结果转换为列表之前，可以使用 `WITH` 子句排序和限制输出结果。

```
查询所有 Tag 为 player 的点数据，并根据 name 属性倒序排列，将前 3 条结果转换为列表返回。
trsgraph> MATCH (v:player) \
 WITH v.player.name AS Name \
 ORDER BY Name DESC \
 LIMIT 3 \
 RETURN collect(Name);
+-----+-----+
| collect(Name) |
+-----+-----+
| ["Yao Ming", "Vince Carter", "Tracy McGrady"] |
+-----+-----+
```

## 结合 RETURN 语句使用

在 `WITH` 子句中设置别名，并通过 `RETURN` 子句输出结果。

```
判断 3 是否存在于 list 数组中。
trsgraph> WITH [1, 2, 3] AS `list` RETURN 3 IN `list` AS r;
+-----+
| r |
+-----+
| true |
+-----+

返回 3 和 4 的比较结果。
trsgraph> WITH 4 AS one, 3 AS two RETURN one > two AS result;
+-----+
| result |
+-----+
| true |
+-----+
```

最后更新: 2025年12月19日

## 4.6.10 UNWIND

`UNWIND` 语句可以将列表拆分为单独的行，列表中的每个元素为一行。

`UNWIND` 可以作为单独语句或语句中的子句使用。

### UNWIND 语句

#### 语法

```
UNWIND <list> AS <alias> <RETURN clause>;
```

#### 示例

- 拆分列表。

```
trsgraph> UNWIND [1,2,3] AS n RETURN n;
+---+
| n |
+---+
| 1 |
| 2 |
| 3 |
+---+
```

### UNWIND 子句

#### 语法

- 原生 nGQL 语句中使用 `UNWIND` 子句。



Note

在原生 nGQL 语句中使用 `UNWIND` 子句时，需要用在管道符 `|` 之后，并使用 `$-` 引用管道符之前的变量。如果 `UNWIND` 后使用语句或子句，需要使用管道符 `|` 并且使用 `$-` 引用管道符之前的变量。

```
<statement> | UNWIND $-.<var> AS <alias> <|> <clause>;
```

- openCypher 语句中使用 `UNWIND` 子句。

```
<statement> UNWIND <list> AS <alias> <RETURN clause>;
```

## 示例

- 在 UNWIND 子句中使用 WITH DISTINCT 可以将列表中的重复项忽略，返回去重后的结果。



Note

原生 nGQL 语句不支持 WITH DISTINCT。

```
// 拆分列表`[1,1,2,2,3,3]`，删除重复行，排序行，将行转换为列表。
```

```
trsgraph> WITH [1,1,2,2,3,3] AS n \
 UNWIND n AS r \
 WITH DISTINCT r AS r \
 ORDER BY r \
 RETURN collect(r);
```

```
+-----+
| collect(r) |
+-----+
```



```
| [1, 2, 3] |
+-----+
```

- **MATCH 语句中使用 UNWIND。**

```
// 将匹配路径上的顶点输出到列表中，拆分列表，删除重复行，将行转换为列表。
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})--(v2) \
 WITH nodes(p) AS n \
 UNWIND n AS r \
 WITH DISTINCT r AS r \
 RETURN collect(r);
+-----+
| collect(r) |
+-----+
| [{"player100":player{age: 42, name: "Tim Duncan"}}, {"player101":player{age: 36, name: "Tony Parker"}}, |
| {"team204":team{name: "Spurs"}}, {"player102":player{age: 33, name: "LaMarcus Aldridge"}}, |
| {"player125":player{age: 41, name: "Manu Ginobili"}}, {"player104":player{age: 32, name: "Marco Belinelli"}}, |
| {"player144":player{age: 47, name: "Shaquille O'Neal"}}, {"player105":player{age: 31, name: "Danny Green"}}, |
| {"player113":player{age: 29, name: "Dejounte Murray"}}, {"player107":player{age: 32, name: "Aron Baynes"}}, |
| {"player109":player{age: 34, name: "Tiago Splitter"}}, {"player108":player{age: 36, name: "Boris Diaw"}}] |
+-----+
```

- **GO 语句中使用 UNWIND。**

```
// 在点列表中查询点关联的边。
trsgraph> YIELD ['player101', 'player100'] AS a | UNWIND $-.a AS b | GO FROM $-.b OVER follow YIELD edge AS e;
+-----+
| e |
+-----+
| [:follow "player101"->"player100" @0 {degree: 95}] |
| [:follow "player101"->"player102" @0 {degree: 90}] |
| [:follow "player101"->"player125" @0 {degree: 95}] |
| [:follow "player100"->"player101" @0 {degree: 95}] |
| [:follow "player100"->"player125" @0 {degree: 95}] |
+-----+
```

- **LOOKUP 语句中使用 UNWIND。**

```
// 查询年龄大于 46 岁球员的所有属性，去掉重复属性，并将结果转换为行。
trsgraph> LOOKUP ON player \
 WHERE player.age > 46 \
 YIELD DISTINCT keys(vertex) as p | UNWIND $-.p as a | YIELD $-.a AS a;
+-----+
| a |
+-----+
| "age" |
| "name" |
+-----+
```

- **FETCH 语句中使用 UNWIND。**

```
// 查询 player101 点的所有 Tag，并将结果转换为行。
trsgraph> CREATE TAG hero(like string, height int);
INSERT VERTEX hero(like, height) VALUES "player101":("deep", 182);
FETCH PROP ON * "player101" \
 YIELD tags(vertex) as t | UNWIND $-.t as a | YIELD $-.a AS a;
+-----+
| a |
+-----+
| "hero" |
| "player" |
+-----+
```

• GET SUBGRAPH 语句中使用 UNWIND。

```
// 查询从点 player100 开始、0~2 跳、serve 类型的出边和入边的子图，并将结果转换为行。
trsgraph> GET SUBGRAPH 2 STEPS FROM "player100" BOTH serve \
YIELD edges as e | UNWIND $-e as a | YIELD $-.a AS a;
```

```
+-----+
| a |
+-----+
| [:serve "player100"->"team204" @0 {}] |
| [:serve "player101"->"team204" @0 {}] |
| [:serve "player102"->"team204" @0 {}] |
| [:serve "player103"->"team204" @0 {}] |
| [:serve "player105"->"team204" @0 {}] |
| [:serve "player106"->"team204" @0 {}] |
| [:serve "player107"->"team204" @0 {}] |
| [:serve "player108"->"team204" @0 {}] |
| [:serve "player109"->"team204" @0 {}] |
| [:serve "player110"->"team204" @0 {}] |
| [:serve "player111"->"team204" @0 {}] |
| [:serve "player112"->"team204" @0 {}] |
| [:serve "player113"->"team204" @0 {}] |
| [:serve "player114"->"team204" @0 {}] |
| [:serve "player125"->"team204" @0 {}] |
| [:serve "player138"->"team204" @0 {}] |
| [:serve "player104"->"team204" @20132015 {}] |
| [:serve "player104"->"team204" @20182019 {}] |
+-----+
```

• FIND PATH 语句中使用 UNWIND。

```
// 找出 player101 到 team204 沿 serve 类型边的最短路径上的所有点，并将结果转换为行。
trsgraph> FIND SHORTEST PATH FROM "player101" TO "team204" OVER serve \
YIELD path as p | YIELD nodes($-.p) AS nodes | UNWIND $-.nodes AS a | YIELD $-.a AS a;
```

```
+-----+
| a |
+-----+
| ("player101") |
| ("team204") |
+-----+
```

最后更新: 2025年12月19日

## 4.6.11 INNER JOIN

`INNER JOIN` 是一种连接查询。它基于两个表之间的共同列值进行匹配，并返回满足条件的记录。`INNER JOIN` 通常用于根据关联列的值来创建一个包含两个表的结果集。在 TRSGraph 中，可以显式地使用 `INNER JOIN` 子句在两个表之间进行连接查询，以获取更加复杂的查询结果。



在 nGQL 语句中，`GO` 的多跳查询隐式地使用了 `INNER JOIN` 子句。例如，`GO 1 TO 2 STEPS FROM "player101" OVER follow YIELD $$player.name AS name, $$player.age AS age` 语句中，`GO` 子句隐式地使用了 `INNER JOIN` 子句，将从 `player101` 出发沿 `follow` 边第一步查询的结果列和第二步查询的起点列进行匹配，然后基于匹配结果返回 `name` 和 `age`。

### openCypher 兼容性

`INNER JOIN` 子句仅适用于原生 nGQL 语法。

### 语法

```
YIELD <column_name_list>
FROM <first_table> INNER JOIN <second_table> ON <join_condition>
```

### 使用说明

- 必须使用 `YIELD` 子句指定返回的列，并且 `YIELD` 子句需放置在 `INNER JOIN` 子句之前。
- 使用 `FROM` 子句指定要连接的两个表。
- `INNER JOIN` 子句必须包含 `ON` 子句，`ON` 子句指定了连接条件，并且连接条件只支持等值连接（即 `==`）。
- `<first_table>` 和 `<second_table>` 是要连接的两个表，两表名不能相同。
- 使用自定义变量来指定表名。详情参见[使用自定义变量](#)。

### 使用示例

以下示例介绍如何通过 `INNER JOIN` 关联 nGQL 语句中的两个查询结果。

**示例一：**通过 `INNER JOIN` 将第一个 `LOOKUP` 查询中的结果 `dst` 列（Tony Parker 的 ID `player101`）和第二个 `GO` 查询中的结果 `src` 列（`player101` 和 `player125` 两个 ID）进行匹配。匹配的结果集为两个查询结果中 `dst` 和 `src` 列都包含的 `player101`，基于这个 `player101` 匹配的结果，然后通过 `YIELD $b.vid AS vid, $a.v AS v, $b.e2 AS e2` 返回最终查询结果。

```
trsgraph> $a = LOOKUP ON player WHERE player.name == 'Tony Parker' YIELD id(vertex) as dst, vertex AS v; \
 $b = GO FROM 'player101', 'player125' OVER follow YIELD id($) as src, id($) as vid, edge AS e2; \
 YIELD $b.vid AS vid, $a.v AS v, $b.e2 AS e2 FROM $a INNER JOIN $b ON $a.dst == $b.src;
+-----+-----+-----+
| vid | v | e2 |
+-----+-----+-----+
"player100"	("player101" :player{age: 36, name: "Tony Parker"})	[:follow "player101"->"player100" @0 {degree: 95}]
"player102"	("player101" :player{age: 36, name: "Tony Parker"})	[:follow "player101"->"player102" @0 {degree: 90}]
"player125"	("player101" :player{age: 36, name: "Tony Parker"})	[:follow "player101"->"player125" @0 {degree: 95}]
+-----+-----+-----+
```

**示例二：**通过 `INNER JOIN` 将第一个 `LOOKUP` 查询中的结果 `src` 列（Tony Parker 的 ID `player101`）和第二个 `FETCH` 查询中的结果 `src` 列（`player101` 到 `player100` 的起点 ID `player101`）进行匹配。匹配的结果集为都包含的 `player101`，基于这个 `player101` 匹配的结果，然后通过 `YIELD $a.src AS src, $a.v AS v, $b.e AS e` 返回最终查询结果。

```
trsgraph> $a = LOOKUP ON player WHERE player.name == 'Tony Parker' YIELD id(vertex) as src, vertex AS v; \
 $b = FETCH PROP ON follow 'player101'->'player100' YIELD src(edge) as src, edge as e; \
 YIELD $a.src AS src, $a.v AS v, $b.e AS e FROM $a INNER JOIN $b ON $a.src == $b.src;
+-----+-----+-----+
| src | v | e |
+-----+-----+-----+
| "player101" | ("player101" :player{age: 36, name: "Tony Parker"}) | [:follow "player101"->"player100" @0 {degree: 95}] |
+-----+-----+-----+
```

示例三：使用 INNER JOIN 关联 LOOKUP、GO 和 FIND PATH 子句的查询的过程示例步骤：

1. 通过 LOOKUP ON 语句，先对 player 表进行查询，找到名为 Tony Parker 的球员顶点，并将其 ID 和属性存储到 \$a.src 和 v 列中。
2. 再通过 GO 查找从 \$a.src 节点出发经过 follow 边往外走 2-5 步的球员节点，其中要求这些节点对应的 player 点年龄大于 30 岁，返回这些节点的 ID，并将它们存储在 \$b.dst 列中。
3. 然后通过 FIND ALL PATH 语句，查找从 \$a.src 到 \$b.dst 的沿 follow 所有路径，并返回路径 \$c.p 以及路径中的终点 \$c.dst。
4. 最后通过 FIND SHORTEST PATH 语句，查找从 \$c.dst 到 \$a.src 的最短路径，并返回路径 \$d.p 和路径起点 \$d.src。
5. 使用 INNER JOIN 将 3 和 4 中得到的结果联接起来，匹配 \$c.dst 和 \$d.src 列，为每个联接匹配的记录返回

YIELD \$c.forward AS forwardPath, \$c.dst AS end, \$d.p AS backwardPath。

```

trsgraph> $a = LOOKUP ON player WHERE player.name == 'Tony Parker' YIELD id(vertex) as src, vertex AS v; \
$b = GO 2 TO 5 STEPS FROM $a.src OVER follow WHERE $.player.age > 30 YIELD id($) AS dst; \
$c = (FIND ALL PATH FROM $a.src TO $b.dst OVER follow YIELD path AS p | YIELD $-.p AS forward, id(endNode($-.p)) AS dst); \
$d = (FIND SHORTEST PATH FROM $c.dst TO $a.src OVER follow YIELD path AS p | YIELD $-.p AS p, id(startNode($-.p)) AS src); \
YIELD $c.forward AS forwardPath, $c.dst AS end, $d.p AS backwardPath FROM $c INNER JOIN $d ON $c.dst == $d.src;

+-----+-----+
+
|
forwardPath
| end | backwardPath
+-----+-----+
+-----+-----+
| <("player101")-[:follow@0 {}]-
>("player102")>
| "player102" |
| <("player102")-[:follow@0 {}]->("player101")>
| "player101" |
| <("player101")-[:follow@0 {}]->("player100")-[:follow@0 {}]->("player101")-[:follow@0 {}]-
>("player102")>
| "player102" | <("player102")-[:follow@0 {}]-
>("player101")>
| <("player101")-[:follow@0 {}]-
>("player125")>
| "player125" |
| <("player125")-[:follow@0 {}]->("player100")-[:follow@0 {}]->("player101")> |
| <("player101")-[:follow@0 {}]->("player100")-[:follow@0 {}]-
>("player125")>
| "player125" | <("player125")-[:follow@0 {}]-
>("player100")-[:follow@0 {}]->("player101")> |
| <("player101")-[:follow@0 {}]->("player100")-[:follow@0 {}]->("player101")-[:follow@0 {}]-
>("player125")>
| "player125" | <("player125")-[:follow@0 {}]->("player100")-[:follow@0 {}]-
>("player101")> |
| <("player101")-[:follow@0 {}]->("player102")-[:follow@0 {}]->("player100")-[:follow@0 {}]-
>("player125")>
| "player125" | <("player125")-[:follow@0 {}]->("player100")-[:follow@0 {}]-
>("player101")> |
| <("player101")-[:follow@0 {}]->("player102")-[:follow@0 {}]->("player101")-[:follow@0 {}]-
>("player125")>
| "player125" | <("player125")-[:follow@0 {}]->("player100")-[:follow@0 {}]-
>("player101")> |
...
+-----+-----+
+-----+-----+

```

最后更新: 2025年12月19日

## 4.7 变量和复合查询

### 4.7.1 复合查询（子句结构）

复合查询将来自不同请求的数据放在一起，然后进行过滤、分组或者排序等，最后返回结果。

TRSGraph 支持三种方式进行复合查询（或子查询）：

- （opencypher 兼容语句）连接各个子句，让它们在彼此之间提供中间结果集。
- （原生 nGQL）多个查询可以合并处理，以英文分号 (;) 分隔，返回最后一个查询的结果。
- （原生 nGQL）可以用管道符 (|) 将多个查询连接起来，上一个查询的结果可以作为下一个查询的输入。

#### OpenCypher 兼容性

在复合查询中，请不要混用 opencypher 兼容语句和原生 nGQL 语句，例如 `MATCH ... | GO ... | YIELD ...`。

- 如果使用 opencypher 兼容语句（`MATCH`、`RETURN`、`WITH` 等），请不要使用管道符或分号组合子句。
- 如果使用原生 nGQL 语句（`FETCH`、`GO`、`LOOKUP` 等），必须使用管道符或分号组合子句。

#### 复合查询不支持事务

例如一个查询由三个子查询 A、B、C 组成，A 是一个读操作，B 是一个计算操作，C 是一个写操作，如果在执行过程中，任何一个操作执行失败，则整个结果是未定义的：没有回滚，而且写入的内容取决于执行程序。



Note

openCypher 没有事务要求。

#### 示例

- opencypher 兼容语句

```
子句连接多个查询。
trsgraph> MATCH p=(v:player{name:"Tim Duncan"})--() \
 WITH nodes(p) AS n \
```

```
UNWIND n AS n1 \
RETURN DISTINCT n1;
```

- 原生 nGQL（分号）

```
只返回边。
trsgraph> SHOW TAGS; SHOW EDGES;

插入多个点。
trsgraph> INSERT VERTEX player(name, age) VALUES "player100":("Tim Duncan", 42); \
 INSERT VERTEX player(name, age) VALUES "player101":("Tony Parker", 36); \
 INSERT VERTEX player(name, age) VALUES "player102":("LaMarcus Aldridge", 33);
```

- 原生 nGQL（管道符）

```
管道符连接多个查询。
trsgraph> GO FROM "player100" OVER follow YIELD dst(edge) AS id | \
 GO FROM $-.id OVER serve YIELD properties($$.name AS Team, \
 properties($^).name AS Player;

+-----+-----+
| Team | Player |
+-----+-----+
"Spurs"	"Tony Parker"
"Hornets"	"Tony Parker"
"Spurs"	"Manu Ginobili"
+-----+-----+
```

---

最后更新: 2025年12月19日

## 4.7.2 自定义变量

TRSGraph 允许将一条语句的结果作为自定义变量传递给另一条语句。

### OpenCypher 兼容性

当引用一个变量的点、边或路径，需要先给它命名。例如：

```
trsgraph> MATCH (v:player{name:"Tim Duncan"}) RETURN v;
+-----+
| v |
+-----+
| ("player100":player{name: "Tim Duncan", age: 42}) |
+-----+
```

示例中的 `v` 就是自定义变量。



在同一个 MATCH 语句的模式中，不能重复使用边变量。例如 `e` 不能重复被写在模式 `p=(v1)-[e*2..2]->(v2)-[e*2..2]->(v3)` 中。

### 原生 nGQL

nGQL 扩展的自定义变量可以表示为 `$var_name`，`var_name` 由字母、数字或下划线（`_`）构成，不允许使用其他字符。

自定义变量仅在当前执行（本复合查询中）有效，执行结束后变量也会释放，不能在其他客户端、执行、session 中使用之前的自定义变量。

用户可以在复合查询中使用自定义变量。复合查询的详细信息请参见[复合查询](#)。



- 自定义变量区分大小写。
- 在包含扩展的自定义变量的复合语句中，用英文分号 ; 结束定义变量的语句。详情参见 [nGQL 风格指南](#)。

### 示例

```
trsgraph> $var = GO FROM "player100" OVER follow YIELD dst(edge) AS id; \
GO FROM $var.id OVER serve YIELD properties($$).name AS Team, \
properties($^).name AS Player;
+-----+
| Team | Player |
+-----+
"Spurs"	"Tony Parker"
"Hornets"	"Tony Parker"
"Spurs"	"Manu Ginobili"
+-----+
```

### 集合运算与变量语句的范围与运算顺序

当我们需要对集合运算的复合语句进行变量赋值时，需要注意用括号包裹语句的范围，比如下边的例子里 `$var` 赋值的来源是两个语句 `INTERSECT` 之后的输出。

```
$var = (\
GO FROM "player100" OVER follow \
YIELD dst(edge) AS id \
INTERSECT \
GO FROM "player100" OVER follow \
YIELD dst(edge) AS id \
); \
GO FROM $var.id OVER follow YIELD follow.degree AS degree
```

最后更新: 2025年12月19日



### 4.7.3 引用属性

nGQL 提供属性引用符以允许用户在 `GO` 语句中引用起始点、目的点、边的属性，以及引用复合查询的输出结果。本文将详细介绍如何在 nGQL 中使用这些属性引用符。

#### openCypher 兼容性

属性引用符仅适用于原生 nGQL。

#### 点属性引用符

| 引用符               | 说明     |
|-------------------|--------|
| <code>\$^</code>  | 引用起始点。 |
| <code>\$\$</code> | 引用目的点。 |

##### 引用语法

```
$^.<tag_name>.<prop_name> # 起始点属性引用
$$.<tag_name>.<prop_name> # 目的点属性引用
```

- `tag_name`：点的 Tag 名称。
- `prop_name`：Tag 内的属性名称。

#### 边属性引用符

| 引用符                | 说明                             |
|--------------------|--------------------------------|
| <code>_src</code>  | 边的起始点                          |
| <code>_dst</code>  | 边的目的点                          |
| <code>_type</code> | 边的类型内部编码，正负号表示方向：正数为正向边，负数为逆向边 |
| <code>_rank</code> | 边的 rank 值                      |

##### 引用语法

nGQL 允许用户引用边的属性，包括自定义的边属性和四种内置的边属性。

```
<edge_type>.<prop_name> # 自定义边属性引用
<edge_type>._src|_dst|_type|_rank # 内置边属性引用
```

- `edge_type`：Edge type。
- `prop_name`：Edge type 的属性名称。

#### 复合查询中的引用符

| 引用符              | 说明                                                |
|------------------|---------------------------------------------------|
| <code>\$-</code> | 引用复合查询中管道符之前的语句输出结果。更多信息请参见 <a href="#">管道符</a> 。 |

#### 示例

##### 使用点属性引用符

```
返回起始点的 Tag player 的 name 属性值和目的点的 Tag player 的 age 属性值。
trsgraph> GO FROM "player100" OVER follow YIELD $^.player.name AS startName, $$player.age AS endAge;
+-----+
| startName | endAge |
```

```
+-----+-----+
| "Tim Duncan" | 36 |
| "Tim Duncan" | 41 |
+-----+-----+
```

 版本兼容性

从TRSGraph 2.6.0 起支持了新的 Schema 相关函数。以上示例在TRSGraph 10.0 中的近似写法如下：

```
GO FROM "player100" OVER follow YIELD properties($^).name AS startName, properties($$).age AS endAge;
```

TRSGraph 10.0 兼容新旧语法。

使用边属性引用符

```
返回 Edge type follow 的 degree 属性值。
trsgraph> GO FROM "player100" OVER follow YIELD follow.degree;
+-----+
| follow.degree |
+-----+
| 95 |
+-----+

返回 EdgeType 是 follow 的起始点 VID、目的点 VID、EdgeType 编码（正数为正向边，负数为逆向边），和边的 rank 值。
trsgraph> GO FROM "player100" OVER follow YIELD follow._src, follow._dst, follow._type, follow._rank;
+-----+-----+-----+-----+
| follow._src | follow._dst | follow._type | follow._rank |
+-----+-----+-----+-----+
| "player100" | "player101" | 17 | 0 |
| "player100" | "player125" | 17 | 0 |
+-----+-----+-----+-----+
```

 版本兼容性

从 TRSGraph 2.6.0 起支持了新的 Schema 相关函数。以上示例在 TRSGraph 10.0 中的近似写法如下：

```
GO FROM "player100" OVER follow YIELD properties(edge).degree;
GO FROM "player100" OVER follow YIELD src(edge), dst(edge), type(edge), rank(edge);
```

TRSGraph 10.0 兼容新旧语法。

复合查询中使用属性引用符

以下复合查询语句示例执行操作如下：

- 1. 使用 `$.id` 引用管道符前面的 `GO FROM "player100" OVER follow YIELD dst(edge) AS id` 语句的结果，即返回 `follow` 边类型的目的点 ID。
- 2. 使用 `properties($^)` 函数获取 `serve` 类型边上起始点的球员的 `name` 属性。
- 3. 使用 `properties($$)` 函数获取 `serve` 类型边上目的点的团队的 `name` 属性。

```
trsgraph> GO FROM "player100" OVER follow \
 YIELD dst(edge) AS id | \
 GO FROM $.id OVER serve \
 YIELD properties($^).name AS Player, properties($$).name AS Team;
+-----+-----+
| Player | Team |
+-----+-----+
"Tony Parker"	"Spurs"
"Tony Parker"	"Hornets"
"Manu Ginobili"	"Spurs"
+-----+-----+
```

## 4.8 图空间语句

### 4.8.1 CREATE SPACE

图空间是 TRSGraph 中彼此隔离的图数据集，与 MySQL 中的 database 概念类似。`CREATE SPACE` 语句可以创建一个新的图空间，或者克隆现有图空间的 Schema。

#### 前提条件

只有 God 角色的用户可以执行 `CREATE SPACE` 语句。详情请参见[身份验证](#)。

#### 语法

##### 创建图空间

```
CREATE SPACE [IF NOT EXISTS] <graph_space_name> (
 [partition_num = <partition_number>],
 [replica_factor = <replica_number>],
 vid_type = {FIXED_STRING(<N>) | INT[64]}
)
[COMMENT = '<comment>'];
```

| 参数                 | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IF NOT EXISTS      | 检测待创建的图空间是否存在，只有不存在时，才会创建图空间。仅检测图空间的名称，不会检测具体属性。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <graph_space_name> | <div>1、在 TRSGraph 实例中唯一标识一个图空间。</div> <div>2、图空间名称设置后无法被修改。</div> <div>3、默认情况下，仅支持 1~4 字节的 UTF-8 编码字符，包括英文字母（区分大小写）、数字、中文等。不能包含下划线（<code>_</code>）以外的特殊字符，且不能以数字开头。</div> <div>4、如果需要使用特殊字符、保留关键字或者以数字开头，请使用反引号（<code>`</code>）包围整个名称。反引号中不能包含英文句号（<code>.</code>）。详情参见<a href="#">关键字和保留字</a>。</div> <div>注意：</div> <div>1、如果以中文为图空间命名，报 <code>SyntaxError</code> 错误时，需使用反引号（<code>`</code>）包围中文字符。</div> <div>2、名称中如需包含反引号（<code>`</code>），使用反斜杠（<code>\</code>）来转义反引号（<code>`</code>），如：<code>\`</code>；如需包含反斜杠（<code>\</code>），反斜杠（<code>\</code>）本身也需转义，如：<code>\\</code>。</div> |
| partition_num      | 指定图空间的分片数量。建议设置为集群中硬盘数量的 20 倍（HDD 硬盘建议为 2 倍）。例如集群中有 3 个硬盘，建议设置 60 个分片。默认值为 10。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| replica_factor     | 指定每个分片的副本数量。建议在生产环境中设置为 3，在测试环境中设置为 1。由于需要基于多数表决，副本数量必须是奇数。默认值为 1。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| vid_type           | <div>必选参数。指定点 ID 的数据类型。可选值为 <code>FIXED_STRING(&lt;N&gt;)</code> 和 <code>INT64</code>。<code>INT</code> 等同于 <code>INT64</code>。</div> <div><code>FIXED_STRING(&lt;N&gt;)</code> 表示数据类型为定长字符串，长度为 <code>N</code> 字节，超出长度会报错。例如，UTF-8 中，一个中文字符的长度为三个字节，如果设置 <code>N</code> 为 12，那么 <code>vid_type</code> 为最多 4 个中文字符。</div> <div><code>INT64</code> 表示数据类型为整数。</div>                                                                                                                                                                                                                     |
| COMMENT            | 图空间的描述。最大为 256 字节。默认无描述。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |



### Caution

- 如果将副本数设置为 1，用户将无法使用 `SUBMIT JOB BALANCE` 命令为 TRSGraph 的存储服务平衡负载或扩容。
- VID 类型变更与长度限制：
  - 在 TRSGraph 1.x 中，VID 的类型只能为 `INT64`，不支持字符型；在 TRSGraph 2.x 中，VID 的类型支持 `INT64` 和 `FIXED_STRING(<N>)`。请在创建图空间时指定 VID 类型，使用 `INSERT` 语句时也需要保持一致，否则会报错 `Wrong vertex id type: 1001`。
  - VID 最大长度必须为 `N`，不可任意长度；超过该长度也会报错 `The VID must be a 64-bit integer or a string fitting space vertex id length limit.`。
- 如果出现 `Host not enough!` 的报错，直接原因是线上 Storage Host 的数量少于即将创建的图空间的 `replica_factor` 的数量。此时，可用 `SHOW HOSTS` 指令查看判断，出现的情况有：
  - 在集群是单 Storage Host 的情况下试图创建多副本（`replica_factor`）图空间，这时候只能选择创建单副本数的图空间，或者扩容 Storage Host 之后再创建图空间。
  - 新创建的集群里 Storage Host 已经被服务发现，但是尚未执行 `ADD HOSTS` 将其激活，这时候需要通过 Console 连接，执行 `SHOW HOSTS` 获取被发现了的 Storage Host，然后执行相应的 `ADD HOSTS` 激活，待有足够的 Online Storage Host 之后再尝试创建图空间。
- 有部分 Storage Host 处在非 Online 状态，需要进行进一步排查。



### 版本兼容性

2.5.0 之前的 2.x 版本中，`vid_type` 不是必选参数，默认为 `FIXED_STRING(8)`。



### Note

`graph_space_name`，`partition_num`，`replica_factor`，`vid_type`，`comment` 设置后就无法改变。除非 `DROP SPACE`，并重新 `CREATE SPACE`。

### 克隆图空间

```
CREATE SPACE [IF NOT EXISTS] <new_graph_space_name> AS <old_graph_space_name>;
```

| 参数                                        | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>IF NOT EXISTS</code>                | 检测待创建的图空间是否存在，只有不存在时，才会克隆图空间。仅检测图空间的名称，不会检测具体属性。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>&lt;new_graph_space_name&gt;</code> | <p>目标图空间名称。该图空间必须未创建。</p> <p>默认情况下，仅支持 1~4 字节的 UTF-8 编码字符，包括英文字母（区分大小写）、数字、中文等，但是特殊字符只能使用下划线。不能以数字开头。</p> <p>如果需要使用特殊字符、保留关键字或者以数字开头，请使用反引号（<code>`</code>）包围整个名称。反引号中不能包含英文句号（<code>.</code>）。详情参见<a href="#">关键字和保留字</a>。</p> <p>创建时会克隆 <code>&lt;old_graph_space_name&gt;</code> 图空间的 Schema，包括图空间本身参数（分片数量、副本数量等）、Tag、Edge type 和 原生索引。</p> <p>注意：</p> <ol style="list-style-type: none"><li>如果以中文为图空间命名，报 <code>SyntaxError</code> 错误时，需使用反引号（<code>`</code>）包围中文字符。</li><li>名称中如需包含反引号（<code>`</code>），使用反斜杠（<code>\</code>）来转义反引号（<code>`</code>），如：<code>\`</code>；如需包含反斜杠（<code>\</code>），反斜杠（<code>\</code>）本身也需转义，如：<code>\\</code>。</li></ol> |
| <code>&lt;old_graph_space_name&gt;</code> | 原始图空间名称。该图空间必须已存在。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

### 示例

```
仅指定 VID 类型，其他选项使用默认值。
trsgraph> CREATE SPACE IF NOT EXISTS my_space_1 (vid_type=FIXED_STRING(30));
```



## 4.8.2 USE

`USE` 语句可以指定一个图空间，或切换到另一个图空间，将其作为后续查询的工作空间。

### 前提条件

执行 `USE` 语句指定图空间时，需要当前登录的用户拥有指定图空间的权限，否则会报错。

### 语法

```
USE <graph_space_name>;
```

### 示例

```
创建示例空间。
trsgraph> CREATE SPACE IF NOT EXISTS space1 (vid_type=FIXED_STRING(30));
trsgraph> CREATE SPACE IF NOT EXISTS space2 (vid_type=FIXED_STRING(30));

指定图空间 space1 作为工作空间。
trsgraph> USE space1;

切换到图空间 space2。检索 space2 时，无法从 space1 读取任何数据，检索的点和边与 space1 无关。
trsgraph> USE space2;
```

### Caution

不能在一条语句中同时操作两个图空间。

与 Fabric Cypher 不同，TRSGraph 的图空间彼此之间是完全隔离的，将一个图空间作为工作空间后，用户无法访问其他空间。使用新图空间的唯一方法是通过 `USE` 语句切换。而在 Fabric Cypher 中可以在一条语句中 (`USE + CALL` 语法) 使用两个图空间。

最后更新: 2025年12月19日

### 4.8.3 SHOW SPACES

---

`SHOW SPACES` 语句可以列出 TRSGraph 示例中的所有图空间。

#### 语法

```
SHOW SPACES;
```

#### 示例

```
trsgraph> SHOW SPACES;
+-----+
| Name |
+-----+
| "cba" |
| "basketballplayer" |
+-----+
```

创建图空间请参见 [CREATE SPACE](#)。

---

最后更新: 2025年12月19日

## 4.8.4 DESCRIBE SPACE

`DESCRIBE SPACE` 语句可以显示指定图空间的信息。

### 语法

你可以用 `DESC` 作为 `DESCRIBE` 的缩写。

```
DESC[RIBE] SPACE <graph_space_name>;
```

### 示例

```
trsgraph> DESCRIBE SPACE basketballplayer;
+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | Partition Number | Replica Factor | Charset | Collate | Vid Type | Comment |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | "basketballplayer" | 10 | 1 | "utf8" | "utf8_bin" | "FIXED_STRING(32)" | |
+-----+-----+-----+-----+-----+-----+-----+
```

最后更新: 2025年12月19日



### 4.8.5 CLEAR SPACE

`CLEAR SPACE` 语句用于清空图空间中的点和边，但不会删除图空间本身以及其中的 Schema 信息。



建议在执行 `CLEAR SPACE` 操作之后，立即执行 `SUBMIT JOB COMPACT` 操作以提升查询性能。需要注意的是，`COMPACT` 操作可能会影响查询性能，建议在业务低峰期（例如凌晨）执行该操作。

#### 权限要求

只有 `God` 角色的用户可以执行 `CLEAR SPACE` 语句。

#### 注意事项

- 数据清除后，如无备份，无法恢复。使用该功能务必谨慎。
- `CLEAR SPACE` 不是原子性操作。如果执行出错，请重新执行，避免残留数据。
- 图空间中的数据量越大，`CLEAR SPACE` 消耗的时间越长。如果 `CLEAR SPACE` 的执行因客户端连接超时而失败，可以增大 `Graph 服务配置` 中 `storage_client_timeout_ms` 参数的值。
- 在 `CLEAR SPACE` 的执行过程中，向该图空间写入数据的行为不会被自动禁止。这样的写入行为可能导致 `CLEAR SPACE` 清除数据不完全，残留的数据也可能受到损坏。



TRSGraph 不支持在运行 `CLEAR SPACE` 的同时禁止写入。

#### 语法

```
CLEAR SPACE [IF EXISTS] <space_name>;
```

| 参数/选项                   | 说明                                                                                                                           |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>IF EXISTS</code>  | 检查待清空的图空间是否存在，如果图空间存在，则继续执行清空操作；如果图空间不存在，则完成执行，并且提示执行成功，不会提示图空间不存在。若不设置该选项，当图空间不存在时， <code>CLEAR SPACE</code> 语句会执行失败，系统会报错。 |
| <code>space_name</code> | 要清除数据的图空间名称。                                                                                                                 |

示例：

```
CLEAR SPACE basketballplayer;
```

#### 保留的数据

图空间中，`CLEAR SPACE` 不会删除的数据包括：

- Tag 信息。
- Edge type 信息。
- 原生索引和全文索引的元数据。

下面的执行示例明确展示了 `CLEAR SPACE` 会删除与保留的数据。

```
进入图空间 basketballplayer。
trsgraph[(none)]> use basketballplayer;
```

Execution succeeded

# 查看 Tag 和 Edge type.

trsgraph[basketballplayer]> SHOW TAGS;

```
+-----+
| Name |
+-----+
| "player" |
| "team" |
+-----+
Got 2 rows
```

trsgraph[basketballplayer]> SHOW EDGES;

```
+-----+
| Name |
+-----+
| "follow" |
| "serve" |
+-----+
Got 2 rows
```

# 统计图空间中的数据。

trsgraph[basketballplayer]> SUBMIT JOB STATS;

```
+-----+
| New Job Id |
+-----+
| 4 |
+-----+
Got 1 rows
```

# 查看统计结果。

trsgraph[basketballplayer]> SHOW STATS;

```
+-----+-----+-----+
| Type | Name | Count |
+-----+-----+-----+
"Tag"	"player"	51
"Tag"	"team"	30
"Edge"	"follow"	81
"Edge"	"serve"	152
"Space"	"vertices"	81
"Space"	"edges"	233
+-----+-----+-----+
Got 6 rows
```

# 查看 Tag 索引。

trsgraph[basketballplayer]> SHOW TAG INDEXES;

```
+-----+-----+-----+
| Index Name | By Tag | Columns |
+-----+-----+-----+
| "player_index_0" | "player" | [] |
| "player_index_1" | "player" | ["name"] |
+-----+-----+-----+
Got 2 rows
```

# ----- CLEAR SPACE 分割线 -----

# 执行 CLEAR SPACE 清空图空间 basketballplayer。

trsgraph[basketballplayer]> CLEAR SPACE basketballplayer;

Execution succeeded

# 更新统计信息。

trsgraph[basketballplayer]> SUBMIT JOB STATS;

```
+-----+
| New Job Id |
+-----+
| 5 |
+-----+
Got 1 rows
```

# 查看统计信息。Tag 和 Edge type 还存在，但所有的点和边都没了。

trsgraph[basketballplayer]> SHOW STATS;

```
+-----+-----+-----+
| Type | Name | Count |
+-----+-----+-----+
"Tag"	"player"	0
"Tag"	"team"	0
"Edge"	"follow"	0
"Edge"	"serve"	0
"Space"	"vertices"	0
"Space"	"edges"	0
+-----+-----+-----+
Got 6 rows
```

# 查看 Tag 索引，它们依然存在。

trsgraph[basketballplayer]> SHOW TAG INDEXES;

```
+-----+-----+-----+
| Index Name | By Tag | Columns |
+-----+-----+-----+
| "player_index_0" | "player" | [] |
| "player_index_1" | "player" | ["name"] |
+-----+-----+-----+
Got 2 rows (time spent 523/978 us)
```

最后更新: 2025年12月19日

### 4.8.6 DROP SPACE

`DROP SPACE` 语句用于删除指定图空间以及其中的所有信息。



Note

`DROP SPACE` 是否删除图空间对应的硬盘数据由 Storage 配置参数 `auto_remove_invalid_space` 决定。`auto_remove_invalid_space` 的默认值为 `true`，表示会删除数据。如需在删除逻辑图空间时保留硬盘数据，将 `auto_remove_invalid_space` 的值修改为 `false`。详情参见 [Storage 服务配置](#)。



Warning

执行 `DROP SPACE` 后，即使快照中存在该图空间的数据，该图空间的数据也无法恢复。

#### 前提条件

只有 God 角色的用户可以执行 `DROP SPACE` 语句。详情请参见[身份验证](#)。

#### 语法

```
DROP SPACE [IF EXISTS] <graph_space_name>;
```

`IF EXISTS` 关键字可以检测待删除的图空间是否存在，只有存在时，才会删除图空间。



兼容性

在 TRSGraph 3.1.0 版本前，`DROP SPACE` 语句不会删除硬盘上对应图空间的目录和文件。



Danger

请谨慎执行删除图空间操作。

#### FAQ

问：执行 `DROP SPACE` 语句删除图空间后，为什么磁盘的大小没变化？

答：如果使用 3.1.0 之前版本的 TRSGraph，`DROP SPACE` 语句仅删除指定的逻辑图空间，不会删除硬盘上对应图空间的目录和文件。如需删除硬盘上的数据，需手动删除相应文件的路径，文件路径为 `<trsgaph_install_path>/data/storage/nebula/<space_id>`。其中 `<space_id>` 可以通过 `DESCRIBE SPACE {space_name}` 查看。

最后更新: 2025年12月19日

## 4.9 Tag 语句

---

### 4.9.1 CREATE TAG

---

`CREATE TAG` 语句可以通过指定名称创建一个 Tag。

#### OpenCypher 兼容性

nGQL 中的 Tag 和 openCypher 中的 Label 相似，但又有所不同，例如它们的创建方式。

- openCypher 中的 Label 需要在 `CREATE` 语句中与点一起创建。
- nGQL 中的 Tag 需要使用 `CREATE TAG` 语句独立创建。Tag 更像是 MySQL 中的表。

#### 前提条件

执行 `CREATE TAG` 语句需要当前登录的用户拥有指定图空间的创建 Tag 权限，否则会报错。

#### 语法

创建 Tag 前，需要先用 `USE` 语句指定工作空间。

```
CREATE TAG [IF NOT EXISTS] <tag_name>
(
 <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']
 [{, <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']} ...]
)
[TTL_DURATION = <ttl_duration>]
```

```
[TTL_COL = <prop_name>]
[COMMENT = '<comment>'];
```

| 参数              | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IF NOT EXISTS   | 检测待创建的 Tag 是否存在，只有不存在时，才会创建 Tag。仅检测 Tag 的名称，不会检测具体属性。                                                                                                                                                                                                                                                                                                                                                                             |
| <tag_name>      | <div>1、每个图空间内的 Tag 必须是唯一的。</div> <div>2、Tag 名称设置后无法修改。</div> <div>3、默认情况下，仅支持 1~4 字节的 UTF-8 编码字符，包括英文字母（区分大小写）、数字、中文等。不能包含下划线（_）以外的特殊字符，且不能以数字开头。</div> <div>4、如果需要使用特殊字符、保留关键字或者以数字开头，请使用反引号（`）包围整个名称。反引号中不能包含英文句号（.）。详情参见<a href="#">关键字和保留字</a>。</div> <div>注意：</div> <div>1、如果以中文命名 Tag，报 <code>SyntaxError</code> 错误时，需使用反引号（`）包围中文字符。</div> <div>2、名称中如需包含反引号（`），使用反斜杠（\）来转义反引号（`），如：\`；如需包含反斜杠（\），反斜杠（\）本身也需转义，如：\\。</div> |
| <prop_name>     | 属性名称。每个 Tag 中的属性名称必须唯一。属性的命名规则与 Tag 相同。                                                                                                                                                                                                                                                                                                                                                                                           |
| <data_type>     | 属性的数据类型，目前支持 <a href="#">数值</a> 、 <a href="#">布尔</a> 、 <a href="#">字符串</a> 、 <a href="#">日期与时间</a> 以及 <a href="#">地理空间</a> 。                                                                                                                                                                                                                                                                                                      |
| NULL   NOT NULL | 指定属性值是否支持为 NULL。默认值为 NULL。指定 NOT NULL 且没有指定 DEFAULT 值时，插入数据必须指定值；同时指定 NOT NULL 和 DEFAULT 值时，插入数据如果没有指定值，则默认插入 DEFAULT 值。                                                                                                                                                                                                                                                                                                          |
| DEFAULT         | 指定属性的默认值。默认值可以是一个文字值或 TRSGraph 支持的表达式。如果插入点时没有指定某个属性的值，则使用默认值。                                                                                                                                                                                                                                                                                                                                                                    |
| COMMENT         | 对单个属性或 Tag 的描述。最大为 256 字节。默认无描述。                                                                                                                                                                                                                                                                                                                                                                                                  |
| TTL_DURATION    | 指定时间戳差值，单位：秒。时间戳差值必须为 64 位非负整数。属性值和时间戳差值之和如果小于当前时间戳，属性就会过期。默认值为 0，表示属性永不过期。                                                                                                                                                                                                                                                                                                                                                       |
| TTL_COL         | 指定要设置存活时间的属性。属性的数据类型必须是 <code>int</code> 或者 <code>timestamp</code> 。一个 Tag 只能指定一个字段为 TTL_COL。更多信息请参见 <a href="#">TTL</a> 。                                                                                                                                                                                                                                                                                                        |

示例

```
trsgraph> CREATE TAG IF NOT EXISTS player(name string, age int);

创建没有属性的 Tag。
trsgraph> CREATE TAG IF NOT EXISTS no_property();

创建包含默认值的 Tag。
trsgraph> CREATE TAG IF NOT EXISTS player_with_default(name string, age int DEFAULT 20);

对字段 create_time 设置 TTL 为 100 秒。
trsgraph> CREATE TAG IF NOT EXISTS woman(name string, age int, \
 married bool, salary double, create_time timestamp) \
 TTL_DURATION = 100, TTL_COL = "create_time";
```

创建 Tag 说明

尝试使用新创建的 Tag 可能会失败，因为创建是异步实现的。为确保数据同步，后续操作能顺利进行，请等待 2 个心跳周期（20 秒）。

如果需要修改心跳间隔，请为[所有配置文件](#)修改参数 `heartbeat_interval_secs`。

最后更新: 2025年12月19日

## 4.9.2 DROP TAG

`DROP TAG` 语句可以删除当前工作空间内所有点上的指定 Tag。

点可以有一个或多个 Tag。

- 如果某个点只有一个 Tag，删除这个 Tag 后，用户就无法访问这个点，下次 Compaction 操作时会删除该点，但与该点相邻的边仍然存在——这会造成悬挂边。
- 如果某个点有多个 Tag，删除其中一个 Tag，仍然可以访问这个点，但是无法访问已删除 Tag 所定义的所有属性。

删除 Tag 操作仅删除 Schema 数据，硬盘上的文件或目录不会立刻删除，而是在下一次 Compaction 操作时删除。



Compatibility

TRSGraph 10.0 中默认不支持插入无 Tag 的点。如需使用无 Tag 的点，在集群内所有 Graph 服务的配置文件（`trsgraph-graphd.conf`）中新增 `--graph_use_vertex_key=true`；在所有 Storage 服务的配置文件（`trsgraph-storaged.conf`）中新增 `--use_vertex_key=true`。

### 前提条件

- 登录的用户必须拥有对应权限才能执行 `DROP TAG` 语句。详情请参见[内置角色权限](#)。
- 确保 Tag 不包含任何索引，否则 `DROP TAG` 时会报冲突错误 `[ERROR (-1005)]: Conflict!`。删除索引请参见[drop index](#)。

### 语法

```
DROP TAG [IF EXISTS] <tag_name>;
```

- `IF EXISTS`：检测待删除的 Tag 是否存在，只有存在时，才会删除 Tag。
- `tag_name`：指定要删除的 Tag 名称。一次只能删除一个 Tag。

### 示例

```
trsgraph> CREATE TAG IF NOT EXISTS test(p1 string, p2 int);
trsgraph> DROP TAG test;
```

最后更新: 2025年12月19日

### 4.9.3 ALTER TAG

`ALTER TAG` 语句可以修改 Tag 的结构。例如增删属性、修改数据类型，也可以为属性设置、修改 `TTL`（Time-To-Live）。

#### 注意事项

- 登录的用户必须拥有对应权限才能执行 `ALTER TAG` 语句。详情请参见[内置角色权限](#)。
- 确保要修改的属性不包含索引，否则 `ALTER TAG` 时会报冲突错误 `[ERROR (-1005)]: Conflict!`。删除索引请参见 [drop index](#)。
- 确保新增的属性名不与已存在或被删除的属性名同名，否则新增属性会失败。

#### 语法

```
ALTER TAG <tag_name>
 <alter_definition> [[, alter_definition] ...]
 [ttl_definition [, ttl_definition] ...]
 [COMMENT = '<comment>'];

alter_definition:
| ADD (prop_name data_type [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>'])
| DROP (prop_name)
| CHANGE (prop_name data_type [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>'])

ttl_definition:
 TTL_DURATION = ttl_duration, TTL_COL = prop_name
```

- `tag_name`：指定要修改的 Tag 名称。一次只能修改一个 Tag。请确保要修改的 Tag 在当前工作空间中存在，否则会报错。
- 可以在一个 `ALTER TAG` 语句中使用多个 `ADD`、`DROP` 和 `CHANGE` 子句，子句之间用英文逗号（,）分隔。
- 当使用 `ADD` 或 `CHANGE` 指定属性值为 `NOT NULL` 时，必需为该属性指定默认值，即定义 `DEFAULT` 的值。
- 当使用 `CHANGE` 修改属性的数据类型时：
  - 仅允许修改 `FIXED_STRING` 和 `INT` 类型的长度为更大的长度，不允许减少长度。
  - 仅允许修改 `FIXED_STRING` 类型为 `STRING` 类型、修改 `FLOAT` 类型为 `DOUBLE` 类型。

#### 示例

```
trsgraph> CREATE TAG IF NOT EXISTS t1 (p1 string, p2 int);
trsgraph> ALTER TAG t1 ADD (p3 int32, p4 fixed_string(10));
trsgraph> ALTER TAG t1 TTL_DURATION = 2, TTL_COL = "p2";
trsgraph> ALTER TAG t1 COMMENT = 'test1';
trsgraph> ALTER TAG t1 ADD (p5 double NOT NULL DEFAULT 0.4 COMMENT 'p5') COMMENT='test2';
// 将 TAG t1 的 p3 属性类型从 INT32 改为 INT64, p4 属性类型从 FIXED_STRING(10) 改为 STRING。
trsgraph> ALTER TAG t1 CHANGE (p3 int64, p4 string);
```

#### 修改 Tag 说明

尝试使用刚修改的 Tag 可能会失败，因为修改是异步实现的。为确保数据同步，后续操作能顺利进行，请等待 2 个心跳周期（20 秒）。

如果需要修改心跳间隔，请为[所有配置文件](#)修改参数 `heartbeat_interval_secs`。

最后更新: 2025年12月19日



## 4.9.4 SHOW TAGS

---

`SHOW TAGS` 语句显示当前图空间内的所有 Tag 名称。

执行 `SHOW TAGS` 语句不需要任何权限，但是返回结果由登录的用户权限决定。

### 语法

```
SHOW TAGS;
```

### 示例

```
trsgraph> SHOW TAGS;
+-----+
| Name |
+-----+
| "player" |
| "team" |
+-----+
```

---

最后更新: 2025年12月19日

### 4.9.5 DESCRIBE TAG

`DESCRIBE TAG` 显示指定 Tag 的详细信息，例如字段名称、数据类型等。

**前提条件**

登录的用户必须拥有对应权限才能执行 `DESCRIBE TAG` 语句。详情请参见[内置角色权限](#)。

**语法**

```
DESC[RIBE] TAG <tag_name>;
```

`DESCRIBE` 可以缩写为 `DESC`。

**示例**

```
trsgraph> DESCRIBE TAG player;
+-----+-----+-----+-----+
| Field | Type | Null | Default | Comment |
+-----+-----+-----+-----+
| "name" | "string" | "YES" | | |
| "age" | "int64" | "YES" | | |
+-----+-----+-----+-----+
```

最后更新: 2025年12月19日

## 4.9.6 DELETE TAG

`DELETE TAG` 语句可以删除指定点上的指定 Tag。

### 前提条件

登录的用户必须拥有对应权限才能执行 `DELETE TAG` 语句。详情请参见[内置角色权限](#)。

### 语法

```
DELETE TAG <tag_name_list> FROM <VID_list>;
```

- `tag_name_list`：指定 Tag 名称。多个 Tag 用英文逗号 (,) 分隔，也可以用 `*` 表示所有 Tag。
- `VID_list`：指定要删除 Tag 的点 ID。可以指定多个 VID，用英文逗号 (,) 分隔。

### 示例

```
trsgraph> CREATE TAG IF NOT EXISTS test1(p1 string, p2 int);
trsgraph> CREATE TAG IF NOT EXISTS test2(p3 string, p4 int);
trsgraph> INSERT VERTEX test1(p1, p2),test2(p3, p4) VALUES "test":("123", 1, "456", 2);
trsgraph> FETCH PROP ON * "test" YIELD vertex AS v;
+-----+
| v |
+-----+
| ("test" :test1{p1: "123", p2: 1} :test2{p3: "456", p4: 2}) |
+-----+

trsgraph> DELETE TAG test1 FROM "test";
trsgraph> FETCH PROP ON * "test" YIELD vertex AS v;
+-----+
| v |
+-----+
| ("test" :test2{p3: "456", p4: 2}) |
+-----+

trsgraph> DELETE TAG * FROM "test";
trsgraph> FETCH PROP ON * "test" YIELD vertex AS v;
+----+
| v |
+----+
+----+
```



#### Compatibility

- 在 openCypher 中，可以使用 `REMOVE v:LABEL` 语句来移除该点 `v` 的 `LABEL`。
- 相同语意，但不同语法。在 nGQL 中使用 `DELETE TAG`。

最后更新: 2025年12月19日

### 4.9.7 增加和删除标签

在 openCypher 中，有增加标签（`SET label`）和移除标签（`REMOVE label`）的功能，可以用于加速查询或者标记过程。

在 TRSGraph 中，可以通过 Tag 变相实现相同操作，创建 Tag 并将 Tag 插入到已有的点上，就可以根据 Tag 名称快速查找点，也可以通过 `DELETE TAG` 删除某些点上不再需要的 Tag。

#### 示例

例如在 basketballplayer 数据集中，部分篮球运动员同时也是球队股东，可以为股东 Tag `shareholder` 创建索引，方便快速查找。如果不再是股东，可以通过 `DELETE TAG` 语句删除相应运动员的股东 Tag。

```
//创建股东 Tag 和索引
trsgraph> CREATE TAG IF NOT EXISTS shareholder();
trsgraph> CREATE TAG INDEX IF NOT EXISTS shareholder_tag on shareholder();

//为点添加 Tag
trsgraph> INSERT VERTEX shareholder() VALUES "player100":();
trsgraph> INSERT VERTEX shareholder() VALUES "player101":();

//快速查询所有股东
trsgraph> MATCH (v:shareholder) RETURN v;
+-----+
| v |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"} :shareholder{}) |
| ("player101" :player{age: 36, name: "Tony Parker"} :shareholder{}) |
+-----+
trsgraph> LOOKUP ON shareholder YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player100" |
| "player101" |
+-----+

//如果 player100 不再是股东
trsgraph> DELETE TAG shareholder FROM "player100";
trsgraph> LOOKUP ON shareholder YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "player101" |
+-----+
```



Note

如果插入测试数据后才创建索引，请用 `REBUILD TAG INDEX <index_name_list>;` 语句重建索引。

最后更新: 2025年12月19日

## 4.10 Edge type 语句

---

### 4.10.1 CREATE EDGE

---

`CREATE EDGE` 语句可以通过指定名称创建一个 Edge type。

#### OpenCypher 兼容性

nQL 中的 Edge type 和 openCypher 中的关系类型相似，但又有所不同，例如它们的创建方式。

- openCypher 中的关系类型需要在 `CREATE` 语句中与点一起创建。
- nQL 中的 Edge type 需要使用 `CREATE EDGE` 语句独立创建。Edge type 更像是 MySQL 中的表。

#### 前提条件

执行 `CREATE EDGE` 语句需要当前登录的用户拥有指定图空间的创建 Edge type 权限，否则会报错。

#### 语法

创建 Edge type 前，需要先用 `USE` 语句指定工作空间。

```
CREATE EDGE [IF NOT EXISTS] <edge_type_name>
(
 <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']
 [{, <prop_name> <data_type> [NULL | NOT NULL] [DEFAULT <default_value>] [COMMENT '<comment>']} ...]
)
[TTL_DURATION = <ttd_duration>]
```

| <pre>[TTL_COL = &lt;prop_name&gt;] [COMMENT = '&lt;comment&gt;'];</pre> |                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 参数                                                                      | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>IF NOT EXISTS</code>                                              | 检测待创建的 Edge type 是否存在，只有不存在时，才会创建 Edge type。仅检测 Edge type 的名称，不会检测具体属性。                                                                                                                                                                                                                                                                                                                                                                             |
| <code>&lt;edge_type_name&gt;</code>                                     | <div>1、每个图空间内的 Edge type 必须是唯一的。</div> <div>2、Edge type 名称设置后无法修改。</div> <div>3、默认情况下，仅支持 1~4 字节的 UTF-8 编码字符，包括英文字母（区分大小写）、数字、中文等。不能包含下划线（_）以外的特殊字符，且不能以数字开头。</div> <div>4、如果需要使用特殊字符、保留关键字或者以数字开头，请使用反引号（`）包围整个名称。反引号中不能包含英文句号（.）。详情参见<a href="#">关键字和保留字</a>。</div> <div>注意：</div> <div>1、如果以中文命名 Edge type，报 <code>SyntaxError</code> 错误时，需使用反引号（`）包围中文字符。</div> <div>2、名称中如需包含反引号（`），使用反斜杠（\）来转义反引号（`），如：\`；如需包含反斜杠（\），反斜杠（\）本身也需转义，如：\\。</div> |
| <code>&lt;prop_name&gt;</code>                                          | 属性名称。每个 Edge type 中的属性名称必须唯一。属性的命名规则与 Edge type 相同。                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>&lt;data_type&gt;</code>                                          | 属性的数据类型，目前支持 <a href="#">数值</a> 、 <a href="#">布尔</a> 、 <a href="#">字符串</a> 、 <a href="#">日期与时间</a> 以及 <a href="#">地理空间</a> 。                                                                                                                                                                                                                                                                                                                        |
| <code>NULL   NOT NULL</code>                                            | 指定属性值是否支持为 <code>NULL</code> 。默认值为 <code>NULL</code> 。当指定属性值为 <code>NOT NULL</code> 时，必需指定属性的默认值，也就是 <code>DEFAULT</code> 的值。                                                                                                                                                                                                                                                                                                                       |
| <code>DEFAULT</code>                                                    | 指定属性的默认值。默认值可以是一个文字值或 TRSGraph 支持的表达式。如果插入边时没有指定某个属性的值，则使用默认值。                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>COMMENT</code>                                                    | 对单个属性或 Edge type 的描述。最大为 256 字节。默认无描述。                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>TTL_DURATION</code>                                               | 指定时间戳差值，单位：秒。时间戳差值必须为 64 位非负整数。属性值和时间戳差值之和如果小于当前时间戳，属性就会过期。默认值为 0，表示属性永不过期。                                                                                                                                                                                                                                                                                                                                                                         |
| <code>TTL_COL</code>                                                    | 指定要设置存活时间的属性。属性的数据类型必须是 <code>int</code> 或者 <code>timestamp</code> 。一个 Edge type 只能指定一个字段为 <code>TTL_COL</code> 。更多 TTL 的信息请参见 <a href="#">TTL</a> 。                                                                                                                                                                                                                                                                                                |

示例

```
trsgraph> CREATE EDGE IF NOT EXISTS follow(degree int);

创建没有属性的 Edge type.
trsgraph> CREATE EDGE IF NOT EXISTS no_property();

创建包含默认值的 Edge type.
trsgraph> CREATE EDGE IF NOT EXISTS follow_with_default(degree int DEFAULT 20);

对字段 p2 设置 TTL 为 100 秒。
trsgraph> CREATE EDGE IF NOT EXISTS e1(p1 string, p2 int, p3 timestamp) \
 TTL_DURATION = 100, TTL_COL = "p2";
```

最后更新: 2025年12月19日

## 4.10.2 DROP EDGE

`DROP EDGE` 语句可以删除当前工作空间内的指定 Edge type。

一个边只能有一个 Edge type，删除这个 Edge type 后，用户就无法访问这个边，下次 Compaction 操作时会删除该边。

删除 Edge type 操作仅删除 Schema 数据，硬盘上的文件或目录不会立刻删除，而是在下一次 Compaction 操作时删除。

### 前提条件

- 登录的用户必须拥有对应权限才能执行 `DROP EDGE` 语句。详情请参见[内置角色权限](#)。
- 确保 Edge type 不包含任何索引，否则 `DROP EDGE` 时会报冲突错误 `[ERROR (-1005)]: Conflict!`。删除索引请参见[drop index](#)。

### 语法

```
DROP EDGE [IF EXISTS] <edge_type_name>
```

- `IF EXISTS`：检测待删除的 Edge type 是否存在，只有存在时，才会删除 Edge type。
- `edge_type_name`：指定要删除的 Edge type 名称。一次只能删除一个 Edge type。

### 示例

```
trsgraph> CREATE EDGE IF NOT EXISTS e1(p1 string, p2 int);
trsgraph> DROP EDGE e1;
```

---

最后更新: 2025年12月19日

### 4.10.3 ALTER EDGE

`ALTER EDGE` 语句可以修改 Edge type 的结构。例如增删属性、修改数据类型，也可以为属性设置、修改 `TTL`（Time-To-Live）。

#### 注意事项

- 登录的用户必须拥有对应权限才能执行 `ALTER EDGE` 语句。详情请参见[内置角色权限](#)。
- 确保要修改的属性不包含索引，否则 `ALTER EDGE` 时会报冲突错误 `[ERROR (-1005)]: Conflict!`。删除索引请参见[drop index](#)。
- 确保新增的属性名不与已存在或被删除的属性名同名，否则新增属性会失败。
- 允许增加 `FIXED_STRING` 和 `INT` 类型的长度。
- 允许 `FIXED_STRING` 类型转换为 `STRING` 类型、`FLOAT` 类型转换为 `DOUBLE` 类型。

#### 语法

```
ALTER EDGE <edge_type_name>
 <alter_definition> [, alter_definition] ...]
 [ttl_definition [, ttl_definition] ...]
 [COMMENT = '<comment>'];

alter_definition:
| ADD (prop_name data_type)
| DROP (prop_name)
| CHANGE (prop_name data_type)

ttl_definition:
 TTL_DURATION = ttl_duration, TTL_COL = prop_name
```

- `edge_type_name`：指定要修改的 Edge type 名称。一次只能修改一个 Edge type。请确保要修改的 Edge type 在当前工作空间中存在，否则会报错。
- 可以在一个 `ALTER EDGE` 语句中使用多个 `ADD`、`DROP` 和 `CHANGE` 子句，子句之间用英文逗号（,）分隔。
- 当使用 `ADD` 或 `CHANGE` 指定属性值为 `NOT NULL` 时，必需为该属性指定默认值，即定义 `DEFAULT` 的值。

#### 示例

```
trsgraph> CREATE EDGE IF NOT EXISTS e1(p1 string, p2 int);
trsgraph> ALTER EDGE e1 ADD (p3 int, p4 string);
trsgraph> ALTER EDGE e1 TTL_DURATION = 2, TTL_COL = "p2";
trsgraph> ALTER EDGE e1 COMMENT = 'edge1';
```

#### 修改 Edge type 说明

尝试使用刚修改的 Edge type 可能会失败，因为修改是异步实现的。为确保数据同步，后续操作能顺利进行，请等待 2 个心跳周期（20 秒）。

如果需要修改心跳间隔，请为[所有配置文件](#)修改参数 `heartbeat_interval_secs`。

最后更新: 2025年12月19日



### 4.10.4 SHOW EDGES

`SHOW EDGES` 语句显示当前图空间内的所有 Edge type 名称。

执行 `SHOW EDGES` 语句不需要任何权限，但是返回结果由登录的用户权限 决定。

#### 语法

```
SHOW EDGES;
```

#### 示例

```
trsgraph> SHOW EDGES;
+-----+
| Name |
+-----+
| "follow" |
| "serve" |
+-----+
```

最后更新: 2025年12月19日

### 4.10.5 DESCRIBE EDGE

`DESCRIBE EDGE` 显示指定 Edge type 的详细信息，例如字段名称、数据类型等。

**前提条件**

登录的用户必须拥有对应权限才能执行 `DESCRIBE EDGE` 语句。详情请参见[内置角色权限](#)。

**语法**

```
DESC[RIBE] EDGE <edge_type_name>
```

`DESCRIBE` 可以缩写为 `DESC`。

**示例**

```
trsgraph> DESCRIBE EDGE follow;
+-----+-----+-----+-----+
| Field | Type | Null | Default | Comment |
+-----+-----+-----+-----+
| "degree" | "int64" | "YES" | | |
+-----+-----+-----+-----+
```

最后更新: 2025年12月19日

## 4.11 点语句

### 4.11.1 INSERT VERTEX

`INSERT VERTEX` 语句可以在 TRSGraph 实例的指定图空间中插入一个或多个点。

#### 前提条件

执行 `INSERT VERTEX` 语句需要当前登录的用户拥有指定图空间的插入点权限，否则会报错。

#### 语法

```
INSERT VERTEX [IF NOT EXISTS] [tag_props, [tag_props] ...]
VALUES VID: ([prop_value_list])

tag_props:
 tag_name ([prop_name_list])

prop_name_list:
 [prop_name [, prop_name] ...]

prop_value_list:
 [prop_value [, prop_value] ...]
```

- `IF NOT EXISTS`：检测待插入的 VID 是否存在，只有不存在时，才会插入，如果已经存在，不会进行修改。

#### Note

- `IF NOT EXISTS` 仅检测 VID + Tag 的值是否相同，不会检测属性值。
- `IF NOT EXISTS` 会先读取一次数据是否存在，因此对性能会有明显影响。

- `tag_name`：点关联的 Tag（点类型）。Tag 的创建，详情请参见 [CREATE TAG](#)。

#### Caution

TRSGraph 10.0 中默认不支持插入无 Tag 的点。如需使用无 Tag 的点，在集群内所有 Graph 服务的配置文件（`trsgraph-graphd.conf`）中新增 `--graph_use_vertex_key=true`；在所有 Storage 服务的配置文件（`trsgraph-storaged.conf`）中新增 `--use_vertex_key=true`。插入无 Tag 点的命令示例如 `INSERT VERTEX VALUES "1":()`。

- `property_name`：需要设置的属性名称。
- `vid`：点 ID。在 TRSGraph 10.0 中支持字符串和整数，需要在创建图空间时设置，详情请参见 [CREATE SPACE](#)。
- `property_value`：根据 `prop_name_list` 填写属性值。如果没有填写属性值，而 Tag 中对应的属性设置为 `NOT NULL`，会返回错误。详情请参见 [CREATE TAG](#)。

#### Caution

`INSERT VERTEX` 与 openCypher 中 `CREATE` 的语意不同：

- `INSERT VERTEX` 语意更接近于 NoSQL(key-value) 方式的 INSERT 语意，或者 SQL 中的 `UPSERT`（`UPDATE or INSERT`）。
- 相同 VID 和 TAG 的情况下，如果没有使用 `IF NOT EXISTS`，新写入的数据会覆盖旧数据，不存在时会新写入。
- 相同 VID 但不同 TAG 的情况下，不同 TAG 对应的记录不会相互覆盖，不存在会新写入。

参考以下示例。

## 示例

```
插入不包含属性的点。
trsgraph> CREATE TAG IF NOT EXISTS t1();
trsgraph> INSERT VERTEX t1() VALUES "10":();
```

```
trsgraph> CREATE TAG IF NOT EXISTS t2 (name string, age int);
trsgraph> INSERT VERTEX t2 (name, age) VALUES "11":("n1", 12);

创建失败, 因为"a13"不是 int 类型。
trsgraph> INSERT VERTEX t2 (name, age) VALUES "12":("n1", "a13");

一次插入 2 个点。
trsgraph> INSERT VERTEX t2 (name, age) VALUES "13":("n3", 12), "14":("n4", 8);
```

```
trsgraph> CREATE TAG IF NOT EXISTS t3(p1 int);
trsgraph> CREATE TAG IF NOT EXISTS t4(p2 string);

一次插入两个 Tag 的属性到同一个点。
trsgraph> INSERT VERTEX t3 (p1), t4(p2) VALUES "21": (321, "hello");
```

一个点可以多次插入属性值，以最后一次为准。

```
多次插入属性值。
trsgraph> INSERT VERTEX t2 (name, age) VALUES "11":("n2", 13);
trsgraph> INSERT VERTEX t2 (name, age) VALUES "11":("n3", 14);
trsgraph> INSERT VERTEX t2 (name, age) VALUES "11":("n4", 15);
trsgraph> FETCH PROP ON t2 "11" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 15, name: "n4"} |
+-----+
```

```
trsgraph> CREATE TAG IF NOT EXISTS t5(p1 fixed_string(5) NOT NULL, p2 int, p3 int DEFAULT NULL);
trsgraph> INSERT VERTEX t5(p1, p2, p3) VALUES "001":("Abe", 2, 3);
```

```
插入失败, 因为属性 p1 不能为 NULL。
trsgraph> INSERT VERTEX t5(p1, p2, p3) VALUES "002":(NULL, 4, 5);
[ERROR (-1009)]: SemanticError: No schema found for `t5`
```

```
属性 p3 为默认值 NULL。
trsgraph> INSERT VERTEX t5(p1, p2) VALUES "003":("cd", 5);
trsgraph> FETCH PROP ON t5 "003" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {p1: "cd", p2: 5, p3: __NULL__} |
+-----+
```

```
属性 p1 最大长度为 5, 因此会被截断。
trsgraph> INSERT VERTEX t5(p1, p2) VALUES "004":("shalalalala", 4);
trsgraph> FETCH PROP ON t5 "004" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {p1: "shala", p2: 4, p3: __NULL__} |
+-----+
```

使用 IF NOT EXISTS 插入已存在的点时，不会进行修改。

```
插入点 1。
trsgraph> INSERT VERTEX t2 (name, age) VALUES "1":("n2", 13);

使用 IF NOT EXISTS 修改点 1, 因为点 1 已存在, 不会进行修改。
trsgraph> INSERT VERTEX IF NOT EXISTS t2 (name, age) VALUES "1":("n3", 14);
trsgraph> FETCH PROP ON t2 "1" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 13, name: "n2"} |
+-----+
```

最后更新: 2025年12月19日

## 4.11.2 DELETE VERTEX

`DELETE VERTEX` 语句可以删除点，但是默认不删除该点关联的出边和入边。



Compatibility

TRSGraph 2.x 默认删除点及关联该点的出边和入边，TRSGraph 10.0 默认只删除点，不删除该点关联的出边和入边，此时将默认存在悬挂边。

`DELETE VERTEX` 语句一次可以删除一个或多个点。用户可以结合管道符一起使用，详情请参见[管道符](#)。



Note

- `DELETE VERTEX` 是直接删除点，不删除关联的边。
- `DELETE TAG` 是删除指定点上的指定 Tag。

### 语法

```
DELETE VERTEX <vid> [, <vid> ...] [WITH EDGE];
```

- `WITH EDGE`: 删除该点关联的出边和入边。

### 示例

```
删除 VID 为 `team1` 的点，不删除该点关联的出边和入边。
trsgraph> DELETE VERTEX "team1";

删除 VID 为 `team1` 的点，并删除该点关联的出边和入边。
trsgraph> DELETE VERTEX "team1" WITH EDGE;
```

```
结合管道符，删除符合条件的点。
trsgraph> GO FROM "player100" OVER serve WHERE properties(edge).start_year == "2021" YIELD dst(edge) AS id | DELETE VERTEX $-id;
```

### 删除过程

TRSGraph 找到目标点并删除，该目标点的所有邻边（出边和入边）将成为悬挂边。



Caution

- 不支持原子性删除，如果发生错误请重试，避免出现部分删除的情况。否则会导致悬挂边。
- 删除超级节点耗时较多，为避免删除完成前连接超时，可以调整 `trsgraph-graphd.conf` 中的参数 `--storage_client_timeout_ms` 延长超时时间。

最后更新: 2025年12月19日

### 4.11.3 UPDATE VERTEX

`UPDATE VERTEX` 语句可以修改点上 Tag 的属性值。

TRSGraph 支持 CAS（compare and set）操作。



一次只能修改一个 Tag。

#### 语法

```
UPDATE VERTEX ON <tag_name> <vid>
SET <update_prop>
[WHEN <condition>]
[YIELD <output>]
```

| 参数                | 是否必须 | 说明                                          | 示例                 |
|-------------------|------|---------------------------------------------|--------------------|
| ON <tag_name>     | 是    | 指定点的 Tag。要修改的属性必须在这个 Tag 内。                 | ON player          |
| <vid>             | 是    | 指定要修改的点 ID。                                 | "player100"        |
| SET <update_prop> | 是    | 指定如何修改属性值。                                  | SET age = age +1   |
| WHEN <condition>  | 否    | 指定过滤条件。如果 <condition> 结果为 false，SET 子句不会生效。 | WHEN name == "Tim" |
| YIELD <output>    | 否    | 指定语句的输出格式。                                  | YIELD name AS Name |

#### 示例

```
// 查看点"player101"的属性。
trsgraph> FETCH PROP ON player "player101" YIELD properties(vertex);
+-----+
| properties (VERTEX) |
+-----+
| {age: 36, name: "Tony Parker"} |
+-----+

// 修改属性 age 的值，并返回 name 和新的 age。
trsgraph> UPDATE VERTEX ON player "player101" \
 SET age = age + 2 \
 WHEN name == "Tony Parker" \
 YIELD name AS Name, age AS Age;
+-----+-----+
| Name | Age |
+-----+-----+
| "Tony Parker" | 38 |
+-----+-----+
```

最后更新: 2025年12月19日

4.11.4 UPSERT VERTEX

UPSERT VERTEX 语句结合 UPDATE 和 INSERT，如果点存在，会修改点的属性值；如果点不存在，会插入新的点。



UPSERT VERTEX 一次只能修改一个 Tag。

UPSERT VERTEX 性能远低于 INSERT，因为 UPSERT 是一组分片级别的读取、修改、写入操作。



并发 UPSERT 同一个 TAG 或 EDGE TYPE 会报错。

语法

```
UPSERT VERTEX ON <tag> <vid>
SET <update_prop>
[WHEN <condition>]
[YIELD <output>]
```

| 参数                | 是否必须 | 说明                          | 示例                 |
|-------------------|------|-----------------------------|--------------------|
| ON <tag>          | 是    | 指定点的 Tag。要修改的属性必须在这个 Tag 内。 | ON player          |
| <vid>             | 是    | 指定要修改或插入的点 ID。              | "player100"        |
| SET <update_prop> | 是    | 指定如何修改属性值。                  | SET age = age +1   |
| WHEN <condition>  | 否    | 指定过滤条件。                     | WHEN name == "Tim" |
| YIELD <output>    | 否    | 指定语句的输出格式。                  | YIELD name AS Name |

插入不存在的点

如果点不存在，无论 WHEN 子句的条件是否满足，都会插入点，同时执行 SET 子句，因此新插入的点的属性值取决于：

- SET 子句。
- 属性是否有默认值。

例如：

- 要插入的点包含基于 Tag player 的属性 name 和 age。
- SET 子句指定 age=30。

不同情况下的属性值如下表。

| 是否满足 WHEN 子句条件 | 属性是否有默认值 | name 属性值 | age 属性值 |
|----------------|----------|----------|---------|
| 是              | 是        | 默认值      | 30      |
| 是              | 否        | NULL     | 30      |
| 否              | 是        | 默认值      | 30      |
| 否              | 否        | NULL     | 30      |

示例如下：

```
// 查看三个点是否存在，结果 “Empty set” 表示顶点不存在。
trsgraph> FETCH PROP ON * "player666", "player667", "player668" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
Empty set

trsgraph> UPSERT VERTEX ON player "player666" \
 SET age = 30 \
 WHEN name == "Joe" \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| __NULL__ | 30 |
+-----+

trsgraph> UPSERT VERTEX ON player "player666" \
 SET age = 31 \
 WHEN name == "Joe" \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| __NULL__ | 30 |
+-----+

trsgraph> UPSERT VERTEX ON player "player667" \
 SET age = 31 \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| __NULL__ | 31 |
+-----+

trsgraph> UPSERT VERTEX ON player "player668" \
 SET name = "Amber", age = age + 1 \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| "Amber" | __NULL__ |
+-----+
```

上面最后一个示例中，因为 `age` 没有默认值，插入点时，`age` 默认值为 `NULL`，执行 `age = age + 1` 后仍为 `NULL`。如果 `age` 有默认值，则 `age = age + 1` 可以正常执行，例如：

```
trsgraph> CREATE TAG IF NOT EXISTS player_with_default(name string, age int DEFAULT 20);
Execution succeeded

trsgraph> UPSERT VERTEX ON player_with_default "player101" \
 SET age = age + 1 \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| __NULL__ | 21 |
+-----+
```

### 修改存在的点

如果点存在，且满足 `WHEN` 子句的条件，就会修改点的属性值。

```
trsgraph> FETCH PROP ON player "player101" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 36, name: "Tony Parker"} |
+-----+

trsgraph> UPSERT VERTEX ON player "player101" \
 SET age = age + 2 \
 WHEN name == "Tony Parker" \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| "Tony Parker" | 38 |
+-----+
```



如果点存在，但是不满足 `WHEN` 子句的条件，修改不会生效。

```
trsgraph> FETCH PROP ON player "player101" YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {age: 38, name: "Tony Parker"} |
+-----+

trsgraph> UPSERT VERTEX ON player "player101" \
 SET age = age + 2 \
 WHEN name == "Someone else" \
 YIELD name AS Name, age AS Age;
+-----+
| Name | Age |
+-----+
| "Tony Parker" | 38 |
+-----+
```

---

最后更新: 2025年12月19日

## 4.12 边语句

### 4.12.1 INSERT EDGE

`INSERT EDGE` 语句可以在 TRSGraph 实例的指定图空间中插入一条或多条边。边是有方向的，从起始点（`src_vid`）到目的点（`dst_vid`）。

`INSERT EDGE` 的执行方式为覆盖式插入。如果已有 Edge type、起点、终点、rank 都相同的边，则覆盖原边。

#### 语法

```
INSERT EDGE [IF NOT EXISTS] <edge_type> (<prop_name_list>) VALUES
<src_vid> -> <dst_vid>[@<rank>] : (<prop_value_list>)
[, <src_vid> -> <dst_vid>[@<rank>] : (<prop_value_list>), ...];

<prop_name_list> ::=
[<prop_name> [, <prop_name>] ...]

<prop_value_list> ::=
[<prop_value> [, <prop_value>] ...]
```

- `IF NOT EXISTS`：用户可以使用 `IF NOT EXISTS` 关键字检测待插入的边是否存在，只有不存在时，才会插入。



Note

- `IF NOT EXISTS` 仅检测<边的类型、起始点、目的点和 rank>是否存在，不会检测属性值是否重合。
- `IF NOT EXISTS` 会先读取一次数据是否存在，因此对性能会有明显影响。

- `<edge_type>`：边关联的 Edge type，只能指定一个 Edge type。Edge type 必须提前创建，详情请参见 [CREATE EDGE](#)。
- `<prop_name_list>`：需要设置的属性名称列表。
- `src_vid`：起始点 ID，表示边的起点。
- `dst_vid`：目的点 ID，表示边的终点。
- `rank`：可选项。边的 rank 值。数据类型为 `int`。默认值为 `0`。



OpenCypher 兼容性

openCypher 中没有 rank 的概念。

- `<prop_value_list>`：根据 `prop_name_list` 填写属性值。如果属性值和 Edge type 中的数据类型不匹配，会返回错误。如果没有填写属性值，而 Edge type 中对应的属性设置为 `NOT NULL`，也会返回错误。详情请参见 [CREATE EDGE](#)。

#### 示例

```
插入不包含属性的边。
trsgraph> CREATE EDGE IF NOT EXISTS e1();
trsgraph> INSERT EDGE e1 () VALUES "10"->"11":();

插入 rank 为 1 的边。
trsgraph> INSERT EDGE e1 () VALUES "10"->"11"@1:();
```

```
trsgraph> CREATE EDGE IF NOT EXISTS e2 (name string, age int);
trsgraph> INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", 1);

一次插入 2 条边。
trsgraph> INSERT EDGE e2 (name, age) VALUES \
 "12"->"13":("n1", 1), "13"->"14":("n2", 2);
```

```
创建失败，因为"a13"不是 int 类型。
trsgraph> INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", "a13");
```

一条边可以多次插入属性值，以最后一次为准。

```
多次插入属性值。
trsgraph> INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", 12);
trsgraph> INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", 13);
trsgraph> INSERT EDGE e2 (name, age) VALUES "11"->"13":("n1", 14);
trsgraph> FETCH PROP ON e2 "11"->"13" YIELD edge AS e;
+-----+
| e |
+-----+
| [:e2 "11"->"13" @0 {age: 14, name: "n1"}] |
+-----+
```

使用 `IF NOT EXISTS` 插入已存在的边时，不会进行修改。

```
插入边。
trsgraph> INSERT EDGE e2 (name, age) VALUES "14"->"15"@1:("n1", 12);
使用 IF NOT EXISTS 修改边，因为边已存在，不会进行修改。
trsgraph> INSERT EDGE IF NOT EXISTS e2 (name, age) VALUES "14"->"15"@1:("n2", 13);
trsgraph> FETCH PROP ON e2 "14"->"15"@1 YIELD edge AS e;
+-----+
| e |
+-----+
| [:e2 "14"->"15" @1 {age: 12, name: "n1"}] |
+-----+
```



- TRSGraph 10.0 允许存在悬挂边（Dangling edge）。因此可以在起点或者终点存在前，先写边；此时就可以通过 `<edgetype>._src` 或 `<edgetype>._dst` 获取到（尚未写入的）点 VID（不建议这样使用）。
- 目前还不能保证操作的原子性，如果失败请重试，否则会发生部分写入。此时读取该数据的行为是未定义的。例如写入操作涉及到多个机器时，可能会出现插入单个边的正反向边只写入成功一个，或者插入多个边时只写入成功一部分，此时会返回报错，请重新执行命令。
- 并发写入同一条边会报 `edge conflict` 错误，可稍后重试。
- 边的 `INSERT` 速度 大约是点的 `INSERT` 速度一半。原因是 `INSERT` 边会对应 `stored` 的两个 `INSERT`，`INSERT` 点 对应 `stored` 的一个 `INSERT`。

最后更新: 2025年12月19日

## 4.12.2 DELETE EDGE

`DELETE EDGE` 语句可以删除边。一次可以删除一条或多条边。用户可以结合管道符一起使用，详情请参见 [管道符](#)。

如果需要删除一个点的所有出边，请删除这个点。详情请参见 [DELETE VERTEX](#)。

### 语法

```
DELETE EDGE <edge_type> <src_vid> -> <dst_vid>[@<rank>] [, <src_vid> -> <dst_vid>[@<rank>] ...]
```



#### Caution

如果不指定 rank，则仅仅删除 rank 为 0 的边。需要删除所有的 rank，见如下示例。

### 示例

```
trsgraph> DELETE EDGE serve "player100" -> "team204"@0;
```

```
结合管道符，删除两点之间同类型的所有rank的边。
trsgraph> GO FROM "player100" OVER follow \
 WHERE dst(edge) == "player101" \
 YIELD src(edge) AS src, dst(edge) AS dst, rank(edge) AS rank \
 | DELETE EDGE follow $-.src -> $-.dst @ $-.rank;
```

最后更新: 2025年12月19日

4.12.3 UPDATE EDGE

UPDATE EDGE 语句可以修改边上 Edge type 的属性。

TRSGraph 支持 CAS（compare and swap）操作。

语法

```
UPDATE EDGE ON <edge_type>
<src_vid> -> <dst_vid> [@<rank>]
SET <update_prop>
[WHEN <condition>]
[YIELD <output>]
```

| 参数                   | 是否必须 | 说明                                          | 示例                                 |
|----------------------|------|---------------------------------------------|------------------------------------|
| ON <edge_type>       | 是    | 指定 Edge type。要修改的属性必须在这个 Edge type 内。       | ON serve                           |
| <src_vid>            | 是    | 指定边的起始点 ID。                                 | "player100"                        |
| <dst_vid>            | 是    | 指定边的目的点 ID。                                 | "team204"                          |
| <rank>               | 否    | 指定边的 rank 值。                                | 10                                 |
| SET<br><update_prop> | 是    | 指定如何修改属性值。                                  | SET start_year =<br>start_year + 1 |
| WHEN <condition>     | 否    | 指定过滤条件。如果 <condition> 结果为 false，SET 子句不会生效。 | WHEN end_year < 2010               |
| YIELD <output>       | 否    | 指定语句的输出格式。                                  | YIELD start_year AS<br>Start_Year  |

示例

```
// 用 GO 语句查看边的属性值。
trsgraph> GO FROM "player100" \
 OVER serve \
 YIELD properties(edge).start_year, properties(edge).end_year;
+-----+-----+
| properties(EDGE).start_year | properties(EDGE).end_year |
+-----+-----+
| 1997 | 2016 |
+-----+-----+

// 修改属性 start_year 的值，并返回 end_year 和新的 start_year。
trsgraph> UPDATE EDGE ON serve "player100" -> "team204"@0 \
 SET start_year = start_year + 1 \
 WHEN end_year > 2010 \
 YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| 1998 | 2016 |
+-----+-----+
```

最后更新: 2025年12月19日

4.12.4 UPSERT EDGE

`UPSERT EDGE` 语句结合 `UPDATE` 和 `INSERT`，如果边存在，会更新边的属性；如果边不存在，会插入新的边。

`UPSERT EDGE` 性能远低于 `INSERT`，因为 `UPSERT` 是一组分片级别的读取、修改、写入操作。



Caution

并发 `UPSERT` 同一个 TAG 或 EDGE TYPE 会报错。

语法

```
UPSERT EDGE ON <edge_type>
<src_vid> -> <dst_vid> [@rank]
SET <update_prop>
[WHEN <condition>]
[YIELD <properties>]
```

| 参数                                   | 是否必须 | 说明                                    | 示例                                           |
|--------------------------------------|------|---------------------------------------|----------------------------------------------|
| <code>ON &lt;edge_type&gt;</code>    | 是    | 指定 Edge type。要修改的属性必须在这个 Edge type 内。 | <code>ON serve</code>                        |
| <code>&lt;src_vid&gt;</code>         | 是    | 指定边的起始点 ID。                           | <code>"player100"</code>                     |
| <code>&lt;dst_vid&gt;</code>         | 是    | 指定边的目的点 ID。                           | <code>"team204"</code>                       |
| <code>&lt;rank&gt;</code>            | 否    | 指定边的 rank 值。数据类型为 <code>int</code> 。  | <code>10</code>                              |
| <code>SET &lt;update_prop&gt;</code> | 是    | 指定如何修改属性值。                            | <code>SET start_year = start_year + 1</code> |
| <code>WHEN &lt;condition&gt;</code>  | 否    | 指定过滤条件。                               | <code>WHEN end_year &lt; 2010</code>         |
| <code>YIELD &lt;output&gt;</code>    | 否    | 指定语句的输出格式。                            | <code>YIELD start_year AS Start_Year</code>  |

插入不存在的边

如果边不存在，无论 `WHEN` 子句的条件是否满足，都会插入边，同时执行 `SET` 子句，因此新插入的边的属性值取决于：

- `SET` 子句。
- 属性是否有默认值。

例如：

- 要插入的边包含基于 Edge type `serve` 的属性 `start_year` 和 `end_year`。
- `SET` 子句指定 `end_year = 2021`。

不同情况下的属性值如下表。

| 是否满足 <code>WHEN</code> 子句条件 | 属性是否有默认值 | <code>start_year</code> 属性值 | <code>end_year</code> 属性值 |
|-----------------------------|----------|-----------------------------|---------------------------|
| 是                           | 是        | 默认值                         | <code>2021</code>         |
| 是                           | 否        | <code>NULL</code>           | <code>2021</code>         |
| 否                           | 是        | 默认值                         | <code>2021</code>         |
| 否                           | 否        | <code>NULL</code>           | <code>2021</code>         |

示例如下：

```
// 查看如下三个点是否有 serve 类型的出边，结果 “Empty set” 表示没有 serve 类型的出边。
trsgraph> GO FROM "player666", "player667", "player668" \
OVER serve \
YIELD properties(edge).start_year, properties(edge).end_year;
+-----+-----+
| properties(EDGE).start_year | properties(EDGE).end_year |
+-----+-----+
Empty set

trsgraph> UPSERT EDGE on serve \
"player666" -> "team200"@0 \
SET end_year = 2021 \
WHEN end_year == 2010 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| __NULL__ | 2021 |
+-----+-----+

trsgraph> UPSERT EDGE on serve \
"player666" -> "team200"@0 \
SET end_year = 2022 \
WHEN end_year == 2010 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| __NULL__ | 2021 |
+-----+-----+

trsgraph> UPSERT EDGE on serve \
"player667" -> "team200"@0 \
SET end_year = 2022 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| __NULL__ | 2022 |
+-----+-----+

trsgraph> UPSERT EDGE on serve \
"player668" -> "team200"@0 \
SET start_year = 2000, end_year = end_year + 1 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| 2000 | __NULL__ |
+-----+-----+
```

上面最后一个示例中，因为 `end_year` 没有默认值，插入边时，`end_year` 默认值为 `NULL`，执行 `end_year = end_year + 1` 后仍为 `NULL`。如果 `end_year` 有默认值，则 `end_year = end_year + 1` 可以正常执行，例如：

```
trsgraph> CREATE EDGE IF NOT EXISTS serve_with_default(start_year int, end_year int DEFAULT 2010);
Execution succeeded

trsgraph> UPSERT EDGE on serve_with_default \
"player668" -> "team200" \
SET end_year = end_year + 1 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| __NULL__ | 2011 |
+-----+-----+
```

## 修改存在的边

如果边存在，且满足 `WHEN` 子句的条件，就会修改边的属性值。

```
trsgraph> MATCH (v:player{name:"Ben Simmons"})-[:serve]-(v2) \
RETURN e;
+-----+-----+
| e |
+-----+-----+
| [:serve "player149"->"team219" @0 {end_year: 2019, start_year: 2016}] |
+-----+-----+

trsgraph> UPSERT EDGE on serve \
"player149" -> "team219" \
SET end_year = end_year + 1 \
WHEN start_year == 2016 \
```

```
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| 2016 | 2020 |
+-----+-----+
```

如果边存在，但是不满足 `WHEN` 子句的条件，修改不会生效。

```
trsgraph> MATCH (v:player{name:"Ben Simmons"})-[e:serve]-(v2) \
RETURN e;
+-----+-----+
| e |
+-----+-----+
| [:serve "player149"->"team219" @0 {end_year: 2020, start_year: 2016}] |
+-----+-----+

trsgraph> UPSERT EDGE on serve \
"player149" -> "team219" \
SET end_year = end_year + 1 \
WHEN start_year != 2016 \
YIELD start_year, end_year;
+-----+-----+
| start_year | end_year |
+-----+-----+
| 2016 | 2020 |
+-----+-----+
```

最后更新: 2025年12月19日



## 4.13 原生索引

### 4.13.1 索引介绍

为了提高查询性能，TRSGraph 支持为点的 Tag 或 Tag 的某个属性，边的 Edge type 或 Edge type 的某个属性创建索引。索引可以基于指定的 Tag、Edge type、属性查询数据，但是索引本身不存储数据，而是存储数据的位置。

TRSGraph 支持两种类型索引：原生索引和全文索引。

#### 使用说明

- 索引可以提高查询性能，但是会降低写入性能。
- 索引是执行 `LOOKUP` 语句时用于定位到数据的前置条件，如果没有索引，执行 `LOOKUP` 语句会报错。
- 使用索引时，TRSGraph 会自动选择最优的索引。
- 具有高选择度的索引，即索引列中不同值的记录数与总记录数的比值较高（例如身份证号的比值为 1）可显著提升查询性能；而对于低选择度的索引（例如国家），查询性能可能不会带来显著提升。

#### 原生索引

原生索引可以基于指定的属性查询数据，有如下特点：

- 包括 Tag 索引和 Edge type 索引。
- 必须手动重建索引（`REBUILD INDEX`）。
- 支持创建同一个 Tag 或 Edge type 的多个属性的索引（复合索引），但是不能跨 Tag 或 Edge type。

#### 原生索引操作

- `CREATE INDEX`
- `SHOW CREATE INDEX`
- `SHOW INDEXES`
- `DESCRIBE INDEX`
- `REBUILD INDEX`
- `SHOW INDEX STATUS`
- `DROP INDEX`
- `LOOKUP`
- `MATCH`
- 地理空间索引

#### 全文索引

全文索引是基于 Elasticsearch 来实现的，用于对字符串属性进行前缀搜索、通配符搜索、正则表达式搜索和模糊搜索，有如下特点：

- 只允许创建一个属性的索引。
- 不支持逻辑操作，例如 `AND`、`OR`、`NOT`。



Note

如果需要整个字符串的匹配，请使用原生索引。

### 没有 NULL 值索引

不支持对值为 NULL 的属性创建索引。

### 没有唯一索引

在 Cypher 中，可以通过 Constrains 实现属性值的唯一性限制。在 MySQL 中，可以建立唯一索引来限制某字段只有唯一值。在 nGQL 中没有属性的唯一索引（用户自行保证属性值的唯一性）。

### 数字、日期和时间类型的范围查询

原生索引还支持对数字、日期和时间类型的属性进行范围查询，不支持其他属性类型的范围查询。

---

最后更新: 2025年12月19日

### 4.13.2 CREATE INDEX

#### 前提条件

创建索引之前，请确保相关的 Tag 或 Edge type 已经创建。如何创建 Tag 和 Edge type，请参见 [CREATE TAG](#) 和 [CREATE EDGE](#)。

如何创建全文索引，请参见[部署全文索引](#)。

#### 使用索引必读

索引的概念和使用限制都较为复杂。在使用索引前，请务必阅读以下内容。

`CREATE INDEX` 语句用于对 Tag、EdgeType 或其属性创建原生索引。通常分别称为“Tag 索引”、“Edge type 索引”和“属性索引”。

- Tag 索引和 Edge type 索引应用于和 Tag、Edge type 自身相关的查询，例如用 `LOOKUP` 查找有 Tag `player` 的所有点。
- “属性索引”应用于基于属性的查询，例如基于属性 `age` 找到 `age == 19` 的所有的点。

如果已经为 Tag `T` 的属性 `A` 建立过属性索引 `i_TA`（`T` 的索引为 `i_T`），索引之间的可替代关系如下（Edge type 索引同理）：

- 查询引擎可以使用 `i_TA` 来替代 `i_T`。
- 在 `MATCH`、`LOOKUP` 语句中 `i_T` 可以替代 `i_TA` 查找属性。



#### 版本兼容性

在此前的版本中，`LOOKUP` 语句中的 Tag 或 Edge type 索引不可替代属性索引用于属性查找。

使用替代索引进行查询虽然能获得相同的结果，但查询性能会根据选择的索引有所区别。



#### Caution

不要任意在生产环境中使用索引，除非很清楚使用索引对业务的影响。索引会导致写性能大幅下降。

变长字符串的属性索引长度最长为 256；定长字符串的属性索引长度为索引名本身的长度，无限制。长索引会降低 Storage 服务的扫描性能，以及占用更多内存，建议将索引长度设置为和被索引的最长字符串相同。

#### 操作步骤

如果必须使用索引，通常按照如下步骤：

1. 初次导入数据至 TRSGraph。
2. 创建索引。
3. [重建索引](#)。
4. 使用 `LOOKUP` 或 `MATCH` 语句查询数据。不需要（也无法）指定使用哪个索引，TRSGraph 会自动计算。



如果先创建索引再导入数据，会因为写性能的下降导致导入速度极慢。

日常增量写入时保持 `--disable_auto_compaction = false`。

新创建的索引并不会立刻生效。创建新的索引并尝试立刻使用（例如 `LOOKUP` 或者 `REBUILD INDEX`）通常会失败（报错 `can't find xxx in the space`）。因为创建步骤是异步实现的，TRSGraph 要在下一个心跳周期才能完成索引的创建。为确保数据同步，后续操作能顺利进行，请等待 2 个心跳周期（20 秒）。如果需要修改心跳间隔，请为所有配置文件修改参数 `heartbeat_interval_secs`。



创建索引，或者删除并再次创建同名索引后，必须 `REBUILD INDEX`。否则无法在 `MATCH` 和 `LOOKUP` 语句中返回这些数据。

语法

```
CREATE {TAG | EDGE} INDEX [IF NOT EXISTS] <index_name> ON {<tag_name> | <edge_name>} ([<prop_name_list>]) [COMMENT '<comment>'];
```

| 参数                                                | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TAG   EDGE</code>                           | 指定要创建的索引类型。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>IF NOT EXISTS</code>                        | 检测待创建的索引是否存在，只有不存在时，才会创建索引。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>&lt;index_name&gt;</code>                   | <div>1、索引名。索引名在一个图空间中必须是唯一的。推荐的命名方式为 <code>i_tagName_propName</code>。</div> <div>2、默认情况下，仅支持 1~4 字节的 UTF-8 编码字符，包括英文字母（区分大小写）、数字、中文等。不能包含下划线（<code>_</code>）以外的特殊字符，且不能以数字开头。</div> <div>3、如果需要使用特殊字符、保留关键字或者以数字开头，请使用反引号（<code>`</code>）包围整个名称。反引号中不能包含英文句号（<code>.</code>）。详情参见<a href="#">关键字</a>和<a href="#">保留字</a>。</div> <div>注意：</div> <div>1、如果以中文为索引命名，报 <code>SyntaxError</code> 错误时，需使用反引号（<code>`</code>）包围中文字符。</div> <div>2、名称中如需包含反引号（<code>`</code>），使用反斜杠（<code>\</code>）来转义反引号（<code>`</code>），如：<code>\`</code>；如需包含反斜杠（<code>\</code>），反斜杠（<code>\</code>）本身也需转义，如：<code>\\</code>。</div> |
| <code>&lt;tag_name&gt;   &lt;edge_name&gt;</code> | 指定索引关联的 Tag 或 Edge 名称。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>&lt;prop_name_list&gt;</code>               | 为变长字符串属性创建索引时，必须用 <code>prop_name(length)</code> 指定索引长度，索引长度最长为 256；为 Tag 或 Edge type 本身创建索引时，忽略 <code>&lt;prop_name_list&gt;</code> 。                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>COMMENT</code>                              | 索引的描述。最大为 256 字节。默认无描述。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

创建 Tag/Edge type 索引

```
trsgraph> CREATE TAG INDEX player_index on player();
```

```
trsgraph> CREATE EDGE INDEX follow_index on follow();
```

为 Tag 或 Edge type 创建索引后，用户可以使用 `LOOKUP` 语句查找 带有该 Tag 的所有点的 VID，或者 所有该类型的边 的 对应起始点 VID、目的点 VID、以及 rank。详情请参见 [LOOKUP](#)。

创建单属性索引

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS player_index_0 on player(name(10));
```

上述示例是为所有包含 Tag `player` 的点创建属性 `name` 的索引，索引长度为 10。即只使用属性 `name` 的前 10 个字符来创建索引。

```
变长字符串需要指定索引长度。
trsgraph> CREATE TAG IF NOT EXISTS var_string(pl string);
```

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS var ON var_string(p1(10));

定长字符串不需要指定索引长度。
trsgraph> CREATE TAG IF NOT EXISTS fix_string(p1 FIXED_STRING(10));
trsgraph> CREATE TAG INDEX IF NOT EXISTS fix ON fix_string(p1);

trsgraph> CREATE EDGE INDEX IF NOT EXISTS follow_index_0 on follow(degree);
```

创建复合属性索引

复合属性索引 用于查找一个 Tag（或者 Edge type）中的多个属性（的组合）。

```
trsgraph> CREATE TAG INDEX IF NOT EXISTS player_index_1 on player(name(10), age);
```



不支持跨 Tag 或 Edge type 创建复合索引。



使用复合属性索引时，遵循"最左匹配原则"，必须从复合属性索引的最左侧开始匹配。

最后更新: 2025年12月19日

### 4.13.3 SHOW INDEXES

`SHOW INDEXES` 语句可以列出当前图空间内的所有 Tag 和 Edge type（包括属性）的索引。

#### 语法

```
SHOW {TAG | EDGE} INDEXES;
```

#### 示例

```
trsgraph> SHOW TAG INDEXES;
+-----+-----+-----+
| Index Name | By Tag | Columns |
+-----+-----+-----+
"fix"	"fix_string"	["pl"]
"player_index_0"	"player"	["name"]
"player_index_1"	"player"	["name", "age"]
"var"	"var_string"	["pl"]
+-----+-----+-----+

trsgraph> SHOW EDGE INDEXES;
+-----+-----+-----+
| Index Name | By Edge | Columns |
+-----+-----+-----+
| "follow_index" | "follow" | [] |
+-----+-----+-----+
```

最后更新: 2025年12月19日

## 4.13.4 SHOW CREATE INDEX

`SHOW CREATE INDEX` 展示创建 Tag 或者 Edge type 时使用的 nGQL 语句，其中包含索引的详细信息，例如其关联的属性。

### 语法

```
SHOW CREATE {TAG | EDGE} INDEX <index_name>;
```

### 示例

用户可以先运行 `SHOW TAG INDEXES` 查看有哪些 Tag 索引，然后用 `SHOW CREATE TAG INDEX` 查看指定索引的创建信息。

```
trsgraph> SHOW TAG INDEXES;
+-----+-----+-----+
| Index Name | By Tag | Columns |
+-----+-----+-----+
| "player_index_0" | "player" | [] |
| "player_index_1" | "player" | ["name"] |
+-----+-----+-----+

trsgraph> SHOW CREATE TAG INDEX player_index_1;
+-----+-----+-----+
| Tag Index Name | Create Tag Index |
+-----+-----+-----+
"player_index_1"	"CREATE TAG INDEX `player_index_1` ON `player` (
	`name` (20)
	)"
+-----+-----+-----+
```

Edge type 索引可以用类似的方法查询：

```
trsgraph> SHOW EDGE INDEXES;
+-----+-----+-----+
| Index Name | By Edge | Columns |
+-----+-----+-----+
| "follow_index" | "follow" | [] |
+-----+-----+-----+

trsgraph> SHOW CREATE EDGE INDEX follow_index;
+-----+-----+-----+
| Edge Index Name | Create Edge Index |
+-----+-----+-----+
| "follow_index" | "CREATE EDGE INDEX `follow_index` ON `follow` (|
| |)" |
+-----+-----+-----+
```

最后更新: 2025年12月19日

### 4.13.5 DESCRIBE INDEX

`DESCRIBE INDEX` 语句可以查看指定索引的信息，包括索引的属性名称（Field）和数据类型（Type）。

#### 语法

```
DESCRIBE {TAG | EDGE} INDEX <index_name>;
```

#### 示例

```
trsgraph> DESCRIBE TAG INDEX player_index_0;
+-----+-----+
| Field | Type |
+-----+-----+
| "name" | "fixed_string(30)" |
+-----+-----+

trsgraph> DESCRIBE TAG INDEX player_index_1;
+-----+-----+
| Field | Type |
+-----+-----+
| "name" | "fixed_string(10)" |
| "age" | "int64" |
+-----+-----+
```

最后更新: 2025年12月19日



### 4.13.6 REBUILD INDEX



**Danger**

- 索引功能不会自动对其创建之前已存在的存量数据生效——在索引重建完成之前，无法基于该索引使用 `LOOKUP` 和 `MATCH` 语句查询到存量数据。
- 索引的重建未完成时，依赖索引的查询仅能使用部分索引，因此不能获得准确结果。

请在创建索引后，选择合适的时间为存量数据重建索引。使用索引的详情请参见 [CREATE INDEX](#)。



**Caution**

通过修改配置文件中的 `rebuild_index_part_rate_limit` 和 `rebuild_index_batch_size` 两个参数，可优化重建索引的速度，另外，更大参数可能会导致更高的内存和网络占用，详情请参见 [Storage服务配置](#)。

#### 语法

```
REBUILD {TAG | EDGE} INDEX [<index_name_list>];

<index_name_list>::=
[index_name [, index_name] ...]
```

- 可以一次重建多个索引，索引名称之间用英文逗号（,）分隔。如果没有指定索引名称，将会重建所有索引。
- 重建完成后，用户可以使用命令 `SHOW {TAG | EDGE} INDEX STATUS` 检查索引是否重建完成。详情请参见 [SHOW INDEX STATUS](#)。

#### 示例

```
trsgraph> CREATE TAG IF NOT EXISTS person(name string, age int, gender string, email string);
trsgraph> CREATE TAG INDEX IF NOT EXISTS single_person_index ON person(name(10));

重建索引，返回任务 ID。
trsgraph> REBUILD TAG INDEX single_person_index;
+-----+
| New Job Id |
+-----+
| 31 |
+-----+

查看索引状态。
trsgraph> SHOW TAG INDEX STATUS;
+-----+-----+
| Name | Index Status |
+-----+-----+
| "single_person_index" | "FINISHED" |
+-----+-----+

也可以使用 SHOW JOB <job_id>查看重建索引的任务状态。
trsgraph> SHOW JOB 31;
+-----+-----+-----+-----+-----+-----+
| Job Id(TaskId) | Command(Dest) | Status | Start Time | Stop Time | Error Code |
+-----+-----+-----+-----+-----+-----+
31	"REBUILD_TAG_INDEX"	"FINISHED"	2021-07-07T09:04:24.000	2021-07-07T09:04:24.000	"SUCCEEDED"
0	"storaged1"	"FINISHED"	2021-07-07T09:04:24.000	2021-07-07T09:04:28.000	"SUCCEEDED"
1	"storaged2"	"FINISHED"	2021-07-07T09:04:24.000	2021-07-07T09:04:28.000	"SUCCEEDED"
2	"storaged0"	"FINISHED"	2021-07-07T09:04:24.000	2021-07-07T09:04:28.000	"SUCCEEDED"
"Total:3"	"Succeeded:3"	"Failed:0"	"In Progress:0"	""	""
+-----+-----+-----+-----+-----+-----+
```

TRSGraph 创建一个任务去重建索引，因此可以根据返回的任务 ID，通过 `SHOW JOB <job_id>` 语句查看任务状态。详情请参见 [SHOW JOB](#)。

最后更新: 2025年12月19日

### 4.13.7 SHOW INDEX STATUS

`SHOW INDEX STATUS` 语句可以查看索引名称和对应作业的状态。

重建索引时的作业状态包括：

- `QUEUE`：队列中
- `RUNNING`：执行中
- `FINISHED`：已完成
- `FAILED`：失败
- `STOPPED`：停止
- `INVALID`：失效



如何创建索引请参见 [CREATE INDEX](#)。

#### 语法

```
SHOW {TAG | EDGE} INDEX STATUS;
```

#### 示例

```
trsgraph> SHOW TAG INDEX STATUS;
+-----+-----+
| Name | Index Status |
+-----+-----+
| "player_index_0" | "FINISHED" |
| "player_index_1" | "FINISHED" |
+-----+-----+
```

最后更新: 2025年12月19日

## 4.13.8 DROP INDEX

---

`DROP INDEX` 语句可以删除当前图空间中已存在的索引。

### 前提条件

执行 `DROP INDEX` 语句需要当前登录的用户拥有指定图空间的 `DROP TAG INDEX` 和 `DROP EDGE INDEX` 权限，否则会报错。

### 语法

```
DROP {TAG | EDGE} INDEX [IF EXISTS] <index_name>;
```

`IF EXISTS`：检测待删除的索引是否存在，只有存在时，才会删除索引。

### 示例

```
trsgraph> DROP TAG INDEX player_index_0;
```

---

最后更新: 2025年12月19日

## 4.14 全文索引

### 4.14.1 全文索引限制

本文介绍全文索引的限制，请在使用全文索引前仔细阅读。



3.6.0 版本重做了全文索引功能，不兼容之前版本的全文索引。如果你想继续使用通配符、正则、模糊匹配等方式，有如下 3 种方式：

- 删除原有全文索引，使用新的方式重建全文索引，使用新的 [查询语法](#)。
- 删除原有全文索引，直接用 TRSGraph 的 [原生索引](#) 和 [字符串运算符](#)。
- 继续使用之前版本的 TRSGraph 及其全文索引功能。

全文索引有如下限制：

- 全文索引当前仅支持 `LOOKUP` 语句。
- 全文索引名称只能包含数字、小写字母、下划线。
- 不同图空间内的全文索引名称不能重复。
- 查询默认返回 10 条数据。可以使用 `LIMIT` 子句返回更多数据，最多可以返回 10000 条。可以修改 Elasticsearch 的参数调整最大返回条数。
- 如果 Tag/Edge type 上存在全文索引，无法删除或修改 Tag/Edge type。
- 属性的类型必须为 `STRING` 或 `FIXED_STRING`。
- 全文索引不支持多个 Tag/Edge type 的搜索。
- 全文索引不支持搜索属性值为 `NULL` 的属性。
- 不支持修改 Elasticsearch 中的索引，只能删除重建。
- 不支持修改分词器，需要删除索引数据后重建索引时指定。
- 确保同时启动了 Elasticsearch 集群和 TRSGraph，否则可能导致 Elasticsearch 集群写入的数据不完整。
- 从写入 TRSGraph，到写入 listener，再到写入 Elasticsearch 并创建索引可能需要一段时间。如果访问全文索引时返回未找到索引，可检查索引任务的状态。
- 使用 K8s 方式部署的 TRSGraph 集群不支持自动部署全文索引，但支持手动部署。

最后更新: 2025年12月19日

## 4.14.2 部署全文索引

TRSGraph 的全文索引是基于 [Elasticsearch](#) 实现，这意味着用户可以使用 Elasticsearch 全文查询语言来检索想要的内容。全文索引由内置的进程管理，当 listener 集群和 Elasticsearch 集群部署后，内置的进程只能为数据类型为定长字符串或变长字符串的属性创建全文索引。

### 注意事项

使用全文索引前，请确认已经了解全文索引的[使用限制](#)。

### 部署 Elasticsearch 集群

部署 Elasticsearch 集群请参见 [Kubernetes 安装 Elasticsearch](#) 或 [单机安装 Elasticsearch](#)。目前仅支持 7.x 版本的 Elasticsearch。



为了支持外网访问 Elasticsearch，请将 `config/elasticsearch.yml` 中的 `network.host` 设置为 `0.0.0.0`。

用户可以配置 Elasticsearch 来满足业务需求，如果需要定制 Elasticsearch，请参见 [Elasticsearch 官方文档](#)。

### 登录文本搜索客户端

部署 Elasticsearch 集群之后，可以使用 `SIGN IN` 语句登录 Elasticsearch 客户端。必须使用 Elasticsearch 配置文件中的 IP 地址和端口才能正常连接，同时登录多个客户端，请在多个 `elastic_ip:port` 之间用英文逗号 (,) 分隔。

#### 语法

```
SIGN IN TEXT SERVICE (<elastic_ip:port>, {HTTP | HTTPS} [, "<username>", "<password>"]) [, (<elastic_ip:port>, ...)];
```

#### 示例

```
trsgraph> SIGN IN TEXT SERVICE (192.168.8.100:9200, HTTP);
```



Elasticsearch 默认没有用户名和密码，如果设置了用户名和密码，请在 `SIGN IN` 语句中指定。



Elasticsearch 客户端只能登录一次，如有修改，需要 `SIGN OUT` 后重新 `SIGN IN`，且客户端对全局生效，多个图空间共享相同的 Elasticsearch 客户端。

### 显示文本搜索客户端

`SHOW TEXT SEARCH CLIENTS` 语句可以列出文本搜索客户端。

#### 语法

```
SHOW TEXT SEARCH CLIENTS;
```

#### 示例

```
trsgraph> SHOW TEXT SEARCH CLIENTS;
+-----+
| Type | Host | Port |
+-----+
| "ELASTICSEARCH" | "192.168.8.100" | 9200 |
+-----+
```

## 退出文本搜索客户端

`SIGN OUT TEXT SERVICE` 语句可以退出所有文本搜索客户端。

### 语法

```
SIGN OUT TEXT SERVICE;
```

### 示例

```
trsgraph> SIGN OUT TEXT SERVICE;
```

---

最后更新: 2025年12月19日

### 4.14.3 部署 Raft listener

全文索引的数据是异步写入 Elasticsearch 集群的。流程是通过 Storage 服务的 Raft listener（简称 listener）这个单独部署的进程，从 Storage 服务读取数据，然后将它们写入 Elasticsearch 集群。

#### 注意事项

- 请保证 TRSGraph 各服务（Metad、Stored、Graphd、listener）有相同的版本。
- 请保证 listener 所在机器的时区与 TRSGraph 各服务所在机器的时区一致。
- 只能为一个图空间一次性添加所有的 listener 机器。尝试向已经存在有 listener 的图空间再添加新 listener 会失败。因此，需在一个命令语句里完整地添加全部的 listener。

#### 部署流程

##### 第一步：安装 LISTENER 服务

listener 服务与 stored 服务使用相同的二进制文件，但是二者配置文件不同，进程使用端口也不同。我们在所有需要部署 listener 的服务器上都安装 TRSGraph，但是不启动服务。

##### 第二步：准备 LISTENER 的配置文件

在 `etc` 目录内将 `trsgraph-stored-listener.conf.default` 或 `trsgraph-stored-listener.conf.production` 去掉后缀命名为 `trsgraph-stored-listener.conf`，然后修改配置内容。

大部分配置与 [Storage 服务](#) 的配置文件相同，本文仅介绍差异部分。

| 名称                                   | 预设值                                            | 说明                                                                      |
|--------------------------------------|------------------------------------------------|-------------------------------------------------------------------------|
| <code>daemonize</code>               | <code>true</code>                              | 是否启动守护进程。                                                               |
| <code>pid_file</code>                | <code>pids/trsgraph-stored-listener.pid</code> | 记录进程 ID 的文件。                                                            |
| <code>meta_server_addrs</code>       | -                                              | 全部 Meta 服务的 IP（或主机名）和端口。多个 Meta 服务用英文逗号（,）分隔。                           |
| <code>local_ip</code>                | -                                              | listener 服务的 IP（或主机名）。请使用真实的 listener 机器 IP 替换 <code>127.0.0.1</code> 。 |
| <code>port</code>                    | -                                              | listener 服务的 RPC 守护进程监听端口。                                              |
| <code>heartbeat_interval_secs</code> | <code>10</code>                                | Meta 服务的心跳间隔。单位：秒（s）。                                                   |
| <code>listener_path</code>           | <code>data/listener</code>                     | listener 的 WAL 目录。只允许使用一个目录。                                            |
| <code>data_path</code>               | <code>data</code>                              | 出于兼容性考虑，可以忽略此参数。填充一个默认值 <code>data</code> 。                             |
| <code>part_man_type</code>           | <code>memory</code>                            | 部件管理器类型，可选值为 <code>memory</code> 和 <code>meta</code> 。                  |
| <code>rocksdb_batch_size</code>      | <code>4096</code>                              | 批处理操作的默认保留字节。                                                           |
| <code>rocksdb_block_cache</code>     | <code>4</code>                                 | BlockBasedTable 的默认块缓存大小。单位：兆字节（MB）。                                    |
| <code>engine_type</code>             | <code>rocksdb</code>                           | 存储引擎类型，例如 <code>rocksdb</code> 、 <code>memory</code> 等。                 |
| <code>part_type</code>               | <code>simple</code>                            | 部件类型，例如 <code>simple</code> 、 <code>consensus</code> 等。                 |

##### 第三步：启动 LISTENER

在需要部署 listener 集群的安装目录下，执行如下命令启动 listener：

```
./bin/trsgraph-stored --flagfile etc/trsgraph-stored-listener.conf
```

第四步：添加 LISTENER 到 TRSGRAPH 集群

用命令行连接到 TRSGraph ，然后执行 `USE <space>` 进入需要创建全文索引的图空间。然后执行如下命令添加 listener：

```
ADD LISTENER ELASTICSEARCH <listener_ip:port> [,<listener_ip:port>, ...]
```



listener 必须使用真实的 IP 地址。  
请在一个语句里完整地添加所有 listener。例如：

```
trsgraph> ADD LISTENER ELASTICSEARCH 192.168.8.100:9789,192.168.8.101:9789;
```

查看 listener

执行 `SHOW LISTENER` 语句可以列出所有的 listener。

示例

```
trsgraph> SHOW LISTENER;
+-----+-----+-----+-----+
| PartId | Type | Host | Host Status |
+-----+-----+-----+-----+
1	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
2	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
3	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
+-----+-----+-----+-----+
```

删除 listener

执行 `REMOVE LISTENER ELASTICSEARCH` 语句可以删除图空间的所有 listener。

示例

```
trsgraph> REMOVE LISTENER ELASTICSEARCH;
```

最后更新: 2025年12月19日



#### 4.14.4 全文搜索

全文搜索是基于全文索引对值为字符串类型的属性进行前缀搜索、通配符搜索、正则表达式搜索和模糊搜索。

在 `LOOKUP` 语句中，使用 `WHERE` 子句指定字符串的搜索条件。

##### 前提条件

请确保已经部署全文索引。详情请参见[部署全文索引](#)和[部署 listener](#)。

##### 注意事项

使用全文索引前，请确认已经了解全文索引的[使用限制](#)。

##### 全文本查询

全文本查询使您能够搜索经过分析的文本字段，使用具有严格语法的解析器，根据提供的查询字符串返回内容。详情参见[Query string query](#)。

##### 语法

###### 创建全文索引

```
CREATE FULLTEXT {TAG | EDGE} INDEX <index_name> ON {<tag_name> | <edge_name>} (<prop_name> [,<prop_name>]...) [ANALYZER="<analyzer_name>"];
```

- 创建全文索引时支持多属性的复合索引。
- `<analyzer_name>` 为分词器名称。默认为 `standard`。使用其他分词器（例如 [IK Analysis](#)）需要确保已经提前在 Elasticsearch 中安装对应分词器。

###### 显示全文索引

```
SHOW FULLTEXT INDEXES;
```

###### 重建全文索引

```
REBUILD FULLTEXT INDEX;
```



Caution

数据量大时，重建全文索引速度较慢，可以修改 Storage 服务的配置文件（`trsgaph-storaged.conf`）中 `snapshot_send_files=false`。

###### 删除全文索引

```
DROP FULLTEXT INDEX <index_name>;
```

###### 使用查询选项

```
LOOKUP ON {<tag> | <edge_type>} WHERE ES_QUERY(<index_name>, "<text>") YIELD <return_list> [| LIMIT [<offset>,<number_rows>];

<return_list>
 <prop_name> [AS <prop_alias>] [, <prop_name> [AS <prop_alias>] ...] [, id(vertex) [AS <prop_alias>]] [, score() AS <score_alias>]
```

- `index_name`：索引名称。
- `text`：搜索条件。WHERE 后只能跟一个 ES\_QUERY，所有判断条件必须写在 text 里。详细语法请参见[Query string syntax](#)。
- `score()`：对符合条件的点做 N 度扩展计算出的分数。默认值为 `1.0`。分数越高，匹配程度越高。返回值默认按照分数从高到低排序。详情参见[Search and Scoring in Lucene](#)。

##### 示例

```
//创建图空间。
trsgraph> CREATE SPACE IF NOT EXISTS basketballplayer (partition_num=3,replica_factor=1, vid_type=fixed_string(30));

//登录文本搜索客户端。
trsgraph> SIGN IN TEXT SERVICE (192.168.8.100:9200, HTTP);

//检查是否成功登录。
trsgraph> SHOW TEXT SEARCH CLIENTS;
+-----+
| Type | Host | Port |
+-----+
| "ELASTICSEARCH" | "192.168.8.100" | 9200 |
+-----+

//切换图空间。
trsgraph> USE basketballplayer;

//添加 listener 到 TRSGraph 集群。
trsgraph> ADD LISTENER ELASTICSEARCH 192.168.8.100:9789;

//检查是否成功添加 listener, 当状态为 Online 时表示成功添加。
trsgraph> SHOW LISTENER;
+-----+
| PartId | Type | Host | Host Status |
+-----+
1	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
2	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
3	"ELASTICSEARCH"	"192.168.8.100":9789	"ONLINE"
+-----+

//创建 Tag。
trsgraph> CREATE TAG IF NOT EXISTS player(name string, city string);

//创建单属性全文索引。
trsgraph> CREATE FULLTEXT TAG INDEX fulltext_index_1 ON player(name) ANALYZER="standard";

//创建多属性全文索引。
trsgraph> CREATE FULLTEXT TAG INDEX fulltext_index_2 ON player(name,city) ANALYZER="standard";

//重建全文索引。
trsgraph> REBUILD FULLTEXT INDEX;

//查看全文索引。
trsgraph> SHOW FULLTEXT INDEXES;
+-----+
| Name | Schema Type | Schema Name | Fields | Analyzer |
+-----+
| "fulltext_index_1" | "Tag" | "player" | "name" | "standard" |
| "fulltext_index_2" | "Tag" | "player" | "name, city" | "standard" |
+-----+

//插入测试数据。
trsgraph> INSERT VERTEX player(name, city) VALUES \
 "Russell Westbrook": ("Russell Westbrook", "Los Angeles"), \
 "Chris Paul": ("Chris Paul", "Houston"), \
 "Boris Diaw": ("Boris Diaw", "Houston"), \
 "David West": ("David West", "Philadelphia"), \
 "Danny Green": ("Danny Green", "Philadelphia"), \
 "Tim Duncan": ("Tim Duncan", "New York"), \
 "James Harden": ("James Harden", "New York"), \
 "Tony Parker": ("Tony Parker", "Chicago"), \
 "Aron Baynes": ("Aron Baynes", "Chicago"), \
 "Ben Simmons": ("Ben Simmons", "Phoenix"), \
 "Blake Griffin": ("Blake Griffin", "Phoenix");

//测试查询
trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"Chris") YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "Chris Paul" |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"Harden") YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {_vid: "James Harden", city: "New York", name: "James Harden"} |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"Da*") YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {_vid: "David West", city: "Philadelphia", name: "David West"} |
| {_vid: "Danny Green", city: "Philadelphia", name: "Danny Green"} |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"*b*") YIELD id(vertex);
+-----+
| id(VERTEX) |
+-----+
| "Russell Westbrook" |
| "Boris Diaw" |
+-----+
```

```

| "Aron Baynes" |
| "Ben Simmons" |
| "Blake Griffin" |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"*b*") YIELD id(vertex) | LIMIT 2,3;
+-----+
| id(VERTEX) |
+-----+
| "Aron Baynes" |
| "Ben Simmons" |
| "Blake Griffin" |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"*b*") YIELD id(vertex) | YIELD count(*);
+-----+
| count(*) |
+-----+
| 5 |
+-----+

trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"*b*") YIELD id(vertex), score() AS score;
+-----+
| id(VERTEX) | score |
+-----+
"Russell Westbrook"	1.0
"Boris Diaw"	1.0
"Aron Baynes"	1.0
"Ben Simmons"	1.0
"Blake Griffin"	1.0
+-----+

//对于包含 b 的词的文档，它的得分将乘以加权因子 4，而对于包含 c 的词的文档，使用默认加权因子 1。
trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_1,"*b^4 OR *c*") YIELD id(vertex), score() AS score;
+-----+
| id(VERTEX) | score |
+-----+
"Russell Westbrook"	4.0
"Boris Diaw"	4.0
"Aron Baynes"	4.0
"Ben Simmons"	4.0
"Blake Griffin"	4.0
"Chris Paul"	1.0
"Tim Duncan"	1.0
+-----+

//使用多属性全文索引查询，此时条件会在索引的所有属性内进行匹配。
trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_2,"*h*") YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {_vid: "Chris Paul", city: "Houston", name: "Chris Paul"} |
| {_vid: "Boris Diaw", city: "Houston", name: "Boris Diaw"} |
| {_vid: "David West", city: "Philadelphia", name: "David West"} |
| {_vid: "James Harden", city: "New York", name: "James Harden"} |
| {_vid: "Tony Parker", city: "Chicago", name: "Tony Parker"} |
| {_vid: "Aron Baynes", city: "Chicago", name: "Aron Baynes"} |
| {_vid: "Ben Simmons", city: "Phoenix", name: "Ben Simmons"} |
| {_vid: "Blake Griffin", city: "Phoenix", name: "Blake Griffin"} |
| {_vid: "Danny Green", city: "Philadelphia", name: "Danny Green"} |
+-----+

//使用多属性全文索引查询时，可以对不同的属性指定不同的文本进行查询。
trsgraph> LOOKUP ON player WHERE ES_QUERY(fulltext_index_2,"name:*b* AND city:Houston") YIELD properties(vertex);
+-----+
| properties(VERTEX) |
+-----+
| {_vid: "Boris Diaw", city: "Houston", name: "Boris Diaw"} |
+-----+

//删除单属性全文索引。
trsgraph> DROP FULLTEXT INDEX fulltext_index_1;

```

最后更新: 2025年12月19日

## 4.15 查询调优与终止

### 4.15.1 EXPLAIN 和 PROFILE

`EXPLAIN` 语句输出 nGQL 语句的执行计划，但不会执行 nGQL 语句。

`PROFILE` 语句执行 nGQL 语句，然后输出执行计划和执行概要。用户可以根据执行计划和执行概要优化查询性能。

#### 执行计划

执行计划由 TRSGraph 查询引擎中的执行计划器决定。

执行计划器将解析后的 nGQL 语句处理为 `action`。`action` 是最小的执行单元。典型的 `action` 包括获取指定点的所有邻居、获取边的属性、根据条件过滤点或边等。每个 `action` 都被分配给一个 `operator`。

例如 `SHOW TAGS` 语句分为两个 `action`，`operator` 为 `Start` 和 `ShowTags`。更复杂的 `GO` 语句可能会被处理成 10 个以上的 `action`。

#### 语法

- `EXPLAIN`

```
EXPLAIN [format= {"row" | "dot" | "tck"}] <your_nGQL_statement>;
```

- `PROFILE`

```
PROFILE [format= {"row" | "dot" | "tck"}] <your_nGQL_statement>;
```

#### 输出格式

`EXPLAIN` 或 `PROFILE` 语句的输出有三种格式：`row`（默认）、`dot` 和 `tck`。用户可以使用 `format` 选项修改输出格式。

row 格式

row 格式将返回信息输出到一个表格中。

EXPLAIN

```
trsgraph> EXPLAIN format="row" SHOW TAGS;
Execution succeeded (time spent 327/892 us)

Execution Plan

-----+-----+-----+-----+-----+
| id | name | dependencies | profiling data | operator info |
-----+-----+-----+-----+-----+
| 1 | ShowTags | 0 | | outputVar: [{"colNames":[],"name":"__ShowTags_1","type":"DATASET"}] |
| | | | | inputVar: |
-----+-----+-----+-----+-----+
| 0 | Start | | | outputVar: [{"colNames":[],"name":"__Start_0","type":"DATASET"}] |
-----+-----+-----+-----+-----+
```

PROFILE

```
trsgraph> PROFILE format="row" SHOW TAGS;
+-----+
| Name |
+-----+
| player |
| team |
+-----+
Got 2 rows (time spent 2038/2728 us)

Execution Plan

-----+-----+-----+-----+-----+
| id | name | dependencies | profiling data | operator info |
-----+-----+-----+-----+-----+
| 1 | ShowTags | 0 | ver: 0, rows: 1, execTime: 42us, totalTime: 1177us | outputVar: [{"colNames":[],"name":"__ShowTags_1","type":"DATASET"}] |
| | | | | inputVar: |
-----+-----+-----+-----+-----+
| 0 | Start | | ver: 0, rows: 0, execTime: 1us, totalTime: 57us | outputVar: [{"colNames":[],"name":"__Start_0","type":"DATASET"}] |
-----+-----+-----+-----+-----+
```

| 参数             | 说明                                                                                                                           |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| id             | operator 的 ID。                                                                                                               |
| name           | operator 的名称。                                                                                                                |
| dependencies   | 当前 operator 所依赖的 operator 的 ID。                                                                                              |
| profiling data | 执行概要文件内容。 ver 表示 operator 的版本； rows 表示 operator 输出结果的行数； execTime 表示执行 action 的时间； totalTime 表示执行 action 的时间、系统调度时间、排队时间的总和。 |
| operator info  | operator 的详细信息。                                                                                                              |

dot 格式

dot 格式将返回 DOT 语言的信息，然后用户可以使用 Graphviz 生成计划图。



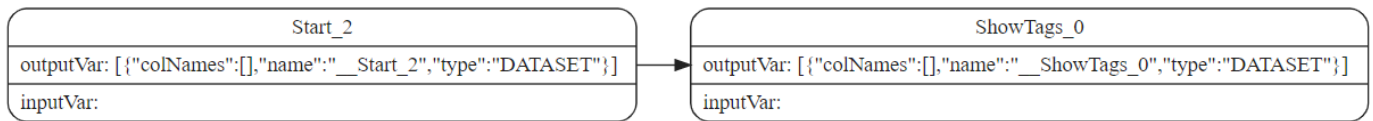
Graphviz 是一款开源可视化图工具，可以绘制 DOT 语言脚本描述的图。Graphviz 提供一个在线工具，可以预览 DOT 语言文件，并将它们导出为 SVG 或 JSON 等其他格式。详情请参见 [Graphviz Online](#)。

```
trsgraph> EXPLAIN format="dot" SHOW TAGS;
Execution succeeded (time spent 161/665 us)
Execution Plan

-----+-----+-----+-----+-----+
plan
-----+-----+-----+-----+-----+
digraph exec_plan {
 rankdir=LR;
 "ShowTags_0" [label="ShowTags_0|outputVar: [{"colNames":[],"name":"__ShowTags_0","type":"DATASET"}]|inputVar:\1", shape=Mrecord];
 "Start_2"->"ShowTags_0";
}
```

```
"Start_2" {label="Start_2" outputVar: [{"colNames":["__Start_2"],"name":"__Start_2","type":"DATASET"}] inputVar: \1", shape=Mrecord};
}
```

将上述示例的 DOT 语言转换为 Graphviz 图，如下所示。



### tck 格式

tck 格式类似于表格，但是没有边框和行之间的分割线。用户可以将结果用在单元测试的测试用例中。关于 tck 格式的测试用例，参见 TCK case。

#### • EXPLAIN

```
trsgaph> EXPLAIN format="tck" FETCH PROP ON player "player_1","player_2","player_3" YIELD properties(vertex).name as name, properties(vertex).age as age;
Execution succeeded (time spent 261µs/613.718µs)

Execution Plan (optimize time 28 us)

id	name	dependencies	profiling data	operator info
2	Project	1		
1	GetVertices	0		
0	Start			

Wed, 22 Mar 2023 23:15:52 CST
```

#### • PROFILE

```
trsgaph> PROFILE format="tck" FETCH PROP ON player "player_1","player_2","player_3" YIELD properties(vertex).name as name, properties(vertex).age as age;

name	age
"Piter Park"	24
"aaa"	24
"ccc"	24

Got 3 rows (time spent 1.474ms/2.19677ms)

Execution Plan (optimize time 41 us)

id	name	dependencies	profiling data	operator info
2	Project	1	{"rows":3,"version":0}	
1	GetVertices	0	{"resp[0]":{"exec":"232(us)","host":"127.0.0.1:9779","total":"758(us)","rows":3,"total_rpc":"875(us)","version":0}}	
0	Start		{"rows":0,"version":0}	

Wed, 22 Mar 2023 23:16:13 CST
```

最后更新: 2025年12月19日

## 4.15.2 终止查询

`KILL QUERY` 命令可以终止正在执行的查询，常用于终止慢查询。



Note

仅 God 角色权限可以终止任意查询，其他角色只能终止自己的查询。

### 语法

```
KILL QUERY (session=<session_id>, plan=<plan_id>);
```

- `session_id`：会话 ID。
- `plan_id`：执行计划 ID。

会话 ID 和执行计划 ID 可以唯一确定一个查询。二者可以通过 `SHOW QUERIES` 语句获取。

### 示例

在一个会话中执行命令终止另一个会话中的查询：

```
trsgraph> KILL QUERY (SESSION=1625553545984255, PLAN=163);
```

另一个会话中的查询会终止，并返回如下信息：

```
[ERROR (-1005)]: ExecutionPlanId[1001] does not exist in current Session.
```

---

最后更新: 2025年12月19日

### 4.15.3 终止会话 (KILL SESSION)

`KILL SESSION` 语句用于关闭未登出的会话 (Session)。



Note

- 只有 `root` 用户可以终止会话。
- 执行 `KILL SESSION` 命令后，所有的 Graph 服务同步最新的会话信息需要等待  $2 * \text{session\_reclaim\_interval\_secs}$  秒，默认等待 120 秒。

#### 语法

`KILL SESSION` 语句支持终止单个和多个会话，语法如下：

- 终止单个会话

```
KILL {SESSION|SESSIONS} <SessionId>
```

- `{SESSION|SESSIONS}`：支持 `SESSION` 和 `SESSIONS` 的写法。
- `<SessionId>`：指会话 Session 的 ID。可执行 `SHOW SESSIONS` 命令查看会话 ID。

- 终止多个会话

```
SHOW SESSIONS
| YIELD $-.SessionId AS sid [WHERE <filter_clause>]
| KILL {SESSION|SESSIONS} $-.sid
```



Note

`KILL SESSION` 语句支持管道操作，即将 `SHOW SESSIONS` 语句与 `KILL SESSION` 语句结合使用，以终止多个会话。

- `[WHERE <filter_clause>]`：
- 可选项，使用 `WHERE` 子句过滤会话；`<filter_expression>` 指过滤表达式，例如 `WHERE $-.CreateTime < datetime("2022-12-14T18:00:00")`。如果不加该选项，则关闭所有当前会话。
- `WHERE` 子句中支持的过滤项有：`SessionId`、`UserName`、`SpaceName`、`CreateTime`、`UpdateTime`、`GraphAddr`、`Timezone`、`ClientIp`。可以执行 `SHOW SESSIONS` 命令查看这些过滤项的含义。
- `{SESSION|SESSIONS}`：支持 `SESSION` 和 `SESSIONS` 的写法。



Caution

请谨慎使用过滤条件以防误删会话。

#### 示例

- 终止单个会话。



```
trsgraph> KILL SESSION 1672887983842984
```

- 终止多个会话。
- 终止创建时间小于 2023-01-05T18:00:00 的所有会话。

```
trsgraph> SHOW SESSIONS | YIELD $-.SessionId AS sid WHERE $-.CreateTime < datetime("2023-01-05T18:00:00") | KILL SESSIONS $-.sid
```

- 终止创建时间最早的两个会话。

```
trsgraph> SHOW SESSIONS | YIELD $-.SessionId AS sid, $-.CreateTime as CreateTime | ORDER BY $-.CreateTime ASC | LIMIT 2 | KILL SESSIONS $-.sid
```

- 终止用户名为 session\_user1 创建的所有会话。

```
trsgraph> SHOW SESSIONS | YIELD $-.SessionId as sid WHERE $-.UserName == "session_user1" | KILL SESSIONS $-.sid
```

- 终止所有会话

```
trsgraph> SHOW SESSIONS | YIELD $-.SessionId as sid | KILL SESSION $-.sid
```

// 或者

```
trsgraph> SHOW SESSIONS | KILL SESSIONS $-.SessionId
```



终止所有会话时，当前会话也会被终止。请谨慎使用。

---

最后更新: 2025年12月19日

## 4.16 作业管理

在 Storage 服务上长期运行的任务称为作业，例如 `COMPACT`、`FLUSH` 和 `STATS`。如果图空间的数据量很大，这些作业可能耗时很长。作业管理可以帮助执行、查看、停止和恢复作业。



所有作业管理命令都需要先选择图空间后才能执行。

### 4.16.1 SUBMIT JOB BALANCE LEADER

`SUBMIT JOB BALANCE LEADER` 语句会启动任务均衡分布所有图空间中的 leader。该命令会返回任务 ID。

示例：

```
trsgraph> SUBMIT JOB BALANCE LEADER;
+-----+
| New Job Id |
+-----+
| 33 |
+-----+
```

### 4.16.2 SUBMIT JOB COMPACT

`SUBMIT JOB COMPACT` 语句会在当前图空间内触发 RocksDB 的长耗时 `compact` 操作。

`compact` 配置详情请参见 [Storage 服务配置](#)。

示例：

```
trsgraph> SUBMIT JOB COMPACT;
+-----+
| New Job Id |
+-----+
| 40 |
+-----+
```

### 4.16.3 SUBMIT JOB FLUSH

`SUBMIT JOB FLUSH` 语句将当前图空间内存中的 RocksDB memfile 写入硬盘。

示例：

```
trsgraph> SUBMIT JOB FLUSH;
+-----+
| New Job Id |
+-----+
| 96 |
+-----+
```

### 4.16.4 SUBMIT JOB STATS

`SUBMIT JOB STATS` 语句会在当前图空间内启动一个作业，该作业对当前图空间进行统计。作业完成后，用户可以使用 `SHOW STATS` 语句列出统计结果。详情请参见 [SHOW STATS](#)。



如果存储在 TRSGraph 中的数据有变化，为了获取最新的统计结果，请重新执行 `SUBMIT JOB STATS`。

示例：

```
trsgraph> SUBMIT JOB STATS;
+-----+
| New Job Id |
+-----+
| 9 |
+-----+
```

### 4.16.5 SUBMIT JOB DOWNLOAD/INGEST

`SUBMIT JOB DOWNLOAD HDFS` 和 `SUBMIT JOB INGEST` 命令用于将 SST 文件导入 TRSGraph。详情参见[导入 SST 文件数据](#)。

`SUBMIT JOB DOWNLOAD HDFS` 语句会下载指定的 HDFS 上的 SST 文件。

`SUBMIT JOB INGEST` 语句会将下载的 SST 文件导入图空间。

示例：

```
trsgraph> SUBMIT JOB DOWNLOAD HDFS "hdfs://192.168.10.100:9000/sst";
+-----+
| New Job Id |
+-----+
| 10 |
+-----+

trsgraph> SUBMIT JOB INGEST;
+-----+
| New Job Id |
+-----+
| 11 |
+-----+
```

### 4.16.6 SHOW JOB

Meta 服务将 `SUBMIT JOB` 请求解析为多个任务，然后分配给进程 `trsgraph-storaged`。`SHOW JOB <job_id>` 语句显示当前图空间内指定作业和相关任务的信息。

`job_id` 在执行 `SUBMIT JOB` 语句时会返回。

示例：

```
trsgraph> SHOW JOB 8;
+-----+-----+-----+-----+-----+-----+
| Job Id(TaskId) | Command(Dest) | Status | Start Time | Stop Time | Error Code |
+-----+-----+-----+-----+-----+-----+
8	"STATS"	"FINISHED"	2022-10-18T08:14:45.000000	2022-10-18T08:14:45.000000	"SUCCEEDED"
0	"192.168.8.129"	"FINISHED"	2022-10-18T08:14:45.000000	2022-10-18T08:15:13.000000	"SUCCEEDED"
"Total:1"	"Succeeded:1"	"Failed:0"	"In Progress:0"	""	""
+-----+-----+-----+-----+-----+-----+
```

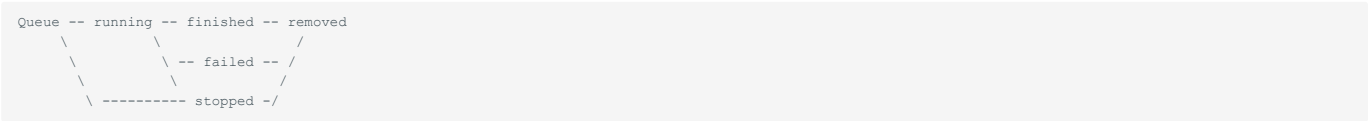
| 参数                          | 说明                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------|
| <code>Job Id(TaskId)</code> | 第一行显示作业 ID，最后一行显示作业相关的任务总数，其他行显示作业相关的任务 ID。                                                  |
| <code>Command(Dest)</code>  | 第一行显示执行的作业命令名称，最后一行显示作业相关的成功的任务数。其他行显示任务对应的 <code>trsgraph-storaged</code> 进程。               |
| <code>Status</code>         | 显示作业或任务的状态，最后一行显示作业相关的失败的任务数。详情请参见 <a href="#">作业状态</a> 。                                    |
| <code>Start Time</code>     | 显示作业或任务开始执行的时间，最后一行显示作业相关的正在进行的任务数。                                                          |
| <code>Stop Time</code>      | 显示作业或任务结束执行的时间，结束后的状态包括 <code>FINISHED</code> 、 <code>FAILED</code> 或 <code>STOPPED</code> 。 |
| <code>Error Code</code>     | 显示作业或任务的错误码。                                                                                 |

作业状态

作业状态的说明如下。

| 状态       | 说明                                            |
|----------|-----------------------------------------------|
| QUEUE    | 作业或任务在等待队列中。此阶段 <code>Start Time</code> 为空。   |
| RUNNING  | 作业或任务在执行中。 <code>Start Time</code> 为该阶段的起始时间。 |
| FINISHED | 作业或任务成功完成。 <code>Stop Time</code> 为该阶段的起始时间。  |
| FAILED   | 作业或任务失败。 <code>Stop Time</code> 为该阶段的起始时间。    |
| STOPPED  | 作业或任务停止。 <code>Stop Time</code> 为该阶段的起始时间。    |
| REMOVED  | 作业或任务被删除。                                     |

状态转换的说明如下。



4.16.7 SHOW JOBS

`SHOW JOBS` 语句列出当前图空间内所有未过期的作业。

作业的默认过期时间为一周。如果需要修改过期时间，请修改 `Meta` 服务的参数 `job_expired_secs`。详情请参见 [Meta 服务配置](#)。

示例：

```
trsgaph> SHOW JOBS;
+-----+-----+-----+-----+-----+
| Job Id | Command | Status | Start Time | Stop Time |
+-----+-----+-----+-----+-----+
34	"STATS"	"FINISHED"	2021-11-01T03:32:27.000000	2021-11-01T03:32:27.000000
33	"FLUSH"	"FINISHED"	2021-11-01T03:32:15.000000	2021-11-01T03:32:15.000000
32	"COMPACT"	"FINISHED"	2021-11-01T03:32:06.000000	2021-11-01T03:32:06.000000
31	"REBUILD_TAG_INDEX"	"FINISHED"	2021-10-29T05:39:16.000000	2021-10-29T05:39:17.000000
10	"COMPACT"	"FINISHED"	2021-10-26T02:27:05.000000	2021-10-26T02:27:05.000000
+-----+-----+-----+-----+-----+
```

4.16.8 STOP JOB

`STOP JOB <job_id>` 语句可以停止当前图空间内未完成的作业。

示例：

```
trsgraph> STOP JOB 22;
+-----+
| Result |
+-----+
| "Job stopped" |
+-----+
```

4.16.9 RECOVER JOB

`RECOVER JOB [<job_id>]` 语句可以重新执行当前图空间内状态为 `FAILED`、`STOPPED` 的作业，未指定 `<job_id>` 时，会从最早的作业开始尝试重新执行，并返回已恢复的作业数量。

示例：

```
trsgraph> RECOVER JOB;
+-----+
| Recovered job num |
+-----+
```

```
| 5 job recovered |
+-----+
```

### 4.16.10 FAQ

#### 如何排查作业问题?

`SUBMIT JOB` 操作使用的是 HTTP 端口，请检查 Storage 服务机器上的 HTTP 端口是否正常工作。用户可以执行如下命令调试：

```
curl "http://{storaged-ip}:19779/admin?space={space_name}&op=compact"
```

最后更新: 2025年12月19日

## 5. 配置与日志

### 5.1 配置

#### 5.1.1 配置管理

TRSGraph 基于 `gflags` 库打造了系统配置，多数配置项都是其中的 flags。TRSGraph 服务启动时，默认会从配置文件中获取配置信息。对于文件中没有的配置项，系统使用默认值。

#### 查看配置项列表与说明

使用以下命令获取二进制文件对应服务的所有配置项信息：

```
<binary> --help
```

例如：

```
获取 Meta 配置项的帮助信息
$ /usr/local/trsgraph/bin/trsgraph-metad --help

获取 Graph 配置项的帮助信息
$ /usr/local/trsgraph/bin/trsgraph-graphd --help

获取 Storage 配置项的帮助信息
$ /usr/local/trsgraph/bin/trsgraph-storaged --help
```

以上示例使用了二进制文件的默认存储路径 `/usr/local/trsgraph/bin/`。如果修改了 TRSGraph 安装路径，使用实际路径查询配置项信息。

#### 查看运行配置

使用 `curl` 命令获取运行中的配置项的值，即 TRSGraph 的运行配置。

例如：

```
获取 Meta 服务的运行配置
curl 127.0.0.1:19559/flags

获取 Graph 服务的运行配置
curl 127.0.0.1:19669/flags

获取 Storage 服务的运行配置
curl 127.0.0.1:19779/flags
```

使用 `-s` 或者 `--silent` 参数可以隐藏进度条和错误信息。例如：

```
curl -s 127.0.0.1:19559/flags
```



实际环境中需使用真实的 IP（或主机名）取代以上示例中的 `127.0.0.1`。

#### 配置文件简介

TRSGraph 为每个服务都提供了两份初始配置文件 `<service_name>.conf.default` 和 `<service_name>.conf.production`，方便用户在不同场景中使用。配置文件路径为 `<install_path>/<tar_package_directory>/etc` TAR 包的安装路径。

初始配置文件中的配置值仅供参考，使用时可根据实际需求调整。如需使用初始配置文件，从上述两个文件选择其一，删除后缀 `.default` 或 `.production` 使其生效。



为确保服务的可用性，同类服务的配置建议保持一致，本机 `local_ip` 除外。例如，一个 TRSGraph 集群中部署了 3 个 Storage 服务器，3 者除 `local_ip` 外的其它配置建议都相同。

下表列出了各服务对应的初始配置文件。

| TRSGraph 服务 | 初始配置文件                                                                                       | 配置说明         |
|-------------|----------------------------------------------------------------------------------------------|--------------|
| Meta        | <code>trsgraph-metad.conf.default</code> 和 <code>trsgraph-metad.conf.production</code>       | Meta 服务配置    |
| Graph       | <code>trsgraph-graphd.conf.default</code> 和 <code>trsgraph-graphd.conf.production</code>     | Graph 服务配置   |
| Storage     | <code>trsgraph-storaged.conf.default</code> 和 <code>trsgraph-storaged.conf.production</code> | Storage 服务配置 |

所有服务的初始配置文件中都包含 `local_config` 参数，预设值为 `true`，表示 TRSGraph 服务会从其配置文件获取配置并启动。



不建议修改 `local_config` 的值为 `false`。修改后 TRSGraph 服务启动后会先尝试从 Meta 服务获取缓存的配置信息，可能导致集群配置不一致，造成未知风险。

修改配置

用户可以在配置文件中修改 TRSGraph 的配置，或使用命令动态修改配置。



同时使用两种方式修改配置会导致配置信息失去统一的管理方式，可能造成配置混乱。建议仅使用配置文件管理配置，或在通过命令动态更新配置后对配置文件做相同的修改，以保证一致性。

在配置文件中修改配置

默认情况下，所有 TRSGraph 服务从配置文件获取配置。用户可以按照以下步骤修改配置并使其生效。

- 针对使用RPM/DEB、TAR 包安装的集群：
  - a. 使用文本编辑器修改目标服务的配置文件并保存。
  - b. 选择合适的时间重启所有 TRSGraph 服务使修改生效。

使用命令动态修改配置

用户可以通过 `curl` 命令动态修改 TRSGraph 服务的配置。例如，修改 Storage 服务的 `wal_ttl` 参数为 `600`，命令如下：

```
curl -X PUT -H "Content-Type: application/json" -d '{"wal_ttl": "600"}' -s "http://192.168.15.6:19779/flags"
```

其中，`{"wal_ttl": "600"}` 为待修改的配置参数及其值；`192.168.15.6:19779` 为 Storage 服务的 IP 地址和 HTTP 端口号。



- 动态修改配置功能仅适用于原型验证和测试环境，不建议在生产环境中使用。因为当 `local_config` 值设置为 `true` 时，动态修改的配置不会持久化，重启服务后配置会恢复为初始配置。
- 仅支持动态修改部分配置参数，具体支持的参数列表，参见各服务配置中是否支持运行时动态修改的描述。

最后更新: 2025年12月19日



### 5.1.2 Meta 服务配置

Meta 服务提供了两份初始配置文件 `trsgraph-metad.conf.default` 和 `trsgraph-metad.conf.production`，方便在不同场景中使用。文件的默认路径为 `/usr/local/trsgraph/etc/`。



- 不建议修改 `local_config` 的值为 `false`。修改后 TRSGraph 服务启动后会先尝试从 Meta 服务获取缓存的配置信息，可能导致集群配置不一致，造成未知风险。
- 不建议修改文档未介绍的配置项，除非已经熟悉源代码并清楚了解配置项作用。

#### 配置文件使用方式

如需使用初始配置文件，从上述两个文件选择其一，删除后缀 `.default` 或 `.production`，Meta 服务才能将其识别为配置文件并从中获取配置信息。

#### 配置文件参数值说明

配置文件内没有设置某个参数表示参数使用的是默认值。文件内只预设了部分参数的值，而且两份初始配置文件内的参数值也略有不同，本文的预设值以 `trsgraph-metad.conf.default` 为准。



配置文件中的部分参数值可以在运行时动态修改。本文将这些参数标记为支持运行时动态修改，并当 `local_config` 值设置为 `true` 时重启服务后配置会恢复为初始配置。详情参见[修改配置](#)。

如需查看所有的参数及其当前值，参见[配置管理](#)。

#### basics 配置

| 名称                         | 预设值                                  | 说明                                                                                                                                                                                      | 是否支持运行时动态修改 |
|----------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>daemonize</code>     | <code>true</code>                    | 是否启动守护进程。                                                                                                                                                                               | 不支持         |
| <code>pid_file</code>      | <code>pids/trsgraph-metad.pid</code> | 记录进程 ID 的文件。                                                                                                                                                                            | 不支持         |
| <code>timezone_name</code> | -                                    | 指定 TRSGraph 的时区。初始配置文件中未设置该参数，如需使用请手动添加。系统默认值为 <code>UTC+00:00:00</code> 。格式请参见 <a href="#">Specifying the Time Zone with TZ</a> 。例如，东八区的设置方式为 <code>--timezone_name=UTC+08:00</code> 。 | 不支持         |



- 在插入时间类型的属性值时，TRSGraph 会根据 `timezone_name` 设置的时区将该时间值（TIMESTAMP 类型例外）转换成相应的 UTC 时间，因此在查询中返回的时间类型属性值为 UTC 时间。
- `timezone_name` 参数只用于转换 TRSGraph 中存储的数据，TRSGraph 进程中其它时区相关数据，例如日志打印的时间等，仍然使用主机系统默认的时区。

logging 配置

| 名称                        | 预设值              | 说明                                                                                                                                            | 是否支持运行时动态修改 |
|---------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| log_dir                   | logs             | 存放 Meta 服务日志的目录，建议和数据保存在不同硬盘。                                                                                                                 | 不支持         |
| minloglevel               | 0                | 最小日志级别，即记录此级别或更高级别的日志。可选值为 0（INFO）、1（WARNING）、2（ERROR）、3（FATAL）。建议在调试时设置为 0，生产环境中设置为 1。如果设置为 4，TRSGraph 不会记录任何日志。                             | 支持          |
| v                         | 0                | VLOG 日志详细级别，即记录小于或等于此级别的所有 VLOG 消息。可选值为 0、1、2、3、4、5。glog 提供的 VLOG 宏允许用户定义自己的数字日志记录级别，并用参数 v 控制记录哪些详细消息。详情参见 <a href="#">Verbose Logging</a> 。 | 支持          |
| logbufsecs                | 0                | 缓冲日志的最大时间，超时后输出到日志文件。0 表示实时输出。单位：秒。                                                                                                           | 不支持         |
| redirect_stdout           | true             | 是否将标准输出和标准错误重定向到单独的输出文件。                                                                                                                      | 不支持         |
| stdout_log_file           | metad-stdout.log | 标准输出日志文件名称。                                                                                                                                   | 不支持         |
| stderr_log_file           | metad-stderr.log | 标准错误日志文件名称。                                                                                                                                   | 不支持         |
| stderrthreshold           | 3                | 要复制到标准错误中的最小日志级别（minloglevel）。                                                                                                                | 不支持         |
| timestamp_in_logfile_name | true             | 日志文件名称中是否包含时间戳。true 表示包含，false 表示不包含。                                                                                                         | 不支持         |

networking 配置

| 名称                   | 预设值            | 说明                                                                                      | 是否支持运行时动态修改 |
|----------------------|----------------|-----------------------------------------------------------------------------------------|-------------|
| meta_server_addrs    | 127.0.0.1:9559 | 全部 Meta 服务的 IP（或主机名）和端口。多个 Meta 服务用英文逗号（,）分隔。                                           | 不支持         |
| local_ip             | 127.0.0.1      | Meta 服务的本地 IP（或主机名）。本地 IP 用于识别 trsgraph-metad 进程，如果是分布式集群或需要远程访问，请修改为对应地址。              | 不支持         |
| port                 | 9559           | Meta 服务的 RPC 守护进程监听端口。同时还会使用相邻的 +1（9560）端口用于 Meta 服务之间的 Raft 通信。                        | 不支持         |
| ws_ip                | 0.0.0.0        | HTTP 服务的 IP 地址。                                                                         | 不支持         |
| ws_http_port         | 19559          | HTTP 服务的端口。                                                                             | 不支持         |
| ws_storage_http_port | 19779          | HTTP 协议监听 Storage 服务的端口，需要和 Storage 服务配置文件中的 ws_http_port 保持一致。仅存算合并版 TRSGraph 需要设置本参数。 | 不支持         |



使用 IP 时建议使用真实的 IP。否则某些情况下 127.0.0.1/0.0.0.0 无法正确解析。

storage 配置

| 名称        | 预设值       | 说明           | 是否支持运行时动态修改 |
|-----------|-----------|--------------|-------------|
| data_path | data/meta | meta 数据存储路径。 | 不支持         |

misc 配置

| 名称                            | 预设值 | 说明                                                               | 是否支持运行时动态修改 |
|-------------------------------|-----|------------------------------------------------------------------|-------------|
| default_parts_num             | 10  | 创建图空间时的默认分片数量。                                                   | 不支持         |
| default_replica_factor        | 1   | 创建图空间时的默认副本数量。                                                   | 不支持         |
| heartbeat_interval_secs       | 10  | 默认心跳间隔。请确保所有服务的 heartbeat_interval_secs 取值相同，否则会导致系统无法正常工作。单位：秒。 | 支持          |
| agent_heartbeat_interval_secs | 60  | Agent 服务发送心跳的时间间隔。该值影响系统确定 Agent 服务离线状态的时间。                      | 不支持         |

rocksdb options 配置

| 名称               | 预设值  | 说明                     | 是否支持运行时动态修改 |
|------------------|------|------------------------|-------------|
| rocksdb_wal_sync | true | 是否同步 RocksDB 的 WAL 日志。 | 不支持         |

最后更新: 2025年12月19日

### 5.1.3 Graph 服务配置

Graph 服务提供了两份初始配置文件 `trsgraph-graphd.conf.default` 和 `trsgraph-graphd.conf.production`，方便在不同场景中使用。文件的默认路径为 `/usr/local/trsgraph/etc/`。



- 不建议修改 `local_config` 的值为 `false`。修改后 TRSGraph 服务启动后会先尝试从 Meta 服务获取缓存的配置信息，可能导致集群配置不一致，造成未知风险。
- 不建议修改文档未介绍的配置项，除非已经熟悉源代码并完全了解配置项的作用。

#### 配置文件使用方式

如需使用初始配置文件，从上述两个文件选择其一，删除后缀 `.default` 或 `.production`，Graph 服务才能将其识别为配置文件并从中获取配置信息。

#### 配置文件参数值说明

配置文件内没有设置某个参数表示参数使用的是默认值。文件内只预设了部分参数的值，而且两份初始配置文件内的参数值也略有不同，本文的预设值以 `trsgraph-graphd.conf.default` 为准。



配置文件中的部分参数值可以在运行时动态修改。本文将这些参数标记为支持运行时动态修改，并当 `local_config` 值设置为 `true` 时重启服务后配置会恢复为初始配置。详情参见[修改配置](#)。

如需查看所有的参数及其当前值，参见[配置管理](#)。

#### basics 配置

| 名称                            | 预设值                                   | 说明                                                                                                                                                                                    | 是否支持运行时动态修改 |
|-------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>daemonize</code>        | <code>true</code>                     | 是否启动守护进程。                                                                                                                                                                             | 不支持         |
| <code>pid_file</code>         | <code>pids/trsgraph-graphd.pid</code> | 记录进程 ID 的文件。                                                                                                                                                                          | 不支持         |
| <code>enable_optimizer</code> | <code>true</code>                     | 是否启用优化器。                                                                                                                                                                              | 不支持         |
| <code>timezone_name</code>    | -                                     | 指定 TRSGraph 的时区。初始配置文件中未设置该参数，使用需手动添加。系统默认值为 <code>UTC+00:00:00</code> 。格式请参见 <a href="#">Specifying the Time Zone with TZ</a> 。例如，东八区的设置方式为 <code>--timezone_name=UTC+08:00</code> 。 | 不支持         |
| <code>default_charset</code>  | <code>utf8</code>                     | 创建图空间时的默认字符集。                                                                                                                                                                         | 不支持         |
| <code>default_collate</code>  | <code>utf8_bin</code>                 | 创建图空间时的默认排序规则。                                                                                                                                                                        | 不支持         |
| <code>local_config</code>     | <code>true</code>                     | 是否从配置文件获取配置信息。                                                                                                                                                                        | 不支持         |



Note

- 在插入时间类型的属性值时，TRSGraph 会根据 `timezone_name` 设置的时区将该时间值（TIMESTAMP 类型例外）转换成相应的 UTC 时间，因此在查询中返回的时间类型属性值为 UTC 时间。
- `timezone_name` 参数只用于转换 TRSGraph 中存储的数据，TRSGraph 进程中其它时区相关数据，例如日志打印的时间等，仍然使用主机系统默认的时区。

logging 配置

| 名称                                     | 预设值                            | 说明                                                                                                                                                                                                                                                  | 是否支持运行时动态修改 |
|----------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>log_dir</code>                   | <code>logs</code>              | 存放 Graph 服务日志的目录，建议和数据保存在不同硬盘。                                                                                                                                                                                                                      | 不支持         |
| <code>minloglevel</code>               | <code>0</code>                 | 最小日志级别，即记录此级别或更高级别的日志。可选值为 <code>0</code> （INFO）、 <code>1</code> （WARNING）、 <code>2</code> （ERROR）、 <code>3</code> （FATAL）。建议在调试时设置为 <code>0</code> ，生产环境中设置为 <code>1</code> 。如果设置为 <code>4</code> ，TRSGraph 不会记录任何日志。                              | 支持          |
| <code>v</code>                         | <code>0</code>                 | VLOG 日志详细级别，即记录小于或等于此级别的所有 VLOG 消息。可选值为 <code>0</code> 、 <code>1</code> 、 <code>2</code> 、 <code>3</code> 、 <code>4</code> 、 <code>5</code> 。glog 提供的 VLOG 宏允许用户定义自己的数字日志记录级别，并用参数 <code>v</code> 控制记录哪些详细消息。详情参见 <a href="#">Verbose Logging</a> 。 | 支持          |
| <code>logbufsecs</code>                | <code>0</code>                 | 缓冲日志的最大时间，超时后输出到日志文件。 <code>0</code> 表示实时输出。单位：秒。                                                                                                                                                                                                   | 不支持         |
| <code>redirect_stdout</code>           | <code>true</code>              | 是否将标准输出和标准错误重定向到单独的输出文件。                                                                                                                                                                                                                            | 不支持         |
| <code>stdout_log_file</code>           | <code>graphd-stdout.log</code> | 标准输出日志文件名称。                                                                                                                                                                                                                                         | 不支持         |
| <code>stderr_log_file</code>           | <code>graphd-stderr.log</code> | 标准错误日志文件名称。                                                                                                                                                                                                                                         | 不支持         |
| <code>stderrthreshold</code>           | <code>3</code>                 | 要复制到标准错误中的最小日志级别（ <code>minloglevel</code> ）。                                                                                                                                                                                                       | 不支持         |
| <code>timestamp_in_logfile_name</code> | <code>true</code>              | 日志文件名称中是否包含时间戳。 <code>true</code> 表示包含， <code>false</code> 表示不包含。                                                                                                                                                                                   | 不支持         |

query 配置

| 名称                                         | 预设值                  | 说明                                                                                   | 是否支持运行时动态修改 |
|--------------------------------------------|----------------------|--------------------------------------------------------------------------------------|-------------|
| <code>accept_partial_success</code>        | <code>false</code>   | 是否将部分成功视为错误。此配置仅适用于只读请求，写请求总是将部分成功视为错误。查询部分成功时，会提示 <code>Got partial result</code> 。 | 支持          |
| <code>session_reclaim_interval_secs</code> | <code>60</code>      | 将 Session 信息发送给 Meta 服务的间隔。单位：秒。                                                     | 支持          |
| <code>max_allowed_query_size</code>        | <code>4194304</code> | 最大查询语句长度。单位：字节。默认为 <code>4194304</code> ，即 4MB。                                      | 支持          |

networking 配置

| 名称                        | 预设值            | 说明                                                                             | 是否支持运行时动态修改 |
|---------------------------|----------------|--------------------------------------------------------------------------------|-------------|
| meta_server_addrs         | 127.0.0.1:9559 | 全部 Meta 服务的 IP（或主机名）和端口。多个 Meta 服务用英文逗号（,）分隔。                                  | 不支持         |
| local_ip                  | 127.0.0.1      | Graph 服务的本地 IP（或主机名）。本地 IP 用于识别 trsgraph-graphd 进程，如果是分布式集群或需要远程访问，请修改为对应地址。   | 不支持         |
| listen_netdev             | any            | 监听的网络设备。                                                                       | 不支持         |
| port                      | 9669           | Graph 服务的 RPC 守护进程监听端口。                                                        | 不支持         |
| reuse_port                | false          | 是否启用 SO_REUSEPORT。                                                             | 不支持         |
| listen_backlog            | 1024           | socket 监听的连接队列最大长度，调整本参数需要同时调整 net.core.somaxconn。                             | 不支持         |
| client_idle_timeout_secs  | 28800          | 空闲连接的超时时间。取值范围为 1~604800，单位：秒。默认 8 小时。                                         | 不支持         |
| session_idle_timeout_secs | 28800          | 空闲会话的超时时间。取值范围为 1~604800。默认 8 小时。单位：秒。                                         | 不支持         |
| num_accept_threads        | 1              | 接受传入连接的线程数。                                                                    | 不支持         |
| num_netio_threads         | 0              | 网络 IO 线程数。0 表示 CPU 核数。                                                         | 不支持         |
| num_max_connections       | 0              | 所有网络线程的最大活动连接数，0 表示没有限制。每个网络线程的最大连接数= num_max_connections / num_netio_threads。 | 不支持         |
| num_worker_threads        | 0              | 执行用户查询的线程数。0 表示 CPU 核数。                                                        | 不支持         |
| ws_ip                     | 0.0.0.0        | HTTP 服务的 IP 地址。                                                                | 不支持         |
| ws_http_port              | 19669          | HTTP 服务的端口。                                                                    | 不支持         |
| heartbeat_interval_secs   | 10             | 默认心跳间隔。请确保所有服务的 heartbeat_interval_secs 取值相同，否则会导致系统无法正常工作。单位：秒。               | 支持          |
| storage_client_timeout_ms | -              | Graph 服务与 Storage 服务的 RPC 连接超时时间。初始配置文件中未设置该参数，使用需手动添加。默认值为 60000 毫秒。          | 不支持         |
| slow_query_threshold_us   | 200000         | 定义超过多长时间的查询为慢查询。单位：微秒。注意：DML 语句的执行时间即使超过该值，也不会被记录为慢查询。                         | 不支持         |
| ws_meta_http_port         | 19559          | HTTP 协议监听 Meta 服务的端口，需要和 Meta 服务配置文件中的 ws_http_port 保持一致。                      | 不支持         |



使用 IP 时建议使用真实的 IP。否则某些情况下 127.0.0.1/0.0.0.0 无法正确解析。

authorization 配置

| 名称                            | 预设值                   | 说明                                                                               | 是否支持运行时动态修改 |
|-------------------------------|-----------------------|----------------------------------------------------------------------------------|-------------|
| <code>enable_authorize</code> | <code>false</code>    | 用户登录时是否进行身份验证。身份验证详情请参见 <a href="#">身份验证</a> 。                                   | 不支持         |
| <code>auth_type</code>        | <code>password</code> | 用户登录的身份验证方式。取值为 <code>password</code> 、 <code>ldap</code> 、 <code>cloud</code> 。 | 不支持         |

memory 配置

| 名称                                              | 预设值              | 说明                                                  | 是否支持运行时动态修改 |
|-------------------------------------------------|------------------|-----------------------------------------------------|-------------|
| <code>system_memory_high_watermark_ratio</code> | <code>0.8</code> | 内存高水位报警机制的触发阈值。系统内存占用率高于该值会触发报警机制，TRSGraph 会停止接受查询。 | 支持          |

metrics 配置

| 名称                                      | 预设值                | 说明                                                                                                                                                                    | 是否支持运行时动态修改 |
|-----------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>enable_space_level_metrics</code> | <code>false</code> | 开启后可打开图空间级别的监控，对应的监控指标名称中包含图空间的名称，例如 <code>query_latency_us{space=basketballplayer}.avg.3600</code> 。支持的监控指标可用 <code>curl</code> 命令查看，详细说明参见 <a href="#">查询监控指标</a> 。 | 不支持         |

session 配置

| 名称                                        | 预设值              | 说明                       | 是否支持运行时动态修改 |
|-------------------------------------------|------------------|--------------------------|-------------|
| <code>max_sessions_per_ip_per_user</code> | <code>300</code> | 相同用户和 IP 地址可以创建的最大活跃会话数。 | 不支持         |

experimental 配置

| 名称                                       | 预设值                | 说明                                                                              | 是否支持运行时动态修改 |
|------------------------------------------|--------------------|---------------------------------------------------------------------------------|-------------|
| <code>enable_experimental_feature</code> | <code>false</code> | 实验性功能开关。可选值为 <code>true</code> 和 <code>false</code> 。                           | 不支持         |
| <code>enable_data_balance</code>         | <code>true</code>  | 是否开启均衡分片功能。仅当 <code>enable_experimental_feature</code> 为 <code>true</code> 时生效。 | 不支持         |



memory tracker 配置



Memory Tracker 是一个内存管理工具，用于监控和限制内存使用。对于大型查询，Memory Tracker 可以防止 OOM 问题。如果在容器环境中使用 Memory Tracker，需要在 Graph 服务的配置文件中添加相关配置。操作如下：

- 1. 创建目录 `/sys/fs/cgroup/graphd/`，添加并配置 `memory.max`。
- 2. 在 `etc/trsgraph-graphd.conf` 添加以下配置。

```
--containerized=true
--cgroup_v2_controllers=/sys/fs/cgroup/graphd/cgroup.controllers
--cgroup_v2_memory_stat_path=/sys/fs/cgroup/graphd/memory.stat
--cgroup_v2_memory_max_path=/sys/fs/cgroup/graphd/memory.max
--cgroup_v2_memory_current_path=/sys/fs/cgroup/graphd/memory.current
```

详细信息，请参见[容器/cgroup 模式](#)。

| 名称                                                       | 预设值                | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 是否支持运行时动态修改 |
|----------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>memory_tracker_limit_ratio</code>                  | <code>0.8</code>   | 取值可设置为： <code>{0, 1}</code> 、 <code>2</code> 、 <code>3</code> 。<br>警惕：设置该参数时请确保 <code>system_memory_high_watermark_ratio</code> 的值不为 <code>1</code> ，否则该参数的值不生效。<br><code>{0, 1}</code> ：可用内存的百分比。计算公式：可用内存的百分比 = 可用内存 / (总内存 - 保留内存)。<br>当正在进行的查询导致内存使用超过配置的内存限制时，该查询会失败，并在失败后释放内存。<br>注意：对于云上和本地节点混合部署的集群，需要根据实际情况调小该参数。例如，当预期 Graphd 只占用 50% 的内存时，该参数的值可设置为小于 <code>0.5</code> 。<br><code>2</code> ：动态自适应模式（Dynamic Self Adaptive），Memory Tracker 会根据系统当前的可用内存，动态调整可用内存。<br>注意：此功能为实验性功能，由于动态自适应不能做到实时监控操作系统内存使用情况，在一些大内存分配的场景，还是会有 OOM 可能。<br><code>3</code> ：关掉 Memory Tracker，Memory Tracker 将只记录内存使用情况，即使超过限额，也不会干预执行。 | 支持          |
| <code>memory_tracker_untracked_reserved_memory_mb</code> | <code>50</code>    | 保留内存的大小，单位：MB。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 支持          |
| <code>memory_tracker_detail_log</code>                   | <code>false</code> | 是否定期生成较详细的内存跟踪日志。当值为 <code>true</code> 时，会定期生成内存跟踪日志。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 支持          |
| <code>memory_tracker_detail_log_interval_ms</code>       | <code>60000</code> | 内存跟踪日志的生成时间间隔，单位：毫秒。仅当 <code>memory_tracker_detail_log</code> 为 <code>true</code> 时，该参数生效。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 支持          |
| <code>memory_purge_enabled</code>                        | <code>true</code>  | 是否定期开启内存清理功能。当值为 <code>true</code> 时，会定期清理内存。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 支持          |
| <code>memory_purge_interval_seconds</code>               | <code>10</code>    | 内存清理的时间间隔，单位：秒。 <code>memory_purge_enabled</code> 为 <code>true</code> 时，该参数生效。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 支持          |

performance optimization 配置

| 名称                      | 预设值   | 说明                                                 | 是否支持运行时动态修改 |
|-------------------------|-------|----------------------------------------------------|-------------|
| max_job_size            | 1     | 最大作业并发数，即查询执行时在可以并发执行的阶段所采用的最大线程数。建议为物理 CPU 核数的一半。 | 支持          |
| min_batch_size          | 8192  | 处理数据集的最小批处理大小。仅在 max_job_size 大于 1 时生效。            | 支持          |
| optimize_appendvertices | false | 启用后，执行 MATCH 语句时不过滤悬挂边。                            | 支持          |
| path_batch_size         | 10000 | 每个线程构建的路径数。                                        | 支持          |

最后更新: 2025年12月19日

### 5.1.4 Storage 服务配置

Storage 服务提供了两份初始配置文件 `trsgraph-storaged.conf.default` 和 `trsgraph-storaged.conf.production`，方便在不同场景中使用。文件的默认路径为 `/usr/local/trsgraph/etc/`。



- 不建议修改 `local_config` 的值为 `false`。修改配置并重启 Storage 服务，会先尝试从 Meta 服务获取缓存的配置信息，可能导致集群配置不一致，造成未知风险。
- 不建议修改文档未介绍的配置项，除非已经熟悉源代码并完全了解配置项的作用。

#### 配置文件使用方式

如需使用初始配置文件，从上述两个文件选择其一，删除后缀 `.default` 或 `.production`，Storage 服务才能将其识别为配置文件并从中获取配置信息。

#### 配置文件参数值说明

配置文件内没有设置某个参数表示参数使用的是默认值。文件内只预设了部分参数的值，而且两份初始配置文件内的参数值也略有不同，本文的预设值以 `trsgraph-storaged.conf.default` 文件为准，其中没有的参数则以 `trsgraph-storaged.conf.production` 文件为准。



配置文件中的部分参数值可以在运行时动态修改。本文将这些参数标记为支持运行时动态修改，并当 `local_config` 值设置为 `true` 时重启服务后配置会恢复为初始配置。详情参见[修改配置](#)。



Raft Listener 的配置和 Storage 服务配置不同，详情请参见[部署 Raft listener](#)。

如需查看所有的参数及其当前值，参见[配置管理](#)。

#### basics 配置

| 名称                         | 预设值                                     | 说明                                                                                                                                                    | 是否支持运行时动态修改 |
|----------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>daemonize</code>     | <code>true</code>                       | 是否启动守护进程。                                                                                                                                             | 不支持         |
| <code>pid_file</code>      | <code>pids/trsgraph-storaged.pid</code> | 记录进程 ID 的文件。                                                                                                                                          | 不支持         |
| <code>timezone_name</code> | <code>UTC+00:00:00</code>               | 指定 TRSGraph 的时区。初始配置文件中未设置该参数，如需使用请手动添加。格式请参见 <a href="#">Specifying the Time Zone with TZ</a> 。例如，东八区的设置方式为 <code>--timezone_name=UTC+08:00</code> 。 | 不支持         |
| <code>local_config</code>  | <code>true</code>                       | 是否从配置文件获取配置信息。                                                                                                                                        | 不支持         |



- 在插入时间类型的属性值时，TRSGraph 会根据 `timezone_name` 设置的时区将该时间值（TIMESTAMP 类型例外）转换成相应的 UTC 时间，因此在查询中返回的时间类型属性值为 UTC 时间。
- `timezone_name` 参数只用于转换 TRSGraph 中存储的数据，TRSGraph 进程中其它时区相关数据，例如日志打印的时间等，仍然使用主机系统默认的时区。

logging 配置

| 名称                                     | 预设值                            | 说明                                                                                                                                                                                                                                                  | 是否支持运行时动态修改 |
|----------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>log_dir</code>                   | <code>logs</code>              | 存放 Storage 服务日志的目录，建议和数据保存在不同硬盘。                                                                                                                                                                                                                    | 不支持         |
| <code>minloglevel</code>               | <code>0</code>                 | 最小日志级别，即记录此级别或更高级别的日志。可选值： <code>0</code> （INFO）、 <code>1</code> （WARNING）、 <code>2</code> （ERROR）、 <code>3</code> （FATAL）。建议在调试时设置为 <code>0</code> ，生产环境中设置为 <code>1</code> 。如果设置为 <code>4</code> ，TRSGraph 不会记录任何日志。                              | 支持          |
| <code>v</code>                         | <code>0</code>                 | VLOG 日志详细级别，即记录小于或等于此级别的所有 VLOG 消息。可选值为 <code>0</code> 、 <code>1</code> 、 <code>2</code> 、 <code>3</code> 、 <code>4</code> 、 <code>5</code> 。glog 提供的 VLOG 宏允许用户定义自己的数字日志记录级别，并用参数 <code>v</code> 控制记录哪些详细消息。详情参见 <a href="#">Verbose Logging</a> 。 | 支持          |
| <code>logbufsecs</code>                | <code>0</code>                 | 缓冲日志的最大时间，超时后输出到日志文件。 <code>0</code> 表示实时输出。单位：秒。                                                                                                                                                                                                   | 不支持         |
| <code>redirect_stdout</code>           | <code>true</code>              | 是否将标准输出和标准错误重定向到单独的输出文件。                                                                                                                                                                                                                            | 不支持         |
| <code>stdout_log_file</code>           | <code>stored-stdout.log</code> | 标准输出日志文件名称。                                                                                                                                                                                                                                         | 不支持         |
| <code>stderr_log_file</code>           | <code>stored-stderr.log</code> | 标准错误日志文件名称。                                                                                                                                                                                                                                         | 不支持         |
| <code>stderrthreshold</code>           | <code>3</code>                 | 要复制到标准错误中的最小日志级别（ <code>minloglevel</code> ）。                                                                                                                                                                                                       | 不支持         |
| <code>timestamp_in_logfile_name</code> | <code>true</code>              | 日志文件名称中是否包含时间戳。 <code>true</code> 表示包含， <code>false</code> 表示不包含。                                                                                                                                                                                   | 不支持         |

networking 配置

| 名称                                   | 预设值                         | 说明                                                                                                                                                                                                                            | 是否支持运行时动态修改 |
|--------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>meta_server_addrs</code>       | <code>127.0.0.1:9559</code> | 全部 Meta 服务的 IP（或主机名）和端口。多个 Meta 服务用英文逗号（,）分隔。                                                                                                                                                                                 | 不支持         |
| <code>local_ip</code>                | <code>127.0.0.1</code>      | Storage 服务的本地 IP（或主机名）。本地 IP 地址用于识别 trsgraph-storaged 进程，如果是分布式集群或需要远程访问，请修改为对应地址。                                                                                                                                            | 不支持         |
| <code>port</code>                    | <code>9779</code>           | Storage 服务的 RPC 守护进程监听端口。同时还会使用相邻的 <code>-1</code> （ <code>9778</code> ）和 <code>+1</code> （ <code>9780</code> ）端口。<br><code>9778</code> ：Admin 服务（Storage 接收 Meta 命令的服务）占用的端口。<br><code>9780</code> ：Storage 服务之间的 Raft 通信端口。 | 不支持         |
| <code>ws_ip</code>                   | <code>0.0.0.0</code>        | HTTP 服务的 IP 地址。                                                                                                                                                                                                               | 不支持         |
| <code>ws_http_port</code>            | <code>19779</code>          | HTTP 服务的端口。                                                                                                                                                                                                                   | 不支持         |
| <code>heartbeat_interval_secs</code> | <code>10</code>             | 默认心跳间隔。请确保所有服务的 <code>heartbeat_interval_secs</code> 取值相同，否则会导致系统无法正常工作。单位：秒。                                                                                                                                                 | 支持          |



使用 IP 时建议使用真实的 IP。否则某些情况下 `127.0.0.1/0.0.0.0` 无法正确解析。

raft 配置

| 名称                                        | 预设值                | 说明                              | 是否支持运行时动态修改 |
|-------------------------------------------|--------------------|---------------------------------|-------------|
| <code>raft_heartbeat_interval_secs</code> | <code>30</code>    | Raft 选举超时时间。单位：秒。               | 支持          |
| <code>raft_rpc_timeout_ms</code>          | <code>500</code>   | Raft 客户端的远程过程调用（RPC）超时时间。单位：毫秒。 | 支持          |
| <code>wal_ttl</code>                      | <code>14400</code> | Raft WAL 的有效时间。单位：秒。            | 支持          |

disk 配置

| 名称                                         | 预设值                                   | 说明                                                                                                                                                                                                                                                                       | 是否支持运行时动态修改 |
|--------------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>data_path</code>                     | <code>data/storage</code>             | 数据存储路径，多个路径用英文逗号 (,) 分隔。一个 RocksDB 实例对应一个路径。                                                                                                                                                                                                                             | 不支持         |
| <code>minimum_reserved_bytes</code>        | <code>268435456</code>                | 每个数据存储路径的剩余空间最小值，低于该值时，可能会导致集群数据写入失败。单位：字节。                                                                                                                                                                                                                              | 不支持         |
| <code>rocksdb_batch_size</code>            | <code>4096</code>                     | 批量操作的缓存大小。单位：字节。                                                                                                                                                                                                                                                         | 不支持         |
| <code>rocksdb_block_cache</code>           | <code>4</code>                        | BlockBasedTable 的默认块缓存大小。单位：兆字节。                                                                                                                                                                                                                                         | 不支持         |
| <code>disable_page_cache</code>            | <code>false</code>                    | 允许或禁止 TRSGraph 使用操作系统的页缓存。默认值为 <code>false</code> ，表示允许使用 page cache。当值为 <code>true</code> 时，禁止 TRSGraph 使用 page cache，此时须设置充足的块缓存 (block cache) 空间。                                                                                                                     | 不支持         |
| <code>engine_type</code>                   | <code>rocksdb</code>                  | 存储引擎类型。                                                                                                                                                                                                                                                                  | 不支持         |
| <code>rocksdb_compression</code>           | <code>lz4</code>                      | 压缩算法，可选值： <code>no</code> 、 <code>snappy</code> 、 <code>lz4</code> 、 <code>lz4hc</code> 、 <code>zlib</code> 、 <code>bzip2</code> 、 <code>zstd</code> 。<br>该参数会修改每一层的压缩算法，如需为不同层级设置不同的压缩算法，请使用 <code>rocksdb_compression_per_level</code> 参数。                             | 不支持         |
| <code>rocksdb_compression_per_level</code> | <code>-</code>                        | 为不同层级设置不同的压缩算法。优先级高于 <code>rocksdb_compression</code> 。例如<br><code>no:no:lz4:lz4:snappy:zstd:snappy</code> 。<br>也可以不设置某个层级的压缩算法，例如<br><code>no:no:lz4:lz4::zstd</code> ，此时 L4、L6 层使用 <code>rocksdb_compression</code> 参数的压缩算法。                                           | 不支持         |
| <code>enable_rocksdb_statistics</code>     | <code>false</code>                    | 是否启用 RocksDB 的数据统计。                                                                                                                                                                                                                                                      | 不支持         |
| <code>rocksdb_stats_level</code>           | <code>kExceptHistogramOrTimers</code> | RocksDB 的数据统计级别。可选值：<br><code>kExceptHistogramOrTimers</code> （禁用计时器统计，跳过柱状图统计）、 <code>kExceptTimers</code> （跳过计时器统计）、 <code>kExceptDetailedTimers</code> （收集除互斥锁和压缩花费时间之外的所有统计数据）、 <code>kExceptTimeForMutex</code> 收集除互斥锁花费时间之外的所有统计数据）、 <code>kAll</code> （收集所有统计数据）。 | 不支持         |

| 名称                                              | 预设值                | 说明                                                                                                          | 是否支持运行时动态修改 |
|-------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------|-------------|
| <code>enable_rocksdb_prefix_filtering</code>    | <code>true</code>  | 是否启用 prefix bloom filter，启用时可以提升图遍历速度，但是会增加内存消耗。                                                            | 不支持         |
| <code>enable_rocksdb_whole_key_filtering</code> | <code>false</code> | 是否启用 whole key bloom filter。                                                                                | 不支持         |
| <code>rocksdb_filtering_prefix_length</code>    | <code>12</code>    | 每个 key 的 prefix 长度。可选值： <code>12</code> （分片 ID+点 ID）、 <code>16</code> （分片 ID+点 ID+TagID/Edge typeID）。单位：字节。 | 不支持         |
| <code>enable_partitioned_index_filter</code>    | <code>false</code> | 设置为 <code>true</code> 可以降低 bloom 过滤器占用的内存大小，但是在某些随机寻道（random-seek）的情况下，可能会降低读取性能。初始配置文件中未设置该参数，如需使用请手动添加。   | 不支持         |

rocksdb options 配置

| 名称                                             | 预设值                                                                                                                     | 说明                            | 是否支持运行时动态修改 |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------|-------------|
| <code>rocksdb_db_options</code>                | <code>{}</code>                                                                                                         | RocksDB database 选项。          | 不支持         |
| <code>rocksdb_column_family_options</code>     | <code>{"write_buffer_size": "67108864", "max_write_buffer_number": "4", "max_bytes_for_level_base": "268435456"}</code> | RocksDB column family 选项。     | 不支持         |
| <code>rocksdb_block_based_table_options</code> | <code>{"block_size": "8192"}</code>                                                                                     | RocksDB block based table 选项。 | 不支持         |

rocksdb options 配置的格式为 `{"<option_name>": "<option_value>"}`，多个选项用英文逗号 (,) 隔开。



`rocksdb_db_options` 和 `rocksdb_column_family_options` 支持的选项如下：

- `rocksdb_db_options`

```
max_total_wal_size
delete_obsolete_files_period_micros
max_background_jobs
stats_dump_period_sec
compaction_readahead_size
writable_file_max_buffer_size
bytes_per_sync
wal_bytes_per_sync
delayed_write_rate
avoid_flush_during_shutdown
max_open_files
stats_persist_period_sec
stats_history_buffer_size
strict_bytes_per_sync
enable_rocksdb_prefix_filtering
enable_rocksdb_whole_key_filtering
rocksdb_filtering_prefix_length
num_compaction_threads
rate_limit
```

- `rocksdb_column_family_options`

```
write_buffer_size
max_write_buffer_number
level0_file_num_compaction_trigger
level0_slowdown_writes_trigger
level0_stop_writes_trigger
target_file_size_base
target_file_size_multiplier
max_bytes_for_level_base
max_bytes_for_level_multiplier
disable_auto_compactions
```

参数的详细说明请参见 [RocksDB 官方文档](#)。

misc 配置



下表中的 snapshot 与 TRSGraph 快照是不同的概念，这里的 snapshot 指 Raft 同步过程中 leader 上的存量数据。

| 名称                            | 预设值      | 说明                                                                                 | 是否支持运行时动态修改 |
|-------------------------------|----------|------------------------------------------------------------------------------------|-------------|
| query_concurrently            | true     | 是否开启多线程查询。开启后可以提高单个查询的时延性能，但大压力下会降低整体的吞吐量。                                         | 支持          |
| auto_remove_invalid_space     | true     | 在执行 DROP SPACE 后，会删除指定图空间，该参数设置是否同时删除指定图空间内的所有数据。当值为 true 时，同时删除指定图空间内的所有数据。       | 支持          |
| num_io_threads                | 16       | 网络 I/O 线程的数量，用于发送 RPC 请求和接收响应。                                                     | 不支持         |
| num_max_connections           | 0        | 所有网络线程的最大活动连接数，0 表示没有限制。<br>每个网络线程的最大连接数= num_max_connections / num_netio_threads。 | 不支持         |
| num_worker_threads            | 32       | Storage 的 RPC 服务的工作线程数量。                                                           | 不支持         |
| max_concurrent_subtasks       | 10       | TaskManager 并发执行子任务的最大个数。                                                          | 不支持         |
| snapshot_part_rate_limit      | 10485760 | Raft leader 向 Raft group 中其它成员同步存量数据时的限速。单位：字节/秒。                                  | 支持          |
| snapshot_batch_size           | 1048576  | Raft leader 向 Raft group 中其它成员同步存量数据时每批发送的数据量。单位：字节。                               | 支持          |
| rebuild_index_part_rate_limit | 4194304  | 重建索引过程中，Raft leader 向 Raft group 中其它成员同步索引数据时的限速。单位：字节/秒。                          | 支持          |
| rebuild_index_batch_size      | 1048576  | 重建索引过程中，Raft leader 向 Raft group 中其它成员同步索引数据时每批发送的数据量。单位：字节。                       | 支持          |

memory tracker 配置



Memory Tracker 是一个内存管理工具，用于监控和限制内存使用。对于大型查询，Memory Tracker 可以防止 OOM 问题。如果在容器环境中使用 Memory Tracker，需要在 Storage 服务的配置文件中添加相关配置。操作如下：

- 1. 创建目录 `/sys/fs/cgroup/storaged/`，添加并配置 `memory.max`。
- 2. 在 `etc/trsgraph-storaged.conf` 添加以下配置。

```
--containerized=true
--cgroup_v2_controllers=/sys/fs/cgroup/storaged/cgroup.controllers
--cgroup_v2_memory_stat_path=/sys/fs/cgroup/storaged/memory.stat
--cgroup_v2_memory_max_path=/sys/fs/cgroup/storaged/memory.max
--cgroup_v2_memory_current_path=/sys/fs/cgroup/storaged/memory.current
```

| 名称                                                       | 预设值                | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 是否支持运行时动态修改 |
|----------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <code>memory_tracker_limit_ratio</code>                  | <code>0.8</code>   | 取值可设置为： <code>{0, 1}</code> 、 <code>2</code> 、 <code>3</code> 。<br><code>{0, 1}</code> ：可用内存的百分比。计算公式：可用内存的百分比 = 可用内存 / (总内存 - 保留内存)。<br>当正在进行的查询导致内存使用超过配置的内存限制时，该查询会失败，并在失败后释放内存。<br>注意：对于云上和本地节点混合部署的集群，需要根据实际情况调小该参数。例如，当预期 Graphd 只占用 50% 的内存时，该参数的值可设置为小于 <code>0.5</code> 。<br><code>2</code> ：动态自适应模式（Dynamic Self Adaptive），Memory Tracker 会根据系统当前的可用内存，动态调整可用内存。<br>注意：此功能为实验性功能，由于动态自适应不能做到实时监控操作系统内存使用情况，在一些大内存分配的场景，还是会在存在 OOM 可能。<br><code>3</code> ：关掉 Memory Tracker，Memory Tracker 将只记录内存使用情况，即使超过限额，也不会干预执行。 | 支持          |
| <code>memory_tracker_untracked_reserved_memory_mb</code> | <code>50</code>    | 保留内存的大小，单位：MB。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 支持          |
| <code>memory_tracker_detail_log</code>                   | <code>false</code> | 是否定期生成较详细的内存跟踪日志。当值为 <code>true</code> 时，会定期生成内存跟踪日志。                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 支持          |
| <code>memory_tracker_detail_log_interval_ms</code>       | <code>60000</code> | 内存跟踪日志的生成时间间隔，单位：毫秒。仅当 <code>memory_tracker_detail_log</code> 为 <code>true</code> 时，该参数生效。                                                                                                                                                                                                                                                                                                                                                                                                                                     | 支持          |
| <code>memory_purge_enabled</code>                        | <code>true</code>  | 是否定期开启内存清理功能。当值为 <code>true</code> 时，会定期清理内存。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 支持          |
| <code>memory_purge_interval_seconds</code>               | <code>10</code>    | 内存清理的时间间隔，单位：秒。 <code>memory_purge_enabled</code> 为 <code>true</code> 时，该参数生效。                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 支持          |

超级节点处理（出入边数量极多的点）

在每个点出发的查询获取到边时，直接截断。目的是避免超级节点的邻边过多，单个查询占用过多的硬盘和内存。截取前 `max_edge_returned_per_vertex` 个边，多余的边不返回。该参数作用于全局，不用于单个 space。

| 属性名                          | 默认值        | 说明                                                | 是否支持运行时动态修改 |
|------------------------------|------------|---------------------------------------------------|-------------|
| max_edge_returned_per_vertex | 2147483647 | 每个稠密点，最多返回多少条边，多余的边截断不返回。初始配置文件中未设置该参数，如需使用请手动添加。 | 不支持         |

数据量大而内存不够时



一个图空间至少占用大约 300 MB 的内存。

如果数据量很大但内存不够，则可以尝试把 storage 配置中的 `enable_partitioned_index_filter` 设置为 `true`，减少 `rocksdb_block_cache`。但由于缓存了较少的 RocksDB 索引，性能可能会受影响。此外，当通过语句拉取大量数据时，仍存在 OOM 的可能。

最后更新: 2025年12月19日

### 5.1.5 Linux 内核配置

本文介绍与 TRSGraph 相关的 Linux 内核配置，并介绍如何修改配置。

#### 资源控制

虽然可以用 `ulimit` 进行资源控制，但是所做的更改仅对当前会话或子进程生效。如果需要永久生效，请编辑文件 `/etc/security/limits.conf`。配置如下：

```
#<domain> <type> <item> <value>
* soft core unlimited
* hard core unlimited
* soft nofile 130000
* hard nofile 130000
```



Note

配置修改后会对新的会话生效。

参数说明如下。

| 参数                  | 说明                                                                                                                                                                                           |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>domain</code> | 控制域。可以是用户名称、用户组名称（以 <code>@</code> 开头），或者用 <code>*</code> 表示所有用户。                                                                                                                            |
| <code>type</code>   | 控制类型。可以是 <code>soft</code> 或 <code>hard</code> 。 <code>soft</code> 表示资源的软阈值（默认阈值）， <code>hard</code> 表示用户可以设置的最大值。可以使用 <code>ulimit</code> 命令提高 <code>soft</code> ，但不能超过 <code>hard</code> 。 |
| <code>item</code>   | 资源类型。例如 <code>core</code> 限制核心转储文件的大小， <code>nofile</code> 限制一个进程可以打开的最大文件描述符数量。                                                                                                             |
| <code>value</code>  | 资源限制值。可以是一个数字，或者 <code>unlimited</code> 表示没有限制。                                                                                                                                              |

可以执行 `man limits.conf` 查看更多帮助信息。

#### 内存

##### VM.SWAPPINESS

`vm.swappiness` 是触发虚拟内存（swap）的空闲内存百分比。值越大，使用 swap 的可能性就越大，建议设置为 0，表示首先删除页缓存。需要注意的是，0 表示尽量不使用 swap。

##### VM.MIN\_FREE\_KBYTES

`vm.min_free_kbytes` 用于设置 Linux 内核保留的最小空闲千字节数。如果系统内存足够，建议设置较大值。例如物理内存为 128 GB，可以将 `vm.min_free_kbytes` 设置为 5 GB。如果值太小，会导致系统无法申请足够的连续物理内存。

##### VM.MAX\_MAP\_COUNT

`vm.max_map_count` 用于限制单个进程的 VMA（虚拟内存区域）数量。默认值为 65530，对于绝大多数应用程序来说已经足够。如果应用程序因为内存消耗过大而报错，请增大本参数的值。

##### VM.DIRTY\_\*

`vm.dirty_*` 是一系列控制系统脏数据缓存的参数。对于写密集型场景，用户可以根据需要进行调整（吞吐量优先或延迟优先），建议使用系统默认值。

##### 透明大页

透明大页（Transparent Huge Pages, THP）是一种 Linux 内核的内存管理特性，可以提高系统使用大页的能力。在多数数据库系统中，透明大页会降低性能，建议关闭。

操作如下：

1. 编辑 GRUB 配置文件 `/etc/default/grub`。

```
sudo vi /etc/default/grub
```

2. 在 `GRUB_CMDLINE_LINUX` 选项中添加 `transparent_hugepage=never`，然后保存并退出。

```
GRUB_CMDLINE_LINUX="... transparent_hugepage=never"
```

3. 更新 GRUB 配置。

- CentOS

```
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

- Ubuntu

```
sudo update-grub
```

4. 重启操作系统。

```
sudo reboot
```

如果不重启可临时关闭透明大页，直到下次重启。

```
echo 'never' > /sys/kernel/mm/transparent_hugepage/enabled
echo 'never' > /sys/kernel/mm/transparent_hugepage/defrag
```

## 网络

### NET.IPV4.TCP\_SLOW\_START\_AFTER\_IDLE

`net.ipv4.tcp_slow_start_after_idle` 默认值为 1，会导致闲置一段时间后堵塞窗口超时，建议设置为 0，尤其适合大带宽高延迟场景。

### NET.CORE.SOMAXCONN

`net.core.somaxconn` 用于限制 socket 监听的连接队列数量。默认值为 128。对于有大量突发连接的场景，建议设置为不低于 1024。

### NET.IPV4.TCP\_MAX\_SYN\_BACKLOG

`net.ipv4.tcp_max_syn_backlog` 用于限制处于 SYN\_RECV（半连接）状态的 TCP 连接数量。默认值为 128。对于有大量突发连接的场景，建议设置为不低于 1024。

### NET.CORE.NETDEV\_MAX\_BACKLOG

`net.core.netdev_max_backlog` 用于限制队列中数据包的数量。默认值为 1000，建议设置为 10000 以上，尤其是万兆网卡。

### NET.IPV4.TCP\_KEEPA\_LIVE\_\*

`net.ipv4.tcp_keepalive_*` 是一系列保持 TCP 连接存活的参数。对于使用四层透明负载均衡的应用程序，如果空闲连接异常断开，请增大 `tcp_keepalive_time` 和 `tcp_keepalive_intvl` 的值。

### NET.IPV4.TCP\_WMEM/RMEM

TCP 套接字发送/接收缓冲池的最小、最大、默认空间。对于大连接，建议设置为 带宽 (GB) \*往返时延 (ms)。

### SCHEDULER

对于 SSD 设备，建议将 `scheduler` 设置为 `noop` 或者 `none`，路径为 `/sys/block/DEV_NAME/queue/scheduler`。

## 其他参数

### KERNEL.CORE\_PATTERN

建议设置为 `core`，并且将 `kernel.core_uses_pid` 设置为 `1`。

## 修改参数

### SYSCTL 命令

- `sysctl <conf_name>`

查看当前参数值。

- `sysctl -w <conf_name>=<value>`

临时修改参数值，立即生效，重启后恢复原值。

- `sysctl -p [<file_path>]`

从指定配置文件里加载 Linux 系统参数，默认从 `/etc/sysctl.conf` 加载。

### PRLIMIT

命令 `prlimit` 可以获取和设置进程资源的限制，结合 `sudo` 可以修改硬阈值，例如，`prlimit --nofile=140000 --pid=$$` 调整当前进程允许的打开文件的最大数量为 `140000`，立即生效，此命令仅支持 RedHat 7u 或更高版本。

---

最后更新: 2025年12月19日

## 5.2 日志

### 5.2.1 运行日志

运行日志通常提供给 DBA 或开发人员查看，当系统出现故障，DBA 或开发人员可以根据运行日志定位问题。

TRSGraph 默认使用 `glog` 打印运行日志，使用 `gflags` 控制日志级别，并在运行时通过 HTTP 接口动态修改日志级别，方便跟踪问题。

#### 运行日志目录

运行日志的默认目录为 `/usr/local/trsgraph/logs/`。

如果在 TRSGraph 运行过程中删除运行日志目录，日志不会继续打印，但是不会影响业务。重启服务可以恢复正常。

#### 配置说明

- `minloglevel`：最小日志级别，即不会记录低于这个级别的日志。可选值为 `0`（INFO）、`1`（WARNING）、`2`（ERROR）、`3`（FATAL）。建议在调试时设置为 `0`，生产环境中设置为 `1`。如果设置为 `4`，TRSGraph 不会记录任何日志。
- `v`：日志详细级别，值越大，日志记录越详细。可选值为 `0`、`1`、`2`、`3`。

Meta 服务、Graph 服务和 Storage 服务的日志级别可以在各自的配置文件中查看，默认路径为 `/usr/local/trsgraph/etc/`。

#### 查看运行日志级别

使用如下命令查看当前所有的 `gflags` 参数（包括日志参数）：

```
$ curl <ws_ip>:<ws_port>/flags
```

| 参数                   | 说明                                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>ws_ip</code>   | HTTP 服务的 IP 地址，可以在配置文件中查看。默认值为 <code>127.0.0.1</code> 。                                                           |
| <code>ws_port</code> | HTTP 服务的端口，可以在配置文件中查看。默认值分别为 <code>19559</code> （Meta）、 <code>19669</code> （Graph）、 <code>19779</code> （Storage）。 |

示例如下：

- 查看 Meta 服务当前的最小日志级别：

```
$ curl 127.0.0.1:19559/flags | grep 'minloglevel'
```

- 查看 Storage 服务当前的日志详细级别：

```
$ curl 127.0.0.1:19779/flags | grep -w 'v'
```

#### 修改运行日志级别

使用如下命令修改运行日志级别：



```
$ curl -X PUT -H "Content-Type: application/json" -d '{"<key>":<value>[,"<key>":<value>]}' "<ws_ip>:<ws_port>/flags"
```

| 参数      | 说明                                                                    |
|---------|-----------------------------------------------------------------------|
| key     | 待修改的运行日志类型，可选值请参见配置说明。                                                |
| value   | 运行日志级别，可选值请参见配置说明。                                                    |
| ws_ip   | HTTP 服务的 IP 地址，可以在配置文件中查看。默认值为 127.0.0.1。                             |
| ws_port | HTTP 服务的端口，可以在配置文件中查看。默认值分别为 19559（Meta）、19669（Graph）、19779（Storage）。 |

示例如下：

```
$ curl -X PUT -H "Content-Type: application/json" -d '{"minloglevel":0,"v":3}' "127.0.0.1:19779/flags" # stored
$ curl -X PUT -H "Content-Type: application/json" -d '{"minloglevel":0,"v":3}' "127.0.0.1:19669/flags" # graphd
$ curl -X PUT -H "Content-Type: application/json" -d '{"minloglevel":0,"v":3}' "127.0.0.1:19559/flags" # metad
```

如果在 TRSGraph 运行时修改了运行日志级别，重启服务后会恢复为配置文件中设置的级别，如果需要永久修改，请修改配置文件中并重启服务。

RocksDB 运行日志

RocksDB 的运行日志通常在 /usr/local/trsgraph/data/storage/nebula/\$sid/data/LOG，其中 \$sid 为实例号。该日志通常用于调试 RocksDB 参数。

回收日志

glog 本身不支持回收日志，如果需要回收日志，可以使用 Linux 系统中定时任务（Cron Jobs）来定期删除旧的日志文件。或者，使用日志管理工具 logrotate 来轮转日志以定期归档和删除日志。

### 使用定时任务回收日志

本文以回收 Graph 服务的运行日志为例，说明如何使用定时任务来定期删除旧的日志文件。操作步骤如下：

1. 在 Graph 服务配置文件中，进行如下配置，然后重启服务。

```
timestamp_in_logfile_name = true
max_log_size = 500
```

- 设置 `timestamp_in_logfile_name` 为 `true`，这样日志文件名中会包含时间戳，以定期删除旧的日志文件。
- 添加 `max_log_size` 参数，设置单个日志文件的最大大小，例如 `500`。超过这个大小后，会自动创建新的日志文件，单位 MB，默认值为 `1800`。

2. 在 Linux 系统中，使用如下命令编辑定时任务：

```
crontab -e
```

3. 在定时任务中添加命令，以定期删除旧的日志文件。

```
***** find <log_path> -name "<YourProjectName>" -mtime +7 -delete
```



#### Caution

以上命令中的 `find` 命令需要使用 `root` 用户或者具有 `sudo` 权限的用户来执行。

- `*****`：定时任务的时间字段，五个星号表示这个任务每分钟都会执行。其他设置，参见 [Cron Expression](#)。
- `<log_path>`：服务运行日志文件的路径，例如 `/usr/local/trsgraph/logs`。
- `<YourProjectName>`：日志文件名，例如 `trsgraph-graphd.*`。
- `-mtime +7`：表示删除更新时间超过 7 天的日志文件。也可以使用 `-mmin +n`，表示删除更新时间超过 `n` 分钟的日志文件。详情参见 [find 命令](#)。
- `-delete`：表示删除满足条件的日志文件。

例如，每天凌晨 3 点自动删除更新时间超过 7 天的 Graph 服务运行日志文件的命令：

```
03*** find /usr/local/trsgraph/logs -name trsgraph-graphd.* -mtime +7 -delete
```

4. 保存定时任务。

### 使用 LOGROTATE 回收日志

用户可以使用 `logrotate` 工具对指定的日志文件进行轮转，以达到归档和回收日志的目的。



#### Note

需要使用 `root` 用户或者具有 `sudo` 权限的用户来安装 `logrotate` 或者运行 `logrotate`。

本文以回收 Graph 服务 `INFO` 级别的日志文件（`/usr/local/trsgraph/logs/trsgraph-graphd.INFO.impl`）为例说明如何使用 `logrotate` 工具。操作步骤如下：

1. 在 **Graph 服务配置文件** 中，将 `timestamp_in_logfile_name` 设置为 `false`，以便 `logrotate` 工具可以识别日志文件名。然后重启服务。

```
timestamp_in_logfile_name = false
```

2. 安装 `logrotate`。

- Debian/Ubuntu:

```
sudo apt-get install logrotate
```

- CentOS/RHEL:

```
sudo yum install logrotate
```

3. 创建 `logrotate` 配置文件，添加日志轮转规则，然后保存配置文件。

在 `/etc/logrotate.d` 目录下，创建一个新的 `logrotate` 配置文件 `trsgraph-graphd.INFO`

```
sudo vim /etc/logrotate.d/trsgraph-graphd.INFO
```

添加以下内容：

```
需配置日志文件的绝对路径
并且文件名不能为软链接文件，如不能为 `trsgraph-graph.INFO`
/usr/local/trsgraph/logs/trsgraph-graphd.INFO.impl {
 daily
 rotate 2
 copytruncate
 nocompress
 missingok
 notifempty
 create 644 root root
 dateext
 dateformat .%Y-%m-%d-%s
 maxsize 1k
}
```

| 参数                                   | 说明                                                                                                                                                                                 |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>daily</code>                   | 每天轮转日志。可用的时间单位有： <code>hourly</code> 、 <code>daily</code> 、 <code>weekly</code> 、 <code>monthly</code> 、 <code>yearly</code> 。                                                     |
| <code>rotate 2</code>                | 在删除前日志文件前，其被轮转的次数。即保留最近生成的 2 个日志文件。                                                                                                                                                |
| <code>copytruncate</code>            | 将当前日志文件复制一份，然后清空当前日志文件。                                                                                                                                                            |
| <code>nocompress</code>              | 不压缩旧的日志文件。                                                                                                                                                                         |
| <code>missingok</code>               | 如果日志文件丢失，不报告错误。                                                                                                                                                                    |
| <code>notifempty</code>              | 如果日志文件为空，不进行轮转。                                                                                                                                                                    |
| <code>create 644 root root</code>    | 创建新的日志文件，并设置适当的权限和所有者。                                                                                                                                                             |
| <code>dateext</code>                 | 在日志文件名中添加日期后缀。<br>默认是当前日期。默认是 <code>-%Y%m%d</code> 的后缀。可用 <code>dateformat</code> 选项扩展配置。                                                                                          |
| <code>dateformat .%Y-%m-%d-%s</code> | 必须配合 <code>dateext</code> 使用，紧跟在下一行出现，定义文件切割后的文件名。<br>在 V3.9.0 之前，只支持 <code>%Y</code> 、 <code>%m</code> 、 <code>%d</code> 、 <code>%s</code> 参数。在 V3.9.0 及之后，支持 <code>%H</code> 参数。 |
| <code>maxsize 1k</code>              | 当日志文件大小超过 1 千字节（1024 字节）或者超过设定的周期（如 <code>daily</code> ）时，进行日志轮转。可用的大小单位有：<br><code>k</code> 、 <code>M</code> ，默认单位为字节。                                                            |

用户可以根据实际需求修改配置文件中的参数。更多关于参数的配置及解释，参见 [logrotate](#)。

4. 测试 `logrotate` 配置。

为了验证 `logrotate` 的配置是否正确，可以使用以下命令来进行测试：

```
sudo logrotate --debug /etc/logrotate.d/trsgraph-graphd.INFO
```

5. 运行 logrotate。

尽管 logrotate 通常由定时作业自动执行，但也可以手动执行以下命令，以立即进行日志轮转：

```
sudo logrotate -fv /etc/logrotate.d/trsgraph-graphd.INFO
```

-fv： f 表示强制执行，v 表示打印详细信息。

6. 查看日志轮转结果。

日志轮转后，会在 /usr/local/trsgraph/logs 目录下看到新的日志文件，例如 trsgraph-graphd.INFO.impl.2024-01-04-1704338204。原始日志内容会被清空，但文件会被保留，新日志继续写入。当日志文件数量超过 rotate 设置的值时，会删除最旧的日志文件。

例如，rotate 2 表示保留最近生成的 2 个日志文件，当日志文件数量超过 2 个时，会删除最旧的日志文件。

```
[test@test logs]$ ll
-rw-r--r-- 1 root root 0 Jan 4 11:18 trsgraph-graphd.INFO.impl
-rw-r--r-- 1 root root 6894 Jan 4 11:16 trsgraph-graphd.INFO.impl.2024-01-04-1704338204 # 当新的日志文件生成时，此文件被删除
-rw-r--r-- 1 root root 222 Jan 4 11:18 trsgraph-graphd.INFO.impl.2024-01-04-1704338287
[test@test logs]$ ll
-rw-r--r-- 1 root root 0 Jan 4 11:18 trsgraph-graphd.INFO.impl
-rw-r--r-- 1 root root 222 Jan 4 11:18 trsgraph-graphd.INFO.impl.2024-01-04-1704338287
-rw-r--r-- 1 root root 222 Jan 4 11:18 trsgraph-graphd.INFO.impl.2024-01-04-1704338339 # 新生成的日志文件
```

如果用户需要对多个日志文件进行轮转，可以在 /etc/logrotate.d 目录下创建多个配置文件，每个配置文件对应一个日志文件。例如，用户需要对 Meta 服务的 INFO 级别日志文件和 WARNING 级别日志文件进行轮转，可以创建两个配置文件 trsgraph-metad.INFO 和 trsgraph-metad.WARNING，并在其中分别添加日志轮转规则。

最后更新: 2025年12月19日

## 6. 数据安全

### 6.1 验证和授权

#### 6.1.1 身份验证

身份验证用于将会话映射到特定用户，从而实现访问控制。

当客户端连接到 TRSGraph 时，TRSGraph 会创建一个会话，会话中存储连接的各种信息，如果开启了身份验证，就会将会话映射到对应的用户。

##### 本地身份验证

本地身份验证是指在服务器本地存储用户名、加密密码，当用户尝试访问 TRSGraph 时，将进行身份验证。

启用本地身份验证

1. 编辑配置文件 `trsgraph-graphd.conf`（默认目录为 `/usr/local/trsgraph/etc/`），设置如下参数：

- `--enable_authorize`：是否启用身份验证，可选值：`true`、`false`。



Note

- 默认情况下，身份验证功能是关闭的，用户可以使用 `root` 用户名和任意密码连接到 TRSGraph。
- 开启身份验证后，默认的 God 角色账号为 `root`，密码为 `trsadmin`。角色详情请参见[内置角色权限](#)。
- `--failed_login_attempts`：可选项，需要手动添加该参数。单个 Graph 节点允许连续输入错误密码的次数。超过该次数时，账户会被锁定。如果有多个 Graph 节点，允许的次数为 `节点数 * 次数`。
- `--password_lock_time_in_secs`：可选项，需要手动添加该参数。多次输入错误密码后，账户被锁定的时间。单位：秒。

2. 重启 TRSGraph 服务。

最后更新: 2025年12月19日

## 6.1.2 用户管理

用户管理是 TRSGraph 访问控制中不可或缺的组成部分，本文将介绍用户管理的相关语法。

开启身份验证后，用户需要使用已创建的用户才能连接 TRSGraph，而且连接后可以进行的操作也取决于该用户拥有的角色权限。



- 默认情况下，身份验证功能是关闭的，用户可以使用 root 用户名和任意密码连接到 TRSGraph。
- 修改权限后，对应的用户需要重新登录才能生效。

### 创建用户 (CREATE USER)

执行 CREATE USER 语句可以创建新的 TRSGraph 用户。当前仅 God 角色用户（即 root 用户）能够执行 CREATE USER 语句。

- 语法

```
CREATE USER [IF NOT EXISTS] <user_name> [WITH PASSWORD '<password>'];
```

- IF NOT EXISTS：检测待创建的用户名是否存在，只有不存在时，才会创建新用户。
- user\_name：待创建的用户名。长度最多为 16 个字符。
- password：用户名对应的密码。默认密码为空字符串（''）。长度最多为 24 个字符。

- 示例

```
trsgraph> CREATE USER user1 WITH PASSWORD 'trsadmin';
trsgraph> SHOW USERS;
+-----+
| Account |
+-----+
| "root" |
| "user1" |
+-----+
```

### 授权用户 (GRANT ROLE)

执行 GRANT ROLE 语句可以将指定图空间的内置角色权限授予用户。当前仅 God 角色用户和 Admin 角色用户能够执行 GRANT ROLE 语句。角色权限的说明，请参见内置角色权限。

- 语法

```
GRANT ROLE <role_type> ON <space_name> TO <user_name>;
```

- 示例

```
trsgraph> GRANT ROLE USER ON basketballplayer TO user1;
```

### 撤销用户权限（REVOKE ROLE）

执行 `REVOKE ROLE` 语句可以撤销用户的指定图空间的内置角色权限。当前仅 God 角色用户和 Admin 角色用户能够执行 `REVOKE ROLE` 语句。角色权限的说明，请参见[内置角色权限](#)。

#### • 语法

```
REVOKE ROLE <role_type> ON <space_name> FROM <user_name>;
```

#### • 示例

```
trsgraph> REVOKE ROLE USER ON basketballplayer FROM user1;
```

### 查看指定用户权限（DESCRIBE USER）

执行 `DESCRIBE USER` 语句可以查看指定用户的角色权限信息。

#### • 语法

```
DESCRIBE USER <user_name>;
DESC USER <user_name>;
```

#### • 示例

```
trsgraph> DESCRIBE USER user1;
+-----+-----+
| role | space |
+-----+-----+
| "ADMIN" | "basketballplayer" |
+-----+-----+
```

### 查看指定空间内用户权限（SHOW ROLES）

执行 `SHOW ROLES` 语句可以查看指定空间内的所有用户（除 `root` 以外）和对应角色权限的信息。

#### • 语法

```
SHOW ROLES IN <space_name>;
```

#### • 示例

```
trsgraph> SHOW ROLES IN basketballplayer;
+-----+-----+
| Account | Role Type |
+-----+-----+
| "user1" | "ADMIN" |
+-----+-----+
```

### 修改用户密码（CHANGE PASSWORD）

执行 `CHANGE PASSWORD` 语句可以修改用户密码，修改时需提供旧密码和新密码。

#### • 语法

```
CHANGE PASSWORD <user_name> FROM '<old_password>' TO '<new_password>';
```

#### • 示例

```
trsgraph> CHANGE PASSWORD user1 FROM 'trsgraph' TO 'trsgraph123';
```



### 修改用户密码（ALTER USER）

执行 `ALTER USER` 语句可以修改用户密码，修改时不需要提供旧密码。当前仅 God 角色用户（即 `root` 用户）能够执行 `ALTER USER` 语句。

#### • 语法

```
ALTER USER <user_name> WITH PASSWORD '<password>';
```

#### • 示例

```
trsgraph> ALTER USER user2 WITH PASSWORD 'change_password';
trsgraph> SHOW USERS;
+-----+
| Account |
+-----+
| "root" |
| "user1" |
| "user2" |
+-----+
```

### 删除用户（DROP USER）

执行 `DROP USER` 语句可以删除用户。当前仅 God 角色用户能够执行 `DROP USER` 语句。



Note

删除用户不会自动断开该用户当前会话，而且权限仍在当前会话中生效。

#### • 语法

```
DROP USER [IF EXISTS] <user_name>;
```

#### • 示例

```
trsgraph> DROP USER user1;
```

### 查看用户列表（SHOW USERS）

执行 `SHOW USERS` 语句可以查看用户列表。当前仅 God 角色用户能够执行 `SHOW USERS` 语句。

#### • 语法

```
SHOW USERS;
```

#### • 示例

```
trsgraph> SHOW USERS;
+-----+-----+
| Account | IP Whitelist |
+-----+-----+
"root"	""
"user1"	""
"user2"	"192.168.10.10"
+-----+-----+
```

最后更新: 2025年12月19日

### 6.1.3 内置角色权限

所谓角色，就是一组相关权限的集合。用户可以把角色分配给创建的用户，从而实现访问控制。

#### 内置角色

TRSGraph 内置了多种角色，说明如下：

- God
- 初始最高权限角色，拥有所有操作的权限。类似于 Linux 中的 `root` 和 Windows 中的 `administrator`。
- Meta 服务初始化时，会自动创建 God 角色用户 `root`，密码为 `trsadmin`。



请及时修改 `root` 用户的密码，保证数据安全。

- 在 `trsgraph-graphd.conf` 文件中（默认目录为 `/usr/local/trsgraph/etc/`），当 `--enable_authorize` 为 `true` 时：
- 一个集群只能有一个 God 角色用户，该用户可以管理集群内所有图空间。
- 不支持手动授权 God 角色，只能使用默认 God 角色用户 `root`。

- Admin
- 对权限内的图空间拥有 Schema 和 data 的读写权限。
- 可以将权限内的图空间授权给其他用户。



只能授权低于 ADMIN 级别的角色给其他用户。

- DBA
- 对权限内的图空间拥有 Schema 和 data 的读写权限。
- 无法将权限内的图空间授权给其他用户。

- User
- 对权限内的图空间拥有 Schema 的只读权限。
- 对权限内的图空间拥有 data 的读写权限。

- Guest
- 对权限内的图空间拥有 Schema 和 data 的只读权限。



- 不支持自行创建角色，只能使用默认的内置角色。
- 一个用户在一个图空间内只能拥有一个角色权限。授权用户请参见[用户管理](#)。

角色权限

各角色的执行权限如下。

| 权限              | God | Admin | DBA | User | Guest | 相关语句                                                                                                                                        |
|-----------------|-----|-------|-----|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Read space      | Y   | Y     | Y   | Y    | Y     | USE、DESCRIBE SPACE                                                                                                                          |
| Read schema     | Y   | Y     | Y   | Y    | Y     | DESCRIBE TAG、DESCRIBE EDGE、DESCRIBE TAG INDEX、DESCRIBE EDGE INDEX                                                                           |
| Write schema    | Y   | Y     | Y   |      |       | CREATE TAG、ALTER TAG、CREATE EDGE、ALTER EDGE、DROP TAG、DELETE TAG、DROP EDGE、CREATE TAG INDEX、CREATE EDGE INDEX、DROP TAG INDEX、DROP EDGE INDEX |
| Write user      | Y   |       |     |      |       | CREATE USER、DROP USER、ALTER USER                                                                                                            |
| Write role      | Y   | Y     |     |      |       | GRANT、REVOKE                                                                                                                                |
| Read data       | Y   | Y     | Y   | Y    | Y     | GO、SET、PIPE、MATCH、ASSIGNMENT、LOOKUP、YIELD、ORDER BY、FETCH VERTICES、Find、FETCH EDGES、FIND PATH、LIMIT、GROUP BY、RETURN                          |
| Write data      | Y   | Y     | Y   | Y    |       | INSERT VERTEX、UPDATE VERTEX、INSERT EDGE、UPDATE EDGE、DELETE VERTEX、DELETE EDGES、DELETE TAG                                                   |
| Show operations | Y   | Y     | Y   | Y    | Y     | SHOW、CHANGE PASSWORD                                                                                                                        |
| Job             | Y   | Y     | Y   | Y    |       | SUBMIT JOB COMPACT、SUBMIT JOB FLUSH、SUBMIT JOB STATS、STOP JOB、RECOVER JOB、BUILD TAG INDEX、BUILD EDGE INDEX、INGEST、DOWNLOAD                  |
| Write space     | Y   |       |     |      |       | CREATE SPACE、DROP SPACE、CREATE SNAPSHOT、DROP SNAPSHOT、BALANCE、CONFIG                                                                        |



Caution

Show operations 为特殊操作，只会在自身权限内执行。例如 SHOW SPACES，每个角色都可以执行，但是只会返回自身权限内的图空间。只有 God 角色可以执行 SHOW USERS 和 SHOW SNAPSHOTS 语句。

最后更新: 2025年12月19日

## 6.2 SSL 加密

TRSGraph 支持在客户端、Graph 服务、Meta 服务和 Storage 服务之间进行 SSL 加密传输，本文介绍如何设置 SSL 加密。

### 6.2.1 注意事项

开启 SSL 加密会轻微影响性能。

### 6.2.2 参数介绍

| 参数                            | 默认值                | 说明                       |
|-------------------------------|--------------------|--------------------------|
| <code>cert_path</code>        | -                  | PEM 证书的路径。               |
| <code>key_path</code>         | -                  | 密钥证书的路径。                 |
| <code>password_path</code>    | -                  | 密码文件证书的路径。               |
| <code>ca_path</code>          | -                  | 受信任 CA 证书文件的路径。          |
| <code>enable_ssl</code>       | <code>false</code> | 是否开启 SSL 加密。             |
| <code>enable_graph_ssl</code> | <code>false</code> | 是否仅在 Graph 服务上开启 SSL 加密。 |
| <code>enable_meta_ssl</code>  | <code>false</code> | 是否仅在 Meta 服务上开启 SSL 加密。  |

### 6.2.3 证书模式

为了使用 SSL 加密，必须有 SSL 证书。TRSGraph 支持两种证书模式：

- 自签名证书模式  
需要自行制作签名证书。需要根据加密策略，在对应的配置文件内设置 `cert_path`、`key_path` 和 `password_path`。
- CA 签名证书模式  
需要在认证机构（Certificate Authority）申请签名证书。需要根据加密策略，在对应的配置文件内设置 `cert_path`、`key_path` 和 `ca_path`。

### 6.2.4 加密策略

TRSGraph 支持三种加密策略。

- 对客户端、Graph 服务、Meta 服务和 Storage 服务之间的传输数据加密。  
需要修改 `trsgraph-graphd.conf`、`trsgraph-metad.conf` 和 `trsgraph-storaged.conf` 配置文件，设置 `enable_ssl = true`。
- 对客户端和 Graph 服务之间的传输数据加密。  
适用于集群设置在同一个机房内，仅对外开放 Graph 服务的端口。因为其他服务可以通过内部网络通信，无需加密。需要修改 `trsgraph-graphd.conf` 配置文件，设置 `enable_graph_ssl = true`。
- 对集群中 Meta 服务相关的传输数据加密。  
适用于向 meta 服务传输需保密的信息。需要修改 `trsgraph-graphd.conf`、`trsgraph-metad.conf` 和 `trsgraph-storaged.conf` 配置文件，设置 `enable_meta_ssl = true`。

## 6.2.5 使用方式

---

1. 确认证书模式和加密策略。
2. 在对应的配置文件内增加证书配置和策略配置。

例如使用自签名证书，并对客户端、Graph 服务、Meta 服务和 Storage 服务之间的数据传输进行加密。需要对三个配置文件都进行如下设置：

```
--cert_path=xxxxxx
--key_path=xxxxxx
--password_path=xxxxxx
--enable_ssl=true
```

3. 客户端设置安全套接字并添加受信任的 CA。

---

最后更新: 2025年12月19日

## 7. 同步与迁移

### 7.1 负载均衡

我们可以提交任务让 TRSGraph 的 Storage 服务实现负载均衡。详细示例请参见 [Storage 负载均衡](#)。



Note

其他作业管理命令请参见[作业管理](#)。

负载均衡相关的语法说明如下。

| 语法                                     | 说明                                                          |
|----------------------------------------|-------------------------------------------------------------|
| <code>SUBMIT JOB BALANCE LEADER</code> | 启动任务均衡分布所有图空间中的 leader。该命令会返回任务 ID ( <code>job_id</code> )。 |

查看、停止、重启任务，请参见[作业管理](#)。

最后更新: 2025年12月19日

## 8. 导入与导出

---

### 8.1 导入导出工具概述

---

#### 8.1.1 导入工具

---

有多种方式可以将数据写入TRSGraph 10.0:

- 使用命令行 `-f` 的方式导入：可以导入少量准备好的 nGQL 文件，适合少量手工测试数据准备。
- 使用 Studio 导入：可以用过浏览器导入本机多个 CSV 文件，格式有限制。
- 使用 Importer 导入：导入单机多个 CSV 文件，大小没有限制，格式灵活。
- 使用 C++/GO/Java/Python SDK：编写程序的方式导入，需要有一定编程和调优能力。

---

最后更新: 2025年12月19日

## 8.2 TRSGraph Importer

TRSGraph Importer（简称 Importer）是一款 TRSGraph 的 CSV 文件单机导入工具，可以读取并批量导入多种数据源的 CSV 文件数据，还支持批量更新和删除操作。

### 8.2.1 功能

- 支持多种数据源，包括本地、S3、OSS、HDFS、FTP、SFTP、GCS。
- 支持导入 CSV 格式文件的数据。单个文件内可以包含多种 Tag、多种 Edge type 或者二者混合的数据。
- 支持过滤数据源数据。
- 支持批量操作，包括导入、更新、删除。
- 支持同时连接多个 Graph 服务进行导入并且动态负载均衡。
- 支持失败后重连、重试。
- 支持多维度显示统计信息，包括导入时间、导入百分比等。统计信息支持打印在 Console 或日志中。
- 支持 SSL 加密。

### 8.2.2 优势

- 轻量快捷：不需要复杂环境即可使用，快速导入数据。
- 灵活筛选：通过配置文件可以实现对 CSV 文件数据的灵活筛选。

### 8.2.3 版本兼容性

TRSGraph Importer 版本和 TRSGraph 内核的版本对应关系如下。

| TRSGraph 版本 | TRSGraph Importer 版本 |
|-------------|----------------------|
| 3.x.x       | 3.x.x、4.x.x          |
| 2.x.x       | 2.x.x、3.x.x          |



Importer 4.0.0 对 Importer 进行了重做，性能得到了提高，但配置文件不兼容旧版本。建议使用新版 Importer。

### 8.2.4 前提条件

在使用 TRSGraph Importer 之前，请确保：

- 已部署 TRSGraph 服务。
- TRSGraph 中已创建 Schema，包括图空间、Tag 和 Edge type，或者通过参数 `manager.hooks.before.statements` 设置。

### 8.2.5 操作步骤

#### 创建 CSV 文件

准备好待导入的 CSV 文件并配置 YAML 文件，即可使用本工具向 TRSGraph 批量导入数据。





YAML 配置文件说明请参见下文的[配置文件说明](#)。

### 下载二进制包运行

在 `trsgraph-importer` 的安装目录下，执行以下命令导入数据。

```
$./<binary_file_name> --config <yaml_config_file_path>
```

## 8.2.6 配置文件说明

TRSGraph Importer 的配置文件用来描述待导入文件信息、TRSGraph 服务器信息等。下文将分类介绍配置文件内的字段。

### Client 配置

Client 配置存储客户端连接 TRSGraph 相关的配置。

示例配置如下：

```
client:
 version: v3
 address: "192.168.1.100:9669,192.168.1.101:9669"
 user: root
 password: trsadmin
 ssl:
 enable: true
 certPath: "/home/xxx/cert/importer.crt"
 keyPath: "/home/xxx/cert/importer.key"
 caPath: "/home/xxx/cert/root.crt"
 insecureSkipVerify: false
 concurrencyPerAddress: 10
 reconnectInitialInterval: 1s
```

```
retry: 3
retryInitialInterval: 1s
```

| 参数                              | 默认值              | 是否必须 | 说明                                                 |
|---------------------------------|------------------|------|----------------------------------------------------|
| client.version                  | v3               | 是    | 指定连接的 TRSGraph 的大版本。当前仅支持 v3。                      |
| client.address                  | "127.0.0.1:9669" | 是    | 指定连接的 TRSGraph 地址。多个地址用英文逗号 (,) 分隔。                |
| client.user                     | root             | 否    | TRSGraph 的用户名。                                     |
| client.password                 | trsadmin         | 否    | TRSGraph 用户名对应的密码。                                 |
| client.ssl.enable               | false            | 否    | 指定是否开启 SSL 认证。                                     |
| client.ssl.certPath             | -                | 否    | 指定 SSL 公钥证书的存储路径。开启 SSL 认证后该参数必填。                  |
| client.ssl.keyPath              | -                | 否    | 指定 SSL 密钥的存储路径。开启 SSL 认证后该参数必填。                    |
| client.ssl.caPath               | -                | 否    | 指定 CA 根证书的存储路径。开启 SSL 认证后该参数必填。                    |
| client.ssl.insecureSkipVerify   | false            | 否    | 指定是否跳过验证服务端的证书链和主机名。如果设置为 true，则接受服务端提供的任何证书链和主机名。 |
| client.concurrencyPerAddress    | 10               | 否    | 单个 Graph 服务的客户端并发连接数。                              |
| client.reconnectInitialInterval | 1s               | 否    | 重连间隔时间。                                            |
| client.retry                    | 3                | 否    | nGQL 语句执行失败的重试次数。                                  |
| client.retryInitialInterval     | 1s               | 否    | 重试间隔时间。                                            |

Manager 配置

Manager 配置是连接数据库后的人为控制配置。

示例配置如下：

```
manager:
 spaceName: basic_string_examples
 batch: 128
 readerConcurrency: 50
 importerConcurrency: 512
 statsInterval: 10s
 hooks:
 before:
 - statements:
 - |
 DROP SPACE IF EXISTS basic_string_examples;
 CREATE SPACE IF NOT EXISTS basic_string_examples(partition_num=5, replica_factor=1, vid_type=int);
 USE basic_string_examples;
 wait: 10s
 after:
 - statements:
```

```
-|
SHOW SPACES;
```

| 参数                                                   | 默认值 | 是否必须 | 说明                                                                 |
|------------------------------------------------------|-----|------|--------------------------------------------------------------------|
| <code>manager.spaceName</code>                       | -   | 是    | 指定数据要导入的图空间。不支持同时导入多个图空间。                                          |
| <code>manager.batch</code>                           | 128 | 否    | 执行语句的批处理量（全局配置）。对某个数据源单独设置批处理量可以使用下文的 <code>sources.batch</code> 。 |
| <code>manager.readerConcurrency</code>               | 50  | 否    | 读取器读取数据源的并发数。                                                      |
| <code>manager.importerConcurrency</code>             | 512 | 否    | 生成待执行的 nGQL 语句的并发数，然后会调用客户端执行这些语句。                                 |
| <code>manager.statsInterval</code>                   | 10s | 否    | 打印统计信息的时间间隔。                                                       |
| <code>manager.hooks.before.<br/>[].statements</code> | -   | 否    | 导入前在图空间内执行的命令。                                                     |
| <code>manager.hooks.before.[].wait</code>            | -   | 否    | 执行 <code>statements</code> 语句后的等待时间。                               |
| <code>manager.hooks.after.<br/>[].statements</code>  | -   | 否    | 导入后在图空间内执行的命令。                                                     |
| <code>manager.hooks.after.[].wait</code>             | -   | 否    | 执行 <code>statements</code> 语句后的等待时间。                               |

Log 配置

Log 配置是设置日志相关配置。

示例配置如下：

```
log:
 level: INFO
 console: true
 files:
 - logs/importer.log
```

| 参数                       | 默认值  | 是否必须 | 说明                                                                                                                                    |
|--------------------------|------|------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>log.level</code>   | INFO | 否    | 日志级别。可选值为 <code>DEBUG</code> 、 <code>INFO</code> 、 <code>WARN</code> 、 <code>ERROR</code> 、 <code>PANIC</code> 、 <code>FATAL</code> 。 |
| <code>log.console</code> | true | 否    | 存储日志时是否将日志同步打印到 Console。                                                                                                              |
| <code>log.files</code>   | -    | 否    | 日志文件路径。需手动创建日志文件目录。                                                                                                                   |

Source 配置

Source 配置中需要配置数据源信息、数据处理方式和 Schema 映射。

示例配置如下：

```
sources:
 - path: ./person.csv # 指定存储数据文件的路径。如果使用相对路径，则路径和当前配置文件目录拼接。也支持通配符文件名，例如：./follower-*.csv，请确保所有匹配的文件具有相同的架构。
 # - s3: # AWS S3
 # endpoint: endpoint # 可选。S3 服务端点，如果使用 AWS S3 可以省略。
 # region: us-east-1 # 必填。S3 服务的区域。
 # bucket: gdelt-open-data # 必填。S3 服务中的 bucket。
 # key: events/20190918.export.csv # 必填。S3 服务中文件的 key。
 # accessKeyId: "" # 可选。S3 服务的访问密钥。如果是公共数据，则无需配置。
 # accessKeySecret: "" # 可选。S3 服务的密钥。如果是公共数据，则无需配置。
 # - oss:
 # endpoint: https://oss-cn-hangzhou.aliyuncs.com # 必填。OSS 服务端点。
 # bucket: bucketName # 必填。OSS 服务中的 bucket。
 # key: objectKey # 必填。OSS 服务中文件的 object key。
 # accessKeyId: accessKey # 必填。OSS 服务的访问密钥。
 # accessKeySecret: secretKey # 必填。OSS 服务的密钥。
 # - ftp:
```

```

host: 192.168.0.10 # 必填。FTP 服务的主机。
port: 21 # 必填。FTP 服务的端口。
user: user # 必填。FTP 服务的用户名。
password: password # 必填。FTP 服务的密码。
path: "/events/20190918.export.csv" # FTP 服务中文件的路径。
- sftp:
host: 192.168.0.10 # 必填。SFTP 服务的主机。
port: 22 # 必填。SFTP 服务的端口。
user: user # 必填。SFTP 服务的用户名。
password: password # 可选。SFTP 服务的密码。
keyFile: keyFile # 可选。SFTP 服务的 SSH 密钥文件路径。
keyData: keyData $ 可选。SFTP 服务的 SSH 密钥文件内容。
passphrase: passphrase # 可选。SFTP 服务的 SSH 密钥密码。
path: "/events/20190918.export.csv" # 必填。SFTP 服务中文件的路径。
- hdfs:
address: "127.0.0.1:8020" # 必填。HDFS 服务的地址。
user: "hdfs" # 可选。HDFS 服务的用户名。
servicePrincipalName: <Kerberos Service Principal Name> # 可选。启用 Kerberos 认证时，HDFS 服务的 Kerberos 服务实例名称。
krb5ConfigFile: <Kerberos config file> # 可选。启用 Kerberos 认证时，HDFS 服务的 Kerberos 配置文件路径，默认为 `/etc/krb5.conf`。
ccacheFile: <Kerberos ccache file> # 可选。启用 Kerberos 认证时，HDFS 服务的 Kerberos ccache 文件路径。
keyTabFile: <Kerberos keytab file> # 可选。启用 Kerberos 认证时，HDFS 服务的 Kerberos keytab 文件路径。
password: <Kerberos password> # 可选。启用 Kerberos 认证时，HDFS 服务的 Kerberos 密码。
dataTransferProtection: <Kerberos Data Transfer Protection> # 可选。启用 Kerberos 认证时的传输加密类型。可选值为 `authentication`、`integrity`、`privacy`。
disablePAFXFAST: false # 可选。是否禁止客户端使用预身份验证（PA_FX_FAST）。
path: "/events/20190918.export.csv" # 必填。HDFS 服务中文件的路径。也支持通配符文件名，例如：/events/*.export.csv，请确保所有匹配的文件具有相同的架构。
- gcs: # Google Cloud Storage
bucket: chicago-crime-sample # 必填。GCS 服务中的 bucket 名称。
key: stats/000000000000.csv # 必填。GCS 服务中文件的路径。
withoutAuthentication: false # 可选。是否匿名访问。默认为 false，即使用凭证访问。
使用凭证访问时，credentialsFile 和 credentialsJSON 参数二选一即可。
credentialsFile: "/path/to/your/credentials/file" # 可选。GCS 服务的凭证文件路径。
credentialsJSON: '{ # 可选。GCS 服务的凭证 JSON 内容。
"type": "service_account",
"project_id": "your-project-id",
"private_key_id": "key-id",
"private_key": "-----BEGIN PRIVATE KEY-----\nxxxxx\n-----END PRIVATE KEY-----\n",
"client_email": "your-client@your-project-id.iam.gserviceaccount.com",
"client_id": "client-id",
"auth_uri": "https://accounts.google.com/o/oauth2/auth",
"token_uri": "https://oauth2.googleapis.com/token",
"auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
"client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/your-client%40your-project-id.iam.gserviceaccount.com",
"universe_domain": "googleapis.com"
}'

```

batch: 256

csv:

delimiter: "|"  
withHeader: false  
lazyQuotes: false

tags:

- name: Person

# mode: INSERT

# filter:

# expr: Record[1] == "XXX"

id:

type: "STRING"  
function: "hash"

# index: 0

concatItems:

- person\_

- 0

- \_id

props:

- name: "firstName"

type: "STRING"

index: 1

- name: "lastName"

type: "STRING"

index: 2

- name: "gender"

type: "STRING"

index: 3

nullable: true

defaultValue: female

- name: "birthday"

type: "DATE"

index: 4

nullable: true

nullValue: \_NULL\_

- name: "creationDate"

type: "DATETIME"

index: 5

- name: "locationIP"

```
 type: "STRING"
 index: 6
 - name: "browserUsed"
 type: "STRING"
 index: 7
- path: ./knows.csv
batch: 256
edges:
- name: KNOWS # person_knows_person
mode: INSERT
filter:
expr: Record[1] == "XXX"
src:
 id:
 type: "STRING"
 concatItems:
 - person_
 - 0
 - _id
dst:
 id:
 type: "STRING"
 concatItems:
 - person_
 - 1
 - _id
props:
- name: "creationDate"
 type: "DATETIME"
 index: 2
 nullable: true
 nullValue: _NULL_
 defaultValue: 0000-00-00T00:00:00
```

配置主要包括以下几个部分：

- 指定数据源信息。
- 指定执行语句的批处理量。
- 指定 CSV 文件格式信息。
- 指定 Tag 的模式映射。
- 指定 Edge type 的模式映射。

| 参数                                                                                                                                                      | 默认值    | 是否必须 | 说明                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sources.path</code> <code>sources.s3</code> <code>sources.oss</code> <code>sources.ftp</code> <code>sources.sftp</code> <code>sources.hdfs</code> | -      | 否    | 指定数据源信息，例如本地文件、HDFS、S3。                                                                                                                                                                            |
| <code>sources.batch</code>                                                                                                                              | 256    | 否    | 导入该数据源时执行语句的批处理量。优先级低。                                                                                                                                                                             |
| <code>sources.csv.delimiter</code>                                                                                                                      | ,      | 否    | CSV 文件的分隔符。仅支持 1 个字符的字符串。支持转义为：" <code>\x03</code> " 或 " <code>\u0003</code> "。关于 yaml 格式转义，请参考 <a href="#">YAML 转义</a> 。                                                                          |
| <code>sources.csv.withHeader</code>                                                                                                                     | false  | 否    | 是否忽略 CSV 文件中的第一条记录。                                                                                                                                                                                |
| <code>sources.csv.lazyQuotes</code>                                                                                                                     | false  | 否    | 是否允许惰性解析引号。如果值为 <code>true</code> ，引号内的内容不会被解析。                                                                                                                                                    |
| <code>sources.tags.name</code>                                                                                                                          | -      | 是    | Tag 名称。                                                                                                                                                                                            |
| <code>sources.tags.mode</code>                                                                                                                          | INSERT | 否    | 批量操作类型，包括导入、更新和删除。可选 <code>INSERT</code> 、 <code>UPDATE</code> 或 <code>DELETE</code> 。                                                                                                             |
| <code>sources.tags.filter.expr</code>                                                                                                                   | -      | 否    | 过滤数据，满足过滤条件的才会导入。支持的表达式为： <code>Record[0] == "Mahinda" or Record[0] == "Michael"</code> 或 <code>Record[0] &lt; "Michael"</code> 。                                                                  |
| <code>sources.tags.id.type</code>                                                                                                                       | STRING | 否    | VID 的类型。                                                                                                                                                                                           |
| <code>sources.tags.id.function</code>                                                                                                                   | -      | 否    | 生成 VID 的函数。目前仅支持 <code>hash</code> 。                                                                                                                                                               |
| <code>sources.tags.id.index</code>                                                                                                                      | -      | 否    | VID 对应的数据文件中的列号。如果未配置 <code>sources.tags.id.index</code> ，则使用 <code>sources.tags.id.type</code> 指定的类型。                                                                                             |
| <code>sources.tags.id.concatItems</code>                                                                                                                | -      | 否    | 用于连接两个或多个数组，连接项可以是 <code>string</code> 、 <code>array</code> 或 <code>object</code> 。                                                                                                                |
| <code>sources.tags.ignoreExistedIndex</code>                                                                                                            | true   | 否    | 是否启用 <code>IGNORE_EXISTED_INDEX</code> ，即插入点冲突时忽略。                                                                                                                                                 |
| <code>sources.tags.props.name</code>                                                                                                                    | -      | 是    | VID 上的属性名称，必须与数据库中的属性名称一致。                                                                                                                                                                         |
| <code>sources.tags.props.type</code>                                                                                                                    | STRING | 否    | VID 上属性的数据类型。目前支持 <code>BOOL</code> 、 <code>INT</code> 、 <code>STRING</code> 、 <code>ARRAY</code> 、 <code>OBJECT</code> 、 <code>GEOGRAPHY (LINESTRING)</code> 、 <code>GEOGRAPHY (POLYGON)</code> 。 |
| <code>sources.tags.props.index</code>                                                                                                                   | -      | 是    | 属性值对应的数据文件中的列号。                                                                                                                                                                                    |
| <code>sources.tags.props.nullable</code>                                                                                                                | false  | 否    | 属性是否可以 NULL，可选 <code>true</code> 或者 <code>false</code> 。                                                                                                                                           |
| <code>sources.tags.props.nullValue</code>                                                                                                               | -      | 否    | <code>nullable</code> 设置为 <code>true</code> 时，属性的值与 <code>nullValue</code> 一致。                                                                                                                     |
| <code>sources.tags.props.alternativeIndices</code>                                                                                                      | -      | 否    | 当 <code>nullable</code> 为 <code>false</code> 时忽略。该属性根据索引值进行匹配。                                                                                                                                     |
| <code>sources.tags.props.defaultValue</code>                                                                                                            | -      | 否    | 当 <code>nullable</code> 为 <code>false</code> 时忽略。根据 <code>index</code> 和 <code>alternativeIndices</code> 进行匹配。                                                                                     |
| <code>sources.edges.name</code>                                                                                                                         | -      | 是    | Edge type 名称。                                                                                                                                                                                      |
| <code>sources.edges.mode</code>                                                                                                                         | INSERT | 否    | 批量操作类型，包括导入、更新和删除。可选 <code>INSERT</code> 、 <code>UPDATE</code> 或 <code>DELETE</code> 。                                                                                                             |
| <code>sources.edges.filter.expr</code>                                                                                                                  | -      | 否    | 过滤数据，满足过滤条件的才会导入。支持的表达式为： <code>Record[0] == "Mahinda" or Record[0] == "Michael"</code> 或 <code>Record[0] &lt; "Michael"</code> 。                                                                  |
| <code>sources.edges.src.id.type</code>                                                                                                                  | STRING | 否    | 边上起点 VID 的数据类型。                                                                                                                                                                                    |
| <code>sources.edges.src.id.index</code>                                                                                                                 | -      | 是    | 边上起点 VID 对应的数据文件中的列号。                                                                                                                                                                              |
| <code>sources.edges.dst.id.type</code>                                                                                                                  | STRING | 否    | 边上终点 VID 的数据类型。                                                                                                                                                                                    |
| <code>sources.edges.dst.id.index</code>                                                                                                                 | -      | 是    | 边上终点 VID 对应的数据文件中的列号。                                                                                                                                                                              |
| <code>sources.edges.rank.index</code>                                                                                                                   | -      | 否    | 边上 RANK 对应的数据文件中的列号。                                                                                                                                                                               |
| <code>sources.edges.ignoreExistedIndex</code>                                                                                                           | true   | 否    | 是否启用 <code>IGNORE_EXISTED_INDEX</code> ，即插入点冲突时忽略。                                                                                                                                                 |
| <code>sources.edges.props.name</code>                                                                                                                   | -      | 否    | 边上属性的名称，必须与数据库中的属性名称一致。                                                                                                                                                                            |

| 参数                                            | 默认值                 | 是否必须 | 说明                                                                                                                             |
|-----------------------------------------------|---------------------|------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>sources.edges.props.type</code>         | <code>STRING</code> | 否    | 边上属性的数据类型。目前支持 <code>BOOL</code> 、 <code>INT</code> 、 <code>GEOGRAPHY (LINESTRING)</code> 、 <code>GEOGRAPHY (POLYGON)</code> 。 |
| <code>sources.edges.props.index</code>        | -                   | 否    | 属性值对应的数据文件中的列号。                                                                                                                |
| <code>sources.edges.props.nullable</code>     | -                   | 否    | 属性是否可以 <code>NULL</code> ，可选 <code>true</code> 或者 <code>false</code> 。                                                         |
| <code>sources.edges.props.nullValue</code>    | -                   | 否    | <code>nullable</code> 设置为 <code>true</code> 时，属性的值与 <code>null</code> 值一致。                                                     |
| <code>sources.edges.props.defaultValue</code> | -                   | 否    | 当 <code>nullable</code> 为 <code>false</code> 时忽略。根据 <code>index</code> 和 <code>type</code> 设置默认值。                              |



CSV 文件中列的序号从 0 开始，即第一列的序号为 0，第二列的序号为 1。

### 8.2.7 常见问题

#### 日志输出的字段含义是什么？

例如以下日志内容：

```
"msg": "44m20s 2h7m10s 25.85%(129 GiB/498 GiB) Records{Finished: 302016726, Failed: 0, Rate: 113538.13/s}, Requests{Finished: 181786, Failed: 0, Latency: 4.046496736s/4.06694393s, Rate: 68.34/s}, Processed{Finished: 908575178, Failed: 0, Rate: 341563.62/s}"
```

字段说明如下：

- `44m20s 2h7m10s 25.85%(129 GiB/498 GiB)` 对应基本信息，没有字段名称。
- 已用时间。
- 预计剩余时间。
- 已处理百分比。
- 已处理的数据大小。
- 总数据大小。
- `Records` 对应 CSV 文件的记录。
- `Finished`：完成的记录数。
- `Failed`：失败的记录数。
- `Rate`：每秒导入的记录数。
- `Requests` 对应请求。
- `Finished`：完成的请求数。
- `Failed`：失败的请求数。
- `Latency`：服务端请求耗时/客户端请求耗时。
- `Rate`：每秒处理的请求数。
- `Processed` 对应点边。
- `Finished`：处理完成的点边数量。
- `Failed`：处理失败的点边数量。
- `Rate`：每秒处理的点边数量。



最后更新: 2025年12月19日

## 9. 最佳实践

### 9.1 Compaction

本文介绍 Compaction 的相关信息。

TRSGraph 中，Compaction 是最重要的后台操作，对性能有极其重要的影响。

Compaction 操作会读取硬盘上的数据，然后重组数据结构和索引，然后再写回硬盘，可以成倍提升读取性能。将大量数据写入 TRSGraph 后，为了提高读取性能，需要手动触发 Compaction 操作（全量 Compaction）。



Note

Compaction 操作会长时间占用硬盘的 IO，建议在业务低峰期（例如凌晨）执行该操作。

TRSGraph 有两种类型的 Compaction 操作：自动 Compaction 和全量 Compaction。

#### 9.1.1 自动 Compaction

自动 Compaction 是在系统读取数据、写入数据或系统重启时自动触发 Compaction 操作，提升短时间内的读取性能。默认情况下，自动 Compaction 是开启状态，可能在业务高峰期触发，导致意外抢占 IO 影响业务。

#### 9.1.2 全量 Compaction

全量 Compaction 可以对图空间进行大规模后台操作，例如合并文件、删除 TTL 过期数据等，该操作需要手动发起。使用如下语句执行全量 Compaction 操作：



Note

建议在业务低峰期（例如凌晨）执行该操作，避免大量占用硬盘 IO 影响业务。

```
trsgraph> USE <your_graph_space>;
trsgraph> SUBMIT JOB COMPACT;
```

上述命令会返回作业的 ID，用户可以使用如下命令查看 Compaction 状态：

```
trsgraph> SHOW JOB <job_id>;
```

#### 9.1.3 操作建议

为保证 TRSGraph 的性能，请参考如下操作建议：

- 数据导入完成后，请执行 `SUBMIT JOB COMPACT`。
- 业务低峰期（例如凌晨）执行 `SUBMIT JOB COMPACT`。
- 为控制 Compaction 的写入速率，请在配置文件 `trsgraph-storaged.conf` 中设置如下参数（注：此参数限制全部写入，包括正常写入和 Compaction）：

```
写入速度限制为 20MB/s。
--rocksdb_rate_limit=20 (in MB/s)
```

### 9.1.4 FAQ

Compaction 相关的日志在哪？

默认情况下，`/usr/local/trsgraph/data/storage/nebula/{1}/data/` 目录下的文件名为 `L0G` 文件，或者类似 `L0G.old.1625797988509303`，找到如下的部分。

| ** Compaction Stats [default] ** |         |         |       |          |        |          |           |          |           |       |          |          |           |                   |           |          |
|----------------------------------|---------|---------|-------|----------|--------|----------|-----------|----------|-----------|-------|----------|----------|-----------|-------------------|-----------|----------|
| Level                            | Files   | Size    | Score | Read(GB) | Rn(GB) | Rnpl(GB) | Write(GB) | Wnew(GB) | Moved(GB) | W-Amp | Rd(MB/s) | Wr(MB/s) | Comp(sec) | CompMergeCPU(sec) | Comp(cnt) | Avg(sec) |
| KeyIn                            | KeyDrop |         |       |          |        |          |           |          |           |       |          |          |           |                   |           |          |
| L0                               | 2/0     | 2.46 KB | 0.5   | 0.0      | 0.0    | 0.0      | 0.0       | 0.0      | 0.0       | 1.0   | 0.0      | 0.0      | 0.53      | 0.51              | 2         |          |
| 0.264                            | 0       | 0       |       |          |        |          |           |          |           |       |          |          |           |                   |           |          |
| Sum                              | 2/0     | 2.46 KB | 0.0   | 0.0      | 0.0    | 0.0      | 0.0       | 0.0      | 0.0       | 1.0   | 0.0      | 0.0      | 0.53      | 0.51              | 2         |          |
| 0.264                            | 0       | 0       |       |          |        |          |           |          |           |       |          |          |           |                   |           |          |
| Int                              | 0/0     | 0.00 KB | 0.0   | 0.0      | 0.0    | 0.0      | 0.0       | 0.0      | 0.0       | 0.0   | 0.0      | 0.0      | 0.00      | 0.00              | 0         |          |
| 0.000                            | 0       | 0       |       |          |        |          |           |          |           |       |          |          |           |                   |           |          |

如果当前的 `L0` 文件数量较多，对读性能影响较大，可以触发 compaction。

可以同时多个图空间执行全量 Compaction 操作吗？

可以，但是此时的硬盘 IO 会很高，可能会影响效率。

全量 Compaction 操作会耗费多长时间？

如果已经设置读写速率限制，例如 `rocksdb_rate_limit` 限制为 20MB/S 时，用户可以通过 `硬盘使用量/rocksdb_rate_limit` 预估需要耗费的时间。如果没有设置读写速率限制，根据经验，速率大约为 50MB/S。

可以动态调整 rocksdb\_rate\_limit 吗？

不可以。

全量 Compaction 操作开始后是否可以停止？

不可以停止，必须等待操作完成。这是 RocksDB 的限制。

最后更新: 2025年12月19日

## 9.2 Storage 负载均衡

用户可以使用 `SUBMIT JOB BALANCE` 语句平衡 Raft leader 的分布。详情请参见 [SUBMIT JOB BALANCE](#)。



Danger

`SUBMIT JOB BALANCE` 命令通过创建和执行一组子任务来迁移数据和均衡分片分布，禁止停止集群中的任何机器或改变机器的 IP 地址，直到所有子任务完成，否则后续子任务会失败。

### 9.2.1 均衡 leader 分布

用户可以使用命令 `SUBMIT JOB BALANCE LEADER` 均衡分布所有图空间中的 Leader 分片副本。

#### 示例

```
trsgraph> SUBMIT JOB BALANCE LEADER;
```

用户可以执行 `SHOW HOSTS` 检查结果。

```
trsgraph> SHOW HOSTS;
+-----+-----+-----+-----+-----+-----+-----+
| Host | Port | Status | Leader count | Leader distribution | Partition distribution | Version |
+-----+-----+-----+-----+-----+-----+-----+
"192.168.10.100"	9779	"ONLINE"	4	"basketballplayer:3"	"basketballplayer:8"	"10.0"
"192.168.10.101"	9779	"ONLINE"	8	"basketballplayer:3"	"basketballplayer:8"	"10.0"
"192.168.10.102"	9779	"ONLINE"	3	"basketballplayer:3"	"basketballplayer:8"	"10.0"
"192.168.10.103"	9779	"ONLINE"	0	"basketballplayer:2"	"basketballplayer:7"	"10.0"
"192.168.10.104"	9779	"ONLINE"	0	"basketballplayer:2"	"basketballplayer:7"	"10.0"
"192.168.10.105"	9779	"ONLINE"	0	"basketballplayer:2"	"basketballplayer:7"	"10.0"
+-----+-----+-----+-----+-----+-----+-----+
```



Caution

在 TRSGraph 10.0 中，Leader 分片副本切换期间，Leader 分片副本会被暂时禁止写入直到切换完成。如果在 Leader 分片副本切换期间，有大量的写入请求，将会导致请求错误（Storage Error `E_RPC_FAILURE`），错误处理方法见 [FAQ](#)。

用户可以在 Storage 配置文件中设置 `raft_heartbeat_interval_secs` 的值来控制 Leader 副本切换的超时时间。有关配置文件的详细信息，请参见 [Storage 配置文件](#)。

最后更新: 2025年12月19日

## 9.3 图建模设计

本文介绍在 TRSGraph 项目中成功应用的一些图建模和系统设计的通用建议。



本文建议是通用的，在特定领域有例外，请结合实际业务情况进行图建模。

### 9.3.1 以性能为第一目标进行建模

目前 TRSGraph 没有完美的建模方法，如何建模取决于想从数据中挖掘的内容。分析数据并根据业务模型创建方便直观的数据模型，测试模型并优化，逐渐适应业务。为了更好的性能，用户可以多次更改或重新设计模型。

#### 设计和评估最重要的查询语句

在测试环节中，通常会验证各种各样的查询语句，以全面评估系统能力。但在大多数生产场景下，每个集群被频繁调用的查询语句的类型并不会太多；根据 20-80 原则，针对重要的查询语句进行建模优化。

#### 避免全量扫描

通过属性索引或者 VID 来先定位到某个（些）点或者边，然后开始图遍历；对于有些查询，它们只给定了一个子图或者路径的（正则）模式，但无法通过属性索引或者 VID 定位到遍历起始的点边，而期望找到库中全部满足该模式的子图，这样的查询是通过全量扫描实现的，这样的性能会很差。TRSGraph 没有实现对于子图或者路径的图结构的索引。

#### Tag 与 Edge type 之间没有绑定关系

任何 Tag 可以与任何 Edge type 相关联，完全交由应用程序控制。不需要在 TRSGraph 中预先定义，也没有命令获取哪些 Tag 与哪些 Edge type 相关联。

#### Tag/Edge type 预先定义了一组属性

建立 Tag（或者 Edge type）时，需要指定对应的属性。通常称为 Schema。

#### 区分“经常改变的部分”和“不经常改变的部分”

改变指的是业务模型和数据模型上的改变（元信息），不是数据自身的改变。

一些图数据库产品是 schema-free 的设计，所以在数据模型上，不论是图拓扑结构还是属性，都可以非常自由。属性可以建模转变为图拓扑，反之亦然。这类系统通常对于图拓扑的访问有特别的优化。

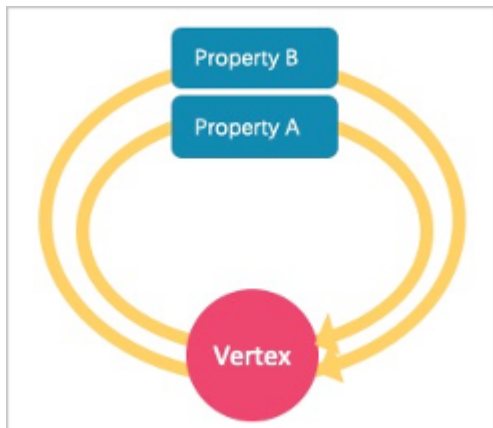
而 TRSGraph 10.0 是强 Schema 的（行存型）系统，这意味着业务数据模型中的部分是“不应该经常改变的”，例如属性 Schema 应该避免改变。类似于 MySQL 中 ALTER TABLE 是应该尽量避免的操作。

而点及邻边可以非常低成本的增删，因此可以将业务模型中“经常改变的部分”建模成点或边（关系），而不是属性 Schema。

例如，在一个业务模型中，人的属性是相对固定的，例如“年龄”，“性别”，“姓名”。而“通信好友”，“出入场所”，“交易账号”，“登录设备”等是相对容易改变的。前者适合建模为属性，后者适合建模为点或边。

#### 自环

TRSGraph 是强 Schema 类型系统，使用 ALTER TAG 的开销很大，而且也不支持 List 类型属性，当用户需要为点添加一些临时属性或者 List 类型的属性时，可以先创建包含所需属性的边类型，然后为点插入一条或多条指向自身的边。查询时只需要查询指向自己的边属性。如下图所示。



示例：

如果要检索点的临时属性，请从自环边中获取。例如：

```
//创建边类型并插入自环属性。
trsgraph> CREATE EDGE IF NOT EXISTS temp(tmp int);
trsgraph> INSERT EDGE temp(tmp) VALUES "player100"->"player100"@1:(1);
trsgraph> INSERT EDGE temp(tmp) VALUES "player100"->"player100"@2:(2);
trsgraph> INSERT EDGE temp(tmp) VALUES "player100"->"player100"@3:(3);

//插入数据后，可以通过查询语句查询，例如：
trsgraph> GO FROM "player100" OVER temp YIELD properties(edge).tmp;
+-----+
| properties(EDGE).tmp |
+-----+
| 1 |
| 2 |
| 3 |
+-----+

//如果需要返回结果为List，可以通过函数实现，例如：
trsgraph> MATCH (v1:player)-[e:temp]->() return collect(e.tmp);
+-----+
| collect(e.tmp) |
+-----+
| [1, 2, 3] |
+-----+
```

对于自环的操作没有封装任何的语法糖，操作方式与普通的边无异。

## 悬挂边

悬挂边 (Dangling edge) 是指一条起点或者终点不存在于数据库中的边。

在 TRSGraph 10.0中，有两种情况可能会出现悬挂边。

第一种情况：在起点和终点插入之前，用 `INSERT EDGE` 语句插入一条边。

第二种情况：使用 `DELETE VERTEX` 语句删除点的时候，没有使用 `WITH EDGE` 选项。此时系统默认不删除该点关联的出边和入边，这些边将变成悬挂边。

TRSGraph 10.0的数据模型中，由于设计允许图中存在“悬挂边”；没有 openCypher 中的 `MERGE` 语句。对于悬挂边的保证完全依赖应用层面。用户可以使用 `GO` 和 `LOOKUP` 语句查询到悬挂边，但无法使用 `MATCH` 语句查询到悬挂边。

示例：

```
// 插入起点为“11”，终点为“13”并且都不存在于数据库中的悬挂边
trsgraph> CREATE EDGE IF NOT EXISTS e1 (name string, age int);
trsgraph> INSERT EDGE e1 (name, age) VALUES "11"->"13":("n1", 1);

// 使用 GO 语句查询
trsgraph> GO FROM "11" over e1 YIELD properties(edge);
+-----+
| properties(EDGE) |
+-----+
| {age: 1, name: "n1"} |
+-----+
```

```
// 使用 LOOKUP 语句查询
trsgraph> LOOKUP ON e1 YIELD EDGE AS r;
+-----+
| r |
+-----+
|[:e2 "11"->"13"@0 {age: 1, name: "n1"}] |
+-----+

// 使用 MATCH 查询，不能查询到悬挂边
trsgraph> MATCH ()-[e:e1]->() RETURN e;
+----+
| e |
+----+
+----+
Empty set (time spent 3153/3573 us)
```

### 广度优先大于深度优先

- TRSGraph 基于图拓扑结构进行深度图遍历的性能较低，广度优先遍历以及获取属性的性能较好。例如，模型 a 包括姓名、年龄、眼睛颜色三种属性，建议创建一个 Tag `person`，然后为它添加姓名、年龄、眼睛颜色的属性。如果创建一个包含眼睛颜色的 Tag 和一个 Edge type `has`，然后创建一个边用来表示人拥有的眼睛颜色，这种建模方法会降低遍历性能。
- “通过边属性获取边”的性能与“通过点属性获取点”的性能是接近的。在一些数据库中，会建议将边上的属性重新建模为中间节点的属性：例如 `(src)-[edge {P1, P2}]->(dst)`，`edge` 上有属性 `P1, P2`，会建议建模为 `(src)-[edge1]->(i_node {P1, P2})-[edge2]->(dst)`。在 TRSGraph 10.0 中可以直接使用 `(src)-[edge {P1, P2}]->(dst)`，减少遍历深度有助于性能。

### 边的方向

查询时，如果需要使用边的逆向查询，可以用如下语法：

```
(dst)<-[edge]-(src) 或者 GO FROM dst REVERSELY;
```

如果不关心边的方向，可以使用如下语法：

```
(src)-[edge]-(dst) 或者 GO FROM src BIDIRECT;
```

因此，通常同一条边没有必要反向再冗余插入一次。

### 合理设置 Tag 属性

在图建模中，请将一组类似的平级属性放入同一个 Tag，即按不同概念进行分组。

### 正确使用索引

使用属性索引可以通过属性查找到 VID。但是索引会导致写性能大幅下降，只有在根据点或边的属性定位点或边时才使用索引。

### 合理设计 VID

参考点 VID 一节。

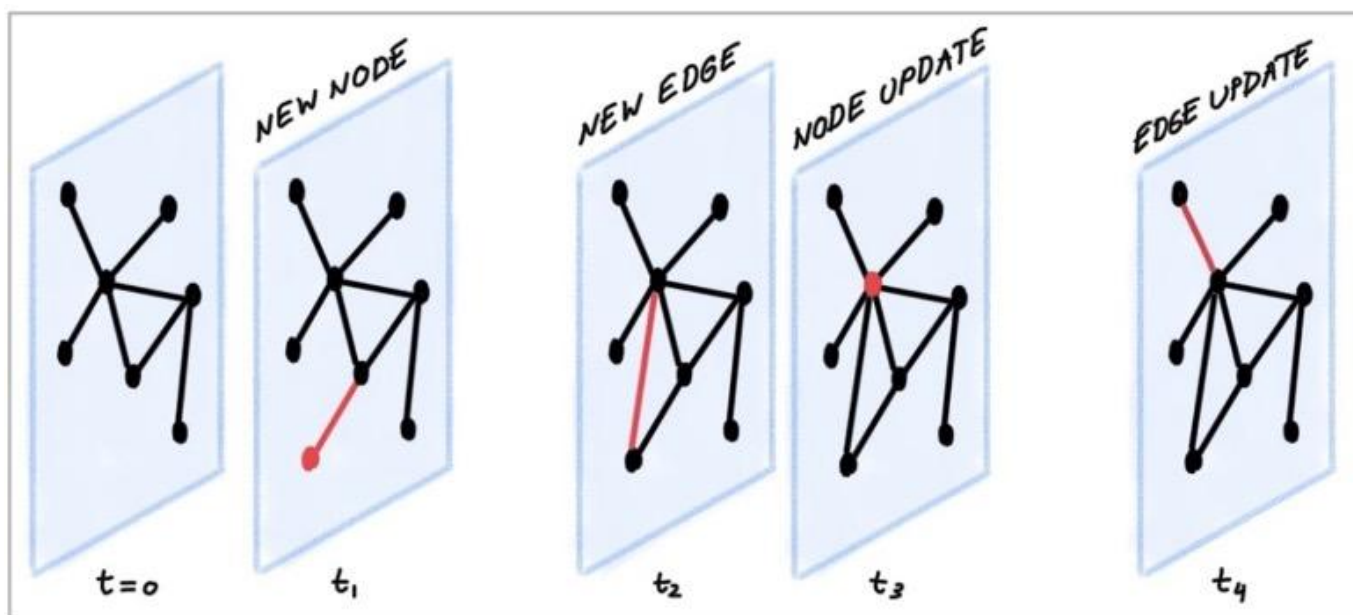
### 长文本

为边创建属性时请勿使用长文本：这些属性会被存储 2 份，导致写入放大问题（write amplification）。此时建议将长文本放在 HBase/ES 中，将其地址存放在 TRSGraph 中。

## 9.3.2 关于支持动态图（时序图）

在某些场景下，图需要同时带有序序信息，以描述整个图的结构随着时间变化的情况<sup>1</sup>。

TRSGraph 10.0 的边可以使用 Rank 字段存放时间信息 (int64)，但是点上没有字段可以存放时间信息（存放在属性会被新写入覆盖）。一个折中的办法是在点上设计自己指向自己的自环，并将时间信息放置在自环的 Rank 上。



### 9.3.3 一些免费的建模工具

arrows.app

1. [https://blog.twitter.com/engineering/en\\_us/topics/insights/2021/temporal-graph-networks](https://blog.twitter.com/engineering/en_us/topics/insights/2021/temporal-graph-networks) ←

最后更新: 2025年12月19日



## 9.4 系统设计建议

---

### 9.4.1 选择 QPS 优先或时延优先

---

- TRSGraph 10.0 更擅长处理（互联网式的）有大量并发的小请求。也即：虽然全图很大（万亿点边），但是每个请求要访问到的子图本身并不大（几百万个点边）——单个请求时延不大；但这类请求的并发数量特别多——QPS 大。
- 但对于一些交互分析型的场景，并发请求的数量不多，而每个请求要访问的子图本身特别大（亿以上）。为降低时延，可以在应用程序中将一个大的请求，拆分为多个小请求，并发送给多个 graphd。这样可以降低单个大请求的时延，降低单个 graphd 的内存占用。另外，也可以使用图计算功能 TRSGraph Algorithm。

### 9.4.2 数据传输与优化

---

- 读写平衡。TRSGraph 适合读写平衡性的在线场景，也即 OLTP 型的“并发的发生写入与读取”；而非数仓 OLAP 型的“一次写入多次读取”。
- 选择不同的写入方式。大批量的数据写入可以使用 sst 加载的方式；小批量的写入使用 `INSERT` 语句。
- 选择合适的时间运行 COMPACTION 和 BALANCE，来分别优化数据格式和存储分布。
- TRSGraph 10.0 不支持关系型数据库意义上的事务和隔离性，更接近 NoSQL。

### 9.4.3 查询预热与数据预热

---

应用端进行预热：

- Graphd 不支持预编译查询及相应生成查询计划，也不支持缓存之前的查询结果；
- Stored 不支持预热数据，只有 RocksDB 自身的 LSM-tree 和 BloomFilter 会启动时加载到内存中。
- 点和边被访问过后，会各自缓存在 Stored 的两种 (LRU) Cache 中。

---

最后更新: 2025年12月19日

## 9.5 执行计划

---

TRSGraph 10.0 实现了基于规则的执行计划。用户无法改变执行计划，无法进行查询的预编译（及相应的计划缓存），无法通过指定索引来加速查询。

要查看执行计划及执行概要，请参考 [EXPLAIN](#) 和 [PROFILE](#)。

---

最后更新: 2025年12月19日

## 9.6 超级顶点（稠密点）处理

### 9.6.1 原理介绍

在图论中，超级顶点（稠密点）是指一个点有着极多的相邻边。相邻边可以是出边（我指向谁）或者是入边（谁指向我）。

由于幂律分布的特点，超级顶点现象非常普遍。例如社交网络中的影响力领袖（网红大 V）、证券市场中的热门股票、银行系统中的四大行、交通网络中的枢纽站、互联网中的高流量网站等、电商网络中的爆款产品。

在 TRSGraph 10.0 中，一个点 和其 属性 是一个 Key-Value（以该点的 VID 以及其他元信息作为 Key），其 Out-Edge Key-Value 和 In-Edge Key-Value 都存储在同一个 partition 中（具体原理详见[存储架构](#)，并且以 LSM-tree 的形式组织存放在硬盘（和缓存）中。

因此不论是 从该点出发的有向遍历，或者 以该点为终点的有向遍历，都会涉及到大量的 顺序 IO 扫描（最理想情况，当完成 Compact 操作之后），或者大量的 随机 IO（有关于 该点 和其 出入边 频繁的写入）。

经验上说，当一个点的出入度超过 1 万时，就可以视为是稠密点。需要考虑一些特殊的设计和处理。



TRSGraph 中没有专用的字段来记录每个点的出度和入度，也没有内置任务来进行统计，因此无法预知哪些点会是超级节点。一个折中的办法是使用 Spark 周期性地计算和统计。

### 重复属性索引

在属性图中，除了网络拓扑结构中的超级顶点，还有一类情况类似于超级顶点——“某属性有极高重复率，也即“相同的 点类型 Tag，不同的 顶点 VID，同一属性字段，拥有相同属性值”。

TRSGraph 10.0 属性索引的设计复用了存储模块 RocksDB 的功能，这种情况下的索引会被建模为 前缀相同的 Key。对于该属性的查找，（如果未能命中缓存，）会对应为硬盘上的“一次随机查找 + 一次前缀顺序扫描”，以找到对应的 点 VID（此后，通常会从该顶点开始图遍历，这样又会发生该点对应 Key-Value 的一次随机读+顺序扫描）。当重复率越高，扫描范围就越大。

经验上说，当重复属性值超过 1 万时，也需要特殊的设计和处理。

### 建议的办法

#### 数据库端的常见办法

1. **截断**: 只访问一定阈值的边，超过该阈值的其他边则不返回。
2. **Compact**: 重新组织 RocksDB 中数据的排列方式，减少随机读，增加顺序读。

## 应用端的常见办法

根据业务意义，将一些超级顶点拆分：

- 删除多条边，合并为一条

例如，一个转账场景：（账户 A）-[转账]->（账户 B）。每次转账建模为一条 AB 之间的边，那么（账户 A）和（账户 B）之间会有着数十次转账的场景。

按日、周、或者月为粒度，合并陈旧的转账明细。也就是批量删除陈旧的边，改为少量的边“月总额”和“次数。而保留最近月的转账明细。

- 拆分相同类型的边，变为多种不同类型的边

例如，（机场）-<-[depart]-（航班）场景，每个架次航班的离港，都建模为一条航班和机场之间的边。那么大型机场的离港航班会极多。

根据不同的航空公司将 depart 这个 Edge type 拆分更细的 Edge type，如 depart\_ceair, depart\_csair 等。在查询（图遍历）时，指定离港的航空公司。

- 切分顶点本身

例如，对于（人）-[借款]->（银行）的借款网络，某大型银行 A 的借款次数和借款人会非常的多。

可以将该大行节点 A 拆分为多个相关联的子节点 A1、A2、A3，

```
（人 1）-[借款]->（银行 A1）， （人 2）-[借款]->（银行 A2）， （人 2）-[借款]->（银行 A3）；
（银行 A1）-[属于]->（银行 A）， （银行 A2）-[属于]->（银行 A）， （银行 A3）-[属于]->（银行 A）。
```

这里的 A1、A2、A3 既可以是 A 真实的三个分行（例如北京、上海、浙江），也可以是三个按某种规则设立的虚拟分行，例如按借款金额划分 A1：1-1000，A2：1001-10000，A3：10000+。这样，查询时对于 A 的任何操作，都转变为对于 A1、A2、A3 的三次单独操作。

---

最后更新: 2025年12月19日

## 10. 客户端

---

### 10.1 客户端介绍

---

TRSGraph 提供多种类型客户端，便于用户连接、管理 TRSGraph 图数据库。

- TRSGraph Console：原生 CLI 客户端
- TRSGraph CPP：C++ 客户端
- TRSGraph Java：Java 客户端
- TRSGraph Python：Python 客户端
- TRSGraph Go：Go 客户端



Note

仅以下类支持线程安全（thread-safe）：

- TRSGraph Java 客户端提供的 NebulaPool 和 SessionPool
- TRSGraph Go 客户端提供的 ConnectionPool 和 SessionPool

---

最后更新: 2025年12月19日

## 10.2 TRSGraph Console

TRSGraph Console 是 TRSGraph 的原生命令行客户端，用于连接 TRSGraph 集群并执行查询，同时支持管理参数、导出命令的执行结果、导入测试数据集等功能。

### 10.2.1 功能说明

#### 连接 TRSGraph

运行二进制文件 `trsgraph-console` 连接 TRSGraph 的命令语法如下：

```
<path_of_console> -addr <ip> -port <port> -u <username> -p <password>
```

- `path_of_console` 是 TRSGraph Console 二进制文件的存储路径。
- 开启 SSL 加密后，需要双向认证时，连接时需要指定 SSL 相关参数。

示例如下：

- 通过 HTTP/1.1 连接 TRSGraph

```
./trsgraph-console -addr 192.168.8.100 -port 9669 -u root -p trsadmin
```

- 开启 SSL 加密且使用私有 CA 证书连接

```
./trsgraph-console -addr 192.168.8.100 -port 9669 -u root -p trsadmin -enable_ssl -ssl_root_ca_path /home/xxx/cert/root.crt -ssl_cert_path /home/xxx/cert/client.crt -ssl_private_key_path /home/xxx/cert/client.key
```

- 开启 SSL 加密且使用受信任的 CA 证书连接

```
./trsgraph-console -addr trsgraph-graph-ncnj7sslssfnb4eqq88g.aws.dev.cloud.nebula-graph.io -port 9669 -u root -p trsadmin -enable_ssl
```

常用参数的说明如下。

| 参数                                     | 说明                                                                                          |
|----------------------------------------|---------------------------------------------------------------------------------------------|
| <code>-h/-help</code>                  | 显示帮助菜单。                                                                                     |
| <code>-addr/-address</code>            | 设置要连接的 Graph 服务的 IP 或主机名。默认地址为 127.0.0.1。                                                   |
| <code>-p/-port</code>                  | 设置要连接的 Graph 服务的端口。默认端口为 9669。                                                              |
| <code>-u/-user</code>                  | 设置 TRSGraph 账号的用户名。未启用身份认证时，可以使用任意已存在的用户名（默认为 <code>root</code> ）。                          |
| <code>-p/-password</code>              | 设置用户名对应的密码。未启用身份认证时，密码可以填写任意字符。若不指定，会弹出密码输入提醒。                                              |
| <code>-t/-timeout</code>               | 设置整数类型的连接超时时间。单位为毫秒，默认值为 120。                                                               |
| <code>-e/-eval</code>                  | 设置字符串类型的 nGQL 语句。连接成功后会执行一次该语句并返回结果，然后自动断开连接。                                               |
| <code>-f/-file</code>                  | 设置存储 nGQL 语句的文件的存储路径。连接成功后会执行该文件内的 nGQL 语句并返回结果，执行完毕后自动断开连接。                                |
| <code>-enable_ssl</code>               | 连接 TRSGraph 时使用 SSL 加密双向认证。                                                                 |
| <code>-ssl_root_ca_path</code>         | 指定私有 CA 颁布的根证书的存储路径。                                                                        |
| <code>-ssl_cert_path</code>            | 指定 SSL 客户端证书存储路径。                                                                           |
| <code>-ssl_private_key_path</code>     | 指定 SSL 客户端密钥的存储路径。                                                                          |
| <code>-ssl_insecure_skip_verify</code> | 指定客户端是否跳过验证服务端的证书链和主机名。默认为 <code>false</code> 。如果设置为 <code>true</code> ，则接受服务端提供的任何证书链和主机名。 |

管理参数

TRSGraph Console 可以保存参数，用于参数化查询。



- VID 不支持参数化查询。
- `SAMPLE` 子句中不支持参数化查询。
- 会话释放后，参数不会保留。

- 保存参数命令如下：

```
trsgraph> :param <param_name> => <param_value>;
```

示例：

```
trsgraph> :param p1 => "Tim Duncan";
trsgraph> MATCH (v:player{name:$p1})-[:follow]->(n) RETURN v,n;
+-----+
| v | n |
+-----+
| ("player100" :player{age: 42, name: "Tim Duncan"}) | ("player125" :player{age: 41, name: "Manu Ginobili"}) |
| ("player100" :player{age: 42, name: "Tim Duncan"}) | ("player101" :player{age: 36, name: "Tony Parker"}) |
+-----+

trsgraph> :param p2 => {"a":3,"b":false,"c":"Tim Duncan"};
trsgraph> RETURN $p2.b AS b;
+-----+
| b |
+-----+
| false |
+-----+
```

- 查看当前保存的所有参数，命令如下：

```
trsgraph> :params;
```

- 查看指定参数，命令如下：

```
trsgraph> :params <param_name>;
```

- 删除指定参数，命令如下：

```
trsgraph> :param <param_name> =>;
```

导出执行结果

导出命令执行的返回结果，可以保存为 CSV 文件、DOT 文件或者 Profile/Explain 结果。





- 文件保存在当前工作目录中，即 Linux 命令 `pwd` 显示的目录。
- 命令只对下一条查询语句生效。
- DOT 文件的内容可以复制后在 [GraphvizOnline](#) 网页中粘贴，生成可视化的执行计划图。

- 导出 CSV 文件命令如下：

```
trsgraph> :CSV <file_name.csv>;
```

- 导出 DOT 文件命令如下：

```
trsgraph> :dot <file_name.dot>;
```

示例：

```
trsgraph> :dot a.dot;
trsgraph> PROFILE FORMAT="dot" GO FROM "player100" OVER follow;
```

- 导出 PROFILE/EXPLAIN 结果到文件命令如下：

```
trsgraph> :profile <file_name>;
```

或者

```
trsgraph> :explain <file_name>;
```



相比于 Studio 中的截图、CSV 文件，因为保有更多信息量和拥有更好的可读性，经由此命令输出的文本文件内容是首推的在 GitHub issue、论坛中报告执行计划、图查询调优的方式。

示例：

```
trsgraph> :profile profile.log
trsgraph> PROFILE GO FROM "player102" OVER serve YIELD dst(edge);
trsgraph> :profile profile.dot
trsgraph> PROFILE FORMAT="dot" GO FROM "player102" OVER serve YIELD dst(edge);
trsgraph> :explain explain.log
trsgraph> EXPLAIN GO FROM "player102" OVER serve YIELD dst(edge);
```

## 加载测试数据集

测试数据集名称为 `basketballplayer`，详细 Schema 信息和数据信息请使用相关 `SHOW` 命令查看。

加载测试数据集命令如下：

```
trsgraph> :play basketballplayer;
```

## 重复执行语句

重复执行下一个命令 N 次，然后打印平均执行时间。命令如下：

```
trsgraph> :repeat N;
```

示例：

```
trsgraph> :repeat 3;
trsgraph> GO FROM "player100" OVER follow YIELD dst(edge);
+-----+
| dst(EDGE) |
+-----+
| "player101" |
| "player125" |
+-----+
Got 2 rows (time spent 2602/3214 us)

Fri, 20 Aug 2021 06:36:05 UTC

+-----+
| dst(EDGE) |
+-----+
| "player101" |
| "player125" |
+-----+
Got 2 rows (time spent 583/849 us)

Fri, 20 Aug 2021 06:36:05 UTC

+-----+
| dst(EDGE) |
+-----+
| "player101" |
| "player125" |
+-----+
Got 2 rows (time spent 496/671 us)

Fri, 20 Aug 2021 06:36:05 UTC

Executed 3 times, (total time spent 3681/4734 us), (average time spent 1227/1578 us)
```

睡眠

睡眠 N 秒。常用于修改 Schema 的操作中，因为修改 Schema 是异步实现的，需要在下一个心跳周期才同步数据。命令如下：

```
trsgraph> :sleep N;
```

断开连接

用户可以使用 `:EXIT` 或者 `:QUIT` 从 TRSGraph 断开连接。为方便使用，TRSGraph Console 支持使用不带冒号 (:) 的小写命令，例如 `quit`。

示例：

```
trsgraph> :QUIT;

Bye root!
```

最后更新: 2025年12月19日

## 10.3 TRSGraph CPP

---

TRSGraph CPP 是一款 C++ 语言的客户端，可以连接、管理 TRSGraph 图数据库。

### 10.3.1 前提条件

---

请确保已安装 C++ 且 GCC 版本为 4.8 及以上。

### 10.3.2 使用方法

---

将 CPP 文件编译为可执行文件即可。接下来以 `SessionExample.cpp` 为例，介绍如何操作。

编译文件，命令如下：

```
$ LIBRARY_PATH=<library_folder_path>:$LIBRARY_PATH g++ -std=c++11 SessionExample.cpp -I<include_folder_path> -lnebula_graph_client -o session_example
```

- `library_folder_path`：TRSGraph 动态库文件存储路径，默认为 `/usr/local/trsgraph/lib64`。

- `include_folder_path`：TRSGraph 头文件存储路径，默认为 `/usr/local/trsgraph/include`。

示例：

```
$ LIBRARY_PATH=/usr/local/trsgraph/lib64:$LIBRARY_PATH g++ -std=c++11 SessionExample.cpp -I/usr/local/trsgraph/include -lnebula_graph_client -o session_example
```

---

最后更新: 2025年12月19日

## 10.4 TRSGraph Java

---

TRSGraph Java 是一款 Java 语言的客户端，可以连接、管理 TRSGraph 图数据库。

### 10.4.1 前提条件

---

已安装 JDK 8。

### 10.4.2 使用方法

---



Note

建议一个线程使用一个会话，如果多个线程使用同一个会话，会降低效率。

将API依赖的JAR包添加到CLASSPATH中。

### 10.4.3 API 文档

---

点击[此处](#)查看 Java 客户端提供的各种类和方法。

---

最后更新: 2025年12月19日

## 10.5 TRSGraph Python

---

TRSGraph Python 是一款 Python 语言的客户端，可以连接、管理 TRSGraph 图数据库。

### 10.5.1 前提条件

---

已安装 Python，版本为 3.6 及以上。

### 10.5.2 安装 TRSGraph Python

---

#### 克隆源码安装

1. 克隆 TRSGraph Python 源码到机器。
2. 进入目录 trsgraph-python。

```
bash
$ cd trsgraph-python
```

3. 执行如下命令安装。

```
bash
$ pip install .
```

### 10.5.3 API 文档

---

点击[此处](#)查看 Python 客户端提供的各种类和方法。

---

最后更新: 2025年12月19日

## 11. TRSGraph Studio

### 11.1 认识 TRSGraph Studio

#### 11.1.1 什么是 TRSGraph Studio

TRSGraph Studio（简称 Studio）是一款可以通过 Web 访问的开源图数据库可视化工具，搭配 TRSGraph 内核使用，提供构图、数据导入、编写 nGQL 查询等一站式服务。

#### 发行版本

用户可以使用tar 包安装部署 Studio。详细信息参考 [部署 Studio](#)。

几种部署方式功能基本相同，在使用 Studio 时可能会受到限制。详细信息，参考 [使用限制](#)。

#### 产品功能

Studio 可以方便管理 TRSGraph 数据，具备以下功能：

- 使用 Schema 管理功能，用户可以使用图形界面完成图空间、Tag（标签）、Edge Type（边类型）、索引的创建，查看图空间的统计数据，快速上手 TRSGraph。
- 使用导入功能，通过简单的配置，用户即能批量导入点和边数据，并能实时查看数据导入日志。
- 使用控制台功能，用户可以使用 nGQL 语句创建 Schema，并对数据执行增删改查操作。

#### 适用场景

如果有以下任一需求，都可以使用 Studio：

- 已经安装部署了 TRSGraph，想使用 GUI 工具创建 Schema、导入数据、执行 nGQL 语句查询。
- 刚开始学习 nGQL(TRSGraph Query Language)，但是不习惯用命令行工具，更希望使用 GUI 工具查看语句输出的结果。

#### 身份验证

因为 TRSGraph 默认不启用身份验证，所以用户可以使用 `root` 账号和任意密码登录 Studio。

当 TRSGraph 启用了身份验证后，用户只能使用指定的账号和密码登录 Studio。关于 TRSGraph 的身份验证功能，参考 [身份验证](#)。

#### 版本兼容性



Studio 版本发布节奏独立于 TRSGraph 内核，其命名方式也不参照内核命名规则，两者兼容对应关系如下表。

| TRSGraph 版本 | Studio 版本 |
|-------------|-----------|
| 10.0        | 10.0      |

最后更新: 2025年12月19日

### 11.1.2 使用限制

---

本文描述使用 Studio 的限制。

#### 系统架构

Studio 目前仅支持 x86\_64 架构。

#### 数据上传

Studio 上传数据仅支持上传无表头的 CSV 文件，但是，单个文件大小及保存时间不受限制，而且数据总量以本地存储容量为准。

#### 数据备份

目前仅支持在 控制台 上以 CSV 格式导出查询结果，不支持其他数据备份方式。

#### nGQL 支持

除以下内容外，用户可以在 控制台 上执行所有 nGQL 语句：

- `USE <space_name>`：只能在 Space 下拉列表中选择图空间，不能运行这个语句选择图空间。
- 控制台 上使用 nGQL 语句时，用户可以直接回车换行，不能使用换行符。

#### 浏览器支持

建议使用最新版本的 Chrome 访问 Studio。否则会出现样式显示异常和交互异常等问题。

---

最后更新: 2025年12月19日

## 11.2 安装与登录

---

### 11.2.1 部署 Studio

---

本文介绍如何在本地通过ar 包部署 Studio。

#### tar 包部署 Studio

前提条件

在部署 tar 包安装的 Studio 之前，用户需要确认以下信息：

- TRSGraph 服务已经部署并启动。详细信息，参考 [TRSGraph 安装部署](#)。
- 确保以下端口未被占用。

| 端口号  | 说明                |
|------|-------------------|
| 7001 | Studio 提供的 web 服务 |

---



安装部署

1. 根据需要下载 tar 包，建议选择最新版本。

| 安装包                      | Studio 版本 | 适用 TRSGraph 版本 |
|--------------------------|-----------|----------------|
| [trsgraph-studio.tar.gz] | 10.0      | 10.0           |

1. 解压 tar 包。

```
tar -xvf trsgraph-studio.tar.gz
```

2. 部署 trs-graph-studio 并启动。

```
$ cd trs-graph-studio
$./server
```

3. 启动成功后，在浏览器地址栏输入 `http://<ip address>:7001`。

如果在浏览器窗口中能看到以下登录界面，表示已经成功部署并启动 Studio。



停止服务

用户可以采用 `kill <pid>` 的方式来关停服务：

```
$ kill $(lsof -t -i :7001) # stop trs-graph-studio
```

最后更新: 2025年12月19日

## 11.2.2 连接数据库

---

在成功启动 Studio 后，用户需要配置连接 TRSGraph。本文主要描述 Studio 如何连接 TRSGraph。

### 前提条件

在连接 TRSGraph 数据库前，用户需要确认以下信息：

- Studio 已经启动。详细信息参考 [部署 Studio](#)。
- TRSGraph 的 Graph 服务本机 IP 地址以及服务所用端口。默认端口为 `9669`。
- TRSGraph 登录账号信息，包括用户名和密码。

操作步骤

按以下步骤连接 TRSGraph：

1. 在浏览器地址栏输入 `http://<ip_address>:7001`。

在浏览器窗口中看到以下登录界面表示已经成功部署并启动了 Studio。



2. 在 Studio 的 配置数据库 页面上，输入以下信息：

- Graphd IP 地址：填写 TRSGraph 的 Graph 服务本机 IP 地址。例如 `192.168.10.100`。

Note

- 即使 TRSGraph 与 Studio 部署在同一台机器上，用户也必须填写这台机器的本机 IP 地址，而不是 `127.0.0.1` 或者 `localhost`。
- 在新的标签页连接另一个 TRSGraph 时，会覆盖旧标签页的会话。如果需要同时登录多个 TRSGraph，可以用不同的浏览器或者无痕模式。

- Port：Graphd 服务的端口。默认为 `9669`。
- 用户名 和 密码：根据 TRSGraph 的身份验证设置填写登录账号和密码。
- 如果未启用身份验证，可以填写默认用户名 `root` 和任意密码。
- 如果已启用身份验证，但是未创建账号信息，用户只能以 GOD 角色登录，必须填写 `root` 及对应的密码 `trsadmin`。
- 如果已启用身份验证，同时又创建了不同的用户并分配了角色，不同角色的用户使用自己的账号和密码登录。

3. 完成设置后，点击 连接 按钮。

Note

一次连接会话持续 30 分钟。如果超过 30 分钟没有操作，会话即断开，用户需要重新登录数据库。

首次登录会显示欢迎页，根据使用流程展示相关功能，并且支持自动下载并导入测试数据集。

想要再次访问欢迎页，单击 。

后续操作

成功连接 TRSGraph 后，用户可以执行以下操作：

- 使用控制台或者 Schema 页面管理 Schema。
- 批量导入数据。
- 在控制台页面上执行 nGQL 语句查询数据。
- 在 Schema 草图页面图形化设计 Schema。



账号的权限决定了能执行哪些操作。详情参见[内置角色权限](#)。

登出

如果需要重新连接 TRSGraph，可以登出后重新配置数据库。

在页面右上角单击用户头像，单击 登出。

最后更新: 2025年12月19日

## 11.3 快速开始

### 11.3.1 规划 Schema

在使用 Studio 之前，用户需要先根据 TRSGraph 的要求规划 Schema（模式）。

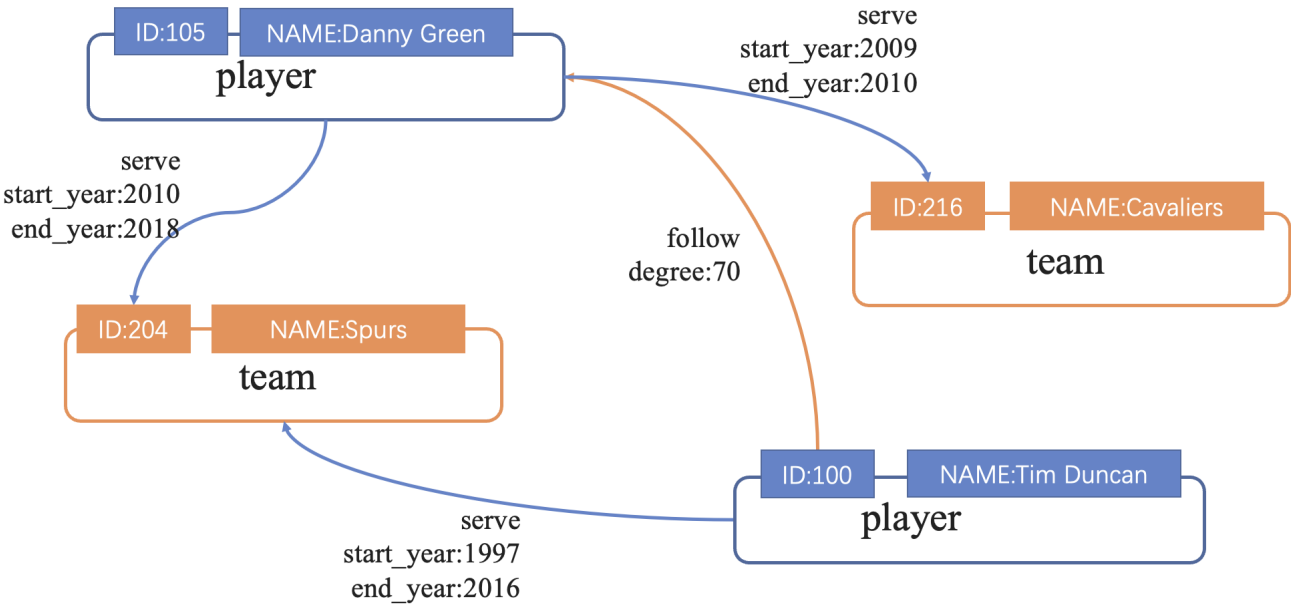
Schema 至少要包含以下要素：

- Tag，以及每种 Tag 的属性。
- Edge type，以及每种 Edge type 的属性。

下表列出了 Schema 要素。

| 类型        | 名称     | 属性名（数据类型）                              | 说明                                       |
|-----------|--------|----------------------------------------|------------------------------------------|
| Tag       | player | - name (string)<br>- age (int)         | 表示球员。                                    |
| Tag       | team   | - name (string)                        | 表示球队。                                    |
| Edge type | serve  | - start_year (int)<br>- end_year (int) | 表示球员的行为。<br>该行为将球员和球队联系起来，方向是从球员到球队。     |
| Edge type | follow | - degree (int)                         | 表示球员的行为。<br>该行为将两个球员联系起来，方向是从一个球员到另一个球员。 |

下图说明示例中 player 类点与 team 类点之间如何发生关系（serve/follow）。



### 11.3.2 创建 Schema

在 TRSGraph 中，用户必须先有 Schema，才能向其中写入点数据和边数据。本文描述如何使用 TRSGraph 的控制台或 Schema 功能创建 Schema。

Note

- 用户可以使用 trsgraph-console 创建 Schema。详情参见 [TRSGraph 使用手册](#)和 [TRSGraph 快速开始](#)。
- 用户可以使用 Schema 草图功能图形化设计 Schema。详情参见 [Schema 草图](#)。

#### 前提条件

在 Studio 上创建 Schema 之前，用户需要确认以下信息：

- Studio 已经连接到 TRSGraph 数据库。
- 账号拥有 GOD、ADMIN 或 DBA 权限。详细信息，参考 [TRSGraph 内置角色](#)。
- 已经规划 Schema 的要素。
- 已经创建图空间。

Note

本示例已经创建图空间。如果账号拥有 GOD 权限，也可以在 控制台 或 Schema 上创建一个图空间。

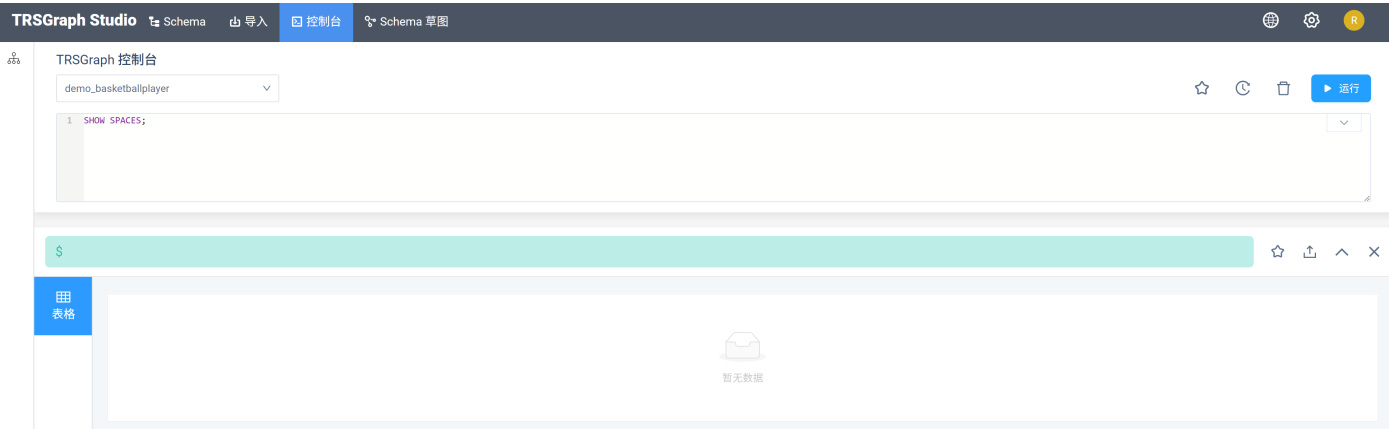
#### 使用 Schema 管理功能创建 Schema

按以下步骤使用 Schema 管理功能创建 Schema：

1. 创建 Tag。详细信息，参考[操作 Tag](#)。
2. 创建 Edge type。详细信息，参考[操作 Edge type](#)。

#### 使用控制台创建 Schema

1. 在顶部导航栏里，点击 控制台 页签。
2. 在 当前 Space 中选择一个图空间。在本示例中，选择 demo\_basketballplayer。



3. 在命令行中，依次输入以下语句，并点击右侧的运行按钮。

```
// 创建 Tag player，带有 2 个属性
CREATE TAG player(name string, age int);
```

```
// 创建 Tag team, 带有 1 个属性
CREATE TAG team(name string);

// 创建 Edge type follow, 带有 1 个属性
CREATE EDGE follow(degree int);

// 创建 Edge type serve, 带有 2 个属性
CREATE EDGE serve(start_year int, end_year int);
```

至此，用户已经完成了 Schema 创建。用户可以运行以下语句查看 Tag 与 Edge type 的定义是否正确、完整。

```
// 列出当前图空间中所有 Tag
SHOW TAGS;

// 列出当前图空间中所有 Edge type
SHOW EDGES;

// 查看每种 Tag 和 Edge type 的结构是否正确
DESCRIBE TAG player;
DESCRIBE TAG team;
DESCRIBE EDGE follow;
DESCRIBE EDGE serve;
```

## 后续操作

创建 Schema 后，用户可以开始[导入数据](#)。

---

最后更新: 2025年12月19日

### 11.3.3 导入数据

Studio 支持界面化地将 CSV 格式数据导入至 TRSGraph 中。

**前提条件**

导入数据之前，需要确认以下信息：

- TRSGraph 里已经创建 Schema。
- CSV 文件符合 Schema 要求。
- 账号拥有 GOD、ADMIN 或 DBA 权限。详情参见 [TRSGraph 内置角色](#)。

**入口**

在顶部导航栏里，单击导入。

**操作步骤**

导入数据主要分为 2 个部分，新建数据源和创建导入任务，接下来将详细介绍。

Note

导入任务还支持 AI 导入，该功能为 Beta 功能，使用前需要在 [系统设置](#) 里设置和启用该功能。

**新建数据源**

在页面右上角单击新建数据源，设置数据来源及其相关设置。当前支持 3 种类型的数据源。

| 数据源类型 | 说明                                               |
|-------|--------------------------------------------------|
| 云存储   | 添加云存储作为 CSV 文件来源，只支持兼容 Amazon S3接口的云服务。          |
| SFTP  | 添加 SFTP 作为 CSV 文件来源。                             |
| 本地文件  | 上传本地 CSV 文件。文件大小不能超过 200 MB，超过限制的文件请放入其他方式的数据源中。 |

Note

- 上传本地 CSV 文件时，一次可以选择多个 CSV 文件。
- 数据源添加后，可以在页面上方单击数据源管理，切换页签即可查看不同类型的数据源详情，也可以编辑或删除数据源。

**创建导入任务**

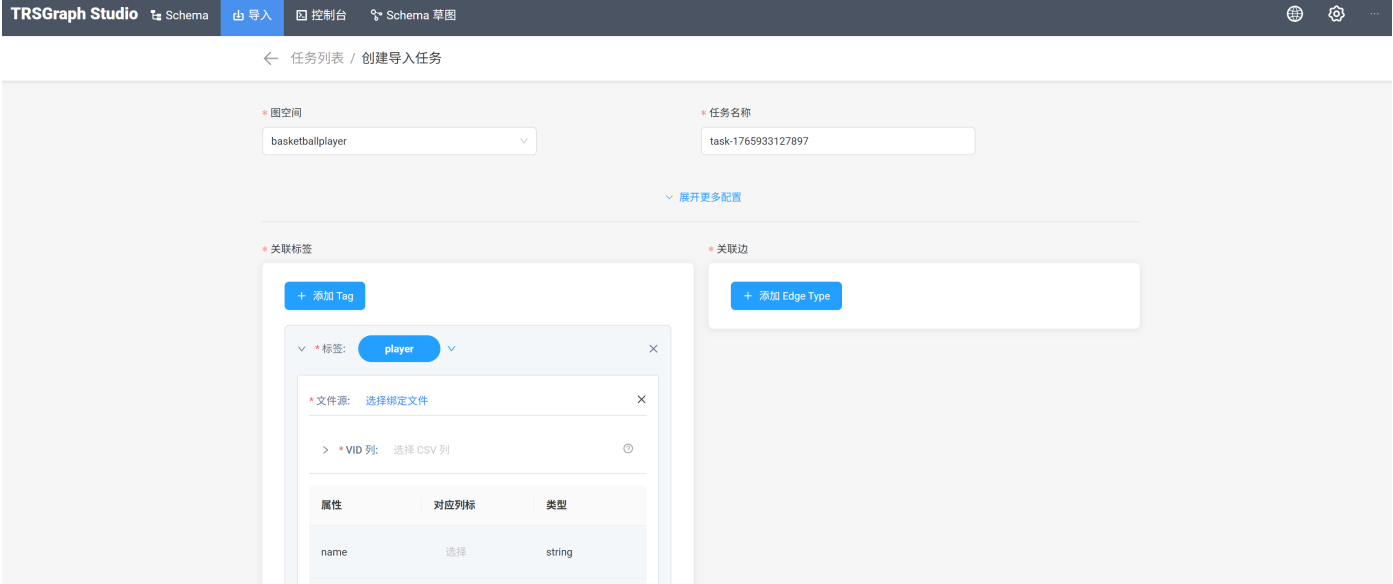
1. 在页面左上角单击创建导入任务，完成如下设置：





用户也可以单击导入模版，下载示例配置文件 `example.yaml`，配置后再上传配置文件。配置方式与 `TRSGraph Importer` 大致相同。

- 图空间：需要导入数据的图空间名称。
- 任务名称：默认自动生成，可以修改。
- 更多配置（可选）：可以自定义设置并发数、批处理量、重试次数、读取并发数和导入并发数。
- 关联标签：
  - a. 单击添加 Tag，然后在下方新增的标签内选择 Tag。
  - b. 单击添加导入文件，在文件源里选择数据源类型和文件路径，找到需要导入的文件，然后单击添加。
  - c. 在预览页面设置文件的分隔符和是否携带表头，然后单击确认。
  - d. 在VID 列为 VID 选择对应的列。支持选择多个列合并为 VID，也可以为 VID 添加前缀或后缀。
  - e. 在属性框内为属性选择对应的列。对于可以为 `NULL` 或设置了 `DEFAULT` 的属性，可以不指定对应的列。
  - f. 重复 2 ~ 5 步骤将步骤 1 所选 Tag 的数据文件全部导入。
  - g. 重复 1 ~ 6 步骤将所有需要导入的 Tag 数据全部导入。
- 关联边：与关联标签的操作相同。



2. 完成设置后，单击导入,输入 TRSGraph 账号的密码并确认。

导入任务创建后，可以在导入数据页签内查看导入任务的进度，支持根据图空间筛选任务、编辑任务、查看日志、下载日志、重新导入、下载配置文件、删除任务等操作。

AI 导入



导入任务执行完成后，请手动检查数据是否正常导入，推荐结合导入页面的任务日志进行查看，排查超时、触发隐私政策、服务中断、编码等问题。

1. 在页面左上角单击AI 导入，完成如下设置：
- 文件：选择需要进行 AI 导入的文件。当前仅支持本地单个文件或本地目录。每次只能选择一个本地文件或目录。文件需要为纯文本文件。
  - 导入图空间：待导入数据的图空间。图空间的 Schema 需要提前创建好。
  - 导出 NGQL 文件路径：导出 ngql 文件的路径，通常无需修改。
  - 附加提示：可以附加其他提示来指导数据导入。提示的精度直接影响导入结果的准确性。  
你可以在配置文件中查看和 AI 导入相关的 `LLM` 参数。

2. 完成设置后，单击下一步，确认待导入的文件和使用的 AI URL，然后单击开始。

导入任务创建后，可以在导入数据页签内查看导入任务的进度，支持查看日志、下载日志、重新导入、删除任务等操作。

后续操作

完成数据导入后，用户可以进入控制台页面。

最后更新: 2025年12月19日

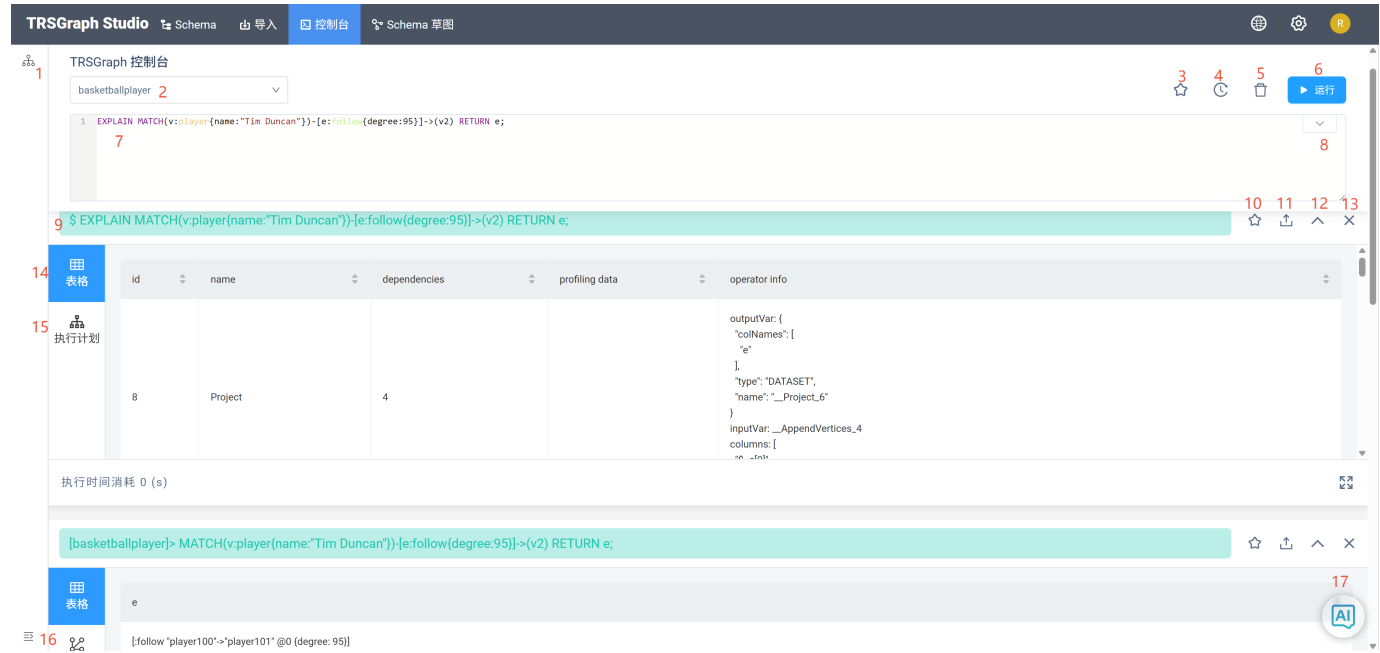
### 11.3.4 控制台界面

本文介绍 Studio 的控制台界面。

#### 入口

在顶部导航栏里，单击控制台。

#### 页面介绍

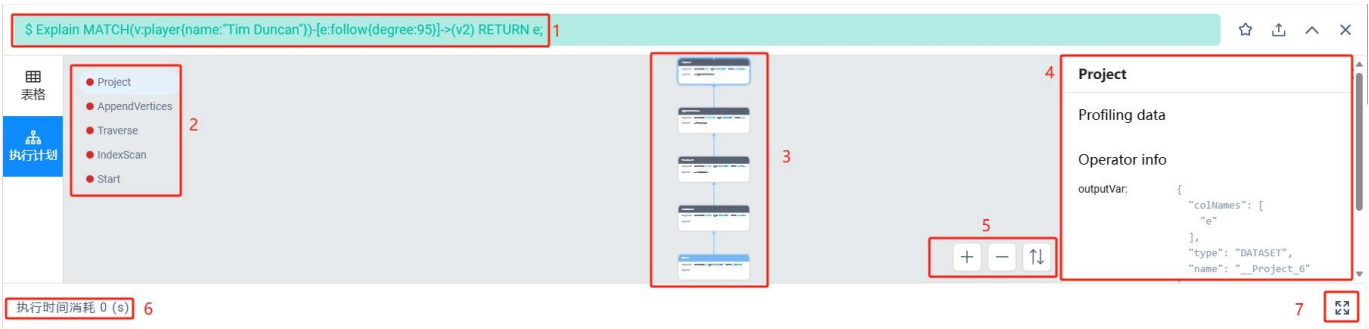


下表列出了控制台界面上的各种功能。

| 编号 | 功能                  | 说明                                                                                                                                                                                                                                                                     |
|----|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 查看 Schema           | 展示图空间的 Schema。                                                                                                                                                                                                                                                         |
| 2  | 选择图空间               | 选择执行 nGQL 的图空间。不支持在控制台执行 <code>USE &lt;space_name&gt;</code> 语句切换图空间。                                                                                                                                                                                                  |
| 3  | 收藏夹                 | 点击  按钮，展开收藏夹，点击其中一个语句，该语句会自动输入到输入框中。                                                                                                                                                  |
| 4  | 历史清单                | 点击  按钮，在语句运行记录列表里，点击其中一个语句，输入框中即自动输入该语句。列表里提供最近 15 次语句运行记录。在输入框中输入 <code>/</code> 可以快速选择历史查询语句。                                                                                       |
| 5  | 清空输入框               | 点击  按钮，清空输入框中已经输入的内容。                                                                                                                                                                 |
| 6  | 运行                  | 在输入框中输入 nGQL 语句后，点击  按钮即开始运行语句。                                                                                                                                                       |
| 7  | 输入框                 | 输入 nGQL 语句的区域。语句会根据 Schema 或字符串展示不同颜色。支持代码补全，可以根据 Schema 快速输入 Tag 或者 Edge type。可以同时输入多个语句按顺序执行，语句之间以 <code>;</code> 分隔。可以用 <code>//</code> 添加注释。支持对选中的语句单击右键，然后执行剪切、复制、运行等操作。                                                                                          |
| 8  | 自定义参数展示             | 点击  按钮可展开查看自定义参数，用于参数化查询。详情信息可见 <a href="#">管理参数</a> 。                                                                                                                                |
| 9  | 语句运行状态              | 运行 nGQL 语句后，这里显示语句运行状态。如果语句运行成功，语句以绿色显示。如果语句运行失败，语句以红色显示。                                                                                                                                                                                                              |
| 10 | 添加到收藏夹              | 点击  按钮，将语句存入收藏夹中，已收藏的语句该按钮以黄色展示。                                                                                                                                                    |
| 11 | 导出 CSV 文件或 PNG 格式图片 | 运行 nGQL 语句返回结果后，返回结果为表格形式时，点击  按钮即能将结果以 CSV 文件的形式导出。切换到可视化窗口，点击  按钮即能将结果以 CSV 文件或 PNG 图片的形式导出。 |
| 12 | 展开/隐藏执行结果           | 点击  按钮，隐藏此条 nGQL 语句返回的结果。                                                                                                                                                           |
| 13 | 关闭执行结果              | 点击  按钮，关闭此条 nGQL 语句返回的结果。                                                                                                                                                           |
| 14 | 表格窗口                | 在表格中显示语句的运行结果。                                                                                                                                                                                                                                                         |
| 15 | 执行计划窗口              | 显示执行计划。如果执行的是 <code>EXPLAIN</code> 或 <code>PROFILE</code> 语句，窗口会以可视化形式呈现执行计划。如何理解执行计划请参见下文说明。                                                                                                                                                                          |
| 16 | 可视化窗口               | 显示语句运行结果。如果结果包含完整的点边信息，窗口会以可视化形式呈现返回的结果。点击右方  按钮，展开数据概览面板。                                                                                                                        |
| 17 | AI 查询语句助手           | 用户可以和助手聊天，将自然语言转换为 nGQL 查询语句，并支持一键复制 nGQL 语句到输入框中。使用前需要在 <a href="#">系统设置</a> 里设置和启用该功能。注意：和助手交互时，会将当前图空间的 Schema 信息发送给大规模语言模型，请注意信息安全。开关 <code>text2match</code> 用于切换普通问答和查询问答。查询问答可以输入自然语言转换为 nGQL 查询语句。                                                             |

执行计划说明

Studio 可以展示语句的执行计划，界面说明如下。



| 序号 | 说明                                                                                                                                       |
|----|------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | EXPLAIN 或 PROFILE 语句。                                                                                                                    |
| 2  | 执行计划涉及的算子并根据执行耗时排序。耗时前三的算子分别标记为红色、橘红色和黄色。单击算子可以直接选中执行流程中的对应算子并展示算子信息。注意：PROFILE 语句会实际执行语句，所以能得到实际的耗时并排序。EXPLAIN 不会执行语句，所有算子耗时视为相同并标记为红色。 |
| 3  | 算子执行流程。每个算子都会显示该算子的入参、出参和耗时。Select、Loop、PassThrough 和 Start 算子有独立配色。箭头展示数据流向和行数，行数越多，箭头越粗。单击算子会在右侧展示算子的详细信息。                             |
| 4  | 算子详细信息。分为 Profiling data 和 Operator info 两部分。Profiling data 展示算子的性能数据，包括接收到的数据行数、执行耗时、总耗时等。Operator info 展示算子的具体操作信息。                    |
| 5  | 放大或缩小界面，还可以倒序显式算子执行流程。                                                                                                                   |
| 6  | 语句耗时。                                                                                                                                    |
| 7  | 全屏展示或取消全屏。                                                                                                                               |

最后更新: 2025年12月19日

## 11.3.5 管理Schema

### 操作图空间

Studio 连接到 TRSGraph 后，用户可以创建或删除图空间。用户可以使用 控制台 或者 Schema 操作图空间。本文仅说明如何使用 Schema 操作图空间。

#### 前提条件

操作图空间之前，用户需要确保以下信息：

- Studio 已经连接到 TRSGraph。
- 当前登录的账号拥有创建或删除图空间的权限，即：
- 如果 TRSGraph 未开启身份验证，用户以默认用户名 `user` 账号和任意密码登录。
- 如果 TRSGraph 已开启身份验证，用户以 `root` 账号及其密码登录。

#### 创建图空间

1. 在顶部导航栏里，点击 Schema 页签。
2. 在图空间列表上方，点击 + 创建图空间 按钮，完成以下配置：
  - 名称：指定图空间名称，本示例中设置为 `basketballplayer`。不可与已有的图空间名称重复。
  - vid type：图空间中点 ID (VID) 的数据类型，可选值为定长字符串 `FIXED_STRING(<N>)` 或 `INT64`，一旦定义无法修改。本示例设置为 `FIXED_STRING(32)`。详细信息，参考 [VID](#)。
  - 描述：图空间的描述，最大为 256 字节。默认无描述。本示例设置为 `Statistics of basketball players`。
  - 可选参数：分别设置 `partition_num`、`replica_factor` 的值。在本示例中，两个参数分别设置为 100、1。详细信息，参考 [CREATE SPACE](#) 语法。在对应的 nGQL 语句 面板上，用户能看到上述设置对应的 nGQL 语句。如下所示：

```
CREATE SPACE basketballplayer (partition_num = 100, replica_factor = 1, vid_type = FIXED_STRING(32)) COMMENT = "Statistics of basketball players"
```

3. 配置确认无误后，点击 创建 按钮。如果页面回到 图空间列表，而且列表中显示刚创建的图空间信息，表示图空间创建成功。

TRSGraph Studio

Schema

由 导入

控制台

Schema 草图

← 图空间列表 / 创建图空间

名称

basketballplayer

Vid Type

FIXED\_STRING

长度

32

描述: 可选

Statistics of basketball players

Partition\_num: 可选

10

Replica\_factor: 可选

1

对应的nGQL语句

```
1 CREATE SPACE 'basketballplayer' (partition_num = 10, vid_type = FIXED_STRING(32)) COMMENT = "Statistics of basketball players"
```

取消

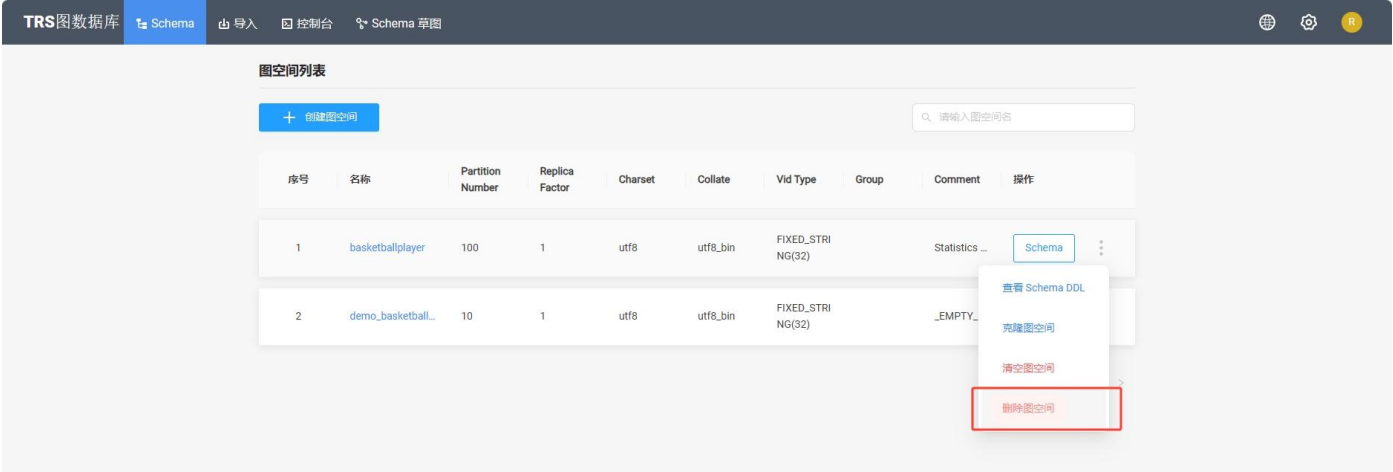
创建

删除图空间

 Danger

删除图空间会删除其中的所有数据，已删除的数据如未备份无法恢复。

1. 在顶部导航栏里，点击 Schema 页签。
2. 在图空间列表里，找到需要删除的图空间，并在 操作 列中，选择 删除图空间。



3. 在弹出的对话框中点击 确认。

后续操作

图空间创建成功后，用户可以开始创建或修改 Schema，包括：

- 操作 Tag
- 操作 Edge type
- 操作索引

最后更新: 2025年12月19日

操作 Tag（点类型）

在 TRSGraph 中创建图空间后，用户需要创建 Tag（点类型）。用户可以选择使用 控制台 或者 Schema 管理功能操作 Tag。本文仅说明如何使用 Schema 管理功能操作 Tag。

前提条件

在 Studio 上操作 Tag 之前，用户必须确认以下信息：

- Studio 已经连接到 TRSGraph。
- 图空间已经创建。
- 当前登录的账号拥有 GOD、ADMIN 或者 DBA 的权限。

创建 TAG

1. 在顶部导航栏中，点击 Schema 页签。
2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 Schema。
3. 在 当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 标签 页签，并点击 + 创建 按钮。
5. 在 创建标签 页面上，完成以下设置：
  - a. 名称：输入 Tag 名称。本示例中，输入 `player`。
  - b. 描述（可选）：输入 Tag 的备注。
  - c. 定义属性（可选）：如果 Tag 需要属性，点击 + 添加属性，添加一个或多个属性：
    - 输入属性名称。
    - 选择数据类型。
    - 选择是否允许空值。
    - （可选）输入默认值。
    - （可选）输入属性备注。
  - d. 设置TTL（存活时间）（可选）：Tag 未设置索引时，用户可以设置 TTL。勾选设置TTL（存活时间），设置 `TTL_COL` 和 `TTL_DURATION`（单位：秒）。详情参考 [TTL 配置](#)。
6. 完成设置后，在 对应的nGQL语句 面板，用户能看到与上述配置等价的 nGQL 语句。

TRSGraph Studio

Schema

由 导入

控制台

Schema 草图

← 图空间列表 / 标签列表 / 创建标签

当前图空间: basketballplayer

\* 名称

player

描述

定义属性

+ 添加属性

| * 属性名称 | * 数据类型       | 允许空值                                   | 默认值 | 描述 |    |
|--------|--------------|----------------------------------------|-----|----|----|
| age    | int64        | <input checked="" type="checkbox"/>    |     |    | 删除 |
| name   | fixed_string | 64 <input checked="" type="checkbox"/> |     |    | 删除 |

☐ 设置TTL（存活时间）

TTL\_COL (属性的数据类型必须是int或者timestamp)

TTL\_DURATION

请输入时间(s)

取消

创建



7. 确认无误后，点击 创建 按钮。

如果 Tag 创建成功，标签 面板会显示这个 Tag 的属性列表。

修改 TAG

- 1. 在顶部导航栏中，点击 Schema 页签。
- 2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 Schema。
- 3. 在 当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
- 4.



点击 标签 页签，找到需要修改的 Tag，并在 操作 列中，点击 图标。

5. 在 编辑 页面，用户可以选择以下操作：

- 如果要修改描述：在 描述 右侧点击 编辑，修改后点击 确认。
- 如果要修改属性：在 定义属性 区域，找到需要修改的属性，在右侧点击 编辑，修改后点击 确认。
- 如果要删除属性：在 定义属性 区域，找到需要删除的属性，在右侧点击 删除，然后点击 确认。
- 如果要添加属性：在 定义属性 区域，点击 + 添加属性，设置属性信息，点击 确认。详细说明参考[创建 Tag](#)。
- 如果要修改 TTL 信息：在 设置 TTL 区域，点击 编辑，修改后点击 确认。详情参考[TTL 配置](#)。
- 如果要删除已经配置的 TTL 信息：在 设置 TTL 区域，取消勾选 设置TTL（存活时间），然后点击 确定。
- 如果要配置 TTL 信息：在 设置 TTL 区域，勾选 设置TTL（存活时间），设置 `TTL_COL` 和 `TTL_DURATION`（单位：秒），点击 确认。详情参考[TTL 配置](#)。



Note

TTL 与索引的共存问题，详情参考[TTL](#)。

删除 TAG



Danger

删除 Tag 前先确认[影响](#)，已删除的数据如未备份无法恢复。

- 1. 在顶部导航栏中，点击 Schema 页签。
- 2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 Schema。
- 3. 在 当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
- 4. 点击 标签 页签，找到需要删除的 Tag，并在 操作 列中，点击 图标。
- 5. 在弹出的对话框中点击 确认。

后续操作

Tag 创建成功后，用户可以在 控制台 上逐条插入点数据，或者使用 导入 功能批量插入点数据。

最后更新: 2025年12月19日

操作 Edge type

在 TRSGraph 中创建图空间后，用户可能需要创建 Edge type（边类型）。用户可以选择使用 控制台 或者 Schema 操作 Edge type。本文仅说明如何使用 Schema 操作 Edge type。

前提条件

在 Studio 上操作 Edge type 之前，用户必须确认以下信息：

- Studio 已经连接到 TRSGraph。
- 图空间已经创建。
- 当前登录的账号拥有 GOD、ADMIN 或者 DBA 的权限。

创建边类型

1. 在顶部导航栏中，点击 Schema 页签。
2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 Schema。
3. 在 当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 边类型 页签，并点击 + 创建 按钮。
5. 在 创建边类型 页面上，完成以下设置：
  - a. 名称：输入 Edge type 名称。本示例中，输入 `serve`。
  - b. 描述（可选）：输入 Edge type 的备注。
  - c. 定义属性（可选）：如果 Edge type 需要属性，点击 + 添加属性，添加一个或多个属性：
    - 输入属性名称。
    - 选择数据类型。
    - 选择是否允许空值。
    - （可选）输入默认值。
    - （可选）输入属性备注。
  - d. 设置 TTL（存活时间）（可选）：Edge type 未设置索引时，用户可以设置 TTL。勾选设置 TTL（存活时间），设置 `TTL_COL` 和 `TTL_DURATION`（单位：秒）。详情参考 [TTL 配置](#)。
6. 完成设置后，在 对应的 nGQL 语句 面板上，用户能看到与上述配置等价的 nGQL 语句。

TRSGraph Studio

Schema

由 导入

回 控制台

Schema 草图

← 图空间列表 / 边类型列表 / 创建边类型

当前图空间: basketballplayer

\* 名称

serve

描述

☒ 定义属性

+ 添加属性

| * 属性名称     | * 数据类型 | 允许空值                                | 默认值 | 描述 |    |
|------------|--------|-------------------------------------|-----|----|----|
| start_year | int64  | <input checked="" type="checkbox"/> |     |    | 删除 |
| end_year   | int64  | <input checked="" type="checkbox"/> |     |    | 删除 |
| teamID     | string | <input checked="" type="checkbox"/> |     |    | 删除 |

☐ 设置TTL（存活时间）

取消

创建

7. 确认无误后，点击 **创建** 按钮。

如果 Edge Type 创建成功，边类型 面板会显示这个 Edge type 的属性列表。

#### 修改 EDGE TYPE

按以下步骤使用 Schema 修改 Edge type：

1. 在顶部导航栏中，点击 **Schema** 页签。
2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 **Schema**。
3. 在当前图空间 里确认图空间名称。用户也可以通过选择图空间名称完成图空间切换。
- 4.



点击 **边类型** 页签，找到需要修改的 Edge Type，并在 操作 列中，点击  图标。

5. 在 编辑 页面，用户可以选择以下操作：

- 如果要修改描述：在 描述 右侧点击 **编辑**，修改后点击 **确认**。
- 如果要修改属性：在 定义属性 区域，找到需要修改的属性，在右侧点击 **编辑**，修改后点击 **确认**。
- 如果要删除属性：在 定义属性 区域，找到需要删除的属性，在右侧点击 **删除**，然后点击 **确认**。
- 如果要添加属性：在 定义属性 区域，点击 **+ 添加属性**，设置属性信息，点击 **确认**。详细说明参考 [创建边类型](#)。
- 如果要修改 TTL 信息：在 设置 TTL 区域，点击 **编辑**，修改后点击 **确认**。详情参考 [TTL 配置](#)。
- 如果要删除已经配置的 TTL 信息：在 设置 TTL 区域，取消勾选 **设置TTL（存活时间）**，然后点击 **确定**。
- 如果要配置 TTL 信息：在 设置 TTL 区域，勾选 **设置TTL（存活时间）**，设置 **TTL\_COL** 和 **TTL\_DURATION**（单位：秒），点击 **确认**。详情参考 [TTL 配置](#)。



#### Note

TTL 与索引的共存问题，详情请见 [TTL](#)。

#### 删除 EDGE TYPE



#### Danger

删除 Edge type 前先确认 [影响](#)，已删除的数据如未备份。

1. 在顶部导航栏中，点击 **Schema** 页签。
2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 **Schema**。
3. 在当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 **边类型** 页签，找到需要修改的 Edge type，并在 操作 列中，点击  图标。
5. 在弹出的对话框中点击 **确认**。

#### 后续操作

Edge type 创建成功后，用户可以在 控制台 上逐条插入边数据，或者使用 **导入** 功能批量插入边数据。

最后更新: 2025年12月19日

## 操作索引

用户可以为 Tag 和 Edge type 创建索引，使得图查询时可以从拥有共同属性的同一类型的点或边开始遍历，使大型图的查询更为高效。用户可以选择使用 控制台 或者 Schema 操作索引。本文仅说明如何使用 Schema 操作索引。



一般在创建了 Tag 或者 Edge type 之后即可创建索引，但是，索引会影响写性能，所以，建议先导入数据，再批量重建索引。关于索引的详细信息，参考 [索引介绍](#)。

### 前提条件

在 Studio 上操作索引之前，用户必须确认以下信息：

- Studio 已经连接到 TRSGraph。
- 图空间、Tag 和 Edge type 已经创建。
- 当前登录的账号拥有 GOD、ADMIN 或者 DBA 的权限。

### 创建索引

1. 在顶部导航栏中，点击 Schema 页签。
2. 在 图空间列表 中，找到图空间，点击图空间名称或者在 操作 列中点击 Schema。
3. 在 当前图空间 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 索引 页签，并点击 + 创建 按钮。
5. 在 创建索引 页面上，完成以下设置：
  - a. 索引类型：选择索引类型，即 Tag 或者 Edge type。本示例中选择 Edge type。
  - b. 关联名称：选择要关联的 Tag 或 Edge type 名称。本示例中选择 follow。
  - c. 索引名称：输入索引名称。本示例中，输入 follow\_index。
  - d. 描述（可选）：输入索引的备注。
  - e. 索引属性（可选）：点击 + 添加属性，在弹出的对话框列表里选择需要关联的属性，点击 确认。如果需要关联多个属性，重复这一步骤。用户可以按界面提示重排索引属性的顺序。本示例中选择 degree。



索引属性的顺序会影响 `LOOKUP` 语句的查询结果。详细信息，参考 [LOOKUP](#)。

6. 完成设置后，在 对应的 nGQL 面板，用户能看到与上述配置等价的 nGQL 语句。

TRSGraph Studio Schema 由 导入 控制台 Schema 草图

← 图空间列表 / 索引列表 / 创建索引 当前图空间: basketballplayer

\* 索引类型  
Edge Type

\* 索引名称  
serve\_index

\* 关联 edge 名称  
serve

描述  
serve\_index

索引属性(可拖拽排序)  
+ 添加属性

对应的nGQL语句

```
1 CREATE EDGE INDEX `serve_index` on `serve` () COMMENT "serve_index"
```

取消 创建

7. 确认无误后，点击 **创建** 按钮。

如果索引创建成功，索引 面板会显示这个索引的属性列表。

[查看索引](#)

1. 在顶部导航栏中，点击 **Schema** 页签。
2. 在 **图空间列表** 中，找到图空间，点击图空间名称或者在 **操作** 列中点击 **Schema**。
3. 在 **当前图空间** 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 **索引** 页签，在列表左上方，选择需要查看的索引类型。
5. 在列表中，找到需要查看的索引，点击索引所在行即可展开查看该索引相关的所有属性名称、数据类型。

[重建索引](#)

1. 在顶部导航栏中，点击 **Schema** 页签。
2. 在 **图空间列表** 中，找到图空间，点击图空间名称或者在 **操作** 列中点击 **Schema**。
3. 在 **当前图空间** 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 **索引** 页签，在列表左上方，选择索引类型。
5. 找到需要重建的索引，在 **操作** 列 点击 **重建索引**。



Note

关于重建索引，详情参见 [REBUILD INDEX](#)。

[删除索引](#)

1. 在顶部导航栏中，点击 **Schema** 页签。
2. 在 **图空间列表** 中，找到图空间，点击图空间名称或者在 **操作** 列中点击 **Schema**。
3. 在 **当前图空间** 里确认图空间名称。用户也可以通过选择图空间名称切换图空间。
4. 点击 **索引** 页签，在列表左上方，选择索引类型。
5. 找到需要删除的索引，并在 **操作** 列中，点击 图标。

6. 在弹出的对话框中点击 确认。

---

最后更新: 2025年12月19日

## 查看 Schema

用户可以在 Studio 上可视化地查看 Schema。

### 操作步骤

1. 在顶部导航栏里，点击 Schema 页签。
2. 在图空间列表中，找到图空间，单击图空间名称或者在操作列中单击 Schema。
3. 单击查看 Schema 页签，单击获取 Schema。

### 其他操作

在图空间列表中，找到图空间，在操作列可以执行如下操作：

- 查看 Schema DDL：显示该图空间的 Schema 创建语句，包括图空间、Tag、Edge type 和索引。
- 克隆图空间：克隆该图空间的 Schema 到新的图空间。
- 删除图空间：删除该图空间，包括 Schema 和所有点边数据。

---

最后更新: 2025年12月19日

### 11.3.6 Schema 草图

Studio 支持 Schema 草图功能。用户可以在画板上自行设计 Schema，可以直观展示点边关系，设计完成后可以将 Schema 应用到指定图空间。

#### 功能说明

- 图形化设计 Schema。
- 应用 Schema 到指定图空间。
- 导出 Schema 为 PNG 格式图片。

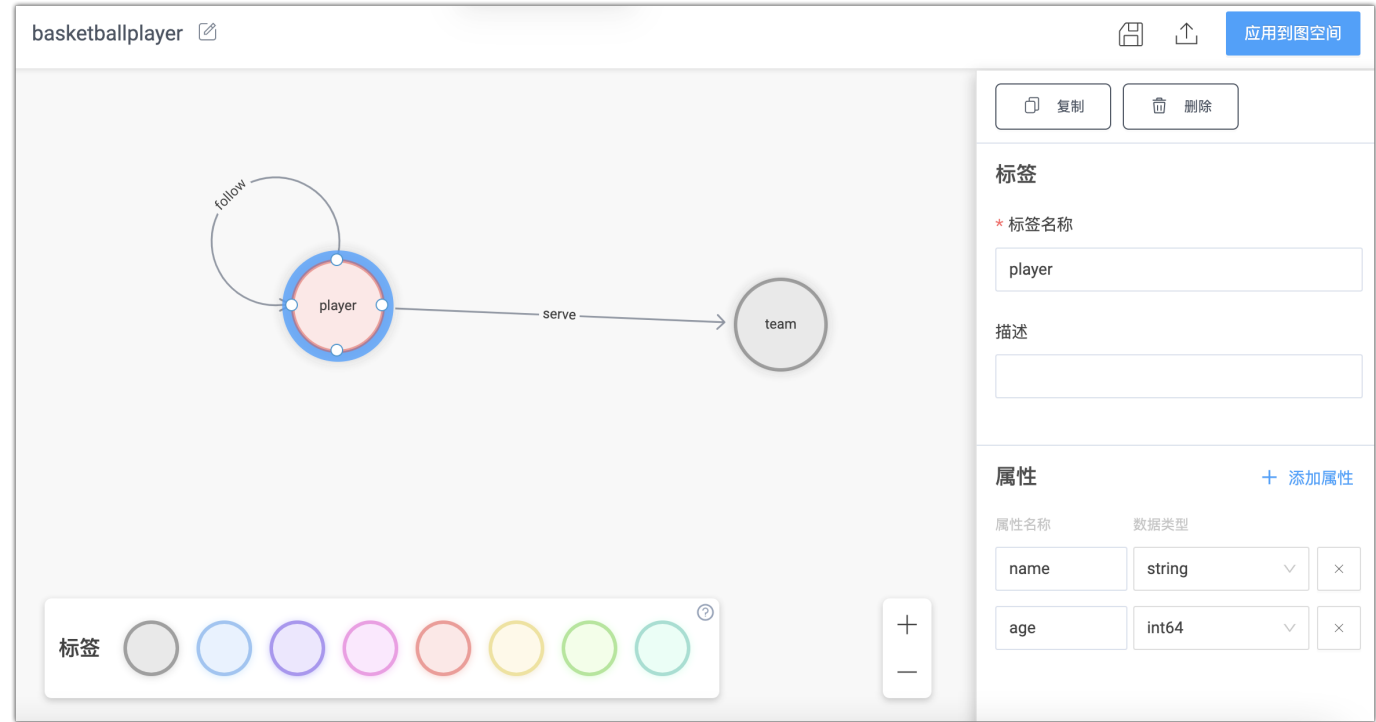
#### 入口

在顶部导航栏里，点击  图标。

#### 设计 Schema

以 `basketballplayer` 的 Schema 为例，说明如何设计 Schema。

1. 在页面左上角单击新建。
2. 在画布下方选择合适颜色的 Tag 标签，按住左键拖拽至画布中，创建一个 Tag。
3. 单击 Tag，在右侧填写标签名称 `player`、描述，并且添加属性 `name` 和 `age`。
4. 再次创建一个 Tag，标签名称为 `team`，属性为 `name`。
5. 从 Tag `player` 上的锚点连接至 Tag `team` 的锚点，单击生成的边，在右侧填写边类型名称 `serve`，并且添加属性 `start_year` 和 `end_year`。
6. 从 Tag `player` 上的锚点连接至自身另一个锚点，单击生成的边，在右侧填写边类型名称 `follow`，并且添加属性 `degree`。
7. 设计完成后，在页面上方单击  修改草图名称，然后在右上角单击  保存草图。



#### 应用 Schema

1. 在页面左侧的草图列表内选择需要导入的 Schema 草图，然后在右上角单击应用到图空间。



2. 选择将 Schema 导入新建图空间或者已创建的图空间，单击确认。

Note

- 创建图空间的参数说明参见[CREATE SPACE](#)。
- 如果图空间中已有重名 Schema，导入操作会失败，并提示修改名称或更换图空间。

修改 Schema

在页面左侧的草图列表内选择需要修改的 Schema 草图，修改完成后在右上角单击  保存。

Note

已应用 Schema 的图空间不会同步修改。

删除 Schema

在页面左侧的草图列表内找到需要删除的 Schema 草图，在缩略图右上角单击X并确认即可删除。

导出 Schema

在页面右上角单击  可以导出 Schema 为 PNG 格式图片。

最后更新: 2025年12月19日

## 11.4 全局设置

本文介绍 Studio 的全局设置，包含语言切换和 Beta 功能。

- 语言：支持切换中文和英文。
- Beta 功能：Beta 功能开关。Beta 功能目前包括[查看 Schema](#)、[文本转查询](#)和 [AI 导入](#)。  
文本转查询和 AI 导入需要设置人工智能相关的配置。详细设置参见下文。

### 11.4.1 文本转查询和 AI 导入

文本转查询和 AI 导入是基于大规模语言模型（LLM）实现的人工智能，需要配置以下参数才能正常使用。

| 参数       | 说明                                                                                                                                                                          |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API type | API 接口类型。支持 OpenAI 和 Aliyun。                                                                                                                                                |
| URL      | API URL。请按照对应接口类型填写正确的 URL 格式。例如 <code>https://{your-resource-name}.openai.azure.com/openai/deployments/{deployment-id}/chat/completions?api-version={api-version}</code> 。 |
| key      | 用于验证 API 的 key。使用在线大语言模型时必填，使用离线大语言模型时根据实际设置选填。                                                                                                                             |
| model    | 大语言模型版本。使用在线大语言模型时必填，使用离线大语言模型时根据实际设置选填。                                                                                                                                    |
| 文本最大长度   | 接收或生成一段文本的最大长度限制。单位：字节。                                                                                                                                                     |

最后更新: 2025年12月19日

## 11.5 故障排查

### 11.5.1 连接数据库错误

#### 问题描述

按[连接 Studio](#) 文档操作，提示 配置失败。

#### 可能的原因及解决方法

用户可以按以下步骤排查问题。

##### 第 1 步。确认 HOST 字段的格式是否正确

必须填写 TRSGraph 图数据库 Graph 服务的 IP 地址（`graph_server_ip`）和端口。如果未做修改，端口默认为 `9669`。即使 TRSGraph 与 Studio 都部署在当前机器上，用户也必须使用本机 IP 地址，而不能使用 `127.0.0.1`、`localhost` 或者 `0.0.0.0`。

##### 第 2 步。确认 用户名 和 密码 是否正确

如果 TRSGraph 没有开启身份认证，用户可以填写任意字符串登录。

如果已经开启身份认证，用户必须使用分配的账号登录。

##### 第 3 步。确认 TRSGRAPH 服务是否正常

检查 TRSGraph 服务状态。关于查看服务的操作：

- 如果在 Linux 服务器上通过编译部署的 TRSGraph，参考[查看 TRSGraph 服务](#)。
- 如果使用 Docker Compose 部署和 RPM 部署的 TRSGraph，参考[查看 TRSGraph 服务状态和端口](#)。

如果 TRSGraph 服务正常，进入第 4 步继续排查问题。否则，请重启 TRSGraph 服务。



Note

如果之前使用 `docker-compose up -d` 启动 TRSGraph，必须运行 `docker-compose down` 命令停止 TRSGraph。

##### 第 4 步。确认 GRAPH 服务的网络连接是否正常

在 Studio 机器上运行命令（例如 `telnet <graph_server_ip> 9669`）确认 TRSGraph 的 Graph 服务网络连接是否正常。

如果连接失败，则按以下要求检查：

- 如果 Studio 与 TRSGraph 在同一台机器上，检查端口是否已暴露。
- 如果两者不在同一台机器上，检查 TRSGraph 服务器的网络配置，例如，防火墙、网关以及端口。

最后更新: 2025年12月19日

## 11.5.2 无法访问 Studio

### 问题描述

我按照文档描述启动 Studio 后访问 `127.0.0.1:7001` 或者 `0.0.0.0:7001`，但是打不开页面，为什么？

### 可能的原因及解决方法

用户可以按以下顺序排查问题。

#### 1. 确认系统架构

需要确认部署 Studio 服务的机器是否为 x86\_64 架构。目前 Studio 仅支持 x86\_64 系统架构。

#### 2. 检查 STUDIO 服务是否正常启动

- 使用 tar 包部署的 Studio，使用 `sudo lsof -i:7001` 查看端口状态。

如果服务没有正常运行，请重新启动 Studio。详细信息，参考[部署 Studio](#)。

#### 3. 确认访问地址

如果 Studio 与浏览器在同一台机器上，用户可以在浏览器里使用 `localhost:7001`、`127.0.0.1:7001` 或者 `0.0.0.0:7001` 访问 Studio。

如果两者不在同一台机器上，必须在浏览器里输入 `<studio_server_ip>:7001`。其中，`studio_server_ip` 是指部署 Studio 服务的机器的 IP 地址。

#### 4. 确认网络连通性

运行 `curl <studio_server_ip>:7001 -I` 确认是否正常。如果返回 `HTTP/1.1 200 OK`，表示网络连通正常。

如果连接被拒绝，则按以下要求检查：

- 如果浏览器与 Studio 在同一台机器上，检查端口是否已暴露。
- 如果两者不在同一台机器上，检查 Studio 所在机器的网络配置，例如，防火墙、网关以及端口。

---

最后更新: 2025年12月19日

## 12. 常见问题 FAQ

本文列出了使用 TRSGraph 10.0 时可能遇到的常见问题，用户可以使用文档中心或者浏览器的搜索功能查找相应问题。

### 12.1 关于本手册

#### 12.1.1 为什么手册示例和系统行为不一致？



如果发现本文档中的错误：

1. 用户可以点击页面顶部右上角的"铅笔"图标进入编辑页面。
2. 使用 Markdown 修改文档。完成后点击页面底部的 "Commit changes"，这会触发一个 GitHub pull request。
3. 完成 CLA 签署，并且至少 2 位 reviewer 审核通过即可合并。

### 12.2 关于历史兼容性



TRSGraph 10.0 与 历史版本（包括 TRSGraph 1.x 和 2.x）的数据格式、客户端通信协议均双向不兼容。使用老版本客户端连接新版本服务端，会导致服务进程退出。数据格式升级参见[升级 TRSGraph 历史版本至当前版本](#)。客户端与工具均需要[下载对应版本](#)。

### 12.3 关于执行报错

#### 12.3.1 如何处理错误信息 `-1005:GraphMemoryExceeded: (-2600)`

这个错误是由 Memory Tracker 发出的，它在观察到内存使用超过了设定阈值时发出警报。这个机制可以帮助避免服务进程被系统的 OOM（Out of Memory）killer 终止。解决步骤：

1. 检查内存使用情况：首先，你需要查看命令执行过程中的内存用量。如果内存使用量确实很高，那么这个错误可能是预期中的情况。
2. 检查 Memory Tracker 的配置：如果内存使用量并不高，检查 Memory Tracker 的相关配置。这包括 `memory_tracker_untracked_reserved_memory_mb`（未被跟踪的预留内存量，单位为 MB），`memory_tracker_limit_ratio`（内存限制比例），以及 `memory_purge_enabled`（是否启用内存清理）。关于 Memory Tracker 的配置，参见[memory tracker 配置](#)。
3. 优化配置：根据实际情况，调整这些配置。例如，如果可用内存限制过低，可以调高 `memory_tracker_limit_ratio` 的值。

#### 12.3.2 如何处理错误信息 `SemanticError: Missing yield clause`

从 TRSGraph 3.0.0 开始，查询语句 `LOOKUP`、`GO`、`FETCH` 必须用 `YIELD` 子句指定输出结果。详情请参见[YIELD](#)。

#### 12.3.3 如何处理错误信息 `Host not enough!`

从 3.0.0 版本开始，在配置文件中添加的 Storage 节点无法直接读写，配置文件的作用仅仅是将 Storage 节点注册至 Meta 服务中。必须使用 `ADD HOSTS` 命令后，才能正常读写 Storage 节点。详情参见[管理 Storage 主机](#)。

### 12.3.4 如何处理错误信息 To get the property of the vertex in 'v.age', should use the format 'var.tag.prop'

从 3.0.0 版本开始，`pattern` 支持同时匹配多个 Tag，所以返回属性时，需要额外指定 Tag 名称。即从 `RETURN 变量名.属性名` 改为 `RETURN 变量名.Tag名.属性名`。

### 12.3.5 如何处理错误信息 Used memory hits the high watermark(0.800000) of total system memory.

报错原因：TRSGraph 的 `system_memory_high_watermark_ratio` 参数指定了内存高水位报警机制的触发阈值，默认为 0.8。系统内存占用率高于该值会触发报警机制，TRSGraph 会停止接受查询。

解决方案：

- 清理系统内存，使其降低到阈值以下。
- 修改 Graph 配置。在所有 Graph 服务器的配置文件中增加 `system_memory_high_watermark_ratio` 参数，为其设置一个大于 0.8 的值，例如 0.9。

需要注意的是 `system_memory_high_watermark_ratio` 参数已被弃用。建议用户使用 Memory Tracker 功能来限制 TRSGraph 的内存使用量。详情请参见 Graph 服务的 [memory tracker 配置](#) 和 Storage 服务的 [memory tracker 配置](#)。

### 12.3.6 如何处理错误信息 Storage Error E\_RPC\_FAILURE

报错原因通常为 Graph 服务向 Storage 服务请求了过多的数据，导致 Storage 服务超时。请尝试以下解决方案：

- 修改配置文件：在 `trsgraph-graphd.conf` 文件中修改 `--storage_client_timeout_ms` 参数的值，以增加 Storage client 的连接超时时间。该值的单位为毫秒（ms）。例如，设置 `--storage_client_timeout_ms=60000`。如果 `trsgraph-graphd.conf` 文件中未配置该参数，请手动增加。提示：请在配置文件开头添加 `--local_config=true` 再重启服务。
- 优化查询语句：减少全库扫描型的查询，无论是否用 `LIMIT` 限制了返回结果的数量；用 GO 语句改写 MATCH 语句（前者有优化，后者无优化）。
- 检查 Stored 是否发生过 OOM。（`dmesg |grep trsgraph`）。
- 为 Storage 服务器提供性能更好的 SSD 或者内存。
- 重试请求。

### 12.3.7 如何处理错误信息 The leader has changed. Try again later

已知问题，通常需要重试 1-N 次（N==partition 数量）。因为 meta client 更新 leader 缓存需要 1-2 个心跳或者通过错误触发强制更新。

如果登录 TRSGraph 时，出现该报错，可以考虑使用 `df -h` 查看磁盘空间，检查本地磁盘空间是否已满。

### 12.3.8 Schema not exist: xxx

查询时提示 `Schema not exist`，请确认：

- Schema 中是否存在该 Tag 或 Edge type。
- Tag 或 Edge type 的名称是否为关键字，如果是关键字，请使用反引号（```）将它们括起来。详情请参见[关键字](#)。

### 12.3.9 编译 Exchange、Connectors、Algorithm 时无法下载 SNAPSHOT 包

现象：编译时提示 `Could not find artifact com.vesoft:client:jar:xxx-SNAPSHOT`。

原因：本地 maven 没有配置用于下载 SNAPSHOT 的仓库。maven 中默认的 central 仓库用于存放正式发布版本，而不是开发版本（SNAPSHOT）。

解决方案：在 maven 的 `setting.xml` 文件的 `profiles` 作用域内增加以下配置：

```
<profile>
 <activation>
 <activeByDefault>true</activeByDefault>
 </activation>
 <repositories>
 <repository>
 <id>snapshots</id>
 <url>https://oss.sonatype.org/content/repositories/snapshots/</url>
```

```
<snapshots>
 <enabled>true</enabled>
</snapshots>
</repository>
</repositories>
</profile>
```

### 12.3.10 如何处理错误信息 [ERROR (-1004)]: SyntaxError: syntax error near?

大部分情况下，查询语句需要有 `YIELD` 或 `RETURN`，请检查查询语句是否包含。

### 12.3.11 如何处理错误信息 can't solve the start vids from the sentence

查询引擎需要知道从哪些 VID 开始图遍历。这些开始图遍历的 VID，或者通过用户指定，例如：

```
> GO FROM ${vids} ...
> MATCH (src) WHERE id(src) == ${vids}
开始图遍历的 VID 通过如上办法指定
```

或者通过一个属性索引来得到，例如：

```
CREATE TAG INDEX IF NOT EXISTS i_player ON player(name(20));
REBUILD TAG INDEX i_player;

> LOOKUP ON player WHERE player.name == "abc" | ... YIELD ...
> MATCH (src) WHERE src.name == "abc" ...
通过点属性 name 的索引，来得到 VID
```

否则，就会抛出这样一个异常 `can't solve the start vids from the sentence`。

### 12.3.12 如何处理错误信息 Wrong vertex id type: 1001

检查输入的 VID 类型是否是 `create space` 设置的 `INT64` 或 `FIXED_STRING(N)`。详情请参见 [create space](#)。

### 12.3.13 如何处理错误信息 The VID must be a 64-bit integer or a string fitting space vertex id length limit.

检查输入的 VID 是否超过限制长度。详情请参见 [create space](#)。

### 12.3.14 如何处理错误信息 edge conflict 或 vertex conflict

Storage 服务在毫秒级时间内多次收到插入或者更新同一点或边的请求时，可能返回该错误。请稍后重试。

### 12.3.15 如何处理错误信息 RPC failure in MetaClient: Connection refused

报错原因通常为 metad 服务状态异常，或是 metad 和 graphd 服务所在机器网络不通。请尝试以下解决方案：

- 在 metad 所在服务器查看下 metad 服务状态，如果服务状态异常，可以重新启动 metad 服务。
- 在报错服务器下使用 `telnet meta-ip:port` 查看网络状态。
- 检查配置文件中的端口配置，如果端口号与连接时使用的不同，改用配置文件中的端口或者修改配置。

### 12.3.16 如何处理 trsgraph-graph.INFO 中错误日志 StorageClientBase.inl:214] Request to "x.x.x.x":9779

```
failed: N6apache6thrift9transport19TTransportExceptionE: Timed Out
```

报错原因可能是查询的数据量比较大，storage 处理超时。请尝试以下解决方法：

- 导入数据时，手动 `compaction`，加速读的速度。
- 增加 Graph 服务与 Storage 服务的 RPC 连接超时时间，在 `trsgraph-graphd.conf` 文件里面修改 `--storage_client_timeout_ms` 参数的值。该值的单位为毫秒（ms），默认值为 60000 毫秒。

12.3.17 如何处理 `trsgraph-storaged.INFO` 中错误日志 `MetaClient.cpp:65] Heartbeat failed, status:Wrong cluster!` 或者 `trsgraph-metad.INFO` 含有错误日志 `HBProcessor.cpp:54] Reject wrong cluster host "x.x.x.x":9771!`

报错的原因可能是用户修改了 metad 的 ip 或者端口信息，或者 storage 之前加入过其他集群。请尝试以下解决方法：

用户到 storage 部署的机器所在的安装目录（默认安装目录为 `/usr/local/trsgraph`）下面将 `cluster.id` 文件删除，然后重启 storaged 服务。

12.3.18 如何处理报错信息 `Storage Error: More than one request trying to add/update/delete one edge/vertex at the same time.`

上述报错表示当前版本不允许同时对一个点或者一条边进行并发操作。如出现此报错请重新执行操作命令。

12.3.19 如何处理错误信息 `The maximum number of statements allowed has been exceeded` 和 `the maximum number of statements for trsgraph is 512`

这两个报错信息都是由于单条插入语句长度超过 `max_allowed_statements` 引起的，通过修改 `trsgraph-graphd.conf` 配置文件的 `etc` 目录下 `--max_allowed_statements` 参数项，能避免语句插入报错。

## 12.4 关于设计与功能

### 12.4.1 返回消息中 `time spent` 的含义是什么？

将命令 `SHOW SPACES` 返回的消息作为示例：

```
trsgraph> SHOW SPACES;
+-----+
| Name |
+-----+
| "basketballplayer" |
+-----+
Got 1 rows (time spent 1235/1934 us)
```

- 第一个数字 `1235` 表示数据库本身执行该命令花费的时间，即查询引擎从客户端接收到一个查询，然后从存储服务器获取数据并执行一系列计算所花费的时间。
- 第二个数字 `1934` 表示从客户端角度看所花费的时间，即从客户端发送请求、接收结果，然后在屏幕上显示结果所花费的时间。

### 12.4.2 为什么在正常连接 TRSGraph 后，`trsgraph-storaged` 进程的端口号一直显示红色？

`trsgraph-storaged` 进程的端口号的红色闪烁状态是因为 `trsgraph-storaged` 在启动流程中会等待 `trsgraph-metad` 添加当前 Storage 服务，当前 Storage 服务收到 Ready 信号后才会正式启动服务。从 3.0.0 版本开始，Meta 服务无法直接读写在配置文件中添加的 Storage 服务，配置文件的作用仅仅是将 Storage 服务注册至 Meta 服务中。用户必须使用 `ADD HOSTS` 命令后，才能使 Meta 服务正常读写 Storage 服务。更多信息，参见[管理 Storage 主机](#)。

### 12.4.3 为什么 TRSGraph 的返回结果每行之间没有横线分隔了？

这是 TRSGraph Console 2.6.0 版本的变动造成的，不是 TRSGraph 内核的变更，不影响返回数据本身的内容。

### 12.4.4 关于悬挂边

悬挂边 (Dangling edge) 是指一条边的起点或者终点在数据库中不存在。

TRSGraph 10.0 的数据模型中，由于设计允许图中存在“悬挂边”；没有 openCypher 中的 MERGE 语句。对于悬挂边的保证完全依赖应用层面。详见 [INSERT VERTEX](#), [DELETE VERTEX](#), [INSERT EDGE](#), [DELETE EDGE](#)。



### 12.4.5 可以在 CREATE SPACE 时设置 replica\_factor 为偶数（例如设置为 2）吗？

不要这样设置。

Storage 服务使用 Raft 协议（多数表决），为保证可用性，要求出故障的副本数量不能达到一半。

当机器数量为 1 时，replica\_factor 只能设置为 1。

当机器数量足够时，如果 replica\_factor=2，当其中一个副本故障时，就会导致系统无法正常工作；如果 replica\_factor=4，只能有一个副本可以出现故障，这和 replica\_factor=3 是一样。以此类推，所以 replica\_factor 设置为奇数即可。

建议在生产环境中设置 replica\_factor=3，测试环境中设置 replica\_factor=1，不要使用偶数。

### 12.4.6 是否支持停止或者中断慢查询

支持。

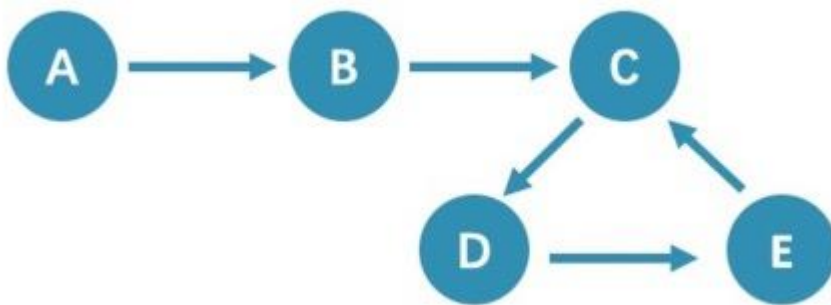
详情请参见[终止查询](#)。

### 12.4.7 使用 GO 和 MATCH 执行相同语义的查询，查询结果为什么不同？

原因可能有以下几种：

- GO 查询到了悬挂边。
- RETURN 命令未指定排序方式。
- 触发了 Storage 服务中 max\_edge\_returned\_per\_vertex 定义的稠密点截断限制。
- 路径的类型不同，导致查询结果可能会不同。
- GO 语句采用的是 walk 类型，遍历时点和边可以重复。
- MATCH 语句兼容 openCypher，采用的是 trail 类型，遍历时只有点可以重复，边不可以重复。

因路径类型不同导致查询结果不同的示例图和说明如下。



从点 A 开始查询距离 5 跳的点，都会查询到点 C（A->B->C->D->E->C），查询 6 跳的点时，GO 语句会查询到点 D（A->B->C->D->E->C->D），因为边 C->D 可以重复查询，而 MATCH 语句查询为空，因为边不可以重复。

所以使用 GO 和 MATCH 执行相同语义的查询，可能会出现 MATCH 语句的查询结果比 GO 语句少。

关于路径的详细说明，请参见[维基百科](#)。

### 12.4.8 如何统计每种 Tag 有多少个点，每个 Edge type 有多少条边？

请参见 [show-stats](#)。

## 12.4.9 如何获取每种 Tag 的所有点，或者每种 Edge type 的所有边？

### 1. 建立并重建索引。

```
> CREATE TAG INDEX IF NOT EXISTS i_player ON player();
> REBUILD TAG INDEX i_player;
```

### 2. 使用 LOOKUP 或 MATCH 语句。例如：

```
> LOOKUP ON player;
> MATCH (n:player) RETURN n;
```

更多详情请参见 [INDEX](#)、[LOOKUP](#) 和 [MATCH](#)。

## 12.4.10 能不能用中文字符做标识符，比如图空间、Tag、Edge type、属性、索引的名称？

能，详情参见[关键字](#)和[保留字](#)。

## 12.4.11 获取指定点的出度（或者入度）？

一个点的“出度”是指从该点出发的“边”的条数。入度，是指指向该点的边的条数。

```
trsgraph > MATCH (s)-[e]->() WHERE id(s) == "given" RETURN count(e); #出度
trsgraph > MATCH (s)<-[e]-() WHERE id(s) == "given" RETURN count(e); #入度
```

因为没有对出度和入度的进行特殊处理（例如索引或者缓存），这是一个较慢的操作。特别当遇到[超级节点](#)时，可能会发生OOM。

## 12.4.12 是否有办法快速获取“所有”点的出度和入度？

没有直接命令。

可以使用 [TRSGraph Algorithm](#)。

## 12.5 关于运维

### 12.5.1 运行日志文件过大时如何回收日志？

TRSGraph使用 [glog](#) 打印日志，glog 没有日志回收的功能。用户可以通过定时任务或日志管理工具 [logrotate](#) 来管理运行日志。操作详情，参见[回收日志](#)。

### 12.5.2 如何查看 TRSGraph 版本

服务运行时：`trsgraph-console` 中执行命令 `SHOW HOSTS META`，详见 [SHOW HOSTS](#)

服务未运行时：在安装路径的 `bin` 目录内，执行 `./<binary_name> --version` 命令，可以查看到 version 和 GitHub 上的 commit ID，例如：

```
$./trsgraph-graphd--version
```

### 12.5.3 修改 Host 名称后，旧的 Host 一直显示 OFFLINE 怎么办？

`OFFLINE` 状态的 Host 将在一天后自动删除。

## 12.5.4 如何查看 dmp 文件？

dmp 文件是错误报告文件，详细记录了进程退出的信息，可以用操作系统自带的 gdb 工具查看。Coredump 文件默认保存在启动二进制的当前目录下（默认为 `/usr/local/trsgraph` 目录），在 TRSGraph 服务 crash 时，系统会自动生成。

1. 查看 Core 文件进程名字，pid 一般为数值。

```
$ file core.<pid>
```

2. 使用 gdb 调试。

```
$ gdb <process.name> core.<pid>
```

3. 查看文件内容。

```
$(gdb) bt
```

例如：

```
$ file core.1316027
core.1316027: ELF 64-bit LSB core file, x86-64, version 1 (SYSV), SVR4-style, from '/home/workspace/fork/trsgraph-debug/bin/trsgraph-metad --flagfile /home/k', realuid: 1008, effectiveuid: 1008, realgid: 1008, effectivegid: 1008, execfn: '/home/workspace/fork/trsgraph-debug/bin/trsgraph-metad', platform: 'x86_64'

$ gdb /home/workspace/fork/trsgraph-debug/bin/trsgraph-metad core.1316027

$(gdb) bt
#0 0x00007f9de58fecf5 in __memcpy_ssse3_back () from /lib64/libc.so.6
#1 0x000000000eb2299 in void std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>>::_M_construct<char*>(char*, char*, std::forward_iterator_tag) ()
#2 0x000000000ef71a7 in nebula::meta::cpp2::QueryDesc::QueryDesc(nebula::meta::cpp2::QueryDesc const&) ()
...
```

## 12.5.5 如何通过 systemctl 设置开机自动启动 TRSGraph 服务？

1. 执行 `systemctl enable` 启动 metad、graphd 和 stored 服务。

```
[root]# systemctl enable trsgraph-metad.service
Created symlink from /etc/systemd/system/multi-user.target.wants/trsgraph-metad.service to /usr/lib/systemd/system/trsgraph-metad.service.
[root]# systemctl enable trsgraph-graphd.service
Created symlink from /etc/systemd/system/multi-user.target.wants/trsgraph-graphd.service to /usr/lib/systemd/system/trsgraph-graphd.service.
[root]# systemctl enable trsgraph-stored.service
Created symlink from /etc/systemd/system/multi-user.target.wants/trsgraph-stored.service to /usr/lib/systemd/system/trsgraph-stored.service.
```

2. 配置 metad、graphd 和 stored 的 service 文件，设置服务自动拉起。



Caution

在配置 service 文件时需注意以下几点：- PIDFile、ExecStart、ExecReload、ExecStop 参数的路径需要与服务器上的一致。- RestartSec 为重启前等待的时长(秒)，可根据实际情况修改。- （可选）StartLimitInterval 为无限次重启，默认是 10 秒内如果重启超过 5 次则不再重启，设置为 0 表示不限次数重启。- （可选）LimitNOFILE 为服务最大文件打开数，默认为 1024，可根据实际情况修改。

配置 metad 服务的 service 文件：

```
$ vi /usr/lib/systemd/system/trsgraph-metad.service

[Unit]
Description=TRS Graph Metad Service
After=network.target

[Service]
Type=forking
Restart=always
RestartSec=15s
PIDFile=/usr/local/trsgraph/pids/trsgraph-metad.pid
ExecStart=/usr/local/trsgraph/scripts/trsgraph.service start metad
ExecReload=/usr/local/trsgraph/scripts/trsgraph.service restart metad
ExecStop=/usr/local/trsgraph/scripts/trsgraph.service stop metad
PrivateTmp=true
StartLimitInterval=0
LimitNOFILE=1024

[Install]
WantedBy=multi-user.target
```

配置 graphd 服务的 service 文件：

```
$ vi /usr/lib/systemd/system/trsgraph-graphd.service
[Unit]
Description=TRS Graph Graphd Service
After=network.target

[Service]
Type=forking
Restart=always
RestartSec=15s
PIDFile=/usr/local/trsgraph/pids/trsgraph-graphd.pid
ExecStart=/usr/local/trsgraph/scripts/trsgraph.service start graphd
ExecReload=/usr/local/trsgraph/scripts/trsgraph.service restart graphd
ExecStop=/usr/local/trsgraph/scripts/trsgraph.service stop graphd
PrivateTmp=true
StartLimitInterval=0
LimitNOFILE=1024

[Install]
WantedBy=multi-user.target
```

配置 stored 服务的 service 文件：

```
$ vi /usr/lib/systemd/system/trsgraph-stored.service
[Unit]
Description=TRS Graph Stored Service
After=network.target

[Service]
Type=forking
Restart=always
RestartSec=15s
PIDFile=/usr/local/trsgraph/pids/trsgraph-stored.pid
ExecStart=/usr/local/trsgraph/scripts/trsgraph.service start stored
ExecReload=/usr/local/trsgraph/scripts/trsgraph.service restart stored
ExecStop=/usr/local/trsgraph/scripts/trsgraph.service stop stored
PrivateTmp=true
StartLimitInterval=0
LimitNOFILE=1024

[Install]
WantedBy=multi-user.target
```

3. reload 配置文件。

```
[root]# sudo systemctl daemon-reload
```

4. 重启服务。

```
$ systemctl restart trsgraph-metad.service
$ systemctl restart trsgraph-graphd.service
$ systemctl restart trsgraph-stored.service
```

## 12.6 关于连接

### 12.6.1 防火墙中需要开放哪些端口？

如果没有修改过[配置文件](#)中预设的端口，请在防火墙中开放如下端口：

服务类型	端口
Meta	9559, 9560, 19559
Graph	9669, 19669
Storage	9777 ~ 9780, 19779

如果修改过配置文件中预设的端口，请找出实际使用的端口并在防火墙中开放它们。

更多端口信息，参见[公司产品端口全集](#)。

## 12.6.2 如何测试端口是否已开放？

用户可以使用如下 telnet 命令检查端口状态：

```
telnet <ip> <port>
```



如果无法使用 telnet 命令，请先检查主机中是否安装并启动了 telnet。

示例：

```
// 如果端口已开放：
$ telnet 192.168.1.10 9669
Trying 192.168.1.10...
Connected to 192.168.1.10.
Escape character is '^['.

// 如果端口未开放：
$ telnet 192.168.1.10 9777
Trying 192.168.1.10...
telnet: connect to address 192.168.1.10: Connection refused
```

## 12.6.3 会话数量达到 max\_sessions\_per\_ip\_per\_user 所设值上限时怎么办？

当会话数量达到上限时，即超过 max\_sessions\_per\_ip\_per\_user 所设值，在连接 TRSGraph 的时候，会出现以下报错：

```
Fail to create a new session from connection pool, fail to authenticate, error: Create Session failed: Too many sessions created from 127.0.0.1 by user root. the threshold is 2. You can change it by modifying 'max_sessions_per_ip_per_user' in trsgraph-graphd.conf
```

用户可以通过以下三种方式解决问题：

1. 执行 `KILL SESSION` 命令，关闭不需要的 Session。详情参见[终止会话](#)。
2. 在 Graph 服务的配置文件 `trsgraph_graphd.conf` 中增大 `max_sessions_per_ip_per_user` 的值，增加可以创建的会话总数。重启 Graph 服务使配置生效。
3. 在 Graph 服务的配置文件 `trsgraph_graphd.conf` 中减小 `session_idle_timeout_secs` 参数的值，降低空闲会话的超时时间。重启 Graph 服务使配置生效。

## 12.7 关于扩容、缩容



集群扩缩容功能未正式在社区版中发布。以下涉及 `SUBMIT JOB BALANCE DATA REMOVE` 和 `SUBMIT JOB BALANCE DATA` 的操作在社区版中均为实验性功能，功能未稳定。如需使用，请先做好数据备份，并且在 Graph 配置文件中将 `enable_experimental_feature` 和 `enable_data_balance` 均设置为 `true`。

### 12.7.1 如何增加或减少 Meta、Graph、Storage 节点的数量

- TRSGraph 10.0 未提供运维命令以实现自动扩缩容，但可参考以下步骤实现手动扩缩容：
- Meta 的扩容和缩容：Meta 不支持自动扩缩容。



Note

用户可以使用脚本工具迁移 Meta 服务，但是需要自行修改 Graph 服务和 Storage 服务的配置文件中的 Meta 设置。

- Graph 的缩容：将该 Graph 的 IP 从 client 的代码中移除，关闭该 Graph 进程。
- Graph 的扩容：在新机器上准备 Graph 二进制文件和配置文件，在配置文件中修改或增加已在运行的 Meta 地址，启动 Graph 进程。
- Storage 的缩容：执行 `SUBMIT JOB BALANCE DATA REMOVE <ip:port>` 和 `DROP HOSTS <ip:port>`。完成后关闭 Storage 进程。



Caution

- 执行该命令迁移指定 Storage 节点中的分片数据前，需要确保其他 Storage 节点数量足够以满足设置的副本数。例如，如果设置了副本数为 3，那么在执行该命令前，需要确保其他 Storage 节点数量大于等于 3。
- 如果待迁移的 Storage 节点中有多个图空间的分片，需要分别在每个图空间内执行该命令以迁移该 Storage 节点中的所有分片。

- Storage 的扩容：在新机器上准备 Storage 二进制文件和配置文件，在配置文件中修改或增加已在运行的 Meta 地址，然后注册 Storage 到 Meta 并启动 Storage 进程。详情参见注册 Storage 服务。

Storage 扩缩容之后，根据需要执行 `SUBMIT JOB BALANCE DATA` 将当前图空间的分片平均分配到所有 Storage 节点中和执行 `SUBMIT JOB BALANCE LEADER` 命令均衡分布所有图空间中的 leader。运行命令前，需要选择一个图空间。

### 12.7.2 如何在 Storage 节点中增加或减少磁盘

目前，Storage 不支持动态识别新的磁盘。可以通过以下步骤在分布式集群的 Storage 节点中增加或减少磁盘（不支持对单个 Storage 节点的集群增加或减少磁盘）：

1. 执行 `SUBMIT JOB BALANCE DATA REMOVE <ip:port>` 将待增加或减少磁盘的 Storage 节点迁到其他 Storage 节点中。



Caution

- 执行该命令迁移指定 Storage 节点中的分片数据前，需要确保其他 Storage 节点数量足够以满足设置的副本数。例如，如果设置了副本数为 3，那么在执行该命令前，需要确保其他 Storage 节点数量大于等于 3。
- 如果待迁移的 Storage 节点中有多个图空间的分片，需要分别在每个图空间内执行该命令以迁移该 Storage 节点中的所有分片。

2. 执行 `DROP HOSTS <ip:port>` 移除待增加或减少磁盘的 Storage 节点。
3. 在所有 Storage 节点配置文件中通过 `--data_path` 配置新增或减少磁盘路径，详情参见 Storage 配置文件。
4. 执行 `ADD HOSTS <ip:port>` 重新添加待增加或减少磁盘的 Storage 节点。
5. 根据需要执行 `SUBMIT JOB BALANCE DATA` 将当前图空间的分片平均分配到所有 Storage 节点中和执行 `SUBMIT JOB BALANCE LEADER` 命令均衡分布所有图空间中的 leader。运行命令前，需要选择一个图空间。

最后更新: 2025年12月19日

## 13. 附录

---

### 13.1 错误码

---

TRSGraph 运行出现问题时，会返回错误码。本文介绍错误码的详细信息。



Note

- 如果出现错误但没有返回错误码，或错误码描述不清，请在[论坛](#)或 [GitHub](#) 反馈。
- 返回 0 表示执行成功。



错误名称	错误码	说明
E_DISCONNECTED	-1	连接断开
E_FAIL_TO_CONNECT	-2	无法建立连接
E_RPC_FAILURE	-3	RPC 失败
E_LEADER_CHANGED	-4	Raft leader 变更
E_SPACE_NOT_FOUND	-5	图空间不存在
E_TAG_NOT_FOUND	-6	Tag 不存在
E_EDGE_NOT_FOUND	-7	Edge type不存在
E_INDEX_NOT_FOUND	-8	索引不存在
E_EDGE_PROP_NOT_FOUND	-9	边属性不存在
E_TAG_PROP_NOT_FOUND	-10	Tag 属性不存在
E_ROLE_NOT_FOUND	-11	当前角色不存在
E_CONFIG_NOT_FOUND	-12	当前配置不存在
E_MACHINE_NOT_FOUND	-13	当前主机不存在
E_LISTENER_NOT_FOUND	-15	listener 不存在
E_PART_NOT_FOUND	-16	当前分区不存在
E_KEY_NOT_FOUND	-17	key 不存在
E_USER_NOT_FOUND	-18	用户不存在
E_STATS_NOT_FOUND	-19	统计信息不存在
E_SERVICE_NOT_FOUND	-20	没有找到当前服务
E_DRAINER_NOT_FOUND	-21	drainer 不存在
E_DRAINER_CLIENT_NOT_FOUND	-22	drainer 客户端不存在
E_PART_STOPPED	-23	当前 partition 已经被停止
E_BACKUP_FAILED	-24	备份失败
E_BACKUP_EMPTY_TABLE	-25	备份的表为空
E_BACKUP_TABLE_FAILED	-26	备份表失败
E_PARTIAL_RESULT	-27	multiget 无法获得所有数据
E_REBUILD_INDEX_FAILED	-28	重建索引失败
E_INVALID_PASSWORD	-29	密码无效
E_FAILED_GET_ABS_PATH	-30	无法获得绝对路径
E_BAD_USERNAME_PASSWORD	-1001	身份验证失败
E_SESSION_INVALID	-1002	无效会话
E_SESSION_TIMEOUT	-1003	会话超时
E_SYNTAX_ERROR	-1004	语法错误
E_EXECUTION_ERROR	-1005	执行错误

错误名称	错误码	说明
E_STATEMENT_EMPTY	-1006	语句为空
E_BAD_PERMISSION	-1008	权限不足
E_SEMANTIC_ERROR	-1009	语义错误
E_TOO_MANY_CONNECTIONS	-1010	超出最大连接数
E_PARTIAL_SUCCEEDED	-1011	访问存储失败（仅有部分请求成功）
E_NO_HOSTS	-2001	主机不存在
E_EXISTED	-2002	主机已经存在
E_INVALID_HOST	-2003	无效主机
E_UNSUPPORTED	-2004	当前命令、语句、功能不支持
E_NOT_DROP	-2005	不允许删除
E_CONFIG_IMMUTABLE	-2007	配置项不能改变
E_CONFLICT	-2008	参数与 meta 数据冲突
E_INVALID_PARM	-2009	无效的参数
E_WRONGCLUSTER	-2010	错误的集群
E_ZONE_NOT_ENOUGH	-2011	listener 冲突
E_ZONE_IS_EMPTY	-2012	主机不存在
E_SCHEMA_NAME_EXISTS	-2013	Schema 名字已存在
E_RELATED_INDEX_EXISTS	-2014	与 Tag 或 Edge Type 相关的索引存在，不能被删除
E_RELATED_SPACE_EXISTS	-2015	仍有图空间在主机上，不能被删除
E_STORE_FAILURE	-2021	存储数据失败
E_STORE_SEGMENT_ILLEGAL	-2022	存储段非法
E_BAD_BALANCE_PLAN	-2023	无效的数据均衡计划
E_BALANCED	-2024	集群已经处于数据均衡状态
E_NO_RUNNING_BALANCE_PLAN	-2025	没有正在运行的数据均衡计划
E_NO_VALID_HOST	-2026	缺少有效的主机
E_CORRUPTED_BALANCE_PLAN	-2027	已经损坏的数据均衡计划
E_IMPROPER_ROLE	-2030	回收用户角色失败
E_INVALID_PARTITION_NUM	-2031	无效的分区数量
E_INVALID_REPLICA_FACTOR	-2032	无效的副本因子
E_INVALID_CHARSET	-2033	无效的字符集
E_INVALID_COLLATE	-2034	无效的字符排序规则
E_CHARSET_COLLATE_NOT_MATCH	-2035	字符集和字符排序规则不匹配
E_SNAPSHOT_FAILURE	-2040	生成快照失败
E_BLOCK_WRITE_FAILURE	-2041	写入块数据失败

错误名称	错误码	说明
E_ADD_JOB_FAILURE	-2044	增加新的任务失败
E_STOP_JOB_FAILURE	-2045	停止任务失败
E_SAVE_JOB_FAILURE	-2046	保存任务信息失败
E_BALANCER_FAILURE	-2047	数据均衡失败
E_JOB_NOT_FINISHED	-2048	当前任务还没有完成
E_TASK_REPORT_OUT_DATE	-2049	任务报表失效
E_JOB_NOT_IN_SPACE	-2050	当前任务不在图空间内
E_JOB_NEED_RECOVER	-2051	当前任务需要恢复
E_JOB_ALREADY_FINISH	-2052	任务已经失败或完成
E_JOB_SUBMITTED	-2053	任务默认状态
E_JOB_NOT_STOPPABLE	-2054	给定任务不支持停止
E_JOB_HAS_NO_TARGET_STORAGE	-2055	leader 分布未上报，因此无法将任务发送到存储
E_INVALID_JOB	-2065	无效的任务
E_BACKUP_BUILDING_INDEX	-2066	备份终止（正在创建索引）
E_BACKUP_SPACE_NOT_FOUND	-2067	备份时图空间不存在
E_RESTORE_FAILURE	-2068	备份恢复失败
E_SESSION_NOT_FOUND	-2069	会话不存在
E_LIST_CLUSTER_FAILURE	-2070	获取集群信息失败
E_LIST_CLUSTER_GET_ABS_PATH_FAILURE	-2071	获取集群信息时无法获取绝对路径
E_LIST_CLUSTER_NO_AGENT_FAILURE	-2072	获取集群信息时无法获得 agent
E_QUERY_NOT_FOUND	-2073	query 未找到
E_AGENT_HB_FAILURE	-2074	agent 没有汇报心跳
E_HOST_CAN_NOT_BE_ADDED	-2082	该主机不能被添加，因为它不是一个 Storage 主机
E_ACCESS_ES_FAILURE	-2090	访问 Elasticsearch 失败
E_GRAPH_MEMORY_EXCEEDED	-2600	Graph 内存超出
E_CONSENSUS_ERROR	-3001	选举时无法达成共识
E_KEY_HAS_EXISTS	-3002	key 已经存在
E_DATA_TYPE_MISMATCH	-3003	数据类型不匹配
E_INVALID_FIELD_VALUE	-3004	无效的字段值
E_INVALID_OPERATION	-3005	无效的操作
E_NOT_NULLABLE	-3006	当前值不允许为空
E_FIELD_UNSET	-3007	字段非空或者没有默认值时，字段值必须设置
E_OUT_OF_RANGE	-3008	取值超出了当前类型的范围
E_DATA_CONFLICT_ERROR	-3010	数据冲突

错误名称	错误码	说明
E_WRITE_STALLED	-3011	写入被延迟
E_IMPROPER_DATA_TYPE	-3021	不正确的数据类型
E_INVALID_SPACEVIDLEN	-3022	VID 长度无效
E_INVALID_FILTER	-3031	无效的过滤器
E_INVALID_UPDATER	-3032	无效的字段更新
E_INVALID_STORE	-3033	无效的 KV 存储
E_INVALID_PEER	-3034	peer 无效
E_RETRY_EXHAUSTED	-3035	重试次数耗光
E_TRANSFER_LEADER_FAILED	-3036	leader 转换失败
E_INVALID_STAT_TYPE	-3037	无效的统计类型
E_INVALID_VID	-3038	VID 无效
E_LOAD_META_FAILED	-3040	加载元信息失败
E_FAILED_TO_CHECKPOINT	-3041	生成 checkpoint 失败
E_CHECKPOINT_BLOCKED	-3042	生成 checkpoint 被阻塞
E_FILTER_OUT	-3043	数据被过滤
E_INVALID_DATA	-3044	无效的数据
E_MUTATE_EDGE_CONFLICT	-3045	并发写入同一条边发生冲突
E_MUTATE_TAG_CONFLICT	-3046	并发写入同一个点发生冲突
E_OUTDATED_LOCK	-3047	锁已经失效
E_INVALID_TASK_PARA	-3051	无效的任务参数
E_USER_CANCEL	-3052	用户取消了任务
E_TASK_EXECUTION_FAILED	-3053	任务执行失败
E_PLAN_IS_KILLED	-3060	执行计划被清除
E_NO_TERM	-3070	收到请求时心跳流程未完成
E_OUTDATED_TERM	-3071	收到旧 leader 的过时心跳（已选举出新的 leader）
E_WRITE_WRITE_CONFLICT	-3073	并发写入时与后到的请求发生冲突
E_RAFT_UNKNOWN_PART	-3500	未知的分区
E_RAFT_LOG_GAP	-3501	raft 日志落后
E_RAFT_LOG_STALE	-3502	raft 日志过期
E_RAFT_TERM_OUT_OF_DATE	-3503	心跳信息已经过期
E_RAFT_UNKNOWN_APPEND_LOG	-3504	未知的追加日志
E_RAFT_WAITING_SNAPSHOT	-3511	等待快照完成
E_RAFT_SENDING_SNAPSHOT	-3512	发送快照过程出错
E_RAFT_INVALID_PEER	-3513	无效的接收端

错误名称	错误码	说明
E_RAFT_NOT_READY	-3514	Raft 没有启动
E_RAFT_STOPPED	-3515	Raft 已经停止
E_RAFT_BAD_ROLE	-3516	错误的角色
E_RAFT_WAL_FAIL	-3521	写入 WAL 失败
E_RAFT_HOST_STOPPED	-3522	主机已经停止
E_RAFT_TOO_MANY_REQUESTS	-3523	请求数量过多
E_RAFT_PERSIST_SNAPSHOT_FAILED	-3524	持久化快照失败
E_RAFT_RPC_EXCEPTION	-3525	RPC 异常
E_RAFT_NO_WAL_FOUND	-3526	没有发现 WAL 日志
E_RAFT_HOST_PAUSED	-3527	主机暂停
E_RAFT_WRITE_BLOCKED	-3528	写入被堵塞
E_RAFT_BUFFER_OVERFLOW	-3529	缓存溢出
E_RAFT_ATOMIC_OP_FAILED	-3530	原子操作失败
E_LEADER_LEASE_FAILED	-3531	leader 租约过期
E_RAFT_CAUGHT_UP	-3532	Raft 已经同步数据
E_STORAGE_MEMORY_EXCEEDED	-3600	Storage 内存超出
E_LOG_GAP	-4001	drainer 日志落后
E_LOG_STALE	-4002	drainer 日志过期
E_INVALID_DRAINER_STORE	-4003	drainer 数据存储无效
E_SPACE_MISMATCH	-4004	图空间不匹配
E_PART_MISMATCH	-4005	分区不匹配
E_DATA_CONFLICT	-4006	数据冲突
E_REQ_CONFLICT	-4007	请求冲突
E_DATA_ILLEGAL	-4008	数据非法
E_CACHE_CONFIG_ERROR	-5001	缓存配置错误
E_NOT_ENOUGH_SPACE	-5002	空间不足
E_CACHE_MISS	-5003	没有命中缓存
E_POOL_NOT_FOUND	-5005	写缓存失败
E_NODE_NUMBER_EXCEED_LIMIT	-7001	机器节点数超出限制
E_PARSING_LICENSE_FAILURE	-7002	解析证书失败
E_UNKNOWN	-8000	未知错误

最后更新: 2025年12月19日